An Introduction to Deep Learning Phenomena: From Perceptrons to Large Models

Authors (sorted by stroke count of Chinese surname):
Zhi-Qin John Xu Yaoyu Zhang

Contributing students:
Zhiwei Wang Zhiwei Bai Tianyi Chen Zhongwang Zhang
Pengxiao Lin Liangkai Hang Zhangchen Zhou Jiajie Zhao Junjie Yao
(sorted by stroke count)

Contact:
Github: `https://github.com/xuzhiqin1990/understanding_dl`
Email: xuzhiqin@sjtu.edu.cn, zhyy.sjtu@sjtu.edu.cn

Tuesday 27$^{\text{th}}$ October, 2026

Contents

1 Introduction to Deep Learning **21**
- 1.1 Data Fitting . . . 23
- 1.2 Introduction to Neural Networks . . . 26
  - 1.2.1 How a single neuron senses information . . . 26
  - 1.2.2 Single-layer neural network . . . 28
  - 1.2.3 Multi-layer neural networks . . . 31
- 1.3 Common Loss Functions . . . 33
  - 1.3.1 Mean squared error loss . . . 33
  - 1.3.2 Mean absolute error loss . . . 34
  - 1.3.3 Cross-entropy . . . 34
- 1.4 Loss Landscape . . . 35
- 1.5 Optimization Methods . . . 37
  - 1.5.1 Gradient computation: back-propagation . . . 37
  - 1.5.2 Gradient descent . . . 38
  - 1.5.3 Stochastic optimization methods . . . 40
  - 1.5.4 Gradient descent with momentum . . . 41
  - 1.5.5 Adaptive optimization methods . . . 42
- 1.6 Parameter Initialization . . . 43
- 1.7 No Free Lunch . . . 46
- 1.8 Understanding Deep Learning . . . 46
  - 1.8.1 The basic elements of deep learning . . . 47
  - 1.8.2 Deep learning theory . . . 49
  - 1.8.3 The generalization mystery and implicit bias of neural networks . . . 49
  - 1.8.4 Research approach: phenomenon-driven theoretical research . . . 52

2 Curse of Dimensionality **57**
- 2.1 Properties of high-dimensional spaces . . . 59
  - 2.1.1 Sparsity of data in high-dimensional spaces . . . 59
  - 2.1.2 Concentration of volume near the surface . . . 60
  - 2.1.3 Distance concentration effect and orthogonality . . . 63
  - 2.1.4 Gaussian annulus effect . . . 65
  - 2.1.5 Dimensionality reduction via random projection . . . 68
  - 2.1.6 Linear separability of data . . . 69
- 2.2 Examples of the curse of dimensionality . . . 70

2.2.1 High-dimensional numerical integration . . . 70
2.2.2 High-dimensional partial differential equations . . . 71
2.2.3 High-dimensional function approximation . . . 72
2.3 Methods for overcoming the curse of dimensionality . . . 74
2.3.1 Monte Carlo method . . . 74
2.3.2 Neural network method . . . 76

3 Data and Neural Network Architecture 78
3.1 Fully connected networks . . . 82
3.2 Residual neural networks . . . 83
3.3 Convolutional neural networks . . . 85
3.3.1 Characteristics of image datasets . . . 85
3.3.2 Image-processing structure of the primary visual cortex . . . 91
3.3.3 Convolutional neural networks . . . 93
3.4 Language tasks and the main paradigms of natural language processing . . . 104
3.4.1 Characteristics of language tasks . . . 104
3.4.2 An introduction to deep learning models for processing language . . . 105
3.5 Recurrent neural networks . . . 108
3.5.1 The basic unit of a recurrent neural network . . . 108
3.5.2 Encoder–Decoder recurrent neural networks . . . 109
3.5.3 Training a recurrent neural network with the BPTT algorithm . . . 110
3.5.4 Long short-term memory networks . . . 111
3.6 Transformer . . . 113
3.6.1 Basic principles of the Transformer . . . 115
3.6.2 Embedding . . . 117
3.6.3 Attention layer . . . 118
3.6.4 Feed-forward network layer . . . 121
3.6.5 Output layer . . . 121
3.6.6 Detailed inference process of a Transformer . . . 122
3.7 Generative models . . . 124
3.7.1 Autoencoders . . . 124
3.7.2 Variational autoencoders . . . 125

4 Frequency Principle 128
4.1 Low-dimensional experiments of the Frequency Principle . . . 130
4.1.1 The “smoothness” preference of neural networks . . . 130
4.1.2 Frequency and the Fourier transform . . . 133
4.1.3 The Frequency Principle . . . 135
4.2 Understanding neural networks from the Frequency Principle . . . 136
4.2.1 Experimental understanding: the necessity of the Frequency Principle . . . 137
4.2.2 A frequency-based understanding of early-stopping . . . 140
4.2.3 Advantages and limitations of neural networks . . . 141

5 Designing Efficient Neural Networks Based on the Frequency Principle 143
5.1 Multi-scale Neural Network Architectures . . . 144
5.1.1 Architecture . . . 145
5.1.2 Neural Networks Based on Subspace Decomposition . . . 146
5.2 Neural Radiance Fields . . . 148
5.3 Fourier Feature Networks . . . 149
5.4 More . . . 150

6 Mechanistic Analysis of the Frequency Principle 151
6.1 Factors Affecting the Frequency Principle . . . 152
6.1.1 Effect of the Magnitude of Initialization Weights . . . 152
6.1.2 Effect of Different Activation Functions . . . 153
6.1.3 Effect of the Form of the Loss Function . . . 154
6.2 A Simple Analysis of the Frequency Principle . . . 155

7 Phase Diagram Analysis 158
7.1 Behavior of neural networks under different initializations . . . 160
7.2 Linear and nonlinear behaviors of neural networks . . . 161
7.2.1 Analysis of parameter evolution . . . 161
7.2.2 Defining linear and nonlinear behavior . . . 163
7.3 Separating the linear and nonlinear regimes . . . 165
7.3.1 Definition of state variables and dynamical phase transition . . . 165
7.3.2 Obtaining the phase diagram experimentally . . . 167
7.3.3 The critical and condensed regimes . . . 169

8 Condensation Phenomenon 172
8.1 Experiments on the condensation phenomenon . . . 175
8.1.1 The condensation process . . . 175
8.1.2 Condensation phenomenon in fully-connected networks . . . 178
8.1.3 Condensation phenomenon in convolutional neural networks . . . 179
8.1.4 Condensation phenomenon in residual neural networks . . . 180
8.2 Discussion of the condensation phenomenon . . . 181
8.2.1 Definition of the condensation phenomenon . . . 181
8.2.2 Understanding the condensation phenomenon . . . 182
8.3 Initial condensation . . . 184
8.3.1 Experiments on initial condensation . . . 184
8.4 Dropout promotes the condensation phenomenon . . . 186
8.4.1 What is dropout? . . . 187
8.4.2 Dropout promotes condensation of neurons . . . 188
8.4.3 Explicit expression of dropout and its implicit regularization . . . 190
8.4.4 Impact of the regularization term on condensation . . . 190
8.4.5 Relationship between dropout and sample size . . . 192

9 The Embedding Principle of the Loss Landscape 193
9.1 Width Similarity and the Embedding Principle . . . 196
9.1.1 Structural similarity at loss-plateau points . . . 196
9.1.2 Theoretical framework: the embedding principle . . . 197
9.1.3 Relationship between the embedding principle and the condensation phenomenon . . . 199
9.1.4 Relationship between the embedding principle and the F-Principle . . . 199
9.2 In-depth Analysis of the Embedding Principle . . . 200
9.2.1 Spectral analysis of the loss-function plateau phenomenon . . . 200
9.2.2 Eigenvalue analysis of the Hessian matrix after critical-point embedding . . . 204
9.2.3 Simplifying the scale of a neural network . . . 205

10 Optimistic Estimate 207
10.1 Quantifying the minimum sample size required for a model to recover a target function: the model rank . . . 209
10.2 Comparison between optimistic sample size and actual experimental performance . . . 213
10.2.1 A simple nonlinear regression model . . . 213
10.2.2 Matrix factorization model . . . 214
10.2.3 Neural network models . . . 216
10.2.4 The role of hyperparameter tuning in nonlinear models . . . 218
10.3 Analyzing neural network architecture design: does the optimistic sample size increase? 219

11 Flatness of Solutions 222
11.1 Definition of Flatness of Solutions . . . 223
11.2 Effect of Batch Size on the Flatness of Solutions . . . 224
11.3 Effect of Stochastic Gradient Descent on the Flatness of Solutions . . . 225
11.3.1 The Importance of the Noise Structure of Stochastic Gradient Descent . . . 226
11.3.2 Relationship between SGD Noise and the Flatness of Solutions . . . 227
11.3.3 Theoretical Analysis of the Implicit Regularization of Stochastic Gradient Descent . . . 228
11.4 Effect of Dropout on the Flatness of Solutions . . . 229
11.5 Edge of Stability Phenomenon . . . 231

12 Anchor Functions: A Class of Simple Functions for Studying Language Models 233
12.1 Challenges in studying Transformer-based language models . . . 235
12.1.1 Unknown tasks . . . 235
12.1.2 High computational and memory demands . . . 236
12.1.3 Difficulty of interpreting the reasoning mechanism . . . 236
12.2 Characteristics of language tasks . . . 236
12.3 Research approach . . . 237
12.4 Anchor functions and language-like tasks . . . 238
12.4.1 Anchor functions . . . 238
12.4.2 Language-like tasks . . . 240
12.5 Data partitioning and model generalization . . . 241
12.5.1 Splitting training and test sets . . . 241
12.5.2 Data generalization and task generalization . . . 244
12.6 Experimental results and discussion . . . 245

12.6.1 Identity-learning task . . . 245
12.6.2 Reading-comprehension task . . . 246
12.6.3 Classification task . . . 246
12.6.4 Composite task . . . 247
12.6.5 Working-memory task . . . 248
12.6.6 Synonym task . . . 249
12.6.7 Forward–backward recitation task . . . 249
12.6.8 Statistical-output task . . . 250
12.6.9 Multi-anchor task . . . 251
12.7 Mechanistic study of the identity-learning task* . . . 253
12.7.1 A brief explanation of the two-layer model . . . 253
12.7.2 Mechanism of the simplified two-layer model . . . 254
12.7.3 Shift and broadcast in Llama2-7B . . . 255
12.7.4 Discussion of the shift and broadcast operations . . . 255
12.8 Experimental setup . . . 256
12.8.1 Model architecture . . . 256
12.8.2 Loss function . . . 256
12.8.3 Hyperparameter settings . . . 257

13 The Effect of Complexity Control on the Reasoning Ability of Language Models 258
13.1 Introduction . . . 260
13.2 Definitions . . . 261
13.2.1 Two-Anchor Composite Functions . . . 261
13.2.2 Data Generation . . . 262
13.2.3 Initialization and Regularization Parameters . . . 263
13.2.4 Hierarchical Structure of the Data . . . 264
13.2.5 Generalization . . . 264
13.2.6 Model Architecture and Basic Experimental Setup . . . 264
13.3 Phase Partition of Solutions for Composite Functions . . . 265
13.4 Mechanism Analysis of Different Phases via Attention Masking Strategy . . . 268
13.4.1 Mechanism Analysis via Masking the Key Token . . . 268
13.4.2 Mechanism Analysis via Masking the Second Anchor . . . 270
13.5 Model Complexity: A Key Factor in Phase Transitions . . . 271
13.5.1 Condensation of Input Weights . . . 271
13.5.2 Structured Organization of the Token Embedding Matrix . . . 272
13.5.3 The Relation Between Condensation and Reasoning . . . 272
13.6 Further Validation on Real-World Tasks . . . 274
13.7 Predictions on the Mechanisms Behind Different Solutions . . . 280

14 More Phenomena in Deep Neural Networks 283
14.1 Scaling law . . . 285
14.2 Densing law for large models (densing law) . . . 286
14.3 In-context learning in large language models . . . 288
14.4 Chain-of-thought in large language models . . . 289
14.5 The grokking phenomenon . . . 290
14.6 The lottery ticket phenomenon . . . 290

14.7 The double descent phenomenon in neural networks . . . 292
14.8 The neural collapse phenomenon . . . 293
14.9 The mode connectivity phenomenon . . . 294

15 Solving Differential Equations with Neural Networks 295
15.1 Example: Newton's Laws of Motion . . . 297
15.2 Parameterized-Solution Methods . . . 299
15.2.1 Least-Squares Method . . . 299
15.2.2 Variational Method . . . 300
15.2.3 Weak-Solution Method . . . 300
15.3 Parameterized-Operator Methods . . . 301
15.4 Strengths and Limitations . . . 302
15.4.1 Strengths . . . 302
15.4.2 Limitations . . . 303
15.5 Different preferences of traditional algorithms and neural networks for low frequencies 304
15.5.1 Multi-Scale Neural Network for Solving PDEs . . . 306
15.5.2 Summary . . . 307

Foreword I

As the core driving force of the field of artificial intelligence, deep learning is reshaping the technological landscape and the boundaries of applications in human society at an unprecedented pace. Nevertheless, many mysteries remain unresolved regarding its internal mechanisms and theoretical foundations. How to systematically understand the principles behind this "black box" has become a focal point of common concern for the academic and industrial communities worldwide.

An Introduction to Deep Learning Phenomena comes into being against this backdrop. Zhi-Qin John Xu, Yaoyu Zhang, and their team, guided by the research philosophy of being "phenomenon-driven," systematically organize the key phenomena and intrinsic mechanisms in deep learning. Starting from the Frequency Principle (F-Principle), the condensation phenomenon, and so on, they progressively unveil the implicit bias and structural characteristics of neural networks in optimization and generalization. The book also covers the design philosophy of mainstream architectures such as convolutional networks, residual structures, and Transformers, and discusses fundamental issues such as the curse of dimensionality, function approximation, and optimization algorithms, reflecting the authors' profound expertise in combining theory with practice.

As a physicist, I particularly appreciate the research paradigm of "from phenomenon to theory" advocated by this book. It resonates with the philosophy we promote in driving original innovation in interdisciplinary fields such as physics, biology, and computation. The essence of deep learning lies not only in its powerful fitting capability, but more importantly in the structures and regularities it embodies. Starting from this vantage point, this book guides readers beyond the surface of "alchemy-style hyperparameter tuning" and into a deeper exploration of the scientific principles underlying neural networks.

During my tenure as President of Shanghai Jiao Tong University, I served as the founding dean of Zhiyuan College. Zhi-Qin John Xu and Yaoyu Zhang were among the first cohort of Zhiyuan students. Their class, also called the science class, required students to study both mathematics and physics. At that time, we invited Professor Shen-Ou Cai, Professor Weinan E, Professor Shi Jin, and other experts to design an integrated curriculum bridging mathematics and physics, with an important goal of cultivating interdisciplinary innovative talents. Fundamental research in deep learning is a new field of research, and it also exhibits very pronounced interdisciplinary characteristics.

It is worth mentioning that the book is written in an accessible style, combining rigorous mathematical derivations with vivid illustrative examples. It is suitable both as an introductory textbook for graduate students and young scholars in universities, and as a theoretical reference for engineers engaged in artificial intelligence research and development.

Today, as artificial intelligence ushers in a new wave of development, the publication of *An Introduction to Deep Learning Phenomena* carries important significance for our time. It is not merely a systematic summary of some existing achievements, but also a powerful guide pointing to future research directions. I believe this book will become an important force in propelling China toward the international frontier of fundamental theoretical research in artificial intelligence.

May every reader find inspiration in it and join together in this exciting journey of exploring intelligence.

Jie Zhang
Academician of the Chinese Academy of Sciences
Chair of the Academic Committee of Shanghai Jiao Tong University

November 2025

Foreword II

At present, the field of artificial intelligence is undergoing breathtakingly rapid development. In particular, the sudden emergence of large models has pushed this wave to unprecedented heights. Yet behind the tremendous success of engineering practice, our understanding of the internal mechanisms of artificial intelligence appears relatively lagged, presenting a situation where "applications come first, and theory follows."

For a long time, one mainstream direction of AI theoretical research has been devoted to using traditional mathematical tools to deconstruct the deep mechanisms of models during training and inference. It is undeniable that this research paradigm, based on rigorous derivation, has played a crucial role ever since the birth of artificial intelligence, laying a solid foundation for our understanding of early learning systems. However, as the technology has moved from deep learning into the era of large models, the speed of engineering iteration and the expansion of computational power have far exceeded expectations, and theoretical research has begun to show clear signs of lagging behind. Faced with modern deep neural networks—a class of systems that are highly nonlinear and extremely complex in structure—the simplifying assumptions on which traditional theoretical analysis relies often appear inadequate, even "unrealistic." This has led to theoretical results frequently exhibiting a gap with experimental phenomena, making it difficult to effectively guide current engineering practice. While there have been occasional successful theoretical cases, they remain few and far between.

Faced with the cutting-edge AI technologies represented by large models, shifting the research perspective is particularly urgent. A "phenomenon-driven" research paradigm, akin to the exploration in natural sciences, may be the key to breaking the impasse. This mode no longer relies solely on a priori axiomatic deduction; rather, it emphasizes extracting universal regularities from complex systems through extensive experimental observation, then creating symbolic language to describe these phenomena, and finally analyzing them with rigorous mathematical reasoning. This may be an effective path toward narrowing the gap between theory and practice, and developing new mathematical tools suitable for understanding frontier AI models.

This book is precisely built upon such a unique perspective. The authors bring together a series of research achievements from recent years, attempting to construct cornerstones for understanding neural networks by capturing and describing key phenomena in deep learning. Among them, the book's discussion of the "Frequency Principle (F-Principle)" is a highly representative example. This research reveals that neural networks tend to preferentially learn low-frequency components during training. As a regularity summarized from extensive experimental observation, it provides an inspiring perspective for understanding the complex mechanisms of model convergence, and has also prompted deeper reflection within the community on the implicit bias of neural networks.

I hope this book will bring inspiration to readers, and add a measure of rationality and insight to the journey of exploring the AI "black box."

This is the preface.

Bin Dong
Boya Distinguished Professor, Peking University
November 2025

Recommendation Preface III

On a casual day in November, Zhi-Qin John Xu invited me to write a short preface for *An Introduction to Deep Learning Phenomena: From Perceptrons to Large Models*. I agreed without hesitation. Having collaborated with Zhi-Qin and Yaoyu for many years, I am certain they fully understand that I am a layperson when it comes to deep learning theory. Since that is the case, their invitation to write this preface must reflect their belief that the book holds considerable value for many laypersons like me.

After reading the book, this is indeed the case.

In today's era, when artificial intelligence is flourishing, there is undoubtedly an expectation for "principles." Yet what are the principles behind artificial intelligence, really? The situation perhaps resembles Einstein's metaphor about knowledge and ignorance: "As our circle of knowledge expands, so does the circumference of darkness surrounding it." The deeper our demand for understanding, the more remote and blurry the ultimate answers appear.

How, then, can we cross the immense difficulties of theoretical research and unveil the "depth" that lies behind the broad frontier of artificial intelligence? More pressingly, how do we dispel the fanatical illusions of daily "disruptive breakthroughs" and arrive at genuine knowledge? Or, on a more practical note, how can a beginner blaze a trail through the exploding mountain of algorithms and open up a path forward?

This book takes "phenomena" as its subject. On the surface, it sidesteps the formidable task of "investigating principles." But in fact, as I understand it, Zhi-Qin and Yaoyu have ingeniously found a viable path. Of course, which experiments count as phenomena, which phenomena are worth excavating, how to study phenomena scientifically, and how research results point toward the essence behind phenomena—these are the concerns of theorists like them.

As a "layperson" reader, this book—starting from phenomena—is remarkably easy to read, easy to understand, and easy to learn from. Because it centers on phenomena, it is concrete, intimate, and not abstract: from kittens to spectra, to features, to invariance. Because it must identify phenomena precisely, it is rich, critical, and historical: the mystery of generalization, high-dimensional sparsity, naming methods and archaeological methods, and so on. Finally, because it observes phenomena, it is always simple, intuitive, and interpretable: the Frequency Principle starts with one-dimensional fitting; the condensation phenomenon begins with two-layer ReLU fully-connected networks; language models start by designing anchor functions. I must say, this is also an introduction full of character, and I derived endless pleasure in the process of reading it.

Finally, facing the ultimate question of deep learning theory, what matters more than describing the answer is perhaps offering a way of thinking. I believe this book has accomplished that as well.

Tianhan Zhang
Professor, Beihang University
November 2025

Preface I

Why We Wrote This Book Deep learning is undoubtedly the most important and central technology in the current development of artificial intelligence. Many researchers have already given comprehensive introductions to deep learning from various angles, yet it is still often described as a "black box" or as "alchemy." Over the years, foundational research in deep learning has gone through many twists and turns, in stark contrast to the rapid progress of its applications. Some have even gone so far as to claim that the foundational theory of deep learning is useless for the development of its applications. In fact, foundational research in recent years has produced considerable progress in our understanding of how deep learning works, but under the currently very results-driven climate, this progress has not been well disseminated. For this reason, a book introducing the progress of foundational research in deep learning carries significant value for students, researchers, and technical practitioners alike.

Of course, there are some unavoidable issues. For example, some works make great theoretical progress but remain at a noticeable distance from real-world applications, and some foundational advances place high demands on the reader's theoretical background. Take the universal approximation theorem as an example: the theorem states that as long as it is wide enough, a two-layer network can approximate any continuous function to arbitrary precision. This certainly gives users of deep neural networks great confidence in employing them, but it also raises many puzzles: if two layers are enough, why do we typically use networks with many more layers rather than two? In polynomial fitting, as long as the degree is high enough (the Weierstrass theorem that we learn in the first year of college), one can also fit any continuous function to arbitrary precision; so why don't we use polynomial fitting and instead use deep neural networks? Does "sufficiently wide" in the theorem have any practical guiding significance for actual training? I think this list of questions could go on and on, which shows that the conclusions reached by the universal approximation theorem are at a substantial distance from practical applications. Understanding why the universal approximation theorem holds also requires substantial theoretical preparation from the reader. On the other hand, many marketing articles report on advances in ways that proclaim a "disruption" every three days and a "revolution" every five, when in reality most such reporting only aims to attract traffic, raising people's unrealistic expectations of the results and creating a vicious cycle in which the achievements must be continually exaggerated. Therefore, which foundational research results to introduce, and in what way to introduce them, becomes especially important.

Why We Start From Phenomena The advantage of starting from phenomena lies, on one hand, in the fact that this is one of the important ways to study deep learning; on the other hand, observing phenomena is more intuitive and places lower demands on the reader, yet it can still lead to a very deep understanding.

Deep neural networks are certainly very complex, but the natural world in which we live exhibits a far higher degree of complexity. Scientific research today has already achieved much understanding of complex natural systems, and put that understanding to use—in mechanics, electricity, quantum mechanics, chemistry, biology, and so on. If humanity can make so much progress in understanding even more complex systems, then for deep neural networks, as long as we adopt reasonable research methods, we can likewise make important progress. As early as 1995, Leo Breiman noted: "Mathematical theory is not critical to the development of machine learning. But scientific inquiry is." The scientific inquiry mentioned here similarly conveys the idea of drawing on the way we study complex systems in nature.

Take the study of astronomy as an example. Tycho Brahe recorded a huge amount of data through observations of the heavens, and Kepler distilled from that data the three laws—which, in essence, amount to identifying three stable phenomena. This played a crucial role in Newton's later discovery of universal gravitation. Such a phenomenon-driven approach is very common, and very effective, across physics, chemistry, biology, and other disciplines. The parameter scale of deep neural networks can already reach the trillions; although their basic constituent units are simple, they exhibit complex dynamics and behaviors across various tasks. Perhaps a few simple equations could characterize all aspects of this complex system, just as the equations of Newton, Maxwell, Einstein, and Schrödinger characterize the natural world; but to guess these core equations purely from theoretical derivation is nearly impossible. Nature's equations were obtained on the basis of extensive experimental observations. Likewise, as introduced in this book, for the complex system of deep neural networks, gradually capturing the features of each aspect of this complex system through phenomena is also an effective approach.

Phenomena play a key foundational and navigational role in promoting in-depth theoretical research, and they also lower the cost of understanding complex systems. Taking the Frequency Principle (F-Principle) mentioned in this book as an example, it points out that neural networks often learn the low frequencies (outline) of the target first, and then the high frequencies (details). This phenomenon is very intuitive and easy to understand; it requires no extensive theoretical background, and one does not even need to know the concept of frequency. For instance, when using a neural network to fit an image, one can expect that it first learns the outline, and finds the depiction of details very difficult; but if an image happens to have a lot of high-frequency noise added (such as many black dots), then when fitting this image with a neural network, the noise is learned the latest, so that through early stopping during training, the noise can be removed. If one wants to learn the details of the image faster, then one needs to do something to the high-frequency part, which has also given rise to many algorithms for accelerating the learning of high frequencies. In classical computational mathematics, Jacobi iteration for solving differential equations is, conversely, slow at learning low frequencies, so one can use a neural network to learn the low frequencies first, and then use Jacobi iteration to solve for the high frequencies, and so on. Many phenomena have similar effects: small experiments, big understanding—and understanding is productivity.

Why We Start From Our Own Research The best way to learn something new is to study it in depth. At present, the field of deep learning makes it very easy to pose a question to which no one yet knows the answer—for example, how the width and depth of a network should be reasonably set to meet requirements on training speed and accuracy. When I first started working with deep neural networks, all sorts of questions kept popping into my mind. Some questions could be resolved relatively easily; more questions had no answer; and some questions directly gave rise to our research results. This research process became my entry point for learning about deep neural networks. Different people will study deep learning from different angles and will arrive at different entry points. Even though the angles of focus may differ, after entering through a research angle, one will gain a fairly profound understanding of deep learning. Starting this book from our own research makes it easier for us to offer readers a way to learn deep learning from a research perspective.

The Intended Audience and Main Content of This Book The core of this book is to start from phenomena and understand some of the basic principles of deep learning. It will be of very direct help to senior undergraduates, graduate students, researchers who study deep learning or use

deep learning in their research, and deep learning technical practitioners—not only to know about deep learning, but more importantly to understand deep learning, and to use deep learning more effectively.

The book first introduces some basic concepts of deep learning and offers a brief discussion of the authors' views on deep learning research. Chapter 2 introduces the characteristics of high-dimensional data, since the main scenarios in which deep learning succeeds are all high-dimensional problems; high-dimensional settings have many important features that cause traditional algorithms to fail while neural networks turn out to be exceptionally effective. Chapter 3 introduces common network architectures and the principles behind their design. Chapters 4 through 6 introduce the fact that the output functions of neural networks typically follow a low-frequency-first Frequency Principle, and the understanding and algorithmic improvements that this brings. Chapters 7 through 10 focus on introducing the condensation phenomenon of neural network parameters, and around the region in which condensation occurs, the embedding structure of the loss landscape, and the optimistic estimate it inspires regarding sample size, they explore the unique properties of neural networks under nonlinear training dynamics—an important mechanism that allows over-parameterized networks to avoid noticeable overfitting. Chapter 11 introduces the influence of the flatness of different solutions on generalization and how hyperparameters and optimization algorithms affect the flatness of solutions. It offers important perspectives on understanding batch size, learning rate, stochastic gradient descent, Dropout optimization, and so on. Chapters 12 and 13 introduce the use of anchor functions to study the properties of Transformer networks, with a focus on how hyperparameters affect the reasoning capability of language models by controlling complexity. Chapter 14 introduces more phenomena in the field of deep learning. Chapter 15 introduces the use of neural networks to solve partial differential equations and explains, from a phenomenon-based perspective, that the convergence of deep neural network solvers and traditional iterative algorithms in frequency is opposite. It carries significant inspiration for researchers in computational mathematics, computational fluid dynamics, and AI for Science.

How This Book Came to Be I began teaching courses related to deep learning in 2020. While preparing these courses, I kept refining my own thinking and understanding of deep learning, and I started to put together lecture notes. From then on, I had the intention of organizing these notes into a book. Each year I would polish them a little more, but because of the unavoidable accumulation of other matters and my own lack of self-discipline, the manuscript was repeatedly delayed. Later, I began to ask my students for help, and the lecture notes and accompanying code gradually became more complete, though still quite rough. In March 2024, both Yaoyu Zhang and I came to recognize the importance of publishing a phenomenon-driven account of how to understand deep learning. So together with several students we focused our efforts, and built on the basis of my lecture notes for nearly two months, producing a relatively complete manuscript. Over the more than a year that followed, we used this manuscript while continuing to revise it, and we asked many colleagues and students for feedback.

The students who participated were mainly (ordered by the number of strokes in their surnames): Zhiwei Wang, Zhiwei Bai, Tianyi Chen, Zhongwang Zhang, Pengxiao Lin, Liangkai Hang, Zhangchen Zhou, Jiajie Zhao, and Junjie Yao. In addition, Lulu Zhang, Qixuan Zhou, Yekun Zhu, Xiaolong Li, and Yuxiao Yi also helped to varying degrees.

My Personal Journey in Deep Learning Research I first encountered deep learning in November 2017. My original field was computational neuroscience, where my research, broadly speaking,

was about understanding how the brain works from a mathematical perspective. More specifically, my work involved processing spiking data generated by high-dimensional networks of neurons, trying to understand how these signals process incoming input signals. But the brain is exceedingly complex, and its dimensionality is exceedingly high: an ordinary human brain has on the order of one hundred billion neurons, and each neuron exchanges signals with thousands more. I did not have much confidence in handling this kind of data. Around that same time, I happened to read a paper whose gist was that applying the current methods of computational neuroscience to study computer chips, the conclusion was that these methods could not help us understand how a chip actually works. Another thing that made me very uneasy was that we not only know very little about the brain, but it is also extremely difficult to obtain data from it.

So at the time we were thinking: could we find a simple network model that can realize complex functions, while at the same time our understanding of this model is still incomplete, so that by studying it we might gain inspiration for our study of the brain? It was the end of 2017, and deep learning was already very popular. In particular, my fellow students had been working with deep learning for some time, so we quickly came to learn about it. Its architecture and training procedure looked simple enough, yet its capabilities were extraordinary, and the related theory was in its embryonic stage.

Thus, my first thought upon entering deep learning was to treat it as a simple model for studying the brain. Clearly, under this "brain-inspired research" framing, what we cared about was the foundational study of deep learning. Even though we were using deep learning as a model to study why the brain has such complex learning abilities, there are still obvious differences between the brain and deep learning. And in terms of knowledge background, ability, and time, it was very difficult for me to delve deeply into these two fields simultaneously, as they still appear to be quite far apart.

I chose to turn entirely toward deep learning. The question I studied was: as an algorithm, what characteristics does deep learning have? The "no free lunch" theorem tells us that, when averaging performance over all possible datasets, all algorithms are equivalent; that is, no algorithm is universally optimal. We need to clarify what kinds of data the deep learning class of algorithms is suited for, and what kinds it is not suited for. In fact, the theory of deep learning is not in its embryonic stage; since the middle of the last century, when it first began to develop, the related theory was already underway, and some important results had been obtained, but overall it is still at a preliminary stage. For me, this was an even more difficult question.

I then turned to treating deep learning as a kind of "toy". By adjusting all sorts of hyperparameters and considering different tasks, I observed what "natural phenomena" it would produce. The goals I set were no longer grand and lofty, but simply interesting: discover an interesting phenomenon, then explain it, and perhaps use it to guide practical applications. With these recognitions in mind, we started from some interesting phenomena in the training of deep neural networks. As for me personally, I learned to write Python and TensorFlow from scratch; more specifically, I found a few pieces of code online and learned by copying and trying to understand them.

Are neural networks really complex?

In traditional learning theory, the number of model parameters is a very important indicator of model complexity. As model complexity increases, the model's ability to fit the training data is enhanced, but this also brings the problem of overfitting on the test set. Von Neumann once famously said: with four parameters I can fit an elephant, and with five I can make him wiggle his trunk.

Therefore, researchers engaged in traditional modeling would often calculate the number of

model parameters when using neural networks, and deliberately use networks with few parameters to avoid overfitting. However, one of the important reasons that neural networks have been so successful today is precisely the use of ultra-large-scale networks. The number of parameters in a network often far exceeds the number of samples, yet it does not overfit in the way that traditional learning theory would predict. This is the generalization mystery that has received enormous attention in recent years. In fact, as early as 1995, Leo Breiman pointed out this problem in one of his articles. Today, when neural networks are extremely popular and important, this mystery has become even more important. We can ask: are neural networks with a vast number of parameters really complex? The answer is yes! Theoretical work from the late 1980s proved that when a two-layer neural network (whose activation function is non-polynomial) is sufficiently wide, it can approximate any continuous function to arbitrary precision; this is the well-known "universal approximation" theorem.

In fact, we should ask a more meaningful question: in actual training, are neural networks really complex?

The solutions proved by approximation theory are almost never encountered in actual training. Actual training requires setting hyperparameters such as the initial values, the optimization algorithm, the network architecture, and so on. For our work to have practical guiding significance, we cannot consider the problem of generalization in isolation from these factors, because generalization itself is inherently a problem that depends on real data.

In the process of learning and training neural networks, we can easily find that the training of neural networks follows certain regularities. In our research, there are two phenomena that are particularly interesting; in the process of studying and explaining them, we have found that they are equally meaningful.

First, we discovered that neural networks, when fitting data, often learn the low frequencies first and then gradually learn the high frequencies. We named this phenomenon the Frequency Principle (F-Principle); other work also calls it Spectral bias. The Frequency Principle can be used both to explain why neural networks, in the over-parameterized regime, generally do not fit data with oscillatory curves, and to point out that high frequencies are an important reason for slow fitting, inspiring the design of more efficient algorithms.

Second, we discovered that during the training process, the directions of the input weights (vectors) of many neurons tend to align with each other, especially when the weight initialization is small. We call this the condensation phenomenon. Neurons with identical input weights process inputs in exactly the same way, so they can be simplified into a single neuron; that is, a large network can be reduced to a small one. Recently, we have also extended the study of condensation to the study of Transformers, and have gained a new understanding of their reasoning ability.

Both of these phenomena reflect a kind of implicit simplicity bias of neural networks during training: a low-frequency bias or an effective-small-network bias. The low-frequency bias is very pervasive, but the small-network bias brought about by condensation is a feature that only emerges in the nonlinear training process.

These studies have given us a very important realization: the phenomenon-driven research approach is a highly effective means of tackling a complex system such as a deep neural network.

We hope that, through this book, we can convey to readers the research of our earlier stages, as well as some of our perceptions of the field. We also greatly look forward to the rapidly evolving developments in artificial intelligence!

Acknowledgments Our way of doing research has been deeply influenced by our late advisor,

David Cai. We dedicate this book to him in gratitude for his guidance.

This series of foundational studies on deep learning has consistently received guidance and assistance from Professor Weinan E. During our postdoctoral years, it was Professor E and our postdoctoral advisor David McLaughlin who gave us the support and courage to venture into the new field of deep learning. We thank Tao Luo and Zheng Ma for beginning the foundational study of deep learning together with us. We also wish to thank many teachers and friends with whom we have discussed and from whom we have learned a great deal, including but not limited to Professor Wei Cai, Professor Bin Dong, Lei Wu, Chao Ma, Tianhan Zhang, Jiequn Han, and others.

We thank Professor Jie Zhang, Professor Tao Tang, Professor Shi Jin, Professor Bin Dong, and Professor Tianhan Zhang for writing the foreword and endorsements for this book.

We also wish to thank Shanghai Jiao Tong University for consistently providing us with an excellent academic environment, the Zhiyuan College for offering us an outstanding undergraduate education, and the School of Mathematical Sciences for providing high-quality graduate training. We thank the Institute of Natural Sciences and the School of Mathematical Sciences for their long-term support of our academic careers. We also thank NYU Abu Dhabi and the Courant Institute of Mathematical Sciences at NYU for their support during the early stages of our deep learning research.

The entire journey of research and writing has been filled with both hardship and joy. Without the support and help of many people, we could hardly have persevered. The list of those to thank could go on and on, and even so we would still miss many, much as one sometimes meets a familiar friend and momentarily forgets their name. I believe that the best way for us to thank the teachers and friends who have supported and helped us is to continue to do good foundational research in deep learning.

Zhi-Qin John Xu
Shanghai Jiao Tong University
November 2025

Preface II

Over the past years, the development of deep learning has far exceeded our expectations. From vision to language, from games to scientific computing, neural networks have demonstrated capabilities across various tasks that continually refresh our understanding of the boundaries of artificial intelligence. However, as a researcher in deep learning theory, I have always felt a striking contrast: engineering applications are expanding rapidly and model capabilities are growing ever stronger, yet our understanding of the roots and mechanisms behind these performances remains insufficient, leaving a vast gulf between theory and practice.

Faced with complex systems containing billions or even hundreds of billions of parameters, our situation is often akin to "blind men feeling an elephant." Different researchers focus on different aspects: the dynamics of optimization, the patterns of generalization, the influence of architecture, the details of training—each facet is real but limited, and it is difficult to directly assemble them into a complete picture. Although traditional statistical learning theory provides an important starting point for understanding models, in the face of modern deep networks many classical conclusions have proven inadequate. For example, even when the number of parameters far exceeds the sample size, neural networks can still exhibit excellent generalization performance, which has become a typical "anomaly" that traditional paradigms struggle to explain. We must acknowledge that, in order to explain these newly emerging phenomena, we need to supplement our work with entirely new perspectives and research paths.

Based on this recognition, we have chosen a relatively plain and grounded route in our research: starting from phenomena rather than from theoretical presuppositions. We try to avoid using existing theoretical frameworks to speculate about how neural networks should learn; instead, we treat them as a complex new system, prioritizing the recording of their actual behavioral trajectories before extracting patterns from them. This approach is like assembling a giant jigsaw puzzle. At the beginning, we hold only a few scattered fragments, unable to judge their mutual relations, let alone know the shape of the final pattern. Rather than expending effort on vague speculation, it is better to strive to uncover new fragments, beginning the search for breakthroughs from the parts most easily observed.

It is precisely on this basis that we began with the simplest one-dimensional function fitting. Although the task itself is not complex, it allowed us for the first time to fully observe the entire evolution of network outputs during training, and from this we noticed a stable feature: no matter how large the network or how complex the structure, it tends to first fit the low-frequency outline before gradually filling in the high-frequency details. The Frequency Principle (F-Principle) emerged from this, becoming a key fragment for understanding the learning preferences of neural networks. We then turned our attention to the more complex parameter space. Through a series of systematic experimental designs, we observed another universal behavior: originally disordered neuron parameters would, during training, cluster along a few directions, forming a clear "condensation phenomenon." This spontaneous process of structuring strongly suggests that the network may achieve good generalization by reducing its effective complexity.

As our research on the condensation phenomenon deepened, we gradually collected more fragments: the phase-transition behavior of dynamical regimes, the embedding principle of the loss landscape, the optimistic estimate of sample efficiency, the task generalization of anchor functions, and so on. They appear to take different perspectives, yet through repeated comparison with the core phenomena they reveal deep connections, as if jointly outlining some more complete underlying picture. We are still far from seeing the full picture, but we can take comfort in saying that the

outline is slowly emerging.

We hope that this book can offer readers a perspective close to the research frontline: not explaining phenomena starting from theory, but extracting patterns from phenomena; not pursuing a tidy system of formulas, but building a reliable understanding of model behavior on the basis of experiments. Each chapter of this book corresponds to a "fragment" that we have dug deeply into through experiments, and together these fragments constitute the important foundation for our understanding of modern neural networks.

Regarding the development of deep learning theory, I am a firm optimist. Exploration will not end in the short term; we need patience, and even more so, we need to maintain a sense of awe for the complexity of the system. We must, through each careful experimental design and controlled-variable comparison, identify which patterns are stable, which phenomena are key, and which mechanisms remain consistent across tasks and structures. I believe that as long as we persist in starting from phenomena—uncovering fragments, organizing fragments, and connecting fragments one by one—a unified picture will eventually emerge naturally. This book may present only one corner of this still-forming larger picture, but I hope it can become a valuable piece of the puzzle in your hands, helping you understand the "elephant" while guiding you to find entirely new fragments. I look forward to working with you in the future to make this picture more complete!

Yaoyu Zhang
Shanghai Jiao Tong University
November 2025

Chapter 1

Introduction to Deep Learning

Deep learning, a term that sounds rather lofty, is quietly transforming our lives. Don't believe it? Let's look at some everyday examples. If you are a heavy user of mobile video apps, you must be impressed by the magical effects of face filters and dynamic stickers in videos. Whether you are transformed into an adorable rabbit or given elaborate makeup, all of this is thanks to the breakthroughs of deep learning in the field of computer vision. By learning from massive amounts of facial data, deep learning models can accurately recognize facial features and achieve real-time face detection, tracking, and effect synthesis.

If you often use smart speakers and talk to them, you will find that their speech recognition and natural language understanding capabilities are becoming increasingly powerful. They can accurately convert your voice commands into text and respond appropriately based on the semantics. Behind this also lies the power of deep learning. By learning from large amounts of speech and text data, deep learning builds acoustic and language models that enable machines to "understand" human language.

Have you ever been pleasantly surprised by the photo albums and video highlights automatically generated in your mobile photo gallery? These seemingly intelligent operations are in fact applications of deep learning in multimedia content understanding. By analyzing the semantics of image and video content, deep learning algorithms can automatically tag photos, identifying people, scenes, objects, and so on, and thereby intelligently generate relevant albums.

Go enthusiasts may have heard about AlphaGo defeating top human Go players. This marked a milestone breakthrough of deep learning in the domain of board games. Through deep reinforcement learning algorithms, AlphaGo autonomously learned from vast amounts of Go game records and mastered superb playing skills, demonstrating extraordinary decision-making ability and opening a new era for artificial intelligence in board games.

These are just the tip of the iceberg of deep learning applications. In medicine, finance, security, autonomous driving, and many other fields, deep learning is demonstrating powerful capabilities and propelling the rapid development of artificial intelligence. So what exactly is deep learning? Why has it achieved such stunning results? Through this book, we will together lift its mysterious veil.

Over the past decade, the development of artificial intelligence algorithms represented by deep learning has been like a revolution, dramatically elevating the intelligence level of machines. From Google's AlphaGo defeating world champion Lee Sedol in Go, to DeepMind's AlphaFold achieving

a major breakthrough on the long-standing protein folding problem in biology, and to OpenAI's general-purpose large language model GPT-4 and video generation model Sora, these achievements are not only technological breakthroughs but also mark a re-examination of the potential of machine intelligence. These remarkable advances are driven by the powerful force of deep learning.

Deep learning, an algorithm inspired by the structure of the human brain, has become the core of contemporary artificial intelligence research and applications. Its essence lies in the ability of neural networks to learn and extract complex patterns and features by fitting massive amounts of data. The power of this approach lies in its ability to adaptively learn the key information needed to solve a problem directly from data, without requiring manual feature extraction and selection, thereby offering a new path to solving problems that were previously intractable.

With the rapid development and application of deep learning, we have witnessed astonishing achievements in many fields, from visual recognition and language understanding to game strategy. However, deep learning is not a technology that was achieved overnight; its success is built upon a deep understanding and continuous exploration of the neural network framework.

Popular Science

What is data fitting?

Data fitting is a method for discovering patterns in observed data. Imagine you have a scatter of data points; data fitting is like drawing a best-fit line or curve through these points. This line or curve should pass as close as possible to all the data points so that it best describes the overall trend of the data. The process is like solving a jigsaw puzzle: we try to find a model or function that best matches and explains the data we observe. This model not only describes the existing data but can also be used to predict new, unseen data. In practice, data fitting is widely used across many fields. From simple linear regression to complex machine learning algorithms, all are attempts to extract meaningful patterns and relationships from data.

What is a neural network? What is its connection to data fitting?

A neural network is a mathematical model inspired by the structure of the human brain, used to fit complex data relationships. It consists of multiple layers of interconnected "neurons," with each layer passing information to the next.

The core of a neural network lies in its ability to automatically learn complex patterns through "training" on large amounts of data. Each neuron has its own weights and biases, which are continuously adjusted during training so that the entire network can better fit the relationship between input data and the expected output. A simple neural network may have only an input layer, a hidden layer, and an output layer, while more complex networks may contain multiple hidden layers, and even different structures, with each layer focusing on extracting different features of the data. This hierarchical structure enables neural networks to handle problems ranging from simple to highly complex, such as image recognition, speech processing, and natural language processing.

Neural networks are closely connected to data fitting. In fact, a neural network can be viewed as a highly flexible and powerful data fitting tool. When we use a neural network for data fitting, the network's goal is to learn a complex function that best describes the relationship between input data and output results.

Which parts of a neural network deserve attention?

In this chapter, we list the following four parts that are especially important in the process by which a neural network learns patterns from data:

Loss function The loss function measures the discrepancy between the model's predictions and the true values.

Loss landscape The loss landscape describes the full picture of how the model's error varies with its parameters; it is the topographic map of the optimization process.

Optimization methods The methods by which a neural network finds an optimal solution within the complex loss landscape.

Parameter initialization Parameter initialization concerns how to choose a suitable starting point in the loss landscape to launch the training process, helping the model train faster and find better solutions; it is also strongly tied to the model's generalization ability.

Are neural networks omnipotent?

Given that neural networks can accomplish many complex tasks, can they perform well on any data fitting problem we throw at them? The answer is no.

No Free Lunch Theorem The core idea of this theorem is that, averaged over all possible problem distributions, all algorithms have the same performance. This theorem tells us that neural networks are not omnipotent and cannot achieve optimal performance on all tasks.

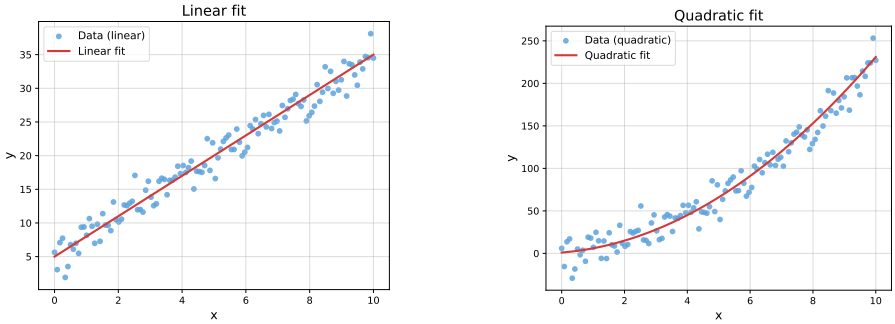
In this chapter, we briefly introduce some basic knowledge of deep learning, hoping to inspire readers' curiosity about deep learning and their enthusiasm for understanding it. While deep learning has its intuitive charm, truly establishing an understanding still requires effort to observe experimental phenomena and to master the theoretical foundations. The subsequent chapters of this book will help readers gain a deeper understanding of the underlying phenomena and mechanisms of deep learning.

1.1 Data Fitting

In the world we live in, we encounter all kinds of data everywhere. From daily life to scientific research, we are constantly engaging with and analyzing various kinds of data. For example, by observing temperature data of a city at different times, we can study its climatic characteristics; or a doctor may collect data on the symptoms and signs of many patients to judge the type and severity of a disease. These examples all illustrate the importance of data for understanding the world and solving problems.

However, simply collecting data does not directly let us see the patterns hidden behind it. We need to use certain mathematical tools and computational methods to mine the valuable information embedded in the data; this is what we usually call **data fitting**. As shown in Figure 1.1, the left subfigure fits the data with a linear function, while the right one fits it with a quadratic

function. Data fitting aims to learn a mathematical model from data that closely approximates the objective law generating the data. We refer to the functional expression of this law as the **target function**. In deep learning, for image classification problems, a reference target function can be viewed as being given by the biological neural network in our brain. For example, when constructing image classification datasets such as ImageNet, besides collecting a large number of images, substantial human effort is required to label them, thereby forming a dataset that uses the human brain as reference. For the ImageNet dataset with one thousand classes, the best current deep learning algorithms have achieved a remarkable test accuracy of about 90%. It is worth noting that this does not necessarily indicate that the learned neural network classification function is already sufficiently close to the human-brain classification function. We often observe that an imperceptible perturbation can drastically change the output of a neural network classifier. For instance, a neural network that generalizes well on natural images may fail to predict accurately when the input image is corrupted by noise. Seeking better deep learning algorithms that more closely approximate the human-brain classification function, especially for noise-corrupted image inputs, remains an important research question in deep learning.



(a) Linear fit

(b) Quadratic fit

Figure 1.1: Illustration of data fitting

Formally, we denote the data as $S = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n \subset \mathcal{X} \times \mathcal{Y}$, where $\mathcal{X} \subset \mathbb{R}^d$ is called the sample set and $\mathcal{Y} \subset \mathbb{R}^{d_o}$ is called the label set. Here d is the data dimension, d_o is the output dimension, and n is the number of samples. For convenience of exposition, we assume the output dimension is $d_o = 1$. The target function is typically denoted by $f^* : \mathcal{X} \to \mathcal{Y}$. Throughout this book, we assume by default that the data are drawn from the target function, i.e., $y_i = f^*(\boldsymbol{x}_i)$ for $i = 1, \cdots, n$. Here, $\mathcal{X}$ denotes the input space and $\mathcal{Y}$ the output space, while $\boldsymbol{x}_i$ and y_i respectively represent the input and output of the i-th observation. We denote the machine learning model we use by $f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$, where $\boldsymbol{\theta}$ is the model's parameters. As $\boldsymbol{\theta}$ varies, $\{f_{\boldsymbol{\theta}}\}_{\boldsymbol{\theta}}$ forms a function space. Given a dataset, we hope to find the best function from this function space to approximate the target function behind the data. To this end, we typically define a **loss function** to measure the gap between the model's predictions and the true values, and then adjust $\boldsymbol{\theta}$ via an optimization algorithm to minimize the loss function. This process is what we usually call model training. By this means, we hope that the resulting model $f_{\boldsymbol{\theta}}$ will be sufficiently close to the target function f^* over the entire input space, and in particular on unseen inputs.

Remark 1. *Linear fitting is a fundamental algorithm in machine learning, whose model is* $f_{\boldsymbol{\theta}}(\boldsymbol{x}) =$

$\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}$, *with parameters* $\boldsymbol{\theta} = (\boldsymbol{A}, \boldsymbol{b})$.

Remark 2. *Deep learning is the most important machine learning algorithm of our time; it uses neural network models to fit data. A simple two-layer fully connected neural network model can be expressed as* $f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{j=1}^{m} a_j \sigma(\boldsymbol{w}_j \cdot \boldsymbol{x} + b_j)$*, where the parameters are* $\boldsymbol{\theta} = \{(a_j, \boldsymbol{w}_j, b_j)\}_{j=1}^{m}$*, and* σ *is a nonlinear function, often called the activation function.*

To characterize the predictive performance of a model, we need to choose a reasonable loss function to measure the distance between the model and the target function. The loss function is typically denoted by $\ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}), y)$, which measures the gap between the predicted label and the true label. An intuitive and widely used loss function is the L^2 loss, also known as the Mean Squared Error (MSE), expressed as:

$$\ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}), y) = \ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}), f^*(\boldsymbol{x})) = (f_{\boldsymbol{\theta}}(\boldsymbol{x}) - f^*(\boldsymbol{x}))^2.$$

Typically, the target function is unknown, and only the training data (or training set) is available. The quality of the model's fit on the training set is measured by the value of the loss function on the training set, namely the training error (also called the empirical risk), defined as:

$$R_S = \frac{1}{n} \sum_{i=1}^{n} \ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), y_i). \tag{1.1}$$

In practice, we obtain a fitted model by minimizing the training error, i.e.,

$$\min_{\boldsymbol{\theta}} R_S.$$

This is the well-known Empirical Risk Minimization (ERM) problem.

For a general machine learning method, such as a neural network, evaluating the gap between the fitted function and the target function is crucial for assessing the algorithm. Theoretically, for data whose inputs follow distribution $\mathcal{D}$, this gap is usually measured by the population risk (or generalization error) of the fitted function:

$$R_{\mathcal{D}} = \mathbb{E}_{\boldsymbol{x} \sim \mathcal{D}} \ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}), f^*(\boldsymbol{x})),$$

where $\mathbb{E}_{\boldsymbol{x} \sim \mathcal{D}}$ denotes expectation with respect to the input distribution.

Remark 3. *We note that if the input distribution* $\mathcal{D}$ *is concentrated on a low-dimensional manifold, a small generalization error does not imply that the fitted function is close to the target function on the whole space. A simple example: the data are distributed on the unit circle* ($x_1^2 + x_2^2 = 1$) *in a two-dimensional space, which is a one-dimensional manifold embedded in two-dimensional space. A small generalization error on these data does not mean the fit is good on the entire two-dimensional space. In weather forecasting, for instance, although a model may exhibit very low generalization error on historical data, indicating it can accurately predict most everyday weather, this does not mean it can accurately predict under extreme climate conditions (such as a rare blizzard or extreme heat), since such situations rarely appear in the data and the model's fit there may be insufficient. Similarly, in image classification, although we have achieved low generalization error on the distribution of noise-free natural images using deep learning—and natural images lie on a specific manifold—for the entire image space, especially on natural-image inputs corrupted by specific noise, these neural network classification functions still differ greatly from the target function.*

In practice, since the target function is generally unavailable, we cannot exactly compute the above expectation. The usual approach is to collect another set of test data $\{(\boldsymbol{x}_j^{test}, y_j^{test})\}_{i=1}^{n'}$ and compute the average loss of the fitted function on this dataset as an estimate of the generalization error. The loss on this test set is called the test loss:

$$R_{test} = \frac{1}{n'} \sum_{j=1}^{n'} \ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}_j^{test}), f^*(\boldsymbol{x}_j^{test})).$$

We usually define the difference between the generalization error and the training error as the generalization gap, i.e.,

$$R_{gap} = R_{\mathcal{D}} - R_S.$$

In practical training, we can shrink the generalization gap by increasing the number of training samples. In this way, when the model achieves a small training error R_S, the generalization error R_D will also be small, meaning that the fitted function is sufficiently close to the target function on the distribution $\mathcal{D}$.

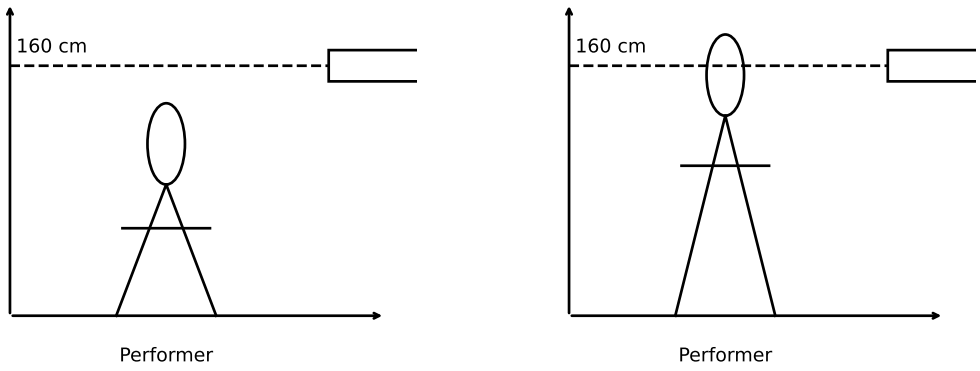
The test set is mainly used to evaluate the generalization ability of the trained model, but it cannot be used directly for tuning the model; otherwise, information about the test set would be implicitly leaked, biasing the evaluation and undermining its fairness. For this reason, a validation set that is unseen during training and is distinct from the test set is usually introduced, to monitor and adjust model performance during training. By analogy, the validation set is like a mock exam, used for periodic checks of learning progress and to guide subsequent learning strategy, while the test set is like the actual final exam, intended to comprehensively and objectively evaluate the model's overall ability.

1.2 Introduction to Neural Networks

In this section, we briefly introduce how neurons function and how neural network architectures mimic neurons.

1.2.1 How a single neuron senses information

The basic condition for sensing information is the ability to distinguish different inputs. To distinguish different pieces of information, we must first be able to extract meaningful content from the signal. For example, suppose we are conscripting soldiers, and one's height must exceed 160 cm to pass the initial screening. So we design a height-information extractor.



(a) Below 160 cm (b) Reaches 160 cm

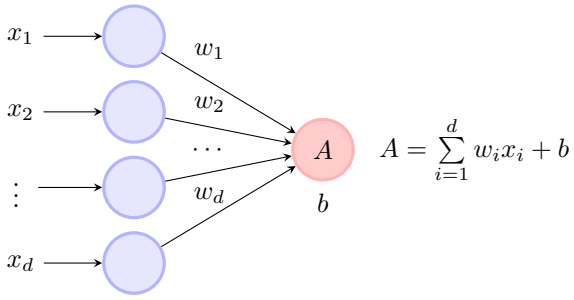
Figure 1.2: Height extractor

As shown in Figure 1.2, we place a laser emitter and receiver at a height of 160 cm. The emitter continuously emits a signal; if the applicant's height reaches 160 cm, the signal is reflected back, and once the receiver detects the signal, the indicator shows "pass," otherwise "fail." Notice that, even though the full information about height is a specific number, we actually do not need the full information—we only need to know whether it exceeds 160 cm, and we only need to output 0 or 1, i.e., one bit of information. So the role of this apparatus can be described as the following threshold function:

$$\gamma(x) = \begin{cases} 1 & x \geq T, \\ 0 & x < T. \end{cases} \tag{1.2}$$

The basic function performed by a neuron is also usually simplified to a similar function: when the strength of an external input exceeds some threshold T, the neuron outputs a spike signal, i.e., 1; otherwise it outputs 0. The key question is how information is transmitted to the neuron.

Let's look at an example. Suppose A has two friends, B and C. Each day they give A information about a stock; this information is a number between 0 and 1, indicating the probability they assign to the stock going up. Their information is denoted b and c, respectively. Even if B and C give no information, A himself has a baseline judgment a_0. If B is an economist and C is an ordinary person, A may consider B more trustworthy, and thus assign B and C weights λ_1 and λ_2, with $\lambda_1 > \lambda_2 > 0$. Ultimately, the information A receives is $a = \lambda_1 b + \lambda_2 c + a_0$. We can generalize this way of transmitting information. Suppose there are d information sources, $x_1, \cdots, x_d$, the neuron's weights for these inputs are $w_1, \cdots, w_d$, and the neuron's own bias is b, as shown in the figure below.



Finally, the total input received by the neuron is

$$w_1 x_1 + \cdots + w_d x_d + b, \tag{1.3}$$
$$= (w_1, \cdots, w_d) \cdot (x_1, \cdots, x_d)^T + b, \tag{1.4}$$
$$= \boldsymbol{w}^T \boldsymbol{x} + b. \tag{1.5}$$

$(\boldsymbol{w}, b)$ are the parameters of the neuron and embody its properties. Let us also understand the way the neuron processes information from a projection viewpoint: $\boldsymbol{w}^T \boldsymbol{x}$ is the inner product of two vectors, which can be understood as the projection of the information $\boldsymbol{x}$ onto the neuron's feature direction $\boldsymbol{w}$.

To further improve the nonlinear fitting ability of neural networks, we wrap the output with a nonlinear function $\sigma(\cdot)$, so that the output of a single neuron becomes $\sigma(\boldsymbol{w}^T \boldsymbol{x} + b)$; this nonlinear function is usually called the activation function.

Remark 4. *The activation function is an indispensable part of a neural network. Theoretically, as long as the activation function is a nonlinear function that is not a polynomial, the neural network can approximate any continuous function of any order. As shown in Figure 1.3, common activation functions include the following:*

- *ReLU:* $\text{ReLU}(x) = max(x, 0)$
- *Tanh:* $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- *Sigmoid:* $\text{Sigmoid}(x) = \frac{1}{1+e^{-x}}$

1.2.2 Single-layer neural network

By simply putting multiple neurons together, we obtain a single-layer neural network (the input is not counted as a layer).

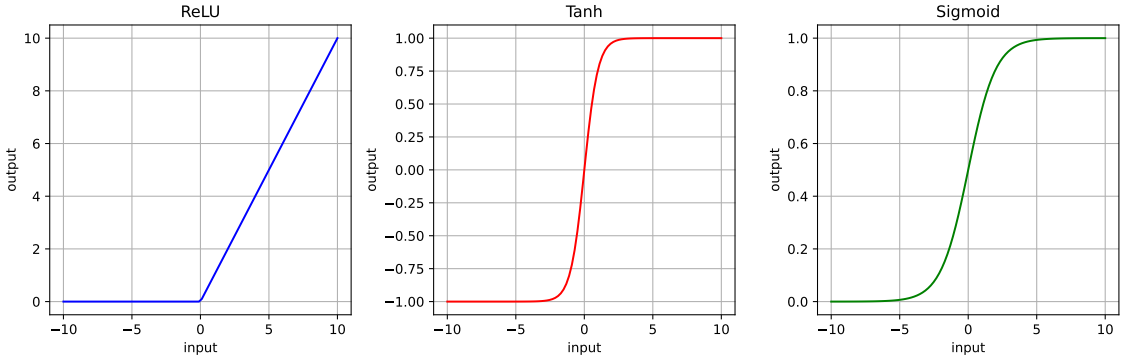
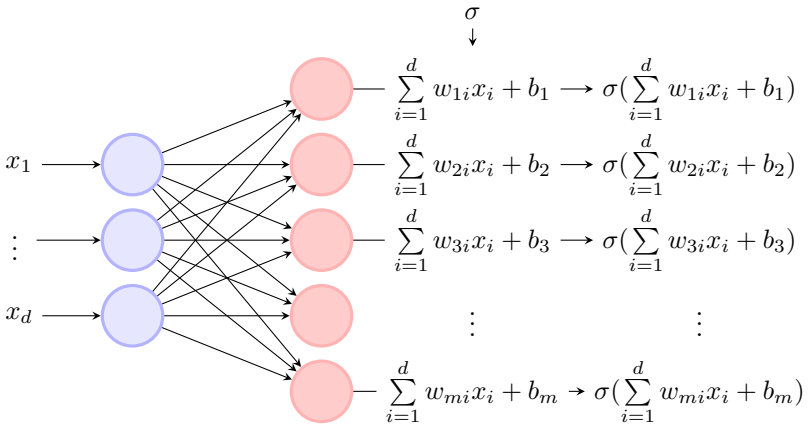


Figure 1.3: Common activation functions



Mathematically, we can write the parameters in the general form:

$$W = \begin{pmatrix} w_{11} & \cdots & w_{1d} \\ \vdots & \ddots & \vdots \\ w_{m1} & \cdots & w_{md} \end{pmatrix}. \tag{1.6}$$

$$\boldsymbol{b} = (b_1, b_2, \cdots, b_m)^T. \tag{1.7}$$

The output of the network is

$$y = \sigma \circ (W\boldsymbol{x} + \boldsymbol{b}), \tag{1.8}$$

where $\circ$ means applying the activation function entrywise. For example, $\sigma \circ ([0.1, 0.3, 0.5]^T) = [\sigma(0.1), \sigma(0.3), \sigma(0.5)]^T$. In the early days of neural network development, due to limited computing power, single-layer neural networks played the dominant role.

From a purely design viewpoint, all neuron parameters are set manually, and a single neuron can perform many logical operations. For example, for the AND operation, consider a two-dimensional

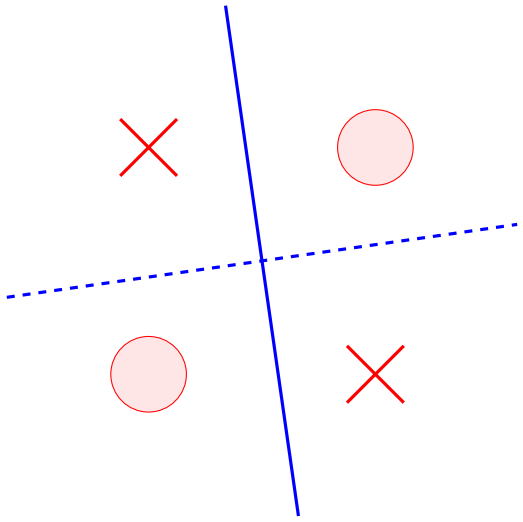


Figure 1.4: Illustration of the XOR problem. The function value is 1 at the circles and 0 at the crosses. As one can see, no linear partition (blue solid or dashed line) can fully separate the two classes.

input $x_1, x_2 \in \{0,1\}^2$, where each coordinate is either 0 or 1. Taking the threshold $T = 2$, the neuron's output is

$$\sigma(x) = \begin{cases} 1 & x_1 + x_2 \geq T, \\ 0 & x_1 + x_2 < T. \end{cases} \tag{1.9}$$

Only when both coordinates are 1 is the output 1, and 0 otherwise—this is the AND operation. If we take $T = 1$, the same neuron performs the OR operation. Recall that the elementary operating units of modern computers are precisely logical operations, so even without considering rules for learning the neuron's parameters, people had already realized that neural networks can accomplish many tasks.

As people delved deeper, they found that single-layer neurons can only solve linearly separable problems Minsky and Papert (1988). For the XOR problem shown in Figure 1.4 (XOR is a logical operation; on binary inputs in two dimensions, it outputs 0 for the combinations $\{0,1\}, \{1,0\}$ and 1 for the combinations $\{0,0\}, \{1,1\}$), since it is not linearly separable, a single-layer neural network cannot solve it well; at least a two-layer network is required. And the difficulty of training multi-layer networks at that time discouraged many researchers, ushering in the first winter of neural networks.

1.2.3 Multi-layer neural networks

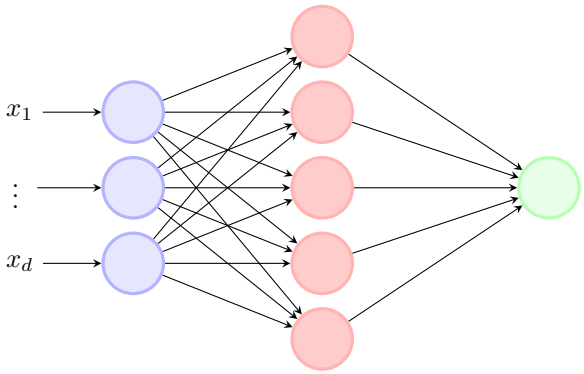


Figure 1.5: A two-layer neural network

Multi-layer neural networks are built by stacking on top of a single-layer neural network. Take a two-layer network as an example, shown in Figure 1.5: the computation from the input layer to the first layer is exactly the same as in a single-layer network, and then, taking the first layer as input, the computation from the first layer to the second layer is another single-layer network. The output of the first layer is invisible to the user and is used internally to process information, and is therefore also called the hidden layer.

Formally, a two-layer neural network is expressed as:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{j=1}^{m} a_j \sigma \circ (\boldsymbol{w}_j \cdot \boldsymbol{x} + b_j), \tag{1.10}$$

where m is the number of neurons in the hidden layer, σ is the activation function, $\boldsymbol{w}_j$ is the input weight, a_j is the output weight, and b_j is the bias. We denote the parameter set by $\boldsymbol{\theta} = (a_1, \cdots, a_m, \boldsymbol{w}_1, \cdots, \boldsymbol{w}_m, b_1, \cdots, b_m)$.

More generally, for a multi-layer neural network with L layers, the network can be expressed as:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{W}^{[L-1]}\sigma \circ (\boldsymbol{W}^{[L-2]}\sigma \circ (\cdots \sigma \circ (\boldsymbol{W}^{[1]}\sigma \circ (\boldsymbol{W}^{[0]}\boldsymbol{x} + \boldsymbol{b}^{[0]}) + \boldsymbol{b}^{[1]}) \cdots) + \boldsymbol{b}^{[L-2]}) + \boldsymbol{b}^{[L-1]}, \tag{1.11}$$

where $\boldsymbol{W}^{[l]} \in \mathbb{R}^{m_{l+1} \times m_l}, \boldsymbol{b}^{[l]} \in \mathbb{R}^{m_{l+1}}, m_0 = d_{in} = d, m_L = d_o$, σ is a vector-valued function, and "$\circ$" denotes entry-wise operation (for example, entrywise multiplication). Note that the input layer is not counted in the layer count (i.e., the number of layers equals the number of hidden layers plus the output layer). We denote the parameter set by

$$\boldsymbol{\theta} = (\boldsymbol{W}^{[0]}, \boldsymbol{W}^{[1]}, \cdots, \boldsymbol{W}^{[L-1]}, \boldsymbol{b}^{[0]}, \boldsymbol{b}^{[1]}, \cdots, \boldsymbol{b}^{[L-1]}),$$

and the elements of $\boldsymbol{W}^{[l]}$ by $\boldsymbol{W}_{ij}^{[l]}$. The network can also be defined recursively:

$$f_{\boldsymbol{\theta}}^{[0]}(\boldsymbol{x}) = \boldsymbol{x} \tag{1.12}$$

$$f_{\boldsymbol{\theta}}^{[l]}(\boldsymbol{x}) = \sigma \circ (\boldsymbol{W}^{[l-1]} f_{\boldsymbol{\theta}}^{[l-1]}(\boldsymbol{x}) + \boldsymbol{b}^{[l-1]}), \quad 1 \le l \le L-1 \tag{1.13}$$

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = f_{\boldsymbol{\theta}}^{[L]}(\boldsymbol{x}) = \boldsymbol{W}^{[L-1]} f_{\boldsymbol{\theta}}^{[L-1]}(\boldsymbol{x}) + \boldsymbol{b}^{[L-1]}. \tag{1.14}$$

There are several ways to understand neural networks; we list a few below.

Linear–nonlinear transformations. When given a data point, the most direct approach is to apply a linear transformation followed by a bias, i.e., an affine transformation. But linear functions cannot handle the many nonlinear problems found in practice. So we wrap the result of the linear transformation with a nonlinear activation function. Yet one such wrap is not enough, so we perform multiple linear transformations and nonlinear activations, and feed each result as the input to another linear–nonlinear transformation. This pattern is also common in biological brains; many similar linear–nonlinear transformations have been found in the primary visual cortex.

Projection to a high-dimensional space. The input dimension of the data is given by the problem. In that space, fitting or classification problems may be difficult. So, using nonlinear activation functions paired with different parameters, we map the data to another high-dimensional space, whose dimension is the number of neurons in the hidden layer, and then perform linear fitting. If a single projection is not enough, we can do multiple projections. This is analogous to kernel methods in traditional approaches, except that in neural network models the kernel itself is also a learnable object.

Monte Carlo sampling in function space. For a two-layer network, if we add a factor of $1/m$ on the right-hand side of Equation 1.10, the expression looks like a weighted average of m functions, or an empirical expectation under some probability distribution—a discrete approximation of a continuous expectation.

Two-layer and deeper neural networks have very strong representational ability; here we construct an example of a two-layer neural network that solves the XOR problem.

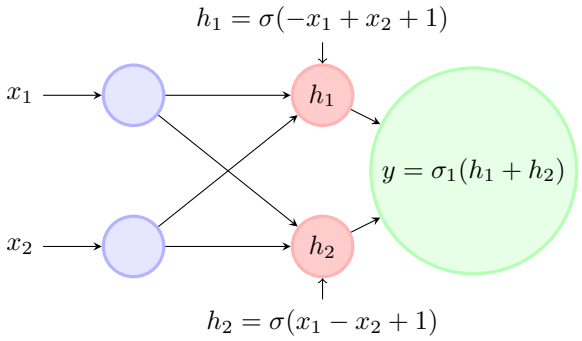


Figure 1.6: A two-layer network solving the XOR problem

As shown in Figure 1.6, take an activation function σ with threshold 1.5. When $x_1 = x_2 = 0$ or $x_1 = x_2 = 1$, we have $h_1 = h_2 = \sigma(1)$; while when $x_1 = 1, x_2 = 0$ or $x_1 = 0, x_2 = 1$, we have $h_1 + h_2 = \sigma(0) + \sigma(2)$. This solves the XOR classification problem.

Furthermore, in the late 1980s, a series of works proved that when the activation function is nonlinear and non-polynomial, and when the hidden layer of the neural network is sufficiently wide, a two-layer neural network can approximate to arbitrary accuracy any continuous function from a finite-dimensional space $\mathcal{X}$ to a finite-dimensional space $\mathcal{Y}$. This is the universal approximation theorem (Cybenko, 1989; Hornik et al., 1989; Leshno et al., 1993).

Subsequently, the academic community began systematically analyzing the approximation error of neural networks. Early work focused mainly on the approximation performance and optimality

of ReLU networks (Yarotsky, 2017, 2018; Lu et al., 2021; Shen et al., 2020, 2022). For example, for a ReLU network with n total parameters, the optimal error rate for approximating Lipschitz functions on $[0,1]^d$ is $O(n^{-2/d})$. On this basis, (Zhang et al., 2024) extended such ReLU-based results to a large class of activation functions, covering nearly all common cases.

As research deepened, the field began to pursue higher-order approximation efficiency. By introducing periodic activation functions such as sine, cosine, and floor, networks can achieve exponential or near-exponential approximation rates (Yarotsky and Zhevnerchuk, 2020; Shen et al., 2021). Going further, researchers discovered that there exists a class of activation functions that allows a fixed-size network to achieve arbitrary approximation accuracy: (Maiorov and Pinkus, 1999) first established the existence; (Yarotsky, 2021) provided an explicit construction; and (Shen et al., 2022) provided a concise, computable construction.

The universal approximation theorem shows that neural networks have strong approximation power and provides an important theoretical underpinning for their use. At the same time, we should note that the theoretical solution constructed by the universal approximation theorem may not be reachable during actual training. Therefore, a neural network's capacity to approximate is only a necessary condition for being a good model; whether it can actually fit a given function well depends on many factors of the training process. This theory also differs significantly from neural networks used in practice. For instance, even though two-layer networks already have strong expressive power, why do we generally use deeper networks in practice? Could the number of neurons grow too large for practical training? Can the solutions constructed in the proof actually be found in practice? What training methods do we need, and how much data? Many such questions cannot be answered by the universal approximation theorem.

In fact, the universal approximation theorem cannot distinguish neural-network fitting from polynomial fitting. Polynomial functions are also dense in the space of continuous functions, that is, polynomials can also approximate continuous functions to arbitrary accuracy—this is the famous but less popularized Weierstrass approximation theorem. In the field of artificial intelligence, "naming" is a frequently used technique: for a new research result, giving it a good name is very important. If the universal approximation theorem were replaced by a person-named theorem, it might be harder to gain wide attention. Of course, "naming" is important in every field.

1.3 Common Loss Functions

The loss function used to train a neural network often differs depending on the task. Generally speaking, designing a loss function that is aligned with the goal and is easy to train is one of the most important subtasks of a machine learning project. For supervised learning tasks, the simplest and most commonly used loss function is the mean squared error between the sample labels and the network outputs, but for problems without explicit labels, such as generative adversarial networks, differential equation solving, and reinforcement learning, one cannot design the loss function as directly. We will discuss loss function design for specific problems in later sections. In this section, we mainly introduce mean squared error loss, mean absolute error loss, and cross-entropy.

1.3.1 Mean squared error loss

The mean squared error (MSE) loss is one of the most commonly used loss functions for regression problems, also known as the L^2 loss. Consider a dataset $S = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^n$ and a model $f_{\boldsymbol{\theta}}(\boldsymbol{x})$,

where $\boldsymbol{\theta}$ are the learnable parameters. The MSE can be expressed as:

$$R_{MSE}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\|f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \boldsymbol{y}_i\|_2^2. \tag{1.15}$$

1.3.2 Mean absolute error loss

The mean absolute error (MAE) loss is another commonly used loss function for regression problems, also known as the L^1 loss. The MAE can be expressed as:

$$R_{MAE}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\|f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \boldsymbol{y}_i\|_1. \tag{1.16}$$

1.3.3 Cross-entropy

For classification problems, cross-entropy is more commonly used than MSE or MAE, mainly because in classification the interpretation of the model output is often based on probability distributions. Cross-entropy measures the difference between two probability distributions, one representing the distribution of true labels and the other the distribution of predicted outputs. For binary classification, the cross-entropy loss is typically expressed as:

$$H(y,p) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) + (1-y_i)\log(1-f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))\right], \tag{1.17}$$

where y_i is the true label of sample i (0 or 1), and $f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$ is the predicted probability that sample i belongs to class 1. For multi-class problems, the cross-entropy loss can be extended to:

$$H(y,p) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} y_{i,c}\log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)_c), \tag{1.18}$$

where C is the total number of classes, $y_{i,c}$ is an indicator variable equal to 1 if sample i belongs to class c and 0 otherwise, and $f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)_c$ is the predicted probability that sample i belongs to class c.

The cross-entropy loss can be derived from Maximum Likelihood Estimation (MLE). MLE is a method for estimating the parameters of a statistical model; it finds the best estimate of the model parameters by maximizing the likelihood of the observed data. In a classification problem, suppose we have a probabilistic model that predicts the probability of each class; we want to choose parameters such that the probability of observing the data distribution is maximized.

For binary classification, suppose p_i is the model's predicted probability that sample i is positive, and $1-p_i$ is the predicted probability that it is negative. If the true label is $y_i = 1$, the likelihood of the observed data is p_i; if $y_i = 0$, the likelihood is $1-p_i$. Therefore, given all samples, the likelihood of the entire dataset is:

$$L = \prod_{i=1}^{N} p_i^{y_i}(1-p_i)^{(1-y_i)}. \tag{1.19}$$

Taking the log-likelihood yields:

$$\log L = \sum_{i=1}^{N} \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right]. \tag{1.20}$$

Maximizing the log-likelihood is equivalent to minimizing its negative, so maximizing the log-likelihood is equivalent to minimizing the cross-entropy loss. This is how the cross-entropy loss is derived from maximum likelihood estimation. In multi-class problems, the derivation is similar, except that the probability over each class for each sample needs to be considered.

Softmax. When using the cross-entropy loss for multi-class classification, the entries of the model's output vector must lie between 0 and 1 and sum to 1. To achieve this, a softmax operation is usually appended after the model output. Suppose the model output is $f_{\boldsymbol{\theta}}(\boldsymbol{x}) = (z_1, z_2, \cdots, z_C)$; the softmax operation transforms the model output to:

$$\text{softmax}(f_\theta(x)) = \left(\frac{e^{z_1}}{\sum_{i=1}^{C} e^{z_i}}, \cdots, \frac{e^{z_C}}{\sum_{i=1}^{C} e^{z_i}} \right). \tag{1.21}$$

It is worth noting that, although cross-entropy is widely used in classification problems and has a nice probabilistic interpretation, it is just one form of loss function that drives the model output to be consistent with the data label. The previously introduced MSE and MAE losses can also fulfill the same role. Moreover, although we often interpret the model output after the softmax transformation as the model's inference about the probability distribution over classes, this interpretation has no firm statistical basis: we cannot infer that the input has an extremely high probability of belonging to a certain class just because the model predicts the probability of that class to be almost 1. After actual training, the output values merely reflect the model's prediction about the input sample based on the training data, and not the actual posterior probability.

Perplexity. In natural language, perplexity is often used to characterize the learning quality, and is defined as

$$PPL = 2^{H(y,p)}. \tag{1.22}$$

This is a concept from information theory; it can be used to measure the uncertainty of a random variable, and can also be used to measure how well a model is trained. Generally, the higher the perplexity of a random variable, the higher its uncertainty; the higher the perplexity at inference time, the worse the model's performance, and vice versa.

Intuitively, we can think of it as the number of guesses needed to hit the target. For example, suppose the dictionary contains ten words and the target output is the first word. At the current learning state, the first five words each have output probability 1/5 and the rest are 0. We can compute the cross-entropy to be $\log_2 5$, and the perplexity to be 5. That is, to guess which word the current output actually is, one has to choose from five candidates—the model is uncertain about which one.

1.4 Loss Landscape

The empirical risk function (loss function) $R_S(\boldsymbol{\theta})$ of a neural network, and especially the geometric shape of the loss value as $\boldsymbol{\theta}$ varies, is often called the loss landscape. We use the term "landscape"

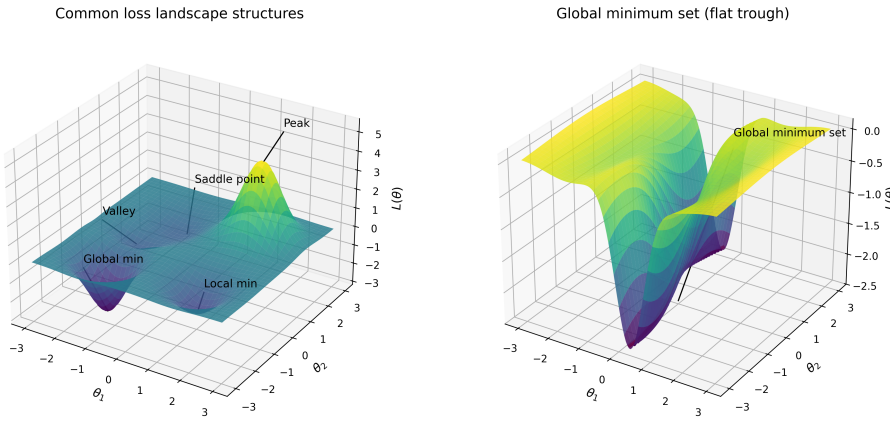


Figure 1.7: Some structures present in the loss landscape

to emphasize the similarity between understanding $R_S(\boldsymbol{\theta})$ and understanding terrain landscapes: (1) peaks (local maxima) and valleys (local minima) are important; we often need to find suitable paths to particular peaks or valleys, such as the lowest valley (global minimum). (2) Convexity is important: a convex landscape guarantees that from any point one can directly "see" the lowest valley, so finding the lowest valley (optimization) is easy, whereas non-convex landscapes are usually much more difficult. (3) For non-convex landscapes, saddle points are important: a saddle point is an ideal "rest stop" and is often the lowest pass from one valley to another. Using this intuitive analogy, we can understand training a neural network as an explorer searching for the lowest valley in a winding, complex terrain. Figure 1.7 shows some common structures present in the loss landscape.

For deep learning algorithms, studying the structure of the loss landscape has special significance: (1) Although the loss landscape is highly non-convex, we find that sufficiently large neural networks usually have no trouble converging to a global minimum; bad local minima that obstruct training are uncommon. To give a vivid example of a bad local minimum: a blind person descending a mountain probes the surrounding terrain with a cane at some point along the slope and finds that everything around is higher than his current spot; he then stays partway down the mountain and cannot reach the foot. But in practice, neural networks usually do reach the foot, suggesting that the loss landscape must have some special structure supporting good optimization. (2) The structure of the loss landscape affects not only optimization performance but also generalization. Since in practice we often use over-parameterized neural networks, the lowest valley of the loss landscape is usually not a single point but a high-dimensional manifold consisting of infinitely many points. The various global minima on this manifold have different generalization properties, and most have large generalization error. Surprisingly, the loss landscape of a neural network seems to have some special structure such that, under common initializations, various optimization algorithms are guided toward those points with good generalization.

The above two points tell us that whether one studies optimization or generalization of neural networks, the loss landscape is an unavoidable key research object. One can foresee that, in the deep learning theory of the future, the theory of the loss landscape will occupy an important position.

1.5 Optimization Methods

In the previous content, we discussed how to use the loss function to measure the performance of a deep learning model, and we have seen that the loss landscape of deep learning exhibits a high-dimensional and non-convex complex structure. Therefore, finding the optimal solution in such a loss landscape is a challenging task. To efficiently optimize model parameters, deep learning widely adopts gradient-based optimization methods to reduce the loss function value. The core idea of gradient-based optimization is to use the gradient information of the objective function to guide the optimization process.

Among various optimization methods, gradient descent is the most basic one. Let's use a simple example to understand it. Suppose you are a mountain-climbing enthusiast and your goal is to reach the lowest point of the valley. But you are at the top of the mountain, surrounded by fog, unable to see the path down. How should you descend? An intuitive method is: look around, find the steepest downhill direction, and take a step in that direction. Then look around again, find the steepest downhill direction, and take another step. Repeat this process until you reach the bottom of the valley. This process embodies the basic idea of gradient descent. In deep learning, the high-dimensional, complex loss landscape is hard to imagine, and we cannot directly "see" the steepest descent direction. This is where the concept of the gradient comes in. The gradient is a vector that points in the direction of fastest ascent of the loss function. So we only need to go in the opposite direction of the gradient, and we can take a step in the direction of fastest decrease of the loss. This is the core idea of gradient descent.

Optimization methods are an important component of deep learning. Effective optimization algorithms can accelerate model training, improve learning efficiency, and help find better solutions. Currently, deep learning primarily uses first-order, gradient-based optimization methods, whose core is computing the gradient of the loss with respect to parameters. In this section, we briefly introduce the back-propagation paradigm followed when computing gradients in deep learning, gradient descent and its stochastic variant, and several variants of gradient descent proposed to improve convergence speed or model performance.

1.5.1 Gradient computation: back-propagation

In deep neural networks, computing the gradient is not a simple matter. Neural networks are usually composed of many layers of nonlinear transformations, and updating the parameters of each layer depends on the gradient information from later layers. In the field of deep learning, gradient computation is also called Back Propagation (BP), because it follows a backward-propagation process from the network output to the input, recursively computing the gradient at each layer. Here, the importance of the "naming" approach is once again on display: in essence, this is just the chain rule for derivatives, but renaming it makes it sound fuller and deeper, and more like a new thing.

Below we use a two-layer neural network $f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{j=1}^{m} a_j \sigma(\boldsymbol{w}_j \cdot \boldsymbol{x} + b_j)$ as an example. We likewise consider the mean squared error loss (to make the expression after differentiation cleaner, we add a coefficient $1/2$ to the standard MSE loss):

$$R_S = \frac{1}{2n} \sum_{i=1}^{n} (y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))^2. \tag{1.23}$$

For the outermost-layer parameter a_j, using gradient descent to update it gives:

$$\frac{\partial R_S}{\partial a_j} = \frac{1}{n}\sum_{i=1}^{n}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - y_i)\sigma(\boldsymbol{w}_j \cdot \boldsymbol{x}_i + b_j). \tag{1.24}$$

For $\boldsymbol{w}_j$, since it is not an outermost-layer parameter, we need to apply the chain rule multiple times:

$$\begin{aligned}
\frac{\partial R_S}{\partial \boldsymbol{w}_j} &= \frac{1}{n}\sum_{i=1}^{n}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - y_i)\frac{\partial f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)}{\partial \boldsymbol{w}_j} \\
&= \frac{1}{n}\sum_{i=1}^{n}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - y_i)\frac{\partial(\sum_{t=1}^{m} a_t\sigma(\boldsymbol{w}_t x_i + b_t))}{\partial \boldsymbol{w}_j} \\
&= \frac{1}{n}\sum_{i=1}^{n}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - y_i)a_j\frac{\partial(\sigma(\boldsymbol{w}_j \cdot \boldsymbol{x}_i + b_j))}{\partial \boldsymbol{w}_j} \\
&= \frac{1}{n}\sum_{i=1}^{n}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - y_i)a_j\frac{\partial\sigma(x)}{\partial x}|_{x=\boldsymbol{w}_j\cdot\boldsymbol{x}_i+b_j}\frac{\partial(\boldsymbol{w}_j x_i)}{\partial \boldsymbol{w}_j} \\
&= \frac{1}{n}\sum_{i=1}^{n}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - y_i)a_j\frac{\partial\sigma(x)}{\partial x}|_{x=\boldsymbol{w}_j\cdot\boldsymbol{x}_i+b_j}\boldsymbol{x}_i.
\end{aligned}$$

Because of the chain rule, the order of gradient computation is $f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) \to a_j \to \sigma \to \boldsymbol{x}_i$, while the order of information flow is $\boldsymbol{x}_i \to \sigma \to a_j \to f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$. The two orders are opposite, which is why it is called back-propagation.

Remark 5. *Because of the chain rule, when differentiating the loss function with respect to parameters, we obtain a product of several expressions. For a very deep network, these products may become very large, causing training instability (gradient explosion), or very small, making training impossible (gradient vanishing). We will discuss solutions to these issues in later chapters on neural network architectures.*

Remark 6. *Many people attribute the discovery of the back-propagation algorithm to the 1986 paper by D. Rumelhart, G. Hinton, and R. Williams. But in fact this 1986 paper explicitly states that it was not the first discovery; David Parker and Yann LeCun, among others, had previously discovered the algorithm. What makes the 1986 paper outstanding is that it describes the algorithm very clearly and demonstrates its effectiveness on complex problems. Here we cannot help but mention another very important research approach in this field, "archaeology." As is well known, the field of deep learning has had several ups and downs, and each wave produced many good ideas that did not flourish for various reasons. When compute, data, and training algorithms become suitable, these buried algorithms may once again influence the academic community. For example, using neural networks to solve differential equations: many good ideas had already emerged in the 1990s; but only after being rediscovered in 2016 did they spread widely across many fields.*

1.5.2 Gradient descent

With gradient information, the most basic optimization algorithm is Gradient Descent (GD). Gradient descent uses the negative gradient direction as the search direction and updates parameters with a fixed step size (i.e., the learning rate). The algorithm is as follows:

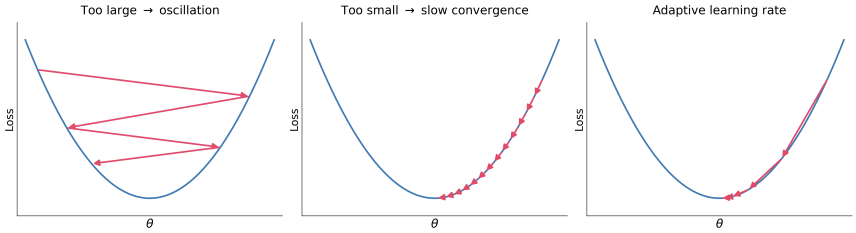


Figure 1.8: Effects of different learning rates on training

$$g_t = \nabla_\theta R_S(\theta_{t-1}), \tag{1.25}$$
$$\theta_t = \theta_{t-1} - \eta g_t. \tag{1.26}$$

Here η is the learning rate (also called step size).

We use the Taylor expansion to explain why gradient descent can guarantee a decrease in the loss when the step size is small.

At θ_{t-1}, the first-order Taylor expansion of the loss function $R_S(\theta)$ is:

$$R_S(\theta_t) \approx R_S(\theta_{t-1}) + (\theta_t - \theta_{t-1})^T \nabla R_S(\theta_{t-1}).$$

Plugging in the gradient descent update rule yields:

$$R_S(\theta_t) \approx R_S(\theta_{t-1}) + (-\eta \nabla R_S(\theta_{t-1}))^T \nabla R_S(\theta_{t-1}) \tag{1.27}$$
$$= R_S(\theta_{t-1}) - \eta \|\nabla R_S(\theta_{t-1})\|^2 \tag{1.28}$$

From the above, we have:

$$R_S(\theta_t) \approx R_S(\theta_{t-1}) - \eta \|\nabla R_S(\theta_{t-1})\|^2 \leq R_S(\theta_{t-1}). \tag{1.29}$$

This means that, as long as the gradient is nonzero and the learning rate is small enough that the first-order Taylor expansion approximates the loss function well, the loss will strictly decrease after the update. This explains why gradient descent can reduce the loss function.

During neural network training, the learning rate is a crucial hyperparameter; it controls the step size in the gradient descent direction. Only with an appropriate learning rate can we successfully achieve good training results. Taking gradient descent as an example, if the learning rate is too large, as shown in the left panel of Figure 1.8, the network parameters will oscillate and fail to converge, bringing instability to training; whereas if the learning rate is too small, as shown in the middle panel of Figure 1.8, the network will need many epochs (an epoch refers to one complete training cycle of the neural network) to reach the minimum, wasting computational resources.

We can understand this instability from a mathematical perspective. Take the quadratic function

$$L = \frac{1}{2}\lambda\theta^2$$

as an example. In high-dimensional space, λ is an eigenvalue of the Hessian (the matrix of second derivatives of the high-dimensional function), and η is the learning rate. According to the gradient-descent update rule,

$$\theta_{t+1} = \theta_t - \eta \frac{\partial L}{\partial \theta},$$

the linear stability condition for the loss to be monotonically decreasing requires

$$|\theta_{t+1}| < |\theta_t|,$$

i.e.,

$$|1 - \lambda\eta| < 1,$$

which gives the relationship between the learning rate η and λ:

$$\eta < \frac{2}{\lambda}. \tag{1.30}$$

This shows that only when the learning rate is less than $\frac{2}{\lambda}$ can the gradient descent training be stable. For stability analysis of stochastic gradient descent, see Wu et al. (2018).

1.5.3 Stochastic optimization methods

With the arrival of the big-data era, deep learning faces unprecedented opportunities and challenges. Massive training data provides richer learning material for models, helping to improve generalization and robustness. On the other hand, large-scale data also poses severe tests for traditional optimization methods. When the amount of data far exceeds the capacity of computation and storage, classical optimization algorithms such as gradient descent become inadequate.

Against this backdrop, stochastic optimization methods emerged and became a breath of fresh air in deep learning optimization. The core idea of stochastic optimization is to use only a small portion of the data at each iteration to update model parameters. This greatly reduces the memory overhead of each parameter update and enables efficient training.

The most representative algorithm of stochastic optimization is Stochastic Gradient Descent (SGD). The principle of SGD is to randomly select $|B|$ samples from all samples to estimate the loss function:

$$R_B = \frac{1}{n} \sum_{i=i_1}^{i_{|B|}} (f_{\boldsymbol{\theta}}(x_i) - y_i)^2, \tag{1.31}$$

where $\{i_1, i_2, \cdots, i_{|B|}\}$ may be chosen randomly at each step, or fixed in advance. SGD performs gradient descent using R_B as the loss function. Because not all samples are used, the computed gradient is not the global one, hence the name stochastic gradient descent.

In practice, the samples are usually divided into several portions, each with $|B|$ data points; each portion is called a batch. In each epoch, one batch is sequentially used to perform gradient descent until all batches have been used once. After completing one epoch, a common practice is to shuffle the data to prevent the model from becoming dependent on the data order, thereby improving generalization. However, in certain situations, such as time series analysis or training certain recurrent neural networks, the order of the data contains important contextual information, in which case it may be better not to shuffle the data and to continue directly with the next epoch, preserving the temporal continuity and contextual integrity of the data.

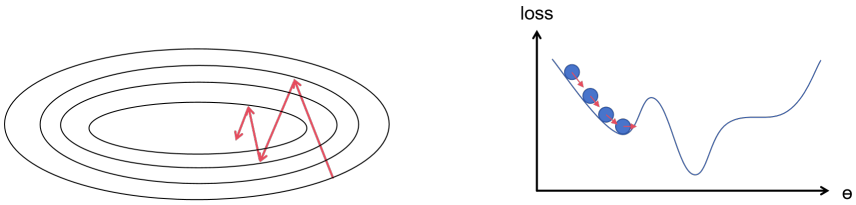


Figure 1.9: Oscillatory behavior of gradient descent and the principle of momentum-based gradient descent. In the left subfigure, the red solid line and arrows depict the optimization trajectory under gradient descent. The right subfigure shows that, when using gradient descent for optimization, if a local minimum is encountered the gradient becomes very small and the optimization stalls.

The impact of stochastic optimization is profound. It has not only changed the training paradigm of deep learning but also provided new ideas for other large-scale optimization problems. In machine learning, operations research and optimization, financial engineering, and other fields, stochastic gradient-type methods are receiving increasing attention and have become powerful tools for solving complex optimization problems. Although the original motivation for introducing SGD was simply to address the computational and memory cost issues, later practice has found that, in most cases, SGD also has advantages over full-data gradient descent in terms of generalization. As a result, the theoretical study of SGD has become an important and active direction of research.

1.5.4 Gradient descent with momentum

In the previous subsections, we introduced gradient descent and its stochastic version, which use gradient information to guide optimization. However, in some cases plain gradient descent may not perform well.

First, as shown in the left side of Figure 1.9, when the contours of the objective function form long, narrow ellipses, gradient descent tends to exhibit "zigzag" oscillations. This is because along the long axis of the ellipse the gradient component is small, so the update step is limited; along the short axis the gradient component is large, so the update step is too big, leading to back-and-forth oscillation. In this case, optimization becomes very slow.

Second, when the loss landscape has many local optima or saddle points, gradient descent easily gets stuck and finds it hard to escape. This is because near a local optimum or a saddle point, the gradient becomes very small in magnitude, and the optimization process almost grinds to a halt.

To overcome these difficulties, researchers introduced the concept of momentum. The core idea of the momentum method is that at each update we consider not only the current gradient direction but also inherit the direction of previous updates. As shown on the right side of Figure 1.9, the red arrow denotes the direction of the momentum-based gradient; even at a local minimum, the "ball" still has momentum, so momentum-based gradient descent gives the optimization a certain "inertia," allowing it to cross long valleys and local optima and move toward a better direction.

Intuitively, momentum is like a ball rolling in a valley. When the ball rolls downward, it is influenced not only by the current slope (gradient) but also retains its previous direction and speed of motion. This "inertia" lets the ball descend rapidly on steep slopes, and once at the bottom, continue forward until new forces stop it.

A basic momentum-based gradient update is given as follows:

$$g_t = \nabla_\theta R(\theta_{t-1}), \tag{1.32}$$
$$m_t = \mu m_{t-1} + g_t, \tag{1.33}$$
$$\theta_t = \theta_{t-1} - \eta m_t. \tag{1.34}$$

Here μ is a hyperparameter, and m_0 can be set to 0.

Nesterov The momentum method was first proposed in Polyak (1964), and the Nesterov Accelerated Gradient (NAG) algorithm (Nesterov, 1983), widely used in deep learning, improves upon the traditional momentum method: it first takes a step along the previous direction and then computes the gradient to correct the direction. This is like climbing a mountain and being able to anticipate the next foothold according to the current plan, and then reconsidering the current decision. This "look-ahead" strategy allows NAG to converge faster to the optimum.

$$g_t = \nabla_\theta R(\theta_{t-1} - \eta\mu m_{t-1}), \tag{1.35}$$
$$m_t = \mu m_{t-1} + g_t, \tag{1.36}$$
$$\theta_t = \theta_{t-1} - \eta m_t. \tag{1.37}$$

1.5.5 Adaptive optimization methods

Adaptive optimization algorithms are a class of optimization algorithms that can automatically adjust the learning rate based on data characteristics during model training. Compared with traditional optimization algorithms (such as gradient descent), adaptive optimization algorithms can dynamically adjust the learning rate of each parameter, thereby more effectively coping with the gradient variations of different parameters during training. These algorithms compute adaptive learning rates by performing moving averages over gradient information or accumulating past updates. Common adaptive optimization algorithms include AdaGrad (Duchi et al., 2011) and Adam (Kingma and Ba, 2014). A key advantage of adaptive optimization algorithms is that they can handle sparse data, high-dimensional spaces, and tasks with non-stationary gradients, improving the convergence speed and stability of the model. Owing to their flexibility, adaptive optimization algorithms have become widely used tools in modern deep learning, and they perform especially well when training complex neural networks.

Adagrad The key idea of AdaGrad is to adapt the learning rate of each parameter by accumulating the squares of past gradients, thereby achieving adaptive learning-rate adjustment. This method addresses the issue of using a fixed learning rate in traditional gradient descent when training models on high-dimensional sparse data. It automatically lowers the learning rate of frequently updated parameters while keeping a relatively high learning rate for sparsely updated parameters, thus handling unbalanced data or features better. Its specific form is as follows:

$$g_t = \nabla_\theta R(\theta_{t-1}), \tag{1.38}$$
$$v_t = v_{t-1} + g_t^2, \tag{1.39}$$
$$\theta_t = \theta_{t-1} - \eta g_t / \sqrt{v_t + \epsilon}. \tag{1.40}$$

However, this method introduces a new problem: in late training, due to long-term accumulation, v_t becomes too large, causing learning to slow down too much. The role of ϵ here is to keep the denominator from being zero.

Adadelta In Adagrad, the update of v_t accumulates the squared gradients without bound, leading to v_t becoming too large and learning too slow in late training. Therefore, Adadelta updates v_t via an exponentially weighted moving average, so that v_t grows smoothly.

$$g_t = \nabla_\theta R(\theta_{t-1}), \tag{1.41}$$

$$v_t = \nu v_{t-1} + (1-\nu) g_t^2, \tag{1.42}$$

$$\theta_t = \theta_{t-1} - \eta g_t / \sqrt{v_t + \epsilon}. \tag{1.43}$$

Here ν is a hyperparameter. **RMSProp** is the special case of Adadelta with $\nu = 0.5$.

Adam Adam combines momentum with adaptive learning rates, further improving optimization performance, and has achieved state-of-the-art results on tasks such as image recognition and language modeling.

$$g_t = \nabla_\theta R(\theta_{t-1}), \tag{1.44}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{1.45}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{1.46}$$

$$\hat{m}_t = \frac{m_t}{1-(\beta_1)^t}, \tag{1.47}$$

$$\hat{v}_t = \frac{v_t}{1-(\beta_2)^t}, \tag{1.48}$$

$$\theta_t = \theta_{t-1} - \eta \hat{m}_t / \sqrt{\hat{v}_t + \epsilon}. \tag{1.49}$$

Here β_1 is the first-moment hyperparameter, β_2 is the second-moment hyperparameter, and η is the learning rate.

Remark 7. *Reddi et al. (2018) established a theoretical analysis of Adam's convergence, pointing out that Adam is not guaranteed to converge—it can even diverge on some simple convex problems. Zhang et al. (2022) identified the key difference between the assumptions of prior theoretical work and real application scenarios: previous theory first fixes β_1 and β_2 and then constructs a problem on which Adam diverges; whereas in real scenarios, we first fix the problem and then tune the hyperparameters β_1 and β_2 based on the specific problem. That paper proved that when the second-moment parameter β_2 is sufficiently large and the first-moment parameter satisfies $\beta_1 < \sqrt{\beta_2} < 1$, Adam can converge to the neighborhood of a critical point. A critical point is a point where the gradient is zero, i.e., $\nabla R_S(\theta) = 0$.*

There are also some recently developed algorithms, such as Adan (Xie et al., 2024) and Sophia (Liu et al., 2023), used to better optimize complex models.

1.6 Parameter Initialization

Parameter initialization, as the name suggests, is the process of setting the initial values of model parameters. It seems simple but has a crucial influence on training and convergence of the model.

Different initializations lead to different training dynamics and also affect the final generalization of the model. When we use a relatively small initialization, the network may start at a saddle point at the beginning of training, leading to slow training or even failure to train; when we use a relatively large initialization, the network may produce poor generalization.

Generally, we hope the initial parameters satisfy the following properties:

- Symmetry breaking: the initial parameters cannot all be the same value, otherwise every neuron in every layer would learn the same features, losing the meaning of the multi-layer structure.
- Maintain moderate variance: the variance of the initial parameters cannot be too large or too small, otherwise it leads to gradient explosion or vanishing, affecting training stability.
- Consider nonlinearity: for different nonlinear activation functions, the initial parameters should follow correspondingly different distributions, to ensure effective propagation of information in the network.

We use a fully connected network as an example to introduce several common initialization methods. Let m_{in} and m_{out} denote the input dimension and the output width of the current layer, i.e., the parameters of a hidden layer are $\boldsymbol{\theta} \in \mathbb{R}^{m_{out} \times m_{in}}$. The following are some common initializations:

Lecun Initialization (LeCun et al., 2012)

- Normal-distribution initialization $\theta \in \boldsymbol{\theta} \sim \mathcal{N}(0, \frac{1}{m_{in}})$.
- Uniform-distribution initialization $\theta \in \boldsymbol{\theta} \sim \mathcal{U}(-\sqrt{\frac{3}{m_{in}}}, \sqrt{\frac{3}{m_{in}}})$.

Xavier Initialization (Glorot and Bengio, 2010)

- Normal-distribution initialization $\theta \in \boldsymbol{\theta} \sim \mathcal{N}(0, \frac{2}{m_{in}+m_{out}})$.
- Uniform-distribution initialization $\theta \in \boldsymbol{\theta} \sim \mathcal{U}(-\sqrt{\frac{6}{m_{in}+m_{out}}}, \sqrt{\frac{6}{m_{in}+m_{out}}})$.

Kaiming Initialization (He et al., 2015)

- Normal-distribution initialization $\theta \in \boldsymbol{\theta} \sim \mathcal{N}(0, \frac{2}{m_{in}})$.
- Uniform-distribution initialization $\theta \in \boldsymbol{\theta} \sim \mathcal{U}(-\sqrt{\frac{6}{m_{in}}}, \sqrt{\frac{6}{m_{in}}})$.

Note that all three common initializations above, whether normal or uniform, share the property that their magnitudes decrease as the width of the network increases, scaling like $1/\sqrt{m}$ (taking the standard deviation into account for Gaussian distributions). This is mainly because, when designing these initializations, one needs to ensure that when the signal $\boldsymbol{x}$ in the neural network flows between layers $l-1$ and l, the strength (i.e., the output of each layer) does not blow up or vanish from layer to layer,

$$\boldsymbol{x}^{[l]} = \boldsymbol{W}^{[l]}\sigma(\boldsymbol{x}^{[l-1]}) + \boldsymbol{b}^{[l]}, \tag{1.50}$$

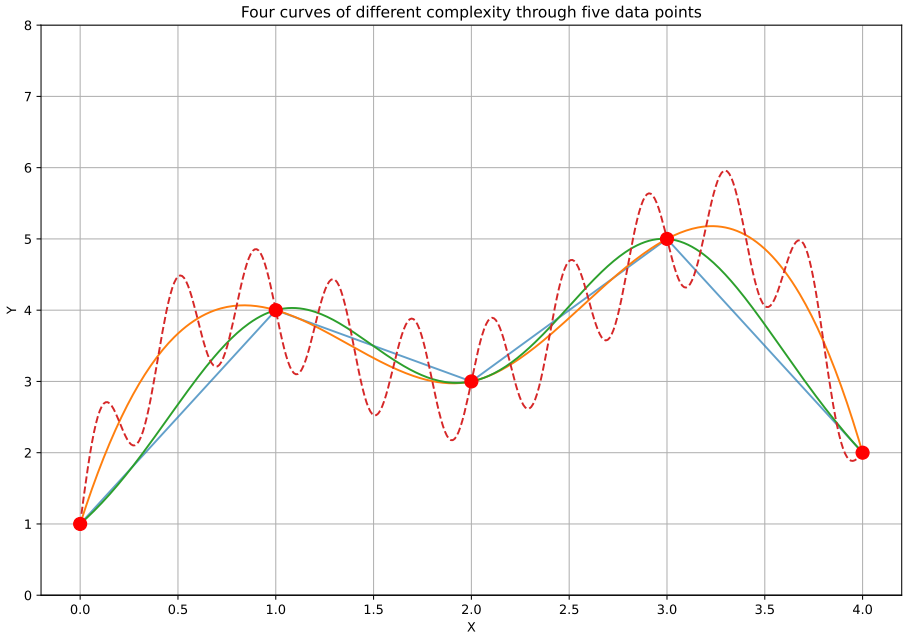


Figure 1.10: Fitting the same set of data points in different ways.

That is, if (1.50) represents the signal flowing between two layers, then we need to satisfy $\mathrm{Var}(\boldsymbol{x}_j^{[l]}) = \mathrm{Var}(\boldsymbol{x}_i^{[l-1]})$. Intuitively, each neuron in the next layer must sum up the outputs of all neurons in the previous layer multiplied by new parameters. Assuming each neuron of the previous layer is independent and has variance $1/m$, the sum of Gaussian-distributed quantities is still Gaussian, with variance equal to the sum of all the variances. Hence the variance of the sum over previous-layer neurons is 1, and since the parameter has variance $1/m$, each neuron in this layer has variance $1/m$, the same as the previous layer. This means that the parameters of the network must shrink as the width increases. At the same time, the random initialization rarely makes any two parameters take the same value.

For more on initialization, see Narkhede et al. (2022). In addition, different initializations often lead to different training dynamics for the neural network, which we will discuss in detail in later chapters.

It should be especially noted that many initialization methods with variance of order $1/m$ have been packaged into commonly used coding frameworks, such as PyTorch and Tensorflow. As a result, deep learning users rarely hear that they need to tune the initialization. However, in current large-model training, adjusting the variance of the initialization distribution is in fact very important and very common. In later chapters, we will also find that the variance of the initialization distribution affects the memorization and reasoning ability of large language models.

1.7 No Free Lunch

The No Free Lunch Theorem (Wolpert and Macready, 1997) is a very famous theorem in the field of machine learning. Its core idea is that, averaged over all possible problem distributions, all algorithms have the same performance. This means that no single algorithm can perform optimally on all possible problems. In other words, an algorithm's advantage on certain specific problems must come at the cost of disadvantages on other problems. For example, consider the situation shown in Figure 1.10: different solid lines represent models fitted to five data points using different algorithms Wu et al. (2017). Each solid line achieves an excellent fit when the remaining data are distributed nearby, but its performance drops significantly when the remaining data are distributed near the dashed line. This example vividly illustrates the essence of the No Free Lunch Theorem: the quality of an algorithm always depends on the specific characteristics of the problem.

The formal statement of the theorem is as follows:

Theorem 1 (No Free Lunch)**.** *Given a finite dataset X and a finite label set Y, let Y^X denote the set of all mappings from data to labels. To optimize a randomly chosen mapping $f : X \to Y$ from Y^X on the set X, no method is better than blind search.*

This theorem was originally proposed by David H. Wolpert and William G. Macready in 1997. In their study of optimization algorithms, they found it impossible to design a universal algorithm that performs best on all problems. This indicates that, considered over all possible problems, the average performance of every algorithm is the same. The theorem has several variants but their basic point is the same: there is no absolute "best" algorithm. Readers interested in the detailed proof may refer to (Wolpert and Macready, 1997).

This theorem emphasizes the importance of choosing an algorithm that matches a specific problem. It tells us that we should choose algorithms based on the specific characteristics and needs of the problem, rather than seeking a universal solution or discussing algorithm performance divorced from dataset features. Taking deep learning as an example, research shows that neural networks generally struggle to learn and generalize parity functions and Fourier transforms (Shalev-Shwartz et al., 2017; Nye and Saxe, 2018). The target functions of these two kinds of tasks are typically highly oscillatory, and neural networks find such functions hard to learn. In later chapters we will go deeper into a class of functions that neural networks struggle with and the mechanisms behind it. But if we turn our gaze to real-world datasets, neural networks display extraordinary capability. In image recognition, speech recognition, natural language processing, and many other fields, neural network models keep setting performance records, and even surpass humans on some tasks. The reason for this discrepancy is that different problems have target functions with different characteristics behind their data.

The No Free Lunch Theorem is a cornerstone of the field of machine learning. It reminds us that in the world of machine learning there is no once-and-for-all panacea, only context-specific "prescriptions." Recognizing this, we can move forward steadily on the path of using and developing machine learning algorithms.

1.8 Understanding Deep Learning

Although deep learning has achieved significant success in many real-world applications, its theoretical foundation has not yet fully revealed the capabilities and limitations of neural network algorithms. This means that, when developing and applying deep learning algorithms, we still rely

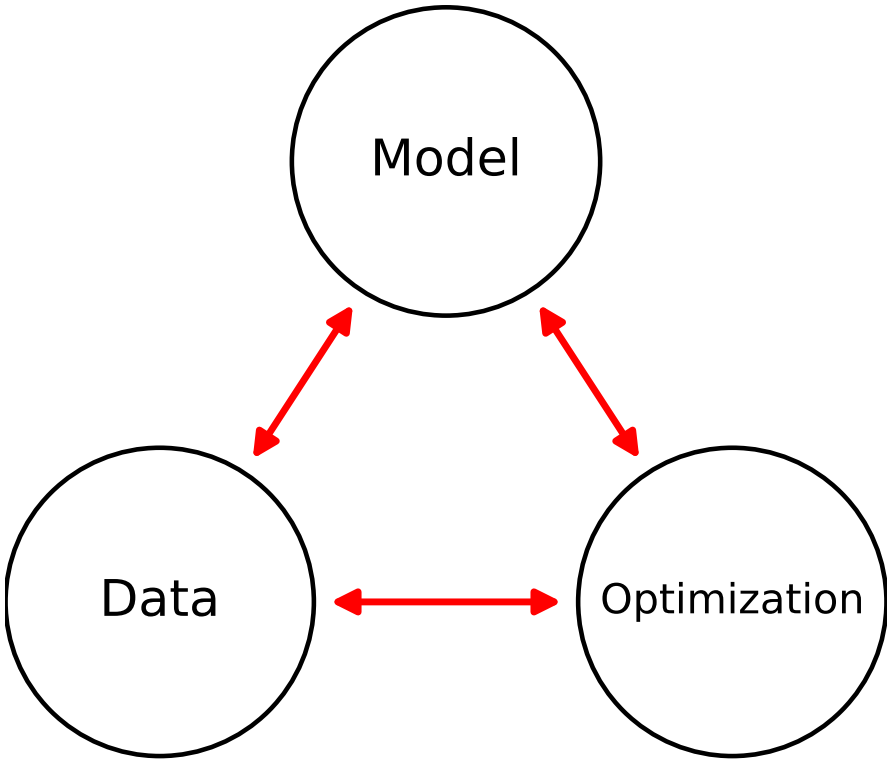


Figure 1.11: The three elements of a machine learning algorithm

heavily on accumulated experience and repeated trial-and-error to push progress. Therefore, further deepening our theoretical understanding of deep learning algorithms is not only key to improving the efficiency of algorithm design, but also a core driving force for the future development of deep learning.

1.8.1 The basic elements of deep learning

Understanding a machine learning algorithm essentially requires understanding the influence of three elements—model, data, and optimization—on the algorithm's performance in fitting the target function (such as generalization error). As shown in Figure 1.11, these three elements influence each other. For some traditional machine learning algorithms, such as kernel methods, we have a clear and quantitative understanding of these influences. However, for deep learning algorithms, the composition of these elements and their influence on fitting performance are much more complex, and establishing theoretical understanding poses a far greater challenge. Below we first give a framework-level introduction to the three elements of deep learning algorithms and their influence on fitting performance.

The fundamental feature of deep learning algorithms is that the model belongs to a special family of brain-inspired nonlinear models, namely neural networks. Many complex tasks, such

as image recognition, natural language processing, and Go, can be fundamentally solved once a suitable neural network is used. This does not mean that other task-specific elements—such as task modeling, data collection and cleaning, optimization tricks, and engineering methods—are not important; but we must clearly recognize the central role of the neural network model in the overall solution, just like an engine is to an aircraft. Going further, neural network models include several components: architecture, scale, activation functions, etc. Among them, the architecture (e.g., fully connected, convolutional, residual, LSTM, Transformer, etc.) is the key. For example, when OpenAI's Radford was training a language model to predict the next character in Amazon user reviews, he found that switching from LSTM to Transformer led to more progress in two weeks than in the prior two years. Beyond the architecture, a sufficiently large scale (including sufficient depth, width, and number of parameters) is an important guarantee for the neural network model to achieve excellent performance. The power-law relationship between model parameters, data amount, and compute, proposed by OpenAI in Kaplan et al. (2020), is called the Scaling Law. Research on Scaling Laws shows that, even on relatively limited training data, increasing model size generally does not significantly hurt performance. This stands in stark contrast to the basic property in traditional machine learning that larger models are more prone to overfitting. This remarkable phenomenon is known as the over-parameterization mystery or the generalization mystery of neural networks, and is one of the key problems in deep learning theory research. In addition, activation functions also have an important modulating effect on the optimization and generalization of the model.

On the data side, the emergence of deep learning algorithms has given us the ability to solve a series of previously unsolvable complex tasks, such as Go, image recognition, and natural language processing. The target functions behind these problems are usually very complex and have no explicit expression. However, they also share some common characteristics, namely that humans can usually perform them well. Many things that humans cannot do well, such as multiplying large numbers, the strongest current large language models also struggle with. Beyond the target function, the amount of data is another key element of the dataset. Scaling Law experiments show that without sufficient training data, many complex tasks, such as in-context learning, cannot be done well. In addition, the data distribution also significantly affects fitting error; for example, a neural network trained on clean natural images is hard to achieve a good test error on images corrupted by noise. Generally, a neural network trained on a low-dimensional manifold in the input space is usually unable to approximate the target function well over the entire space.

On the optimization side, initialization, namely the choice of initial values of the network weights, not only greatly affects training speed but also significantly affects the network's generalization performance. This is mainly because we often use over-parameterized neural networks to fit the training data. In this setting, the loss landscape has infinitely many global minima with different generalization errors. In traditional machine learning theory, such a setting would lead to overfitting, but in neural network practice this often does not occur. Under common initializations, the network usually converges to a global minimum with good generalization, but at the same time, unreasonable initializations can easily lead to overfitting. In addition, the training strategy of the network is also an important element. A suitable training strategy can improve training efficiency while ensuring good generalization; for example, in training large language models, extensive experiments show that AdamW can achieve the best results. Furthermore, dropout, batch normalization, and other optimization tricks can also bring improvements in training efficiency or generalization.

1.8.2 Deep learning theory

Neural network theory has been developed for decades and is still incomplete. Current theory does not yet adequately explain the influence of data, model, and optimization on fitting performance. As a result, when we use and develop deep learning algorithms, we rely mainly on experience and extensive trial-and-error to make progress. The reasons for this are multifaceted; we believe the following two are the most important: (1) Neural networks and their training process can in some sense be regarded as a high-dimensional and highly nonlinear complex system. Although we can obtain the activation of every neuron and the size of every weight, and the training process can be precisely recorded, the complex nonlinear interactions among these quantities make it very difficult to observe and analyze the statistical regularities of the overall training and prediction process; it has essentially become a "black box." (2) We can hardly analyze high-dimensional, structurally complex real data, nor can we describe the target functions behind such data. According to the No Free Lunch Theorem, the core determinant of neural network performance is the match between the algorithm and specific data. In principle, we can understand the "preferences" of neural networks by analyzing the data on which they perform well. However, our understanding of the structure and underlying target functions of such data, like image classification and protein folding, is extremely limited; they are essentially also "black boxes."

From a machine learning perspective, opening either of the "black boxes"—real data or the neural network—is an extremely important research goal. However, in practice, these two "black boxes" are often tightly coupled, which adds great difficulty to observing and analyzing the intrinsic structure of either. Faced with this challenge, a basic strategy in our research is to first use synthetic data to dissect the regularities and preferences of the neural network "black box," and then use those regularities and preferences to analyze the structure of the real-data "black box." From another perspective, since deep neural network models play the role of an "engine" in solving many complex problems, dissecting the model itself also has special significance. The following chapters of this book deepen the introduction of the significant progress current research has made in opening the neural network "black box." We also look forward to gradually delving into the dissection of the real-data "black box." Through everyone's joint efforts, we hope that in the future we no longer use "alchemy" to describe the design, use, and improvement of deep learning algorithms.

1.8.3 The generalization mystery and implicit bias of neural networks

Deep neural networks, with their powerful expressive ability and flexible learning ability, have achieved remarkable success on a wide variety of machine learning tasks. However, despite the astonishing performance demonstrated in practice, our theoretical understanding of their generalization ability is still very limited.

As early as 1995, Professor Breiman raised the generalization mystery shown in Figure 1.12: why do over-parameterized neural networks not exhibit significant overfitting. Specifically, in his paper (Breiman, 1995), he keenly pointed out that many results in statistical learning theory are built on complexity. Complexity can be very roughly reflected by the number of model parameters, or rigorously characterized by VC dimension or Rademacher complexity. VC dimension is a way to measure model complexity: it represents the maximum number of samples the model can classify in all possible ways. Rademacher complexity measures the highest accuracy a model achieves after fully shuffling the labels of the dataset. Take linear fitting as an example: when the number of model parameters exceeds the number of samples, multiple possible solutions can arise. In this case, some parameter combinations may significantly reduce the model's generalization ability.

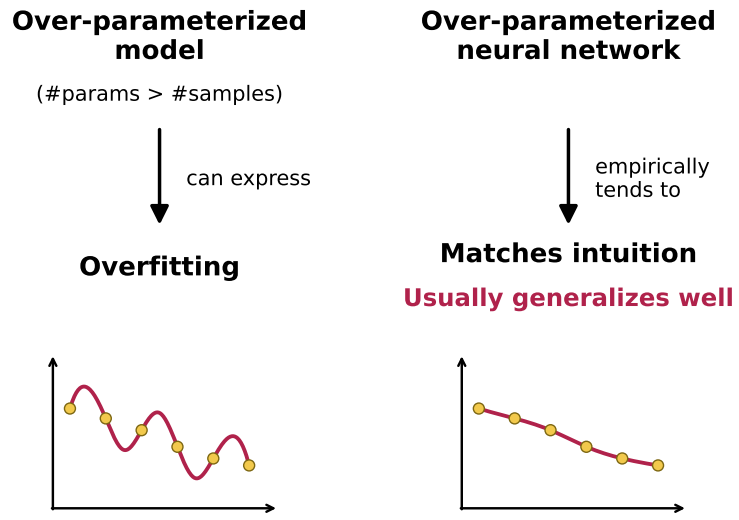


Figure 1.12: Illustration of the generalization mystery

Therefore, when choosing model complexity, one must strike a balance between fitting ability and generalization. As shown in Figure 1.13, traditional learning theory suggests that if a model's complexity is too high, the gap between training and test errors will widen, leading to degraded generalization performance.

But in the real world, many machine learning models (especially neural networks) often have far more parameters than the number of training samples. Traditional theory holds that such over-parameterization must lead to overfitting, with the model completely memorizing the features of the training data and thus losing the ability to generalize to new data. However, in practice, over-parameterized neural networks not only easily fit the training data very well, but often also achieve excellent performance on test data. This suggests that deep neural networks have some remarkable generalization ability that frees them from the curse of overfitting. Breiman thus raised a great question: what exactly gives over-parameterized neural networks their good generalization? What is the mechanism behind it? This question later came to be known as the "Generalization Puzzle."

Although the generalization mystery was raised as early as 1995, since deep neural networks had not yet entered large-scale practical use at the time, the question did not attract much attention. It was not until recent years, with deep neural networks shining in many fields, that this mystery once again drew interest. In 2017, Zhang Chiyuan and others, in their ICLR paper (Zhang et al., 2017), systematically empirically demonstrated the existence of the generalization mystery, sparking widespread discussion in the academic community and eventually winning that year's best-paper award.

Zhang et al. (2017) adopted a technique called "random labels." Specifically, they replaced the true labels of the original training set with completely random labels and then trained standard neural networks on the data with these random labels. Several common neural network architectures

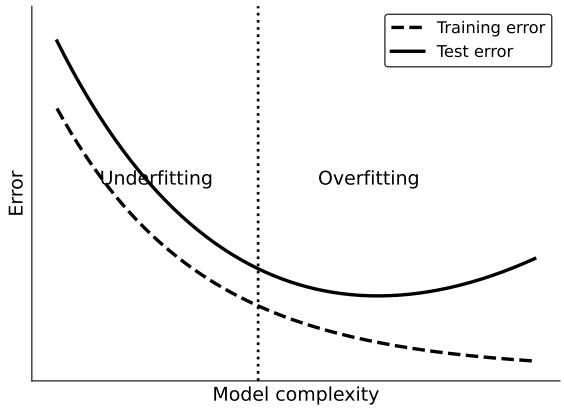


Figure 1.13: The U-shaped curve corresponding to traditional statistical learning theory; the dashed line denotes training error and the solid line denotes test error.

could easily learn the training set with random labels very well: the training error kept decreasing until it was close to zero. The network's strong fitting ability indicates that its model complexity is extremely high, sufficient to "brute-force memorize" huge amounts of training samples. Yet such high-complexity models still have very high generalization ability on real problems. Take the network called Inception as an example: with over 1.6 million parameters, when used to classify CIFAR10 (60,000 training images and 10,000 test images), it can achieve nearly 90% accuracy, far above the 10% accuracy of random guessing. In traditional optimization, one typically adds some regularization to such high-complexity models to find solutions that generalize well. To control the uncertainty introduced by regularization, Zhang et al. (2017) conducted multiple experiments, including training with different regularization terms and training with no regularization at all; the models always exhibited strong generalization ability. These experiments show that classical statistical learning theory cannot give a satisfactory explanation for the generalization behavior of deep neural networks.

The generalization mystery reveals a puzzling but unignorable side of deep neural networks. It does not vaguely ask why neural networks are so powerful; it sharply points to the paradox between the number of parameters and generalization performance. This paradox poses a severe challenge to the theoretical foundation of deep learning:

On the one hand, from the optimization perspective, a large number of parameters gives the model extremely strong expressive ability, enabling it to learn highly complex patterns embedded in the training data. From this perspective, the more parameters the better. On the other hand, from the generalization perspective, too many parameters mean the model can easily overfit and lose the applicability of the learned patterns to new data. From this perspective, the fewer parameters the better.

The relationship between the amount of data and the number of model parameters is a key consideration in machine learning. For linear models, having more model parameters than data leads to overfitting, and fewer than data leads to underfitting. However, for neural network models, this trade-off rarely happens in practice. In modern deep learning practice, we find that even using particularly large-scale neural networks to fit relatively simple training data can still yield good

generalization. We call this phenomenon the generalization mystery of neural networks; it is a key characteristic of neural network models. This new property—that increasing the model's expressive power does not harm generalization—allows deep learning to break free from the previous constraint that data amount and parameter count must trade off, greatly lowering the difficulty of designing neural network models. In practical training, whenever we find it hard to drive training error low, we usually improve results by enlarging the network.

Conceptually, many researchers attribute the generalization mystery of neural networks to a certain implicit bias (also called implicit regularization) of the neural network algorithm. The reason it is called implicit is that we observe that, in the over-parameterized regime, even without adding any extra regularization to the loss function, neural networks under common initializations still pick out, from the infinitely many global minima, points with specific properties that often generalize well, as if some hidden preference (regularization) is at work behind the training process. From the No Free Lunch perspective, understanding this implicit bias will directly help us figure out which target functions are well-matched to neural networks.

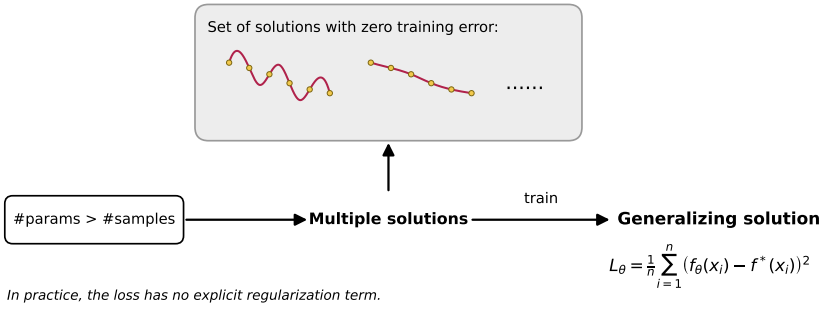


Figure 1.14: Illustration of the implicit bias of neural networks

This book focuses on the general **implicit bias (also known as inductive bias)** of neural networks: what kinds of properties do functions favored by this special model family typically have? The second half focuses on the unique preferences of specific architectures: what functions in its function space does a neural network of a specific architecture prefer? What are the degree and mechanism of the preference? As shown in Figure 1.14, the word "implicit" is used because, although there is no explicit regularization term in the loss function, the neural network can still choose solutions with good generalization during training. The frequency principle and the condensation phenomenon emphasized in this book show that during training neural networks implicitly tend to fit data with low-complexity functions, hoping to explain the generalization mystery in this way.

1.8.4 Research approach: phenomenon-driven theoretical research

In recent years, the complexity of deep learning systems has continuously increased, and their internal mechanisms have become increasingly elusive. Take the GPT series of language models as

examples: their parameter sizes and training costs are jaw-dropping. Even a small flaw could cost an astronomical amount to retrain. Facing such highly complex systems, how should we explore their intrinsic regularities, and how should we guide their future development?

Scientific research tells us that, in exploring complex systems, phenomenon observation and theoretical analysis often go hand in hand and complement each other. It is precisely the rich and diverse phenomena that drive theoretical progress, and mature theory in turn explains an even richer set of phenomena.

Take biology as an example. As organisms evolved, the structure of biological brains became increasingly complex. To understand the workings of the brain, scientists have invested enormous effort in both phenomenon observation and theoretical research. On one hand, they obtained firsthand data on brain structure and activity through dissection, microscopy, brain imaging, and other means; on the other hand, they attempted to build mathematical models to simulate the transmission and processing of information by neurons. It is on the basis of a large accumulation of phenomena that the theoretical system of neuroscience was gradually built and refined. Hubel and Wiesel, through systematic electrophysiological experiments, discovered the selective response characteristics of visual cortex neurons to visual stimuli, laying the theoretical cornerstone of visual information processing. Their work not only revealed the working principles of the visual system but also provided important inspiration for the design of convolutional neural networks.

Throughout the history of science, examples of phenomena driving theory abound. The astronomer Tycho Brahe tirelessly recorded data on celestial motion, providing a solid observational basis for Kepler to discover the laws of planetary motion. Kepler analyzed Tycho's data and summarized the three laws of planetary motion. Although these laws are essentially empirical summaries of observed phenomena, they inspired Newton to establish the system of classical mechanics, achieving a deep understanding and precise prediction of celestial motion.

In the field of electromagnetism, through many experiments, Faraday, Ampère, and other scientists discovered various connections between electric current and magnetic field. Faraday discovered that an electric current can produce a magnetic field, and a changing magnetic field can produce an electric current, revealing the phenomenon of electromagnetic induction. These experimental facts greatly enriched our understanding of electromagnetic interactions. Maxwell, building on the experiments of his predecessors and through profound theoretical reflection, proposed the famous Maxwell equations, achieving the great unification of electromagnetic theory. Maxwell's equations not only explained existing electromagnetic phenomena but also predicted the existence of electromagnetic waves, paving the way for wireless communication.

The development of thermodynamics likewise testifies to the important role of phenomena in driving theory. Through extensive experimental observations, scientists discovered a series of key phenomena such as heat engine efficiency, and distilled corresponding empirical laws. These phenomena and laws constitute the cornerstone of thermodynamic theory. Carnot, by analyzing an idealized heat-engine cycle, derived the upper bound on heat-engine efficiency, laying the foundation for the second law of thermodynamics. Later, Clausius, Kelvin, and others, based on experimental facts, systematically built the theoretical system of thermodynamics, making it an important branch of physics.

In summary, an important paradigm of scientific development is that phenomena are first observed and accumulated, after which theory is proposed and developed. Phenomenon observation points the direction for theoretical research, providing material and basis. Without Tycho's and Kepler's observational data, the birth of Newtonian mechanics would be hard to imagine; without Faraday's and Ampère's experimental discoveries, Maxwell could not have built the edifice of elec-

tromagnetic theory. At the same time, mature theory can in turn explain existing phenomena and predict new ones. Newtonian mechanics not only explains Kepler's laws but also predicts many new astronomical phenomena, such as tides and the motion of comets; Maxwell's equations not only unified electromagnetic phenomena but also predicted the existence of electromagnetic waves, opening a new era of wireless communication. We see that phenomenon accumulation and theoretical research are like two wheels of a cart, or two wings of a bird, jointly driving the progress of science.

Compared with purely theoretical research, phenomenon-driven research has unique advantages. First, it helps us quickly pin down the key influencing factors of a complex system. Pure theoretical research often requires introducing many assumptions and premises, and the derivation is often tedious and time-consuming. By cleverly designing experiments and controlling variables, we can more efficiently explore the intrinsic regularities of a complex system. This is like studying a black-box system: rather than wildly guessing its internal structure, it is better to first observe its external input–output relationship. Second, phenomenon-driven research can provide broader directions and more inspiration for theoretical analysis. When we focus too much on a specific theoretical model, we tend to fall into mental ruts and ignore other aspects of the problem. Observing a wide range of phenomena helps us jump out of fixed theoretical frameworks and discover new research directions and breakthroughs.

However, not every observed result rises to the level of a "phenomenon." A true scientific phenomenon should be a natural fact that occurs stably under specific conditions and exhibits some regularity. Therefore, distilling phenomena from observations requires rigorous and careful experimental design. We need to carefully control experimental conditions, exclude various confounding factors, and ensure reproducibility; we need to systematically record and analyze experimental data and distill key statistical regularities; we also need to examine the stability and universality of the phenomenon under different experimental settings. Only after strict and systematic experimental verification can an observation truly become a scientific phenomenon guiding theoretical research.

In current deep learning research, since both models and data exhibit a high degree of complexity, carefully designing experiments to observe phenomena is particularly important and necessary. On one hand, we lack a comprehensive understanding of the structure and distribution of real-world data, and it is difficult to analyze and predict directly from theory. But we can construct synthetic data with specific structures to examine the learning ability and generalization performance of models. For example, we can design target functions with different frequency characteristics, generate corresponding training data, and then observe how well the model fits, thereby verifying whether the model has properties such as frequency bias. On the other hand, although we still lack a thorough understanding of the internal mechanisms of models, we can infer general rules of how models process information by systematically observing their performance on different tasks. For example, we can compare the performance of models on various visual benchmarks and analyze their sensitivity to different types of features, thereby probing different architectures of models.

Phenomenon research and mathematical theoretical research are both crucial parts of basic research. In the usual sense, (mathematical) theoretical research should include both phenomenon summarization and the proof of mathematical theorems, and should not be narrowly equated with theorem proving. In recent years, some have argued that basic research lags severely behind applied research and contributes little to applications. In fact, this is a very wrong view. On the contrary, it is precisely basic research that has driven every important applied advance. Take Scaling Laws as an example: precisely because this phenomenon was observed did OpenAI firmly choose to scale up the model and data, ultimately leading the development of large language models. The process

of discovering Scaling Laws embodies a strong scientific spirit: through observation of experiments, distilling an empirical formula, making predictions, then verifying through experiments, repeatedly, and finally arriving at a series of reliable conclusions that have a profound impact on applications.

Another example is AlphaGo. As the number of stones placed increases, the possibilities of the game grow exponentially; training a neural network to evaluate the board via exhaustive sampling is impossible. AlphaGo's success largely lies in solving the high-dimensional sampling problem. Its core idea is the observation that not all board positions are equally valuable; a deep neural network can quickly help judge which positions are worthless, so the search need not continue down those branches, while high-quality data can in turn improve the network's accuracy. This is the essence of combining Monte Carlo Tree Search with deep learning. Many similar examples can be found in this book.

The reason such misleading views about basic research in deep learning arise is mainly that people lack correct understanding and reasonable expectations of basic research or theoretical research. Basic research does not tell you how to tune every parameter; rather, it can tell you which direction is doomed and which is more promising; what the model is good at and what its limitations are. For example, the frequency principle introduced in this book tells us that models are good at solving low-frequency problems but struggle with high-frequency ones. Tuning hyperparameters on a general network is difficult, but if one stretches a high-frequency function into a low-frequency one, parameter tuning becomes easier. As for how to "stretch," different papers proposed different methods; though they differ, their core ideas and goals are consistent, namely making parameter tuning easier. Similar situations occur in traditional science. The first law of thermodynamics states the conservation of energy: a perpetual-motion machine of the first kind is impossible, thus avoiding wasted effort. The second law states that it is impossible to absorb energy from a single heat source and completely convert it to useful work with no other effect: a perpetual-motion machine of the second kind is impossible, also avoiding wasted effort. Chemical theory points out that alchemy turning copper into gold is impossible, avoiding wasted effort. Turning charcoal into diamond under high temperature and pressure is possible but extremely difficult, and after long-term development, synthetic diamonds are now quite common.

An excellent basic (theoretical) research can be judged by the following aspects:

Basic conditions:

1. Important problem: discovers or studies interesting phenomena or theoretical problems.
2. Tractable: simple enough to analyze, yet complex enough to provide insight into the phenomena or problems of interest.
3. Verifiable: can be verified by experiments.

Advanced conditions:

1. Can lead to more interesting phenomena or theoretical problems, helping us understand more deeply.
2. Helps guide the design of practical algorithms.
3. Helps cultivate theoretical or applied talent.

Phenomenon observation and theoretical research complement each other and jointly drive the development of deep learning and indeed the whole of science. In today's increasingly complex

deep learning landscape, starting from rich phenomena, systematically summarizing regularities, and building a theoretical framework will be an important path to exploring the mysteries of deep learning and guiding its future development. Let us advance hand in hand, illuminated by both phenomena and theory, and together create a bright future for deep learning.

Chapter 2

Curse of Dimensionality

The curse of dimensionality is a fundamental challenge in machine learning and data science. As the dimensionality of the data we deal with increases, the performance of many algorithms degrades significantly, and the computational complexity and sample size requirements grow exponentially. This chapter will give an in-depth introduction to the properties of high-dimensional data, and on this basis explore the essence, impact, and mitigation strategies of the curse of dimensionality.

Section 2.1 of this chapter will introduce several counter-intuitive geometric and statistical properties of high-dimensional spaces. For example, the volume of a high-dimensional unit ball decreases rapidly to nearly zero as the dimension increases; two points randomly chosen in a high-dimensional unit ball are almost surely separated by an almost constant distance and are very likely to be orthogonal; data points are almost entirely concentrated near the surface of the ball rather than in its interior, and so on. In Section 2.2, we will further explore the curse of dimensionality encountered in practical problems through several concrete examples.

In recent years, deep learning, with its remarkable representation and generalization ability and flexible structural design, has brought new hope for overcoming the curse of dimensionality. Experimental results show that deep neural networks far outperform traditional methods on high-dimensional data (such as natural images, natural language, the game of Go, etc.), and have successfully solved many extremely challenging high-dimensional problems. This undoubtedly raises high expectations for the potential of deep learning. But how exactly is deep learning able to accomplish these tasks? In what sense can it overcome the curse of dimensionality? In Section 2.3, we will focus on the theme of "strategies for overcoming the curse of dimensionality," and discuss the understanding of how the Monte Carlo method and neural network methods overcome the curse of dimensionality.

Popular Science

What is the curse of dimensionality?

The curse of dimensionality (CoD) refers to the phenomenon that, as the dimensionality of the data (e.g., the number of features) increases, the number of parameters or samples required by many machine learning and statistical methods grows exponentially with the dimension. This term was first proposed by Richard Bellman in the 1950s to describe the problem in dynamic programming

whereby an increase in dimensionality leads to an exponential rise in computational complexity. With the development of data science, the curse of dimensionality has been widely applied in machine learning, optimization, numerical analysis, and other fields.

Why should we care about the curse of dimensionality?

In practical applications, datasets often exhibit high-dimensional characteristics. Image and natural language data typically lie in very high-dimensional spaces. For example, a 256×256 grayscale image can be viewed as a 65536-dimensional vector, and a natural language sentence, if each word is represented by a 1000-dimensional word vector, can be viewed as a 100000-dimensional vector for a sentence of 100 words. As the dimension increases, data points become sparse, the computational complexity grows, and many traditional methods struggle to handle the situation effectively. The sparsity and complexity of such high-dimensional spaces pose enormous challenges to the training and prediction of machine learning models, so understanding and overcoming the curse of dimensionality is an important topic in data science.

Why does machine learning encounter the curse of dimensionality?

A core class of problems in machine learning is to learn an unknown function f^*. Take a classical regression problem as an example: given a training dataset $\{(x_i, y_i)\}_{i=1}^n$, where $y_i = f^*(x_i)$, our goal is to find, based on these data, a function $\hat{f}$ such that $\hat{f}$ approximates f^* as accurately as possible. However, when the data we deal with have high-dimensional characteristics, this seemingly simple task becomes extraordinarily difficult.

Intuitively, in a high-dimensional space, data points are usually very sparse. Simply put, if we partition an interval into multiple equal parts, then in a high-dimensional space the number of such small cells grows exponentially with the dimension. For instance, in a two-dimensional space, if we divide an interval into 10 equal parts, we have 100 small cells; in a three-dimensional space we have 1000 small cells. When the dimension is very high, even millions of data points appear extremely sparse in the high-dimensional space and cannot cover all small cells. Thus, the distribution of data points in a high-dimensional space is extremely sparse, making it extraordinarily difficult to approximate a high-dimensional function. Classical function approximation theory (DeVore, 1998) precisely characterizes this point: in a high-dimensional space, the sample size required to approximate a target function grows exponentially with the dimension, leading to the curse of dimensionality.

What methods are there to overcome the curse of dimensionality?

One of the classical methods to overcome the curse of dimensionality is the Monte Carlo method. Consider an example of high-dimensional numerical integration. In the high-dimensional case, if we use the traditional grid-based method, the number of grid points grows exponentially with the dimension, hence we encounter the curse of dimensionality. But by using random sampling via Monte Carlo, the integration accuracy is independent of the dimension d, thus overcoming the curse of dimensionality.

2.1 Properties of high-dimensional spaces

High-dimensional spaces have many counter-intuitive properties that differ greatly from the low-dimensional spaces we are familiar with. In this section, we will introduce several important properties of high-dimensional spaces to help the reader build an intuitive feel for them.

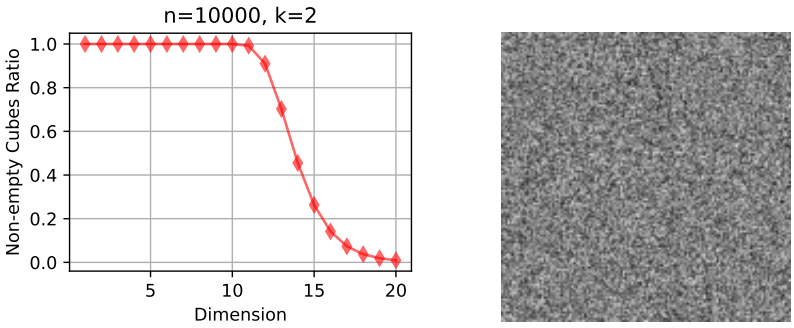
2.1.1 Sparsity of data in high-dimensional spaces

In a high-dimensional space, the distribution of data points is typically extremely sparse. Imagine scattering n points randomly inside a d-dimensional unit hypercube $[0,1]^d$, where each coordinate of each point is independently and uniformly distributed on the interval $[0,1]$. If we partition this interval into k equal-length subintervals, then the whole hypercube is divided into k^d small cubes. As the dimension d increases, even millions of data points are insufficient to cover every one of the k^d small cubes. In other words, most cubes will be empty, and this situation is ubiquitous in high-dimensional spaces.

To ensure that each small cube contains at least one data point, the required sample size n must satisfy:

$$n \geq k^d. \tag{2.1}$$

This means that the sample size n must grow exponentially with the dimension d. In practical applications, the data dimension d may be very high (for example, a 256×256 grayscale image has a dimension as high as 65536), while the available sample size n is usually much smaller than k^d, so the sparsity of data in high-dimensional spaces is a problem that cannot be ignored.



(a) Fraction of non-empty cubes

(b) Randomly sampled images

Figure 2.1: Experiments on data sparsity in high-dimensional spaces.

To present data sparsity intuitively, we consider the following experiment: inside the d-dimensional unit hypercube $[0,1]^d$, randomly sample $n = 10000$ points, where each coordinate of each point is independently and uniformly distributed on $[0,1]$. We divide this interval into $k = 2$ equal-length subintervals, so the whole hypercube is partitioned into 2^d small cubes, and we count the fraction of non-empty small cubes. From Fig. 2.1(a), we can see that as the dimension increases, the fraction of non-empty cubes drops rapidly, which shows that the data become sparser and sparser in high-dimensional spaces. Specifically, in a 10-dimensional space, 10000 data points still appear relatively dense, but in a 20-dimensional space, the fraction of non-empty hypercubes has already

been close to zero. This means that in a 20-dimensional space, the distribution of 10000 data points is extremely sparse. If we further consider the roughly 150,000-dimensional ImageNet image space, $2^{150000} \approx 10^{45154}$. This is an unimaginably huge number! To give the reader a reference, the number of atoms in the universe is on the order of 10^{80}. Even if we collected one image per second and kept doing so day and night, the time required to complete 2^{150000} collections would still vastly exceed the age of the universe (about 13.8 billion years, 4.4×10^{17} seconds). In other words, **high-dimensional spaces are almost "empty"**: once the dimension of the space becomes high enough, the space occupied by the neighborhoods of the data points actually collected in practice is almost negligible.

Image and natural language data typically lie in very high-dimensional spaces. For example, a 256×256 grayscale image can be viewed as a 65536-dimensional vector, and a natural language sentence, if each word is represented by a 1000-dimensional word vector, can be viewed as a 100000-dimensional vector for a sentence of 100 words. In such high-dimensional spaces, data points are naturally very sparse. Fig. 2.1(b) shows the result of random sampling in a high-dimensional image space; the generated images look like random noise, i.e., "snow patterns," and differ greatly from natural images. This shows that in a high-dimensional image space, the overwhelming majority of regions correspond to unnatural, random-noise-like images, while natural images occupy only a negligibly small subregion. Therefore, generating a natural image in a very high-dimensional image space is in theory extremely difficult, and requires very special algorithms and model designs, such as generative adversarial networks (GANs) and diffusion models.

Similarly, randomly sampling in a high-dimensional natural language space, the generated text usually lacks grammatical structure and semantic coherence, and differs greatly from natural language text. Although the famous "infinite monkey theorem" theoretically explores the possibility that random keystrokes over infinite time will eventually produce the works of Shakespeare, in practice meaningful natural language text is very sparse in a high-dimensional space, and random sampling within a finite time is almost never going to accidentally produce coherent, meaningful text. This reveals a basic property of high-dimensional spaces: natural, meaningful objects occupy only an extremely small subset of the whole space, and random sampling can hardly hit them by chance. This is also why we need specially designed generative models (such as the language model GPT) to generate high-quality natural language text.

The sparsity of data in high-dimensional spaces poses major challenges for machine learning. However, we should also note that even though data in a high-dimensional space are sparse, they usually carry some kind of structure. Taking image data and natural language processing as examples, the pixels in an image are not randomly distributed but have local correlations, and such structure allows us to use convolutional neural networks (CNNs) to capture translation invariance and local features in images. Likewise, natural language text has long-range dependencies, i.e., different words in a sentence have dependency relations among them. Therefore, the Transformer model uses the self-attention mechanism to model relations between any pair of words in a sentence. For example, in the sentence "the cat sits on the mat," there is a direct informational connection between "cat" and "mat," and this connection can be effectively captured and exploited by the self-attention mechanism to produce a more effective text representation.

2.1.2 Concentration of volume near the surface

In high-dimensional spaces, one striking phenomenon is that most of the volume is concentrated near the surface of an object. To understand this, let us consider a compact object Ω in the d-

dimensional Euclidean space $\mathbb{R}^d$, where "compact" means that it is closed and of finite size. If we shrink each dimension of the object Ω by a small factor ϵ to obtain a new inner object $(1-\epsilon)\Omega$, then the ratio of the volumes of these two objects satisfies:

$$\frac{V((1-\epsilon)\Omega)}{V(\Omega)} = (1-\epsilon)^d. \tag{2.2}$$

This relation can be understood by partitioning the object Ω into infinitely many infinitesimal cubes. If we shrink the side length of each cube in Ω to $1-\epsilon$ times its original value, then the volume of each cube shrinks to $(1-\epsilon)^d$ times its original value, and hence the volume of the whole object also shrinks by a factor of $(1-\epsilon)^d$.

Using the inequality $1 - x \le e^{-x}$ $(x > 0)$, we obtain:

$$\frac{V((1-\epsilon)\Omega)}{V(\Omega)} = (1-\epsilon)^d \le e^{-\epsilon d}. \tag{2.3}$$

If we fix ϵ and let the dimension d tend to infinity, the above volume ratio quickly approaches zero:

$$\lim_{d\to\infty} (1-\epsilon)^d \approx \lim_{d\to\infty} e^{-\epsilon d} = 0. \tag{2.4}$$

This means that when the dimension d is large enough, almost all the volume is concentrated in the thin layer $\Omega \setminus (1-\epsilon)\Omega$ near the surface of Ω, while the volume of the inner object $(1-\epsilon)\Omega$ is almost negligible compared with the volume of the whole object Ω.

To make this more concrete, let us consider a d-dimensional unit ball $\boldsymbol{B}^d$. A d-dimensional ball is the generalization of a ball in three-dimensional space. A d-dimensional ball of radius r can be viewed as the geometric object centered at the origin consisting of all points whose Euclidean distance to the origin does not exceed r. Consider the case of radius 1. The 1-dimensional ball is the segment $[-1, 1]$. The 2-dimensional ball is the disk bounded by the unit circle, given by $x^2 + y^2 \le 1$. The 3-dimensional ball (which we usually call a "ball") is given by $x^2 + y^2 + z^2 \le 1$. We can extend this definition to any dimension:

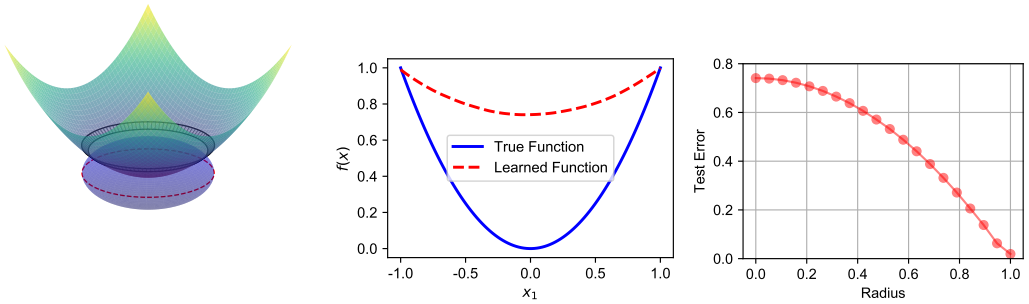
$$\boldsymbol{B}^d = \left\{ (x_1, \ldots, x_d) \in \mathbb{R}^d \middle| \sum_{i=1}^{d} x_i^2 \le 1 \right\}.$$

According to the analysis above, when d is large, most of the volume of the d-dimensional unit ball $\boldsymbol{B}^d$ is concentrated in the thin spherical shell outside the radius $1-\epsilon$. Table 2.1 lists, for $\epsilon = 0.01$ and various dimensions d, the ratio of the volume of the inner ball $(1-\epsilon)\boldsymbol{B}^d$ to the volume of the whole unit ball.

Dimension d	1	2	3	5	10	100	1000
Inner-ball volume ratio	99.00%	98.01%	97.03%	95.10%	90.44%	36.60%	$< 0.01\%$

Table 2.1: As the dimension d increases, the ratio of the volume of the inner ball $(1-\epsilon)\boldsymbol{B}^d$ to that of the unit ball drops rapidly.

As we can see from Table 2.1, as the dimension d increases, the proportion of the inner ball's volume drops rapidly. In a 1000-dimensional space, the inner ball of radius 0.99 accounts for less



(a) Schematic of a quadratic function in high-dimensional space

(b) Comparison of the network-learned function and the true function

(c) Test error at different radii

Figure 2.2: Experiment on fitting a quadratic function in a high-dimensional space.

than 0.01% of the unit ball's volume, which means that 99.99% of the volume is concentrated in the thin spherical shell of thickness 0.01.

To illustrate the impact of this volume-concentration phenomenon on practical machine learning, we design an experiment of fitting a high-dimensional quadratic function. In this experiment, we randomly sample 10000 data points $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{10000}$ inside a 100-dimensional unit ball, where $y_i = \|\boldsymbol{x}_i\|_2^2$, and use a two-layer ReLU neural network $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ to fit these data points.

Fig. 2.2(a) shows a 2-dimensional schematic of this experiment. The semi-transparent blue region indicates the region where data points are sampled, i.e., the unit ball. After training the neural network (with training loss less than 10^{-6}), we evaluate the model's performance from two perspectives:

1. As shown in Fig. 2.2(b), we fix the other 99 coordinates to 0 and sample 500 points along the first coordinate axis from -1 to 1, and compare the neural network's output $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ with the true quadratic function value $y = x_1^2$ at these points. We can see that near the boundary (-1 and 1), the neural network's predictions almost exactly coincide with the true values, while in the central region there is a noticeable deviation. This is because the overwhelming majority of points uniformly sampled inside a high-dimensional ball lie near the boundary, so the model learns well in these regions, while in the data-sparse interior region, the learned function deviates significantly from the true function.

2. As shown in Fig. 2.2(c), we randomly sample 1000 points $\boldsymbol{x}$ on spheres of various radii r, and compute the neural network's mean absolute error $\frac{1}{1000}\sum_{i=1}^{1000} |f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \|\boldsymbol{x}_i\|_2^2|$ at these points. We can see that as the radius r increases, the average error gradually decreases and reaches its lowest value at $r = 1$. The underlying mechanism is not hard to understand: in regions close to the center, data are scarce, and the generalization error of the neural network is large; whereas near the surface, data are more abundant, and the generalization error is small.

This experiment clearly demonstrates the impact of the volume-concentration phenomenon in high-dimensional spaces on machine learning. In practical applications, if we randomly sample data points from a high-dimensional space, then most data points will be distributed in the thin layer

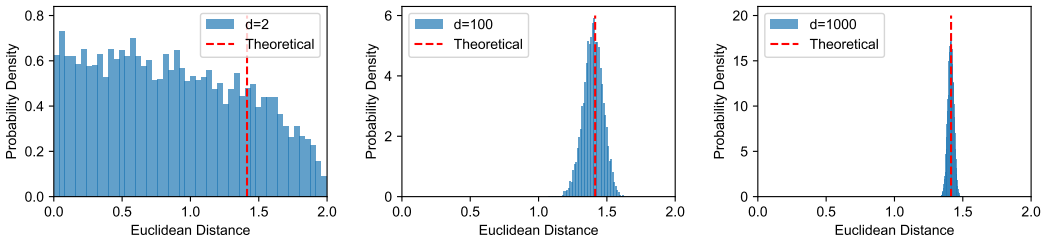
near the boundary, while data points in the interior region are very sparse. This characteristic of data distribution will affect the training and generalization of machine learning models. If we care not only about the model's generalization performance over most of the volume but also about its generalization performance in the small-volume interior, we can adopt the strategy of "importance sampling," i.e., deliberately sampling more data in the regions we care about, to alleviate this problem brought by high-dimensional spaces.

2.1.3 Distance concentration effect and orthogonality

In high-dimensional spaces, the distances between data points exhibit a "concentration effect," i.e., these distances tend to be highly concentrated around some particular value. At the same time, two randomly chosen vectors also tend to be orthogonal to each other. To understand these phenomena intuitively, we consider randomly sampling two points $\boldsymbol{x}$ and $\boldsymbol{y}$ inside a d-dimensional unit ball, and analyze the distributions of the Euclidean distance and the angle between them experimentally.

First, let us look at the distance concentration effect. For two points $\boldsymbol{x}$ and $\boldsymbol{y}$ randomly sampled inside the d-dimensional unit ball $\boldsymbol{B}^d$, the Euclidean distance between them can be expressed as:

$$\|\boldsymbol{x}-\boldsymbol{y}\|_2 = \left(\sum_{i=1}^{d}(x_i-y_i)^2\right)^{\frac{1}{2}}.$$



(a) d=2 (b) d=100 (c) d=1000

Figure 2.3: Distribution of the Euclidean distance between randomly sampled point pairs inside a high-dimensional unit ball. The red dashed line indicates the distance $\sqrt{2}$.

Fig. 2.3 shows the distribution of the Euclidean distance computed for 10000 random point pairs sampled inside unit balls of dimensions 2, 100, and 1000. From Fig. 2.3 we can see that as the dimension d increases, the distribution of the Euclidean distance between randomly sampled point pairs becomes more and more concentrated around $\sqrt{2} \approx 1.414$ (shown by the red dashed line in Fig. 2.3), and its variance becomes smaller and smaller. This is a manifestation of the distance concentration effect in high-dimensional spaces.

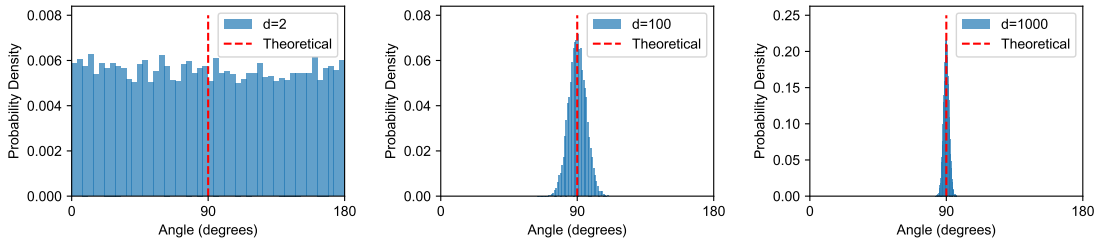
Consider traditional machine learning algorithms, for example the classical k-nearest neighbor (kNN) algorithm, which relies on distances for classification and regression. However, as Fig. 2.3 shows, when the dimension increases, the difference between the nearest and farthest neighbor distances becomes smaller and smaller. This means that in a high-dimensional space, all points are almost "equally far away," and the notion of distance becomes blurred and loses its discriminative power. In other words, in a high-dimensional space "**the concept of nearest neighbor**

disappears": in two- or three-dimensional spaces, the concept of a neighbor is clear and intuitive; whereas in a high-dimensional space, the distances from the nearest and the farthest neighbor are almost the same. This makes the kNN algorithm and other distance-based algorithms ineffective.

Next, we analyze the angles between randomly sampled vectors. For two points $\boldsymbol{x}$ and $\boldsymbol{y}$ randomly sampled inside a d-dimensional unit ball, the angle θ between them can be computed via the dot product:

$$\cos\theta = \frac{\boldsymbol{x}\cdot\boldsymbol{y}}{\|\boldsymbol{x}\|\|\boldsymbol{y}\|}$$

Fig. 2.4 shows the distribution of the angles between 10000 random point pairs sampled inside unit balls of dimensions 2, 100, and 1000. From Fig. 2.4 we can see that as the dimension d increases, the distribution of the angle between randomly sampled vectors becomes more and more concentrated around 90° (shown by the red dashed line in Fig. 2.4), and its variance becomes smaller and smaller. This phenomenon is called the angle concentration effect of vectors in high-dimensional spaces.



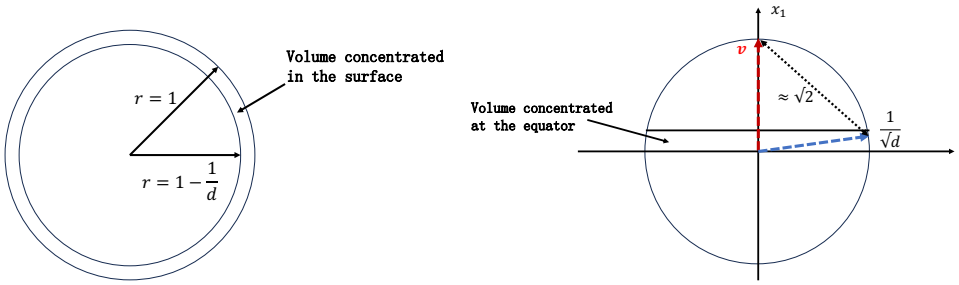
(a) d=2 (b) d=100 (c) d=1000

Figure 2.4: Distribution of angles between randomly sampled vectors inside a high-dimensional unit ball. The red dashed line indicates the angle of 90°.

Similar to the problems that the nearest-neighbor algorithm encounters in high-dimensional spaces, the angle concentration effect also leads to challenges in certain machine learning algorithms. For example, in the fields of text processing and information retrieval, a common method is to use cosine similarity to measure the similarity between two documents or vectors. Cosine similarity essentially evaluates similarity by computing the angle between vectors, with a smaller angle indicating greater similarity. However, as Fig. 2.4 shows, in a high-dimensional space, the angles between most vectors tend toward 90°, which means that their cosine similarities are close to zero, making all vectors look dissimilar. This poses a challenge to algorithms that rely on cosine similarity.

Below we analyze why distances and angles exhibit concentration effects. First, we know that the unit ball S in d-dimensional space is defined as the set of all points whose distance to the origin is at most 1. According to the volume-concentration-near-the-surface observation above, at least a fraction $1 - e^{-\epsilon d}$ of the volume is concentrated in a thin spherical shell of thickness ϵ near the boundary of the unit ball. In particular, as shown in Fig. 2.5(a), most of the volume of the high-dimensional unit ball is concentrated in a spherical shell of thickness $\frac{1}{d}$ near the boundary. If the ball has radius r, then the thickness of the shell is roughly $\frac{r}{d}$.

According to the analysis of the volume concentration near the surface, if we sample uniformly at random inside a high-dimensional unit ball, with very high probability we will land in a spherical



(a) Volume concentrates in the spherical shell (b) Volume concentrates near the equator

Figure 2.5: Schematic illustration of the concentration of the unit ball's volume in the spherical shell and near the equator.

shell of thickness about $\frac{1}{d}$. In addition, another interesting fact is that in high-dimensional spaces, the volume of the unit ball is concentrated not only near the surface but also near the "equator." Here the "equator" refers to the intersection of the hyperplane orthogonal to an arbitrarily chosen unit vector $\boldsymbol{v}$ with the spherical shell. Specifically, for any vector $\boldsymbol{v}$ on the unit sphere, the set of points in the unit ball whose dot product with this vector is small occupies most of the volume of the unit ball. To illustrate this, as shown in Fig. 2.5(b), suppose without loss of generality that $\boldsymbol{v} = [1, 0, \cdots, 0]^\top$; we can show that the points in the unit ball satisfying $|x_1| \leq 1/\sqrt{d}$ occupy most of the volume, i.e., most of the volume of the unit ball lies in the thin layer where the dot product with $\boldsymbol{v}$ is about $1/\sqrt{d}$ (the specific mathematical derivation is omitted here; interested readers can refer to Blum et al. (2020)).

One immediate consequence of this analysis is that if we randomly pick two points from the unit ball, for example the vectors represented by the red and blue dashed lines in Fig. 2.5(b), they are very likely to be almost orthogonal. More specifically, according to the previous analysis of volume concentration in the spherical shell, both points are very likely to be near the surface, with lengths around $1 - 1/d$. If we define the direction of the first point as "north," then the second point's projection onto this direction is very likely to be only $\pm 1/\sqrt{d}$, and so their dot product will be about $\pm 1/\sqrt{d}$, which tends to 0 as the dimension d grows. This means that in a high-dimensional space, the distance between two vectors is almost $\sqrt{2}$, and the angle between them is almost $90°$. Therefore, in a high-dimensional space, randomly chosen points are not only almost all located near the boundary of the unit ball, but are also almost orthogonal to each other. This also explains the concentration effects of distances and angles in high-dimensional spaces.

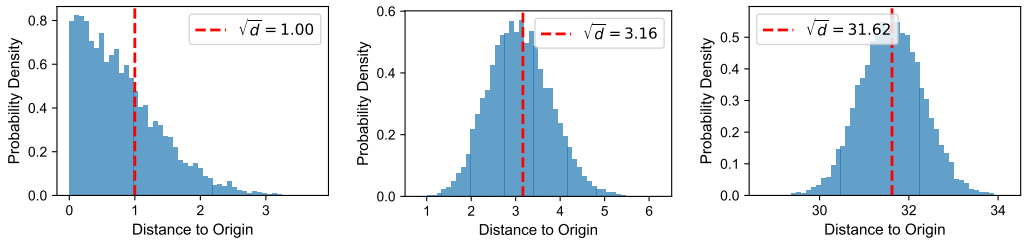
2.1.4 Gaussian annulus effect

Consider a d-dimensional Gaussian distribution $N(0, I_d)$ with unit covariance, whose probability density function is:

$$p(\boldsymbol{x}) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{\|\boldsymbol{x}\|_2^2}{2}\right).$$

To intuitively show the properties of Gaussian distributions in different dimensions, consider Fig. 2.6. This figure shows the distribution of the distance from the origin for 10000 data points sampled from 1-, 10-, and 1000-dimensional Gaussian distributions.

In the 1-dimensional case, as shown in Fig. 2.6(a), the sampled data points are mainly concentrated near the origin, which matches our intuitive understanding: most points are not far from the origin. In 10- and 100-dimensional spaces, however, we observe the phenomenon shown in Fig. 2.6(b, c): most data points are concentrated at distances roughly $\sqrt{d}$ from the origin, and this distribution forms an annular region whose width is almost a fixed constant (and remains unchanged as the dimension continues to increase).



(a) d=1 (b) d=10 (c) d=1000

Figure 2.6: Distribution of the distance from the origin for data points sampled from Gaussian distributions of different dimensions.

The experiment in Fig. 2.6 means that as the dimension increases, data points are no longer concentrated near the origin, but are "pushed" to a specific range of distances. This phenomenon is called the "Gaussian annulus effect": Gaussian random samples in a high-dimensional space "surround" the origin rather than "cluster" around it.

We now attempt to give an intuitive explanation of this phenomenon. First, let us consider the volume of a ball of radius r in a high-dimensional space. The volume $V(d)$ of a d-dimensional ball of radius r can be expressed by the following formula:

$$V(d) = \frac{2\pi^{\frac{d}{2}}}{d\Gamma\left(\frac{d}{2}\right)} r^d.$$

Here the gamma function $\Gamma(d)$ is the generalization of the factorial concept to the real and complex numbers. Intuitively, r^d and $\pi^{\frac{d}{2}}$ are rapidly growing exponential functions of d and $\frac{d}{2}$, while $\Gamma\left(\frac{d}{2}\right)$ grows like the factorial function as the dimension d becomes large. Hence, comparing the two, the volume $V(d)$ rapidly tends to 0 as the dimension d increases:

$$\lim_{d\to\infty} V(d) = 0. \tag{2.5}$$

This shows that as the dimension increases, the volume of the d-dimensional ball tends to 0, and this holds for all radii r.

Next, we explain why data points sampled from a high-dimensional Gaussian distribution concentrate in an annulus:

- In a high-dimensional space, the probability density function attains its maximum at the origin (i.e., $\boldsymbol{x} = \mathbf{0}$), but the neighborhood of the origin contains an extremely small volume. Although the probability density is highest at the origin, by Eq. (2.5) the volume of the unit

ball decreases rapidly in high-dimensional spaces, so even though the probability density is largest near the origin, the integrated probability is almost zero overall.

- To obtain a more significant probability, we need to enlarge the integration region and consider the neighborhood of the spherical shell of radius roughly $\sqrt{d}$. In this shell region, although the probability density decreases somewhat, the volume of the ball becomes relatively large, so the total integrated probability becomes significant.
- When $\|\boldsymbol{x}\|_2^2$ is significantly larger than d, due to the exponential decay, the probability density $p(\boldsymbol{x})$ quickly tends to zero. This means that although the volume of the ball increases with the radius, the increase in probability is very limited, because the probability density decreases faster than the volume increases.

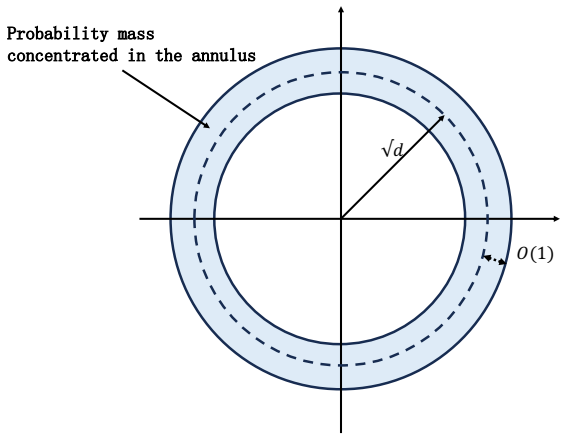


Figure 2.7: Schematic illustration of the Gaussian annulus theorem.

Mathematically, the Gaussian Annulus Theorem formally states this concentration trend (for details, see Blum et al. (2020)). As shown in Fig. 2.7, for a d-dimensional Gaussian distribution with variance 1 in each direction, this theorem asserts that almost all of the probability is concentrated in an annulus of radius $\sqrt{d}$ and width a fixed constant.

The Gaussian annulus effect has important implications for our understanding of high-dimensional noise. For example, in diffusion models, we need to add random noise to a 256×256 natural image. Each pixel of this noise image is independent and follows a $d = 65536$-dimensional Gaussian distribution. The Gaussian annulus theorem guarantees that, after adding high-dimensional randomly sampled noise, the distance between the noisy image and the original natural image is almost always a fixed constant (namely $\sqrt{d}$). Moreover, in diffusion models, noise is added in multiple steps. Since the multi-step superposition of Gaussian distributions is still a Gaussian distribution, the distance from the noisy image to the original image after multiple steps is still a fixed positive value. Therefore, no matter how we add noise, we almost never need to worry that the noise will cancel out and return us to the original natural image. However, for a one-dimensional problem, in the process of multi-step superposition of Gaussian noise, there is a significant probability of returning near the origin.

In addition, in large-scale text classification tasks, documents are usually represented as high-dimensional vectors, where each dimension corresponds to a vocabulary item. Due to the huge vocabulary size, these vectors lie in a high-dimensional space. The Gaussian naive Bayes algorithm assumes that the document features of each class are modeled by a high-dimensional Gaussian distribution. The Gaussian annulus effect means that, although text vectors may lie in a high-dimensional space, the distances within each class will concentrate in a specific range. Therefore, using Gaussian distributions to model the differences between document classes can effectively distinguish different classes. For example, suppose there are documents from two classes, forming two different Gaussian distributions in a high-dimensional space. The Gaussian annulus effect ensures that the document vectors within each class lie in a fixed range of distances, which makes the classifier still effective in this setting.

2.1.5 Dimensionality reduction via random projection

In modern machine learning, especially in deep learning, we often need to handle high-dimensional data, such as image data and natural language data. Suppose we have a dataset containing n data points, each lying in the d-dimensional space $\mathbb{R}^d$. As the data dimension d increases, directly analyzing and processing data in the original high-dimensional space brings considerable storage and computational burden. This calls for dimensionality reduction techniques.

An important property of high-dimensional spaces is that we can use dimensionality reduction via random projection to preserve the distance relations between points. A classical theorem—the Johnson-Lindenstrauss lemma—uses the Gaussian annulus effect to show that one can embed the original data into a k-dimensional space (with k depending on $\log n$), which is usually far lower than the original dimension d, via a random linear map, while the distances between points are approximately preserved (for details, see Blum et al. (2020)).

The random map f in this lemma is very simple and can be realized by a $k \times d$ random matrix $\boldsymbol{R}$:

$$\boldsymbol{R} = \frac{1}{\sqrt{k}} \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1d} \\ r_{21} & r_{22} & \cdots & r_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ r_{k1} & r_{k2} & \cdots & r_{kd} \end{pmatrix} \in \mathbb{R}^{k \times d}.$$

where r_{ij} are i.i.d. Gaussian random variables with mean 0 and variance 1. Then, for any d-dimensional vector $\boldsymbol{x}$, its random projection $\boldsymbol{f}(\boldsymbol{x})$ can be written as $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{R}\boldsymbol{x} \in \mathbb{R}^k$.

The Johnson-Lindenstrauss lemma states that n points in a high-dimensional space can be embedded into a low-dimensional space of dimension about $O(\log n)$, with the pairwise distances approximately preserved, and that this embedding can be realized by a simple random projection map. In practice, taking a dimension on the order of $\log n$ usually yields a good embedding effect.

In machine learning, the random projection lemma suggests that we can process high-dimensional data by embedding or projecting them into a lower-dimensional space, thereby reducing the consumption of storage and computational resources. For example, in natural language processing tasks, we usually represent each word by a high-dimensional one-hot word vector, whose dimension depends on the size of the vocabulary n and can reach the hundreds of thousands. The random projection lemma tells us that we can use random projection to embed these word vectors into an $O(\log n)$-dimensional lower-dimensional space, while the semantic similarity between words (typically measured by the distance between vectors) is still well preserved. In the image domain, consider that we have a set of high-resolution images, each of which is a high-dimensional digital

representation of a sequence of pixel values. Traditional processing of high-dimensional image data is both time-consuming and laborious. Through dimensionality reduction via random projection, we can project each image into a low-dimensional space, thereby reducing computational complexity. More importantly, even after dimensionality reduction, the similarity between these images (typically measured by the distance between vectors) is still well preserved, enabling us to use the dimension-reduced data for efficient image classification and search.

2.1.6 Linear separability of data

Although handling high-dimensional data in machine learning often comes with many challenges, high-dimensional spaces also bring some unique advantages. One notable advantage is the enhanced linear separability of data, which makes techniques such as kernel methods in traditional machine learning more effective.

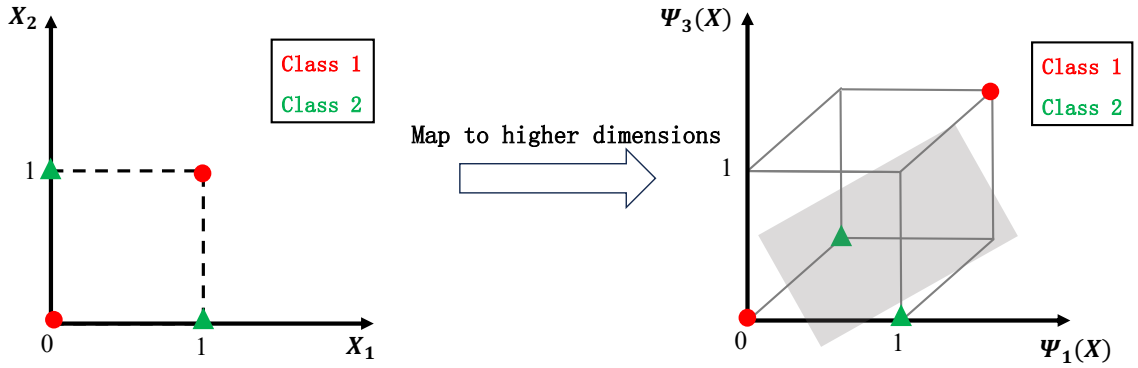


Figure 2.8: Example of the data distribution of the XOR problem in two and three dimensions.

Linear separability means that there exists a hyperplane in the space (a straight line in 2D, a plane in 3D) that can completely separate data points of different classes. We take the classical XOR problem as an example to explore this phenomenon. As shown in Fig. 2.8, in a 2-dimensional space, we have two classes of points, red and green. These points are interleaved and cannot be separated by a single straight line. Yet something magical happens: when we map the data to the higher-dimensional 3D space, these points become linearly separable. This is because high-dimensional spaces have more "degrees of freedom" or "dimensions" to find a hyperplane that can separate data points of different classes.

Kernel methods are a classical technique that fully exploits the linear separability of high-dimensional spaces. Kernel methods nonlinearly map the original low-dimensional data and embed them into a high-dimensional space. In this high-dimensional space, the data are usually linearly separable, and we can then apply a linear classifier such as a support vector machine (SVM) in the high-dimensional space. The principle behind this method is that by adding new dimensions, we provide the data points with extra "room to move," making it feasible to find a hyperplane that separates these points in the new space. This technique is not limited to the XOR problem; in many practical applications, we often use similar methods to solve linearly inseparable problems.

Kernel methods realize such nonlinear mapping by introducing a kernel function, so that data that are hard to separate in a low-dimensional space become separable in a high-dimensional space.

Neural network models can also be understood as a technique that uses nonlinear transformations to enhance the linear separability of data. In fact, a neural network can be viewed as a stack of complex nonlinear transformations, whose goal is to make the data linearly separable at the penultimate layer of the network, after which classification is carried out by the linear classifier in the last layer.

2.2 Examples of the curse of dimensionality

The curse of dimensionality refers to the phenomenon that, as the dimension of the data grows, the number of parameters or amount of data required by an algorithm to maintain its accuracy grows exponentially with the dimension. The curse of dimensionality is a key challenge that must be faced when solving many high-dimensional problems. In this section, we will showcase the manifestations and consequences of the curse of dimensionality through some concrete examples, giving the reader a more intuitive and deeper understanding of this concept.

2.2.1 High-dimensional numerical integration

Numerical integration is an important tool in applied mathematics; it allows us to approximate the integral of a function defined on some region. In one-dimensional space, there are many efficient numerical integration methods, such as the trapezoidal rule, Simpson's rule, and Gaussian quadrature. These methods divide the integration interval into a number of small subintervals and use simple functions (such as polynomials) on each subinterval to approximate the original function, thereby converting the continuous integral into a discrete sum.

However, when we try to generalize these methods to high-dimensional spaces, we encounter severe problems. Consider a simple example in which we want to compute the integral of a function $f(\boldsymbol{x})$ over the d-dimensional unit hypercube $[0,1]^d$:

$$I = \int_{[0,1]^d} f(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x}.$$

In the one-dimensional case, we can approximate this integral using a grid of n equally spaced nodes with spacing $1/n$ between adjacent nodes. This method is called the composite trapezoidal rule. By dividing the interval $[0,1]$ into n subintervals, the composite trapezoidal rule approximates the integral as:

$$I \approx \frac{1}{n}\left(\frac{f(x_0)+f(x_n)}{2} + \sum_{i=1}^{n-1} f(x_i)\right),$$

where $x_i = i/n$ is the coordinate of the i-th node. For a smooth integrand $f(x)$, the error of the composite trapezoidal rule is usually proportional to n^{-2}. This means that if we increase the number of nodes n by a factor of 10, the approximation error of the integral decreases by roughly a factor of 100.

Now let us consider generalizing the composite trapezoidal rule to a d-dimensional space. A natural generalization is to place n nodes uniformly along each coordinate axis, yielding a d-dimensional grid. This grid divides the entire integration region $[0,1]^d$ into n^d small hypercubes, each of side length $1/n$. If we apply the one-dimensional composite trapezoidal rule on each hypercube, we

obtain an approximation to the full integral:

$$I \approx \left(\frac{1}{n}\right)^d \sum_{i_1=0}^{n} \cdots \sum_{i_d=0}^{n} w(i_1, \ldots, i_d) f(x_{i_1}, \ldots, x_{i_d}),$$

where $w(i_1, \ldots, i_d)$ is a weighting function, and $x_{i_j} = i_j/n$ is the coordinate of the i_j-th node along the j-th dimension.

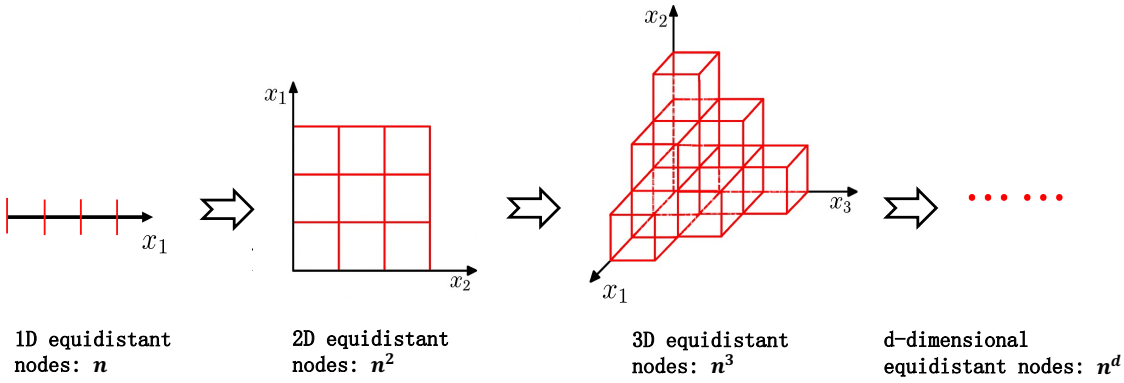


Figure 2.9: Schematic illustration of the curse of dimensionality encountered by discrete grid-based numerical methods.

At first sight, this looks like a reasonable generalization. But let us look more carefully at how many nodes this method requires. As shown in Fig. 2.9, in the one-dimensional case, we need n nodes. In the two-dimensional case, we need to sample on an $n \times n$ grid, with n^2 nodes in total. In the d-dimensional case, we need to sample on an $n \times n \times \cdots \times n$ grid, with n^d nodes in total. This is the intuitive manifestation of the curse of dimensionality. The number of nodes, i.e., the number of function evaluations required to compute the integral, grows exponentially with the dimension d. Even for relatively small d and n, the computational cost quickly becomes huge, and computation becomes infeasible. As a concrete example, suppose we want to compute an integral in a 10-dimensional space, and we use 10 nodes per dimension; then we need a total of $10^{10} = 10,000,000,000$ nodes. This means we have to evaluate the function $f(\boldsymbol{x})$ 10 billion times! Even if each evaluation takes only 1 nanosecond, completing all the evaluations still requires 10 seconds. And if the dimension is increased to 20, still with 10 nodes per dimension, we need 10^{20} function evaluations, and the time grows to about 3000 years, which is completely infeasible in practice.

This simple example reveals a fundamental difficulty of high-dimensional problems: the computational complexity often grows exponentially with the dimension, the so-called "curse of dimensionality." Many algorithms that work well in low-dimensional spaces face this obstacle in high-dimensional spaces and are therefore unable to handle high-dimensional data effectively.

2.2.2 High-dimensional partial differential equations

High-dimensional partial differential equations (PDEs) have broad and important applications in modern science and engineering, but they also face enormous challenges in terms of computational

complexity. Let us examine these applications and challenges through some concrete examples:

The Schrödinger equation in quantum physics In quantum physics, the wave function describing a quantum system composed of d particles satisfies the Schrödinger equation, and this equation is precisely a partial differential equation defined on the high-dimensional space $\mathbb{R}^{3d}$. In other words, even for a quantum system containing only $d = 10$ particles, the dimension of the space on which the Schrödinger equation is defined is already as high as 30. As the number of particles d further increases, the dimension of the Schrödinger equation becomes even larger. This brings tremendous difficulties to solving the Schrödinger equation using classical numerical methods.

The Black-Scholes equation in financial engineering In financial engineering, option pricing is a core problem, and the Black-Scholes partial differential equation is an important tool for solving option pricing problems. However, the spatial dimension of the Black-Scholes equation corresponds exactly to the number of financial assets considered. For example, for an investment portfolio composed of 100 assets, the corresponding Black-Scholes equation is a 100-dimensional partial differential equation. As one can imagine, for more complex financial derivative products, the number of assets involved may far exceed 100, and the resulting dimension of the Black-Scholes equation may become extremely high. This poses a great challenge for financial engineers to perform option pricing using numerical methods.

To intuitively understand the curse of dimensionality in high-dimensional PDEs, let us consider solving a simple Poisson equation on the d-dimensional unit cube $[0, 1]^d$:

$$\begin{cases} -\Delta u = f, & x \in \Omega, \\ u = 0, & x \in \partial\Omega. \end{cases}$$

Here Ω denotes the interior of the unit cube, and $\partial\Omega$ denotes the boundary of the unit cube.

If we discretize this equation using the finite difference method (i.e., approximating derivatives by differences), with n grid points uniformly placed along each coordinate direction, similar to what is shown in Fig. 2.9, then the discretized equation has $N = n^d$ degrees of freedom in the unknown u. We can see that the size N of the discrete equation grows exponentially with the spatial dimension d. This means that when d is large, even if n is small, N will be an astronomical number. Consequently, even for moderate-sized high-dimensional problems, classical numerical methods may become unaffordable in terms of computational resources.

High-dimensional partial differential equations have important applications in many fields, but solving them must overcome the challenge of the "curse of dimensionality." Classical numerical methods may completely fail in high-dimensional problems, so it is urgent to develop new theories and methods. We will later discuss methods that use neural networks to solve high-dimensional problems, in the hope of providing new ideas for this difficult problem.

2.2.3 High-dimensional function approximation

In many machine learning tasks, such as regression and classification, our goal is to learn from sample data a mapping function from a high-dimensional input space to an output space. The output space can be the real numbers (for regression) or a finite set of categories (for classification). Function approximation theory studies how to find a function that is as close as possible to the target function on the given data.

Classical function approximation theory tells us that in high-dimensional spaces, the difficulty of approximating a target function rises exponentially with the dimension. In other words, as the dimension d increases, the number of samples n must grow exponentially in order to maintain the same approximation accuracy. Specifically, if we have a target function f^* and we approximate it by sampling n points, then in a high-dimensional space, our approximation error ε (i.e., the gap between the approximating function we find and the true target function) roughly obeys the following law (DeVore, 1998):

$$\varepsilon \approx n^{-s/d} \tag{2.6}$$

Here, s is the smoothness of the target function, d is the dimension of the data, and n is the number of samples. As the dimension d increases, the sample size n must grow exponentially in order to maintain the same approximation error. One major problem faced by high-dimensional function approximation is the so-called "curse of dimensionality." Even if the target function is very smooth (i.e., s is large), approximating it in a high-dimensional space still requires a large number of samples. This makes it very difficult to efficiently learn and approximate high-dimensional functions in practical applications.

To gain a more intuitive understanding of the high-dimensional function approximation problem, we use a concrete example to illustrate the impact of smoothness and dimension on function approximation. First, let us look at how the smoothness of the target function affects our approximation. If the target function f^* is a linear function, since a linear function varies smoothly, the number of sample points needed is relatively small. With just a few points, we can approximate this function well. However, if the smoothness of the target function is poor, the function values vary drastically in different regions. To accurately approximate such a function, we need to sample more points, especially concentrated in regions of drastic variation, to ensure that the approximation error is small.

Next, we consider the impact of increasing the dimension. Suppose our input space is a d-dimensional unit cube, i.e., each dimension is in the range $[0, 1]$. We want to approximate a target function in this space using n sample points. When $d = 1$, we can uniformly distribute n sample points in the interval $[0, 1]$. The distance between two adjacent sample points is then about $1/n$. Since the target function is smooth, we can approximate it well, and as long as the gap between sample points is small, the approximation error is small, roughly n^{-s}. As shown in Fig. 2.10, when the dimension d increases, the spacing between sample points rapidly grows. In d-dimensional space, there are roughly $n^{1/d}$ sample points per dimension, so the distance between adjacent sample points is about $n^{-1/d}$. As the dimension d grows, the number of sample points per dimension shrinks drastically, the distance between sample points becomes large, and we have difficulty capturing the variations of the target function. Consequently, the approximation error becomes large, roughly $n^{-s/d}$; the higher the dimension, the larger the approximation error.

The curse of dimensionality in the high-dimensional function approximation problem tells us that, in the high-dimensional case, even if the target function is very smooth, we still need a number of samples growing exponentially with the dimension in order to guarantee a certain approximation accuracy. This poses a great challenge to machine learning algorithms, because the number of training samples obtained in practice is usually far smaller than what theory requires. However, even though the sample complexity of high-dimensional function approximation is generally very high, for certain specific function spaces, neural networks can achieve good approximation with relatively few samples. We will discuss the understanding of how neural networks overcome the curse of dimensionality in later sections.

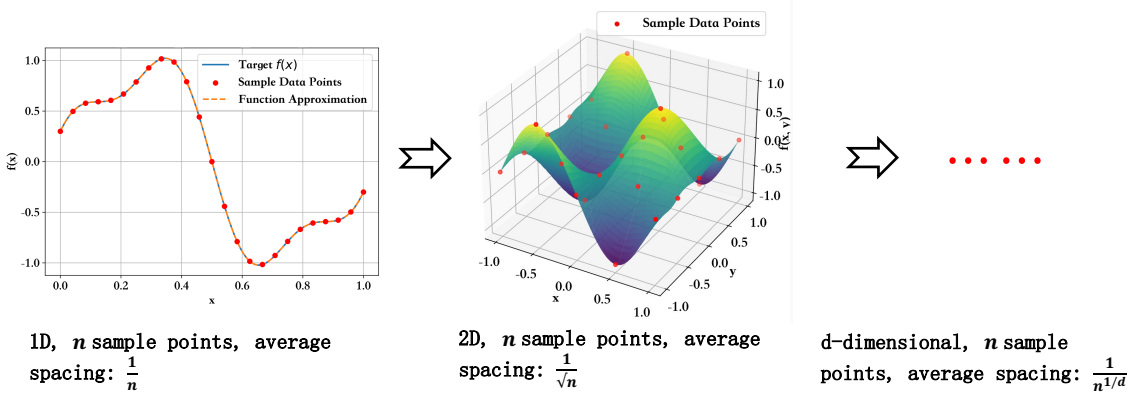


Figure 2.10: Schematic illustration of the curse of dimensionality encountered by high-dimensional function approximation.

2.3 Methods for overcoming the curse of dimensionality

2.3.1 Monte Carlo method

The Monte Carlo method is a class of numerical methods based on random sampling. It uses ideas from probability and statistics to solve high-dimensional problems and thereby overcomes the curse of dimensionality. Unlike traditional deterministic algorithms, the Monte Carlo method introduces randomness, randomly sampling in the solution space of a problem and taking statistical averages over these random samples to estimate the quantity of interest.

The Monte Carlo method was originally proposed by Stanislaw Ulam, John von Neumann, and others during their work on atomic bomb design. Its name comes from the famous casino city of Monte Carlo, symbolizing the randomness and game-like nature involved. Since its birth in the 1940s, the Monte Carlo method has flourished and has been widely applied in physics, chemistry, biology, engineering, finance, and many other fields.

Why can the Monte Carlo method overcome the curse of dimensionality? We illustrate this through a simple example. Consider computing the integral over the d-dimensional unit cube $[0,1]^d$

$$I = \int_{[0,1]^d} f(\boldsymbol{x})\, dx. \tag{2.7}$$

If we estimate this integral using the Monte Carlo method with uniform sampling, i.e., we independently draw n random sample points $\{x_i\}_{i=1}^n$ from $[0,1]^d$, and approximate the integral by the sample mean:

$$I_n = \frac{1}{n} \sum_{i=1}^{n} f(\boldsymbol{x}_i). \tag{2.8}$$

It is easy to verify that I_n is an unbiased estimator of I, i.e., $\mathbb{E}[I_n] = I$. The variance of I_n is

$$\mathrm{Var}(I_n) = \frac{\mathrm{Var}(f)}{n}, \tag{2.9}$$

where $\mathrm{Var}(f)$ is the variance of the integrand f on $[0,1]^d$, defined as:

$$\mathrm{Var}(f) = \int_{[0,1]^d} \left(f(\boldsymbol{x}) - \mathbb{E}[f(\boldsymbol{x})]\right)^2 d\boldsymbol{x}. \tag{2.10}$$

Therefore, the mean squared error of the Monte Carlo estimator is

$$\mathbb{E}\left[(I - I_n)^2\right] = \frac{\mathrm{Var}(f)}{n}. \tag{2.11}$$

We can see that this error depends only on the number of samples n and the variance of the integrand, and is independent of the dimension d. In other words, no matter how high the dimension of the problem, the Monte Carlo method can approximate the integral with a convergence rate of $O(n^{-1/2})$. This dimension-independence is precisely the fundamental reason why the Monte Carlo method overcomes the curse of dimensionality.

Intuitively, the reason the Monte Carlo method can avoid the curse of dimensionality is that it does not attempt to fill the entire high-dimensional space with a regular grid; instead, it randomly scatters some sample points in this space. In a high-dimensional space, the vast majority of the volume is concentrated in a narrow region near the boundary. The Monte Carlo method can adaptively place more sample points in these regions without wasting computational resources on the interior regions that contribute little to the integral. This adaptivity allows the Monte Carlo method to perform numerical integration efficiently and avoids the curse of dimensionality.

However, we must also point out that, although the Monte Carlo method overcomes the curse of dimensionality, its sampling efficiency may still be insufficient in some specific problems. For example, in the earlier experiment of fitting a high-dimensional quadratic function with a neural network (Fig. 2.2), the Monte Carlo method adaptively sampled a large number of points near the spherical shell of the unit ball, so that the model's generalization error in the integral sense was indeed very small; however, when we focused on those test points in the interior of the unit ball close to the center, the generalization error was particularly large. Therefore, judging the sampling efficiency of the Monte Carlo method needs to be done on a case-by-case basis. In practical applications, many methods have been developed to improve Monte Carlo sampling efficiency, such as importance sampling, stratified sampling, Markov chain Monte Carlo, etc., to further enhance its sampling efficiency. In addition, although the convergence rate $O(n^{-1/2})$ of Monte Carlo is independent of the dimension, reducing the error by one order of magnitude still requires a 100-fold increase in the sample size. Such a convergence rate is usually not competitive for low-dimensional problems.

In a specific high-dimensional problem like the game of Go, as the number of moves increases, the number of possible board positions grows exponentially, and sampling faces the curse of dimensionality. Therefore, training an algorithm to evaluate board positions by exhaustive sampling is unrealistic. The success of AlphaGo largely lies in solving the difficulty of high-dimensional sampling. Its core idea is, first, to use Monte Carlo Tree Search to avoid the curse of dimensionality. Second, it is to observe that at each step, not all possible move positions have the same probability of occurring. One can use a deep neural network to learn a sampling probability distribution over moves given a board position, quickly eliminating positions that are essentially impossible or of no value, and focusing the random sampling and search on positions of significant value. This is precisely the idea of importance sampling. In this way, AlphaGo achieved efficient sampling and an approximation to the optimal policy in the enormous state space.

In summary, the Monte Carlo method is a powerful and general class of numerical methods, particularly suited for handling high-dimensional problems. Through the ideas of random sampling and statistical averaging, it elegantly circumvents the curse of dimensionality. In the current era of big data, as the scale and complexity of problems continue to increase, the Monte Carlo method and its ideas are playing an increasingly important role.

2.3.2 Neural network method

Artificial neural networks, especially deep learning models, have shown excellent performance beyond classical machine learning methods on high-dimensional complex tasks (such as image and natural language tasks). So, can it be rigorously proved that neural networks can overcome the curse of dimensionality in some cases?

To theoretically understand this "magic" of neural networks, mathematicians have introduced a key concept—the Barron space. This is a class of function spaces designed specifically for analyzing the performance of neural networks. It was first introduced by Barron in his pioneering 1993 work (Barron, 1993), and was further developed and refined in the works of Weinan E, Lei Wu, Chao Ma, and others (E et al., 2019). Functions in the Barron space have a special property: they can all be represented in the following integral form:

$$f(\boldsymbol{x}) = \int_{\mathbb{S}^d} a(\boldsymbol{w})\sigma(\boldsymbol{w}^T\boldsymbol{x})d\pi(\boldsymbol{w}). \tag{2.12}$$

Here, $\mathbb{S}^d = \{\boldsymbol{w} \mid \|\boldsymbol{w}\|_1 = 1\}$ denotes the unit sphere under the 1-norm, σ is an activation function satisfying homogeneity (such as ReLU), $\boldsymbol{w}$ represents a direction on the sphere, $\pi(\boldsymbol{w})$ is a probability distribution, and $a(\boldsymbol{w})$ describes the "weight" or "contribution" in each direction.

We can understand this integral representation as an infinitely wide single-hidden-layer neural network: each hidden neuron corresponds to a direction $\boldsymbol{w}$ on the sphere, with a corresponding weight $a(\boldsymbol{w})$. In fact, finite-width two-layer neural networks also belong to the Barron space. A two-layer network of width m given by $f(\boldsymbol{x}) = \sum_{k=1}^m a_k\sigma(\boldsymbol{w}_k^T\boldsymbol{x})$ can be viewed as a discrete version of the above integral representation, where the probability distribution becomes:

$$\pi(\boldsymbol{w}) = \frac{1}{m}\sum_{k=1}^m \delta(\boldsymbol{w} - \boldsymbol{w}_k).$$

Here $\delta(\cdot)$ is the Dirac δ function. By the universal approximation theorem, continuous functions can all be approximated to arbitrary accuracy by finite-width two-layer neural networks. This means that the Barron space is dense in the space of continuous functions—it can approximate almost all continuous functions we are interested in. Therefore, the Barron space is a very "large" function space.

Now let us look at an important result on the approximation of Barron-space functions by neural networks (E et al., 2019): for any function f in the Barron space, there exists a two-layer neural network $f(\cdot;\tilde{\boldsymbol{\theta}})$ of width m such that:

$$\mathbb{E}_{\boldsymbol{x}}[(f(\boldsymbol{x}) - f(\boldsymbol{x};\tilde{\boldsymbol{\theta}}))^2] \leq \frac{3\gamma_2^2(f)}{m}.$$

Here $\gamma_2(f) := \inf_{(a,\pi)\in\Theta_f}\left(\int_{\mathbb{S}^d}|a(\boldsymbol{w})|^2\,\mathrm{d}\pi(\boldsymbol{w})\right)^{1/2}$ is a certain Barron norm of the function f, independent of the data dimension d, and Θ_f denotes the set of all (a,π) pairs that represent the function f.

The proof of this result cleverly uses the idea of **Monte Carlo** random sampling. Specifically, for a function f in the Barron space, we know that it has the integral representation in Eq. (2.12). Now, if we independently draw m random samples $\{\boldsymbol{w}_k\}_{k=1}^m$ from the distribution π, we can construct the following two-layer neural network to approximate f:

$$f(\boldsymbol{x};\tilde{\boldsymbol{\theta}}) = \frac{1}{m}\sum_{k=1}^{m} a(\boldsymbol{w}_k)\sigma(\boldsymbol{w}_k^T\boldsymbol{x}).$$

This is precisely the essence of the Monte Carlo method: approximating an integral by an average over finitely many random samples! We can verify that $\mathbb{E}_{\boldsymbol{w}}[f(\boldsymbol{x};\tilde{\boldsymbol{\theta}})] = f(\boldsymbol{x})$, i.e., the expected value of the neural network equals the target function. Analogously to Eq. (2.11), we can also bound the variance of the approximation error as approximately $\frac{\gamma_2(f)}{m}$. The revolutionary significance of this result is: **the approximation accuracy $O(m^{-1/2})$ is completely independent of the data dimension d, and hence the curse of dimensionality is overcome!**

Chapter 3

Data and Neural Network Architecture

One definition of data is raw, unprocessed informational material—the primitive form of information. From the very first day life appeared on Earth, organisms have faced the daunting challenge of surviving and reproducing in a harsh and uncertain environment. Collecting various kinds of information from the surroundings, so as to better exploit advantages and avoid harm, is therefore crucial for survival. The appearance of sensory organs such as the eye, for instance, allowed organisms to efficiently gather light information from their surroundings. The eye acts like a camera, continuously collecting visual data. Yet data alone is not enough: an organism must be able to extract from this data the key information relevant to its survival—what the object in front of it is, whether it can be eaten, whether it poses a threat, and so on. The animal nervous system is precisely the system that processes data and extracts such key information. Only when the value of the information extracted by the organism exceeds the energy consumed in perceiving and processing the data can the nervous system continually "upgrade" through evolution, eventually giving rise to powerful biological intelligence. One may even say that data, rich in survival-relevant information, is the fundamental driver behind the growth of biological intelligence.

It is by no means obvious that the various kinds of data captured by our senses contain so much survival-relevant information. Deep in the ocean, for example, where the environment is extremely dark, even when light has not entirely vanished the effective information it carries is exceedingly scarce—just like taking a photograph in extremely low light, where it is hard to make out the shapes of objects amid the noise. In such environments, the eyes of many organisms have degenerated. Without the drive of visual data, the intelligence level of their nervous systems is also harder to evolve. Fortunately, in the environments humans normally inhabit, the visual, auditory, olfactory, gustatory and tactile data captured by our senses are rich in information. The need to effectively extract survival-related information from such data continually drives the evolution of the human brain and the enhancement of intelligence. One may say that biological intelligence algorithms, carried by the nervous system, evolved precisely to perform such large classes of information-extraction tasks.

How, then, should we design machine learning algorithms to perform such complex information-extraction tasks from data? In today's era, where the pace of brain evolution lags far behind the explosion (exponential growth) of data, this question becomes particularly important. Relying

solely on the human brain to extract information from such data is no different from mining ore with bare hands—it is extremely inefficient. According to the No Free Lunch theorem, the algorithms we design can achieve good performance only if they match the characteristics of the tasks behind the data. In this respect, biological neural networks provide an important reference. There is no doubt that the structures of biological neural networks and their learning algorithms are highly compatible with the various image and language tasks we care about. It is therefore natural to design an artificial neural network algorithm by imitating biological neural networks. At present, we clearly see that, after breaking through the two bottlenecks of data and compute, deep learning methods based on artificial neural networks have achieved unprecedented success. For instance, a large language model based on the Transformer neural network can extract the information we care about from virtually all of humanity's textual data. Such information-extraction capability was completely unimaginable in earlier years.

Similar to the evolution of biological nervous systems, the evolution of deep learning algorithms comes from the survival of the fittest with respect to the ability to extract information from datasets (measured, for instance, by test accuracy). The main progress of deep learning algorithms in recent years has come from the evolution of neural network architectures, for example from fully connected networks, convolutional networks, long short-term memory networks, residual networks to Transformers. Empirically, we observe that the better an architecture matches the intrinsic structure of the data it processes, the better the performance it can achieve.

This chapter explores the close connection between data characteristics and neural network architectures. We will see how different data characteristics inspire different network designs, and how different network architectures embody different prior assumptions about the data. Through this discussion, we hope to build a bridge between data, tasks and models, and to provide readers with a comprehensive perspective for understanding and designing neural network architectures.

Popular-science part

What is the connection between data and neural network architectures?

The architectural design of a neural network is closely tied to the type of data it processes, a connection that can be explained by the No Free Lunch theorem. The No Free Lunch theorem states that, for any optimization algorithm, its average performance over all possible problem spaces is the same. This means that no single universal algorithm can outperform all others in every situation. Therefore, specific data types require specific network architectures in order to optimize performance. For example,

- **Image data**: Image data exhibits local correlation and spatial structural characteristics. Convolutional neural networks (CNNs), through the use of convolutional layers, can capture local features in images such as edges and textures, and through pooling layers reduce the amount of computation while retaining important spatial information. The weight-sharing and local-connectivity properties of CNNs make them especially suitable for processing image data.

- **Language data**: Language data is temporal data with context dependence. The Transformer architecture, through the self-attention mechanism, can attend to all words in the entire sequence simultaneously during processing, capturing long-range dependencies. This makes the Transformer perform excellently on language tasks (such as machine translation,

text generation and semantic analysis). Compared with recurrent neural networks (RNNs), the Transformer can better handle long-range dependencies, and parallel computation also improves training efficiency.

The No Free Lunch theorem reminds us that different types of data require different architectures to fully exploit their characteristics, so as to achieve optimal performance.

What is a residual neural network? What problem does it solve?

Residual Neural Networks (ResNets) are an improved deep neural network architecture proposed by He et al. in 2015. Their core idea is to introduce residual connections (skip connections) to address the degradation problem of deep neural networks: as the number of layers increases, deep neural networks may suffer from an increase in training error, i.e. the degradation phenomenon. This is not due to overfitting, but rather to the vanishing or exploding gradient problem, which makes training difficult.

Residual connections add the input of a certain layer directly to the output of that layer, so that the network learns a residual function (the difference between the actual output and the input) instead of directly learning the mapping from input to output. This design greatly alleviates the vanishing-gradient problem in deep networks.

Mathematically, suppose the input of a certain layer is x, and the corresponding ideal mapping is $H(x)$. A traditional network learns $H(x)$ directly, whereas in a ResNet, by introducing the residual connection, it learns the residual function $F(x) = H(x) - x$. Therefore, the output of this layer is $H(x) = F(x) + x$.

Its main advantages include:

- **Alleviating the vanishing-gradient problem**: residual connections make it easier for gradients to be back-propagated to earlier layers, thereby alleviating the vanishing-gradient problem.
- **Improved training efficiency**: since each residual block contains a direct skip connection, the network can converge more quickly.
- **Support for deeper network architectures**: by introducing residual blocks, ResNet can train very deep networks (e.g. more than 100 layers) while still achieving good performance.

These features have enabled ResNet to achieve remarkable success in tasks such as image classification and object detection, and it is widely used in various deep learning applications.

What are the characteristics of image data? Why are convolutional neural networks more compatible with image data?

The characteristics of image data include:

- **Local correlation**: pixels in an image are usually strongly correlated with their neighbouring pixels. For example, a region of a photograph may represent a particular object, and the neighbouring pixels jointly define the features of that object.
- **Spatial invariance**: patterns in an image (such as edges and corners) exhibit similar characteristics when they appear at different locations.

Convolutional neural networks (CNNs) are more compatible with image data for the following reasons:

- **Convolution operation**: a convolutional layer slides a filter (or convolution kernel) over the input image, allowing it to detect local image features such as edges and textures. The filter parameters are learned through training, so as to adaptively extract the features most beneficial to the task.
- **Weight sharing**: the same convolution kernel shares parameters across the entire image, which greatly reduces the number of model parameters, lowers the computational complexity and memory requirements, and enhances the generalization ability of the model.
- **Pooling layer**: pooling operations (such as max pooling and average pooling) downsample feature maps to reduce their size, thereby reducing the amount of computation while retaining the main features. This helps achieve a certain degree of translation invariance—i.e. an object can still be recognized when it moves within the image.

These characteristics enable CNNs to effectively extract and exploit spatial information when processing image data, and to perform excellently on tasks such as image classification and object detection.

What are the characteristics of language data? Why is the Transformer better at handling language tasks?

The characteristics of language data include:

- **Sequentiality**: language data has a natural order, and the order of words has an important influence on the meaning of a sentence.
- **Context dependence**: words in a language often need to depend on their surrounding context to determine their precise meaning.

The Transformer performs excellently on language tasks for the following reasons:

- **Self-attention mechanism**: through the self-attention mechanism, the Transformer can attend to all other words in the sentence simultaneously when processing each word. This mechanism allows the model to capture long-range dependencies without being constrained by the length of the sequence.
- **Parallel computation**: unlike traditional RNNs, the Transformer can process all positions in a sequence in parallel, which significantly improves training speed and efficiency. This is because RNNs must process the input at each time step sequentially, whereas the Transformer can process the entire sequence at once.
- **Hierarchical structure**: the Transformer is composed of multiple stacked encoder and decoder layers, each of which contains a multi-head self-attention mechanism and a feed-forward neural network. The multi-head self-attention mechanism allows the model to attend to different information at different positions in the input sequence, thereby enhancing its representation ability.

These features give the Transformer significant advantages when handling language tasks (such as machine translation, text generation and semantic analysis), especially when capturing complex contextual dependencies and long-range relationships is needed.

3.1 Fully connected networks

A fully connected network is a classic and fundamental neural network structure, usually composed of an input layer, hidden layers and an output layer, each containing a number of neuron nodes. Apart from the input layer, every neuron in the network receives input from all neurons in the previous layer, forming a "fully connected" structure. These connections all carry learnable weight parameters, allowing the network to adapt to different tasks through training.

In the forward pass of a fully connected network, each hidden-layer neuron takes a weighted sum of the input signals from the previous layer and then transforms it through a nonlinear activation function (such as ReLU, Sigmoid, etc.) to enhance the representational ability of the network. Such layer-by-layer nonlinear transformations enable fully connected networks to fit complex functional relationships, and they have been widely applied in many fields.

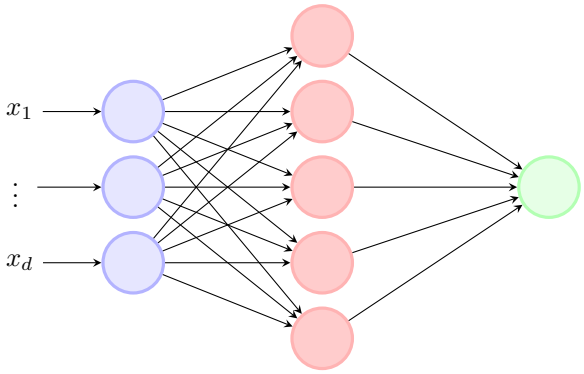


Figure 3.1: A two-layer neural network.

Figure 3.1 shows a schematic diagram of a simple two-layer fully connected neural network. From a mathematical point of view, for an L-layer fully connected network the forward pass can be expressed by the following formula:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{W}^{[L-1]}\sigma \circ (\boldsymbol{W}^{L-2}\sigma \circ (\cdots (\boldsymbol{W}^{[1]}\sigma \circ (\boldsymbol{W}^{[0]}\boldsymbol{x} + \boldsymbol{b}^{[0]}) + \boldsymbol{b}^{[1]}) \cdots) + \boldsymbol{b}^{[L-2]}) + \boldsymbol{b}^{[L-1]}, \quad (3.1)$$

where $\boldsymbol{W}^{[l]} \in \mathbb{R}^{m_{l+1} \times m_l}, \boldsymbol{b}^{[l]} \in \mathbb{R}^{m_{l+1}}, m_0 = d_{in} = d, m_L = d_o$ denote the weight matrix and bias vector of the l-th layer respectively, σ is a nonlinear activation function which is applied to each element of the input vector separately, and the symbol "$\circ$" denotes the entry-wise operation. Note that when describing the number of layers in a network, we usually do not include the input layer, i.e. the number of layers equals the sum of hidden layers and the output layer.

For notational convenience, we denote all parameters of the network by

$$\boldsymbol{\theta} = (\boldsymbol{W}^{[0]}, \boldsymbol{W}^{[1]}, \cdots, \boldsymbol{W}^{[L-1]}, \boldsymbol{b}^{[0]}, \boldsymbol{b}^{[1]}, \cdots, \boldsymbol{b}^{[L-1]}),$$

where $\boldsymbol{W}_{ij}^{[l]}$ denotes the entry of $\boldsymbol{W}^{[l]}$ in row i and column j.

Apart from the above forward-pass formula, we can also define a fully connected network recursively:

$$f_{\boldsymbol{\theta}}^{[0]}(\boldsymbol{x}) = \boldsymbol{x} \tag{3.2}$$

$$f_{\boldsymbol{\theta}}^{[l]}(\boldsymbol{x}) = \sigma \circ (\boldsymbol{W}^{[l-1]} f_{\boldsymbol{\theta}}^{[l-1]}(\boldsymbol{x}) + \boldsymbol{b}^{[l-1]}), \quad 1 \leq l \leq L-1, \tag{3.3}$$

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = f_{\boldsymbol{\theta}}^{[L]}(\boldsymbol{x}) = \boldsymbol{W}^{[L-1]} f_{\boldsymbol{\theta}}^{[L-1]}(\boldsymbol{x}) + \boldsymbol{b}^{[L-1]}. \tag{3.4}$$

This recursive form is more concise and highlights the hierarchical-structure property of fully connected networks.

Each connection in a fully connected network has its own independent weight parameter, with no parameter sharing. Consequently, when the input dimension is high, the number of parameters of the network grows combinatorially. This characteristic makes fully connected networks face challenges such as difficult parameter optimization and high computational complexity when processing high-dimensional data.

Despite this, the fully connected network remains an important fundamental structure in deep learning. It is mainly used to process vectorized structured data and has strong function-fitting ability. In practice, fully connected networks are often used as components of more complex network structures—for example, the classifier part of convolutional neural networks (CNNs).

However, because fully connected networks fail to exploit prior knowledge about the data, their generalization performance is often unsatisfactory when directly processing high-dimensional unstructured data such as raw images or sequences. To overcome this limitation, researchers have developed network structures more suitable for specific types of data, such as convolutional neural networks and recurrent neural networks. By introducing certain prior assumptions (such as local connectivity and weight sharing), these networks improve parameter efficiency while also enhancing the model's ability to capture data features.

3.2 Residual neural networks

With the development of deep learning, researchers found that increasing the depth of a network can improve its expressive power, leading to better performance on tasks such as image classification and object detection. However, as the number of layers grows, training a deep network becomes increasingly difficult. There are two main reasons: first, the vanishing- or exploding-gradient problem, which makes deep networks hard to train; second, the degradation problem, namely that as the network depth increases, performance saturates or even degrades (He et al., 2016).

To solve these problems, Kaiming He et al. (He et al., 2016) proposed the Residual Neural Network (ResNet) in 2015. The core idea of ResNet is to introduce a structure called a "residual connection", so that the network can learn a residual function rather than directly learning the target function. This design greatly alleviates the difficulty of training deep networks, allowing researchers to train networks with hundreds or even thousands of layers and achieve state-of-the-art results on many tasks.

Here we introduce the simplest and most basic form of residual connection:

$$F_{l+1} = \sigma(W_l F_l + b_l) + F_l,$$

where F_l is the output of the l-th layer. This form applies when F_l and F_{l+1} have the same

dimensionality. When their dimensions differ, we can use the form

$$F_{l+1} = \sigma(W_l F_l + b_l) + W_s F_l,$$

where W_s is a trainable matrix whose purpose is to transform the dimension into that of layer $l+1$.

The key point of the residual is that we add the input x directly to the output of that layer, forming a "shortcut" or "bypass" connection.

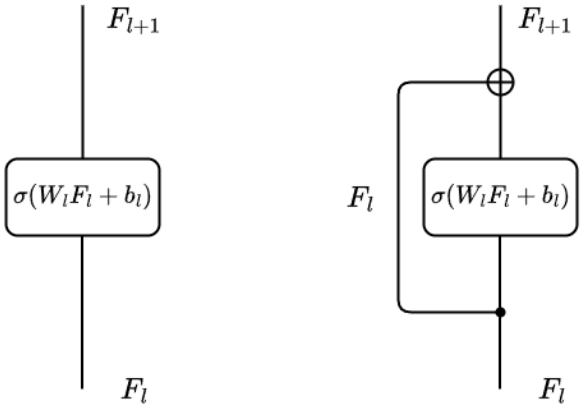


Figure 3.2: A fully connected network architecture versus an architecture with residual connections.

Figure 3.2 contrasts the structural differences between a traditional network and a residual network. We can see that in a residual network, the input F_l is connected to the output via a bypass and added to the output of a fully connected layer. The advantage of this design is that, even if the mapping learned by the fully connected layer is not ideal, the network can still pass the input information through the bypass connection, ensuring that the network's performance does not deteriorate. In terms of learning difficulty, residual connections turn learning into incremental learning—each module only has to learn some new knowledge that is added on top of the preceding modules. Intuitively, this is easier than learning the complete mapping directly.

From another perspective, residual connections provide a way to fuse features from different layers. By passing shallow features directly to deeper layers, the network can flexibly choose which features to use, thereby enhancing its representation capacity. In addition, residual connections act somewhat like a "gradient highway", allowing gradients to flow more smoothly through a deep network and alleviating the vanishing-gradient problem.

The proposal of residual networks is a milestone in the history of deep learning. It not only achieved the best results of its time on the ImageNet image classification task, but also opened up an entirely new network design paradigm. Subsequently, many classic network architectures—such as Inception-ResNet (Szegedy et al., 2017), DenseNet (Huang et al., 2017), and ResNeXt (Xie et al., 2017)—all drew on the idea of residual connections and extended it to broader application scenarios.

The influence of residual networks is not limited to computer vision; they have also been widely applied in natural language processing, speech recognition, and other fields. For example, Google's

machine translation system (Wu et al., 2016) uses residual connections to build deep encoder–decoder networks, greatly improving translation quality. Microsoft's speech recognition system also uses residual networks to extract speech features (Xiong et al., 2018).

The proposal of residual networks solved a key difficulty in training deep networks and promoted the further development of neural networks. It not only had a profound impact in academia but has also been widely adopted in industry, advancing the development of AI technology. One may say that residual networks are one of the cornerstones of modern deep learning, laying a solid foundation for subsequent research.

3.3 Convolutional neural networks

Convolutional Neural Networks (CNNs) are the most powerful tool in deep learning for processing image data. Since the 1980s, CNNs have emerged in image recognition. In 1989, LeCun et al. (LeCun et al., 1989) first applied CNNs to handwritten-digit recognition, achieving a landmark result. Since then, CNNs have shone in computer vision, continually setting new performance records on tasks such as image classification (Krizhevsky et al., 2012), object detection (Girshick et al., 2014), and semantic segmentation (Long et al., 2015), far surpassing traditional machine learning methods. In 2012, AlexNet, proposed by Krizhevsky et al. (Krizhevsky et al., 2012), won the ImageNet image classification challenge by a large margin, igniting the deep learning research boom and cementing the dominance of CNNs in computer vision.

The excellent performance of CNNs on image tasks owes much to their unique network architecture. Unlike traditional fully connected neural networks, CNNs introduce a series of innovative structures tailored to the characteristics of images, such as local receptive fields, weight sharing, and pooling. These designs enable CNNs to efficiently extract local image features and combine them layer by layer to form more abstract and robust representations. One may say that it is precisely thanks to the full exploitation of image features that CNNs achieve such outstanding performance.

To truly understand the elegant ideas behind CNNs, we should start from a more fundamental question: what characteristics do image datasets possess? Only by grasping the essential properties of image data can we appreciate the design ideas behind CNNs, and then better apply and innovate on this tool. At the same time, we should also pay attention to how humans and other organisms process image data—what inspiration does the biological visual cortex provide for the design of CNNs? Therefore, in this section, we will first introduce the characteristics of image datasets, then introduce the image-processing structure of the biological primary visual cortex, which is of great inspirational significance for CNN design, and finally systematically introduce the core structure of CNNs, revealing their internal working mechanisms.

3.3.1 Characteristics of image datasets

Visual information plays an extremely important role among the information received by humans. Some studies estimate that visual information may account for more than 80% of the total information humans acquire. Although this proportion is difficult to quantify precisely, it reveals a fact: humans are "visual animals", and vision plays a dominant role in our perceptual system. Therefore, to study visual artificial intelligence, we must deeply understand the characteristics of natural-image data.

A great deal of research and experience indicates that correlation and invariance are two key features of image data. Correlation means that pixels in an image are not independent but exhibit

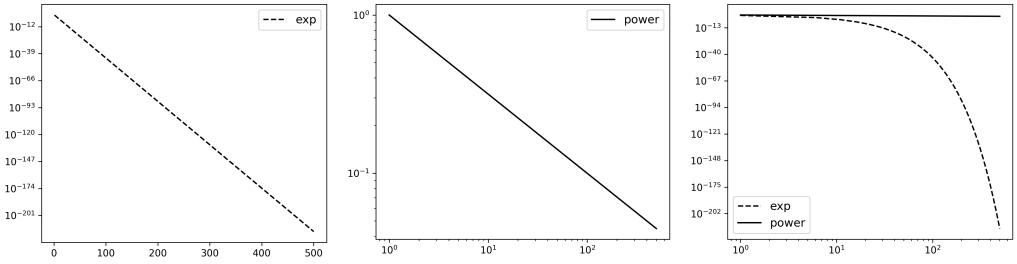
strong correlations. This correlation is reflected in the spatial domain (neighbouring pixels tend to take similar values) and in the frequency domain (the spectrum of an image is usually concentrated in the low-frequency part). This suggests that tasks such as image compression and denoising can exploit this correlation. Invariance means that the semantic content of an image is, to a certain extent, invariant under transformations. For example, even if a photo of a cat has been translated, rotated or scaled, we can still recognize that it is a cat. This suggests that a visual intelligent system needs to be robust to such transformations.

Correlation

Many data with spatial or temporal structure exhibit local correlation, that is, the correlation strength between data at two spatial points decreases as the distance grows. This local correlation is especially apparent in natural images. The pixels in an image are not independently distributed but exhibit strong spatial correlations. Neighbouring pixels usually have similar grey values or colours, which stems from the continuity of object surfaces in the real world and the smooth variation of lighting. Understanding and exploiting this correlation is crucial for tasks such as image processing, compression, and feature extraction.

To characterize the decay features of the correlation, we first introduce two common decay models: exponential decay and power-law decay. Exponential decay is characterized by a definite decay period—after one decay period the initial value drops to half of the original value, and we call this decay period the half-life. For example, for an exponential decay model of the form $y = e^{-\frac{x}{\tau}}$, its half-life is $\tau \ln 2$. Exponential decay is reflected in many physical processes, such as the decay of radioactive elements and the discharge of a capacitor.

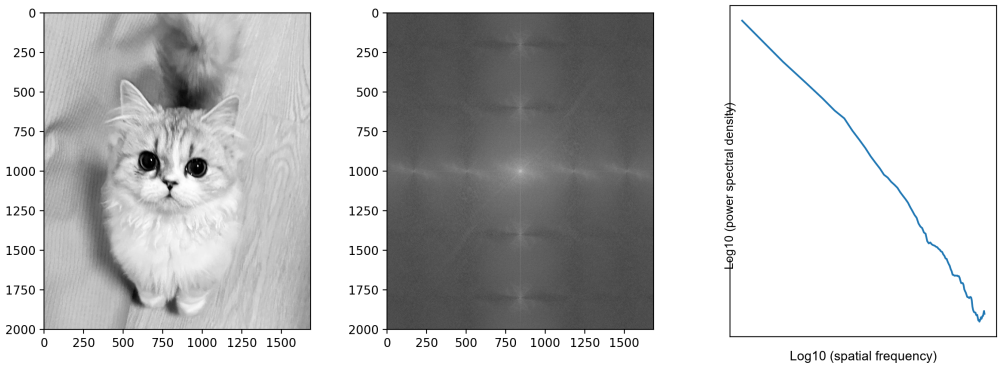
Power-law decay has no decay period, as illustrated in Figure 3.3(c). It is much slower than exponential decay; for instance, for $y = 1/x$, the larger x is, the slower its decay. Another way to understand power-law decay is through exponential decay: power-law decay can be regarded as the sum of exponential decays with infinitely many half-lives. For example, $\int_0^\infty \exp(-\alpha x) d\alpha = \frac{1}{x}$, where the left-hand side is the integral of exponential decays with different periods and the right-hand side is a power-law decay. Power-law decay is very useful for describing many natural phenomena, such as the relationship between gravitational potential energy and distance, or the distribution of urban populations.



(a) Exponential decay (b) Power-law decay (c) Comparison of decay rates: exponential vs. power-law

Figure 3.3: Data decay

To verify the local-correlation properties of natural images, we can compute two commonly used



(a) Image (b) 2D spectrum (c) Radially averaged spectrum

Figure 3.4: The 2D spectrum and the radially averaged spectrum of an image.

quantities: the power spectrum and the correlation function. In fact, these two quantities can be transformed into one another via the Fourier transform.

The power spectrum of an image reflects the distribution of energy at different frequencies. By performing a Fourier transform on the image and computing the squared modulus of each frequency component, we obtain the power spectrum. As shown in Figure 3.4, in a spectrum diagram a higher brightness indicates greater energy at that frequency component. Analyzing the power spectra of a large number of natural images, we find that they generally exhibit a power-law distribution: low-frequency components dominate, while high-frequency components occupy a smaller but non-negligible share. This means that most of the image's energy is concentrated in the lower frequency range, which is consistent with the local smoothness and slow variation of images. At the same time, the presence of high-frequency components reflects structural information such as details, edges, and textures in the image. The power-law property of the power spectrum provides important prior knowledge for image processing and analysis.

Apart from frequency-domain analysis, we can also study the correlation of an image directly in the spatial domain. A commonly used measure is the correlation function, which describes how the degree of correlation between two pixels in the image relates to their distance. For two pixels ϕ_1 and ϕ_2, their correlation can be measured by the mutual information I:

$$I = \iint p_2\left(\phi_1, \phi_2\right) \log \frac{p_2\left(\phi_1, \phi_2\right)}{p_1\left(\phi_1\right) p_1\left(\phi_2\right)} \mathrm{d}\phi_1 \mathrm{d}\phi_2, \tag{3.5}$$

where $p_2(\phi_1, \phi_2)$ is the joint probability distribution of the two pixels and $p_1(\phi_1)$ and $p_1(\phi_2)$ are their respective marginal distributions. When the two pixels are correlated, $I > 0$; when they are independent, $p_2\left(\phi_1, \phi_2\right) = p_1\left(\phi_1\right) p_1\left(\phi_2\right)$, and hence $I = 0$.

By computing the average mutual information between pixel pairs at different distances, we obtain the correlation function of the image. The correlation function quantitatively characterizes how the local correlation of an image decays with distance. As shown in Figure 3.5, experiments and studies indicate that for natural images the correlation function usually exhibits a power-law-

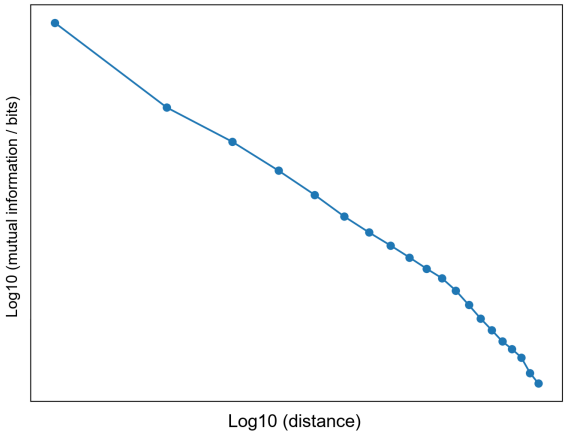


Figure 3.5: The correlation between pixels of a natural image decays with the inter-pixel distance in a power-law form.

like decay: the correlation between nearby pixels is strong, while that between distant pixels is weaker, but not completely absent. This non-local nature of spatial correlation suggests that, when processing images, we should not only consider information from a local neighbourhood but also appropriately exploit some long-range dependencies.

Image data exhibits clear spatial correlations, which is both an opportunity and a challenge. On the one hand, we can use this prior knowledge to design more efficient and more robust image-processing and analysis algorithms; on the other hand, we also need to develop machine learning models and training strategies that fully exploit image correlations, so as to mine valuable information from massive amounts of data. Deep learning models, such as convolutional neural networks, are a good example: through local connectivity and weight sharing, they exploit the spatial correlations of images well, achieving great success in computer vision.

Invariance

The invariance of image data refers to the property that the semantic content of an image remains unchanged under certain transformations. These transformations include translation, rotation, scaling, brightness changes, and so on; Figure 3.6 shows various kinds of invariance. Invariance is a key requirement for visual intelligence, because in the real world the same object may appear in different poses, angles, and sizes, yet we still want an intelligent system to recognize it accurately. Below, we examine the invariance of image data in detail from a mathematical perspective.

Translation invariance Translation invariance means that the spatial movement of an image does not affect its content. Suppose an image can be represented as a 2D function $f(x, y)$, where (x, y) are the pixel coordinates. Translating image f horizontally to the right by a units and

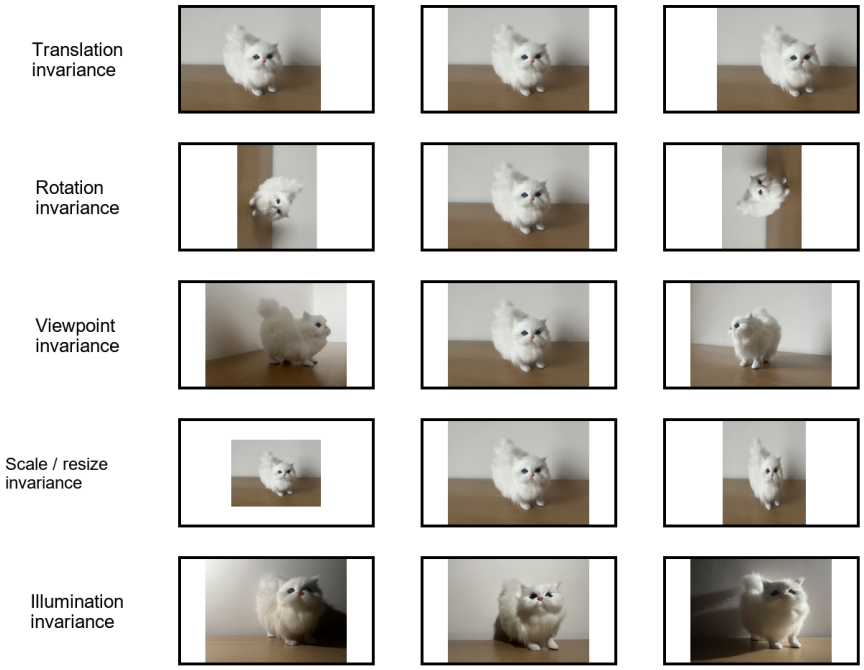


Figure 3.6: Various invariances of images.

vertically upward by b units can be expressed as

$$f'(x, y) = f(x - a, y - b).$$

If an image-processing algorithm or feature is invariant to translation, then its output on the original image f and on the translated image f' should be the same.

Rotation invariance Rotation invariance means that the content of an image does not change after rotation. For an image $f(x, y)$, if we rotate it counter-clockwise by θ degrees, the rotated image $f'(x, y)$ can be expressed as

$$f'(x, y) = f(x \cos\theta - y \sin\theta, x \sin\theta + y \cos\theta).$$

If an algorithm is invariant to rotation, its output on f and on f' should be the same.

Scale invariance Scale invariance means that scaling an image does not affect its content. If we scale image $f(x, y)$ by factors α and β in the horizontal and vertical directions respectively, the scaled image $f'(x, y)$ can be expressed as

$$f'(x, y) = f(\frac{x}{\alpha}, \frac{y}{\beta}).$$

If an algorithm is invariant to scaling, its output on f and on f' should be the same.

Brightness invariance Brightness invariance means that changes in image brightness do not affect its content. If we increase or decrease the brightness of image $f(x, y)$ by a constant c, the modified image $f'(x, y)$ can be expressed as

$$f'(x, y) = f(x, y) \pm c.$$

If an algorithm is invariant to brightness, its output on f and on f' should be the same.

Illumination invariance Illumination invariance means that the content of an image should remain unchanged under variations in lighting conditions. Whether the light source is on the left or the right of the image, the semantic content of the image should be unaffected.

Suppose an image $f(x, y)$ is captured under illumination condition L_1, yielding the image $f_{L_1}(x, y)$. If we change the position and angle of the light source to obtain another illumination condition L_2, the image captured under this condition is $f_{L_2}(x, y)$. If an image-recognition algorithm is invariant to illumination, then its recognition results on f_{L_1} and on f_{L_2} should be the same.

Mathematically, we can model an illumination change as a function g that maps an image under the original illumination condition to an image under the new illumination condition:

$$f_{L_2}(x, y) = g(f_{L_1}(x, y)).$$

If a recognition algorithm R is invariant to illumination, then it should satisfy

$$R(f_{L_1}(x, y)) = R(f_{L_2}(x, y)) = R(g(f_{L_1}(x, y))).$$

That is, regardless of whether the image is captured under illumination L_1 or L_2, the recognition algorithm should give the same result. This property is very important for deploying visual intelligent systems in the real world, because real-world illumination conditions are variable, and we want our algorithms to operate stably under different lighting.

Statistical invariance Statistical invariance means that the statistical properties of an image should be consistent across different regions. This suggests that we can use the same method to extract features from different parts of the image.

For example, suppose we want to detect edges in an image. Intuitively, the edges of objects in an image correspond to regions where the pixel values change rapidly. We can design an operator that at each location computes the difference between the pixel value on the left and that on the right; such an operator can extract vertical edges. If statistical invariance holds, then this operator should behave consistently across different regions of the image, because the statistical properties of edges are the same.

Another example is the autocorrelation function of an image. The autocorrelation function measures the correlation between pixels at different positions, and depends only on the relative distance between these positions, not on their absolute positions. Formally, for all images $f(x, y)$ in a sufficiently large natural dataset $\mathcal{S}$, the autocorrelation function $C_{x,y}(a, b)$ is defined as

$$C_{x,y}(a, b) = \mathbb{E}_f f(x, y) f(x - a, y - b).$$

If statistical invariance holds, then $C_{x,y}(a, b)$ should be independent of the choice of (x, y) and only depend on the distance $\sqrt{a^2 + b^2}$. This is because the statistical properties of images are invariant under translation.

Intuitively, we randomly pick a natural image, record the grey value at pixel (x, y), multiply it by the grey value at the position obtained by moving a units to the left and b units upward, and take the expectation over a large number of natural images. We then obtain the autocorrelation function $C_{x,y}(a, b) = \mathbb{E}_f\big[f(x, y)\, f(x - a, y - b)\big]$. If the two points often become bright or dark at the same time, the expected product is large; if they are often one bright and one dark, the expectation is small or even negative. Hence $C_{x,y}(a, b)$ quantifies "the degree to which two points separated by (a, b) vary synchronously in brightness".

Statistical invariance reflects a basic property of natural images: they have similar statistical characteristics across different regions. This property has wide applications in tasks such as texture analysis and image synthesis. It also provides important prior knowledge for understanding and designing image-processing algorithms: thanks to statistical invariance, we can repeatedly use the same feature-extraction algorithm across the entire image, drastically reducing the number of parameters.

In deep learning, we usually achieve or exploit the invariance of image data in the following ways:

1. Data augmentation: during training we apply random translation, rotation, scaling, brightness changes and other operations to the images, generating more training samples. This can improve the invariance and robustness of the model.

2. Feature design: some traditional image features such as SIFT (scale-invariant feature transform) and HOG (histogram of oriented gradients) are designed mathematically so as to be invariant to translation, rotation, scaling, etc. These features can serve as inputs to deep learning or as inspiration for the design of deep network architectures.

3. Convolutional neural networks (CNNs): through local receptive fields, weight sharing and pooling, CNNs fully exploit the translation invariance of image data. Furthermore, through data augmentation and special network design, CNNs can also obtain a certain degree of rotation and scale invariance. This is exactly what we will focus on next.

The invariance of image data is an important foundation of visual intelligence. Through mathematical analysis and algorithm design, we can better understand and exploit this property, building visual intelligent systems that are more robust and generalize better. In the deep-learning era, the realization and application of invariance are also continually updated and iterated, injecting new vitality into the development of computer vision.

3.3.2 Image-processing structure of the primary visual cortex

In the previous section, we explored the key characteristics of image data, such as correlation and invariance. These properties reflect the intrinsic regularities of the visual world and point the way for the development of visual intelligence. At the same time, we should also note that nature, through a long evolutionary process, has already created an extraordinary visual system—biological vision. So, how does the biological visual system process image information? And what insights does it offer for the design of artificial neural networks?

To answer these questions, we need to take a closer look at the structure and function of the visual cortex. In the 1950s and 1960s, the neuroscientists David Hubel and Torsten Wiesel carried out pioneering studies on the visual cortex of cats and monkeys, revealing the mechanisms by which visual information is processed in the brain. Their work not only deepened our understanding of visual cognition but also provided an important biological basis for later convolutional neural networks.

Hubel and Wiesel found that the primary visual cortex (V1) contains two functionally distinct types of cells: simple cells and complex cells. Simple cells are highly sensitive to lines or edges at specific positions and orientations in the visual field. When a stimulus aligned with the cell's preferred orientation appears in the visual field, the simple cell produces a strong electrophysiological response. Different simple cells prefer stimuli of different orientations, which means that the primary visual cortex actually extracts basic features in the image, such as edges and lines.

Figure 3.7 shows an idealized model of a simple cell. The receptive field of this cell consists of a series of "on" regions (excitatory) and "off" regions (inhibitory), arranged in a regular spatial pattern. When a visual stimulus matches the receptive-field structure of the cell—that is, bright regions fall in on-regions and dark regions fall in off-regions—the cell will be activated. This fine spatial integration enables simple cells to selectively respond to edges or lines of a specific orientation.

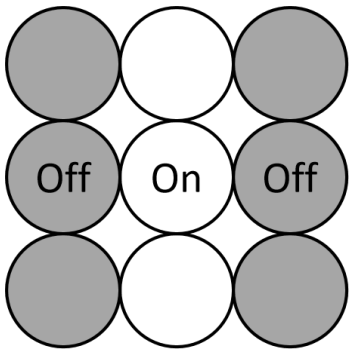


Figure 3.7: An example receptive field of a simple cell in area V1.

Further research shows that complex cells can integrate inputs from multiple simple cells and respond selectively to more complex features. For example, a complex cell may prefer an edge of a certain orientation but be insensitive to the exact position of that edge in the visual field. This property is called position invariance or position tolerance. The discovery of complex cells indicates that the visual cortex extracts increasingly abstract and complex features in a hierarchical, recursive manner.

These findings in neuroscience have important implications for the design of artificial neural networks. We can imagine that, on top of a fully connected network, we introduce a feature extractor similar to the simple cells of the visual cortex. Such an extractor should be able to detect the same feature at different positions and possess a certain degree of position invariance. In fact, this is precisely the prototype of a key component in convolutional neural networks—the convolution kernel. At the same time, by combining these basic feature extractors, we can construct units that are sensitive to more complex patterns, just like complex cells in the visual cortex. In addition, the hierarchical structure of the visual cortex suggests that, by stacking these feature-extraction units layer by layer, a neural network can learn hierarchical feature representations and thereby form a deep understanding of the input image.

The pioneering work of Hubel and Wiesel revealed the basic computational principles of the visual cortex, opening a new era for the study of visual information processing. It not only deepened our understanding of biological vision but also laid the foundation for the development of artificial neural networks, especially convolutional neural networks. Although we have not yet explicitly

introduced the concept of CNNs, we can already see their prototype in the structure of the visual cortex. In the following sections, we will further explore how to draw on these biological insights to build powerful visual intelligent models.

3.3.3 Convolutional neural networks

In the previous section, we explored the biological visual system, in particular the remarkable ability of the primary visual cortex to process visual information. The discovery of simple cells and complex cells reveals some basic computational principles of the visual cortex, such as local feature extraction, orientation selectivity and position invariance. These principles not only deepen our understanding of biological vision but also provide important inspiration for designing efficient artificial visual systems.

Inspired by the structure of the visual cortex, researchers proposed an artificial neural network model called the convolutional neural network (CNN). The design of CNNs cleverly exploits two key characteristics of image data: local correlation and translation invariance. By introducing the convolution operation and the pooling operation, CNNs can effectively extract local features of an image and are robust to changes in the position of objects.

A standard CNN is usually composed of the following modules:

- Convolutional layer: the core operation of this layer is convolution. The convolution kernels in a convolutional layer can be regarded as local feature extractors, similar to the simple cells in the visual cortex. Each kernel slides over the image and extracts local features at different positions. Multiple kernels can extract different types of features such as edges and textures. The convolution operation has translation invariance: no matter where the target feature appears in the image, it can be detected by the same kernel. This echoes the translation invariance of images and of the visual cortex.

- Pooling layer: a pooling layer usually follows a convolutional layer; its role is to compress the spatial dimensions. Common pooling operations include max pooling and average pooling. The pooling layer can extract contour information from the image and further enhance the model's translation invariance. Taking max pooling as an example, within each local region, max pooling always outputs the maximum value of the data in that region; no matter how the feature is translated within this local region, the maximum value remains unchanged. Such invariance helps the model focus on the key features of the image while ignoring trivial details.

- Fully connected layer: after several convolution–pooling layers, a CNN usually flattens the resulting 2D feature maps into a 1D vector and then feeds it into one or two fully connected layers. This part has a structure similar to a traditional fully connected network and can be regarded as processing and predicting from the high-level features extracted by the earlier layers.

- Output layer: the number of neurons in the output layer depends on the type of task. For example, in an image-classification problem, the number of output neurons corresponds to the number of categories; in an object-detection problem, the output layer needs to predict the position and class of the target; in an image-generation problem, the output layer may be a high-dimensional image vector. The design of the output layer is closely tied to the task.

By stacking convolutional and pooling layers, a CNN gradually extracts local features of an image and abstracts them into high-level representations, achieving a mapping from pixels to semantics. This hierarchical, progressive structure mimics, to some extent, the information-processing process of the visual cortex. The clever design of CNNs has led to their great success on tasks such as image recognition and object detection, and for a long time made them the mainstream model in computer vision.

In the following subsections, we will dive into the key components of a CNN—such as the convolution kernel, padding and stride, multi-channel processing, and pooling—revealing their internal working principles and giving the reader a more comprehensive understanding of CNNs.

The convolution operation

Convolution is the core operation of convolutional neural networks and has a long history in image processing and computer vision. Its earliest roots can be traced back to 1980, when Kunihiko Fukushima proposed the Neocognitron, a model inspired by biological neural networks that, for the first time in machine learning, introduced a concept similar to convolution to simulate the feature-extraction process of the human visual system Fukushima (1980). Subsequently, in 1989, Yann LeCun combined the idea of convolution with the back-propagation algorithm, proposed the convolutional neural network (CNN), and achieved a breakthrough on the handwritten-digit recognition task LeCun et al. (1989). These early works not only demonstrated the effectiveness of convolution in extracting features from images but also laid the foundation for the subsequent vigorous development of deep learning.

Today, convolution has become an indispensable tool in deep learning, and its importance can hardly be overstated. In modern convolutional neural networks, the convolution operation effectively captures spatial features of an image through the design of local receptive fields, greatly improving computational efficiency and feature-extraction quality. This enables CNNs to demonstrate strong performance on tasks such as image classification, object detection and semantic segmentation, and to be widely used in many computer-vision applications.

So what exactly is convolution? Simply put, convolution is a mathematical operation that describes the interaction between two functions. In image processing, these two functions are usually the input image and the convolution kernel (also called the filter). The convolution process can be viewed as sliding the kernel over the input image and computing a weighted sum at each position. This process can extract local features such as edges and textures.

Mathematical definition of convolution To better understand the convolution operation in convolutional neural networks, we first give its mathematical definition. Let $f(t)$ and $g(t)$ be two integrable functions on the real line. We define their convolution $(f * g)(t)$ as the following integral transform:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) \, \mathrm{d}\tau. \tag{3.6}$$

Since images are two-dimensional, here we consider the 2D convolution, where $f(x, y)$ denotes the input feature map and $g(x, y)$ denotes the convolution kernel. The continuous version of the convolution formula is then:

$$(f * g)(t_1, t_2) = \int_{-\infty}^{\infty} f(\tau_1, \tau_2) g(t_1 - \tau_1, t_2 - \tau_2) \, \mathrm{d}\tau_1 \, \mathrm{d}\tau_2. \tag{3.7}$$

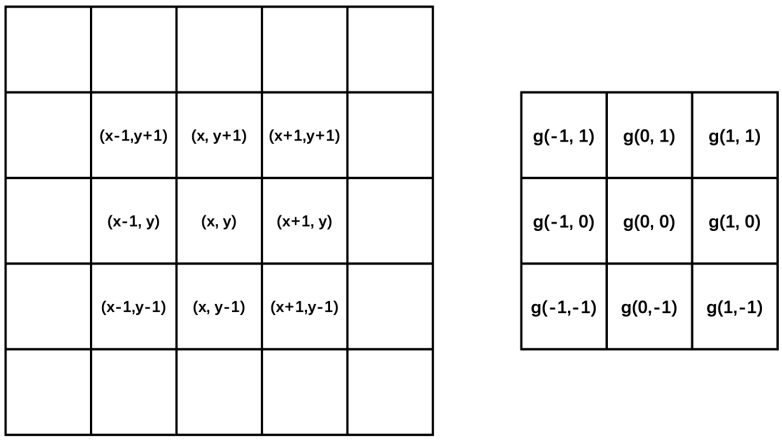


Figure 3.8: Illustration of the convolution operation.

Figure 3.8 illustrates the convolution operation: the left panel is the input feature map $f(x, y)$, and the right panel is the convolution kernel $g(x, y)$. The convolution operation can be viewed as sliding the kernel over the input feature map and computing a weighted sum at each position. Mathematically, the 2D convolution operation can be written as

$$\begin{aligned} F(x,y) = {} & f(x-1,y+1)g(-1,1) + f(x,y+1)g(0,1) + f(x+1,y+1)g(1,1) \\ & + f(x-1,y)g(-1,0) + f(x,y)g(0,0) + f(x+1,y)g(1,0) \\ & + f(x-1,y-1)g(-1,-1) + f(x,y-1)g(0,-1) + f(x+1,y-1)g(1,-1) \end{aligned}$$

This formula shows that the convolution operation is computed by element-wise multiplying the convolution kernel with a local region of the input feature map and summing the results, yielding the value at each position of the output feature map.

Feature extraction with a convolution kernel The convolution kernel plays a crucial role in a convolutional neural network: it determines what kind of features the network can extract. By designing different kernels, we can make the network apply various transformations and filtering operations to the input image, yielding rich and diverse feature maps. These feature maps contain information about the image at different scales, orientations and abstraction levels, providing strong support for the subsequent recognition and classification tasks.

Let us look at some concrete examples. Figure 3.9 shows feature maps obtained by applying different convolution kernels to an input image. Among them, a Blur kernel can smooth the image and remove high-frequency noise, while a Sharpen kernel can enhance image details and edges. These two operations are very common in traditional image processing, while a CNN can automatically optimize the parameters of such kernels through learning, adapting them to the requirements of different tasks. This is very similar to the feature-extraction function of simple cells in the visual cortex.

Padding and stride In practical applications, we often need to adjust some details of the convolution operation to meet different needs. Among them, padding and stride are two commonly

Sharpen	Box blur (normalized)	Gaussian blur (approximation)	Edge detect
$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$	$\frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$	$\frac{1}{16}\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$	$\begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$

Figure 3.9: Feature maps obtained with different convolution kernels.

used techniques.

Consider the convolution operation in Figure 3.10: if we only convolve a 5×5 image with a 3×3 kernel, we end up with a 3×3 output, which is smaller than the original image.

Sometimes we do not want the image to "shrink" after convolution. To address this, we introduce the zero-padding operation, i.e. we pad some zeros around the original image before performing convolution, so as to ensure that the output size after convolution is as close as possible to the original image size. Figure 3.11 shows an example of zero padding. The original image is a 4×4 image; convolving it with a 3×3 kernel yields a 2×2 output. After adding one layer of zero padding (the dashed border in the figure), the image becomes 6×6, and the result of the convolution becomes 4×4. After adding two layers of zero padding, the image becomes 8×8, and the result of the convolution becomes 6×6. This approach has two benefits: first, it can preserve information at the edges of the image; second, it makes the depth and the receptive field of the network easier to control.

Remark 8. *The receptive field refers to the region or range in the input image that a certain neuron in the network can "see". It describes which part of the input image affects the activation of that neuron. As convolutional layers are stacked, the receptive field generally grows, that is, neurons in deeper layers of the network can "see" a larger range of information in the input image.*

Stride refers to the step size with which the convolution kernel slides over the input feature map. By adjusting the stride, we can control the size of the output feature map after convolution while reducing the amount of computation. A larger stride lets the kernel quickly sweep across the input feature map, extracting more global information; a smaller stride captures more local details. The introduction of stride gives convolutional neural networks greater flexibility, allowing them to adapt to inputs of different scales and resolutions. Figure 3.12 shows an example of the stride operation: a 5×5 original image becomes a 7×7 image after zero padding and is convolved with a 3×3 kernel.

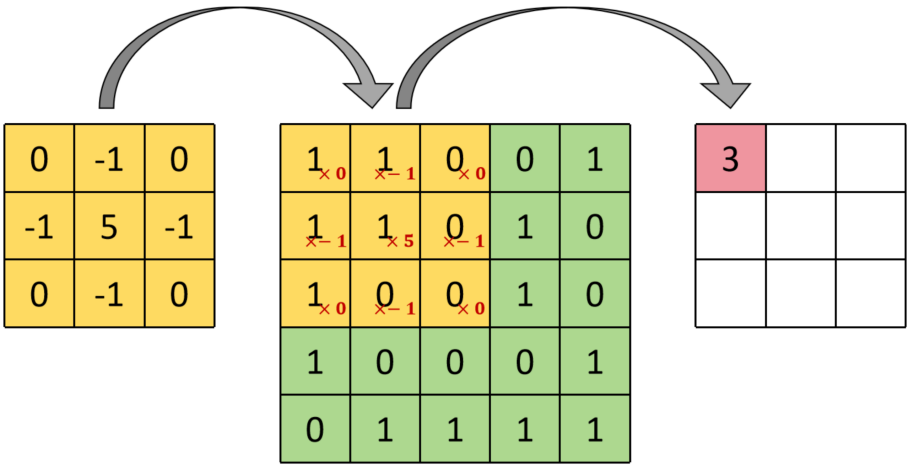


Figure 3.10: Feature extraction from an image by convolution.

The first row does not use stride and yields a 5×5 output; the second row uses stride $(2, 2)$—i.e. a step size of 2 in both the horizontal and vertical directions—and yields a 3×3 output.

Multi-channel convolution In the real world, images are often not single-channel grey-scale images but multi-channel colour images. For example, an RGB colour image has three channels: red, green and blue. To handle such cases, convolutional neural networks introduce the concept of multi-channel convolution.

The basic idea of multi-channel convolution is that, for each input channel, an independent set of convolution kernels is used to perform the convolution, and then the convolution results are summed together and a bias term is added to obtain the final output feature map. This design allows the network to extract features from different channels of the input image and to automatically learn the interactions between them. Multi-channel convolution greatly enriches the expressive power of convolutional neural networks, enabling them to handle more complex and diverse visual tasks. In the example shown in Figure 3.13, three 3×3 kernels are used to convolve with the three channels separately; the values obtained at each point are summed together and a common bias term is added to produce one output. Because each neuron shares parameters, the bias term is the same; in this example, the bias is always b. In this convolution operation, the three kernels provide $3 \times 3 \times 3 = 27$ parameters; together with the bias term, there are 28 parameters in total.

The convolution operation is the core of convolutional neural networks: it cleverly exploits the local correlation and translation invariance of images and uses the convolution kernel to achieve efficient feature extraction. Techniques such as padding, stride and multi-channel convolution further enhance the flexibility and adaptability of convolution operations, enabling CNNs to handle a wide variety of complex visual tasks. These designs are inspired not only by traditional image-processing methods but also by the information-processing mechanisms of the biological visual system. One may say that the convolution operation is a bridge connecting images, biological vision and artificial intelligence, opening up a promising path for the development of computer vision.

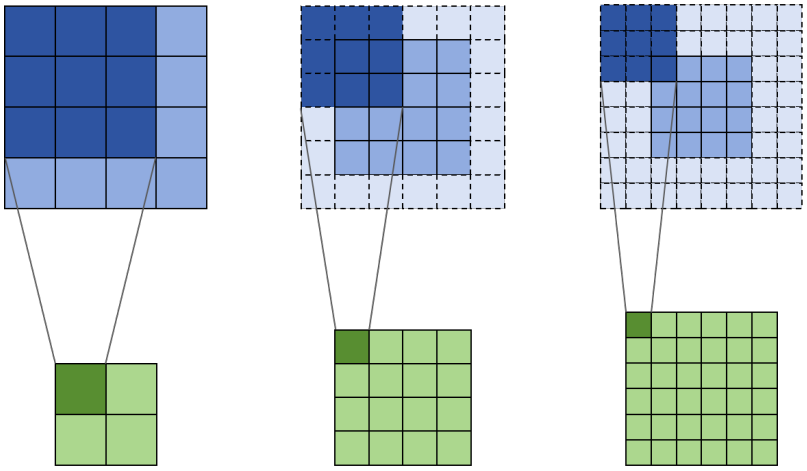


Figure 3.11: An example of the zero-padding operation.

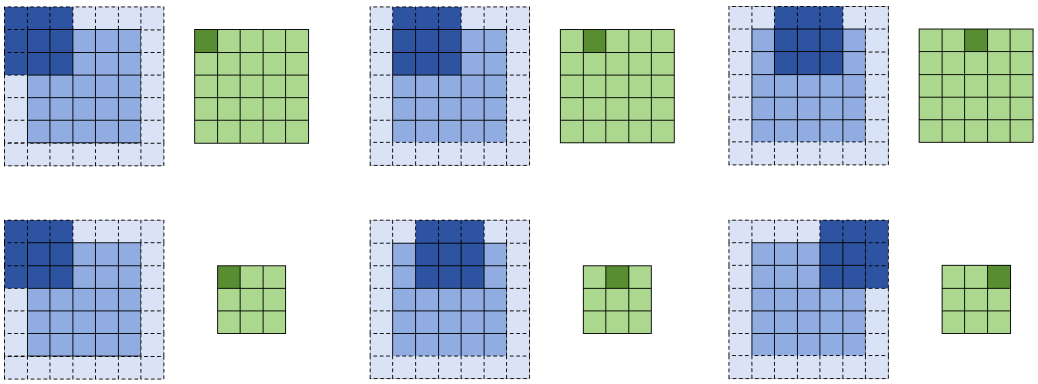


Figure 3.12: An example of the stride operation.

Why introduce convolution? The spatial correlation of images affects not only the design of image-processing algorithms but also poses challenges for the training of machine learning models. Suppose we want to learn a discriminative model that predicts whether two pixels are correlated. If the two pixels are independent, by the law of large numbers we need a large amount of sample data to accurately estimate their independence. Specifically, suppose two pixels x and y are mutually independent, with zero mean and unit variance; we can derive the statistical properties of their covariance.

First, the covariance is defined as:

$$\text{cov}(x, y) = E((x - E(x))(y - E(y))) \tag{3.8}$$

Since x and y have zero mean, we have

$$\text{cov}(x, y) = E(xy) \tag{3.9}$$

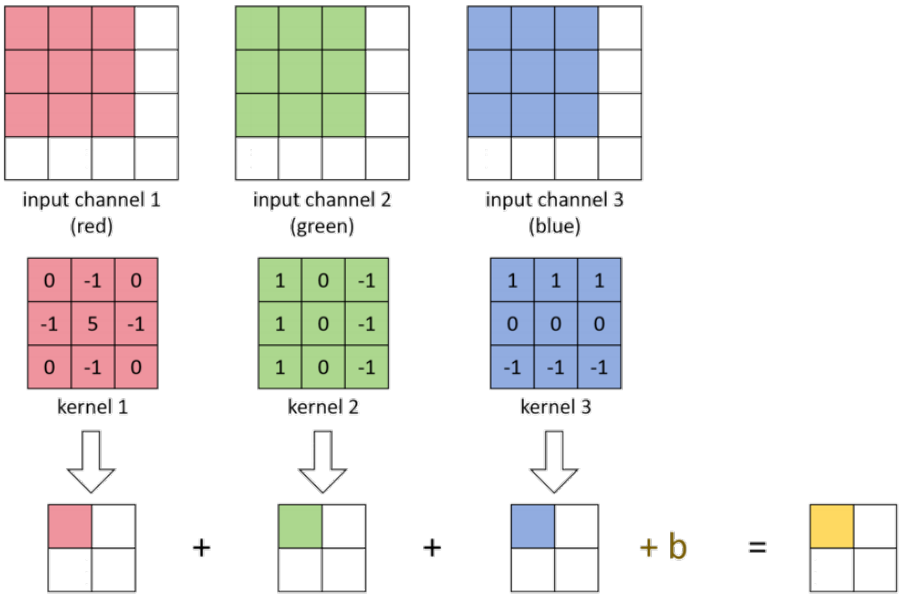


Figure 3.13: An example of multi-channel convolution.

For n samples, the estimate of the covariance is

$$\text{cov}(x, y) = \frac{1}{n} \sum_{i=1}^{n} x_i y_i \tag{3.10}$$

Since x and y are mutually independent, $E(xy) = E(x)E(y) = 0$, so

$$\mathbb{E}(\text{cov}(x, y)) = 0 \tag{3.11}$$

The variance of the covariance estimate is

$$\begin{aligned}
\text{Var}(\text{cov}(x, y)) &= E((\text{cov}(x, y) - E(\text{cov}(x, y)))^2) \\
&= E((\frac{1}{n} \sum_{i=1}^{n} x_i y_i)^2) = \frac{1}{n^2} E((\sum_{i=1}^{n} x_i y_i)^2) \\
&= \frac{1}{n^2} \sum_{i=1}^{n} E(x_i^2 y_i^2) = \frac{1}{n^2} \sum_{i=1}^{n} E(x_i^2) E(y_i^2) \\
&= \frac{1}{n^2} \sum_{i=1}^{n} 1 \cdot 1 = \frac{1}{n}
\end{aligned}$$

Therefore, the standard deviation of the covariance estimate is

$$\sqrt{\text{Var}(\text{cov}(x, y))} = \frac{1}{\sqrt{n}} \tag{3.12}$$

We see that the mean of the covariance is 0 and its standard deviation is inversely proportional to the square root of the sample size. This means that if we want our algorithm to detect a covariance whose standard deviation is below 10^{-3} in order to claim that the two pixels are uncorrelated, we need on the order of 10^6 data points. This places very high demands on both data collection and model training.

To further illustrate the importance of convolution, we conduct an experiment on recognizing local correlations. In the experiment, the input $\mathbf{x}$ is a 100-dimensional vector, where each dimension is sampled uniformly from $[0, 1]$, and the output y is the sum of the first 10 dimensions of $\mathbf{x}$:

$$y = \sum_{i=1}^{10} x_i, \tag{3.13}$$

The purpose of this design is to ensure that the output y depends only on a local part of the input x, so that the dataset satisfies the local-correlation assumption.

Suppose there is a small amount of noise in the training data set, that is $y_{\text{train}} = \sum_{i=1}^{10} x_i + \mathcal{N}(0, 0.01)$. We separately use a fully connected network (DNN) and a neural network with a one-dimensional convolution of size and stride both equal to 5 in front of the fully connected layer (conv+DNN) to fit this function. For the neural network with a convolutional layer, the 100-dimensional input vector is transformed into a 20-dimensional vector after the convolution and then fed into the fully connected network to produce the output. In this experiment, we focus on the case where data is insufficient and contains noise, so the training set size is set to 50. Figure 3.14 shows the experimental results. We can see that, when the data are insufficient and noisy, the fully connected network uses the other 90 unrelated input dimensions to fit the noise in the training data, and thus has no generalization on the test set. In contrast, the neural network with a convolutional layer, thanks to convolution's ability to extract local information, essentially fails to learn the noise and generalizes well on the test set.

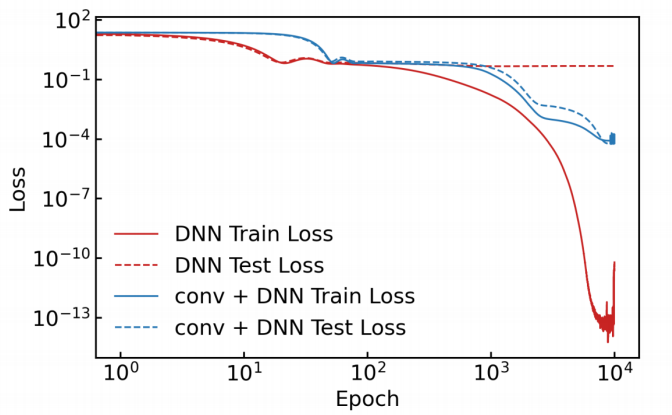


Figure 3.14: Performance of a fully connected network and a neural network with convolution when fitting a noisy dataset with local-correlation structure.

This experiment visually illustrates the advantage of convolution in handling data with local correlation. In image-recognition tasks, the great success of convolutional neural networks is largely

due to the fact that convolutional layers can effectively exploit the spatial correlation of images, greatly reducing the number of parameters and the computational complexity of the model.

In addition, convolution endows a network with translation equivariance, that is, the input and output transform in the same way. Suppose the image is $f(x, y)$ and g is a translation transformation: $g(f(x, y)) = f(x - a, y - b)$. Let conv be the convolution operation with stride 1; then

$$\text{conv}(g(f(x, y))) = g(\text{conv}(f(x, y))). \tag{3.14}$$

This equation shows that, if we first apply a translation to the image and then perform convolution, the result is the same as if we first performed the convolution and then applied the translation. This property stems from the weight-sharing mechanism of convolution: the convolution kernel slides over the image and extracts local features at different positions in the same way, so no matter where an object appears in the image, it can be effectively captured.

This translation equivariance enables convolutional neural networks to excel at tasks such as image recognition. A traditional fully connected network needs to learn a separate weight for each pixel position, which not only requires a huge number of parameters but also makes it hard to generalize to new positions. In contrast, a convolutional neural network, through weight sharing, greatly reduces the number of parameters and improves learning efficiency; at the same time, thanks to translation equivariance, the model can adapt well to changes in the position of objects, achieving stronger generalization.

In summary, convolution endows neural networks with important prior knowledge—local connectivity, weight sharing, translation equivariance, and so on—enabling them to efficiently learn visual patterns and achieve great success on image-related tasks. This prior knowledge alleviates the data-learning burden, speeds up the training process and improves the generalization performance of the model.

Pooling

For very high-resolution images, even after the convolution operation, the output may still be too large for the computer to handle efficiently. To improve computational efficiency, we can perform appropriate downsampling, and here we introduce a new operation: pooling. Pooling can be divided into max pooling and average pooling: max pooling extracts the maximum value in the selected region as the output, while average pooling takes the average of all values in the region as the output. As shown in Figure 3.15, the input is a 4×4 image, which is divided into four 2×2 blocks. Max pooling takes the maximum value of each block as the output, while average pooling takes the mean of each block as the output. Both pooling methods yield a 2×2 output.

For the purposes of computation, during back-propagation gradients are allocated through the pooling layer according to the following rules:

- For max pooling, we record the position of the maximum-value point, and during back-propagation the entire gradient is passed to that position; that is, gradient descent is performed only at that position while the others remain unchanged.
- For average pooling, during back-propagation the gradient is evenly distributed across every unit to perform gradient descent.

Figure 3.16 illustrates this process in detail. For max pooling (left), the maximum-value point in the figure is at the top-right corner, so the back-propagated gradient is entirely used to update

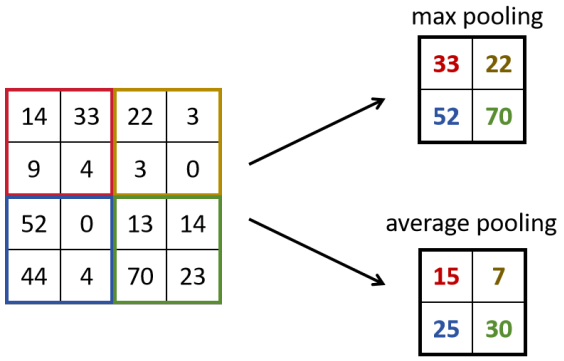


Figure 3.15: Illustration of the pooling operation.

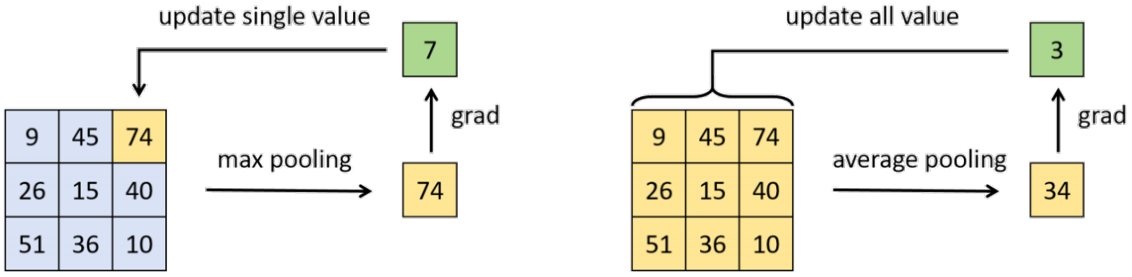


Figure 3.16: Gradient propagation through pooling operations.

the parameter at the top-right corner. For average pooling (right), each unit receives one-ninth of the back-propagated gradient.

The overall architecture of a convolutional neural network

With the two core operations of convolution and pooling, we can build a complete convolutional neural network. The overall architecture of a CNN can be viewed as an ordered combination of convolutional layers, pooling layers and fully connected layers, together with some nonlinear activation functions (such as ReLU) and normalization operations (such as Softmax).

Let us illustrate the workflow of a CNN with a concrete example, as shown in Figure 3.17. Suppose our input is an RGB colour image with three channels: red, green and blue.

First, we use three convolution kernels to perform convolution on each of the three input channels, then sum the results and add a bias term, obtaining one output channel of the first convolutional layer. Next, we use another set of three convolution kernels to convolve with the original input, obtaining the second output channel of the first convolutional layer. By analogy, we eventually obtain a multi-channel output of the first convolutional layer.

Then, this multi-channel output is passed through a nonlinear activation function (such as ReLU) element-wise, and is then fed into the first pooling layer. The pooling operation does not change the number of output channels but reduces the size of each channel. Suppose the output of

the first pooling layer has n channels.

Next, we use m groups of convolution kernels (each group containing n kernels) to convolve with the output of the first pooling layer, obtaining the output of the second convolutional layer, with a total of m channels. This output is again passed through ReLU and the second pooling layer, yielding a feature map with m channels of even smaller size.

We repeat this "convolution–ReLU–pooling" pattern multiple times, obtaining a series of feature maps whose sizes gradually shrink while the number of channels gradually grows. This hierarchical structure allows a CNN to gradually extract image features—from low-level edges and textures to high-level object parts and scenes—forming a feature hierarchy that proceeds from local to global and from simple to complex. At the same time, the introduction of convolution and pooling operations greatly reduces the number of parameters and the computational complexity, enabling CNNs to handle large-sized, high-resolution image inputs.

Finally, we "flatten" the output of the last pooling layer into a one-dimensional vector and feed it into one or more fully connected layers. The fully connected layer can be viewed as a traditional fully connected network that performs nonlinear combinations of the features extracted by the convolutional layers, producing a more compact and abstract representation.

At the output end of the CNN, we usually use a Softmax layer to convert the output of the fully connected layer into a probability distribution. This distribution represents the probability that the input image belongs to each class, with the class of the highest probability being the prediction of the CNN.

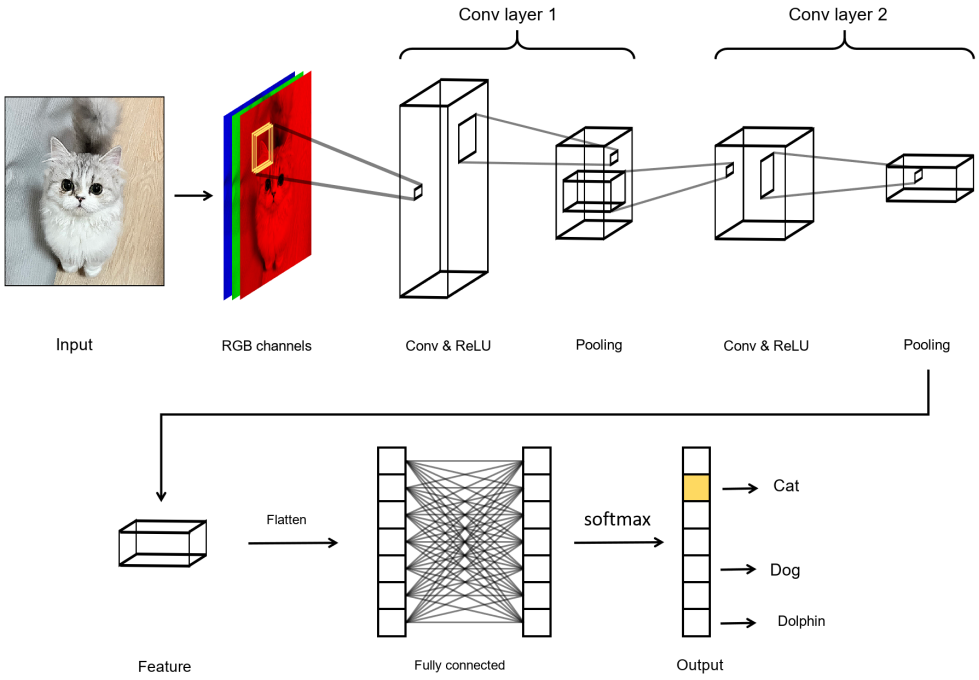


Figure 3.17: The architecture of a convolutional neural network.

The success of CNNs demonstrates the effectiveness of structures such as local connectivity, weight sharing and pooling-based downsampling. They not only greatly reduce the complexity of the network but also align well with the characteristics of the visual system. One may say that CNNs are a perfect combination of deep learning and computer vision, opening a new era of artificial intelligence and giving machines the ability to "see". With the continued development of CNNs, we have reason to believe that computer vision will play an important role in more and more fields and may eventually reach or even surpass the human level.

3.4 Language tasks and the main paradigms of natural language processing

In daily life, we are almost constantly in contact with and using language: reading the news, writing emails, conversing with others, and so on. However, behind seemingly effortless and natural language communication lies a highly complex structural and semantic system. To allow computers to understand and generate language, we must not only recognize the surface form of these symbols, but also deeply grasp the underlying grammatical rules, semantic relations and contextual logic. This places very high demands on a model's expressive power and reasoning ability.

In order to systematically understand the technical road map of natural language processing, we need to discuss the topic from two perspectives: on the one hand, what structural characteristics language itself has and why they pose challenges to computational models; on the other hand, how modern deep learning models have gradually established the ability to understand language. This section will start from the essence of language tasks and gradually unfold the discussion together with typical modelling paradigms.

3.4.1 Characteristics of language tasks

Natural language processing is an important research direction in artificial intelligence. Its goal is to enable computers to understand, generate and process human language. Compared with continuous data such as images and audio, language, as a symbolic system, has unique structural properties and modelling challenges. We describe the main characteristics of language tasks along several key dimensions:

Complex hierarchical structure. Language units have a natural hierarchy: morphemes form words, words form phrases, phrases combine to form sentences, which further form paragraphs and discourses. Each level follows specific combination rules. For instance, "unbelievable" can be decomposed into "un-" (a negative prefix), "believe" (the root) and "-able" (an adjective suffix); its meaning is jointly determined by these parts.

Strict grammatical rules. A complete sentence must contain a subject and a predicate, and many sentences also include objects, adverbials and other components. Changes in word order and sentence type can significantly change the meaning, so modelling needs to pay attention to the structure and order of components.

Recursive structures. Language allows clauses to be nested, forming recursive structures. For example, in "I believe he worked late yesterday", "he worked late yesterday" is itself a complete clause used as the object of the main clause. Similar recursion can, in principle, be nested without bound, reflecting the generative power of language.

Discrete symbolic sequences. Unlike pixel values in images, language is composed of discrete words or characters. These symbols come from a finite vocabulary, but the number of possible combinations is almost unbounded, which leads to a combinatorial explosion for language models.

Long-range dependencies. Words in a sentence may have semantic or grammatical relations across long distances. For example, in "If you come early today, we will be able to finish the task together", "if" and "then" (here implicit in the English version) form a conditional dependency. Modelling such dependencies challenges the memory ability of the model.

Strong context dependence. The same word may have different meanings in different contexts. For example, "traffic" in "I love Shanghai Jiao Tong University" (where "Jiaotong" literally means "transportation/traffic") refers to the name of the university, while in "I don't like the traffic in Shanghai" it refers to the actual traffic conditions in the city.

Limitations of traditional models. Early neural networks such as fully connected networks concatenated all word vectors into a fixed-length representation before processing, which cannot preserve word-order information and is not good at capturing local or long-range relations between words. In addition, the lengths of language sequences are not fixed, and forcing them into fixed-length representations also causes information loss. As a result, traditional architectures struggle to handle language modelling tasks.

3.4.2 An introduction to deep learning models for processing language

Modern natural language processing (NLP) models usually follow a pipeline "from characters to probabilities": first the text is split into the smallest processable units, then it is gradually converted into vectors, and finally a neural network produces a probability distribution over the next token. We now describe the five stages of this process in turn:

Tokenization

Natural language is essentially an unstructured sequence of characters with no inherent boundaries. To make it amenable to model processing, the raw text needs to be split into discrete, semantically meaningful basic units—tokens. Common tokenization methods include:

- **Rule-based tokenization**: e.g. splitting by spaces and punctuation. Simple and intuitive, but limited when dealing with complex linguistic phenomena such as abbreviations and proper names.
- **Character-based tokenization**: treats each character directly as a token; often used for processing Chinese characters or for character-level modelling.
- **Dictionary-based tokenization**: uses a fixed vocabulary to look up word boundaries; suitable for specific tasks but with limited generalization.
- **Subword-based tokenization**: statistical methods such as BPE (Byte-Pair Encoding) or WordPiece, which build a vocabulary from frequent subwords. They strike a good balance between vocabulary size and coverage, and are now the mainstream practice.

For example, the sentence

```
Transformer is a powerful tool for nlp.
```

is split, with space-based tokenization, into

[Transformer, is, a, powerful, tool, for, nlp, .]

while with BPE it might be split into

[Trans, former, is, a, power, ful, tool, for, n, l, p, .]

Such subword representations can effectively reduce the vocabulary size and the risk of out-of-vocabulary (OOV) words, which is especially important for large models.

Mapping

Each token needs to be mapped to an integer `id` $\in [0, V)$, where V is the vocabulary size. The vocabulary is usually sorted by frequency, and some special tokens are reserved for use by the model, such as:

- `<pad>`: used to pad samples of different lengths.
- `<unk>`: represents out-of-vocabulary words.
- `<bos>` / `<eos>`: marks the beginning and end of a sentence.
- `[CLS]`: represents the entire sentence, often used for classification tasks.
- `[SEP]`: separates different sentences or marks the end of a sentence.
- `[MASK]`: used in masked-language-modelling tasks.

These markers provide structural hints to the model; their embedding vectors are learned together with other word vectors during training, and are thus endowed with concrete semantics by the model.

Encoding

The most direct encoding method is one-hot encoding: represent a token with a V-dimensional vector, with the `id`-th coordinate equal to 1 and all others equal to 0. But because of its sparsity and high dimensionality, the computational cost is large and it cannot express semantic similarities between words.

To address this, an embedding matrix $E \in \mathbb{R}^{V \times d}$ is introduced to project the one-hot vector into a low-dimensional dense space:

$$\mathbf{x} = E^\top \mathbf{o} \in \mathbb{R}^d.$$

Here $\mathbf{o}$ is the one-hot encoding and $\mathbf{x}$ is the corresponding embedding vector. This vector is optimized together with the network parameters by gradient descent during training, thereby learning a semantic representation of the language.

Adding special tokens and building the sequence

For training or generation tasks, we add necessary structural markers to the input sequence:

- Prepend `<bos>` or `[CLS]` at the beginning of the sequence, as the start marker or sentence representation.
- Append `<eos>` or `[SEP]` at the end of the sequence, marking the end or separating sentences.

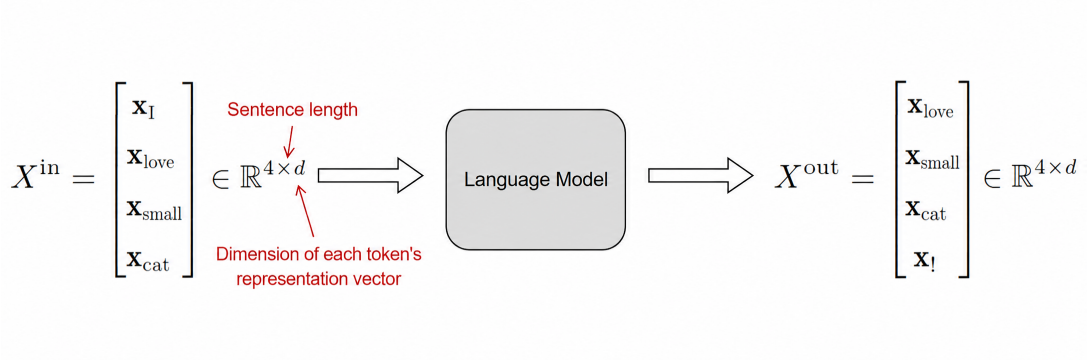


Figure 3.18: Illustration of the next-token prediction paradigm of a language model.

- If the input lengths differ, pad with `<pad>`.
- If unknown words appear, replace them with `<unk>`.
- In masked language models, replace some tokens with `[MASK]` for the model to predict.

After these preprocessing steps, we obtain a sequence of embeddings $\mathbf{x}_{1:T}$ of uniform length, ready to be fed into a neural network model such as an RNN, an LSTM or a Transformer.

Next-Token Prediction

Language models take as their core objective "given the preceding context $t_{1:T}$, predict the next token t_{next}". The network outputs logits $\mathbf{z} \in \mathbb{R}^V$, which are then normalized by the Softmax function into a probability distribution

$$P(t_i \mid t_{1:T}) = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}, \qquad i = 1, \ldots, V.$$

During training, we maximize the log-likelihood of the true next token; during inference, we can directly take the arg max, or sample according to the probabilities to generate coherent text.

To better understand the five stages of natural language processing above, let us look at an example of next-token prediction. Suppose we want to predict what should fill the "[ ]" position in "I love kittens [ ]". Figure 3.18 shows this process, and we now decompose it in detail:

1. **Tokenization result**

   ["I", "love", "little", "cat"]

2. **Word → index mapping (example)**

   "I":100, "love":205, "little":310, "cat":402

3. **One-hot encoding (assuming vocabulary size $V = 10000$)**

$$\begin{aligned}
\text{"I"}: &\ [\ldots, 1, \ldots, 0, \ldots, 0, \ldots, 0, \ldots] && \text{(position } 100 = 1) \\
\text{"love"}: &\ [\ldots, 0, \ldots, 1, \ldots, 0, \ldots, 0, \ldots] && \text{(position } 205 = 1) \\
\text{"little"}: &\ [\ldots, 0, \ldots, 0, \ldots, 1, \ldots, 0, \ldots] && \text{(position } 310 = 1) \\
\text{"cat"}: &\ [\ldots, 0, \ldots, 0, \ldots, 0, \ldots, 1, \ldots] && \text{(position } 402 = 1)
\end{aligned}$$

4. **Word-embedding vectors (embedding dimension $d=50$)**

$$E_{100} = [0.11,\, 0.07,\, \ldots,\, 0.54],\ E_{205} = [0.23,\, 0.18,\, \ldots,\, 0.41],$$
$$E_{310} = [0.05,\, 0.32,\, \ldots,\, 0.29],\ E_{402} = [0.19,\, 0.27,\, \ldots,\, 0.38]$$

5. **Input matrix received by the model**

$$X = \begin{bmatrix} 0.11 & 0.07 & \ldots & 0.54 \\ 0.23 & 0.18 & \ldots & 0.41 \\ 0.05 & 0.32 & \ldots & 0.29 \\ 0.19 & 0.27 & \ldots & 0.38 \end{bmatrix} \in \mathbb{R}^{4\times 50}$$

6. **Model output (Softmax over the entire vocabulary; only Top-3 are shown)**

$$p(t \mid \text{“I love little cat”}) = \begin{cases} 0.87, & t = \text{“!”} \\ 0.08, & t = \text{“.”} \\ 0.05, & t = \text{“,”} \\ 0, & \text{other tokens} \end{cases}$$

7. **Final prediction**

$$\arg\max_t p(t) = \text{“!”} \quad \Longrightarrow \quad \boxed{\texttt{I love little cat!}}$$

We see that, by learning contextual information, a language model can fairly accurately predict the missing word in a sentence. Recursively repeating this process allows it to generate coherent text. Next-token prediction is one of the main pre-training tasks in modern natural language processing, and famous models such as GPT are trained in this way.

3.5 Recurrent neural networks

In order to address the challenges posed by the characteristics of language, researchers proposed the recurrent neural network (RNN). By introducing recurrent connections, an RNN can build state memory inside the neural network and, when computing the output at each time step, combine information from the current input and the previous state, naturally capturing and exploiting the dynamic behaviour and a certain degree of long-range dependency present in sequence data, thereby exhibiting excellent sequence-modelling ability.

3.5.1 The basic unit of a recurrent neural network

As shown in Figure 3.19, this is a basic recurrent neural network unit. Below we describe in detail how this unit works. Our input is a time series $x_1, x_2, \cdots, x_{t-1}, x_t, x_{t+1}, \cdots, x_n$, and the corresponding output sequence $o_1, o_2, \cdots, o_{t-1}, o_t, o_{t+1}, \cdots, o_n$ is computed via Eqs. (3.15), (3.16) and (3.17).

$$S_0 = 0, \tag{3.15}$$
$$S_t = f(\mathbf{U} \cdot x_t + \mathbf{W} \cdot S_{t-1} + b), \quad t = 1, 2, 3, \cdots, n, \tag{3.16}$$
$$o_t = g(\mathbf{V} \cdot S_t) \tag{3.17}$$

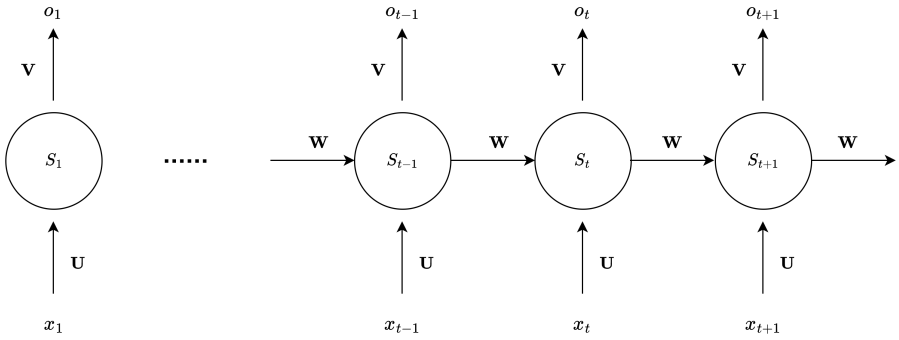


Figure 3.19: The basic structure of a recurrent neural network.

where f is the activation function, usually the Tanh function, and $g = \text{Softmax}(x)$ is the softmax function. S_t is the state memory, and $\mathbf{U}$, $\mathbf{W}$, $\mathbf{V}$ and b are respectively the input weight matrix, the recurrent weight matrix, the output weight matrix and the bias term.

For a multi-layer recurrent neural network, suppose there are L layers. The hidden state and output of each layer are fed as the input to the next layer. The specific mathematical expressions are as follows:

$$S_t^{(1)} = f(\mathbf{U}^{(1)} \cdot x_t + \mathbf{W}^{(1)} \cdot S_{t-1}^{(1)} + b^{(1)}), \quad t = 1, 2, 3, \cdots, n, \tag{3.18}$$

$$o_t^{(1)} = g(\mathbf{V}^{(1)} \cdot S_t^{(1)}), \tag{3.19}$$

The input of the l-th layer is the output of the previous layer $o_1^{(l-1)}, o_2^{(l-1)}, \cdots, o_n^{(l-1)}$, and the hidden state and output of this layer are computed as

$$S_t^{(l)} = f(\mathbf{U}^{(l)} \cdot o_t^{(l-1)} + \mathbf{W}^{(l)} \cdot S_{t-1}^{(l)} + b^{(l)}), \quad t = 1, 2, 3, \cdots, n, \tag{3.20}$$

$$o_t^{(l)} = g(\mathbf{V}^{(l)} \cdot S_t^{(l)}), \tag{3.21}$$

where $S_0^{(l)} = 0$, $l = 1, \cdots, L$. The output sequence of the last layer is the final output $o_1^{(L)}, o_2^{(L)}, \cdots, o_n^{(L)}$.

The above is the most basic architecture of a recurrent neural network. We see that the lengths of its input and output sequences are the same; such a structure is suitable for tasks where the input and output have the same length, for example frame-by-frame classification of video frames or time-series forecasting.

3.5.2 Encoder–Decoder recurrent neural networks

The architecture of the recurrent neural network introduced above has input and output of equal length, which entails many limitations. In general, for tasks such as machine translation where the input and output have different lengths, a more widely used variant is the encoder–decoder structure:

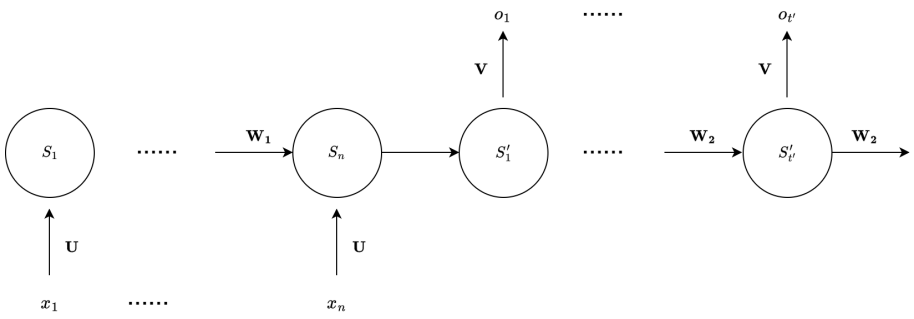


Figure 3.20: An encoder–decoder recurrent neural network.

$$S_0 = 0, \tag{3.22}$$
$$S_t = f_1(\mathbf{U} \cdot x_t + \mathbf{W}_1 \cdot S_{t-1} + b_1), t = 1, 2, 3, \cdots, n, \tag{3.23}$$
$$S'_0 = S_n \tag{3.24}$$
$$S'_t = f_2(\mathbf{W}_2 \cdot S'_{t-1} + b_2), \quad t = 1, 2, 3, \cdots, n', \tag{3.25}$$
$$o_t = g(\mathbf{V} \cdot S'_t) \tag{3.26}$$

Take a translation task as an example: we need to translate "I like eating apples" (in Chinese) into "I like eating apples" (in English). We encode the Chinese sentence character-by-character into $x_1, \ldots, x_6$ and the English sentence word-by-word into $o_1^*, \ldots, o_4^*$. We specify in advance a maximum output sentence length, say 7 here, so the input to our model is $x_1, x_2, x_3, x_4, x_5, x_6$ and the label is $o_1^*, o_2^*, o_3^*, o_4^*, 0, 0, 0$ (zeros are appended to the labels to reach length 7). Through the computation of the basic unit above, we obtain $o_1, o_2, o_3, o_4, o_5, o_6, o_7$ and compute its cross-entropy loss against the labels, then update the weights via back-propagation.

Remark 9. *Recurrent neural networks can also perform regression tasks; in that case the Softmax function g is not added.*

3.5.3 Training a recurrent neural network with the BPTT algorithm

Recurrent neural networks are also trained by gradient descent and back-propagation. However, due to their special form—the computation of o_t involves information from the previous $t-1$ steps—computing gradients also involves multiple time steps. We call this back-propagation through time (BPTT). Specifically, taking the basic RNN architecture as an example, given a loss function

$$L = \sum_{t=1}^{n} L_t(o_t, o_t^*),$$

BPTT first propagates the gradient of the loss L with respect to the state S_t, the output o_t and other variables at the current time step t back to every earlier time step via the chain rule, and thereby updates the weights at all time steps.

For the loss L_t at the t-th step, we can compute its gradients with respect to the weights $\mathbf{U}$, $\mathbf{W}$, $\mathbf{V}$ and the bias b in turn via the chain rule. Because S_t depends on the state S_{t-1} at the previous time step, the error at each time step has to be propagated back through time.

Specifically, the gradients are computed as follows:

$$\frac{\partial L_t}{\partial \mathbf{V}} = \frac{\partial L_t}{\partial o_t} \cdot \frac{\partial o_t}{\partial \mathbf{V}} = \delta_t \cdot S_t^T$$

$$\frac{\partial L_t}{\partial S_t} = \delta_t \cdot \mathbf{V}$$

$$\frac{\partial L_t}{\partial \mathbf{W}} = \sum_{k=1}^{t} \left(\frac{\partial L_t}{\partial S_t} \cdot \frac{\partial S_t}{\partial S_{t-k}} \right) \cdot \frac{\partial S_{t-k}}{\partial \mathbf{W}}$$

$$\frac{\partial L_t}{\partial \mathbf{U}} = \sum_{k=1}^{t} \left(\frac{\partial L_t}{\partial S_t} \cdot \frac{\partial S_t}{\partial S_{t-k}} \right) \cdot \frac{\partial S_{t-k}}{\partial \mathbf{U}}$$

where $\delta_t = \frac{\partial L_t}{\partial o_t} \cdot g'(\mathbf{V} \cdot S_t)$.

However, due to the recursive structure of recurrent neural networks, the gradient of the state S_t has to be computed through a chain that spans multiple time steps. As a result, the gradients gradually shrink or grow sharply over long time dependencies, leading to vanishing and exploding gradients, which is especially severe for longer sequences and makes the model struggle to learn long-term dependencies. Moreover, BPTT requires back-propagating gradients at every time step, and the state at each time step depends on the previous states; therefore the computational cost of each parameter update is high, and training becomes even slower when handling long sequences.

To address these problems, researchers proposed the long short-term memory network (LSTM) and the gated recurrent unit (GRU). Through gating mechanisms, they effectively alleviate the vanishing-gradient problem and can therefore better handle tasks with long-term dependencies.

3.5.4 Long short-term memory networks

The Long Short-Term Memory (LSTM) network is a special class of RNN proposed by Schmidhuber and Hochreiter in 1997, mainly to solve the vanishing- and exploding-gradient problems during training on long sequences. Compared with a vanilla recurrent neural network, an LSTM performs better on longer sequences.

When reading an article, humans do not rethink everything from scratch at every moment; the knowledge they have seen is not immediately forgotten, but is gradually consolidated. This shows that memory information is very important during human thinking and reflection. A recurrent neural network stores this memory information by passing the hidden state forward. However, human memory is not static; while reading new content, we keep updating new knowledge and forgetting old knowledge. Hence the acquisition and forgetting of memory information are important for effective human learning. As shown in Eq. (3.16), each layer of an RNN uses parameter matrices $\mathbf{U}$ and $\mathbf{W}$ to process new input information and memory information, but this structure is too simple, lacking interaction over long-range information and having no forgetting process.

Below we formally introduce the main architecture of an LSTM. As in a recurrent neural network, an LSTM stacks the same sub-module to form the entire network. However, unlike the simple module of a recurrent neural network, the LSTM sub-module (called a gated memory cell) has

a more complex structure. It consists mainly of three different gates: the input gate, the forget gate, and the output gate. In addition to the hidden state S_t shared with the recurrent neural network, an LSTM adds a memory-state channel C_t (cell state), forming a dual-path parallel flow of information. The three gates protect and exploit the memory state to supplement the hidden state. Figure 3.21 gives a schematic illustration of an LSTM; we now describe these three types of gates in detail. We agree that the σ appearing in this section denotes the sigmoid function.

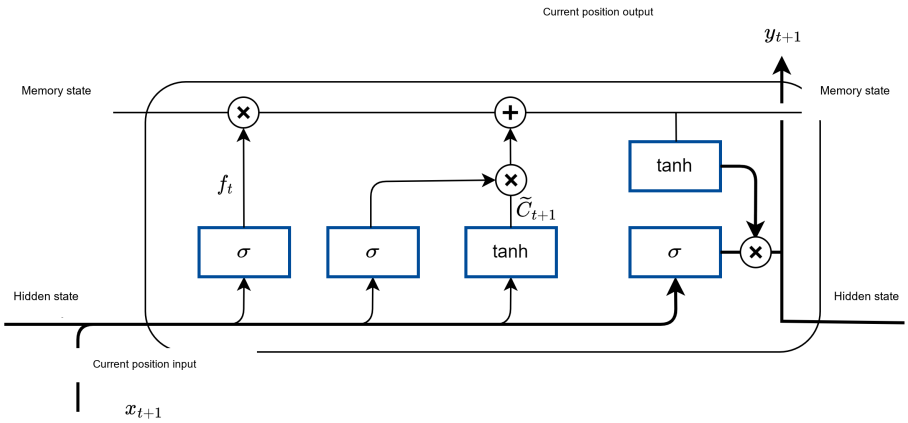


Figure 3.21: Schematic diagram of a single gated memory neuron.

Gated memory neuron: the forget gate

At the first step when information is fed into the LSTM, the network needs to decide which memories in C_t should be forgotten. The forget gate is a sigmoid network that simultaneously receives information from the hidden state S_t and the current input x_{t+1}. Since the output of a sigmoid is a number between 0 and 1, its meaning is the fraction of the memory state C_t that should be retained. If we let f_t denote the retention ratio at this step, we have:

$$f_t = \sigma(W_{Sf} S_t + W_{xf} x_{t+1} + b_f). \tag{3.27}$$

Gated memory neuron: the input gate

In the second step, the gated memory neuron needs to decide which parts of the input should be memorized and to update the memory state accordingly. This step is split into two parts: the first part is the input gate, which decides which memory values to update; the second part provides a new candidate memory state $\tilde{C}_t$ that is controlled by the input gate. Finally, the entire candidate memory is added directly to the memory state. The corresponding formulas are:

$$i_t = \sigma(W_{Si} S_t + W_{xi} x_{t+1} + b_i), \tag{3.28}$$

$$\tilde{C}_{t+1} = \tanh(W_{SC} S_t + W_{xC} x_{t+1} + b_C), \tag{3.29}$$

$$C_{t+1} = f_t * C_t + i_t * \tilde{C}_{t+1}, \tag{3.30}$$

where "$*$" denotes element-wise multiplication.

Gated memory neuron: the output gate

In the last step we need to decide the current output. By now the memory state has been fully updated to its latest state. Using the hidden state of the previous step and the current input, a sigmoid layer decides which parts of the memory to output; the memory state is normalized to the range -1 to 1 via a tanh layer and then multiplied by the output of the sigmoid layer to produce the final output y_{t+1} at the current position. The corresponding formulas are:

$$o_t = \sigma(W_{So} S_t + W_{xo} x_{t+1} + b_o), \tag{3.31}$$

$$y_{t+1} = S_{t+1} = o_t * \tanh(C_{t+1}). \tag{3.32}$$

LSTM partially solves the vanishing-gradient problem of RNNs

From a mathematical viewpoint, the key to how an LSTM can solve the exploding- or vanishing-gradient problem of RNNs is that it stabilizes gradient propagation through the memory state C_t and the design of the gating mechanism.

In an RNN, gradients are back-propagated through time, and the update depends on the Jacobian matrix at each time step. Suppose the gradient of the loss function L with respect to the hidden state S_t at time t is $\frac{\partial L}{\partial S_t}$; by the chain rule, in an RNN this gradient is computed as

$$\frac{\partial L}{\partial S_t} = \frac{\partial L}{\partial S_{t+1}} \cdot \frac{\partial S_{t+1}}{\partial S_t}$$

Since S_{t+1} is obtained by passing through some activation function (such as tanh or sigmoid) and multiplying by the weight matrix, the eigenvalues of the Jacobian matrix $\frac{\partial S_{t+1}}{\partial S_t}$ may be less than 1 (causing vanishing gradients) or greater than 1 (causing exploding gradients). As the number of time steps grows, the chain-multiplied gradient decays or explodes rapidly.

LSTM introduces a memory state C_t and uses gating mechanisms to control the flow of information. According to the update rule of the memory state

$$C_{t+1} = f_t * C_t + i_t * \tilde{C}_{t+1},$$

we have

$$\frac{\partial L}{\partial C_t} = \frac{\partial L}{\partial C_{t+1}} \cdot f_t.$$

Since the output of the forget gate f_t is usually between 0 and 1, the gating mechanism keeps the gradient within a reasonable range; it does not grow or decay exponentially as in an RNN. For example, if the task requires it, the model may learn $f_t = 1$ for all t, which is similar to a residual network. This constant error flow lets gradients decay more slowly as they are propagated through many time steps, thereby alleviating the vanishing-gradient problem.

3.6 Transformer

In the previous sections, we mentioned that RNNs have a number of limitations. Although many innovative algorithms have subsequently been developed within the RNN framework, none of them can fully overcome the drawbacks of RNNs or the inherent difficulties of language problems. Against

this background, the Transformer emerged. The birth of the Transformer has greatly advanced natural language processing; its most representative opening work is the paper "Attention is All You Need" Vaswani et al. (2017). This paper first proposed the Transformer model, which abandons the RNN paradigm and relies entirely on the attention mechanism to model sequences. The attention mechanism was first introduced by Bahdanau et al. Bahdanau et al. (2014) in the context of machine translation, and was later widely applied to various natural language processing tasks. Building on this, Lin Zhouhan et al. Lin et al. (2017) further proposed structured self-attention, allowing any two positions in a sequence to interact directly, no matter how far apart they are. The Transformer model is precisely built upon this structured self-attention. Such a structure naturally has the ability to capture long-range dependencies. At the same time, self-attention computation is highly parallelizable, which greatly speeds up training.

The original Transformer is composed of several encoders and decoders. The encoder extracts features from the input sequence and converts it into a fixed-length internal representation; the decoder generates the target (predicted) content based on the encoder's internal representation and the already known content. Each encoder and decoder uses self-attention and feed-forward neural networks. It should be noted that when computing self-attention, the encoder does not use a mask, so it can see global information and causal relations across the entire input sequence. This global information is encoded into context vectors and passed to the decoder. The decoder then predicts the next output word based on the context vectors and the previously generated output sequence. The encoder captures the global semantics of the input sequence, while the decoder uses this semantic information to generate the output; the two cooperate to perform end-to-end sequence transformation. This encoder–decoder architecture of the Transformer was initially mainly applied to machine translation, where the input is a source-language sequence (original text) and the output is a target-language sequence (translation), which are asymmetric, fitting naturally with the encoder–decoder design philosophy.

The input and output lengths of a Transformer are variable and do not need to be fixed. Taking machine translation as an example, the source-language sentence and the target-language sentence usually have different lengths, and the Transformer can handle this well. This is thanks to its unique positional-encoding mechanism, which assigns a unique vector to each position so that the model can distinguish between words at different positions; even if the input length changes, the representational ability is not affected.

However, with further research, it became clear that the potential of the Transformer goes far beyond machine translation. In many other natural language processing tasks, such as language modelling, text summarization and dialogue generation, the input and output are not entirely asymmetric. Take a language model as an example: its goal is to predict the next word given the preceding words, and the input and output are of the same kind of language. Researchers therefore proposed a Decoder-Only variant of the Transformer. In this architecture, the model has only the decoder, with no encoder.

Most existing large language models, such as the GPT series, are based on the Decoder-Only architecture shown in Figure 3.22. In such a Decoder-Only architecture, the input of the model is first mapped via the embedding layer into the embedding space, yielding the embedding representation. These embeddings are then fed into a Transformer composed of multiple decoder layers. During the computation, residual connections and layer normalization are also widely used to improve training stability. Finally, the Transformer maps the output of the last decoder layer back to the vocabulary space, producing a probability distribution over the output sequence. Through this entirely new structural design, the Transformer architecture exhibits excellent parallelism and

long-range modelling ability, achieving breakthrough progress on many sequence-related tasks and pushing forward the development of artificial intelligence. It is said that, after switching to the Transformer architecture, OpenAI made more progress in two weeks than it had made in two years of RNN-based research. Some theoretical work has also pointed out that Transformers have a strong fitting ability for sequence-form data (Wang et al., 2024) and do not face the memory catastrophe of RNNs when dealing with long-memory tasks.

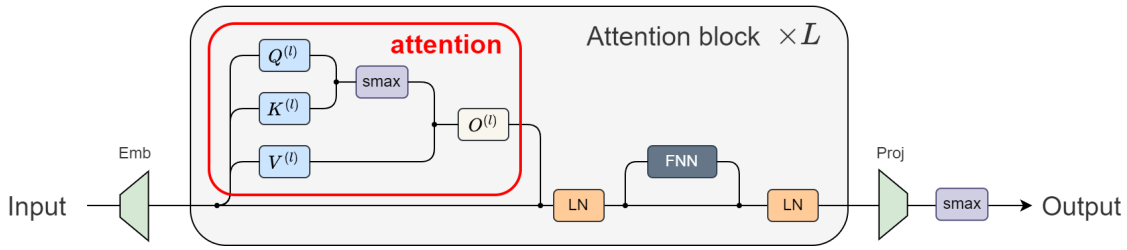


Figure 3.22: The attention module in a Transformer model. This figure also shows the "Post-LayerNorm" architecture, i.e. where layer normalization occurs after the other main modules.

The proposal of the Transformer is a milestone in natural language processing. It laid the foundation for subsequent language models such as BERT (Devlin et al., 2018) and GPT (Radford et al., 2018). After pretraining on large-scale corpora, these models acquire rich linguistic knowledge and exhibit astonishing performance on downstream tasks, sometimes even surpassing humans. One may say that the Transformer led a revolution in natural language processing, greatly enhancing the ability of machines to understand and generate language.

In this section, taking a Decoder-Only model as an example, we will describe, module by module, how data is processed and computed inside the Transformer.

3.6.1 Basic principles of the Transformer

Before delving into the internal structure of the Transformer model, let us first look at its basic principles. The Transformer model is essentially an autoregressive language model whose basic task is to predict the $(n+1)$-th word given the previous n words; this is also known as next-token prediction.

Specifically, suppose we have an input sequence of length n, $X^{\text{in}} = (\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_n)^\mathsf{T} \in \mathbb{R}^{n\times d}$, where $\boldsymbol{x}_i \in \mathbb{R}^d$ is the d-dimensional one-hot representation of the i-th word and d is the vocabulary size; this vector is 1 only at the position corresponding to the word in the dictionary and 0 elsewhere. The goal of the Transformer model is to predict the next word $\boldsymbol{x}_{n+1}$ based on X^{in}.

Remark 10. *In a sense, a one-hot vector can also be understood as a probability distribution that outputs the corresponding word in the dictionary with probability* 100% *and other words with probability* 0.

The model's output is also an $n \times d$ matrix $X^{\text{out}} \in \mathbb{R}^{n\times d}$, whose i-th row X_i^{out} is intended to predict the $(i+1)$-th word in the input. For a well-trained model, we should have $X_i^{\text{out}} \approx \boldsymbol{x}_{i+1}$. We can convert X_i^{out} into a probability distribution over the vocabulary via a linear transformation followed by a softmax, and then select the word with the highest probability as the prediction for position $i+1$.

Returning to our previous example, suppose the input sequence is "I love kittens"; the corresponding input matrix is

$$X^{\text{in}} = \begin{bmatrix} \mathbf{x}_{\text{I}} \\ \mathbf{x}_{\text{love}} \\ \mathbf{x}_{\text{little}} \\ \mathbf{x}_{\text{cat}} \end{bmatrix} \in \mathbb{R}^{4\times d}.$$

Then for next-token prediction we expect the output of the Transformer model to be

$$X^{\text{out}} = \begin{bmatrix} \mathbf{x}_{\text{love}} \\ \mathbf{x}_{\text{little}} \\ \mathbf{x}_{\text{cat}} \\ \mathbf{x}_{!} \end{bmatrix} \in \mathbb{R}^{4\times d},$$

where $\mathbf{x}_{!}$ indicates that the model predicts the next word to be the exclamation mark "!".

During inference, the Transformer model performs next-token prediction based on the input information: each time it generates a new token, it merges this token with the input information and feeds the result into the model for the next prediction. This continues until a vector $\mathbf{x}_{\text{EOS}}$ corresponding to the termination symbol is generated. The collection of outputs obtained is then the inference result of the Transformer model. After we have introduced the specific architecture of the Transformer, we will give a more concrete example in Section 3.6.6 to detail the inference process of the Transformer.

During training, the Transformer model computes the cross-entropy loss at each position of the entire sentence separately, and then takes the average as the final loss function. Specifically, suppose the target output sequence corresponding to the input $X^{\text{in}} = (\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_n)^{\mathsf{T}}$ is $X^{\text{out}} = (\boldsymbol{x}_2, \boldsymbol{x}_3, ..., \boldsymbol{x}_{n+1})^{\mathsf{T}}$ and the model's predicted output is $\hat{X}^{\text{out}} = (\hat{\boldsymbol{x}}_2, \hat{\boldsymbol{x}}_3, ..., \hat{\boldsymbol{x}}_{n+1})^{\mathsf{T}}$, where $\hat{\boldsymbol{x}}_{i+1}$ is the predicted probability distribution at the i-th position. Then the cross-entropy loss over the entire sequence can be written as

$$\mathcal{L}(X, \hat{X}) = -\frac{1}{n}\sum_{i=2}^{n+1}\sum_{j=1}^{d} \boldsymbol{x}_{ij} \log \hat{\boldsymbol{x}}_{ij},$$

where $\boldsymbol{x}_{ij}$ is the j-th element of $\boldsymbol{x}_i$ (0 or 1) and $\hat{\boldsymbol{x}}_{ij}$ is the j-th element of $\hat{\boldsymbol{x}}_i$ (the predicted probability).

The goal of the model is to minimize this loss, so that the predicted results are as close as possible to the true results. This way of computing the loss separately at every position of the entire sequence more accurately measures the model's prediction quality at each position and provides finer-grained supervision signals.

Remark 11. *The idea of computing cross-entropy loss separately at each position in a Transformer can be viewed as performing multiple next-token predictions on a single corpus. For example, the sentence "I love little cat!" can be regarded as ("I") predicts "love"; ("I", "love") predicts "little"; ("I", "love", "little") predicts "cat"; ("I", "love", "little", "cat") predicts "!". Thus a single corpus is used multiple times, greatly improving training efficiency and corpus utilization.*

The above outlines the basic framework of the Transformer model. In the following subsections, we will describe the internal structure of the Transformer in detail and see how it carries out the next-token prediction task.

3.6.2 Embedding

When processing language tasks, we need to represent each word in a sentence (sequence) by a vector. The simplest representation is a length-d one-hot vector, where d is the dictionary size and only the position corresponding to that word is 1, with all other positions equal to 0. This representation is simple and intuitive, but the vectors are mutually orthogonal, and the Euclidean distance between any two of them is $\sqrt{2}$, so it cannot describe the similarities between words. Moreover, the matrix of all word vectors in this representation is $d \times d$, which is hard to store and compute. To allow the model to better understand and exploit the semantic information among words, we can use a linear transformation to map these word vectors into a continuous, low-dimensional vector space; this is the role of the embedding layer.

Specifically, we first represent the input sequence as a one-hot matrix $X^{\text{in}} \in \mathbb{R}^{n \times d}$, where n is the sequence length. To overcome the limitations of the one-hot representation, we introduce an embedding layer. The embedding layer contains a learnable embedding matrix $W^{\text{em}} \in \mathbb{R}^{d \times d_m}$, where d_m is the dimension of the embedding vectors, usually much smaller than the dictionary size d. Using the embedding matrix, we can map each one-hot vector into a dense embedding vector:

$$X^{\text{em}} = X^{\text{in}} W^{\text{em}} \in \mathbb{R}^{n \times d_m}.$$

During training, the embedding matrix W^{em} is randomly initialized and learned jointly with the rest of the model. Through learning, words with similar meanings are mapped to nearby points in the embedding space, while words with different meanings are mapped to distant points. This continuous, high-dimensional representation can characterize semantic relations among words very well, providing better features for the subsequent learning of the model.

Since the Transformer model itself has no ability to record positional information about the sequence, we also need to explicitly encode a position vector for each position: $X^{\text{pos}} \in \mathbb{R}^{n \times d_m}$. The purpose of positional encoding is to introduce a unique, meaningful representation for each position, helping the model distinguish words at different positions. A simple and effective positional encoding is given by sine and cosine functions:

$$X^{\text{pos}}[k, 2i] = \sin\left(\frac{k}{10000^{2i/d_m}}\right),$$
$$X^{\text{pos}}[k, 2i+1] = \cos\left(\frac{k}{10000^{2i/d_m}}\right),$$

where k denotes the position (ranging over $[1, n]$) and i denotes the dimension (ranging over $[0, d_m/2-1]$). This positional encoding satisfies the property that $X^{\text{pos}}[k] \cdot X^{\text{pos}}[k+dk]$ depends only on dk and not on the position k, which helps the model learn relative positional relations. Moreover, this encoding constrains the values within $[-1, 1]$, which helps generalization to sequences of various complexities.

Finally, we add the embedding representation and the positional encoding together to form the input of the Transformer block:

$$X^{(1)} = X^{\text{em}} + X^{\text{pos}} \in \mathbb{R}^{n \times d_m}.$$

This summation can be viewed as fusing word information and positional information. After this processing, the original word sequence is transformed into a continuous vector sequence carrying both semantic and positional information, ready for the subsequent self-attention computation and feature extraction.

Remark 12. *There are other ways to fuse the embedding representation and the positional encoding, such as directly concatenating them. Current mainstream large models still use the additive form, which saves hidden-space dimensions.*

Remark 13. *In recent years, Rotary Position Embedding (RoPE) (Su et al., 2024) has become a commonly used positional encoding for large models. By introducing 2D rotation matrices, RoPE preserves relative positional relations while allowing the model to capture positional information more effectively (compared with the sine/cosine encoding mentioned in the main text), thereby enhancing the ability to handle sequence tasks. At the same time, many new positional-encoding schemes are continually being proposed.*

3.6.3 Attention layer

Single-head attention mechanism

In the Transformer model, the self-attention mechanism plays a central role. It allows the model, when processing each position, to take into account information from all other positions in the sequence, thereby capturing long-range dependencies between words.

The computation of self-attention can be divided into three steps: the first step is to compute the query, key and value matrices; the second step is to compute the attention-score matrix; the third step is to compute the attention output. We discuss them one by one below.

First step: for the input $X^{(l)} \in \mathbb{R}^{n\times d_m}$ of the l-th layer, we use three different learnable matrices $W^{Q(l)}, W^{K(l)}, W^{V(l)}$ to transform it into the query matrix $Q^{(l)}$, key matrix $K^{(l)}$ and value matrix $V^{(l)}$:

$$\begin{aligned} Q^{(l)} &= X^{(l)}W^{Q(l)} \in \mathbb{R}^{n\times d_q}, \quad W^{Q(l)} \in \mathbb{R}^{d_m\times d_q}, \\ K^{(l)} &= X^{(l)}W^{K(l)} \in \mathbb{R}^{n\times d_k}, \quad W^{K(l)} \in \mathbb{R}^{d_m\times d_k}, \\ V^{(l)} &= X^{(l)}W^{V(l)} \in \mathbb{R}^{n\times d_v}, \quad W^{V(l)} \in \mathbb{R}^{d_m\times d_v}. \end{aligned}$$

Intuitively, the query matrix $Q^{(l)}$ encodes the queries made by the current position to the other positions, the key matrix $K^{(l)}$ encodes the key features at each position, and the value matrix $V^{(l)}$ represents the actual information at each position. Through such transformations, we map the original word vectors into three different semantic spaces, preparing for the subsequent attention computation.

Remark 14. *For computational convenience, we usually set $d_q = d_k = d_v$.*

Remark 15. *In neural networks, we usually do not use a piece of data directly; instead, we first apply a linear transformation. This operation is very common, and its purpose is to extract features.*

Second step: based on the query matrix and the key matrix, we compute the attention-score matrix $\mathrm{Attn}^{(l)} \in \mathbb{R}^{n\times n}$:

$$\mathrm{Attn}^{(l)} = \mathrm{softmax}\left(\frac{\mathrm{mask}(Q^{(l)}(K^{(l)})^T)}{\sqrt{d_k}}\right) \in \mathbb{R}^{n\times n}.$$

Each row of the attention-score matrix represents the attention distribution of the current position to all other positions. Specifically, $\mathrm{Attn}_{ij}^{(l)}$ is the attention score from the i-th position to the j-th

position; it is obtained by computing the dot product (inner product) of $Q_i^{(l)}$ and $K_j^{(l)}$ and dividing by $\sqrt{d_k}$. Dividing by $\sqrt{d_k}$ here is to alleviate the problem of too-large variance in the dot-product results, which helps training.

It should be noted that, in the Decoder, in order to preserve the autoregressive property, we usually apply an upper-triangular mask to the attention-score matrix. This mask sets the values at positions (i, j) with $i < j$ in the upper triangle to negative infinity, so that after the softmax these attention scores become 0. This masking operation ensures that, when generating each word, the model can only see the current word and those on the left, and cannot see those on the right, thereby preserving the autoregressive property of the language model.

The softmax function is applied row-wise, that is, for a matrix $A = \{a_{ij}\}_{n\times m}$,

$$\text{softmax}(A)_{ij} = \frac{\mathrm{e}^{a_{ij}}}{\sum_{j=1}^{m} \mathrm{e}^{a_{ij}}}, \tag{3.33}$$

whose effect is to turn a set of real numbers into a probability distribution. It guarantees that the sum of the attention scores in each row is 1 and that each score lies in $[0, 1]$. This gives the attention scores a probabilistic meaning, representing the importance of the other positions to the current position.

Third step: based on the attention-score matrix and the value matrix, we compute the attention output $X^{\text{qkv}(l)}$:

$$X^{\text{qkv}(l)} = \text{Attn}^{(l)} V^{(l)} \in \mathbb{R}^{n\times d_v}.$$

Each row of the attention output is a weighted average of the rows of the value matrix, with the attention scores as weights. Intuitively, the model uses the attention scores to selectively focus on different positions in the sequence and extract relevant information from these positions to form a new representation of the current position.

To ensure that the dimensions of the attention output match those of the input, we use another learnable matrix $W^{O(l)}$ to map $X^{\text{qkv}(l)}$ back to d_m dimensions:

$$X^{\text{pr}(l)} = X^{\text{qkv}(l)} W^{O(l)} \in \mathbb{R}^{n\times d_m}, \quad W^{O(l)} \in \mathbb{R}^{d_v \times d_m}.$$

Finally, we add the output of the attention layer $X^{\text{pr}(l)}$ to the input $X^{(l)}$ and normalize:

$$X^{\text{ao}(l)} = \text{LayerNorm}(X^{(l)} + X^{\text{pr}(l)}) \in \mathbb{R}^{n\times d_m}.$$

where

$$[\text{LayerNorm}(X)]_{i,j} = \gamma * \frac{X_{i,j} - \sum_l X_{i,l}/d_m}{\text{std}(X_{i,1}, \cdots, X_{i,d_m})} + \beta.$$

γ and β are two trainable parameters. The residual connection and layer normalization (LN) here are two important components of the Transformer. The residual connection helps the network more easily learn the identity mapping and alleviates the vanishing-gradient problem in deep networks; layer normalization helps stabilize training and speed up convergence. By normalizing the d_m-dimensional output at each position of each layer to have mean 0 and variance 1, it alleviates the covariate-shift problem between different layers.

Multi-head attention mechanism

Although single-head attention can already capture dependencies in a sequence, it still has some limitations. Intuitively, single-head attention can be viewed as examining the sequence from one perspective, which may not fully capture its internal structure.

To overcome this issue, the Transformer introduces the multi-head attention mechanism. The basic idea of multi-head attention is to learn multiple groups of different query, key and value matrices simultaneously, compute multiple attention functions in parallel, and finally concatenate the outputs of the different heads. Formally, for the i-th head of the l-th layer, we have:

$$\begin{aligned}
Q_i^{(l)} &= X^{(l)} W_i^{Q(l)} \in \mathbb{R}^{n \times d_q}, \quad W_i^{Q(l)} \in \mathbb{R}^{d_m \times d_q}, \\
K_i^{(l)} &= X^{(l)} W_i^{K(l)} \in \mathbb{R}^{n \times d_k}, \quad W_i^{K(l)} \in \mathbb{R}^{d_m \times d_k}, \\
V_i^{(l)} &= X^{(l)} W_i^{V(l)} \in \mathbb{R}^{n \times d_v}, \quad W_i^{V(l)} \in \mathbb{R}^{d_m \times d_v}.
\end{aligned}$$

The attention-score matrix and the output of each head are computed similarly to single-head attention:

$$\begin{aligned}
\mathrm{Attn}_i^{(l)} &= \mathrm{softmax}\left(\frac{Q_i^{(l)} (K_i^{(l)})^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{n \times n}, \\
\mathrm{head}_i^{(l)} &= \mathrm{Attn}_i^{(l)} V_i^{(l)} \in \mathbb{R}^{n \times d_v}.
\end{aligned}$$

We then concatenate the outputs of all heads and apply a linear transformation to obtain the multi-head attention output:

$$X^{\mathrm{pr}(l)} = \mathrm{Concat}(\mathrm{head}_1^{(l)}, ..., \mathrm{head}_h^{(l)}) W^{O(l)} \in \mathbb{R}^{n \times d_m}, \quad W^{O(l)} \in \mathbb{R}^{h d_v \times d_m}$$

where h is the number of heads and Concat denotes column-wise concatenation. Finally, similar to single-head attention, we apply residual connection and layer normalization to the multi-head attention output:

$$X^{\mathrm{ao}(l)} = \mathrm{LayerNorm}(X^{(l)} + X^{\mathrm{pr}(l)}).$$

The multi-head attention mechanism allows the model to learn different representations of the sequence in different subspaces. Intuitively, each head can be viewed as understanding the sequence from a different angle and capturing different dependency patterns. For instance, some heads may focus on short-distance grammatical relations, while others may focus on long-range semantic relations. By combining these different representations, the model can obtain a more comprehensive and robust understanding of the sequence.

Another advantage of multi-head attention is that it increases the expressive power of the model without increasing the computational complexity. If we put all parameters into a single head, the expressive power of the model is limited. By distributing parameters across multiple heads, the model can learn richer representations, and multiple heads can run in parallel to improve computational efficiency.

Practice shows that multi-head attention significantly improves the performance of the Transformer on various tasks. It has become a standard component of the Transformer model and is widely used in all areas of natural language processing.

3.6.4 Feed-forward network layer

Apart from the self-attention layer, another important component of the Transformer model is the feed-forward network (FFN) layer. The FFN layer further nonlinearly transforms the output of the self-attention layer, enhancing the model's expressive power. Structurally, the FFN layer is essentially the two-layer fully connected network we introduced earlier, with a ReLU activation in the middle, followed by a residual connection and layer normalization at the end. Specifically, for the input $X^{\mathrm{ao}(l)}$ of the l-th layer, the FFN layer is computed as

$$X^{\mathrm{do}(l)} := \mathrm{FNN}(X^{\mathrm{ao}(l)}) = \mathrm{LayerNorm}\left(X^{\mathrm{ao}(l)} + \sigma(X^{\mathrm{ao}(l)}W^{l,1})W^{l,2}\right) \in \mathbb{R}^{n\times d_m},$$

where $\sigma(\cdot)$ denotes the ReLU activation, and $W^{l,1} \in \mathbb{R}^{d_m\times d_{ff}}$ and $W^{l,2} \in \mathbb{R}^{d_{ff}\times d_m}$ are two learnable weight matrices. Here d_{ff} is usually larger than d_m, for example $d_{ff} = 4d_m$. In essence, the data first passes through a two-layer fully connected network and then through a residual connection followed by layer normalization.

It should be noted that this feed-forward network is applied to each position separately: the n d_m-dimensional vectors are passed through the feed-forward network independently and then reassembled into an $n \times d_m$ sentence matrix. This structure increases the nonlinearity of the model. Since the self-attention layer is essentially a linear transformation (ignoring the nonlinearity of the softmax), introducing the FFN layer enhances the nonlinear expressive power of the model. Moreover, the FFN layer places no requirements on the sequence-length dimension n, so it can be conveniently inserted at various points in the model and can handle sentences of different lengths, exhibiting good compatibility.

Although the FFN layer is structurally simple, it plays an important role in the Transformer. Together with the self-attention layer, it forms the basic computational unit of the Transformer. The $n \times d_m$ matrix obtained after the FFN serves as the input of the self-attention layer in the next computational unit; that is,

$$X^{(l+1)} = X^{\mathrm{do}(l)} \in \mathbb{R}^{n\times d_m}. \tag{3.34}$$

3.6.5 Output layer

At the end of the Transformer model, we need to convert the output of the last layer $X^{\mathrm{do}(L)}$ into a concrete prediction. This typically involves two steps: the first step maps $X^{\mathrm{do}(L)}$ to the dimensionality of the target vocabulary; the second step chooses the most likely output based on the result of this mapping.

The mapping in the first step is done by a linear transformation, which we call the projection layer:

$$X^{\mathrm{out}} = X^{\mathrm{do}(L)}W^{\mathrm{proj}} + b^{\mathrm{proj}} \in \mathbb{R}^{n\times d}, \quad W^{\mathrm{proj}} \in \mathbb{R}^{d_m\times d}, \quad b^{\mathrm{proj}} \in \mathbb{R}^{d},$$

where W^{proj} and b^{proj} are the projection matrix and bias vector. This linear transformation maps the d_m-dimensional hidden state into a d-dimensional output space, where each dimension corresponds to a word in the vocabulary.

After obtaining the projected result X^{out}, we need to choose the most likely output word according to X^{out}. There are usually two approaches: greedy decoding and sampling decoding.

The idea of greedy decoding is, at each position, to select the word with the highest probability as the output:

$$\mathrm{Output} = \mathrm{argmax}(\mathrm{softmax}(X^{\mathrm{out}})).$$

Here argmax returns the index of the maximum, applied along the vocabulary dimension; it returns the index of the word with the highest probability. Greedy decoding is a deterministic decoding strategy: it always selects the locally optimal result, but does not guarantee a globally optimal one.

Different from greedy decoding, sampling decoding is a stochastic decoding strategy. In sampling decoding, we first turn X^{out} into a probability distribution via the softmax function and then sample a word according to this distribution:

$$\text{Output} \sim \text{softmax}(X^{\text{out}}).$$

Sampling decoding allows the model to generate diverse results, but may also generate unreasonable ones. In practice, we usually introduce some techniques such as temperature and top-k sampling to balance the diversity and quality of the outputs.

The design of the output layer is closely related to the type of task and the evaluation metrics. For instance, in machine translation we typically use beam search to find a globally optimal translation; beam search maintains multiple candidate results when generating sequences, instead of just a single optimal path. In dialogue generation, we may pay more attention to the diversity and naturalness of the generated results. Therefore, the choice of the output layer needs to be balanced according to the specific task and requirements.

3.6.6 Detailed inference process of a Transformer

In the previous subsections, we have introduced in detail the internal structure and working principle of the Transformer model. Now let us use a concrete example to see how the Transformer model makes predictions.

Suppose we have a trained Transformer model and we want to use it to answer the question: "My name is Mike, what is my name?". After tokenization, this question can be represented as

$$[\text{“[CLS]”}, \text{“My”}, \text{“name is”}, \text{“Mike”}, \text{“I”}, \text{“am called”}, \text{“what”}, \text{“[SEP]”}].$$

We can generate the answer through the following steps:

First step: convert the tokenized question into an input format that the model can handle. Suppose, after encoding, the input matrix corresponding to this question is

$$X_0 = \begin{bmatrix} \mathbf{x}_{\text{[CLS]}} \\ \mathbf{x}_{\text{My}} \\ \mathbf{x}_{\text{name is}} \\ \mathbf{x}_{\text{Mike}} \\ \mathbf{x}_{\text{I}} \\ \mathbf{x}_{\text{am called}} \\ \mathbf{x}_{\text{what}} \\ \mathbf{x}_{\text{[SEP]}} \end{bmatrix} \in \mathbb{R}^{8\times d}.$$

Second step: we feed X_0 into the Transformer model. After many layers of computation, we obtain the output matrix X_0^{out}. We turn the last row of X_0^{out} into a probability distribution via the softmax, and choose the token "Your" with the highest probability as the first generated token.

Third step: we append the embedding $\mathbf{x}_{\text{Your}}$ of "Your" to the end of X_0, forming a new input matrix:

$$X_1 = \begin{bmatrix} \mathbf{x}_{[\text{CLS}]} \\ \vdots \\ \mathbf{x}_{[\text{SEP}]} \\ \mathbf{x}_{\text{Your}} \end{bmatrix} \in \mathbb{R}^{9\times d}.$$

Remark 16. *Note that X_1 here and the output $X^{(1)}$ of the first layer mentioned earlier are two completely different concepts. The former is the $9 \times d$ matrix obtained by appending the output word embedding $\mathbf{x}_{Your}$, produced by passing the input sentence X_0 all the way through the Transformer model, to the end of X_0. The latter is the output obtained by passing an input matrix X through the first Decoder layer of the Transformer. For example, if the input matrix is $X = X_0$, then $X^{(1)} \in \mathbb{R}^{8\times d_m}$.*

We then feed X_1 into the Transformer model again, obtaining a new output matrix X_1^{out}. We select the token "name is" corresponding to the last row of X_1^{out} as the second generated token.

Fourth step: we repeat the third step, appending the embedding $\mathbf{x}_{\text{name is}}$ of "name is" to the end of X_1 to form a new input matrix X_2, and feed it into the Transformer model to obtain X_2^{out}. We select the token "Mike" corresponding to the last row of X_2^{out} as the third generated token.

Fifth step: we continue to repeat the third step, appending the embedding of "Mike" to the end of X_2 to form X_3, and feed it into the Transformer model to obtain X_3^{out}. We find that the last row of X_3^{out} corresponds to the special end-of-sentence symbol "[SEP]", indicating that generation has finished.

At this point we have obtained the complete answer:

$$[\text{“Your”}, \text{“name is”}, \text{“Mike”}, \text{“[SEP]”}].$$

We see that the Transformer model continually feeds previously generated tokens back into the input, generating the complete answer step by step. This process is autoregressive: at every step, the model predicts the next token based on its previous outputs, until the end symbol is produced. This example illustrates the inference process of the Transformer model in practice. Of course, in practice, we usually use more efficient decoding strategies such as Beam Search to obtain better generation results. But regardless of which decoding strategy is used, the basic idea is the same: continuously feed the previous outputs back into the input to implement an autoregressive generation process. This is also an important feature of the Transformer model: it can dynamically adjust its prediction at each step based on the preceding context. This gives the Transformer model strong contextual understanding and generation ability, allowing it to handle a wide variety of complex natural-language-processing tasks.

The above describes the overall structure and workflow of the Transformer model. From the input word embeddings, through the alternation of self-attention layers and feed-forward layers, to the final output layer, these components together build a powerful language-sequence processing model. The design of the Transformer model fully takes into account the characteristics of natural language, such as long-range dependencies and semantic complexity, while also accounting for computational efficiency and scalability. Of course, the Transformer model is not perfect. Its huge model size and computational cost place very high demands on hardware and energy; its black-box nature also poses challenges for model interpretation and debugging. Furthermore, applying the Transformer model to broader scenarios such as multimodal learning and reinforcement learning is also a worthwhile direction. Nonetheless, the Transformer model has become the mainstream

paradigm in natural language processing today, and its influence continues to grow. In the future, as our understanding of language deepens and AI technology continues to advance, the Transformer model will undoubtedly shine in more fields, opening the door to the era of intelligence.

3.7 Generative models

Generative models are mainly used for generation tasks. Given a batch of real data $\{\boldsymbol{x}_i\}_{i=1}^N$ whose underlying distribution μ we do not know, a generative model models this unknown distribution as follows: it randomly samples a latent variable $\boldsymbol{z}$ from a known distribution and then maps it through a decoder $\boldsymbol{g}$ to generate a sample $\boldsymbol{x}$, so that $\boldsymbol{x}$ also follows the distribution μ. There are many types of generative models; this is not the focus of this book, and we will give a brief introduction using the variational autoencoder as an example.

3.7.1 Autoencoders

We would like the images to be continuously interpolated, i.e. when image x_1 and image x_2 are close (some spatial norm of $x_1 - x_2$ is small), the corresponding latents z_1 and z_2 are also sufficiently close (some spatial norm of $z_1 - z_2$ is small); otherwise, the map from z to x would be a high-frequency map, and by the frequency principle $\boldsymbol{g}$ would be very hard to fit. Hence, we may assume that z is a continuous map of x, which guarantees that $\boldsymbol{g}$ is not a high-frequency map. The encoder $\boldsymbol{f}$ is precisely the tool that ensures this. A structure made up directly of an encoder and a decoder is called an autoencoder.

- Encoder $\boldsymbol{f}$: compresses high-dimensional input data $\boldsymbol{x} \in \mathrm{R}^n$ into a lower-dimensional latent variable $\boldsymbol{z} \in \mathrm{R}^d$ with $d \ll n$.
- Decoder $\boldsymbol{g}$: reconstructs the low-dimensional latent variable $\boldsymbol{z}$ back into the original high-dimensional input form $\boldsymbol{x}$.

An autoencoder is a special type of neural network. It belongs to the class of unsupervised learning algorithms. Its main goal is to learn, through training, an efficient representation (encoding) of the input data and then to reconstruct an output (decoding) that is as close as possible to the original input. This network consists of two main parts: the encoder and the decoder.

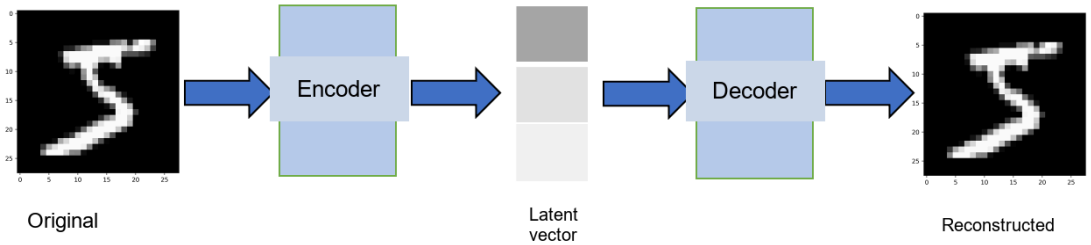


Figure 3.23: Schematic diagram of an autoencoder.

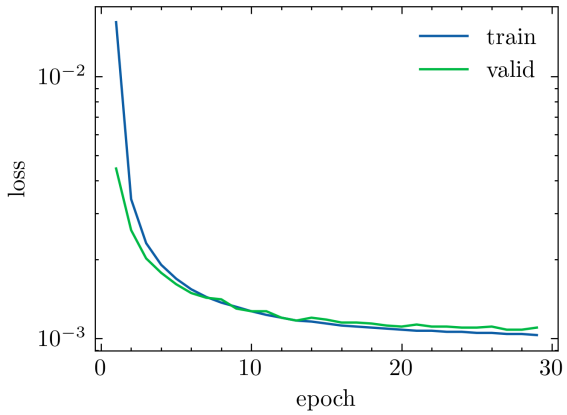


Figure 3.24: Loss curve of the autoencoder during training.

By parameterizing the encoder $\boldsymbol{f}$ and the decoder $\boldsymbol{g}$ with neural networks $\boldsymbol{f}_\theta$ and $\boldsymbol{g}_\theta$ respectively, and minimizing the following loss:

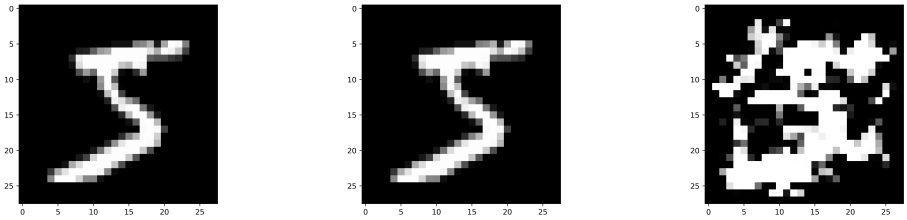
$$L = \frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{x}_i - \boldsymbol{g}_\theta(\boldsymbol{f}_\theta(\boldsymbol{x}_i)\|^2, \tag{3.35}$$

where x_i is the input data. The final goal is to compress the intrinsic structure and patterns of high-dimensional data into a low-dimensional latent space via $\boldsymbol{f}_\theta$, and to perform a generation task via the decoder $\boldsymbol{g}_\theta$ (given a latent variable $\boldsymbol{z}$, the decoder returns a high-dimensional datum $\boldsymbol{g}_\theta(\boldsymbol{z})$).

3.7.2 Variational autoencoders

Although the idea of the autoencoder is very appealing, in practice the data we have is usually extremely sparse relative to the actual space in which it lives. As a result, although autoencoders perform well at compressing and reconstructing data, they may not be ideal at generating new data. The main reason is that the autoencoder cannot effectively fill the gaps in the data space, so the generated content often does not meet expectations. To illustrate this more intuitively, we trained an autoencoder on the Fashion-MNIST dataset. The training loss curve is shown in Figure 3.24; the reconstruction quality on both the training and validation sets is fairly good. Figure 3.25(a) shows an image of the handwritten digit 5 in Fashion-MNIST, while Figure 3.25(b) is the reconstruction by the autoencoder. However, if we add Gaussian noise with mean 0 and standard deviation 5 to the latent vector obtained by passing Figure 3.25(a) through the encoder, the latent vector no longer lies in the original data distribution, and the reconstructed image is no longer a handwritten digit or a meaningful picture.

To address the above problem, we can switch our perspective: instead of simply mapping an image to a single "numerical encoding", we map it to a "distribution". Because this is a "distribution" map, we denote the encoder and decoder by $q_\theta(\boldsymbol{z}|\boldsymbol{x})$ and $p_\theta(\boldsymbol{z}|\boldsymbol{x})$ respectively. For the encoder, the natural forward process turns an image into a latent variable; for the decoder, the natural forward process is the reverse: turning a latent variable into an image. Therefore, in the Variational Autoencoder (VAE) paper Kingma (2013), the following three assumptions are made:



(a) Original image (b) Reconstruction by the autoencoder (c) Perturbed reconstruction

Figure 3.25: Generation and reconstruction performance of an autoencoder.

- Encoder $q_\theta(\boldsymbol{z}|\boldsymbol{x})$: assumed to be a Gaussian distribution, where the mean μ_θ and the variance σ_θ are predicted by the network, and $\boldsymbol{z}$ is sampled from $\mathcal{N}(\mu_\theta, \sigma_\theta^2)$.
- Decoder $p_\theta(\boldsymbol{x}|\boldsymbol{z})$: assumed to be a Gaussian distribution.
- Prior distribution: $p(\boldsymbol{z})$ is a standard Gaussian distribution.

Let us recall our goal: we want to randomly sample a sample from the standard normal distribution $p(\boldsymbol{z})$ and pass it through the decoder $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ to obtain a meaningful image, while the encoder is a necessary tool for training the decoder. The key question is: how can we simultaneously train $p_\theta(\boldsymbol{z}|\boldsymbol{x})$ and $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ so that the decoder $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ has the generative capability we want? To answer this, we need to consider the following two aspects:

- The core training objective is to maximize the log-likelihood $\log(p_\theta)$, but clearly this probability cannot be obtained directly from our three assumptions. By the conditional probability formula, we have

  $$p_\theta(\boldsymbol{x}) = \frac{p_\theta(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_\theta(\boldsymbol{z}|\boldsymbol{x})},$$

  but since $p_\theta(\boldsymbol{z}|\boldsymbol{x})$ is unknown, we cannot directly compute this integral. Therefore, in the VAE we construct a lower bound to approximate this log-likelihood. The idea of the construction is to arrange terms in the form of an entropy or relative entropy, since these forms are always non-negative and can be used to bound things from above or below. The derivation is as follows:

  $$\log p_\theta(\boldsymbol{x}) = \log \frac{p_\theta(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_\theta(\boldsymbol{z}|\boldsymbol{x})} \tag{3.36}$$

  Note that $p_\theta(\boldsymbol{z}|\boldsymbol{x})$ is unknown, while only $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ and $q_\theta(\boldsymbol{z}|\boldsymbol{x})$ are known. Therefore, we use $q_\theta(\boldsymbol{z}|\boldsymbol{x})$ to construct a quantity that is always non-negative. We can consider constructing a relative entropy between $q_\theta(\boldsymbol{z}|\boldsymbol{x})$ and $p_\theta(\boldsymbol{z}|\boldsymbol{x})$.

$$
\begin{aligned}
\log p_\theta(\boldsymbol{x}) &= \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})} \log p_\theta(\boldsymbol{x}) \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})} \log \frac{p_\theta(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_\theta(\boldsymbol{z}|\boldsymbol{x})} \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})} \log \frac{p_\theta(\boldsymbol{x}|\boldsymbol{z})}{q_\theta(\boldsymbol{z}|\boldsymbol{x})} \frac{q_\theta(\boldsymbol{z}|\boldsymbol{x})}{p_\theta(\boldsymbol{z}|\boldsymbol{x})} p(\boldsymbol{z}) \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log \frac{p_\theta(\boldsymbol{x}|\boldsymbol{z})}{q_\theta(\boldsymbol{z}|\boldsymbol{x})}] + \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log \frac{q_\theta(\boldsymbol{z}|\boldsymbol{x})}{p_\theta(\boldsymbol{z}|\boldsymbol{x})}] + \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log p(\boldsymbol{z})] \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log \frac{p_\theta(\boldsymbol{x}|\boldsymbol{z})}{q_\theta(\boldsymbol{z}|\boldsymbol{x})}] + KL[q_\theta(\boldsymbol{z}|\boldsymbol{x})|p_\theta(\boldsymbol{z}|\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log p(\boldsymbol{z})] \\
&\geq \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log \frac{p_\theta(\boldsymbol{x}|\boldsymbol{z})}{q_\theta(\boldsymbol{z}|\boldsymbol{x})}] + \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log p(\boldsymbol{z})],
\end{aligned}
$$

By maximizing this lower bound, we can indirectly optimize the objective function and train the decoder so as to improve its generative capability. We therefore obtain the final loss function,

$$
\begin{aligned}
L &= \mathbb{E}_{\boldsymbol{x}} \left[-\mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log \frac{p_\theta(\boldsymbol{x}|\boldsymbol{z})}{q_\theta(\boldsymbol{z}|\boldsymbol{x})}] - \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})}[\log p(\boldsymbol{z})] \right] \\
&= \mathbb{E}_{\boldsymbol{x}} \left[KL(q_\theta(\boldsymbol{z}|\boldsymbol{x})|p(\boldsymbol{z})) - \mathbb{E}_{\boldsymbol{z}\sim q_\theta(\boldsymbol{z}|\boldsymbol{x})} p_\theta(\boldsymbol{x}|\boldsymbol{z}) \right].
\end{aligned}
\tag{3.37}
$$

Let the mean and variance of $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ be $\boldsymbol{\mu_{\theta_1}}(\boldsymbol{z}) = (\mu^1_{\theta_1}, ..., \mu^K_{\theta_1})$ and $\boldsymbol{\Sigma^2_{\theta_1}}(\boldsymbol{z}) = (\sigma^1_{\theta_1}, ..., \sigma^K_{\theta_1})$, and let the mean and variance of $q_{\theta_2}(\boldsymbol{z}|\boldsymbol{x})$ be $\boldsymbol{\mu_{\theta_2}}(\boldsymbol{z}) = (\mu^1_{\theta_2}, ..., \mu^M_{\theta_2})$ and $\boldsymbol{\Sigma^2_{\theta_2}}(\boldsymbol{x}) = (\sigma^1_{\theta_2}, ..., \sigma^M_{\theta_2})$. Then,

$$
\begin{aligned}
L \simeq & \frac{1}{N} \sum_{n=1}^{N} \sum_{m=1}^{M} \left(-1 + \sigma^m_{\theta_2}(\boldsymbol{x}_n) + \mu^m_{\theta_2}(\boldsymbol{x}_n) - \log \sigma^m_{\theta_2}(\boldsymbol{x}_n) \right) \\
& - \frac{1}{N} \sum_{n=1}^{N} \left(\sum_{j=1}^{P_n} \sum_{k=1}^{K} \frac{(\boldsymbol{x}_n - \mu^k_{\theta_1}(\boldsymbol{z}_j))^2}{\sigma^k_{\theta_1}(\boldsymbol{z}_j)} \right).
\end{aligned}
\tag{3.38}
$$

Remark 17. *In fact, an attentive reader may have noticed that the $\boldsymbol{z}$ in the loss function is sampled through the encoder $q_\theta(\boldsymbol{z}|\boldsymbol{x})$, and the process of sampling directly from this distribution is non-differentiable. To solve this problem, the VAE uses the reparameterization trick. Specifically, we first sample a noise η from a standard normal distribution and then transform it via $\boldsymbol{z} = \mu_\theta + \sigma_\theta \eta$, so that by the chain rule $\frac{\partial}{\partial \theta} = \frac{\partial}{\partial z}(\frac{\partial \mu_\theta}{\partial \theta} + \eta \frac{\partial \sigma_\theta}{\partial \theta})$.*

Chapter 4

Frequency Principle

In the preceding chapters, we introduced some basic knowledge of deep learning, and also pointed out the challenge of understanding a complex system such as a neural network. For this reason, finding a suitable entry point from which to analyze neural networks is especially important for dissecting the inner mechanism of this "black box". In scientific research, exploration of complex systems often originates from observation and analysis of basic phenomena. For example, in genetics research, Mendel chose the relatively simple pea plant as his study object and, by observing the segregation and combination patterns of pea traits, summarized the basic laws of heredity. These laws were later applied to explain more complex biological inheritance phenomena. Another example: in the study of electromagnetism, scientists such as Ampère and Faraday first studied the relatively simple phenomena of electric and magnetic fields, and discovered basic laws such as the law of electromagnetic induction. On this foundation, Maxwell further unified these laws into Maxwell's equations, explained more complex electromagnetic phenomena, and predicted the existence of electromagnetic waves.

Inspired by this, when we study neural networks we may likewise start from very simple problems, observe, analyze, and summarize regularities in simple models, and then attempt to extend these insights to more complex neural networks. Of course, such extension must be made with care: not every conclusion obtained from a simple model can be directly applied to a complex one. But this research methodology can help us grasp the key aspects of a problem and provide useful ideas and inspiration for further exploration of complex networks. Based on these considerations, in our experimental exploration of neural networks we choose to start from the simplest problem of fitting a one-dimensional function. On the one hand, a one-dimensional problem has a simple, intuitive structure that makes it easy to observe and analyze a network's properties; on the other hand, many basic characteristics of neural networks—such as the nonlinearity of the learning process and the absence of obvious overfitting under standard settings—are already fully manifested in one-dimensional problems. Through the study of one-dimensional problems, we can obtain a preliminary understanding of some important features of neural networks, laying a foundation for further exploration of more complex networks. Therefore, in this section we will conduct our discussion on the basis of one-dimensional problems, illustrating to the reader through concrete examples the general patterns exhibited by neural networks during learning.

Popular Science Introduction

What is the Frequency Principle of neural networks?

The Frequency Principle (F-Principle, also known as spectral bias) of neural networks is an important discovery in deep learning in recent years. This principle describes a specific pattern exhibited by neural networks during learning: they tend to learn the low-frequency information in the data first, and then gradually move on to learning the high-frequency information. This principle offers us a new perspective for understanding the optimization process and performance of neural networks.

What is frequency?

In signal processing and data analysis, frequency refers to the rate of variation of a signal or data along the temporal or spatial dimension. From a mathematical viewpoint, low-frequency components represent slowly varying global trends or patterns, while high-frequency components correspond to rapidly varying details and local features. For example, a smooth curve mainly contains low-frequency components, whereas a waveform with sharp variations contains more high-frequency components.

What is the essence of the Frequency Principle?

The Frequency Principle reveals an important characteristic of neural networks during training: the network tends to learn and fit the low-frequency information in the data first, and then gradually captures and adapts to the high-frequency information. This process can be divided into the following stages:

- Initial stage: In the early phase of training, the neural network mainly learns the low-frequency components of the data, i.e., the overall trends and dominant patterns. At this point, the network parameters are more sensitive to macroscopic variations in the data.
- Intermediate stage: As training proceeds, the network begins to capture more detailed information, which is typically contained in the mid-frequency components. At this stage, the predictive capability of the network gradually improves, exhibiting finer feature-recognition ability.
- Late stage: In the later phase of training, the network is able to learn the high-frequency components, which may include subtle variations and noise in the data. At this point, the network exhibits sensitivity to fine patterns and outliers when handling high-frequency information.

What is the significance of the Frequency Principle?

The discovery of the Frequency Principle is of multiple importance for deep learning research and applications:

- Understanding generalization: By learning the low-frequency information first, the neural network can build up a relatively smooth base model, which helps to reduce the risk of overfitting. Overfitting refers to the phenomenon that a model performs excellently on the training data but generalizes poorly to new data.

- Deepening the understanding of learning mechanisms: The Frequency Principle provides researchers with a new angle from which to analyze the learning mechanism of neural networks, helping to improve existing training methods and model architecture designs in order to boost the efficiency of high-frequency learning.

What practical guidance does the Frequency Principle provide?

In practical applications, an understanding of the Frequency Principle can help us:

- Optimize training strategies, such as early stopping.
- Design more effective feature-extraction methods that take into account the capture of both low-frequency and high-frequency information.

4.1 Low-dimensional experiments of the Frequency Principle

In this section, we introduce, through some simple experiments, an implicit bias of neural networks during training—the Frequency Principle.

4.1.1 The "smoothness" preference of neural networks

One advantage of the one-dimensional problem is that we can directly observe the graph of the function output by the neural network (i.e., its "input–output" mapping), and thus intuitively recognize the global properties of the output function, such as smoothness and fitting error. We observed the learning results of network models of different depth, as shown in Fig. 4.1, where black points denote the training data sampled from the target function $\sin(x)$. The curves of different colors represent the function outputs of neural networks of different sizes; one of their salient features is that they form smooth interpolating curves between the training data points. This "implicit" preference of neural networks for smooth functions arouses our curiosity: why is it that, during fitting, neural networks of different sizes all tend to learn smooth functions?

To investigate how the smooth fitting at the training points produced by a neural network arises, we track the entire training process. We construct a target function that is the superposition of a smooth low-frequency function $\sin(x)$ and a rapidly oscillating high-frequency function $\sin(5x)$, and sample points from this target function to fit. As shown in Fig. 4.2, the blue dashed line denotes the target function $\sin(x) + \sin(5x)$, and the red solid line denotes the output of the neural network at training step $t = 1, 2000, 10000$. In the early stage of training (at step 2000), the curve learned by the neural network is essentially very close to the $\sin(x)$ component of the target function, while it has not yet fit the strongly oscillating part. From this we obtain the following intuitive impression of the learning process of neural networks: a neural network tends to quickly capture the overall outline of the training set first, and only after a relatively long period of training does it gradually start to fit more detailed information. At the same time, at the position where the $\sin(x)$ component has been fit well, one can see an obvious loss stagnation phenomenon in the loss plot.

In classification problems, the target function is usually discrete, taking a constant value on different ranges of the independent variable. To investigate the process by which a neural network fits this kind of function, we construct a target function shown as the red solid line in Fig. 4.3, where the blue dashed line denotes the target function and the red solid line denotes the output of the neural network at training step $t = 0, 1000, 10000$. One observes that the neural network

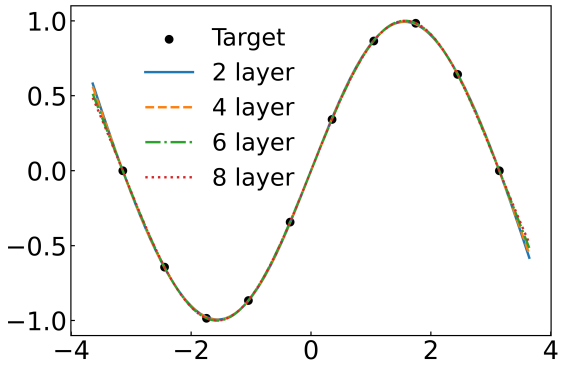
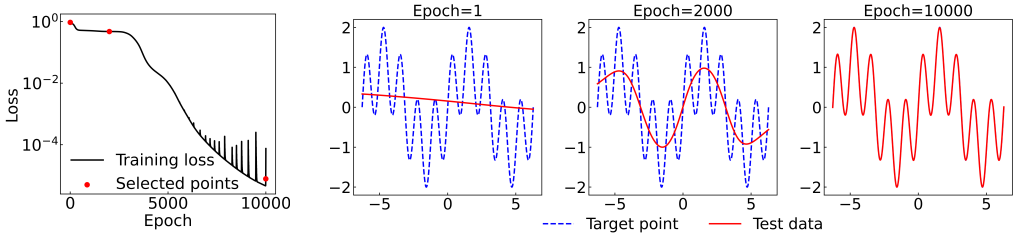


Figure 4.1: Function outputs learned by deep neural networks (DNNs) of different sizes.

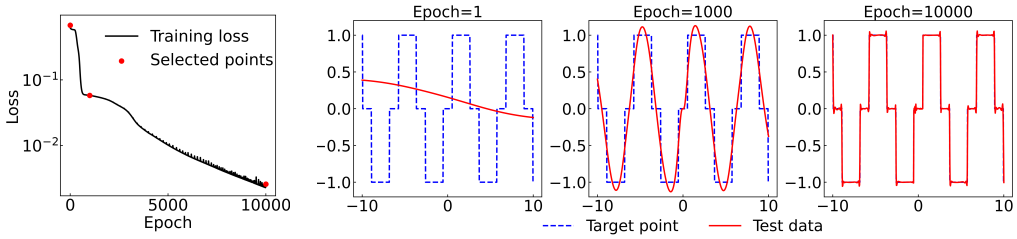


(a) loss (b) model output

Figure 4.2: Illustration of the training process of a deep neural network (DNN).

first captures the low-frequency components and obtains an output resembling a trigonometric function, and then begins to capture the high-frequency features of the target function, gradually learning the "staircase" part of the target function. This function is a periodic jump function whose output takes only three discrete values, which can be analogized to three different class labels. To simplify the problem, we did not use the common cross-entropy loss function but instead adopted the mean-squared error as the loss function, treating the problem as a regression task. In order to observe more clearly the process by which the neural network fits the (high-frequency) jumps, we used very densely sampled data points to ensure that the training data preserve the high-frequency information of the target function as much as possible. From the figure, one can see that the neural network exhibits a regular learning pattern during training. First, the network quickly learns and captures the overall outline of the data. Then, the network begins to gradually learn the detailed information of the data, slowly capturing the jumping parts. Finally, the network output gradually approaches the true target function. This experiment again demonstrates the "from global to local" learning pattern of neural networks. Although the target function is a discontinuous jump function, the network is still able to approximate it via regression. In fact, the choice of such a discontinuous

jump function was originally accidental. In retrospect, however, using such a discontinuous jump target function to observe the training process of neural networks turns out to be very beneficial: in frequency space, this function is composed of multiple harmonics of different frequencies, making it easy to observe the difference in convergence rate among different frequency components.



(a) loss (b) model output

Figure 4.3: Illustration of the training process of a deep neural network (DNN).

In two-dimensional fitting tasks, we can likewise observe the characteristic that neural networks preferentially learn the overall outline. Taking the fitting of a two-dimensional image as an example: a black-and-white photograph can in fact be viewed as a two-dimensional function whose independent variables are the pixel coordinates and whose dependent variable is the grayscale value at the corresponding position. If a neural network completely memorizes this image, we can sequentially feed each pixel coordinate into the network, obtain the output grayscale value at the corresponding position, and finally piece them together to recover the original image. For a color picture, with the three channels (red, green, blue, i.e., RGB), we can use a neural network with two-dimensional input and three-dimensional output to fit it.

Fig. 4.4 shows the training process of using a fully connected network to fit a picture, where the training points are the pixels of the odd columns of the original image, and we observe the model output on the odd columns (training data) and the even columns (test data). We find that at training step 20000 the network can only capture the rough location information of the cat in the image; when the number of steps reaches 50000, the contour information of features such as the face has been essentially recovered; only when the training step reaches 1000000 can the network capture details such as fur and achieve a relatively complete restoration of the entire image. By observing this training process, we can see that when learning a two-dimensional image, the neural network does not memorize one part first and then another, but rather learns a blurry overall outline first, and then gradually adds detailed information until eventually producing a clear and complete image. This indicates that the learning process of neural networks is not chaotic "rote memorization", but follows the principle of "global first, local later".

Furthermore, we can also use a neural network to reconstruct a three-dimensional object. For example, given a set of point cloud data, i.e., the three-dimensional coordinates and intensity values of an object, the fitting process of a neural network is similar to the above examples. It can often fit the outline of the object quickly, but it is difficult to reconstruct the details within only a few training epochs.

This "global first, local later" learning pattern of neural networks is quite similar to the way we humans recognize unfamiliar things. When we come into contact with a new object, we likewise

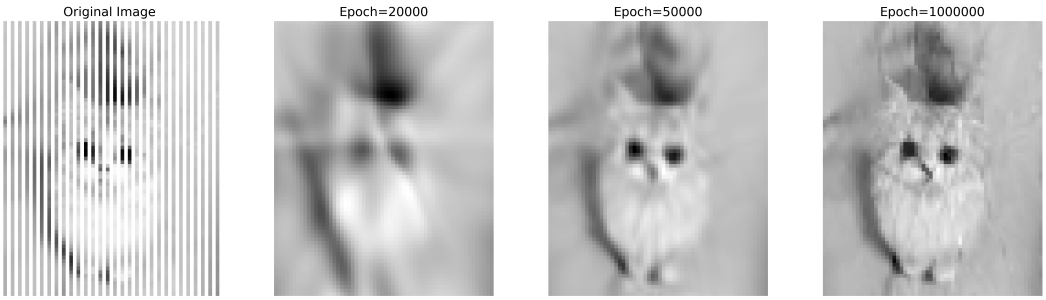


Figure 4.4: The process by which a neural network fits an image.

often grasp its rough outline first, and then gradually learn more details. This process of fitting training data from simple to complex provides us with an interesting perspective for understanding neural networks. However, to analyze this process more finely and quantitatively, we still need to transform the intuitive descriptions of "outline" and "detail" into experimentally observable quantities.

The Frequency Principle points out that neural networks learn low frequencies much faster than high frequencies. This indicates both their advantage in learning low frequencies and their difficulty in learning high frequencies, e.g., when solving differential equations or reconstructing high-resolution images, high-frequency details are often very important. For such problems, the learning of an ordinary neural network is very difficult. Inspired by the Frequency Principle, many related algorithms have been proposed to improve high-frequency learning and have been widely applied, such as PhaseDNN Cai et al. (2020), multi-scale neural networks (MscaleDNN) Liu et al. (2020), Fourier feature networks Tancik et al. (2020), neural radiance fields Mildenhall et al. (2020), and implicit neural representations Sitzmann et al. (2020), etc. Further introductions will be given in later chapters.

4.1.2 Frequency and the Fourier transform

Through the preceding analysis, we have roughly summarized the general pattern of neural network learning, namely learning the outline first and the details later. To quantitatively compare the rate at which the model learns the outline and the details, we need to introduce a quantity to characterize the degree to which the neural network captures outline and detail during learning. Mathematically, a common way to describe "outline" and "detail" is precisely frequency. Intuitively, if a function varies slowly, we call it a low-frequency function; conversely, if a function varies rapidly, we call it a high-frequency function.

To quantitatively characterize the frequency properties of a function, we typically adopt the Fourier transform. The core idea of the Fourier transform is that the vast majority of functions can be represented as superpositions of sine and cosine functions of different frequencies. Here we briefly introduce the Fourier transform (for more, please see the appendix):

Continuous Fourier Transform (CFT): Consider the transform of a function $g(x)$ from the

time-domain x space to the frequency ξ space:

$$\mathcal{F}[g(x)](\xi) = \int_{-\infty}^{\infty} g(x)\mathrm{e}^{-2\pi\mathrm{i}\xi x}\mathrm{d}x, \tag{4.1}$$

This is in fact a form of inner product. Note that for the inner product of two functions, $< u(x), v(x) >= \int u\bar{v}\mathrm{d}x$, the latter function $v(x)$ should be conjugated; therefore the Fourier transform projects $g(x)$ onto the direction of $\mathrm{e}^{2\pi\mathrm{i}\xi x}$, i.e., it decomposes onto the basis functions sin and cos (since $\mathrm{e}^{2\pi\mathrm{i}\xi x} = \cos(2\pi\xi x) + \mathrm{i}\sin(2\pi\xi x)$). If $g(x)$ is a low-frequency, flat function, then its projection components in the low-frequency directions will dominate; conversely, if $g(x)$ is a high-frequency, oscillatory function, then its projection components in the high-frequency directions will be more pronounced.

Correspondingly, we also give the formula for the inverse Fourier transform:

Continuous Inverse Fourier Transform (CIFT): Consider the transform of a function $\hat{g}(\xi)$ from the frequency ξ space to the time-domain x space:

$$\mathcal{F}^{-1}[\hat{g}(\xi)](x) = \int_{-\infty}^{\infty} \hat{g}(\xi)\mathrm{e}^{2\pi\mathrm{i}\xi x}\mathrm{d}\xi. \tag{4.2}$$

To intuitively display the effect of the Fourier transform, we take a picture of a cat as an example. By applying the Fourier transform to the image, we can decompose it into components of different frequencies, and thereby effectively distinguish the high-frequency and low-frequency information in the image. As shown in Fig. 4.5, after the Fourier transform, the low-frequency components characterize the overall outline and basic shape of the image, reflecting its global structure; the high-frequency components correspond to the detailed information in the image, such as the texture and edge features of the cat's fur and eyes. Through the Fourier transform, we can decompose a function into components of different frequencies and thereby deeply understand its properties in the frequency domain. This tool provides a new perspective for analyzing the learning behavior of neural networks.

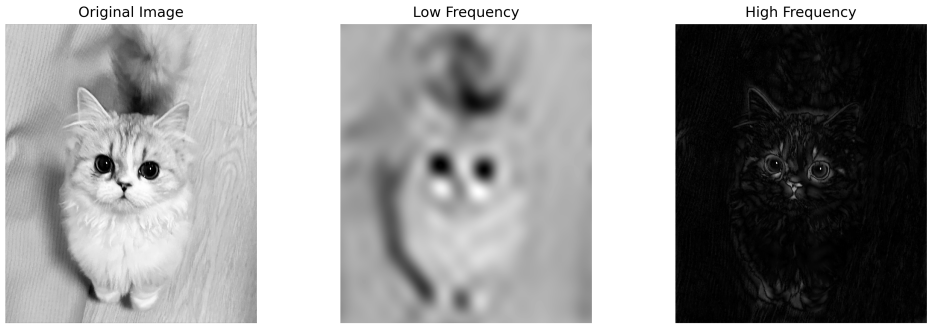


Figure 4.5: After a Fourier transform, the image is separated into different frequency components.

Below, we will further use the perspective of frequency to observe the relative difference in speed at which the model learns high-frequency and low-frequency components. By comparing the learning speed of the model on different frequencies, we can carry out quantitative experimental studies on the "smoothness" preference of neural networks during training.

4.1.3 The Frequency Principle

In the previous section, we introduced the Fourier transform, a powerful mathematical tool that enables us to quantitatively study the fitting rate of neural networks for components of different frequencies during learning. In this chapter, we will exhibit the training process of neural networks in the frequency domain. From it, we summarize an important feature of the training process, namely:

Frequency Principle: Neural networks often fit training data in the order from low to high frequencies.

As shown in Fig. 4.6, we apply the Fourier transform to the target function and the model output at the steps corresponding to those in Fig. 4.3, and observe the learning process of the model in frequency space. The blue dashed line is the frequency distribution of the target function, and the red solid line is the frequency distribution of the neural network fitting result at this moment; from left to right, the training step increases. From the results in the frequency domain, one can more intuitively see that the neural network first learns the low-frequency components of the data, and then, as the number of training steps increases, learns the high-frequency components of the data. By analyzing the transformed results, we discovered an interesting phenomenon: when a deep neural network (DNN) fits a function, it tends to learn the low-frequency components first, while the high-frequency components require a longer training time to be effectively captured. As training proceeds, the neural network gradually learns more and more high-frequency details, and finally achieves an accurate fit of the entire function. We summarize this phenomenon as the "Frequency Principle" of the neural network training process.

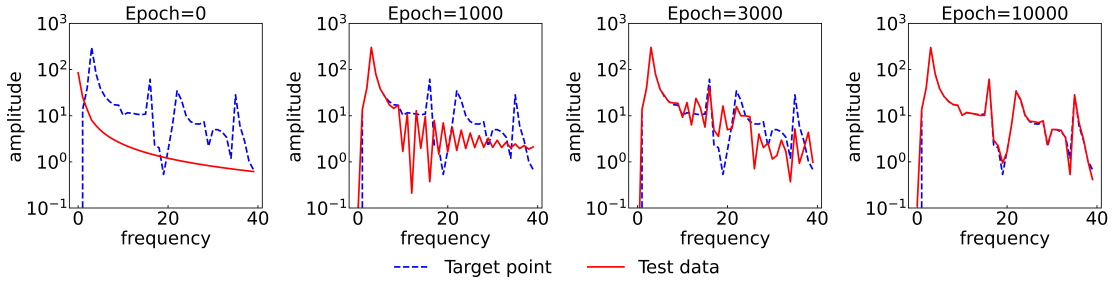


Figure 4.6: The frequency distribution of the neural network output as a function of training time.

According to the Frequency Principle, neural networks often fit training data in the order from low to high frequencies. It is worth noting that, when all frequency components of the training data are captured, the training error decreases to zero and training terminates. Therefore, the highest frequency contained in the neural network's fitting result generally does not exceed the highest frequency of the training data. This also intuitively explains why a neural network always uses a relatively smooth function to fit the training data.

However, careful readers may notice that the above experiments still have certain shortcomings, and are not sufficient to establish that the difference in training speed of different components is caused by the level of their frequencies. Specifically, in Fig. 4.6, we observe that as the frequency increases, the corresponding amplitude also decreases. Therefore, an alternative possible explanation

for the above phenomenon is that the magnitude of the amplitudes determines the training speed of the different frequency components. In order to determine which is the key factor influencing the training speed—frequency or amplitude—

We deliberately designed a controlled-variable experiment, observing the training process of the neural network when it fits a target function with components of the same amplitude but different frequencies. We choose the target function $f(x) = \sin(x)+\sin(3x)+\sin(5x)$, where the amplitude of each frequency component is 1. The experimental results are shown in Fig. 4.7; the left panel shows the target function, the middle panel shows the distribution of the target function and the model output in frequency space, and the right panel shows the evolution of the relative error of the neural network fitting result at different frequency components, $\Delta F(k) = \left|\hat{h}_k - \hat{f}_k\right| / \left|\hat{f}_k\right|$, as a function of training steps (here, $\hat{h}_k$ and $\hat{f}_k$ denote the model output and the ground truth, respectively). As shown in the middle panel, we only consider the peak positions of the frequency distribution, so as to avoid inaccuracies of the frequency amplitudes caused by numerical computation. From the figure, one can see that even with the same amplitude, the low-frequency components still exhibit a much faster convergence rate. This helps us establish that frequency is the core factor influencing the convergence rate.

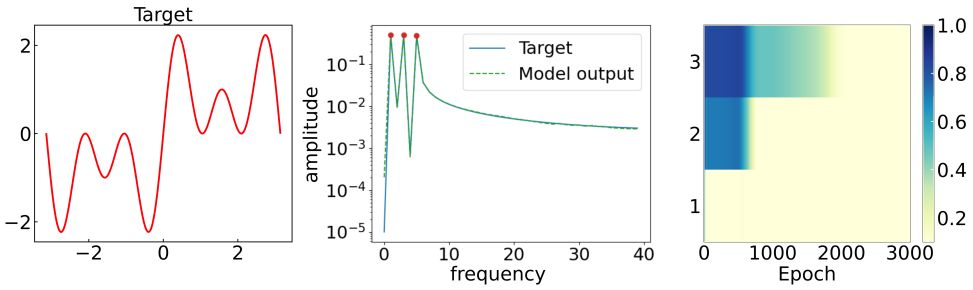


Figure 4.7: A one-dimensional data experiment controlling for the same amplitude of the target function.

The above experimental results verify that the Frequency Principle is a widely existing phenomenon. In fact, this qualitative feature of "low frequency first" is not sensitive to specific factors of the neural network such as its architecture, activation function, or loss function, and is a general law of the neural network training process. This phenomenon offers us a new perspective for understanding the training and generalization of neural networks. Below, we shall discuss this in depth.

4.2 Understanding neural networks from the Frequency Principle

It must first be pointed out that no algorithm can have good generalization on all data sets. This is precisely the fact revealed by the No Free Lunch theorem introduced earlier. Therefore, deeply understanding the learning process of a model and finding ways to make it match the requirements of the task is an important topic in machine learning research. The Frequency Principle provides us

with a new perspective from which to examine the learning behavior of neural networks. Based on the Frequency Principle, we can better understand the performance of neural networks in different tasks. In the chapters that follow, we will further explore the applications of the Frequency Principle in the optimization of neural networks, generalization theory, and the design of new models, demonstrating its important value in deep learning research.

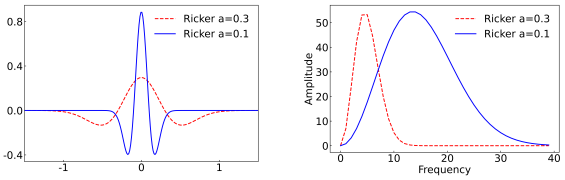
4.2.1 Experimental understanding: the necessity of the Frequency Principle

One-dimensional experiments

To intuitively understand the importance of the Frequency Principle, we consider using a fully connected neural network to fit the function $\sin(x) + \sin(3x) + \sin(5x)$. In this example, we choose the Ricker function with parameter a (as shown in Fig. 4.8) as the activation function:

$$\sigma(x) = \frac{1}{15a}\pi^{1/4}\left(1 - \left(\frac{x}{a}\right)^2\right)\exp\left(-\frac{1}{2}\left(\frac{x}{a}\right)^2\right). \tag{4.3}$$

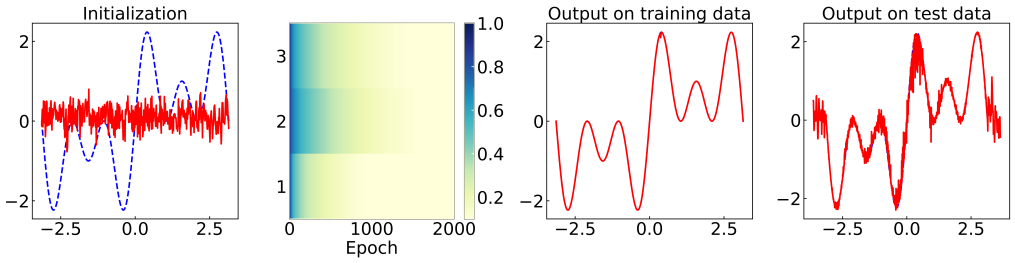
By tuning the value of the parameter a, we can control the fitting speed of the neural network for high-frequency and low-frequency components. The larger a is, the faster the network fits the low-frequency components.



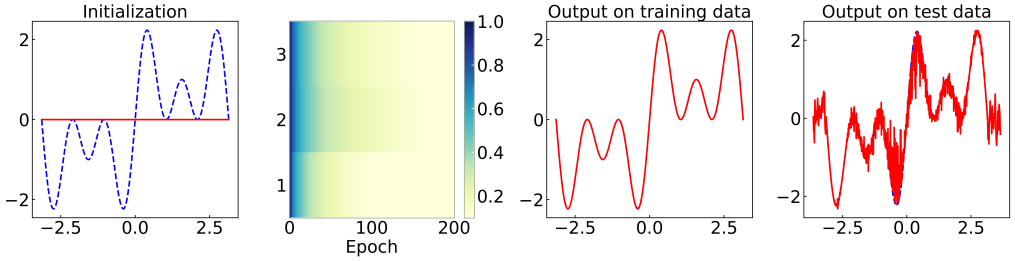
(a) spatial domain (b) frequency domain

Figure 4.8: Illustration of the Ricker activation function.

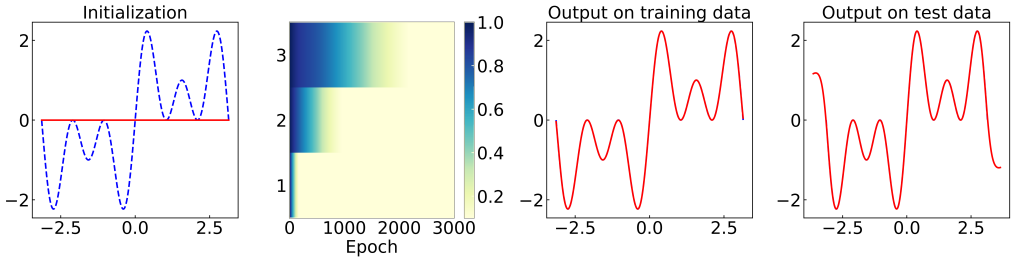
Fig. 4.9 shows the fitting effect of the neural network under different values of a. In the first row of Fig. 4.9, we choose a relatively small a so that the fitting speeds of the high-frequency and low-frequency components are close, i.e., the Frequency Principle is no longer prominent. In this case, although the network can fit the training points perfectly, the output on the test points exhibits obvious oscillations and differs substantially from the true values. This shows that the network overfits the training data, leading to a decrease in generalization. However, we can observe that at the initialization stage, the model already contains a certain amount of high-frequency components, which may directly contribute to the high-frequency oscillations of the fitting result. Therefore, we cannot yet confirm the relationship between overfitting and the weakening of the Frequency Principle. In order to rule out the influence of an oscillatory initial function on the training result, we use a common trick—antisymmetrical initialization (ASI) Zhang et al. (2020)—to enforce that the initial output function is the zero function. Specifically, ASI generates two independent subnetworks with identical initialization parameters, and takes the difference of the two networks as



(a) a=0.1, without ASI trick



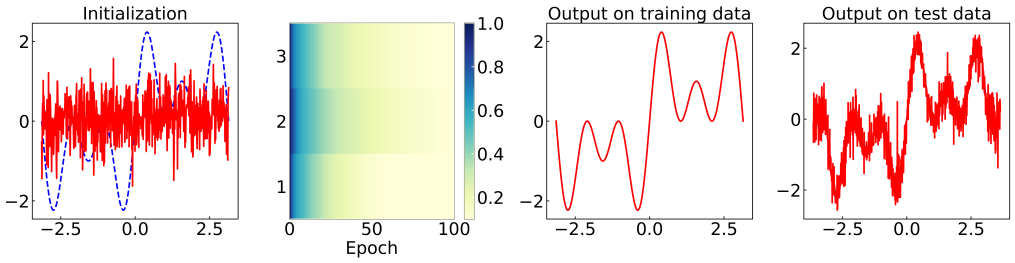
(b) a=0.1, with ASI trick



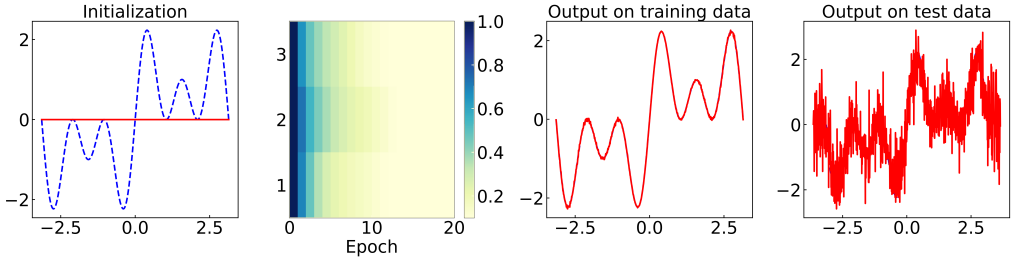
(c) a=0.3, with ASI trick

Figure 4.9: Neural network fitting under different Ricker activation functions.

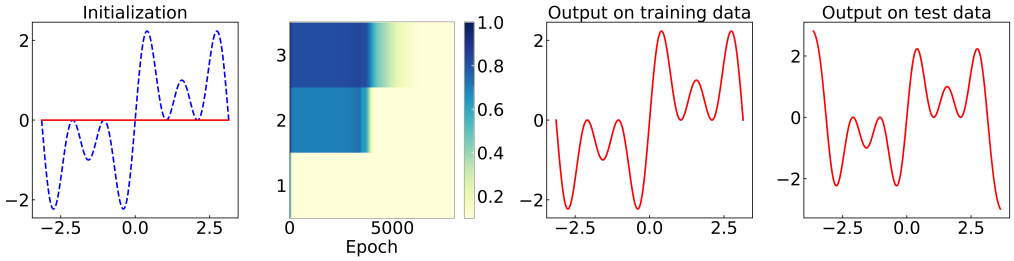
the model output. This trick can effectively ensure consistency of the initial frequency distribution of models with different activation functions. In the second row of Fig. 4.9, we again choose a relatively small a but use ASI to enforce a zero model output; one can easily observe that, in this case, the fitting process of the model still does not follow the Frequency Principle, and the generalization of the model is still poor. In the third row of Fig. 4.9, we choose a relatively large a so that the network fits the low-frequency components more quickly. In this case, although the fitting of the network on the training points is not perfect, its output on the test points is very close to the true values. This indicates that the network captures the overall structure of the function well, achieving good generalization. Similarly, for the activation function $\sigma(x) = sin(ax)$, we can change the frequency distribution of the activation function by modifying the value of a. As shown in Fig. 4.10, for a sufficiently large a, the Frequency Principle is weakened or even disappears, and in this case the model exhibits poor generalization.



(a) a=10, without ASI trick



(b) a=10, with ASI trick



(c) a=1, with ASI trick

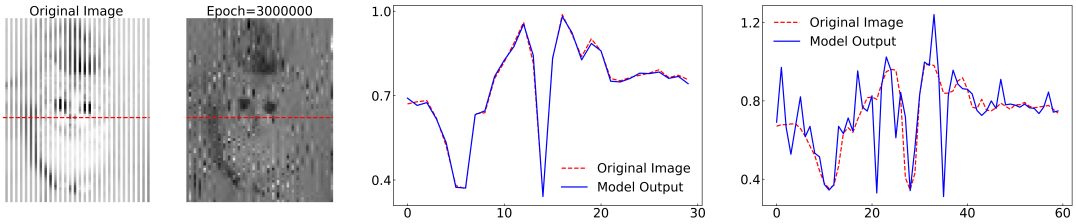
Figure 4.10: Neural network fitting under the activation function $\sigma(x) = \sin(ax)$.

From these two simple examples, we can already appreciate the importance of the Frequency Principle. When the Frequency Principle does not hold, i.e., when the network does not exhibit an obvious low-frequency preference, the model output produced by the fitting often contains a large amount of high-frequency oscillatory components, which typically signals poor generalization performance.

Two-dimensional experiments

For two-dimensional, or even real-world, data, there exists a similar phenomenon. In the two-dimensional experiment, we use a large initialization to break the Frequency Principle. As shown in Fig. 4.11, when we only use the pixels of the odd columns of the image and adopt relatively large initialization weights for training, the network can fit these pixels well on the training set;

but when we feed all pixels into the trained network for testing, the result becomes unrecognizable. Plotting the test output as a curve along the x direction, we find that it contains a large amount of high-frequency oscillations. The reason for this phenomenon is similar to that in the previous one-dimensional experiment: the high-frequency fitting rate is increased, so that the model learns a great deal of high-frequency noise while fitting the low frequencies.



(a) output (b) output on training data (c) output on all data

Figure 4.11: A two-dimensional image fitting experiment with large initialization weights.

4.2.2 A frequency-based understanding of early-stopping

In practical applications of neural networks, people often find that reducing the training error to its minimum is not necessarily the optimal strategy. During training, we typically observe that the training error keeps decreasing, but the test error exhibits a tendency of first decreasing and then increasing. Fig. 4.12 shows a typical example, where we use a neural network to fit the blue dashed curve (panel b) with noise. Panel (a) shows that, as training proceeds, the training error (blue curve) keeps decreasing, but the test error (red curve) first decreases and then increases. Clearly, in this example, we need to stop training early at the lowest point of the test error (the black dashed line) so as to avoid overfitting.

To understand the effectiveness of early-stopping in this example, we first observe the output of the neural network at the black dashed line. From the blue curve in panel (b), one can see that the output of the neural network at this moment is a smooth sine curve. In fact, our data are obtained by sampling from the function $\sin(x)$ plus noise. By contrast, at the end of training, as shown in panel (c), the neural network has fit the high-frequency noise information, and the model overfits. Next, we examine the fitting effect of the neural network at this moment in the frequency domain.

Fig. 4.12(d) shows the amplitude spectrum of the noise-free true data in the frequency domain (blue dashed line). Now we observe the frequency-domain behavior of the neural network on the training and test sets. Taking the moment indicated by the black dashed line in panel (a) (the red solid line in panel (d)), we find that at this moment the low-frequency part of the neural network output agrees well with the true data, while the amplitude of the high-frequency part is still small. This benefits from the Frequency Principle of neural networks, namely that the network tends to learn the low-frequency components first. At this moment, the network has not yet fit the high-frequency part that is heavily influenced by noise, so there is no obvious overfitting. At the end of training (the green dotted line in panel (d)), the high-frequency components are fit, and overfitting occurs.

In many real-world data, the influence of noise on the high-frequency components is often larger. For such data, the Frequency Principle makes the neural network preferentially learn the low-

frequency signals with a high signal-to-noise ratio, which are precisely the dominant features of the data. Adopting the early-stopping trick can effectively prevent the neural network from overfitting the noise-dominated high-frequency components, thereby avoiding severe overfitting problems.

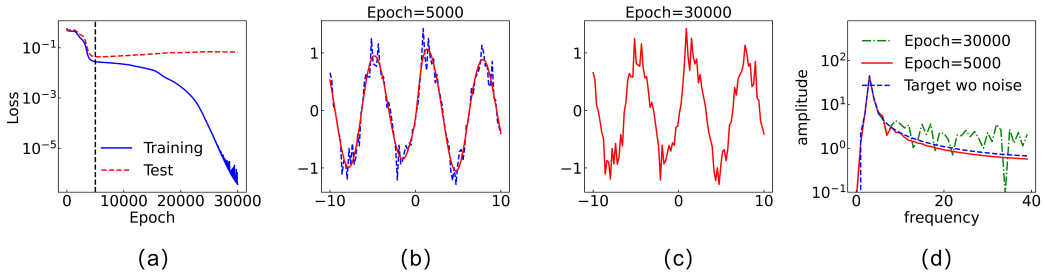


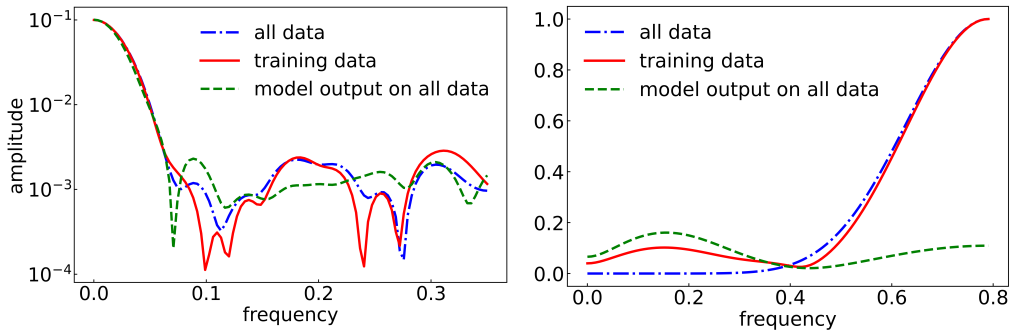
Figure 4.12: An experiment of a neural network fitting noisy data.

4.2.3 Advantages and limitations of neural networks

Earlier, when discussing the generalization problem, we mentioned the famous generalization puzzle that overparameterized neural networks do not easily overfit. In fact, by the No Free Lunch theorem, this generalization puzzle is specific to certain classes of data, such as image classification recognition tasks. From the Frequency Principle we know that neural networks have a low-frequency preference; do those data on which neural networks generalize well themselves possess low-frequency-dominated characteristics? Can we find some high-frequency-dominated data, demonstrating that neural networks cannot generalize well on such data? In this subsection, we will verify these conjectures through frequency-domain analysis of several classes of data.

Fig. 4.13 shows the result of applying the Fourier transform along the principal-component directions of CIFAR10 (for the specific procedure, see the next section). The red line is computed from the training set, the blue dashed line is the result on the test set plus the training set, and the green dashed line is the result of applying the Fourier transform to the neural network's outputs at the training and test points; here the chosen direction is the principal-component direction of the real dataset. One can see that on a low-frequency-dominated dataset, the neural network's fitting result fits the low-frequency information well, while the amplitude of the high-frequency components is small, so that even if the high frequencies are not fit well it does not cause a particularly large error.

Now consider a high-frequency-dominated dataset: define a function in a d-dimensional space, where each dimension takes only 1 or -1, i.e., $\boldsymbol{x} = (x_1, \cdots, x_d) \in \{-1, 1\}^d$, and the function is $f(\boldsymbol{x}) = \prod_{j=1}^{d} x_j$. When an even number of -1's appear among all dimensions, $f(\boldsymbol{x})$ takes the value 1; if an odd number of -1's appear, the function value is -1. This function is called the parity function and is often used in IQ tests. Once one notices that the output only depends on the number of -1's, it becomes very easy to predict the values at other points. Taking $d = 10$ as an example, this function is only defined on $2^{10} = 1024$ points. If we only give 200 of these



(a) CIFAR10 (b) parity function

Figure 4.13: Neural networks fitting real data.

data points as the training set and use the remaining points as the test set, the neural network can accurately predict the output at every point in the training set, i.e., it can memorize the outputs corresponding to all training points, but on the test points its accuracy stays around 50%. One sees how unintelligent artificial intelligence can be in this example! Likewise, we use frequency space to analyze this example.

Performing a discrete Fourier transform on this function:

$$\frac{1}{2^d}\sum_{x\in\Omega}\prod_{j=1}^{d} x_j \mathrm{e}^{-\mathrm{i}2\pi \boldsymbol{k}\cdot\boldsymbol{x}} = (-\mathrm{i})^d \prod_{j=1}^{d} \sin 2\pi k_j. \tag{4.4}$$

We display the curve of this transform as the blue line in Fig. 4.13(b). When the frequency k grows, its amplitude also grows slowly, indicating that this is a set of high-frequency-dominated data. When our sampling is incomplete, the result of the discrete Fourier transform on the training set is represented by the red line. Clearly, in the low-frequency part with small amplitude, it differs noticeably from the true spectrum. In fact, due to insufficient sampling, the training data produce some spurious low frequencies (the frequency aliasing effect).

Now using a neural network to fit this function, one can find that the neural network produces many spurious low frequencies. That is, for data with unknown labels, the neural network tends to fit with low frequencies, causing the high frequencies learned by the neural network to be much fewer than the high frequencies of the true data. Therefore, the neural network's spectrum (green dashed line) is completely different from the spectrum of the true data, which also explains why the neural network is completely unable to make good predictions on the parity function.

Chapter 5

Designing Efficient Neural Networks Based on the Frequency Principle

In many scientific and engineering problems, multi-scale phenomena in the frequency domain are very common. For example, in image processing, an image often contains both low-frequency regions where the color varies smoothly inside an object and high-frequency components such as object edges, details and textures where the variation is abrupt; in problems of solving differential equations, the solutions of many equations are obtained by composing periodic functions of different frequency scales. According to the Frequency Principle (F-Principle, also known as spectral bias) introduced in earlier chapters, common neural network architectures converge fast on low-frequency information during training but relatively slowly on high-frequency information, which means that they often face the difficulty of learning high frequencies in real applications. It is therefore necessary to design suitable network architectures to accelerate the convergence of neural networks on high frequencies, so as to cope with multi-scale problems arising in real applications.

In recent years, the frequency perspective has received increasing attention in neural network optimization. Researchers have approached the problem from multiple angles such as model architecture, solving strategy and feature representation, and have proposed a series of frequency-analysis-based improvements that significantly enhance the ability of neural networks to handle multi-scale problems. For example, in computer vision, by introducing multi-scale feature extraction modules, networks can better capture object information at different scales and achieve significant performance gains on tasks such as image classification and object detection. In scientific computing, designing multi-scale neural network architectures enables more efficient solution of multi-scale partial differential equations, greatly broadening their range of applications. It can be foreseen that the combination of the frequency perspective with neural networks still has great potential.

This chapter focuses on several important efficient neural networks designed based on the Frequency Principle, including multi-scale neural networks, neural radiance fields, and Fourier feature networks. By introducing the ideas, methods and experimental results of these models, we hope to further illustrate the importance of the Frequency Principle and to inspire more thinking on related

topics.

Popular Science

What impact does the Frequency Principle have on neural network training?

During training, neural networks usually converge fast on low-frequency information and slowly on high-frequency information (such as the details and edges in an image), so that the model performs poorly when capturing complex features and fine details.

What multi-scale phenomena in the frequency domain arise in real problems?

In real scientific and engineering problems, multi-scale phenomena in the frequency domain are very common. In image processing, an image typically contains components of different frequencies: low-frequency components include large smooth regions, such as sky or walls, which appear as slowly varying colors; high-frequency components include details and edges, such as the contours and textures of objects, which appear as rapidly varying pixel values. In climate models, multi-scale phenomena span time scales from a few days to several centuries, where seasonal and interannual variations are low-frequency variations, while weather changes and short-term meteorological events are high-frequency variations. In audio signal processing, low-frequency components usually correspond to the pitch of the sound and the region where most of the energy is concentrated, such as the bass in music; high-frequency components correspond to the details and texture of the sound, such as the harmonics of instruments and the clarity of human voices. Biomedical signals such as electrocardiograms (ECG) or electroencephalograms (EEG) also exhibit multi-scale phenomena: low-frequency components reflect the overall activity trends of the heart or brain, while high-frequency components reflect rapidly varying electrophysiological events such as heartbeats or neural firing. These examples show that understanding and handling multi-scale phenomena in the frequency domain is crucial for analyzing and solving real-world problems.

In existing works, what methods have researchers used to alleviate the "high-frequency difficulty" of neural networks?

Researchers have approached the problem from multiple angles, including model architecture, solving strategy and feature representation, and have proposed a series of frequency-analysis-based improvements that significantly enhance the ability of neural networks to handle multi-scale problems.

5.1 Multi-scale Neural Network Architectures

Based on the property that neural networks converge fast on low frequencies and slowly on high frequencies, a natural idea is to convert high-frequency data into low-frequency data so as to accelerate training. Based on this idea, PhaseDNN Cai et al. (2020) partitions each frequency dimension into grids and then transforms the frequency on each grid into a low frequency. However, operating

along each dimension leads to the curse of dimensionality, making it hard to handle high-dimensional problems. So how can we realize such a transformation in high-dimensional problems? We can observe the two sub-figures in Fig. 5.1: the oscillation of the left one is clearly more violent than that of the right one. To convert the left into the right, we only need to apply a simple scaling to the coordinates. The rationale of this simple idea is also justified by the scaling property of the Fourier transform:

$$\mathcal{F}(f(kx))(\zeta) = \frac{1}{|k|}\widehat{f}(\frac{\zeta}{k}). \tag{5.1}$$

In a general problem, we usually do not know the frequency information of the original data in advance, nor can we determine what scaling factor should be used. To address this issue, we can provide multiple scaling channels and let the neural network choose the appropriate frequency transformation by itself. In this way, even if we do not know the specific frequency of the data, the neural network can still adaptively adjust to fit information of different frequencies. Based on this idea, Liu et al. (2020) designed multi-scale neural networks, which alleviate the difficulty of learning high frequencies by applying scaling transformations to the network input.

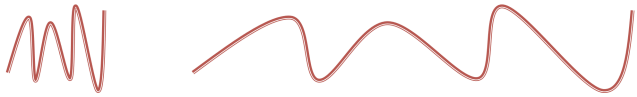
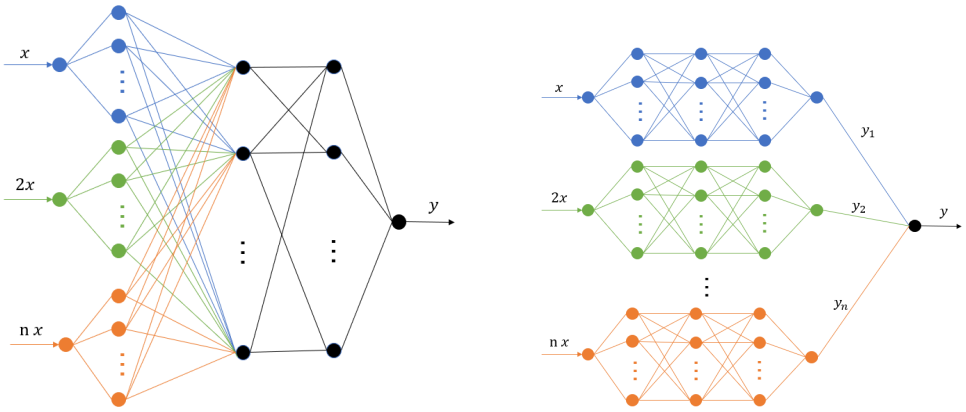


Figure 5.1: Turning a high-frequency signal into a low-frequency one via scaling.

5.1.1 Architecture



(a) MscaleDNN-1 (b) MscaleDNN-2

Figure 5.2: Two types of multi-scale neural network architectures. Adapted from Liu et al. (2020).

To implement a multi-scale neural network, we partition the neurons of the first hidden layer into A groups. The input to the neurons in the i-th group is $i\boldsymbol{x}$, that is, the corresponding output

is $\sigma(i\boldsymbol{w}\cdot\boldsymbol{x}+b)$, where $\boldsymbol{w}$, $\boldsymbol{x}$ and $\boldsymbol{b}$ denote the weight, the input and the bias term respectively. The complete MscaleDNNs are defined as follows,

$$h(\boldsymbol{x}) = \boldsymbol{W}_L\sigma\circ(\boldsymbol{W}_{L-1}\sigma\circ(\cdots\sigma\circ(\boldsymbol{W}_1\sigma\circ(\boldsymbol{K}\odot\boldsymbol{W}_0\boldsymbol{x}+\boldsymbol{b}_0)+\boldsymbol{b}_1)\cdots)+\boldsymbol{b}_{L-1}), \tag{5.2}$$

where $\boldsymbol{x}\in\mathbb{R}^d$, $\boldsymbol{W}_l\in\mathbb{R}^{n_{l+1}\times n_l}$, n_l is the number of neurons in the l-th hidden layer, $n_0=d$, $b_l\in\mathbb{R}^{n_{l+1}}$, σ denotes the activation function of the network, $\circ$ denotes elementwise operation, and $\odot$ denotes the Hadamard product (elementwise multiplication at corresponding positions),

$$\boldsymbol{K} = (\underbrace{1,1,\cdots,1}_{\text{1st part}},\underbrace{1,1,\cdots,1}_{\text{2nd part}},2,\cdots,i-1,\underbrace{i,i,\cdots,i}_{\text{(i+1)th part}},i+1,\cdots,\underbrace{A-1,A-1,\cdots,A-1}_{\text{Ath part}})^T. \tag{5.3}$$

Fig. 5.2 shows two common multi-scale network architectures. Architecture (a) attaches a coefficient to each neuron in the first hidden layer of an ordinary fully connected network. The second architecture is composed of small sub-networks of different scales; since there are no connections between sub-networks, the number of parameters is much smaller than in the first one.

Note that if we only use a single very large scaling factor to stretch the function down to a very low frequency, it may look as if the function has become low-frequency, but in practice the neural network still has difficulty learning it well. On the one hand, the neural network may have a specific frequency at which learning is fastest, and a too-low frequency may actually slow down learning. On the other hand, such a uniform scaling still leaves the problem multi-scale, and learning at different scales will compete with each other and slow down the overall learning speed. By providing the network with multiple scales, we expect that, under algorithms such as gradient descent, the network will choose the fastest learning path to update the output and reach an optimal speed. The example in the next subsection experimentally verifies that the parts with different scalings indeed learn components of different frequencies.

Next, we examine a two-dimensional porous example from Liu et al. (2020) to verify the feasibility of the multi-scale network. Consider the Poisson equation with the following source term:

$$f(\boldsymbol{x}) = 2\mu^2\sin\mu x_1\ \sin\mu x_2, \mu = 7\pi. \tag{5.4}$$

The exact solution is

$$u(\boldsymbol{x}) = \sin\mu x_1\sin\mu x_2. \tag{5.5}$$

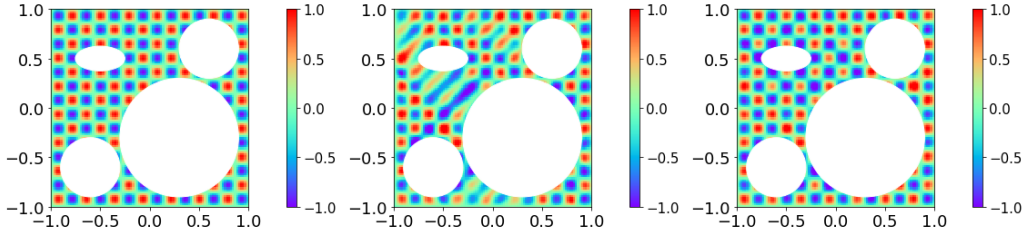
We use the exact solution to set the boundary conditions as well. The sampling and the network architecture are the same as before. By using such an architecture to parameterize the equation, the loss has two parts: the first part is the equation residual term L_{eq}, which uses the energy of the equation; the second part L_{bd} is the boundary loss. The form of the loss function is as follows:

$$L_{sum} = L_{eq} + L_{bd}, \tag{5.6}$$

For details we refer to the content on neural network solvers for partial differential equations in Section A13. As shown in Fig. 5.3, the multi-scale neural network can accurately capture the oscillations in the exact solution.

5.1.2 Neural Networks Based on Subspace Decomposition

Li et al. (2023) proposed a Subspace Decomposition based DNN (SD^{2}NN) architecture, as shown in Fig. 5.4. This architecture consists of two parts: the first part is a low-frequency MscaleDNN



(a) Exact solution (b) Standard neural network (c) MscaleDNN

Figure 5.3: A two-dimensional porous example. Figure reproduced from Liu et al. (2020).

sub-module or an ordinary fully connected neural network for capturing the smooth component of the multi-scale solution; the second part consists of one or several high-frequency MscaleDNN sub-modules for capturing the oscillatory component of the multi-scale solution. To separate the smooth and oscillatory components of the solution, an orthogonality penalty term is added to the loss function, enforcing orthogonality between subspaces at different scales. This work uses an activation function of the form $\sigma(z) = s[\cos(z), \sin(z)]$, where $s \in (0, 1]$ is a relaxation parameter used to control the output range, and s is set to 0.5 in the paper.

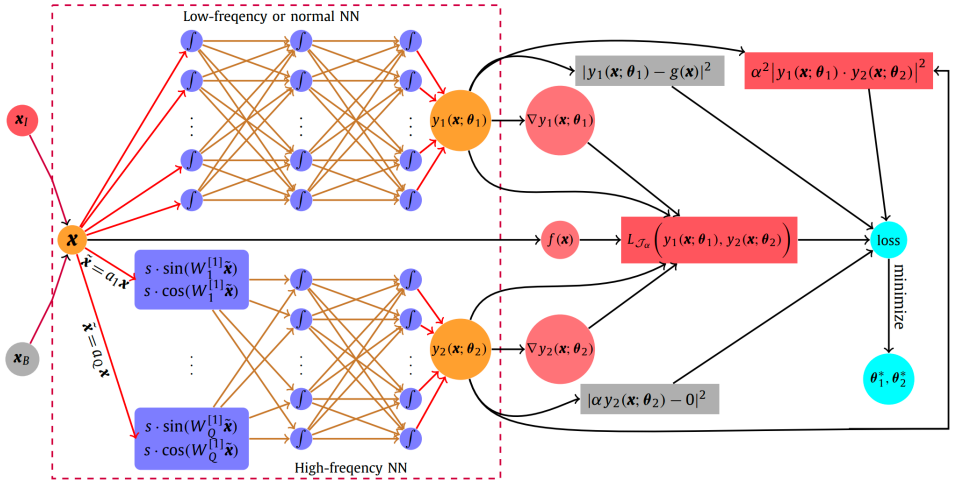


Figure 5.4: Diagram of the SD^{2}NN architecture, adapted from Li et al. (2023)

To verify the effectiveness of the proposed architecture, the paper presents the following one-dimensional elliptic equation example:

$$\begin{cases} -\operatorname{div}\left(A(x)\nabla u(x)\right) = f(x), & x \in [0,1], \\ u(0) = u(1) = 0, \end{cases} \tag{5.7}$$

where the coefficient $A(x)$ has three scales $\mathcal{O}(1), \mathcal{O}(\varepsilon_1), \mathcal{O}(\varepsilon_2)$, with $1 \gg \varepsilon_1 = 0.1 \gg \varepsilon_2 = 0.01$, and the explicit form is:

$$A(x) = \left(2 + \cos\left(2\pi\frac{x}{\epsilon_1}\right)\right)\left(2 + \cos\left(2\pi\frac{x}{\epsilon_2}\right)\right), \tag{5.8}$$

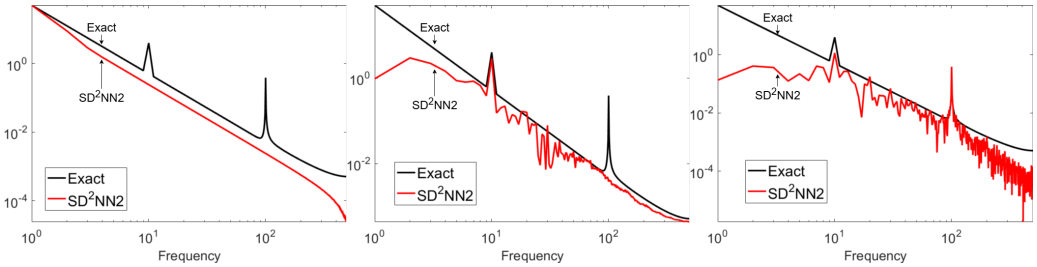
In this case, the analytical solution of Eq. (5.7) is:

$$u(x) = x - x^2 + \frac{\epsilon_1}{4\pi}\sin\left(2\pi\frac{x}{\epsilon_1}\right) + \frac{\epsilon_2}{4\pi}\sin\left(2\pi\frac{x}{\epsilon_2}\right). \tag{5.9}$$

On this example the paper uses an architecture denoted SD^{2}NN2, in which the first part adopts a low-frequency MscaleDNN module (without adding any extra scaling factor), and the second part consists of a mid-frequency MscaleDNN and a high-frequency MscaleDNN (the scaling factors added to the mid-frequency module are smaller than those of the high-frequency one). The sum of the outputs of the three networks is used to approximate the target function. With this architecture parameterizing the equation, the loss function has three parts: the first part is the equation residual term L_{eq}, which uses the energy of the equation so that the network output satisfies the governing equation; the second part L_{orth} is the orthogonality constraint loss, which encourages the outputs of modules of different frequencies to be as orthogonal as possible; the third part L_{bd} is the boundary loss, which makes the network satisfy the boundary conditions. Its form is as follows:

$$L_{sum} = L_{eq} + L_{orth} + L_{bd}, \tag{5.10}$$

For details we refer to the content on neural network solvers for PDEs in Section A12. Fig. 5.5 shows the results learned by the three sub-modules of SD^{2}NN2. We can see that each sub-module successfully captures the frequency component of the corresponding scale, agrees well with the analytical solution, and reconstructs the parts at different scales accurately.



(a) Low-frequency module result of SD^{2}NN2 (b) Mid-frequency module result of SD^{2}NN2 (c) High-frequency module result of SD^{2}NN2

Figure 5.5: A three-scale one-dimensional example. Figure reproduced from Li et al. (2023).

5.2 Neural Radiance Fields

Neural Radiance Fields (NeRF) Mildenhall et al. (2021) can learn from a set of images and generate high-quality novel views, with a visual quality reaching the state-of-the-art level at that time. The NeRF demonstrations are impressive and have inspired many follow-up works based on this new approach. In subsequent developments, NeRF has been widely used in 3D reconstruction, view

synthesis, virtual reality and related areas. Within four years of its introduction, the NeRF paper has been cited more than 6400 times (Google Scholar).

(a) Ground truth (b) Prediction with positional encoding (c) Prediction without positional encoding

Figure 5.6: Comparison with positional encoding removed. Figure reproduced from Mildenhall et al. (2021).

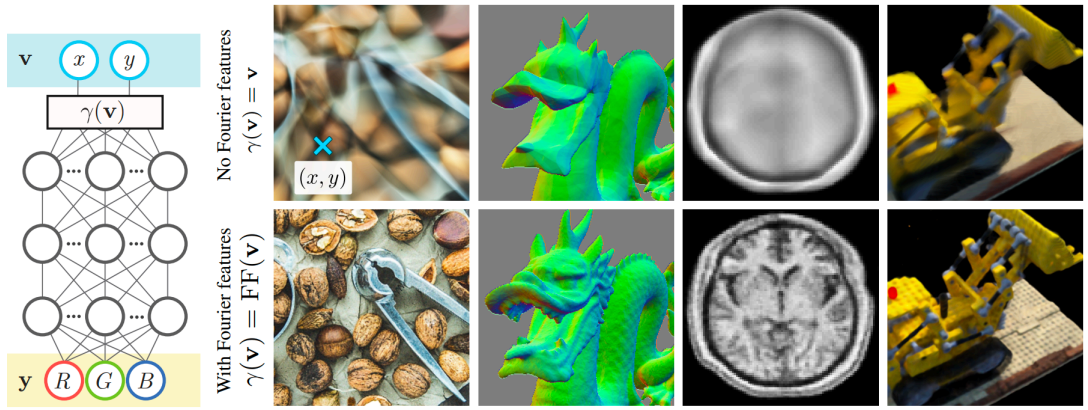
The core idea of NeRF is to use a neural network to represent the continuous three-dimensional space and to generate new views via volumetric rendering. In NeRF, positional encoding plays a crucial role. It essentially uses sine and cosine functions as the activation function of the first hidden layer. In the original NeRF paper, the authors use a multi-layer fully connected neural network to fit the neural radiance field. For an input $\boldsymbol{x}$, NeRF first maps each dimension to a higher-dimensional space using the mapping function $\gamma(p) = (\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p))$, where p is one dimension of $\mathbf{x}$ and L is a hyperparameter. This is a special kind of multi-scale architecture in which all trainable parameters are set to constants, so that the scaling factors play the key role. To visualize the effect of positional encoding, Fig. 5.6 shows a comparison of the NeRF rendering results with and without positional encoding. We can see that with positional encoding, the images generated by NeRF contain more high-frequency details and the visual quality is markedly improved. This illustrates the importance of positional encoding for the performance of NeRF.

5.3 Fourier Feature Networks

Tancik et al. (2020) introduced a more general Fourier feature mapping γ, defined for $\mathbf{v} \in \mathbb{R}^d$ by:

$$\gamma(\mathbf{v}) = \left[a_1 \cos\left(2\pi \mathbf{b}_1^\top \mathbf{v}\right), a_1 \sin\left(2\pi \mathbf{b}_1^\top \mathbf{v}\right), \ldots, a_m \cos\left(2\pi \mathbf{b}_m^\top \mathbf{v}\right), a_m \sin\left(2\pi \mathbf{b}_m^\top \mathbf{v}\right)\right]^\top, \tag{5.11}$$

where $\{\mathbf{b}_i\}$ are frequency vectors sampled from some probability distribution. Fig. 5.7 depicts the model architecture of Fourier features as well as the effect of including or omitting them; comparing the two rows of results, we can see that after adding the Fourier feature mapping, the images generated by the neural network contain richer high-frequency details and the visual quality is markedly improved. This intuitively illustrates that Fourier feature mapping can enhance the ability of neural networks to model high-frequency signals.



(a) Coordinate-based MLP (b) Image regression $(x,y) \rightarrow$ RGB (c) 3D shape regression $(x,y,z) \rightarrow$ occupancy (d) MRI reconstruction $(x,y,z) \rightarrow$ density (e) Inverse rendering $(x,y,z) \rightarrow$ RGB, density

Figure 5.7: Diagram of the Fourier feature architecture and the effect of including or omitting Fourier features on model predictions, adapted from Tancik et al. (2020)

5.4 More

Many works on neural network algorithms have designed methods to improve the learning of high frequencies; we list a few examples here.

To accelerate the learning of high frequencies, designing new activation functions is also a common approach, such as adaptive activation functions Jagtap and Karniadakis (2023), mixed activation functions Jagtap et al. (2022), and hidden units with sharper variations Agarwal et al. (2020).

Combining deep neural networks with traditional iterative methods can accelerate the convergence on low and high frequencies in computational problems respectively Xu et al. (2020); Huang et al. (2020). For example, when solving an equation, one can first use a DNN to obtain a solution through a certain number of optimization steps (or iterations). Then, using the value obtained by the trained DNN as the initial value, one performs traditional iterations, exploiting the smoothing property of iterative methods to eliminate high-frequency errors. However, this approach may only handle relatively low-dimensional problems; for problems with complex boundaries, mesh-free neural network methods still have advantages.

The random feature method (RFM) Chen et al. (2022) for solving partial differential equations partitions the solution space into several sub-regions, shifts the center of each sub-region to the origin, and equips it with a multi-scale coefficient, for example

$$\frac{\boldsymbol{x} - c_i}{r_i}, \tag{5.12}$$

where c_i is the center of the i-th sub-region and r_i is a prescribed random number that plays the role of a scaling factor as in multi-scale networks.

Chapter 6

Mechanistic Analysis of the Frequency Principle

In Chapter 4, we introduced the concept of the Frequency Principle (F-Principle, also known as spectral bias) and verified its validity through experiments. We also discussed several explanations that the Frequency Principle offers for phenomena arising during neural network training. The Frequency Principle states that neural networks typically fit training data in the order from low frequencies to high frequencies. We further observed that models which do not satisfy the Frequency Principle tend to exhibit poor generalization. Therefore, systematically characterizing the conditions under which the Frequency Principle holds and understanding why it holds are important.

In this chapter, we delve into the underlying mechanism of the Frequency Principle. We first analyze in detail the main factors that influence the Frequency Principle, including the magnitude of initialization weights, the choice of activation function, and the form of the loss function. Through experiments and theoretical analysis, we reveal how these factors affect the frequency convergence behavior of neural networks during training. Furthermore, we introduce the Linear Frequency Principle (LFP) model and, through theoretical derivation and experimental verification, uncover the dynamical mechanism behind neural network frequency convergence and its equivalent variational problem. We also discuss the Neural Tangent Kernel (NTK) theory in the linear regime and its explanation of the Frequency Principle, analyzing how the Frequency Principle manifests in higher dimensions and in more complex neural networks. In addition, we examine the limiting problem of frequency convergence, studying the minimum possible decay rate of neural network frequency convergence in different data dimensions. We verify the theoretical analysis through numerical experiments and discuss the implications of these results for neural network training and generalization. Through these analyses and experiments, we not only deepen our understanding of the Frequency Principle, but also provide theoretical grounds and guidance for the initialization and training of neural networks in practical applications.

Popular Science Section

What is the Frequency Principle? Which factors may affect it?

The Frequency Principle refers to the phenomenon that neural networks during training typically first fit the low-frequency components of the data, and then gradually fit the higher-frequency components. The main factors that influence this phenomenon include the magnitude of initialization weights, the choice of activation function, and the form of the loss function, among others.

First, the **magnitude of initialization weights** has a significant effect on the Frequency Principle. When the initialization weights are large, the neuron outputs contain more high-frequency components, which speeds up the fitting of high frequencies and thus weakens or even destroys the Frequency Principle. Therefore, choosing an appropriate range of initialization weights is very important to ensure the validity of the Frequency Principle.

Second, the **choice of activation function** also affects the Frequency Principle. Common activation functions such as tanh and ReLU have amplitudes that are monotonically decreasing with frequency in the frequency domain, and the Frequency Principle is easily observed in such cases. However, if a function whose amplitude in the frequency domain first increases and then decreases with frequency is used—such as the Ricker function—the Frequency Principle may fail.

Finally, the **form of the loss function** has a significant effect on the convergence speed at different frequencies. If certain frequencies are explicitly assigned large weights in the loss function, the convergence of these frequency components is accelerated. For example, a loss function that incorporates an additional derivative term can significantly accelerate the convergence of high-frequency components.

6.1 Factors Affecting the Frequency Principle

Before theoretically proving the Frequency Principle, we first discuss the factors that influence it, which will help us understand the conditions under which the Frequency Principle holds. Intuitively, the magnitude of neural network initialization weights is likely to affect the Frequency Principle, and we begin our investigation from this point.

6.1.1 Effect of the Magnitude of Initialization Weights

The scale of initialization weights plays an important role in the Frequency Principle. Under large-initialization settings, the Frequency Principle becomes weaker or even disappears. From the time-domain perspective, we consider the output of a simplified tanh neuron $\tanh(wx)$. We take $w = 1$ and $w = 100$ to represent the small- and large-initialization settings under the same activation function. As shown in Figure 6.1, when the initialization is larger, the output of the simplified neuron varies more sharply near $x = 0$, which means that its output contains more high-frequency components. When a neural network is composed of such activation functions, the model's high-frequency fitting speed increases, which causes the Frequency Principle of the model to be weakened or to fail.

As shown in Figure 6.2, the red solid line is the initial model output, and the blue dashed line is the curve to be fitted. When the initialization weights are small, the neural network does not fit information beyond the highest frequency in the dataset. However, if the initialization weights are set too large, a large amount of high-frequency information is introduced in the initial state, some

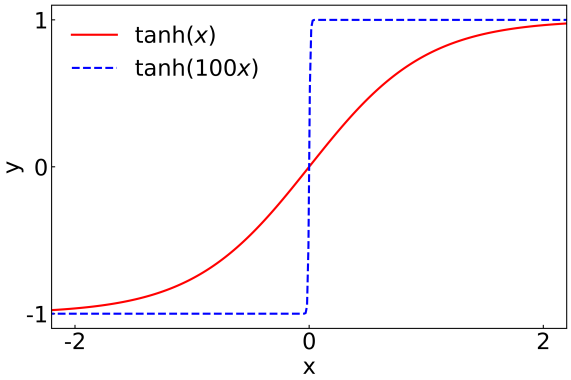
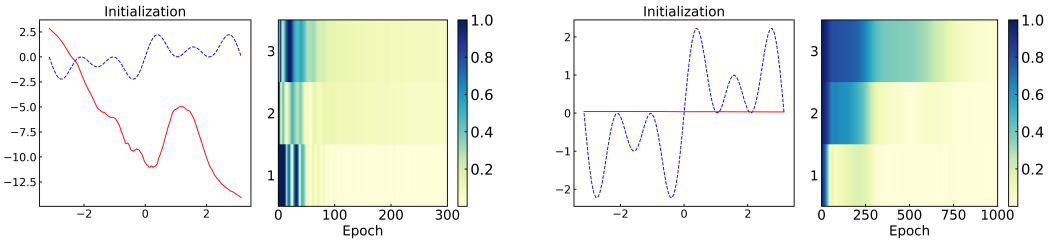


Figure 6.1: Output of a tanh neuron under small-initialization and large-initialization settings

of which even exceeds the highest frequency in the dataset. In this case, the Frequency Principle of the model is weakened or fails. Meanwhile, the neural network does not learn information of frequencies higher than those in the dataset, which means that the training process cannot fully cancel the high-frequency interference introduced by initialization. This is confirmed by the relative-error plot: under large initialization weights, the high-frequency fitting speed in the initial training phase exceeds that of low frequencies and exhibits large oscillations. Under small initialization weights, the fitting speed decreases sequentially from low to high frequencies, which helps the network learn the main features of the data stably and avoids overfitting.



(a) large initialization

(b) small initialization

Figure 6.2: Fitting result for a function under large and small initialization weights

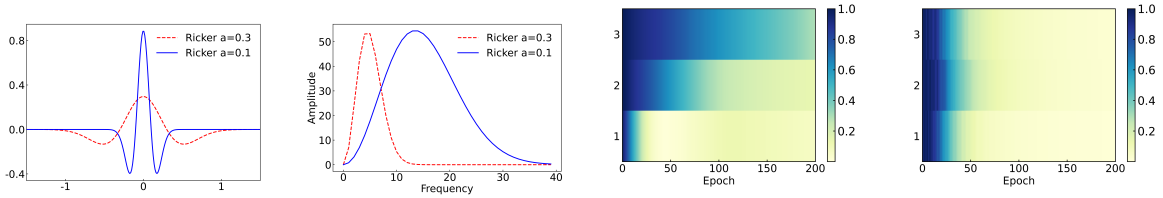
6.1.2 Effect of Different Activation Functions

In addition to the initialization weights, we find that different activation functions also affect the frequency convergence speed. For most activation functions, such as tanh and ReLU, they are monotonically decreasing in the frequency domain. Therefore, in these cases, we can easily observe

the Frequency Principle. However, if we design a class of functions that is not monotonically decreasing in the frequency domain—for example, a function whose amplitude in the frequency domain first increases monotonically up to some high frequency and then decreases monotonically—then we may observe the failure of the Frequency Principle. To verify this point, we try to use the Ricker function with parameter a as the activation function:

$$\frac{1}{15a}\pi^{1/4}\left(1-\left(\frac{x}{a}\right)^2\right)\exp\left(-\frac{1}{2}\left(\frac{x}{a}\right)^2\right). \tag{6.1}$$

When a is small, the Ricker function starts to decay from a higher frequency. We use a fully connected network to fit $\sin(x)+\sin(3x)+\sin(5x)$. As shown in Figure 6.3, for the case of $a = 0.3$, when the activation function starts to decay from a relatively low frequency, we can clearly observe that the low-frequency components converge faster; however, when we set $a = 0.1$, so that the activation function starts to decay from a relatively high frequency, we cannot observe a clear order of frequency convergence. We see that the behavior of the activation function in the frequency domain significantly affects the order in which a neural network learns different frequency components, so the frequency characteristics of the activation function are an important factor that influences the Frequency Principle.



(a) spatial domain (b) frequency domain (c) $a = 0.3$ (d) $a = 0.1$

Figure 6.3: The Ricker activation function in the spatial and frequency domains, and the effect of different values of the parameter a on the fitting speed of high- and low-frequency components.

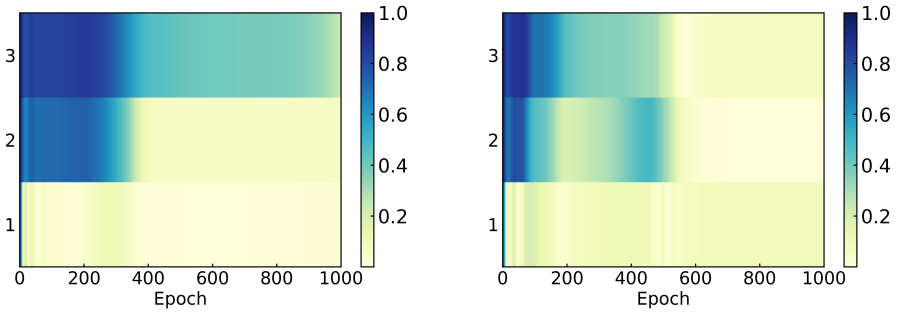
6.1.3 Effect of the Form of the Loss Function

In addition to the factors above, the form of the loss function can also significantly affect the convergence of different frequencies. For example, if we explicitly assign large weights to certain frequencies in the loss function, the convergence of these frequency components may be accelerated. To verify this, we can consider two loss functions: one is the ordinary mean-squared loss L_{nongrad}, and the other is the loss L_{grad} obtained by adding an additional derivative term to L_{nongrad},

$$L_{\text{nongrad}} = \sum_{i=1}^{n}(f_\theta(x_i)-f^*(x_i))^2/n, \tag{6.2}$$

$$L_{\text{grad}} = L_{\text{nongrad}} + \sum_{i=1}^{n}(\nabla_x f_\theta(x_i)-\nabla_x f^*(x_i))^2/n. \tag{6.3}$$

Likewise, we use a neural network to fit $\sin(x)+\sin(3x)+\sin(5x)$. As shown in Figure 6.4, after adding the derivative term to the loss function, the convergence speed of high-frequency components is significantly accelerated. The key reason behind this is that, in the frequency domain, the Fourier transform of a derivative equals the Fourier transform of the original function multiplied by the corresponding frequency. This effectively imposes a larger weight on the high-frequency components. Therefore, in certain specialized tasks such as solving differential equations, where the loss function often involves gradient information, the Frequency Principle may be weakened or disappear.



(a) L_{nongrad} (b) L_{grad}

Figure 6.4: The convergence speed at each major frequency under the loss functions L_{nongrad} and L_{grad}

6.2 A Simple Analysis of the Frequency Principle

First, let us consider the following network structure: a two-layer deep neural network (DNN) using $\sigma(x)=\tanh(x)$ as the activation function to fit a one-dimensional target function $f(x)$. We choose $\tanh(x)$ as the activation function because its Fourier transform is easy to compute. Both the input and the output of the network are one-dimensional, expressed as

$$h(x)=\sum_{j=1}^{m} a_j\sigma\left(w_j x+b_j\right). \tag{6.4}$$

Taking the Fourier transform of the activation function $\sigma(w_j x+b_j)$, we obtain

$$\hat{\sigma(w_j x+b_j)}(k)=\frac{2\pi i}{|w_j|}\exp\left(\frac{ib_j k}{w_j}\right)\frac{1}{\exp(-\frac{\pi k}{2w_j})-\exp(\frac{\pi k}{2w_j})}. \tag{6.5}$$

From the above expression, we see that as the frequency increases, $\tanh(x)$ decays exponentially in the frequency domain. The main reason for this fast decay is that $\tanh(x)$ is a smooth, infinitely

differentiable function in the time domain. Taking the continuous Fourier transform of $h(x)$, we obtain

$$\hat{h}(k) = \sum_{j=1}^{m} \frac{2\pi i a_j}{|w_j|} \exp\left(\frac{i b_j k}{w_j}\right) \frac{1}{\exp(-\frac{\pi k}{2w_j}) - \exp(\frac{\pi k}{2w_j})} \tag{6.6}$$

$$\approx \sum_{j=1}^{m} a_j \exp\left(\frac{i b_j k}{w_j}\right) \exp\left(-\left|\frac{\pi k}{2w_i}\right|\right). \tag{6.7}$$

Note that this section is a qualitative analysis. The right-hand side of the first line has a singularity at frequency $k = 0$, but we do not analyze it in depth here. The approximate equality on the second line holds approximately for relatively large frequencies. Define the loss at a given frequency as

$$L(k) = \frac{1}{2}|\hat{h}(k) - \hat{f}(k)|^2. \tag{6.8}$$

According to Parseval's theorem, the total loss in the frequency domain equals the total loss in the time domain, that is,

$$L = \int L(k)dk = \int \frac{1}{2}|f(x) - h(x)|^2 dx. \tag{6.9}$$

We use gradient descent to train the network, and the parameter update is given by

$$\theta^{(n+1)} \leftarrow \theta^{(n)} - \eta \sum \frac{\partial L(k)}{\partial \theta}, \tag{6.10}$$

where η is the learning rate. Next, we examine the expression for $\frac{\partial L(k)}{\partial \theta}$:

$$\left|\frac{\partial L(k)}{\partial \theta}\right| \approx |\hat{h}(k) - \hat{f}(k)| \exp\left(-\left|\frac{\pi k}{2w_i}\right|\right) G(\theta, k), \tag{6.11}$$

where $G(\theta, k)$ is an $O(1)$ function. From the above expression, we see that if the low-frequency components have not yet converged, that is, $A(k) = |\hat{h}(k) - \hat{f}(k)| > 0$, and the parameters w_i are small, then the term $\exp(-|\pi k/2w_i|)$ dominates. This means that the contribution of low-frequency components to the gradient is much greater than that of high-frequency components, and the direction of gradient descent is in fact closer to the direction of low-frequency convergence. Therefore, from this idealized model, we obtain an important insight: the bias of deep learning toward low frequencies stems from the smoothness and regularity of the activation function, together with gradient-descent-based training methods. Similarly, for the ReLU(x) activation function, one can also prove that it decays polynomially in the frequency domain.

According to Eq. 6.11, we can see the connection between whether the Frequency Principle holds and the three factors discussed in the previous section. Here $|\hat{h}(k) - \hat{f}(k)|$ is affected by the form of the loss function, the form of $\exp\left(-\left|\frac{\pi k}{2w_i}\right|\right)$ benefits from the form of the activation function (with tanh as the example in the derivation above), and the magnitude of this term is affected by w_i, i.e., the size of the parameters.

If we cast the idealized model above into a rigorous mathematical statement, we have

Theorem 2. *Consider a two-layer neural network using $\sigma(x) = \tanh(x)$ as the activation function. For any two frequencies k_1, k_2 such that $\left|\hat{f}(k_1)\right| > 0, \left|\hat{f}(k_2)\right| > 0$ and $|k_2| > |k_1| > 0$, there exist*

positive constants c and C such that, for sufficiently small δ,

$$\frac{\mu\left(\left\{W:\left|\frac{\partial L(k_1)}{\partial \theta_{lj}}\right|>\left|\frac{\partial L(k_2)}{\partial \theta_{lj}}\right| \quad \text{for all} \quad l,j\right\}\cap B_\delta\right)}{\mu\left(B_\delta\right)}\geq 1-C\exp(-c/\delta) \tag{6.12}$$

where $B_\delta \subset \mathbb{R}^m$ is a ball centered at the origin with radius δ, and $\mu(\cdot)$ is the Lebesgue measure.

In the above theorem, for any two frequencies k_1, k_2 that have not yet converged, one can prove that as long as our parameters lie in some ball, when the ball's radius is small—i.e., the parameters are small—the measure of the region where the low-frequency gradient exceeds the high-frequency gradient, divided by the measure of the whole space, tends to 1 exponentially. That is, as $\delta \to 0$, the gradient corresponding to the low frequency exceeds the gradient corresponding to the high frequency almost everywhere.

This piece of theory not only reveals the reason behind the Frequency Principle, but also tells us under which conditions the Frequency Principle becomes less pronounced or even fails, which is consistent with the experimental phenomena described above.

Chapter 7

Phase Diagram Analysis

In the discussions of our previous chapters, we revealed an interesting phenomenon of neural network models, namely that the training process satisfies the **frequency principle**. This principle states that, during learning, neural networks possess an intrinsic, implicit preference to first learn the low-frequency components and then transition to high frequencies. Since training does not explicitly require any difference in frequency convergence, this phenomenon can be regarded as an implicitly occurring form of regularization. From a more macroscopic, function-space perspective, it reveals how neural networks gradually grasp and build up an understanding of the data. This insight not only deepens our understanding of the neural network training process, but can also guide us in designing neural network architectures with better performance.

Nevertheless, the frequency principle still only provides a relatively coarse framework of understanding. It allows us to grasp at a high level the learning tendency of neural networks—namely, prioritizing the low frequencies and then moving to the high frequencies—but it does not provide us with a precise tool to quantitatively study how the network parameters evolve along with the learning process. Furthermore, relying solely on this principle, it is difficult to dig deeply into what makes neural network models unique compared to traditional kernel methods, or to distinguish among different neural network architectures, since they all exhibit a similar low-to-high frequency learning progression.

To depict and understand these differences more finely, we need to dig into the detailed level of the model. In this chapter, we will examine in detail the behavior of fully connected neural networks under different hyperparameter configurations, with particular attention to those hyperparameters that affect parameter initialization.

Popular Science

The phase diagram analysis of neural networks is a method for studying the behavior of neural networks under different initialization conditions. Specifically, phase diagram analysis can help us understand whether a neural network exhibits linear or nonlinear behavior during training. Here, the linearity or nonlinearity refers to whether the model is linear or nonlinear with respect to the parameters. In fact, whether the model is linear in its parameters determines the complexity of the problem.

If the model is linear in its parameters (for example, linear regression models, random feature models, kernel methods), then the loss function (such as the mean squared error, MSE) is typically a convex function. In this case, the optimization problem is relatively simple, since the loss function has a unique global minimum and no local minima. However, if the model is nonlinear in its parameters, the loss function is non-convex, which makes the optimization process much more complex.

Why do we develop phase diagram analysis for neural networks?

- **Phenomenon-driven:** Experiments show that the scale of parameter initialization has a significant impact on the function ultimately learned by the model. However, the frequency principle of neural networks only offers a coarse framework of understanding, and it is difficult to quantitatively analyze the concrete evolution of network parameters during the learning process. To depict and understand these differences more finely, we need to delve deeper into the details of the model at the parameter level.

- **Understanding the learning mechanism:** It is very important to identify the hyperparameter regions in which the training process of neural networks is nonlinear. If theoretical analysis only focuses on the regions where the training process of neural networks is approximately linear, then we cannot explain why other linear models (such as kernel methods) have not achieved the same kind of huge success as neural networks.

What does the phase diagram of neural network training dynamics actually look like? What is its relationship to initialization?

In thermodynamic systems, describing a phase requires identifying suitable state variables, such as the temperature and pressure of water; these state variables can vary independently and clearly distinguish among different phases. Inspired by this, we raise the question: can we define analogous state variables for neural networks to determine the state they are in—linear or nonlinear? More specifically, under suitably defined state variables, can we draw a phase diagram of neural networks analogous to the water phase diagram (the diagram of changes among the three states of water: solid, liquid, and gas)?

The answer is yes. The phase diagram of a neural network is a chart that shows the behavior of the network under different initialization conditions. In the phase diagram of two-layer ReLU neural networks, we use two state-variable parameters (γ and γ') to describe the initialization conditions, and show whether the training behavior of the network under these initialization conditions is linear or nonlinear. Here, the state-variable parameter γ characterizes the magnitude of the output function at initialization, and the state-variable parameter γ' characterizes the magnitude difference between the inner-layer and outer-layer weights at initialization.

The initialization conditions have an important impact on the training process of neural networks. Specifically, the scale of initialization determines the initial values of the network parameters, and these initial values affect the evolution of the network during training.

- **Linear regime:** When the initialization is large, the neural network can already fit the data well near initialization, the relative range of variation of the parameters is very small, and therefore the model can be approximated near initialization by a first-order Taylor expansion, and the network exhibits linear behavior.

- **Condensed regime:** When the initialization is small, the neural network cannot fit the data well near initialization, the relative range of variation of the network parameters is large, and therefore the network cannot be approximated by a linear model and exhibits highly nonlinear behavior. In experiments, we find that the nonlinear behavior is highly correlated with the "condensation phenomenon" of neurons (i.e., many neurons exhibit converging behavior), and hence we call this region the condensed regime. This has also inspired a series of subsequent studies on the condensation phenomenon.
- **Critical regime:** The boundary between the linear and condensed regimes, in which the network exhibits an intermediate level of nonlinear behavior.

Through experiments and theoretical analysis, we can draw a phase diagram showing the behavior of the network under different initialization conditions. For example, one part of the diagram indicates that the network behaves linearly under those conditions, while another part indicates that the network exhibits either condensed or critical behavior under those conditions. This is the phase diagram of neural networks.

What is phase diagram analysis of neural networks good for?

- **Understanding and optimizing the training process:** Phase diagram analysis can help us understand and optimize the training process of neural networks. From the phase diagram, we can determine whether the network behaves linearly or nonlinearly under different initialization conditions, and hence choose the desired initialization conditions to improve the training efficiency and performance of the model.
- **Inspiring subsequent research:** Phase diagram analysis can inspire a series of subsequent studies. For example, we can further investigate: why does the condensation phenomenon arise in the nonlinear regime of neural networks? What is the relationship between condensation and feature learning? What impact does condensation have on understanding the optimization and generalization of neural networks?

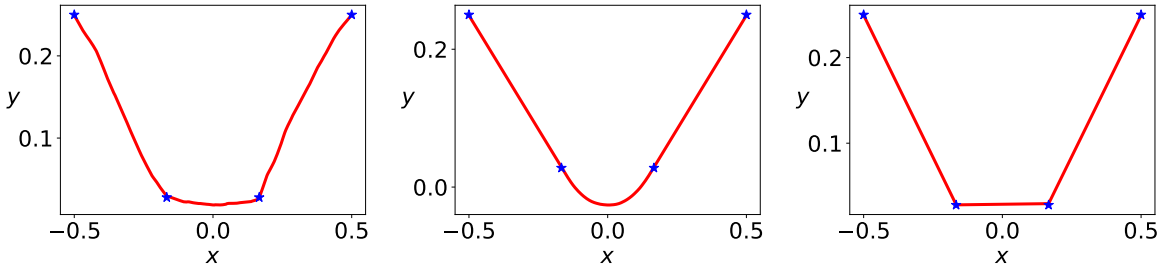
7.1 Behavior of neural networks under different initializations

To observe the behavior of neural networks under different initialization scales, let us start with a simple experiment. As shown in Fig. 7.1, we use a simple two-layer neural network to fit a set of simple data points (the blue stars in Fig. 7.1). The expression of the network is:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{k=1}^{m} a_k \sigma(\boldsymbol{w}_k^\top \boldsymbol{x}),$$

where the width of the hidden layer is $m = 1000$, and we set the initializations of a_k and $\boldsymbol{w}_k$ to follow Gaussian distributions with zero mean and different variances. Throughout this text, we refer to **the size of the variance of the Gaussian distribution as the initialization scale of the parameters**.

In this experiment, we observe that the scale of parameter initialization has a significant impact on the function that the model ultimately learns. From Fig. 7.1(a), we can see that when the



(a) Initialization scale: $\frac{1}{\sqrt{m}}$ (b) Initialization scale: $\frac{1}{m}$ (c) Initialization scale: $\frac{1}{m^{1.75}}$

Figure 7.1: Results of two-layer ReLU neural networks (width $m = 1000$) with different initialization scales fitting four data points.

parameter initialization is relatively large, the function learned by the neural network exhibits mild sawtooth-like fluctuations. In Fig. 7.1(b), as the initialization is reduced, the function learned by the neural network becomes progressively smoother, resembling a cubic spline. Finally, in Fig. 7.1(c), we observe that when the parameter initialization is further reduced, the model tends to learn a function that is almost a linear interpolation. This reflects the subtle yet critical role played by the scale of parameter initialization in the learning outcome.

Although the experiment shown in Fig. 7.1 is simple, it suggests that the behavior of neural networks can differ under different hyperparameter configurations, especially those hyperparameters that affect parameter initialization. In the remainder of this chapter, we will analyze how these hyperparameters influence the behavior of the network during training and the final learning result. In Section 7.2, we will introduce the linear and nonlinear behaviors of neural networks, and present an important phenomenon associated with the nonlinear behavior—condensation.

7.2 Linear and nonlinear behaviors of neural networks

The frequency principle mentioned in the previous chapters focuses on characterizing the evolution in function space, but the essence of neural network training lies in the evolution in parameter space. Therefore, to gain a more refined understanding of the intrinsic mechanism of neural networks, we will focus on the evolution of the parameters, and distinguish the linear and nonlinear behaviors of neural networks by the changes in the parameter weights.

7.2.1 Analysis of parameter evolution

Consider a neural network with a two-layer structure and the ReLU activation function, whose mathematical expression can be written as:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{k=1}^{m} a_k \mathrm{ReLU}(\boldsymbol{w}_k^\top \boldsymbol{x}), \tag{7.1}$$

where $\boldsymbol{x} \in \mathbb{R}^d$ denotes the input vector of the model. The parameter set $\boldsymbol{\theta}$ of the network can be expressed via the vectorization operation $\mathrm{vec}(\cdot)$ as $\boldsymbol{\theta} = \mathrm{vec}(\boldsymbol{\theta}_a, \boldsymbol{\theta}_{\boldsymbol{w}})$, where $\boldsymbol{\theta}_a = \mathrm{vec}(\{a_k\}_{k=1}^m)$ and $\boldsymbol{\theta}_{\boldsymbol{w}} = \mathrm{vec}(\{\boldsymbol{w}_k\}_{k=1}^m)$ represent the output-layer weights and the input-layer weights, respectively. Their initial values $(a_k^0, \boldsymbol{w}_k^0)$ are initialized via Gaussian distributions $a_k^0 \sim \mathcal{N}(0, \beta_1^2)$ and $\boldsymbol{w}_k^0 \sim \mathcal{N}(0, \beta_2^2 \mathbf{I}_d)$. In Eq. 7.1, the bias term b_k can be incorporated into the model by expanding the input $\boldsymbol{x}$ and the weights $\boldsymbol{w}_k$ to $(\boldsymbol{x}^\top, 1)^\top$ and $(\boldsymbol{w}_k^\top, b_k)^\top$, respectively.

To explore the details of how the network parameters evolve in space under different initialization conditions shown in Fig. 7.1, we note that the ReLU activation function is positively homogeneous, i.e., $\sigma(ax) = a\sigma(x), \forall a > 0$. Hence, we can decompose the parameters $(a_k, \boldsymbol{w}_k)$ of each neuron into a unit direction vector $\hat{\boldsymbol{w}}_k = \boldsymbol{w}_k / \|\boldsymbol{w}_k\|_2$ and an amplitude $A_k = |a_k| \|\boldsymbol{w}_k\|_2$ representing the neuron's contribution to the output, i.e., $(\hat{\boldsymbol{w}}_k, A_k)$.

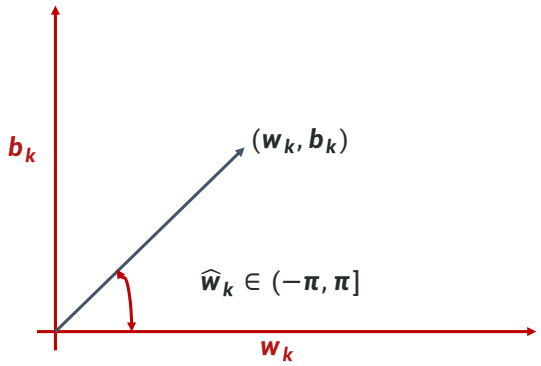


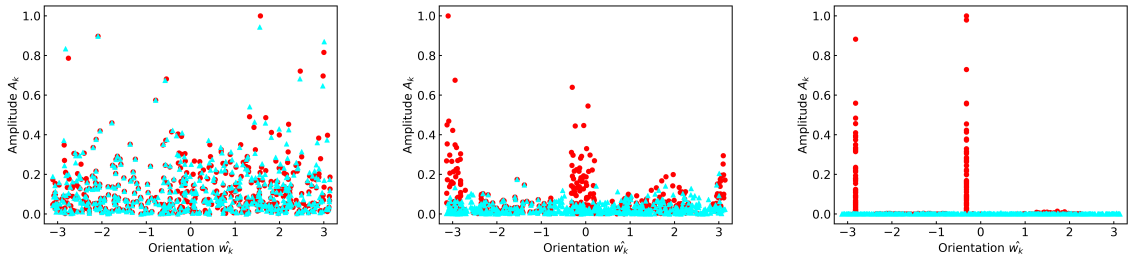
Figure 7.2: Geometric intuition of the directional feature $\hat{\boldsymbol{w}}_k = \boldsymbol{w}_k / \|\boldsymbol{w}_k\|_2$ of a neuron's parameters.

In the experiment shown in Fig. 7.1, we use two-layer ReLU neural networks (width $m = 1000$) with different initialization scales (Gaussian distributions with zero mean and different variances) to learn four data points. Since the input is one-dimensional and the model contains a bias term, the weight vector $\boldsymbol{w}_k := (w_k, b_k)^\top$ is in fact two-dimensional. Therefore, we can represent the direction of each $\hat{\boldsymbol{w}}$ by its angle with respect to the x-axis in $[-\pi, \pi)$, as shown in Fig. 7.2. Based on this representation $(\hat{\boldsymbol{w}}_k, A_k\}_{k=1}^m$, we compare the final learned parameters of the model (red dots) with the initial parameters of the model (cyan triangles) under different initialization conditions, as shown in Fig. 7.3. Each panel is normalized by the largest amplitude among all points in the panel, so that the largest amplitude is 1. For example, in Fig. 7.3(c), the final amplitudes are much larger than the initial ones, so after normalization the initial amplitudes are almost zero.

From Fig. 7.3, we can observe two notable phenomena:

(i) When the initialization is relatively large, the parameters learned by the network barely differ from the initial parameters (in Fig. 7.3(a), the cyan triangles and the red dots almost coincide), whereas as the initialization is reduced, the difference between the learned parameters and the initial parameters gradually increases (in Fig. 7.3(b) and Fig. 7.3(c), the distributions of the cyan triangles and the red dots are clearly distinguishable).

(ii) Under smaller initial parameter settings, the active neurons with significant amplitudes tend to concentrate along specific directions. This phenomenon appears as moderate "condensation" in Fig. 7.3(b), and as the initialization is further reduced, Fig. 7.3(c) shows strong



(a) Initialization scale: $\frac{1}{\sqrt{m}}$ (b) Initialization scale: $\frac{1}{m}$ (c) Initialization scale: $\frac{1}{m^{1.75}}$

Figure 7.3: Scatter plots of the initial states (cyan triangles) and the final states (red dots) of neural networks with different initialization scales fitting four data points.

"condensation" behavior. Here, the two condensed directions in Fig. 7.3(c) correspond exactly to the two ReLUs that combine to form the output function in Fig. 7.1(c). In this case, the neural network can be effectively equivalent to a network with only two effective neurons.

These observations reveal to us that the scale of parameter initialization is very important for the learning behavior of neural networks, and we will study these behaviors carefully in the following parts.

7.2.2 Defining linear and nonlinear behavior

When discussing the dynamical behavior of neural networks, it is crucial to understand the network's linear and nonlinear behaviors. Although the neural network model $f_{\boldsymbol{\theta}}$ is nonlinear with respect to its parameters $\boldsymbol{\theta}$, in certain situations, when the variation of $\boldsymbol{\theta}$ is extremely small, we can use a linear model to approximate this nonlinear model. Specifically, at any training time t, if the variation of $\boldsymbol{\theta}$ is sufficiently small, then the neural network model $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ can be well approximated by the following linear model $f_{\boldsymbol{\theta}}^{\text{lin}}(\boldsymbol{x})$:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) \approx f_{\boldsymbol{\theta}}^{\text{lin}}(\boldsymbol{x}) = f_{\boldsymbol{\theta}(0)}(\boldsymbol{x}) + \nabla_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}(0)}(\boldsymbol{x}) \cdot (\boldsymbol{\theta}(t) - \boldsymbol{\theta}(0)). \tag{7.2}$$

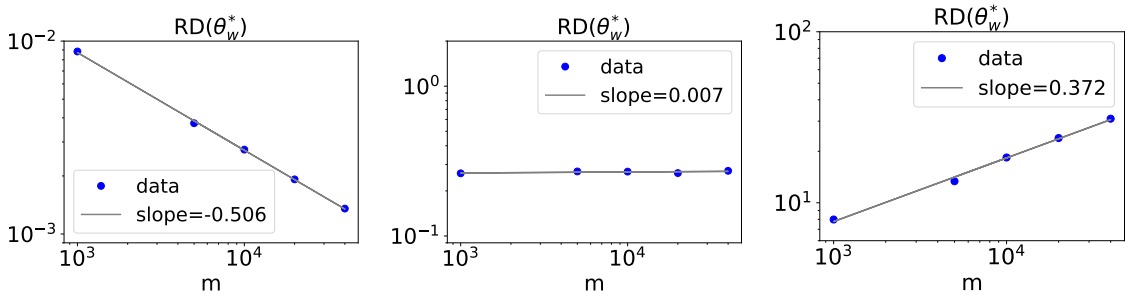
This linear approximation is based on a first-order Taylor expansion, and is usually valid only when $\boldsymbol{\theta}(t)$ stays in a small neighborhood of $\boldsymbol{\theta}(0)$. For two-layer neural networks, since the model $f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{k=1}^{m} a_k \sigma(\boldsymbol{w}_k \cdot \boldsymbol{x})$ is always linear in the output-layer parameters a_k, the requirement of staying in a small neighborhood applies mainly to the input weights $\boldsymbol{\theta}_{\boldsymbol{w}}(t)$, i.e., $\boldsymbol{\theta}_{\boldsymbol{w}}(t)$ needs to remain in a small neighborhood of $\boldsymbol{\theta}_{\boldsymbol{w}}(0)$. Therefore, we refer to the region where this linear approximation is valid as the **"linear regime"**.

Whether a neural network can be accurately approximated by a linear model depends on how much the input-weight parameters $\boldsymbol{\theta}_{\boldsymbol{w}}(t)$ change during training. To quantify this change, we define the **relative distance** (RD) to measure how far $\boldsymbol{\theta}_{\boldsymbol{w}}(t)$ deviates from its initial value $\boldsymbol{\theta}_{\boldsymbol{w}}(0)$ during training:

$$\text{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}(t)) = \frac{\|\boldsymbol{\theta}_{\boldsymbol{w}}(t) - \boldsymbol{\theta}_{\boldsymbol{w}}(0)\|_2}{\|\boldsymbol{\theta}_{\boldsymbol{w}}(0)\|_2}. \tag{7.3}$$

Specifically, we focus on $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$, which represents the relative deviation of $\boldsymbol{\theta}_{\boldsymbol{w}}(t)$ over the entire training process from its initial value to its final learned state, where $\boldsymbol{\theta}_{\boldsymbol{w}}^* := \boldsymbol{\theta}_{\boldsymbol{w}}(\infty)$ denotes the parameter value at the end of training.

Using this metric, we can quantitatively describe the relative parameter changes observed in the experiment of Fig. 7.3. In practice, we often use networks of different widths to learn the data, and the initialization scale of these networks is typically related to the network width by a power law. Therefore, for the data in Fig. 7.3, we plot the relationship between $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ and the network width m under different initialization scales, as shown in Fig. 7.4. As the network width $m \to \infty$, the relationship between the relative distance $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ and the width m exhibits different trends. For Fig. 7.4(a-c), the five blue dots represent the computed values of $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ on neural networks with $1000, 5000, 10000, 20000, 40000$ hidden neurons, respectively. The gray line is a linear fit of the data points, with the slope indicated in the label.



(a) Initialization scale: $\frac{1}{\sqrt{m}}$ (b) Initialization scale: $\frac{1}{m}$ (c) Initialization scale: $\frac{1}{m^{1.75}}$

Figure 7.4: Relationship between the relative distance $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ and the width m as the network width $m \to \infty$.

From Fig. 7.4, we can clearly see that, once the initialization scale is fixed, the relative distance $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ approximately follows a power-law relationship with the network width m. As m tends to infinity, if $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ tends to zero, this indicates that the change of $\boldsymbol{\theta}$ is very small. Based on the first-order Taylor linear expansion above, the behavior of the neural network can then be well approximated by a linear model. In this case, the training dynamics of the network are predominantly linear, and we refer to this region as the **"linear regime"**. Conversely, if this relative distance $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ approaches a constant $O(1)$ or tends to infinity, then the training dynamics of the network exhibit significant nonlinear behavior.

In particular, when $\boldsymbol{\theta}_{\boldsymbol{w}}$ deviates very far from its initial value, i.e., $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ tends to infinity, this corresponds to highly nonlinear dynamics, and in this case we observe a notable condensation behavior in parameter space (as shown in Fig. 7.3(c)). This phenomenon reflects the tendency of the parameters to concentrate along specific directions during training, and hence we refer to this region as the **"condensed regime"**. For the case where $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ tends to a constant $O(1)$, the neural network exhibits intermediate nonlinearity, and we call this region the **"critical regime"**.

In summary, through the analysis of the parameter initialization scale, we find that the network exhibits linear behavior under relatively large initializations, and highly nonlinear behavior under smaller initializations. A salient feature of the nonlinear behavior is the **condensation of neurons**,

which is closely related to the **feature learning** capability of the network, namely the ability of the model to learn certain specific features from the data—those directions along which the parameters condense. This kind of feature learning is widely regarded as a key factor behind the remarkable success of neural networks. If the behavior of a neural network were approximately that of a linear model, we would find it difficult to explain why other linear models, such as kernel methods, have not achieved the same kind of huge success as neural networks. Therefore, the power of neural networks comes from their intrinsic **nonlinearity**, which is the key that distinguishes them from traditional linear models.

7.3 Separating the linear and nonlinear regimes

In the previous experiments, we observed that neural networks may exhibit linear or nonlinear behavior under different initialization conditions. To distinguish linear and nonlinear behavior more precisely, this section attempts to separate the linear regime from the nonlinear regime.

7.3.1 Definition of state variables and dynamical phase transition

We found that the network exhibits linear behavior under relatively large initializations and nonlinear behavior under relatively small ones, which is analogous to phase transitions in physics. In statistical mechanics, in order to observe phase transitions, researchers usually extend the number of particles to infinity. In the experiment shown in Fig. 7.4, as the network width m increases, we observed that $\mathrm{RD}(\boldsymbol{\theta}^*_{\boldsymbol{w}})$ may tend to $0, O(1), +\infty$. Therefore, to make this distinction more explicit, we may similarly consider the limit $m \to \infty$, namely the **dynamical phase transition of infinitely wide neural networks**.

In thermodynamic systems, describing a phase requires identifying suitable state variables, such as the temperature and pressure of water, which can vary independently and clearly distinguish among different phases. Inspired by this, we raise the question: can we define analogous state variables for neural networks to determine the state they are in—linear or nonlinear? More specifically, under suitably defined state variables, can we draw a phase diagram of neural networks analogous to the water phase diagram (the diagram of changes among the three states of water: solid, liquid, and gas)?

We again take two-layer ReLU neural networks as our object of study. Consider a given dataset $S = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, where $\boldsymbol{x}_i \in \mathbb{R}^d$, $i \in [n]$, the network width is m, and the activation function is $\sigma = \mathrm{ReLU}$. Here we introduce a scaling parameter $1/\alpha$ to control the overall magnitude of the output of the sum, which allows us to analyze the common initialization methods of two-layer infinitely wide neural networks in a unified manner:

$$f^\alpha_{\boldsymbol{\theta}}(\boldsymbol{x}) = \frac{1}{\alpha} \sum_{k=1}^m a_k \sigma(\boldsymbol{w}_k^\top \boldsymbol{x}), \tag{7.4}$$

where a_k and $\boldsymbol{w}_k$ are initialized at different scales β_1 and β_2, respectively:

$$a_k^0 \sim N(0, \beta_1^2), \quad \boldsymbol{w}_k^0 \sim N(0, \beta_2^2 \boldsymbol{I}_d). \tag{7.5}$$

Consider the following mean squared error loss function:

$$R_S(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n (f^\alpha_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - y_i)^2. \tag{7.6}$$

Under a gradient-descent-based optimization strategy, in the continuous-time limit, the parameters $\boldsymbol{\theta}$ of the neural network follow the gradient flow dynamics:

$$\frac{\mathrm{d}\boldsymbol{\theta}}{\mathrm{d}t} = -\nabla_{\boldsymbol{\theta}} R_S(\boldsymbol{\theta}). \tag{7.7}$$

This equation describes the trajectory of the parameters in time, pointing in the direction of steepest descent of the loss function. Specifically, for our two-layer ReLU neural network model, the parameters $\boldsymbol{\theta}$ can be expressed as $\boldsymbol{\theta} = \mathrm{vec}(\{\boldsymbol{q}_k\}_{k=1}^m)$, where $\boldsymbol{q}_k = (a_k, \boldsymbol{w}_k^\top)^\top$, $k \in [m]$. The time evolution of the parameters is then described by the following equations:

$$\begin{aligned}
\frac{\mathrm{d}a_k}{\mathrm{d}t} &= -\frac{1}{n}\sum_{i=1}^n \frac{1}{\alpha}\sigma(\boldsymbol{w}_k^\top \boldsymbol{x}_i)\left(\frac{1}{\alpha}\sum_{k'=1}^m a_{k'}\sigma(\boldsymbol{w}_{k'}^\top \boldsymbol{x}_i) - y_i\right) \\
\frac{\mathrm{d}\boldsymbol{w}_k}{\mathrm{d}t} &= -\frac{1}{n}\sum_{i=1}^n \frac{1}{\alpha}a_k\sigma'(\boldsymbol{w}_k^\top \boldsymbol{x}_i)\boldsymbol{x}_i\left(\frac{1}{\alpha}\sum_{k'=1}^m a_{k'}\sigma(\boldsymbol{w}_{k'}^\top \boldsymbol{x}_i) - y_i\right).
\end{aligned}$$

To remove the effects of time and parameter scales, we perform the following rescaling:

$$\bar{a}_k = \beta_1^{-1} a_k, \quad \bar{\boldsymbol{w}}_k = \beta_2^{-1}\boldsymbol{w}_k, \quad \bar{t} = \frac{1}{\beta_1\beta_2}t, \tag{7.8}$$

This rescaled model allows us to establish a normalized basis for comparing different initialization scales and training dynamics. We thus obtain the normalized dynamics:

$$\begin{aligned}
\frac{\mathrm{d}\bar{a}_k}{\mathrm{d}\bar{t}} &= -\frac{\beta_2}{\beta_1}\frac{1}{n}\sum_{i=1}^n \frac{\beta_1\beta_2}{\alpha}\sigma(\bar{\boldsymbol{w}}_k^\top \boldsymbol{x}_i)\left(\frac{\beta_1\beta_2}{\alpha}\sum_{k'=1}^m \bar{a}_{k'}\sigma(\bar{\boldsymbol{w}}_{k'}^\top \boldsymbol{x}_i) - y_i\right), \\
\frac{\mathrm{d}\bar{\boldsymbol{w}}_k}{\mathrm{d}\bar{t}} &= -\frac{\beta_1}{\beta_2}\frac{1}{n}\sum_{i=1}^n \frac{\beta_1\beta_2}{\alpha}\bar{a}_k\sigma'(\bar{\boldsymbol{w}}_k^\top \boldsymbol{x}_i)\boldsymbol{x}_i\left(\frac{\beta_1\beta_2}{\alpha}\sum_{k'=1}^m \bar{a}_{k'}\sigma(\bar{\boldsymbol{w}}_{k'}^\top \boldsymbol{x}_i) - y_i\right).
\end{aligned}$$

Here we note that there are only two independent quantities that are independent of the time scale of the gradient flow dynamics. This motivates us to introduce two scaling parameters:

$$\kappa := \frac{\beta_1\beta_2}{\alpha}, \quad \kappa' := \frac{\beta_1}{\beta_2}, \tag{7.9}$$

where κ and κ' are called the energetic scaling parameter and the dynamical scaling parameter, respectively. κ can be understood as the average output magnitude of each neuron at initialization, while κ' reflects the magnitude difference between a_k and $\boldsymbol{w}_k$, or, from the perspective of their evolution dynamics, κ' reflects the difference in their evolution speeds. Finally, the normalized dynamics can be concisely expressed as:

$$\frac{\mathrm{d}\bar{a}_k}{\mathrm{d}\bar{t}} = -\frac{1}{\kappa'}\frac{1}{n}\sum_{i=1}^n \kappa\sigma(\bar{\boldsymbol{w}}_k^\top \boldsymbol{x}_i)\left(\kappa\sum_{k'=1}^m \bar{a}_{k'}\sigma(\bar{\boldsymbol{w}}_{k'}^\top \boldsymbol{x}_i) - y_i\right), \tag{7.10}$$

$$\frac{\mathrm{d}\bar{\boldsymbol{w}}_k}{\mathrm{d}\bar{t}} = -\kappa'\frac{1}{n}\sum_{i=1}^n \kappa\bar{a}_k\sigma'(\bar{\boldsymbol{w}}_k^\top \boldsymbol{x}_i)\boldsymbol{x}_i\left(\kappa\sum_{k'=1}^m \bar{a}_{k'}\sigma(\bar{\boldsymbol{w}}_{k'}^\top \boldsymbol{x}_i) - y_i\right). \tag{7.11}$$

In what follows, we will focus on the non-dimensionalized model defined by Eqs. (7.10) and (7.11), and for brevity we will drop the "$\bar{\cdot}$" in $\bar{a}_k$, $\bar{\boldsymbol{w}}_k$, $\bar{t}$.

As the network width m tends to infinity, the scales of common initializations all have a power-law relationship with the network width m. To capture this relationship, we define the $\log-\log$ relation of κ and κ' with m as follows:

$$\gamma = \lim_{m\to\infty} -\frac{\log\kappa}{\log m}, \quad \gamma' = \lim_{m\to\infty} -\frac{\log\kappa'}{\log m}. \tag{7.12}$$

These two parameters γ and γ' are independent quantities that do not depend on the time scale, and they are the state variables we will use below to separate the linear and nonlinear regimes. Different initialization methods will cause these parameters to take different values, thereby affecting the training dynamics of the model and the final learning outcome.

To demonstrate this, we list some commonly used initialization methods in the literature, along with their related works and their corresponding scaling parameters. This information is summarized in Table 7.1.

Hyperparameter (related work)	α	β_1	β_2	κ $(\frac{\beta_1\beta_2}{\alpha})$	κ' $(\frac{\beta_1}{\beta_2})$	γ $(\lim_{m\to\infty}\frac{\log 1/\kappa}{\log m})$	γ' $(\lim_{m\to\infty}\frac{\log 1/\kappa'}{\log m})$
LeCun (LeCun et al., 2012)	1	$\sqrt{\frac{1}{m}}$	$\sqrt{\frac{1}{d}}$	$\sqrt{\frac{1}{md}}$	$\sqrt{\frac{d}{m}}$	$\frac{1}{2}$	$\frac{1}{2}$
He (He et al., 2015)	1	$\sqrt{\frac{2}{m}}$	$\sqrt{\frac{2}{d}}$	$\sqrt{\frac{4}{md}}$	$\sqrt{\frac{d}{m}}$	$\frac{1}{2}$	$\frac{1}{2}$
Xavier (Glorot and Bengio, 2010)	1	$\sqrt{\frac{2}{m+1}}$	$\sqrt{\frac{2}{m+d}}$	$\sqrt{\frac{4}{(m+1)(m+d)}}$	$\sqrt{\frac{m+d}{m+1}}$	1	0
NTK (Jacot et al., 2018)	$\sqrt{m}$	1	1	$\sqrt{\frac{1}{m}}$	1	$\frac{1}{2}$	0
Mean-field (Mei et al., 2018) (Sirignano and Spiliopoulos, 2020) (Rotskoff and Vanden-Eijnden, 2018)	m	1	1	$\frac{1}{m}$	1	1	0
E et al. (E et al., 2020)	1	β	1	β	β	$\lim_{m\to\infty}\frac{\log 1/\beta}{\log m}$	$\lim_{m\to\infty}\frac{\log 1/\beta}{\log m}$

Table 7.1: Common initialization methods and their scaling parameters.

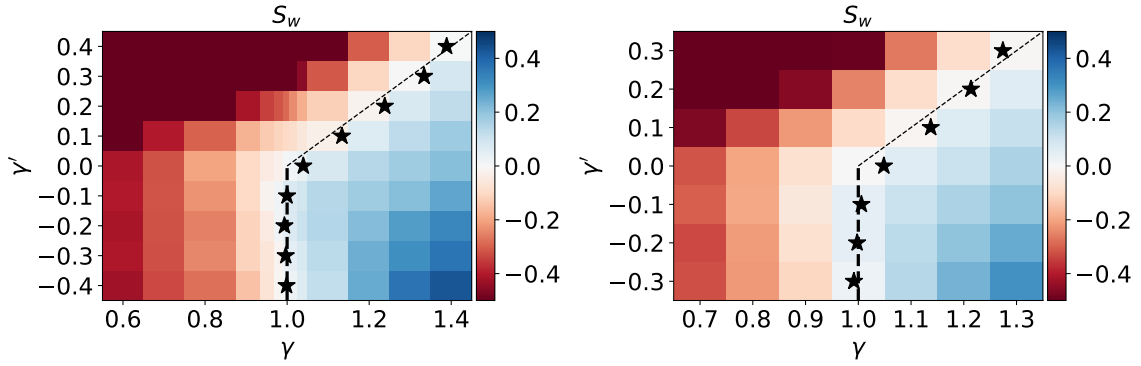
7.3.2 Obtaining the phase diagram experimentally

With the state variables that separate the linear and nonlinear regimes in hand, we can design experiments to obtain a phase diagram of transitions among different phases, especially the boundary that separates the linear regime from the condensed regime. The core of the experiment is to estimate the relative distance $\mathrm{RD}(\boldsymbol{\theta}^*_{\boldsymbol{w}})$ of the network weight updates.

Although ideally we would like to perform experiments at infinite model width m, this is infeasible in practice. We therefore quantify how $\mathrm{RD}(\boldsymbol{\theta}^*_{\boldsymbol{w}})$ changes as the model width m increases. As shown in subplots (a)-(c) of Fig. 7.4, this change approximately follows a power law. Based on this, we define the **scaling exponent** $S_{\boldsymbol{w}}$ of the relative weight-update difference as follows:

$$S_{\boldsymbol{w}} = \lim_{m\to\infty} \frac{\log \mathrm{RD}(\boldsymbol{\theta}^*_{\boldsymbol{w}})}{\log m}, \tag{7.13}$$

which can be obtained by estimating the slope in the log-log plot in Fig. 7.4, thus empirically determining the trend of $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ as $m \to \infty$. If the slope $S_{\boldsymbol{w}} < 0$, then $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ tends to zero as m increases, corresponding to the linear regime; conversely, if $S_{\boldsymbol{w}} > 0$, then $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ tends to infinity as m increases, corresponding to the nonlinear regime.



(a) One-dimensional data

(b) MNIST data

Figure 7.5: Empirical phase diagrams obtained on one-dimensional data and on MNIST data.

Next, we can experimentally obtain the phase diagram empirically by scanning $S_{\boldsymbol{w}}$ over the phase space. For example, for the one-dimensional fitting problem of Fig. 7.1, we show the experimentally obtained phase diagram in Fig. 7.5(a), where $S_{\boldsymbol{w}}$ is estimated on two-layer ReLU neural networks with 1000, 5000, 10000, 20000 and 40000 hidden units. In Fig. 7.5(a), the red zone represents $S_{\boldsymbol{w}}$ less than zero, indicating that $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ tends to zero as $m \to \infty$, characterizing the linear regime. Conversely, in the blue zone, $S_{\boldsymbol{w}}$ is greater than zero and $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ grows to infinity as $m \to \infty$, indicating a highly nonlinear region.

The boundary between these two regions can be approximated through interpolation, indicated by stars in Fig. 7.5(a), where the magnitude of $\mathrm{RD}(\boldsymbol{\theta}_{\boldsymbol{w}}^*)$ is approximately $O(1)$. Such interpolation not only gives us an intuitive partition of the regions, but also provides a clear baseline for further theoretical analysis and experimental design.

To verify whether the phase diagram obtained in Fig. 7.5(a) also applies to real datasets, we trained a two-layer ReLU neural network on the MNIST dataset, with the mean squared error (MSE) as the loss function. $S_{\boldsymbol{w}}$ is estimated on two-layer ReLU neural networks with 1000, 10000, 50000, 250000 and 400000 hidden units. The stars mark the zero crossings obtained by linear interpolation under fixed γ'. The dashed lines represent the boundaries determined by theoretical analysis. In this experimental setup, the input is a 784-dimensional vector representing the pixel values of the image, and the output is a one-dimensional value representing the image label (with values from 0 to 9). As shown in Fig. 7.5(b), we observe that the phase diagram obtained from the one-dimensional data is highly consistent with that obtained from the actual high-dimensional MNIST dataset.

Through these experiments, we have successfully separated the linear and nonlinear regimes and accordingly constructed an empirical phase diagram. In the following parts, we will present the behavior of neural networks in the critical and condensed regimes during actual training, as well as how to use theoretical analysis to precisely characterize the boundary between the linear and

nonlinear regimes.

7.3.3 The critical and condensed regimes

In the previous subsection, through the analysis of Fig. 7.3, we noted the condensation phenomenon of the neural network's representation $\{(\hat{\boldsymbol{w}}_k, A_k)\}_{k=1}^m$ in parameter space, which is a prominent feature of the nonlinear training dynamics of neural networks. Having now obtained the empirical phase diagram, we can also focus on the condensation behavior as the network width m tends to infinity within the phase diagram, namely when the relative variation of $\boldsymbol{\theta}_w$ tends to infinity. We use the same one-dimensional dataset as in Fig. 7.3 and observe the distribution pattern of $(\hat{\boldsymbol{w}}_k, A_k)$. By analyzing two-layer networks of three different widths $m = 10^3, 10^4, 10^6$, we explore the trend as $m \to +\infty$. The results are shown in Fig. 7.6: in each subplot, the colors represent the scatter distribution of $\{(\hat{\boldsymbol{w}}_k, A_k)\}_{k=1}^m$ for the neural network corresponding to that γ and γ'. The numbers of hidden neurons are $m = 10^3$ (blue), 10^4 (red), 10^6 (yellow). The horizontal axis represents γ, and the vertical axis represents γ'. One can clearly see that to the right of the boundary indicated by the blue frame, in the nonlinear regime, the formation of the condensed state becomes more and more pronounced as $m \to \infty$. This matches our expectation that the further $\boldsymbol{\theta}_{\boldsymbol{w}}$ deviates from its initial state, the stronger the nonlinearity of the network becomes. As stated earlier, we define this region as the condensed regime. In the critical regime between the linear and condensed regimes, the degree of condensation appears to remain unchanged as $m \to \infty$, exhibiting behavior similar to that of mean-field theory. At present, the mechanism of condensation and its implicit regularization effect are not yet fully understood, and this remains an important topic for future research.

Furthermore, we also study the condensation behavior of neural networks on the MNIST dataset. Since this is a high-dimensional dataset, we cannot directly visualize the distribution in the high-dimensional feature space. Therefore, we adopt a projection method: we project each $\hat{\boldsymbol{w}}$ onto a reference direction $\boldsymbol{p}$, and plot A_k against $I_{\hat{\boldsymbol{w}}} = \hat{\boldsymbol{w}} \cdot \boldsymbol{p}$. It is worth noting that the choice of the reference direction is arbitrary and does not affect our conclusions. Without loss of generality, we choose the direction $\boldsymbol{p} = \mathbf{1}_n/\sqrt{n}$, where $\mathbf{1}_n$ denotes the n-dimensional vector with all components equal to 1. If neurons indeed condense along certain directions in the high-dimensional feature space, then their one-dimensional projections should also exhibit condensation at certain points. As shown in Fig. 7.7, in each panel, each color represents the scatter distribution of $\{(I_{\hat{\boldsymbol{w}}}, A_k)\}_{k=1}^m$ for the neural network corresponding to that γ and γ'. The numbers of hidden neurons are $m = 10^3$ (blue), 10^4 (red), 2.5×10^5 (yellow). The horizontal axis is γ and the vertical axis is γ'. Similar to the one-dimensional case, we can observe condensation in the condensed regime. As the network width increases, the condensation becomes more pronounced.

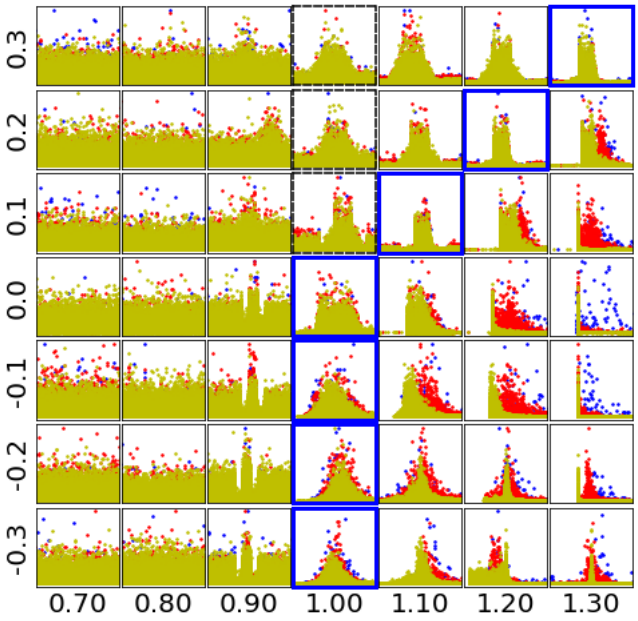


Figure 7.6: Distribution of condensed states on one-dimensional synthetic data.

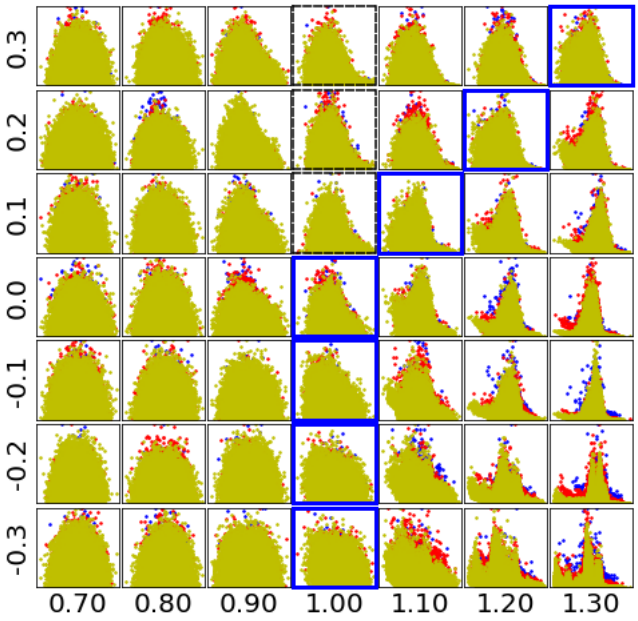


Figure 7.7: Distribution of condensed states on the MNIST dataset.

Chapter 8

Condensation Phenomenon

In our previous exploration of deep learning, we have frequently encountered a number of important concepts whose relationships are not entirely clear, such as non-linearity, the frequency principle, feature learning, the loss landscape, generalization, and complexity. At first glance these concepts may appear isolated; however, upon deeper investigation, we find that they are all closely related to one core phenomenon, which is the "condensation phenomenon".

Our attention to the condensation phenomenon stems from in-depth study and understanding of phase-diagram analysis and the frequency principle. In the phase diagram, the training process of a neural network can be divided into two phases: a linear regime and a non-linear regime. In the linear regime, the learning process of a neural network is equivalent to a kernel method (a method that is linear with respect to the parameters), and can be well characterized by the frequency principle. In contrast, although the behavior of a neural network in the non-linear regime still broadly conforms to the frequency principle, it exhibits more complex features.

In our exploration of the non-linear regime, we found that the condensation phenomenon is ubiquitous. In a neural network, the condensation phenomenon refers to the tendency of neurons in the same layer to become identical. The identity of neurons is essentially the identity of the functions of different neurons (here a neuron's function refers to the mapping from the network input to the output of that neuron).

The content of this chapter will use a series of experiments and theoretical analyses to illustrate the ubiquity of the condensation phenomenon in non-linear training processes, summarized by the following four points: first, the condensation phenomenon does not depend on the width or depth of the network. Whether in infinitely wide neural networks or in commonly used networks of finite width, we can observe the condensation phenomenon. This means that the condensation phenomenon is not limited by the network scale. Second, the condensation phenomenon does not depend on the architecture of the network. Fully-connected networks, convolutional networks, residual networks, etc., all exhibit the condensation phenomenon. Third, the condensation phenomenon is insensitive to the amount of data. Whether the network is over-parameterized or under-parameterized, the condensation phenomenon can occur. Fourth, the condensation phenomenon is insensitive to the loss function and the optimization algorithm.

Behind the ubiquity of the condensation phenomenon lie many structures that guide condensation. In fact, at every stage of neural-network training there are mechanisms that guide condensation. At the initial stage of training, if a small initialization is used, then each neuron follows the

same dynamical equation while the initial points of the neurons are very close to each other, so it is easy for different neurons to become identical. In the middle stage of training, neural networks often pass through the vicinity of certain saddle points. Near these saddle points, one can observe that the output function of the neural network is very similar to the output function of some equivalent smaller network, which is also a way of guiding condensation. We will analyze this phenomenon in detail in the next chapter. At the final stage of training, the global minima of the network have a specific geometric structure, and this geometric structure promotes the occurrence of condensation. Therefore, from the beginning to the end of training, there are structures in the loss landscape of the neural network that guide condensation. In addition, some regularization techniques can also guide condensation, such as dropout.

The condensation phenomenon is closely related to generalization; it is an effective way of controlling complexity. By making neurons identical, condensation makes a wide network equivalent to some narrow network. This reduces the effective complexity of the model, helps prevent the model from over-fitting the training data, and thus improves the generalization ability of the model.

The condensation phenomenon plays an important role in feature learning. Feature learning is a key advantage of deep learning, allowing a model to automatically extract valuable features from raw data. Each neuron in a neural network can be regarded as a feature extractor, and the weights it learns determine the features it extracts. When the condensation phenomenon occurs, the function mappings of neurons in the same layer tend to become identical, meaning that they learn similar features, which captures the important information in the data using as few key features as possible. This both fits the training data and ensures good generalization. In addition, the condensation phenomenon helps enhance the robustness of feature learning: even if some neurons are affected by noise, other neurons can still provide similar features, thereby maintaining model performance and helping improve the robustness and generalization ability of the model. The condensation phenomenon may enable neural networks to learn more effective and more robust features, improving model performance. However, the in-depth relationship between the condensation phenomenon and feature learning remains an active research area, and more theoretical and experimental work is needed to explore it further.

In this chapter, we will first demonstrate the existence of the condensation phenomenon at the experimental level. Then we will introduce one mechanism for realizing condensation—initial condensation—and analyze and explore it in detail.

Popular science section

What is the condensation phenomenon of neural networks?

The condensation phenomenon refers to the tendency of neurons in the same layer of a neural network to become identical during training. For example, multiple neurons may eventually produce very similar outputs for the same input. Although the neurons are initially randomly initialized, they tend to become identical through training. This phenomenon reveals that there is an implicit mechanism inside neural networks that controls the effective complexity of the model, helping us better understand how neural networks operate.

Ubiquity of the condensation phenomenon

The condensation phenomenon occurs ubiquitously during the training of neural networks, as evidenced by the following four points:

- The condensation phenomenon does not depend on the width or depth of the network.
- The condensation phenomenon does not depend on the architecture of the network.
- The condensation phenomenon is insensitive to the amount of data.
- The condensation phenomenon is insensitive to the loss function and the optimization algorithm.

Why is the condensation phenomenon important?

Studying the condensation phenomenon can help us understand many important aspects of neural networks, such as non-linearity, feature learning, and generalization.

- **Understanding the behavior of the non-linear regime:** In studying neural networks, we have found that the training process can be roughly divided into two parts: the linear regime and the non-linear regime. In the linear regime, the learning process of a neural network is much like a traditional linear method (kernel method) and can be well characterized by the frequency principle. However, in the non-linear regime the behavior of the neural network is more complex; the frequency principle can only provide a qualitative description, but the network still follows certain regularities overall—for example, the condensation phenomenon arises at this stage.
- **Understanding the generalization ability of neural networks:** The condensation phenomenon is closely related to generalization; it is an effective way of controlling complexity. By making neurons identical, condensation makes a wide network equivalent to some narrow network. This reduces the effective complexity of the model, helps prevent the model from over-fitting the training data, and thus improves the generalization ability of the model.
- **Understanding feature learning in neural networks:** Each neuron in a neural network can be regarded as a feature extractor, and the weights it learns determine the features it extracts. When the condensation phenomenon occurs, the function mappings of neurons in the same layer tend to become identical, meaning that they learn similar features, which captures the important information in the data using as few key features as possible.

Initial condensation

Under a small initialization, the loss function exhibits a plateau at the very beginning of training. During this plateau stage, the directions of the input weights of the neurons tend to become identical, even when the directions of the input weights are completely random at initialization. We call this phenomenon initial condensation.

- **Experimental verification:** We take a two-layer neural network as an example. Suppose we have the following network architecture:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{j=1}^{m} a_j \sigma(\boldsymbol{w}_j \cdot \boldsymbol{x}), \tag{8.1}$$

where $\sigma(\cdot)$ is the activation function and $\boldsymbol{w}_j$ is the input weight. We choose various activation functions (such as $\tanh(x)$, $x\tanh(x)$, $x^2\tanh(x)$), adopt a small initialization, train the network to fit some target function, and record the behavior of each neuron in the early stage of training. We use cosine-similarity heatmaps to visualize whether condensation occurs in the neural network, as well as the number of condensed directions (see Fig. 8.10).

8.1 Experiments on the condensation phenomenon

In this section, we will explore and demonstrate in depth various experimental results on the condensation phenomenon. These experiments cover different types of neural-network architectures, including fully-connected networks, convolutional neural networks, and residual networks. Starting from the experimental results, we will explore how the condensation phenomenon arises during the training of neural networks.

8.1.1 The condensation process

We first use a one-dimensional example to illustrate how condensation occurs during the training of a neural network. We choose the target function (shown in Fig. 8.1) as

$$f(x) = -\mathrm{ReLU}(x) + \mathrm{ReLU}(2 \times (x + 0.3)) - \mathrm{ReLU}(1.5 \times (x - 0.4)) + \mathrm{ReLU}(0.5 \times (x - 0.8)), \tag{8.2}$$

where $\mathrm{ReLU}(x) = \max\{0, x\}$. We use a two-layer ReLU neural network to fit this function. The width of the neural network is $m = 100$. The model of this neural network is as follows:

$$f_{\boldsymbol{\theta}}(x) = \sum_{k=1}^{m} a_k \mathrm{ReLU}(\boldsymbol{w}_k \cdot \boldsymbol{x}) = \sum_{k=1}^{m} a_k \mathrm{ReLU}(w_k x + b_k), \tag{8.3}$$

where $\boldsymbol{w}_k = (w_k, b_k)$, $\boldsymbol{x} = (x, 1)$, and $\boldsymbol{w}_k \cdot \boldsymbol{x} = w_k x + b_k$.

It is worth noting that for the ReLU activation function, the parameter pair $(a_k, \boldsymbol{w}_k)$ of each neuron can be separated into a unit direction feature $\hat{\boldsymbol{w}}_k = \boldsymbol{w}_k / \|\boldsymbol{w}_k\|_2$ and an amplitude $A_k = |a_k| \|\boldsymbol{w}_k\|_2$ representing its contribution to the output, i.e., $(A_k, \hat{\boldsymbol{w}}_k)$. For one-dimensional inputs, since a bias term is included, $\boldsymbol{w}_k$ is two-dimensional. Therefore, we can represent its direction by the angle $\hat{\Omega}_k$ of each $\hat{\boldsymbol{w}}_k$ within $[-\pi, \pi)$ relative to the x-axis, that is, $\hat{\Omega}_k = \arctan(\frac{b_k}{w_k})$.

First, let us look at how the loss function changes during training (see Fig. 8.2(a)). We find that the loss function exhibits multiple plateaus, which indicates that the training process is highly non-linear. If the model were linear with respect to the parameters—for example, if the model were $wx + b$—then the mean-squared loss would be a quadratic function of w and b, with a unique minimum, i.e. a convex problem. Traditional kernel methods are such convex problems and are relatively well understood; for non-linear problems, the properties are generally more complex.

In order to study in more detail what happens during the non-linear training process, we choose several time points to further observe the details of how parameters change. We choose the initial

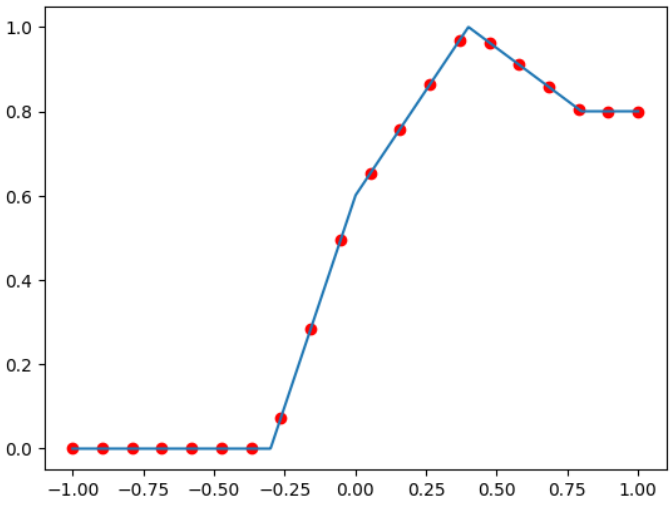


Figure 8.1: Plot of the function in Eq. (8.2)

moment and the six time points during training corresponding to the red dots in Fig. 8.2(a), and plot their training trajectories in Fig. 8.2(b) and Fig. 8.3. These figures show polar scatter plots of $(\hat{\Omega}_k, A_k)_{k=1}^m$ at each training stage. In the polar plot, the radius represents the magnitude of $\log_{10}(A_k)$, and the polar angle $\hat{\Omega}_k$ represents the direction of $\hat{\boldsymbol{w}}_k$. Here A_k is defined as before, and $\hat{\Omega}_k = \text{acrtan}(\frac{b_k}{w_k})$. The dots represent the distribution of $(\hat{\Omega}_k, A_k)$ at the current moment, and the solid lines are the trajectories of the magnitudes of the corresponding neurons evolving over time. The output of the neural network at the corresponding moments is shown in Fig. 8.4, where the blue pentagrams represent the true training data points and the red solid line represents the function output by the model.

As shown in Fig. 8.2(b), at epoch = 0, since the input weights are randomly initialized, the orientations of the neuron input weights are also different from each other, and at this point $\hat{\Omega}_k$ is uniformly distributed over all angles. However, in Fig. 8.3(a) at epoch = 100, the input weights of the neurons with larger amplitudes have already condensed onto a specific direction, and at this point $\hat{\Omega}_k$ is concentrated at the same angle. From Fig. (a) to Fig. (f), we can observe that as training proceeds, the number of directions onto which neurons condense gradually increases, and the magnitudes of neurons in each direction also grow. Figuratively speaking, the neurons gradually "grow" condensed directions. Moreover, the neurons that grow new directions come from two sources. One is from those neurons with very small amplitudes, and the other is from neurons that have already condensed onto some direction with a relatively large amplitude. At the end of training, as shown in Fig. (f), the neurons have grown a total of six directions.

In a two-layer neural network, for an input x, the contribution of a certain neuron to the total output of the network can be expressed as $a_k\text{ReLU}(w_k x + b_k)$. According to the polar coordinate transformation, the two-dimensional weight vector $\boldsymbol{w}_k$ that includes the bias term can be represented by its magnitude $\|\boldsymbol{w}_k\|$ and its direction angle $\hat{\Omega}_k$, that is, $w_k = \|\boldsymbol{w}_k\| \cos\hat{\Omega}_k$ and $b_k = \|\boldsymbol{w}_k\| \sin\hat{\Omega}_k$. Substituting these into the formula for the neuron's output, and using the positive homogeneity of the ReLU function (i.e., for any non-negative scalar $c \geq 0$, $\text{ReLU}(cz) = c\text{ReLU}(z)$), we can factor

out the non-negative magnitude $\|\boldsymbol{w}_k\|$:

$$a_k \mathrm{ReLU}(w_k x + b_k) = a_k \mathrm{ReLU}(\|\boldsymbol{w}_k\| \cos\hat{\Omega}_k \cdot x + \|\boldsymbol{w}_k\| \sin\hat{\Omega}_k) = a_k \|\boldsymbol{w}_k\| \mathrm{ReLU}(\cos\hat{\Omega}_k x + \sin\hat{\Omega}_k)$$

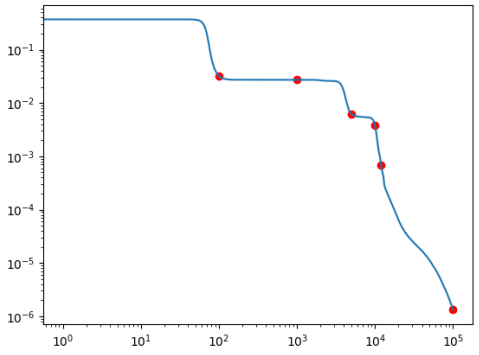
When some neurons (let them form the set S) condense during training so that their angles $\hat{\Omega}_k$ all coincide (denoted by a common $\hat{\Omega}$), the total contribution of these neurons to the network output is:

$$\sum_{k\in S} a_k \mathrm{ReLU}(w_k x + b_k) = \left(\sum_{k\in S} a_k \|\boldsymbol{w}_k\|\right) \mathrm{ReLU}(x\cos\hat{\Omega} + \sin\hat{\Omega})$$

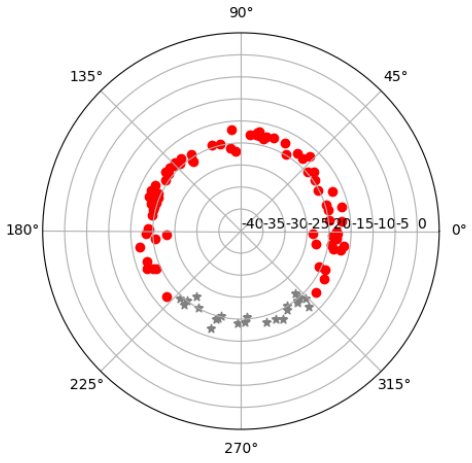
From the above equation, the superposition of this group of neurons is mathematically completely equivalent to a single neuron—whose equivalent input weight is $\cos\hat{\Omega}$, equivalent bias is $\sin\hat{\Omega}$, and equivalent output weight is $\sum_{k\in S} a_k \|\boldsymbol{w}_k\|$.

Therefore, in a two-layer neural network with ReLU as the activation function, if different neurons have the same $\hat{\Omega}_k$, these neurons can be merged, thereby reducing the original network to an equivalent simplified network with fewer neurons.

It should be noted that when $wx + b < 0$ holds for all inputs x, $\mathrm{ReLU}(wx + b)$ remains 0 throughout the entire training process, because the gradient is identically zero and the parameters will not be updated. Under this experimental setup, the neurons marked by pentagrams in Fig. 8.2 satisfy $wx+b < 0$ for all training data, so the outputs of these neurons are always 0. The parameters of these neurons do not change during training, and they do not participate in the condensation process.

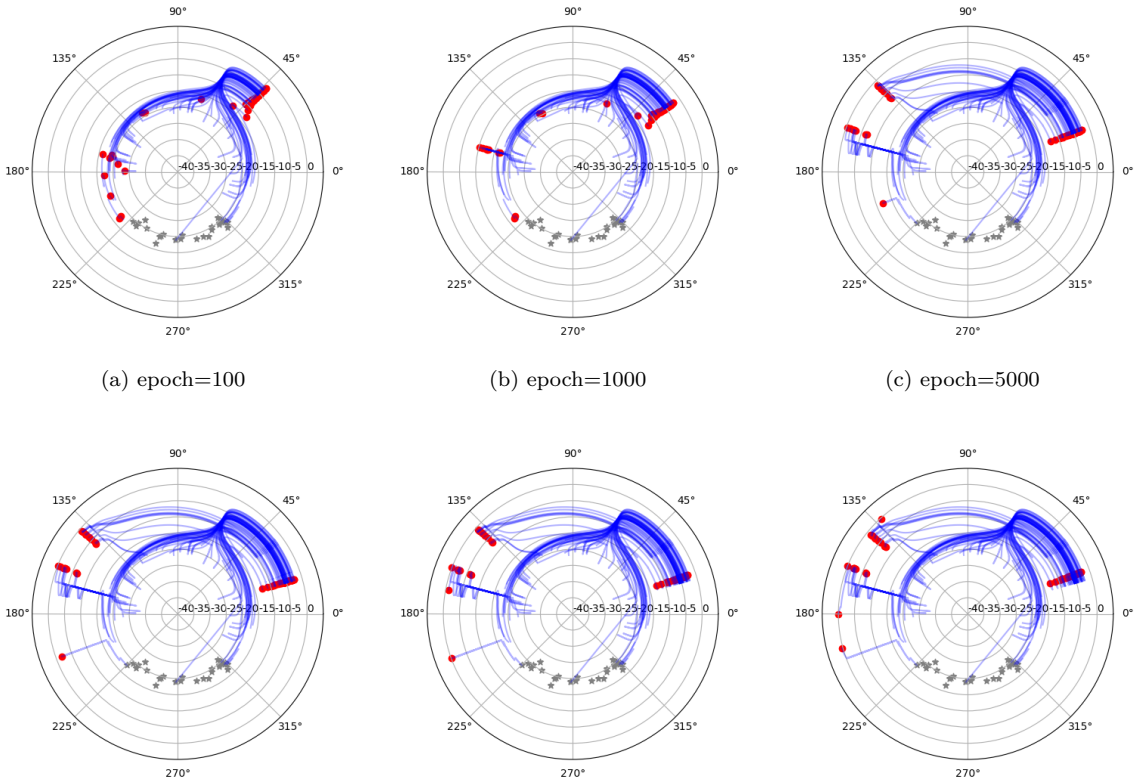


(a) Loss function



(b) epoch= 0

Figure 8.2: Left: the loss function during the training of a two-layer ReLU neural network fitting the function in Eq. (8.2). Right: the distribution of $(\hat{\Omega}_k, A_k)$ at the initial moment of training.



(a) epoch=100 (b) epoch=1000 (c) epoch=5000

(d) epoch=10000 (e) epoch=12000 (f) epoch=100000

Figure 8.3: Time evolution of the $(\hat{\Omega}_k, A_k)$ distribution of the neurons of a two-layer ReLU neural network fitting Eq. (8.2); the radius is plotted on a log scale.

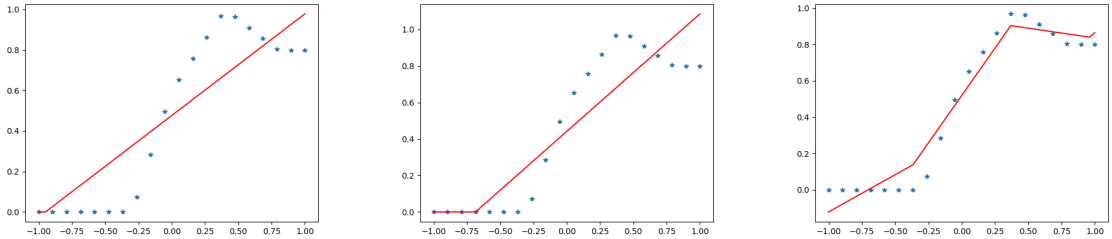
8.1.2 Condensation phenomenon in fully-connected networks

In a fully-connected neural network, the m_{L+1}-dimensional output $\boldsymbol{h}_{L+1}$ of the $(L+1)$-th layer can be expressed as:

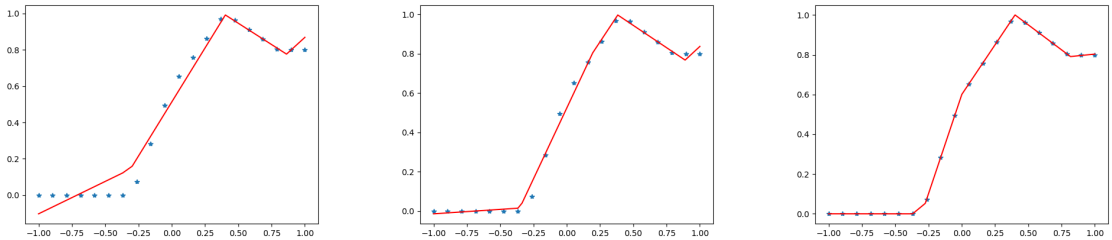
$$\boldsymbol{h}_{L+1}(\boldsymbol{x}) = \sigma(\boldsymbol{W}^{[L]}\boldsymbol{h}_L(\boldsymbol{x}) + \boldsymbol{b}^{[L]}),$$

where $\boldsymbol{W}^{[L]} \in \mathbb{R}^{m_{L+1}\times m_L}$ is the weight matrix, $\boldsymbol{b}^{[L]} \in \mathbb{R}^{m_{L+1}\times 1}$ is the bias vector, and σ is the activation function. We can combine $\boldsymbol{W}^{[L]}$ and $\boldsymbol{b}^{[L]}$ into an augmented weight matrix $\bar{\boldsymbol{W}}^{[L]} \in \mathbb{R}^{m_{L+1}\times(m_L+1)} := (\boldsymbol{W}^{[L]}, \boldsymbol{b}^{[L]})$, so that $\bar{\boldsymbol{W}}^{[L]}$ contains m_{L+1} vectors of dimension $m_L + 1$, and we want to observe whether these vectors exhibit the condensation phenomenon. In some simple cases, such as the condensation of two-layer ReLU networks in the chapter on phase-diagram analysis and the condensation process shown in Sec. 8.1.1, since the input is one-dimensional, one can judge whether condensation occurs by observing the distribution of the neuron direction feature $\hat{\boldsymbol{w}}_k$ in polar coordinates.

For high-dimensional data, polar coordinates cannot be used. In this case, we can use cosine



(a) epoch=100 (b) epoch=1000 (c) epoch=5000



(d) epoch=10000 (e) epoch=12000 (f) epoch=100000

Figure 8.4: Network outputs at the moments corresponding to Fig. 8.3.

similarity to measure the angle between two vectors.

Cosine similarity: The cosine similarity of two vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ is defined as

$$D(\boldsymbol{u},\boldsymbol{v}) = \frac{\boldsymbol{u}^\intercal \boldsymbol{v}}{(\boldsymbol{u}^\intercal \boldsymbol{u})^{1/2}(\boldsymbol{v}^\intercal \boldsymbol{v})^{1/2}}. \tag{8.4}$$

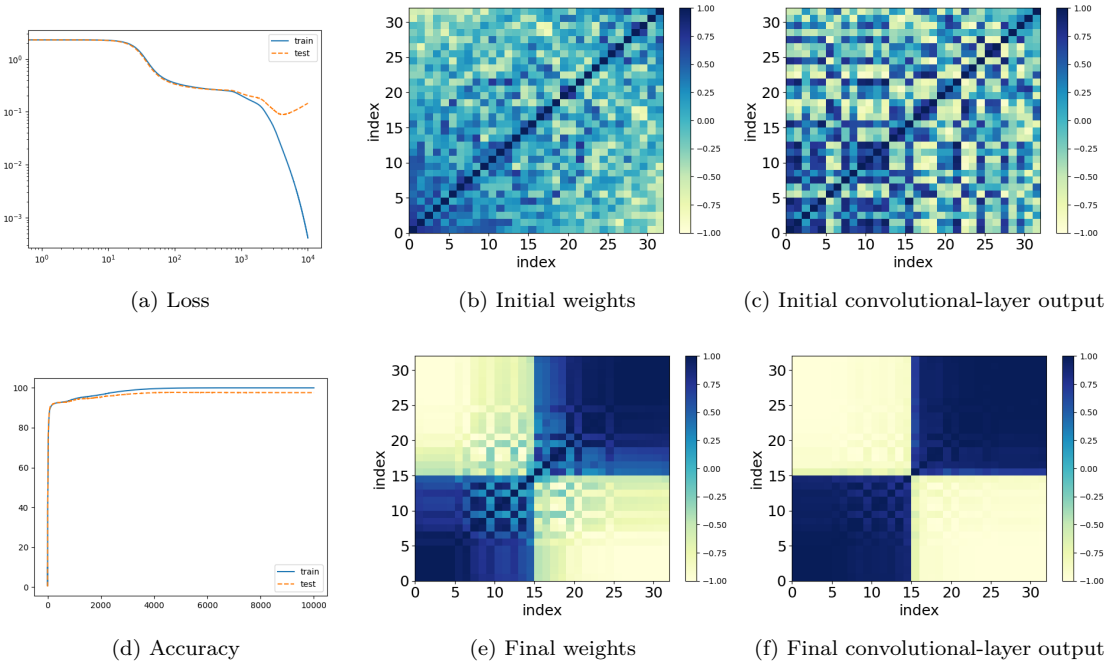
For a set of vectors, we can compute their cosine-similarity matrix. Having obtained this matrix, we sort the neurons according to the magnitudes of the different components of the eigenvector corresponding to its largest eigenvalue. Then, in this order, we plot the cosine-similarity heatmap of the vectors. By inspecting the heatmap, we can detect whether condensation occurs between the vectors.

8.1.3 Condensation phenomenon in convolutional neural networks

As shown in Fig. 8.5, we train a convolutional neural network consisting of just one convolutional layer plus one fully-connected layer. The training data is MNIST (a commonly used small image dataset), the loss function is cross-entropy, and the activation function is tanh(x). The colors in the heatmap represent $D(\boldsymbol{u},\boldsymbol{v})$ between different convolutional kernels. The output layer uses softmax, the optimizer is Adam, and we use full-batch training. The kernel size is $m = 3$ and the learning rate is 2×10^{-4}. Training is stopped when the training accuracy reaches 100%, at which point the test accuracy is 97.62%.

Fig. 8.5(a) and (d) show the error and accuracy during training, respectively. Fig. 8.5(b) and (e) show the cosine-similarity heatmap of the convolutional kernels at the initial time and at the moment when the training accuracy reaches 100%, respectively. The convolutional layer has 32 channels in total, and the kernel size is 3×3, so this is the pairwise cosine similarity between 32 vectors of dimension 9. Fig. 8.5(c) and (f) are obtained by passing all 70000 training and test data through this convolutional layer, yielding a $70000 \times 32 \times 28 \times 28$ four-dimensional tensor. We fix the second dimension and flatten the remaining dimensions to obtain 32 neural-network output vectors of dimension $70000 \times 28 \times 28$, between which we compute cosine similarities.

It can be seen from the figure that, first, there is no condensation relationship between the vectors at the beginning of training, whereas after training is complete, both the convolutional layer and the training data passed through the convolutional layer exhibit a block structure, indicating that the condensation phenomenon has clearly occurred, with condensation approximately onto two opposite directions. Second, the block structure in panel (f) is clearer than that in panel (e), which indicates that the degree of condensation of the convolutional-layer output at the final state is stronger than that of the weights at the final state.



(a) Loss (b) Initial weights (c) Initial convolutional-layer output

(d) Accuracy (e) Final weights (f) Final convolutional-layer output

Figure 8.5: Condensation at the final stage of training of a single-layer CNN under a small initialization (both the convolutional and fully-connected layers are initialized from $\mathcal{N}(0, 96^{-8})$).

8.1.4 Condensation phenomenon in residual neural networks

The condensation phenomenon also occurs in residual neural networks. We take the deep-learning model ResNet18 as an example to demonstrate the condensation phenomenon during its training.

ResNet18 is a convolutional neural network used for vision tasks; it is good at processing data such as images and videos. The network is composed of 18 learnable-parameter layers (17 convolutional layers and 1 linear layer) together with batch-normalization layers, pooling layers, and so on. These layers are organized in a specific manner through residual blocks. Although ResNet18 is relatively small among deep-learning models, it can achieve top-1 accuracy of 73.16% and top-5 accuracy of 91.03% on the ImageNet dataset (data source: `https://huggingface.co/timm/resnet18.a1_in1k`).

In residual neural networks, we handle convolutional kernels in a manner similar to that used for convolutional neural networks; the only difference is that for multi-channel kernels on color images we need to flatten over both the channels and the kernel dimensions. As for the network output, we randomly select 256 training images and 256 test images to form a batch of 512 images, and within this batch we perform a treatment similar to the one used for convolutional neural networks to observe the condensation between the vectors.

In Fig. 8.6, panels (a)(e) and (c)(g) are obtained from the weights of the first convolutional layer and of the last convolutional layer of ResNet18, respectively, expanded along the output channels. The outputs are obtained by merging one batch of training data and one batch of test data, passing them through the convolutional layers of ResNet18, and expanding the resulting features along the channels. The loss function is cross-entropy, the optimizer is Adam, the training batch size is 128, the test batch size is 100, and the learning rate is 10^{-3}. At the end of training, the training accuracy is 99.7% and the test accuracy is 92.5%.

We observe that after training, the weights and outputs of the last layer, as shown in Fig. 8.6(g) and (h), exhibit a fairly clear block structure compared with the initial moment as shown in Fig. 8.6(c) and (d). This indicates that the neurons in this layer exhibit a certain degree of condensation. However, not every convolutional layer exhibits significant condensation. As shown in Fig. 8.6(e) and (f), the weights and outputs of the first layer do not exhibit clear condensation after training.

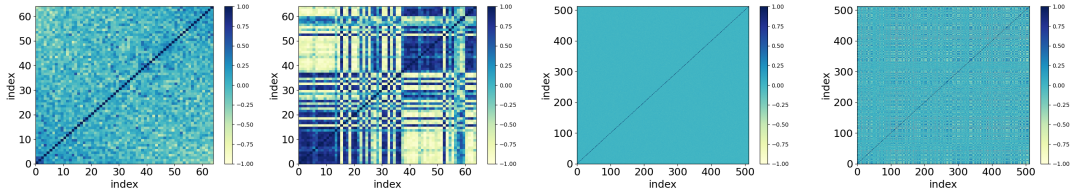
It is worth noting that we also observe similar phenomena in the ResNet-18 model pretrained on ImageNet. As shown in Fig. 8.7(b) and (d), both the weights and outputs of the last convolutional layer exhibit condensation, whereas the weights and outputs of the first layer (as shown in Fig. 8.7(a) and (c)) do not exhibit such clear condensation.

8.2 Discussion of the condensation phenomenon

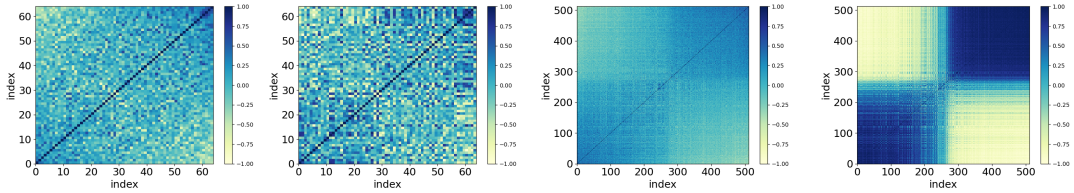
In this section, we will further discuss the condensation phenomenon. We will start from its definition, analyze its causes, and explain the relationship between the condensation phenomenon and generalization.

8.2.1 Definition of the condensation phenomenon

In the previous section, we observed a very important phenomenon: during the training of a neural network, neurons in the same layer exhibit a clear tendency to become identical. This tendency may manifest in the outputs of the neurons, i.e. the responses of the neurons to inputs become identical, or in the weights of the neurons, i.e. their weight vectors become identical during training. We name this tendency of neurons in the same layer to become identical the "condensation phenomenon". In multi-layer networks, the mappings of neurons can be identical even if their input weights are different, which is another common form of becoming identical.



(a) Initial weights of the first layer (b) Initial output of the first layer (c) Initial weights of the last layer (d) Initial output of the last layer



(e) Final weights of the first layer (f) Final output of the first layer (g) Final weights of the last layer (h) Final output of the last layer

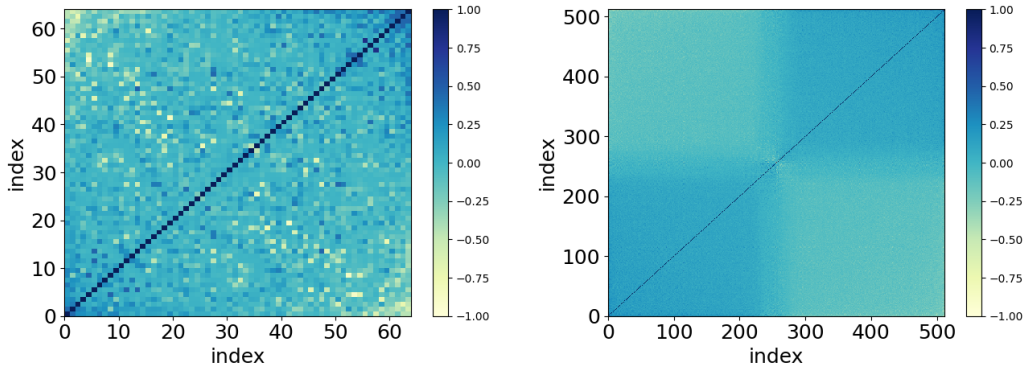
Figure 8.6: Condensation phenomenon during the training of ResNet-18 on the CIFAR10 dataset under the default initialization.

We give an idealized schematic of condensation. As shown in Fig. 8.8, although the input weights of the neurons differ significantly at initialization (the thicknesses and line types of the connections in the schematic differ), after some training the middle hidden neurons split into two classes: the first two neurons form one class and the last three neurons form another class. Within each class, the input weights of different neurons are exactly the same (the corresponding connections have the same type and thickness), so their outputs are also the same. This shows that ideal condensation has occurred. One point that needs to be emphasized is that condensation is a process. Even if ideal condensation is not achieved in the end, as long as the neurons exhibit a tendency to become identical during training, this is also condensation.

8.2.2 Understanding the condensation phenomenon

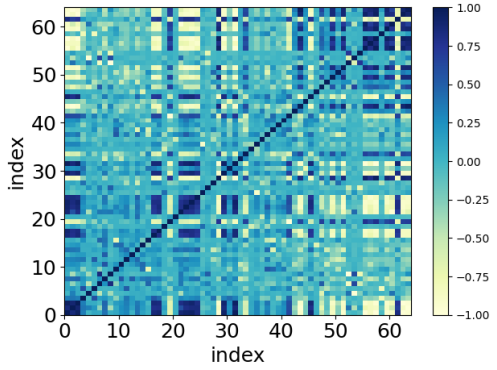
Having confirmed that the condensation phenomenon is ubiquitous, we can explore it further in two directions: one is to analyze the causes of the condensation phenomenon from the perspective of dynamics or the loss landscape; the other is to explore whether we can use this condensation phenomenon to gain a deeper understanding of the generalization performance of neural networks.

Regarding the causes of the condensation phenomenon, we can study it in depth from the perspective of dynamics. During training, the state of each neuron changes over time. This change can be regarded as a dynamical system, whose stability and convergence properties may influence whether neurons exhibit condensation. For example, if the dynamical system of the neurons has some special stable points or attractors, the neurons may tend towards these stable points, and condensation may occur. On the other hand, we can also analyze it from the perspective of the loss landscape. During the training of a neural network, we usually want to minimize some loss function, which defines a loss landscape, and the state of each neuron is searched in this landscape.

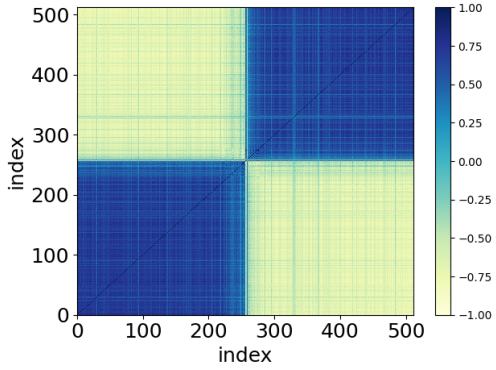


(a) Weights of the first convolutional layer

(b) Weights of the last convolutional layer



(c) Output of the first convolutional layer



(d) Output of the last convolutional layer

Figure 8.7: Condensation phenomenon in the ResNet-18 model pretrained on ImageNet. Source of model parameters: `https://huggingface.co/timm/resnet18.a1_in1k`.

If the loss landscape has some special attractive regions, the neurons may be attracted to these regions, and condensation may occur.

Regarding the impact of the condensation phenomenon on the generalization performance of neural networks, we can discuss it from two aspects. First, condensation means that a neural network can be represented by a smaller neural network. This means that its effective complexity is less than its number of model parameters. If the data itself is not complex, the complexity can be lowered via condensation. On the other hand, if the effective complexity of the neural network is too low, the expressive power of the model can be increased by growing new directions. Therefore, condensation provides an adaptive mechanism for controlling the fitting complexity, which can encourage the neural network to learn a low-complexity fit to the data.

Our research indicates that the condensation phenomenon is a key phenomenon of the non-linear dynamics of neural networks. By studying the condensation phenomenon, we can gain a deeper understanding of the behavior and performance of neural networks.

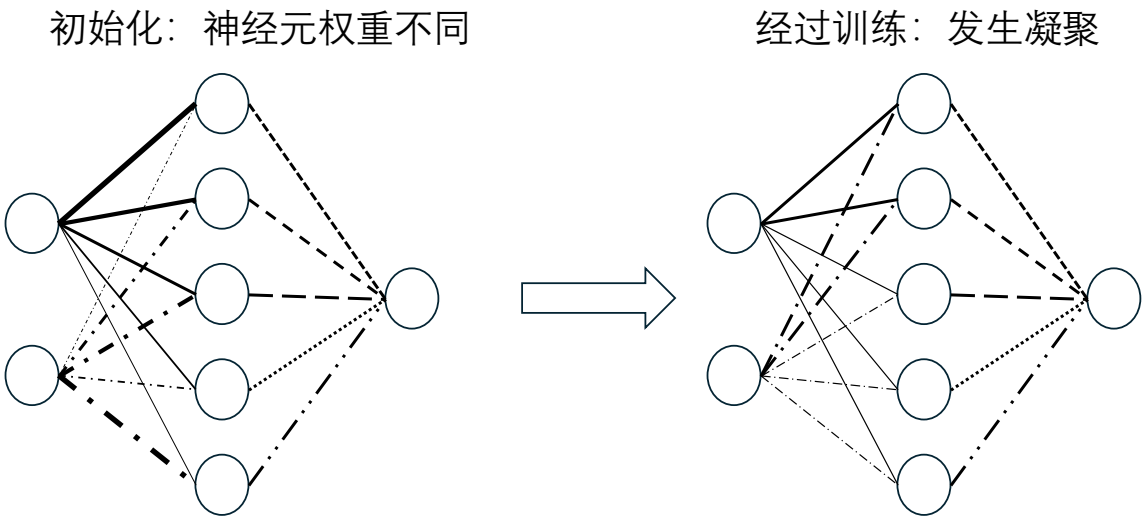


Figure 8.8: Ideal condensation phenomenon.

8.3 Initial condensation

The dynamical behavior at the beginning of training is crucial for the whole training process, because it largely determines the training dynamics of the neural network and the region in which it ultimately settles, which in turn affects the properties of the neural network at the final stage of training. Therefore, studying the behavior at the initial stage of training on its own is an important research direction.

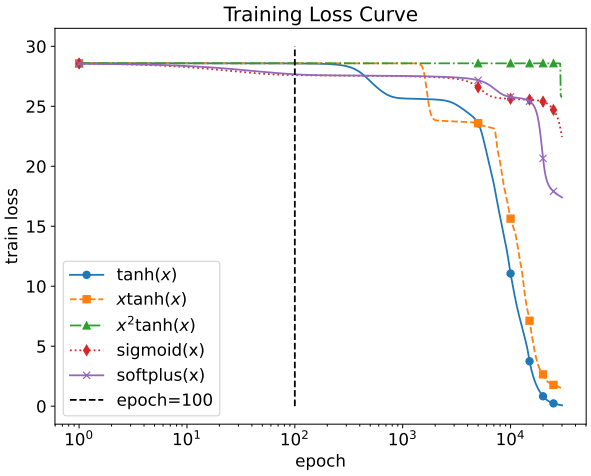
In our study of the phase diagram, we found that when a small initialization is used, the training process of the network exhibits a high degree of non-linearity, with strong condensation features. This condensation manifests itself very strongly already at the initial stage of training. By condensing different neurons onto a few specific features, this initial-stage condensation reduces the network's sensitivity to the randomness in the initialization—that is, different random seeds for the initialization can more easily lead to the same effect. In this section, we will observe the initial condensation phenomenon of neural networks under small initialization, and explore the mechanism by which initial condensation occurs (Zhou et al., 2022).

8.3.1 Experiments on initial condensation

In our experiments, we observe an interesting phenomenon. Although the parameters are initialized completely randomly, at the initial stage of training neural networks undergo a behavior that we call "initial condensation". Specifically, under a small initialization, this phenomenon makes the directions of the input weights of certain neurons become identical, even when their weight directions are completely random at the initial stage. In this section, starting from the experimental results,

we will examine this phenomenon in detail.

Initial condensation occurs at the initial stage of training. So what is the initial stage of training? We know that the point where all parameters equal 0 is a saddle point of a two-layer (multi-layer) neural network. Under a small initialization, the initial values of the neural network are very close to 0, and thus very close to this saddle point, so the loss function will experience a plateau at the very beginning of training, and we call this plateau the initial stage of training. As shown in Fig. 8.9, before epoch= 100 can roughly be regarded as the initial stage of training.



(a)

Figure 8.9: Loss function corresponding to Fig. 8.10.

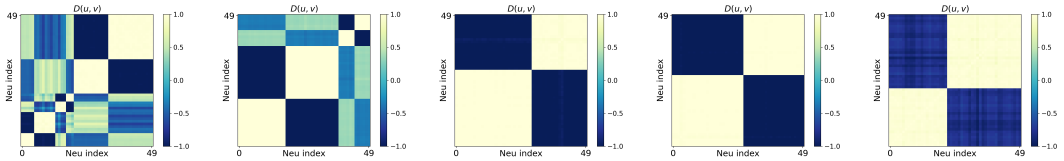
Initial condensation in two-layer neural networks

Fig. 8.10 shows the condensation phenomenon at the initial stage of training. We use a two-layer fully-connected neural network with widths 5-50-1 to fit $n = 80$ training data points sampled from the five-dimensional function $\sum_{k=1}^{5} 3.5\sin(5x_k + 1)$, where $\boldsymbol{x} = (x_1, x_2, \cdots, x_5)^\intercal \in \mathbb{R}^5$, and each x_k is sampled uniformly at random from $[-4, 2]$. At epoch $= 100$, we plot a heatmap to visualize the cosine similarity between the input weights of different neurons. The color intensity of the heatmap represents the magnitude of the cosine similarity, and the horizontal and vertical coordinates represent the indices of the neurons. Panels (a) through (e) use different activation functions, which are given in the captions of the sub-figures. The color represents $D(\boldsymbol{u}, \boldsymbol{v})$ between the input weights of two hidden-layer neurons at epoch 100. The learning rates of sub-figures (a, b, d), (c), and (e) are $\mathrm{lr} = 10^{-3}, 8 \times 10^{-4}, 2.5 \times 10^{-4}$, respectively.

The figure reveals several important phenomena. First, regardless of which activation function we use, the heatmaps in panels (a) through (e) all exhibit clear block structures, indicating that the neurons have condensed onto specific directions. Second, when we change the activation function, the number of directions onto which neurons condense may also change. Specifically, in panels (c),

(d), (e) the neural network condenses onto two opposite directions, i.e. onto one line. In panel (b), the neurons condense onto two lines, for a total of four condensed directions. In panel (a), the neurons condense onto three lines, for a total of six condensed directions.

To summarize, we have observed that, at the initial stage of training, the neurons condense onto specific directions, and the number of condensed directions is related to the activation function of the network.



(a) $x^2\tanh(x)$ (b) $x\tanh(x)$ (c) $\text{sigmoid}(x)$ (d) $\tanh(x)$ (e) $\text{softplus}(x)$

Figure 8.10: Initial condensation of two-layer neural networks.

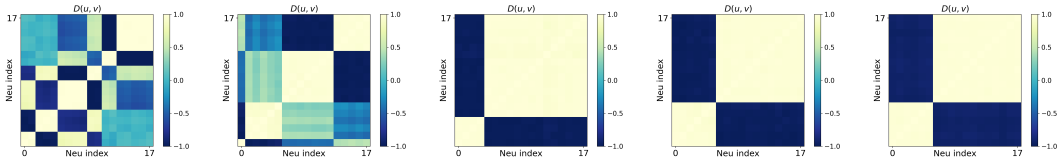
Initial condensation in multi-layer neural networks

Condensation can occur both in multi-layer fully-connected networks and in multi-layer residual networks. Here, we take residual networks as an example to demonstrate the condensation phenomenon. In deep learning, residual connections are often introduced in order to overcome the vanishing-gradient problem. The structure of a residual connection is $\boldsymbol{h}_{l+1}(\boldsymbol{x}) = \sigma(\boldsymbol{W}_l\boldsymbol{h}_l(\boldsymbol{x}) + \boldsymbol{b}_l) + \boldsymbol{h}_l(\boldsymbol{x})$, where $\boldsymbol{h}_l(\boldsymbol{x})$ is the output of the l-th layer. We conduct experiments on a six-layer neural network with residual connections. The training set consists of 80 points sampled from $\sum_{k=1}^{d} 4\sin(12x_k+1)$, where each x_k is sampled uniformly on $[-4,2]$. $n=80$, $d=3$, $m=18$, $d_{\text{out}}=1$, $\text{var}=0.01^2$, $\text{lr}=4\times10^{-5}$. To showcase the differences among various activation functions, we set the activation functions of hidden layers 1 through 5 to $x^2\tanh(x)$, $x\tanh(x)$, $\text{sigmoid}(x)$, $\tanh(x)$, and $\text{softplus}(x)$, respectively. We plot the heatmap of the cosine similarities of the input weights of the neurons within each layer. The experimental results are shown in Fig. 8.11. The steps in the sub-figures are epoch 1000, 900, 900, 1400, and 1400, respectively.

First, in Fig. 8.11, the input weights condense onto three, two, one, one, and one lines in hidden layers 1 through 5, respectively. This indicates that condensation occurs in every layer of the neural network. Second, comparing Fig. 8.11 with Fig. 8.10, we note that although the network architecture, the data points, the layer position, etc., are all different, as long as the activation function is the same, the number of condensed directions of the neurons in that layer is the same. This indicates that there is a very close relationship between the number of condensed directions and the activation function.

8.4 Dropout promotes the condensation phenomenon

Dropout is a technique commonly used for training neural networks (Srivastava et al., 2014), and it can significantly improve the generalization ability of a model . This section will introduce how dropout, through implicit regularization, guides the occurrence of the condensation phenomenon.



(a) layer 1 (b) layer 2 (c) layer 3 (d) layer 4 (e) layer 5

Figure 8.11: Initial condensation of a six-layer neural network with residual connections.

8.4.1 What is dropout?

The basic operating principle of dropout is: for each neuron in the network, with probability p its output is multiplied by $1/p$, and with probability $1-p$ its output is set to zero; this random operation is performed independently during each forward pass.

Specifically, consider the l-th layer of a neural network, whose output is $\boldsymbol{f}_{\boldsymbol{\theta}}^{[l]}(\boldsymbol{x}) \in \mathbb{R}^{m_l}$. When dropout is applied, at the beginning of each training step, we first sample a random vector $\boldsymbol{\eta}$ of length m_l, in which each entry $(\boldsymbol{\eta})_k$ satisfies:

$$(\boldsymbol{\eta})_k = \begin{cases} \frac{1-p}{p} & p \\ -1 & 1-p \end{cases},$$

where $k \in [m_l]$ is the index of $\boldsymbol{\eta}$. It is easy to verify that $\boldsymbol{\eta}$ is a random variable with mean zero. Next, we apply dropout to the output of the l-th layer by the following computation:

$$\boldsymbol{f}_{\boldsymbol{\theta},\boldsymbol{\eta}}^{[l]}(\boldsymbol{x}) = (\mathbf{1}+\boldsymbol{\eta}) \odot \boldsymbol{f}_{\boldsymbol{\theta}}^{[l]}(\boldsymbol{x}),$$

where $\odot$ denotes the Hadamard product, i.e. the element-wise product of two vectors or matrices of the same shape. In the subsequent computations, we use $\boldsymbol{f}_{\boldsymbol{\theta},\boldsymbol{\eta}}^{[l]}(\boldsymbol{x})$ in place of the original $\boldsymbol{f}_{\boldsymbol{\theta}}^{[l]}(\boldsymbol{x})$ as the output of the l-th layer.

For simplicity, we use $\boldsymbol{\eta}$ to denote the dropout vector across all layers of the network. For a given input $\boldsymbol{x}$ and noise $\boldsymbol{\eta}$, the output of the model $\boldsymbol{f}_{\boldsymbol{\theta}}(\boldsymbol{x})$ under dropout is denoted by $\boldsymbol{f}_{\boldsymbol{\theta},\boldsymbol{\eta}}^{\text{drop}}(\boldsymbol{x})$. Correspondingly, the loss function of the network on the training set $S = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^n$ is written as:

$$R_S^{\text{drop}}(\boldsymbol{\theta}, \boldsymbol{\eta}) = \frac{1}{n}\sum_{i=1}^{n} \ell\left(\boldsymbol{f}_{\boldsymbol{\theta},\boldsymbol{\eta}}^{\text{drop}}(\boldsymbol{x}_i), \boldsymbol{y}(\boldsymbol{x}_i)\right) = \mathbb{E}_S \ell\left(\boldsymbol{f}_{\boldsymbol{\theta},\boldsymbol{\eta}}^{\text{drop}}(\boldsymbol{x}), \boldsymbol{y}\right).$$

To intuitively understand the impact of dropout on model training, we first conduct experiments on a simple regression task. Fig. 8.12 shows the fits of a neural network with a large initialization (the setting of the linear regime (Jacot et al., 2018)) with and without dropout. The width of the hidden layer is 1000 and the learning rate is 1×10^{-3} in all experiments. The blue and orange curves in the figure are, respectively, the outputs of the neural networks with and without dropout, and the black dots are the target points. We can see that without dropout, the model tends to fit the given discrete points with a sharply oscillating curve, which is consistent with what we saw in the phase-diagram analysis. After dropout is added, the curve learned by the neural network exhibits a clear piecewise structure, suggesting that condensation of the internal parameters of the model may have occurred. Inspired by this, in the next section we observe that dropout has a promoting

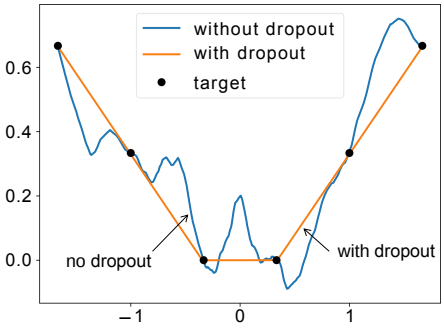


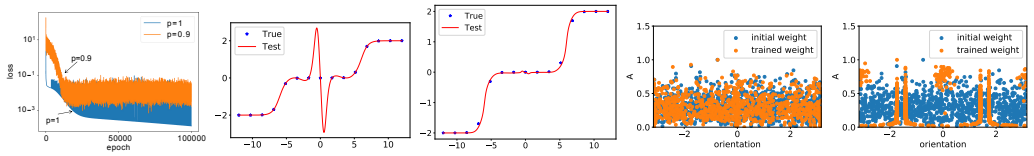
Figure 8.12: Experimental results of training a two-layer ReLU neural network with and without dropout.

effect on condensation, and use this as a starting point to understand the regularizing behavior of dropout.

8.4.2 Dropout promotes condensation of neurons

One-dimensional experiment

In this section we study the role of dropout in promoting condensation of neurons under different settings, and attempt to explain the intrinsic mechanism by which it produces condensation. First, let us consider a simple one-dimensional function-fitting example.



(a) Loss (b) $p = 1$, output (c) $p = 0.9$, output (d) $p = 1$, parameter features (e) $p = 0.9$, parameter features

Figure 8.13: Outputs and parameter features of a neural network with tanh activation under different dropout keep probabilities.

As shown in Fig. 8.13, we take the target function

$$f(x) = \sigma(x - 6) + \sigma(x + 6),$$

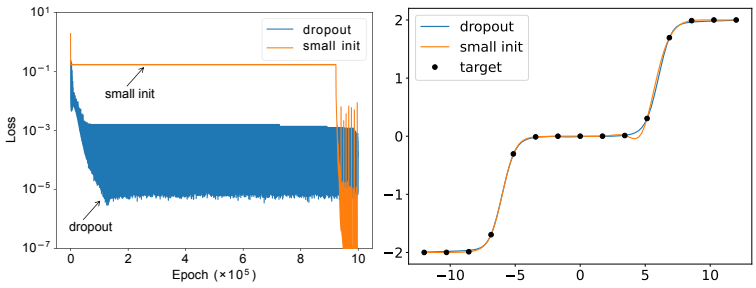
where $\sigma(x) = \tanh(x)$. Using data generated by the above one-dimensional function, we train a tanh neural network with hidden-layer width 1000, using the mean squared error (MSE) as the loss function. The learning rate is fixed to 1×10^{-3} in all experiments. In (d, e), the blue and orange dots represent the weight distributions at the beginning and at the end of training, respectively. In each case we consider the distribution of the input-layer weights. In the three-layer network, we add dropout layers both between the two hidden layers and after the last hidden layer (Fig. 8.13).

To more clearly observe the impact of dropout on the condensation phenomenon, we deliberately use a large parameter initialization. From the previous discussion, we already know that this initialization on its own does not lead to clear condensation. Under the same initialization, with or without dropout, the trained networks can both fit the training data well.

By comparing the training processes, we find that with dropout the network loss does not exhibit a clear plateau (Fig. 8.13(a)), indicating that the parameters are not trapped near a saddle point. In addition, the fitting results show that the network output without dropout has a lot of oscillations (Fig. 8.13(b)), whereas the network with dropout learns a smoother function (Fig. 8.13(c)). To better understand the mechanism of action of dropout, we further observe the distribution features in parameter space.

Using the method from the phase-diagram analysis, we can decompose the parameters $(a_j, \boldsymbol{w}_j)$ of each neuron into a unit direction vector $\hat{\boldsymbol{w}}_j = \boldsymbol{w}_j/\|\boldsymbol{w}_j\|_2$ and an amplitude $A_j = |a_j|\|\boldsymbol{w}_j\|_2$ representing its contribution to the output, i.e. $\{(\hat{\boldsymbol{w}}_j, A_j)\}_{j=1}^m$. We make a scatter plot of the parameter distribution $\{(\hat{\boldsymbol{w}}_j, A_j)\}_{j=1}^m$ of the neural network (Fig. 8.13(d, e)), where the horizontal axis represents the angle of each $\hat{\boldsymbol{w}}_j$ relative to the x-axis and the vertical axis represents the neuron amplitude A_j. For ease of comparison, the parameters of each model are normalized so that the largest neuron magnitude equals 1. We see that without dropout, the weight distribution after training (orange) is close to the initial distribution (blue); by contrast, with dropout, the non-zero weight vectors after training form clear clusters along certain discrete directions, exhibiting a clear tendency to condense.

In what follows, we briefly explain why dropout guides condensation. The basic idea of dropout is that at each training step a portion of the neurons is randomly dropped, and the other neurons in the same layer compensate for this. To give a simple example: if in some training step half of the neurons in a layer are randomly masked, the total output magnitude of that layer is bound to drop significantly. To compensate for this loss, we multiply the outputs of the retained neurons by 2 before performing gradient descent. If training converges to a stable solution, the most ideal case is that the masked and retained neurons are functionally as identical as possible, so that the impact of the masking introduced by dropout on the network behavior is minimized. This state of the network, in which the neurons are identical, is exactly the condensation phenomenon we observe in the experiments.



(a) Loss function (b) Model output

Figure 8.14: Comparison of gradient-descent training under small initialization (orange) and training with a normal initialization but using dropout (blue), in terms of loss and output function.

A natural question is: what is the difference between guiding condensation via small initialization

and via dropout? When we introduced the embedding principle, we observed that under the small-initialization setting, the occurrence of condensation is often accompanied by a plateau in the loss, that is, the loss value lingers near some value for a long time in the curve of loss versus training step. From the perspective of phase-diagram analysis, we need to trade off the size of the initialization. A smaller initialization makes the condensation more pronounced, but at the same time slows down training; while a larger initialization can speed up training but causes the behavior of the neural network to approach that of a random-feature model.

Compared with small initialization, dropout provides a more ideal training scheme: it can induce condensation without significantly increasing training time. For example, in Fig. 8.14(a), we compare the training processes and network outputs of a model trained with dropout under large initialization with those of a model initialized in the condensed regime. We find that the dropout training (blue) reduces the loss to the order of 10^{-5} within about 1×10^5 epochs, which is far less than the number of iterations required by the small-initialization training (orange). At the same time, the model output obtained by dropout training is smoother (Fig. 8.14(b)). By contrast, for a model with large initialization, if dropout is not used, even when the parameter scale is already large enough, the function ultimately learned still exhibits clear oscillations, and the condensation phenomenon does not occur (Fig. 8.13(b)).

8.4.3 Explicit expression of dropout and its implicit regularization

In this section, we will give the explicit form of the implicit regularization effect of dropout, and show that dropout amounts to imposing an L_2-norm constraint on the output of each neuron. In a later section, we will discuss in detail the meaning of this explicit regularization term, in particular its contribution to guiding the condensation of the neural network. For the specific derivation of this explicit expression, please refer to Zhang and Xu (2022).

For ease of analysis, we consider an L-layer ($L \geq 2$) fully-connected network, and assume that the dropout layer appears only after the last hidden layer (i.e. the $L-1$-th layer). At the same time, we choose the mean squared error (MSE) as the loss function. Under these assumptions, we can decompose the expected loss of the model with dropout on the dataset S, $\mathbb{E}_{\boldsymbol{\eta}}(R_S^{\text{drop}}(\boldsymbol{\theta}, \boldsymbol{\eta}))$, into the sum of two terms:

$$\mathbb{E}_{\boldsymbol{\eta}}(R_S^{\text{drop}}(\boldsymbol{\theta}, \boldsymbol{\eta})) = R_S(\boldsymbol{\theta}) + R_1(\boldsymbol{\theta}),$$

where $R_S(\boldsymbol{\theta})$ is the standard empirical risk without dropout, i.e. $R_S(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \ell(\boldsymbol{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}(\boldsymbol{x}_i))$, and $R_1(\boldsymbol{\theta})$ is an additional regularization term brought about by the dropout noise, whose specific form is

$$R_1(\boldsymbol{\theta}) := \frac{1-p}{2np} \sum_{i=1}^{n} \sum_{j=1}^{m_{L-1}} \sum_{k=1}^{m_L} \left(\boldsymbol{w}_{j,k}^{[L]} f_{\boldsymbol{\theta},j}^{[L-1]}(\boldsymbol{x}_i) \right)^2. \tag{8.5}$$

Here $\boldsymbol{w}_{j,k}^{[L]}$ denotes the element in the j-th column and k-th row of the last-layer weight matrix $\mathbf{W}^{[L]}$, i.e. the value in the k-th coordinate of the output weight of the j-th neuron of the last hidden layer, and $f_{\boldsymbol{\theta},j}^{[L-1]}(\boldsymbol{x}_i)$ denotes the output of the j-th neuron of the last hidden-layer output vector $\boldsymbol{f}_{\boldsymbol{\theta}}^{[L-1]}(\boldsymbol{x}_i)$.

8.4.4 Impact of the regularization term on condensation

From the regularization term $R_1(\boldsymbol{\theta})$ in the previous section, we can see that dropout regularization imposes an additional L_2-norm constraint on the output of each neuron. In this section we point out

that this constraint is closely related to the condensation phenomenon. To gain a more intuitive understanding of the mechanism by which $R_1(\boldsymbol{\theta})$ works, we consider a simple two-layer ReLU network. Specifically, we use a two-layer ReLU network of the following form to perform a one-dimensional function-fitting task:

$$f_{\boldsymbol{\theta}}(x) = \sum_{j=1}^{m} a_j \sigma(\boldsymbol{w}_j \cdot \boldsymbol{x}) = \sum_{j=1}^{m} a_j \sigma(w_j x + b_j),$$

where $\boldsymbol{x} := (x, 1)^\intercal \in \mathbb{R}^2$, $\boldsymbol{w}_j := (w_j, b_j) \in \mathbb{R}^2$, and $\sigma(x) = \mathrm{ReLU}(x)$. For simplicity, we set the network width to $m = 2$, and assume that the network can perfectly fit a training set consisting of two data points $\boldsymbol{x}_1, \boldsymbol{x}_2$ generated by the target function $\sigma(\boldsymbol{w}^* \cdot \boldsymbol{x})$, denoted by $\boldsymbol{o}^* := (\sigma(\boldsymbol{w}^* \cdot \boldsymbol{x}_1), \sigma(\boldsymbol{w}^* \cdot \boldsymbol{x}_2))$. We further assume that $\boldsymbol{w}^* \cdot \boldsymbol{x}_i > 0, i = 1, 2$. Denote the output of the j-th neuron on the samples by

$$\boldsymbol{o}_j = (a_j \sigma(\boldsymbol{w}_j \cdot \boldsymbol{x}_1), a_j \sigma(\boldsymbol{w}_j \cdot \boldsymbol{x}_2)).$$

After sufficient training, the output of the network should equal the target values on the training data, i.e.

$$\boldsymbol{o}^* = \boldsymbol{o}_1 + \boldsymbol{o}_2.$$

In fact, since the degrees of freedom of the neural network are more than 2, there are infinitely many combinations of $\boldsymbol{o}_1$ and $\boldsymbol{o}_2$ that can fit $\boldsymbol{o}^*$ well. However, for networks using the dropout regularization trick, the additional $R_1(\boldsymbol{\theta})$ term guides the training process to converge to a particular solution. Noting that

$$\|\boldsymbol{o}_j\|_2^2 = \sum_{i=1}^{2} (a_j \sigma(\boldsymbol{w}_j \cdot \boldsymbol{x}_i))^2,$$

$R_1(\boldsymbol{\theta})$ can be written as

$$R_1(\boldsymbol{\theta}) = \|\boldsymbol{o}_1\|_2^2 + \|\boldsymbol{o}_2\|_2^2,$$

As shown in Fig. 8.15, to minimize $R_1(\boldsymbol{\theta})$, when the model is well trained, the components of $\boldsymbol{o}_j$ perpendicular to $\boldsymbol{o}^*$ need to cancel each other out. Therefore, to minimize the sum of their norms, $\boldsymbol{o}_1$ and $\boldsymbol{o}_2$ must be parallel to $\boldsymbol{o}^*$. At this point we obtain $\boldsymbol{w}_1 // \boldsymbol{w}_2 // \boldsymbol{w}^*$, which is precisely the condensation phenomenon.

Figure 8.15: An intuitive schematic of why the $R_1(\boldsymbol{\theta})$ term promotes the occurrence of the condensation phenomenon.

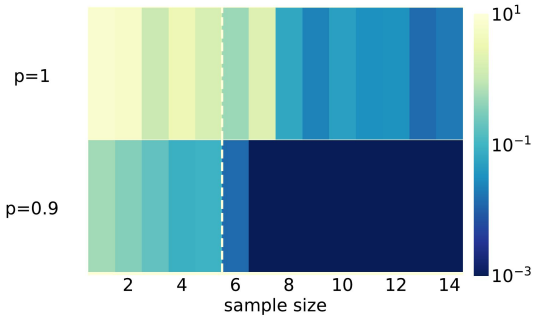


Figure 8.16: Average test error versus sample size of a two-layer tanh neural network under different dropout keep probabilities.

8.4.5 Relationship between dropout and sample size

Although neural networks in the over-parameterized regime can fit and obtain a function with small generalization error, achieving a small generalization error requires a minimum number of samples. Dropout can effectively reduce this minimum-sample-size requirement. To verify this, we take the target function

$$f(x) = \sigma(x - 6) + \sigma(x + 6),$$

where $\sigma(x) = \tanh(x)$. We use a two-layer tanh network with 1000 neurons to learn this two-layer tanh network with two neurons (the teacher–student setting). The number of free parameters in the teacher network is 6, so the minimum sample size required to recover the target function is 6. As shown in Fig. 8.16, in all experiments the width of the hidden layer is 1000, and the learning rate of the Adam optimizer is 1×10^{-4}. Each test error is the average of 10 independent trials with random initialization. When the number of samples is larger than 6, the generalization ability of the model with dropout becomes good, whereas the generalization ability of the model without dropout remains poor. This indicates that dropout can significantly reduce the sample size required to recover the target function in practice, bringing it closer to the optimistic sample size. Therefore, dropout provides a simple and effective method to reduce the sample size needed to recover the target function in actual training, improving the generalization ability of over-parameterized models with a finite sample size. This sample size will be discussed in detail in the chapter on optimistic estimates.

Chapter 9

The Embedding Principle of the Loss Landscape

In our in-depth exploration of the complexity and learning process of neural networks, we encountered a fascinating phenomenon—**condensation of neurons**. This means that different neurons within the same layer tend to converge during training. As discussed in the previous chapter, this condensation phenomenon reveals the collective behavioral pattern exhibited by neurons during nonlinear training. This finding offers us a window through which to explore a series of profound questions.

First, we note that during the training of neural networks, due to the occurrence of condensation, the effective number of neurons is often far below the total size of the network. This raises an intuitive question: if a small-scale network is sufficient to capture the complexity of the target function, why don't we directly train a smaller model? Given that both large and small networks are capable of adequately expressing the target function, what are the essential commonalities and differences between them? Moreover, why do we consistently observe a monotonic increase in the number of condensed directions during training?

Through experimental studies of the nonlinear dynamics of neural networks, we further find that the condensation phenomenon is typically accompanied by the **plateau phenomenon** of the loss function. The plateau of the loss function refers to certain stages of training during which the loss value decreases unusually slowly, which typically indicates that the training trajectory is in the vicinity of a critical point. By a critical point, we mean a point where the gradient vanishes (also called a stationary point), including saddle points, local minima, and global minima.

In a one-dimensional function-fitting problem with a two-layer ReLU neural network, when we place side by side the loss curves obtained from training networks of different widths, we observe a striking regularity. As shown in Fig. 9.1, we find that under small-initialization conditions, although the network width m differs, the loss values plateau at almost the same point, revealing a kind of **width similarity**. This phenomenon prompts us to consider: there may be some essential connection among the critical points of networks of different widths.

However, we must recognize a key issue: networks of different widths have different parameter dimensions. Comparing critical points across spaces of different dimensions is precisely the challenge we face. How can we identify and define similarities among these networks in a high-dimensional parameter space? Specifically, does Fig. 9.1 imply that critical points with the same loss value

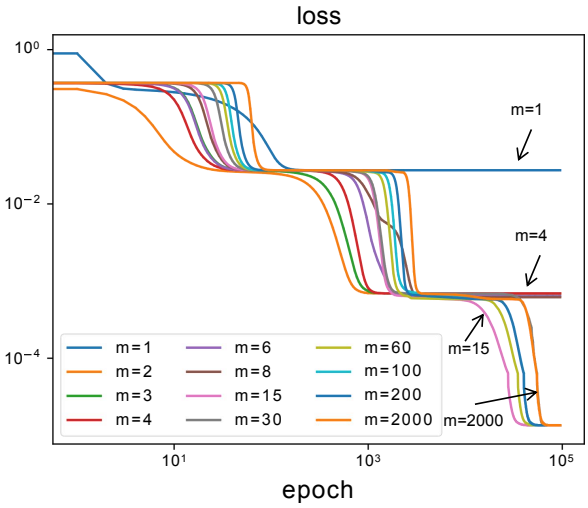


Figure 9.1: Loss curves of fully-connected neural networks of different widths.

always exist across networks of different widths?

In the following sections, we will delve deeper and attempt to answer the questions above. In Sec. 9.1, starting from the phenomenon of width similarity, we will explore how the width of a network affects the learning dynamics and the structure of the loss landscape, and establish the embedding principle of the loss landscape.

Popular Science

What is the embedding principle of neural networks?

The embedding principle of neural networks states that the loss landscape of a network "contains" all critical points of any narrower network. Here a critical point refers to a point where the gradient is zero, which can have a significant impact on the training process. The word "contains" here is not used in the conventional geometric sense, but rather in a functional sense: through a specific embedding operation, every critical point of a narrower network can be embedded into the loss landscape of the wider network while keeping the output function unchanged.

How does the embedding principle work?

- **Experimental verification.** In experiments, when we train neural networks of different widths, we find that their training-loss curves exhibit a notable consistency. Specifically, the loss curve of a wider network plateaus at the same value as that of a narrower network during training, while the output function remains identical. This shows that the wide network encounters critical points embedded from the narrow network.

- **Theoretical support.** The theoretical support of the embedding principle relies on the "splitting" operation of neurons. Through the splitting operation, a network can be embedded into a wider network while preserving the output function and the properties of the critical point. Specifically, one can split a hidden-layer neuron into two: the new neurons have the same input weights as the original, while the output weights are distributed proportionally. Such an operation gradually embeds a narrower network into a wider one while preserving output consistency and the extremal properties of critical points.

Why does the embedding principle matter?

- **Revealing intrinsic connections.** The embedding principle reveals the intrinsic relationships among neural networks of different widths, helping us understand key properties of neural network structure. It explains theoretically that, through appropriate embedding operations, the critical points of a narrower network can exist within the loss landscape of a wider network. This principle is highly general: it does not depend on the specific training data or the choice of loss function, but is closely tied to the layer structure of the network itself.

- **Characterizing the simplicity bias.** The embedding principle is closely related to the F-Principle and the condensation phenomenon, and together they depict the preference of neural networks for simple patterns. Compared with the F-Principle, the embedding principle is more refined: it characterizes behaviors such as the critical points encountered during training. Compared with the condensation phenomenon, the embedding principle characterizes an ideal condensed state—although such an ideal state is not necessarily fully attained in experiments, the network can approach this state through condensation during training.

- **Extending to depth.** The embedding principle can be extended to depth: the loss landscape of a deeper neural network also contains all critical points of any shallower network. Experiments show that even very deep networks plateau during training at the same loss values as shallower networks. This indicates that deep networks encounter critical points embedded from shallower networks. Combined with the embedding principle in width introduced earlier, the embedding principle in depth provides a complete picture of the intrinsic hierarchical structure of the loss landscape of neural networks. This picture strongly supports the experimentally observed similarities in training and generalization across neural networks of varying scales.

What is the embedding principle useful for?

The applications of the embedding principle are manifold:

1. **Optimizing the training process.** The embedding principle helps us understand how wider networks encounter, during training, saddle points whose outputs coincide with the global minima of narrower networks, as well as the properties of the Hessian matrix at these saddle points. Such understanding can be used to optimize the training process and improve model performance.

2. **Model compression and the condensation phenomenon.** The embedding principle is closely related to the condensation phenomenon of networks: a large network can be

compressed into a small network without changing its output function. This has guiding significance for model compression and the efficient deployment of deep-learning models.

3. **Universality across multiple network architectures.** The embedding principle applies to a wide range of common neural-network architectures, such as convolutional neural networks (CNNs), Transformers, and ResNets, and is therefore highly universal.

9.1 Width Similarity and the Embedding Principle

After observing the plateau phenomenon revealed in Fig. 9.1, we are led to a question: when networks of different widths share the same loss value at their plateau points, does this hint at some degree of similarity at the structural level?

9.1.1 Structural similarity at loss-plateau points

In this section, we study in depth the **output behavior** and **internal representation** of neural networks of different widths at specific loss-plateau points. For the output behavior, we compare the output functions of networks of different widths at the same plateau point in order to reveal the external behavior of the networks. As shown in Fig. 9.2(a), we select a specific loss value—the minimum of the loss function for a width-2 neural network—and then compare the output functions of networks of different widths at this loss value.

For the internal representation, we can use a visualization similar to that in previous chapters. Specifically, for a two-layer neural network with a one-dimensional input, as in Eq. (9.1):

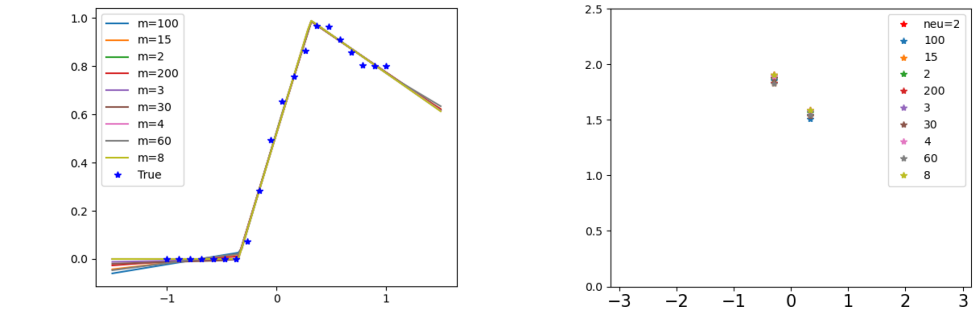
$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{j=1}^{m} a_j \sigma(\boldsymbol{w}_j^\top \boldsymbol{x}) = \sum_{j=1}^{m} a_j \sigma(w_j x + b_j), \tag{9.1}$$

we can probe the internal representation by examining the parameter pair $(a_k, \boldsymbol{w}_k)$ of each neuron. In particular, for the ReLU activation function, we decompose the parameters of each neuron into a unit direction feature $\hat{\boldsymbol{w}} = \boldsymbol{w}/\|\boldsymbol{w}\|_2$ and an amplitude $A = |a|\|\boldsymbol{w}\|_2$ representing its contribution to the output, namely $(A, \hat{\boldsymbol{w}})$. Since the input is one-dimensional, $\boldsymbol{w}$ is in fact a two-dimensional vector that includes the bias term. We therefore use the angle of each $\hat{\boldsymbol{w}}$ relative to the x-axis, in the range $[-\pi, \pi)$, to describe its direction, and use A to represent the amplitude along that direction. Based on this representation, we can sum the amplitudes of neurons with the same direction, thereby forming an "effective neuron," while ignoring those effective neurons whose amplitudes are close to zero. As shown in Fig. 9.2(b), such a visualization reveals the tendency of the internal structure to concentrate.

By analyzing Fig. 9.2, we uncover two key findings:

(i) At plateau points with specific loss values, networks of different widths attain a **high consistency of the output function**. As shown in Fig. 9.2(a), the output functions of networks of different widths almost perfectly overlap, indicating that plateau points with the same loss value are not a superficial coincidence but instead exhibit deep consistency at the level of the output function.

(ii) Independently of the network width, all networks at these specific loss-plateau points display a pronounced **condensation property** of their internal parameters. As shown in Fig. 9.2(b),



(a) Output function of the network

(b) Internal representation of the network

Figure 9.2: Output functions and internal parameter representations of neural networks of different widths at the loss value of the minimum point of the width-2 neural network.

the neurons of networks of different widths condense along two specific directions, and their amplitudes along these directions are also strikingly similar. This condensation property indicates that, at certain stages of training, even a network of large width can achieve effective output behavior through an internal parameter-simplification process, thereby exhibiting a property of **low complexity**.

These observations suggest that a wide neural network may share critical points with a narrow neural network at which the output functions agree, and such critical points typically possess properties of low complexity. Critical points are extremely important structures in the loss landscape, and they have important research value in optimization: we usually hope that the training process can avoid local minima, find a global minimum, and among the many possible global minima, locate those solutions with the best generalization ability. In addition, critical points play a central role during training. For instance, saddle points—one of the key structures in the loss landscape—may cause the training process to plateau in their vicinity and reduce training efficiency, but their minimality along certain dimensions can also attract the training trajectory, providing a sort of navigation that guides the trajectory toward lower values of the loss function, thereby exerting a positive influence on the network's learning path and ultimate generalization ability at a more macroscopic level.

In addition, Fig. 9.2(b) shows that these critical points with consistent output functions resonate with our earlier discussion of the condensation phenomenon: the condensation phenomenon indicates that under appropriate conditions, even a network of large width can, through some internal condensation mechanism, approximate a smaller network in its output behavior. These findings motivate us to dig deeper into the essence of this interconnection.

9.1.2 Theoretical framework: the embedding principle

When we place these findings within the macroscopic perspective of the loss landscape, we arrive at an embedding principle for deep neural networks (for a rigorous proof, see Zhang et al. (2021a)):

Embedding Principle: the loss landscape of a neural network "contains" all critical points of any narrower network.

Here "narrower network" refers to a network of the same depth whose width in each layer is no greater than that of the target neural network. The word "contains" is not used in the conventional geometric sense, because the parameter spaces of neural networks of different widths are different. Nevertheless, this containment is reasonable in a certain sense: through a particular class of embedding operations, any critical point of a narrower network can be embedded into the loss landscape of the wider network while keeping the output function unchanged. We call this special embedding operation a **critical embedding**, because it preserves the network's properties at critical points.

We refer to this concept as a "principle" because it expresses an intrinsic property of deep neural networks—a property that does not depend on the specific training data or the choice of loss function, but is closely tied to the **layer structure** of the network. Furthermore, as the experimental results below show, this principle is tightly connected to the actual training of deep neural networks: in practical training, a wider network encounters saddle points whose outputs coincide with the global minima of narrower networks, which can be viewed as embedded variants in the wider network of the global minima of narrower networks.

Before we formally prove the embedding principle, let us first develop an intuitive understanding of the key ingredient of the proof—the "splitting" operation of neurons. Through the splitting operation, we can embed a network into a wider network while keeping the output function unchanged and at the same time preserving the properties of the critical point.

We first consider one-step embedding. As shown in Fig. 9.3, we can split any hidden-layer neuron (e.g., the black neuron in the network on the left of Fig. 9.3) into two neurons (the blue and purple neurons in the network on the right of Fig. 9.3). The two new neurons share the same input weights as the original, while the output weights are distributed in the proportions α and $(1-\alpha)$ (where $\alpha \in \mathbb{R}$ is a hyperparameter). A multi-step embedding is the composition of several one-step embeddings. Since each one-step embedding can add one neuron to a chosen layer, any narrower network can, through multi-step embedding, be embedded into any wider network. This process not only preserves the consistency of the output function but also preserves the extremal properties of the critical point.

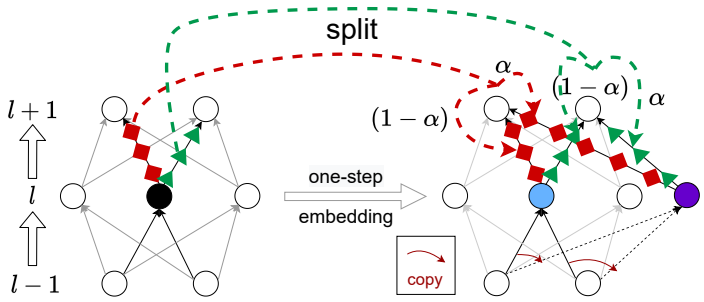


Figure 9.3: Illustration of one-step embedding.

In fact, the splitting operation is not restricted to fully-connected neural networks. Any network with a layer structure and with neuron indices that are exchangeable admits a corresponding embedding principle that can be proved via the splitting operation; the principle can therefore

be extended to a variety of network architectures. In a convolutional neural network (CNN), the splitting operation can be understood as the duplication and transformation of convolution kernels, which increases the width of the network while keeping the receptive field and functionality unchanged. For the Transformer architecture, the splitting operation can be likened to increasing the number of attention heads, which extends the model's capacity and allows it to handle complex sequence relationships in a more diverse manner. Besides the splitting operation, other ways of realizing network embedding do exist. In what follows, we will explore additional embedding operations, which not only theoretically broaden our understanding of the embedding principle of neural networks but also provide new perspectives and tools for the design and training of networks in practice.

9.1.3 Relationship between the embedding principle and the condensation phenomenon

The splitting operation proposed above is highly related to the condensation phenomenon in neural-network training. The condensation phenomenon describes the convergence of neurons during training; in particular, different neurons may develop similar or identical weight directions during learning. This phenomenon is directly related to the splitting operation shown in Fig. 9.3: the splitting operation means that by duplicating one neuron, we obtain two neurons with exactly the same input weights.

The embedding principle provides a powerful theoretical framework for understanding key properties of neural-network structure. From an existence standpoint, it explains how neural networks of different widths are related through critical embedding operations. The condensation phenomenon, in turn, demonstrates from an experimental perspective how these theoretical relationships manifest concretely in actual training. The combination of theory and practice reveals a profound insight: the embedding principle is not only a high-level theoretical concept but also forms a tight connection with the actual training process of neural networks. This connection offers valuable insight that helps us better understand and optimize the training dynamics of deep-learning models.

In addition, from the standpoint of the embedding principle, we can provide a further understanding of condensation. If a larger network can be generated from a smaller network through a critical embedding operation, then we may say that this larger network is in an **ideal condensed state**. Although such a state is hard to attain exactly in experiments, during training a neural network can approach this state through condensation. Under this understanding, condensation not only signifies the convergence of neurons but also implies the compressibility of the network. A large network in the ideal condensed state can be compressed into a smaller network without changing its output function. The discovery of the embedding principle provides a new line of thought for building a theoretical understanding of the condensation phenomenon: by uncovering more critical embedding operations, we can deepen our understanding of the ideal condensed state, which in turn helps us design better indicators for identifying condensation in experiments.

9.1.4 Relationship between the embedding principle and the F-Principle

We have already explored two profound principles of neural networks—the Frequency Principle (F-Principle) and the embedding principle. The F-Principle states that during learning, neural networks tend to first capture the low-frequency components of the target function, and then gradually learn the high-frequency details. This principle reflects the spectral learning dynamics

of neural networks: starting from simple (low-frequency) patterns and gradually transitioning to complex (high-frequency) patterns. The embedding principle, from the perspective of parameter space, reveals the inheritance relationship of critical points among neural networks of different widths, showing that critical points found in the loss landscape of a narrower network also have corresponding critical points in the loss landscape of a wider network.

Both principles are highly general; they are not specific to a particular data distribution, and they apply to a variety of common architectures including convolutional neural networks (CNNs), Transformers, and ResNets. To deepen our understanding of the connection between these two principles, we conducted an experiment in which a two-layer neural network with width 500 and tanh activation was used to fit a one-dimensional target function (the blue dashed line in Fig. 9.4(b)).

We recorded the output function and its frequency spectrum at different training epochs. As shown in Fig. 9.4, we observe the learning process of the neural network from two perspectives: the output function (Fig. 9.4(a)) and the frequency domain (Fig. 9.4(b)). From Fig. 9.4(b) we can see that in the early stage of training, the network output agrees well with the target function in the low-frequency region, and as training progresses, the high-frequency parts gradually align with the target function. At specific epochs, as shown in Fig. 9.4(a), the function learned by the network evolves gradually: at Epoch=10000, the network first learns an output function corresponding to a single tanh neuron (the second image from the left in Fig. 9.4(a)). This is in fact the global minimum of a narrower network, and this point can be embedded, via the embedding principle, as a critical point of the wider network. This embodies the fact that the loss landscape of the wide network contains critical points of the narrow network. Finally, at Epoch=40000, the network successfully fits the target function (corresponding to the global minimum of a 3-tanh-neuron network).

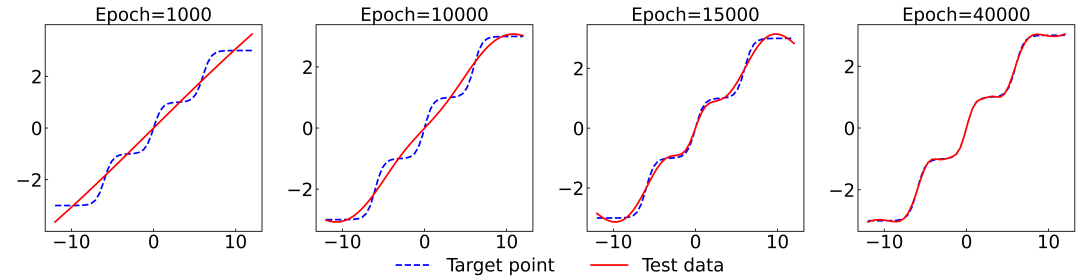
This experiment not only demonstrates how a neural network progressively approximates a complex function, but also reflects the role of the embedding principle in actual training: a wide network can encounter, during training, critical points embedded from narrow networks, thereby controlling the complexity of its output function. According to the F-Principle, we regard functions dominated by low-frequency components as the "simple" functions that the network implicitly prefers during training. Here, "simple" functions are defined via the embedding principle—namely, those functions corresponding to critical points of narrower networks. The phenomenon in which the neural network first learns critical functions embedded from narrow networks and then learns critical functions embedded from wider networks is consistent with the learning process in the F-Principle, which proceeds from low to high frequency.

9.2 In-depth Analysis of the Embedding Principle

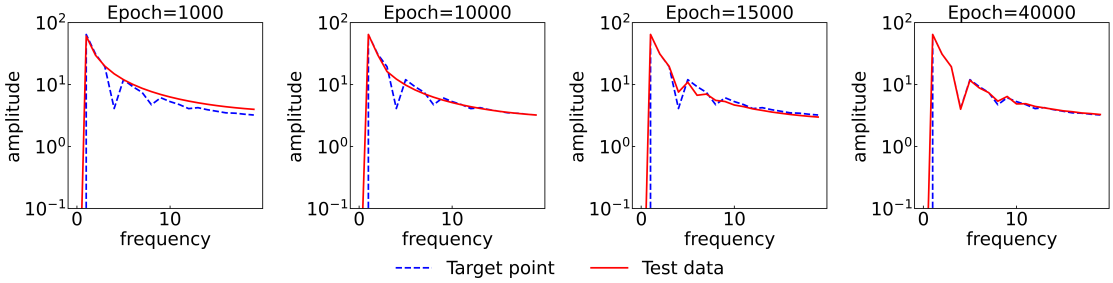
The embedding principle provides a theoretical framework that reveals that, in theory, neural networks of different widths can share critical points with consistent output functions. Through Fig. 9.1 and Fig. 9.2, we observe that wide networks can indeed encounter, during actual training, the critical points defined by narrow networks, which prompts further investigation into this phenomenon and its generality.

9.2.1 Spectral analysis of the loss-function plateau phenomenon

To investigate the generality of the phenomenon in which wide networks plateau at the same loss values as narrow networks, we adopt a systematic experimental approach. Specifically, for each network width we conduct 100 independent experiments, each with a different random-seed



(a) Output-function perspective



(b) Frequency perspective

Figure 9.4: The learning process of a two-layer tanh neural network with width 500 fitting a one-dimensional function, viewed from both the output-function and frequency perspectives.

initialization. This repeated-experiment design is intended to rule out the influence of randomness on the results, ensuring that the observed phenomena are determined by the properties of the network structure itself rather than by chance. Based on the phase-diagram analysis discussed in the previous chapter, we choose a particular parameter-initialization strategy here, ensuring that the parameter initializations of all networks lie at the same position in the condensed regime. Specifically, this means that the initial states of the networks are set so as to have similar dynamical properties, ensuring that every network starts from the nonlinear training regime. The choice of this initialization method is based on previous observations of the condensation phenomenon and on our understanding of dynamical-system behavior.

In each experiment, we closely monitor the evolution trajectory of the loss function, and in particular record the number of steps the loss function spends in each small interval. If the loss function spends an unusually large number of steps in a certain small interval, this will form an obvious "plateau" in the evolution graph of the loss function, as shown in Fig. 9.1. By combining and aggregating the training-loss functions across different experiments, we construct a distribution plot of loss values, as shown in Fig. 9.5. Each row of this distribution plot represents the loss-value distribution corresponding to a network of a specific width, where the color intensity represents the logarithm of the frequency with which the loss value falls within a specific interval.

Fig. 9.5 is similar in form to an energy spectrum in physics, and it can provide key information about the loss function. For instance, the discreteness and finiteness of the spectral lines in the energy-spectrum diagram can help us analyze at which specific values the loss function plateaus,

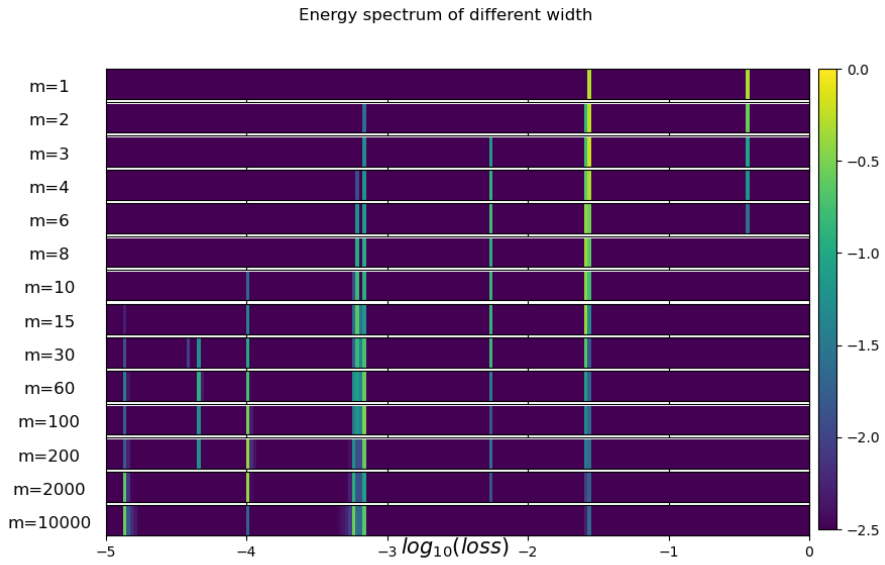


Figure 9.5: Loss spectrum obtained from 100 training runs of two-layer neural networks of different widths.

and how networks of different widths relate to one another. In this energy spectrum, the vertical axis represents the variation of network width and the horizontal axis is $\log_{10}(\text{loss})$. We divide the interval $[-5, 0]$ into 200 equally spaced subintervals and, for each width, record the frequency p_i with which the loss values from the 100 experiments fall within each subinterval. This yields a set $\{p_i\}_{i=1}^{200}$, namely the number of times the loss value falls within each subinterval over the 100 experiments. To clearly display spectral lines with small frequencies, we use a logarithmic scale for the color, namely $\log_{10} \frac{p_i}{\sum_{i=1}^{200} p_i}$, to represent the intensity of the color bar. Intuitively, this means that subintervals appearing frequently across the 100 experiments (i.e., intervals where the plateau phenomenon of the loss function occurs) are marked in brighter colors in the figure.

The results of the statistical analysis reveal several key findings, which deepen our understanding of the plateau phenomenon of the loss function in networks of different widths during training:

(i) For a network of a specific width, the loss functions across different random-seed initializations exhibit notable consistency. Specifically, loss functions of different experiments tend to spend long periods at specific loss values (as shown in Fig. 9.5, specific loss values appear as highlights in the same row). This consistency indicates that, despite the variety of initialization conditions, the training dynamics of the network are to some extent predictable, and certain loss values appear to be key "attractors" during training.

(ii) Networks of different widths exhibit similar plateau patterns in their loss functions (the highlighted positions in different rows of Fig. 9.5 are similar), but a wide network does not necessarily encounter all critical points embedded from narrow networks (Fig. 9.5 also shows cases where narrow networks are highlighted but wide networks are not). Thus, despite the

differing scales of the networks, wide networks do encounter critical points embedded from narrow networks. But which specific critical points they encounter and why remain worth exploring further.

(iii) As the network width increases, the probability that the loss function plateaus at smaller values also increases. This can be seen clearly from Fig. 9.5: networks of larger width have smaller loss values at the end of training. This observation suggests that wide networks may possess inherent advantages in optimization, hinting that wider networks may more easily escape saddle points and attain smaller loss values.

These observations provide a new perspective on the plateau phenomenon of the loss function during neural-network training, and also suggest how network width affects training efficiency and final performance.

Fig. 9.6 offers a visual interpretation of the above concepts. Imagine a traveler setting out from a starting point with the goal of reaching a distant castle, with various challenges and dangers along the way. In this analogy, the critical point corresponding to the global minimum of a network made up of a single neuron acts like a traffic officer, guiding the traveler to the first milestone—which we may call the first way station. Then, the critical point corresponding to the global minimum of a network containing two neurons plays the role of the second traffic officer, continuing to guide the traveler to the second way station. In practical training, the traveler may skip some intermediate way stations; for instance, he may be directly attracted by the critical point corresponding to the global minimum of a network containing three neurons, and proceed directly to the castle. The critical points and guidance along the way form a special structured path that guides the traveler through a complex and dangerous loss landscape.

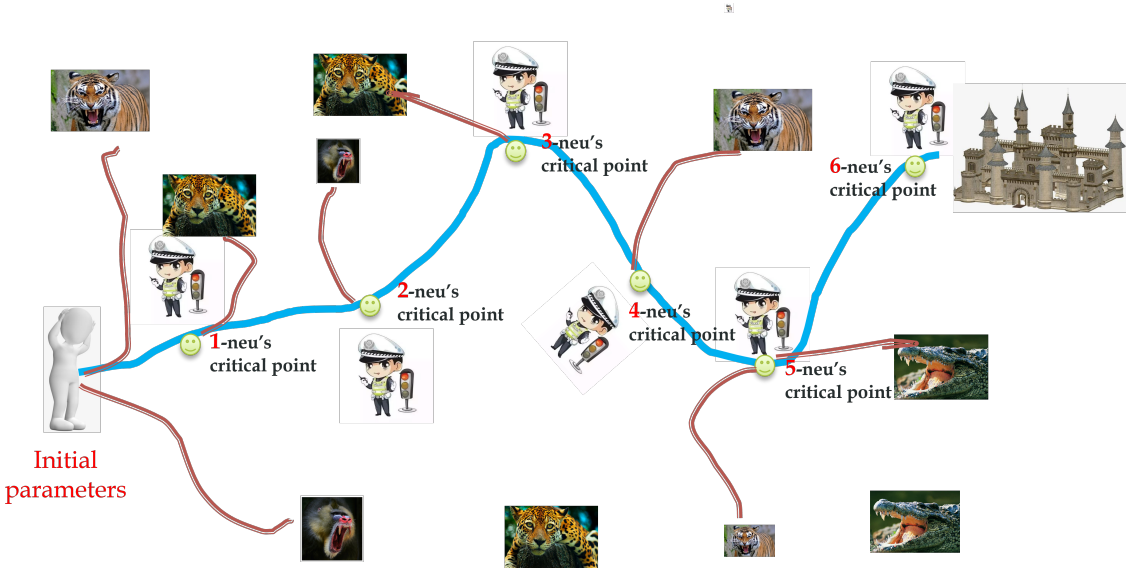


Figure 9.6: An intuitive illustration of the optimization path in the loss landscape.

Although the loss landscape of a neural network is extremely complex, the special properties

induced by its layer structure allow the network to gradually increase complexity during training. This understanding of the parameter space is consistent with the F-Principle's understanding of the function space: low-frequency components are more easily captured and optimized by the network, while high-frequency components are only gradually learned in the later stages of training.

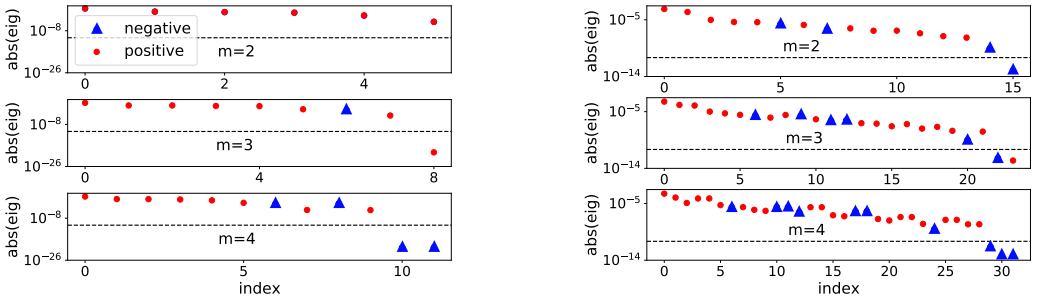
9.2.2 Eigenvalue analysis of the Hessian matrix after critical-point embedding

The embedding principle guarantees that the loss landscape of any network "contains" all critical points of any narrower network, and in Fig. 9.5 we also observe that different networks plateau during training at the same loss values corresponding to critical points. We can further investigate the differences between critical points before and after embedding. Specifically, there are two natural entry points: first, we study the number of descent directions, i.e., the number of negative eigenvalues of the Hessian matrix—clearly, the more descent directions there are, the easier the optimization; second, we study the degree of degeneracy, i.e., the number of zero eigenvalues of the Hessian matrix.

First, through experiments, we observe how the eigenvalues change under the action of the embedding principle. We train a two-layer neural network of width $m_{\text{small}} = 2$ to learn a set of data sampled from a one-dimensional function (see Fig. 9.7(a)) or the Iris dataset (see Fig. 9.7(b)). When the loss function descends so slowly that it nearly plateaus, we consider the network to be near a critical point, and we examine the gradient values at that point. In Fig. 9.7(a), the L_1 norm of the gradient of the loss function at the critical point is approximately 7.15×10^{-15}, and in Fig. 9.7(b) it is 3.72×10^{-13}; these extremely small values justify treating these points as critical points.

Next, we transfer these critical points to networks of width $m = 3$ and $m = 4$ via one-step or two-step embedding. As shown in Fig. 9.7, the auxiliary dashed line in each subfigure is $y = 10^{-11}$. In these subfigures, we evenly split one neuron of the width-2 two-layer neural network at the critical point into k neurons ($k = 2, 3$): their input weights remain unchanged, while their output weights become $1/k$ of the original neuron's. As shown in Fig. 9.7, each embedding step introduces additional zero eigenvalues into the Hessian matrix. More importantly, in Fig. 9.7(a), all eigenvalues of the $m = 2$ network are positive (red dots), indicating that the critical point obtained from training is a local or global minimum. After embedding, the appearance of negative eigenvalues (blue triangles) turns these points into saddle points. In both Fig. 9.7(a) and (b), we observe that the number of negative eigenvalues increases as the network width increases. For instance, examining the portion above the auxiliary dashed line, the count grows from 0 to 1 to 2 in (a), and from 3 to 5 to 7 in (b). This experiment shows that, after embedding, negative eigenvalues appear among the eigenvalues of the Hessian matrix, indicating that the corresponding critical points in wider neural networks have changed and that the difficulty of escaping these critical points has decreased, which supports our observations in the spectral analysis.

In fact, the paper Zhang et al. (2021b) proves that for general critical embedding operations, the number of zero eigenvalues (degenerate directions), positive eigenvalues (ascent directions), and negative eigenvalues (descent directions) of the Hessian matrix of the larger network never decreases. We know that as long as a negative-eigenvalue direction exists, a critical point cannot be a local minimum. Therefore, when a local minimum of a small network is embedded into a wider network, since descent directions are very likely to be added, it has a high probability of becoming a saddle point; moreover, the more descent directions there are, the easier it is for the training process to



(a) One-dimensional synthetic data

(b) Iris data

Figure 9.7: Eigenvalues of the Hessian matrix of the neural network corresponding to the critical points for the one-dimensional synthetic data and the Iris dataset.

escape this saddle point. This understanding is consistent with the phenomenon observed in the loss spectrum, namely that wider networks more easily learn smaller loss values.

Our experimental and theoretical analyses reveal a key phenomenon in neural-network training: when critical points of a smaller network are transferred to a wider network via embedding operations, the new critical points exhibit more degenerate directions and more descent directions. This means that, within the loss landscape of the wide network, these embedded critical points are more likely to attract the training trajectory due to the increased number of degenerate directions. At the same time, the additional descent directions also mean that, compared with the small network, the wide network can more easily optimize the loss function to a lower value.

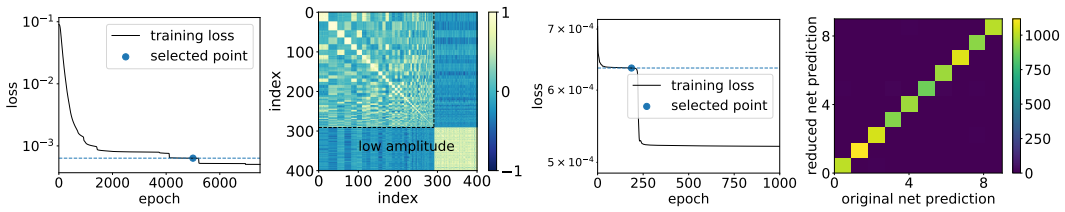
9.2.3 Simplifying the scale of a neural network

In practical training, a neural network rarely lies fully within the theoretical condensed regime, so it is difficult to observe absolute condensation. Nevertheless, we can still observe a convergence behavior among neurons in the same layer, i.e., different neurons tend to become similar to one another, and this convergence can serve as a mechanism for controlling model complexity.

To investigate whether, when a neural network approaches a critical point embedded from a smaller network, we can effectively shrink the network's scale, we conduct the following experiment. According to the embedding principle, we expect that, in the vicinity of a critical point embedded from a narrower network, a group of condensed neurons can be replaced by a single neuron without sacrificing the network's performance.

This prediction is verified in the experiment shown in Fig. 9.8. On 1000 samples of the MNIST dataset, we train a two-layer ReLU neural network of width 400, $\boldsymbol{f}_{\boldsymbol{\theta}} = \sum_{k=1}^{m} a_k \sigma(\boldsymbol{w}_k^T \tilde{\boldsymbol{x}})$ (where $\tilde{\boldsymbol{x}} = [\boldsymbol{x}^\intercal, 1]^\intercal$). During training, we find that at the position marked by the blue dot in Fig. 9.8(a), the loss decreases very slowly, indicating that the network may be near a saddle point. To analyze the similarity among neurons, we compute the normalized inner product of each pair of neuron input-weight vectors.

As shown in Fig. 9.8(b), we plot the normalized inner products of the input weights of different neurons. The horizontal and vertical axes are neuron indices. Neurons in the "low-amplitude" region have amplitudes much smaller than the others and are therefore removed. We identify 58



(a) Loss curve (b) Directional similarity (c) Loss curve after simplification (d) Prediction similarity

Figure 9.8: The MNIST training process.

neuron groups in which the neurons have an input-weight-vector similarity of at least 0.9. For each such similarity group S_{similar}, we randomly choose one neuron j, and adjust its output weight to be the weighted sum of the output weights of all neurons in the group, with weights given by the norms of the input-weight vectors. We then remove the remaining neurons in the group. In this way, the width of the neural network is reduced from 400 to 58.

Starting from the adjusted parameters $\boldsymbol{\theta}_{\text{redu}}$, we continue training the simplified network. As shown in Fig. 9.8, after several training steps the simplified network's loss plateaus at the same value as the blue dot marked in Fig. 9.8(a) for the original network; this is indicated by the blue dashed line in the figure and represented by the blue dot in Fig. 9.8(c). Furthermore, on 10000 test samples we compare the predictions of the original network and the simplified network at the corresponding blue points, as shown in Fig. 9.8(d). In this figure, the horizontal axis represents the predictions of the original wide network on the 10 digits $0-9$ of MNIST, and the vertical axis represents the predictions of the simplified small network on the 10 digits. The color of each grid cell represents the frequency with which the two models give the same prediction. The bright cells along the diagonal indicate a high prediction consistency between the two models, with overall consistency of about 98.5%.

The results of this experiment clearly demonstrate the relevance of our embedding principle to training on real-world datasets. The simplified neural network of width 58 behaves at the critical point in a manner highly consistent with the original neural network of width 400, showing that under certain conditions we can simplify the scale of the network by reducing the number of neurons without sacrificing too much performance. This offers a possible path toward more efficient networks, and provides new theoretical support for optimizing network structure.

Chapter 10

Optimistic Estimate

In the field of deep learning, over-parameterization is a common practice, referring to the case where the number of model parameters exceeds the number of training samples. Generally speaking, the more parameters a model has, the higher its complexity. According to classical learning theory, if the number of parameters exceeds the sample size, the model is likely to overfit. However, deep learning has overturned this conventional wisdom. Even when the number of parameters far exceeds the number of samples, neural networks still exhibit remarkable generalization performance. This phenomenon is referred to as the **"generalization mystery"**. This puzzle prompts us to think: how should we understand this phenomenon? And how should we construct a generalization theory to explain it?

When discussing generalization, we need to comprehensively consider the characteristics of both the model and the data. Usually, the characteristics of the data are rather complex. However, regardless of the nature of the data, the model we use is always a neural network. Therefore, our primary task is to study the fitting characteristics of neural networks, and then to understand the data through these characteristics. Abstractly speaking, the data and the neural network are like two black boxes; we need to first open the black box of the neural network, and then open the black box of the data.

A complete generalization theory should include three elements: data, optimization method, and model. This theory needs to clearly explain how these three jointly affect generalization performance. Since the data is a black box, we need to simplify and clarify the data as much as possible, so that the influence of the neural network model can be seen more clearly. Therefore, in experimental and theoretical studies, the data we choose is sampled from a target function that can be represented by some small network. The purpose of this is to reduce the complexity of the data, enabling us to observe and understand the effect of the neural network model more directly.

During the training of neural networks, we frequently observe the condensation phenomenon. Condensation is unique to nonlinear systems, and we expect it to be closely related to the generalization ability of neural networks. However, this relationship has not been fully clarified, and it remains unclear exactly where the advantage of condensation in generalization is manifested. From an intuitive standpoint, if condensation occurs during training, then the neural network can be viewed as a smaller network in its training dynamics. If the target function can be represented by this small network, then we may be able to recover the target function with only a small number of samples. The stronger the condensation, the smaller the sample size required to recover the target

function. Therefore, condensation can help neural networks learn simple target functions from less data, thereby enhancing their generalization ability.

In this chapter, we conduct a series of experiments to explore the possibility of neural networks learning simple functions. We then propose the concept of optimistic estimate to estimate the sample size required for learning simple functions.

Popular Science Section

In the field of machine learning, traditional learning theory emphasizes that, in order to achieve good generalization, the complexity of the model should not be too high. In particular, if we increase the scale of the model (i.e., its complexity), we should additionally include more samples to prevent overfitting. However, in deep learning the actual situation often runs counter to the theory. Even when the number of parameters of the model far exceeds the sample size, deep neural networks can still exhibit remarkable generalization ability. We call this phenomenon the "generalization mystery". This chapter will develop a new "optimistic estimate" theoretical framework to explore the deep mechanisms behind this phenomenon.

What is optimistic estimate?

Optimistic estimate is a theoretical tool used to estimate the minimum sample size required for a neural network to fit a target function. Its core idea is: consider the best-case scenario, in which the parameter initialization is chosen near an ideal point, so as to maximize the recovery ability of the neural network. Near the ideal parameter point, we obtain the minimum sample size required to recover the target function by linearizing the model, which serves as a lower bound on the sample size required to recover the target function. Although the optimistic sample size is derived from a somewhat impractical optimistic initialization, our experimental results show that, without optimistic initialization, with merely a small initialization plus a good choice of learning-rate hyperparameters, this theoretically derived optimistic sample size can be fully attained or closely approached in real models.

What experimental phenomena motivate optimistic estimate?

Through a series of experiments, we find that in matrix factorization models and neural networks, the sample size required to recover a target function is significantly smaller than the parameter count of the model. In particular, we find that enlarging the model size does not require additional samples of the same amount to maintain generalization. The experiments also reveal the recovery ability of neural networks under different initialization conditions. For instance, using a smaller initialization can reduce the sample size required for recovery, and the core reason is the condensation phenomenon we mentioned in earlier chapters.

What is the theoretical foundation of optimistic estimate?

We define the model rank $R_{f_{\boldsymbol{\theta}}}(\boldsymbol{\theta}')$:

$$R_{f_{\boldsymbol{\theta}}}(\boldsymbol{\theta}') := \dim\left(\operatorname{span}\left\{\partial_{\theta_i} f(\cdot;\boldsymbol{\theta}')\right\}_{i=1}^{M}\right). \tag{10.1}$$

It represents the degrees of freedom of the linearization of the model $f_{\boldsymbol{\theta}}$ at a given parameter initialization $\boldsymbol{\theta}'$. The theoretical result shows that the model rank equals the minimum sample size required to linearly recover the target function.

What are the applications of optimistic estimate?

Matrix factorization

We consider a classical problem: recovering a matrix from partial observations of its entries—the well-known matrix completion problem. Analysis of this problem shows that the sample size required to recover a matrix is related to its rank. Experiments show that the sample size required to recover a low-rank matrix in the over-parameterized regime equals the model rank $R_{f_{\boldsymbol{\theta}}}(\boldsymbol{\theta}')$.

$$R_{\boldsymbol{f_\theta}}(\boldsymbol{M}^*) = 2r_{\boldsymbol{M}^*}d - r_{\boldsymbol{M}^*}^2, \tag{10.2}$$

where $r_{\boldsymbol{M}^*}$ is the matrix rank of the target matrix $\boldsymbol{M}^*$.

Neural networks

For neural network models, optimistic estimate exhibits excellent applicability. We find that for a given target function, the optimistic sample size depends only on the complexity of the target function (its intrinsic width), and is independent of the actual width of the neural network.

Definition 1. *Intrinsic width $k(f^*)$: defined as the minimum width of a neural network that can represent f^*; that is, f^* can be represented by a neural network of width $k(f^*)$, but cannot be represented by any narrower neural network.*

By computation, the optimistic sample size of a neural network is:

$$R_{\mathrm{NN}_m}(f^*) = k(f^*)(d+1), \tag{10.3}$$

where $k(f^*)$ denotes the intrinsic width of the target function f^*, and d is the input dimension. Experimental results show that with proper hyperparameter tuning, the minimum sample size required to recover the target function approaches the optimistic sample size.

Potential and challenges of optimistic estimate

Although optimistic estimate provides a reasonable explanation for the generalization mystery of neural networks, its extension and application still face many challenges. First, how can one effectively realize optimistic initialization in practical training, so that the training process approaches the best-case scenario? Second, for more complex multi-layer neural networks and advanced models (such as Transformer), how can one accurately estimate their optimistic sample sizes?

10.1 Quantifying the minimum sample size required for a model to recover a target function: the model rank

Under over-parameterization, there are many solutions that achieve a training error of 0, but most of them generalize poorly, so in the worst case we cannot recover the target function. However,

practice frequently shows that nonlinear models have the ability to generalize well, or even recover the target function, under over-parameterization (as will be seen in later experiments). To explain these experimental phenomena, we propose a new theoretical framework—**optimistic estimate**. It enables us to theoretically analyze the minimum sample size required to recover the target function.

In a regression problem, we use a model $f_{\boldsymbol{\theta}}$ to fit n data points sampled from a target function f^*, with the goal of recovering f^*. To make "recovery" feasible, we consider target functions that can be represented by the model $f_{\boldsymbol{\theta}}$, i.e., $f^* \in \mathcal{F} := \{f(\cdot;\boldsymbol{\theta}) | \boldsymbol{\theta} \in \mathbb{R}^M\}$. By default, we use gradient descent to solve the regression problem:

$$\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} \left(f\left(\boldsymbol{x}_i;\boldsymbol{\theta}\right) - f^*(\boldsymbol{x}_i)\right)^2. \tag{10.4}$$

The solution to which gradient descent converges is denoted by $f_{\boldsymbol{\theta}_\infty}$, where $\boldsymbol{\theta}_\infty$ denotes the model parameters at convergence.

We use the linear approximation $f(\cdot;\boldsymbol{\theta}) \approx f^{\mathrm{lin}}(\cdot;\boldsymbol{\theta}) := f^*(\cdot) + \nabla f_{\boldsymbol{\theta}'}(\cdot)^{\mathrm{T}}(\boldsymbol{\theta} - \boldsymbol{\theta}')$ to analyze the sample size required for fitting in a small neighborhood $N(\boldsymbol{\theta}', \epsilon)$, where $\boldsymbol{\theta}' \in \boldsymbol{\Theta}_{f^*}$. Here, $\boldsymbol{\Theta}_{f^*} := \left\{\boldsymbol{\theta} \mid f_{\boldsymbol{\theta}} = f^*, \boldsymbol{\theta} \in \mathbb{R}^M\right\}$ denotes the set of parameters that can recover the target function, which we call the target set. The regression problem (10.4) then becomes

$$\min_{\boldsymbol{\theta} \in N(\boldsymbol{\theta}', \epsilon)} \frac{1}{n} \sum_{i=1}^{n} \left(f^{\mathrm{lin}}\left(\boldsymbol{x}_i;\boldsymbol{\theta}\right) - f^*(\boldsymbol{x}_i)\right)^2. \tag{10.5}$$

This is a problem that is linear in the parameter $\boldsymbol{\theta}$, and $f^{\mathrm{lin}}\left(\boldsymbol{x}_i;\boldsymbol{\theta}\right)$, as a function of $\boldsymbol{\theta}$, lies in a linear space, namely the space spanned by the tangent functions of the model $f(\boldsymbol{x};\boldsymbol{\theta})$. Its dimension is $\mathrm{rank}(\nabla f_{\boldsymbol{\theta}'}(\cdot)^{\mathrm{T}}) = \dim\left(\mathrm{span}\left\{\partial_{\theta_i} f(\cdot;\boldsymbol{\theta}')\right\}_{i=1}^{M}\right)$. For such linear models (such as classical linear regression), the number of samples required equals the number of independent parameters; that is, we need $n = \dim\left(\mathrm{span}\left\{\partial_{\theta_i} f(\cdot;\boldsymbol{\theta}')\right\}_{i=1}^{M}\right)$ samples to recover f^*. Inspired by the notion of rank in differential topology, we call this quantity the **model rank**, defined for any $\boldsymbol{\theta}' \in \mathbb{R}^M$ as

$$R_{f_{\boldsymbol{\theta}}}(\boldsymbol{\theta}') := \dim\left(\mathrm{span}\left\{\partial_{\theta_i} f(\cdot;\boldsymbol{\theta}')\right\}_{i=1}^{M}\right). \tag{10.6}$$

For a nonlinear model, the model rank may differ at different points in the target set. We would like to choose a $\boldsymbol{\theta}'$ in the target set with the smallest model rank $R_{f_{\boldsymbol{\theta}}}(\boldsymbol{\theta}')$, and initialize near this $\boldsymbol{\theta}'$. Therefore, for any function $f^* \in \mathcal{F}$, the model rank with respect to f^* is defined as the minimum rank in $\boldsymbol{\Theta}_{f^*}$, i.e.,

$$R_{f_{\boldsymbol{\theta}}}(f^*) := \min_{\boldsymbol{\theta}^* \in \boldsymbol{\Theta}_{f^*}} R_{f_{\boldsymbol{\theta}}}(\boldsymbol{\theta}^*). \tag{10.7}$$

Note that, for convenience, we have slightly overloaded the notation $R_{f_{\boldsymbol{\theta}}}$. Whether it refers to the rank of a function or the rank at a parameter point depends on its input. The above linear approximation analysis is valid only in a small neighborhood of $\boldsymbol{\theta}'$, so we consider a special initialization scheme, which we call **optimistic initialization:** initialize in a neighborhood of an "optimal" point $\boldsymbol{\theta}^*$ in the target set $\boldsymbol{\Theta}_{f^*}$. Under optimistic initialization, the sample size required to fit f^* is $R_{f_{\boldsymbol{\theta}}}(f^*)$, which we call the **optimistic sample size** required to recover f^*. It determines the minimum sample size required to fit f^* using the model $f_{\boldsymbol{\theta}}$.

Although the optimistic sample complexity derived from optimistic initialization seems impractical in real applications—since prior to performing a specific machine learning task, the target function f^* and its corresponding parameter set (i.e., the target set) $\boldsymbol{\Theta}_{f^*} := \left\{\boldsymbol{\theta} \mid f_{\boldsymbol{\theta}} = f^*,\ \boldsymbol{\theta} \in \mathbb{R}^M\right\}$ are usually unknown, so we cannot initialize the model near an ideal parameter $\boldsymbol{\theta}^*$ in the target set—the training trajectory of a neural network that converges to a global minimum can be divided into two stages: the training stage and the convergence stage. In the convergence stage, the trajectory lies near a global minimum. The purpose of the training stage can be understood as providing a suitable "initialization" for the convergence stage. Therefore, the optimistic estimate framework developed from optimistic initialization provides important insight into the behavior of nonlinear models. This framework describes the sample size required for a model to recover the target function in the best case—if the model cannot recover the target function even under optimistic initialization, then it will most likely fail under any worse initialization scheme as well.

Moreover, as we have noted in earlier chapters, under random small initialization, the training process of neural networks typically exhibits a **condensation property**—namely, a large number of neurons tend to become aligned during training. This phenomenon implies that, in actual training, even without adopting a specific "optimistic initialization", merely using a standard random small initialization combined with appropriate learning-rate tuning can usually recover the target function within a range close to the optimistic sample complexity. The intuitive explanation behind this phenomenon is: **the stronger the condensation, the more similar the neurons become, and hence the lower the model rank, so the smaller the sample size required**. In the extreme idealized condensation case, the model rank reaches its minimum, and the sample size required to recover the target function then approaches the theoretical optimistic sample size. In fact, for structurally simple models such as the matrix factorization model, the training dynamics naturally possesses ideal condensation properties (Bai et al., 2024), and therefore in later experiments we will observe that this model can exactly attain the optimistic sample size. For more complex models (such as neural networks), although their structure is more elaborate, the training process also exhibits a high degree of condensation, and therefore in practice they can usually come very close to this theoretical bound. The strong agreement between the theoretically derived optimistic sample complexity and the actual performance observed in experiments validates the effectiveness of the optimistic estimate framework. This indicates that the optimistic sample size is not merely a theoretical construct, but a meaningful and predictive quantity in practice.

Example 1. *Consider the classical linear regression model $f(\boldsymbol{x};\boldsymbol{\theta}) = a_1x_1 + \ldots + a_kx_k$, where $\boldsymbol{\theta} = (a_1, \ldots, a_k) \in \mathbb{R}^k$. Suppose the target function is $f^*(\boldsymbol{x}) = x_1$. Let us compute the model rank. For $\forall \boldsymbol{\theta} \in \mathbb{R}^k$, the partial derivative of $f(\boldsymbol{x};\boldsymbol{\theta})$ with respect to each parameter θ_i is $\partial_{\theta_i} f(\cdot;\boldsymbol{\theta}) = x_i$, and $\{x_1, x_2, \cdots, x_k\}$ as k functions are linearly independent. Here, linear independence is defined in the classical sense, namely $\sum_{i=1}^k \alpha_i x_i = 0 \implies \alpha_j = 0, \forall 1 \le j \le k$.*

By definition, the model rank is the dimension of the space spanned by the tangent functions obtained by taking derivatives of the model with respect to the parameters, so the model rank is $R_{f_{\boldsymbol{\theta}}}(\boldsymbol{\theta}') := \dim\left(\operatorname{span}\left\{\partial_{\theta_i} f(\cdot;\boldsymbol{\theta})\right\}_{i=1}^M\right) = k$. We find that, in linear models, the model rank is a constant and does not vary with $\boldsymbol{\theta}'$.

Example 2. *Consider the following two-neuron network with fixed bias: $f_{\boldsymbol{\theta}}(x) = a_1 \tanh(\boldsymbol{w}_1 \cdot \boldsymbol{x} + 1) + a_2 \tanh(\boldsymbol{w}_2 \cdot \boldsymbol{x} + 1)$, with parameters $\boldsymbol{\theta} = (a_1, \boldsymbol{w}_1, a_2, \boldsymbol{w}_2), \boldsymbol{w}_i \in \mathbb{R}^2, a_i \in \mathbb{R}, \boldsymbol{x} = (x_1, x_2) \in \mathbb{R}^2$. Let the target function be $f^*(x) = \bar{a} \tanh(\bar{\boldsymbol{w}} \cdot \boldsymbol{x} + 1)$. We consider the target set that can recover the target function. There are three cases in total:*

(1) Both neurons take the weight $\bar{\boldsymbol{w}}$, and the coefficients a_1, a_2 satisfy $a_1 + a_2 = \bar{a}$, so the target set in this case is a 1-dimensional space: $Q_1 = \{(a, \bar{\boldsymbol{w}}, \bar{a} - a, \bar{\boldsymbol{w}}) : a \in \mathbb{R}\}$.
(2) The weight $\boldsymbol{w}_1$ of the first neuron is $\bar{\boldsymbol{w}}$ and the coefficient a_1 is $\bar{a}$, while the weight $\boldsymbol{w}_2$ of the other neuron is arbitrary and the coefficient a_2 is 0. So the target set in this case is a 2-dimensional space: $Q_2 = \{(\bar{a}, \bar{\boldsymbol{w}}, 0, \boldsymbol{w}) : \boldsymbol{w} \in \mathbb{R}^2\}$.
(3) The weight $\boldsymbol{w}_2$ of the second neuron is $\bar{\boldsymbol{w}}$ and the coefficient a_2 is $\bar{a}$, while the weight $\boldsymbol{w}_1$ of the other neuron is arbitrary and the coefficient a_1 is 0. So the target set in this case is a 2-dimensional space: $Q_3 = \{(0, \boldsymbol{w}, \bar{a}, \bar{\boldsymbol{w}}) : \boldsymbol{w} \in \mathbb{R}^2\}$.

By a simple computation, in Q_1 and Q_2, Q_3 the model rank $R_{f_{\boldsymbol{\theta}}}(\boldsymbol{\theta}')$ is 3, 4, 4, respectively (left as an exercise). The optimistic sample size is then the minimum of the model ranks over all parameters in the target set, so the optimistic sample size for recovering the single-neuron target function is 3.

Remark 18. ***(1) Upper and lower bounds of the optimistic sample size:*** *$0 \leqslant R_{f_{\boldsymbol{\theta}}}(f^*) \leqslant M$. For a model $f(\boldsymbol{x}; \boldsymbol{\theta})$ that is linear in the parameter $\boldsymbol{\theta}$, Example 1 shows that its optimistic sample size is a constant and equals the parameter count M. This agrees with our common intuition: for linear models, we must sample M points to guarantee that the model recovers the target function. If we keep enlarging the model's parameter count without correspondingly increasing the same number of samples, the model's generalization will become increasingly poor. However, for nonlinear models such as neural networks, Example 2 shows that the optimistic sample size can be smaller than the model parameter count M. This is similar to the phenomenon we will see later in the experimental Figure 10.3: if we enlarge the model's parameter count, the sample size required to recover the target function does not need to grow proportionally with the parameter count; it is significantly less than the parameter count.*
(2) The optimistic sample size is influenced by both the model architecture $f_{\boldsymbol{\theta}}$ and the target function f^*. *This is consistent with the "no free lunch" theorem. The theorem states that there does not exist a single algorithm that can effectively learn all possible target functions; that is, the complexity of a target function must be tied to the specific model attempting to learn it. A function that looks complex under one model may be simple under another. The model rank quantifies this architecture–target-function relationship and provides a mathematical framework for evaluating architecture choices. From the perspective of machine learning, one of our goals is to design the most suitable architecture for a specific class of tasks. For instance, if the target function is one with local structure (such as natural images), then as shown in the later experimental Figure 10.3, convolutional neural networks (CNNs) have a significantly lower sample complexity (computed via the model rank) compared with fully-connected networks (FNNs).*
(3) Optimistic estimate provides a measure of effective parameters, which can precisely characterize over-parameterization in nonlinear regression. *The traditional definition of over-parameterization (sample size $n < M$) can be misleading. For example, consider fitting the linear function $f^*(x) = x$ with $f_{\boldsymbol{\theta}} = (\theta_1 + \theta_2)x$: although there are $M = 2$ parameters, only one sample is needed to fit this model. To address the limitation of this definition, we use the model rank to define over-parameterization as the case $n < M_{\mathrm{I}}$, where:*

$$M_{\mathrm{I}} := \max_{\boldsymbol{\theta}^* \in \mathbb{R}^M} R_{f_{\boldsymbol{\theta}}}(\boldsymbol{\theta}^*) \tag{10.8}$$

Here, M_{I} represents the maximum effective parameter count of the model $f_{\boldsymbol{\theta}}$. When for all $f^* \in \mathcal{F}$, $R_{f_{\boldsymbol{\theta}}}(f^*) = M_{\mathrm{I}}$, the model $f_{\boldsymbol{\theta}}$ cannot achieve fitting under over-parameterization. For models linear in the parameters, it is easy to verify that the model rank remains constant (equal to M_{I}),

which rules out their ability to recover target functions under over-parameterization. In contrast, nonlinear models—such as matrix factorization models and neural networks—can achieve target-function recovery under over-parameterization, demonstrating the fundamental advantage brought about by nonlinearity.

10.2 Comparison between optimistic sample size and actual experimental performance

This section presents a series of experiments on nonlinear models. We focus on observing whether nonlinear models can successfully recover target functions at the optimistic sample size.

10.2.1 A simple nonlinear regression model

First, we consider a basic linear model $f_{\mathrm{L}}(\boldsymbol{x};\boldsymbol{\theta}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$, which has $M = 3$ parameters. For any $\boldsymbol{\theta}' = [\theta_0', \theta_1', \theta_2']^T \in \mathbb{R}^3$, we have $R_{f_{\mathrm{L}}}(\boldsymbol{\theta}') = \dim(\operatorname{span}\{1, x_1, x_2\}) = 3$. Therefore, for any $f' \in \mathcal{F}_{\mathrm{L}} := \{f_{\mathrm{L}}(\cdot;\boldsymbol{\theta}) | \boldsymbol{\theta} \in \mathbb{R}^3\}$, we have $R_{f_{\mathrm{L}}}(f') = M_{\mathrm{I}} = 3$. Therefore, recovering any function using the model $f_{\boldsymbol{\theta}}(x)$ requires at least 3 samples, so f_{L} cannot recover the target function under over-parameterization. Similarly, we can show that all linear models have a fixed optimistic sample size. Therefore, all linear models cannot recover the target function under over-parameterization. This is consistent with the theory of linear regression.

When we reparameterize the linear model as $f_{\mathrm{NL}}(\boldsymbol{x};\boldsymbol{\theta}) = \theta_0 + \theta_1 x_1 + \theta_2\theta_3 x_2$, the optimistic sample size changes. Although the function space remains $\mathcal{F}_{\mathrm{NL}} = \mathcal{F}_{\mathrm{L}}$, the model becomes nonlinear in the parameters. The optimistic sample size of the model is estimated as follows. The model rank as a function on the parameter space $\mathbb{R}^4$ is:

$$R_{f_{\mathrm{NL}}}(\boldsymbol{\theta}') = \dim(\operatorname{span}\{1, x_1, \theta_3' x_2, \theta_2' x_2\}) = \begin{cases} 2, & \theta_2' = \theta_3' = 0, \\ 3, & \text{others.} \end{cases} \tag{10.9}$$

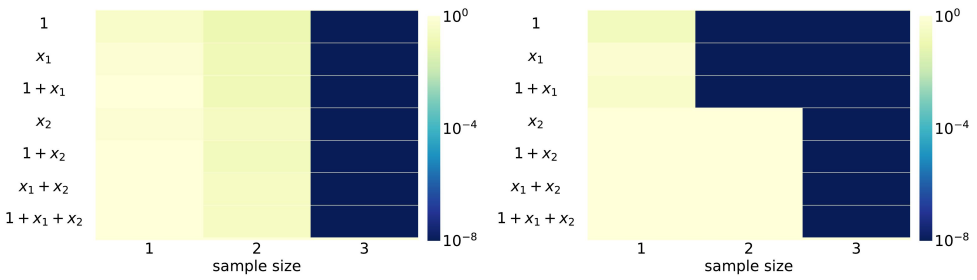
So the effective parameter count of the model is $M_{\mathrm{I}} = 3$. By solving the minimization problem (10.7), we obtain the model rank as a function on the function space $\mathcal{F}$ as

$$R_{f_{\mathrm{NL}}}(f^*) = \begin{cases} 2, & f^* \in \{a_0 + a_1 x_1 | a_0, a_1 \in \mathbb{R}\}, \\ 3, & f^* \in \{a_0 + a_1 x_1 + a_2 x_2 | a_2 \neq 0, a_0, a_1, a_2 \in \mathbb{R}\}. \end{cases} \tag{10.10}$$

The above estimate shows that the nonlinear model f_{NL} can recover $f^* \in \{a_0 + a_1 x_1 | a_0, a_1 \in \mathbb{R}\}$ under over-parameterization using two samples, but cannot recover $f^* \in \{a_0 + a_1 x_1 + a_2 x_2 | a_2 \neq 0, a_0, a_1, a_2 \in \mathbb{R}\}$ under over-parameterization.

In Figure 10.1, we use the mean-square loss over the data points as the loss function, and run gradient descent from small random initialization to train these two models. The model parameters are initialized using a Gaussian distribution with mean 0 and variance 10^{-8}. The training data consist of one, two, or three data points randomly drawn from the target function; the ordinate indicates different target functions, and the abscissa indicates the number of samples. The color of each small square represents the average test (generalization) error, taken as the average over 100 trials of random initialization.

Although the function spaces of the linear and nonlinear models are exactly the same, the sample sizes required for "recovery" differ noticeably between the two models. We see that **the optimistic**



(a) $f_{\mathrm{L}}(\boldsymbol{x};\boldsymbol{\theta})$ (b) $f_{\mathrm{NL}}(\boldsymbol{x};\boldsymbol{\theta})$

Figure 10.1: The average test error (color) for different target functions (ordinate) versus the number of training samples (abscissa).

sample size analysis perfectly predicts the experimental phenomena in Figure 10.1. For the linear model f_{L}, each target function requires three samples to achieve a generalization error close to 0. In contrast, for the nonlinear model f_{NL}, only two samples are needed to recover the target functions 1, x_1, and $1+x_1$. This shows that, for this simple model, the optimistic sample size as a lower bound for the sample size required for "recovery" is attainable.

10.2.2 Matrix factorization model

We consider a nonlinear matrix factorization model $\boldsymbol{f_\theta} = \boldsymbol{AB}$. This model has applications in the matrix completion task: our goal is to recover the target matrix $\boldsymbol{M}^*$ from n observed entries $S = \left\{\left((i_s, j_s), \boldsymbol{M}^*_{i_s j_s}\right)\right\}_{s=1}^n$, where (i_s, j_s) denotes the row and column indices in the matrix M^*. In other words, we are given the values of some entries of the target matrix $\boldsymbol{M}^*$, and we want to recover the values of the other entries of $\boldsymbol{M}^*$.

Consider a very simple low-rank matrix completion task:

$$\boldsymbol{M} = \begin{bmatrix} 1 & 2 \\ 3 & \star \end{bmatrix} \tag{10.11}$$

If we directly set a model that is linear in the parameters, $f_{\boldsymbol{\theta}} = \boldsymbol{W} \in \mathbb{R}^{d\times d}$, and use gradient descent from near-zero initialization to minimize the following objective:

$$\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{s=1}^{n} \left([\boldsymbol{W}]_{i_s j_s} - \boldsymbol{M}^*_{i_s j_s}\right)^2, \tag{10.12}$$

the entry at the second row and second column of $\boldsymbol{W}$ receives no gradient and therefore will not be trained, so a low-rank solution will generally not be learned.

But if we consider a nonlinear matrix factorization model $\boldsymbol{f_\theta} = \boldsymbol{AB}, \boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{d\times d}$, and use gradient descent to minimize the following objective:

$$\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{s=1}^{n} \left([\boldsymbol{AB}]_{i_s j_s} - \boldsymbol{M}^*_{i_s j_s}\right)^2, \tag{10.13}$$

where $\boldsymbol{M}^* \in \mathbb{R}^{d\times d}$, $\boldsymbol{\theta} = (\boldsymbol{A}, \boldsymbol{B})$, and $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{d\times d}$, with $[\boldsymbol{AB}]_{i_s j_s}$ denoting the entry of the matrix $\boldsymbol{W} = \boldsymbol{AB}$ at the i_s-th row and j_s-th column. In this nonlinear model, both the second row of $\boldsymbol{A}$ and the second column of $\boldsymbol{B}$ will be trained, so the model has the potential to learn a low-rank solution, and thus recover this rank-1 matrix from only 3 samples.

By applying the standard optimistic estimate procedure, we determine the optimistic sample size for any $\boldsymbol{M}^* \in \mathbb{R}^{d\times d}$ to be:

$$R_{\boldsymbol{f}_{\boldsymbol{\theta}}}(\boldsymbol{M}^*) = 2r_{\boldsymbol{M}^*}d - r_{\boldsymbol{M}^*}^2, \tag{10.14}$$

where $r_{\boldsymbol{M}^*} = \operatorname{rank}(\boldsymbol{M}^*)$ is the matrix rank of $\boldsymbol{M}^*$ (see Zhang et al. (2023) for the proof). It is worth noting that for fixed d, the optimistic sample size increases with $r_{\boldsymbol{M}^*}$. In particular, for large d, the optimistic sample size of a target matrix of rank r is $2rd - r^2$, which is much smaller than the effective parameter count $M_{\mathrm{I}} = d^2$. Therefore, the matrix factorization model may recover low-rank matrices under over-parameterization.

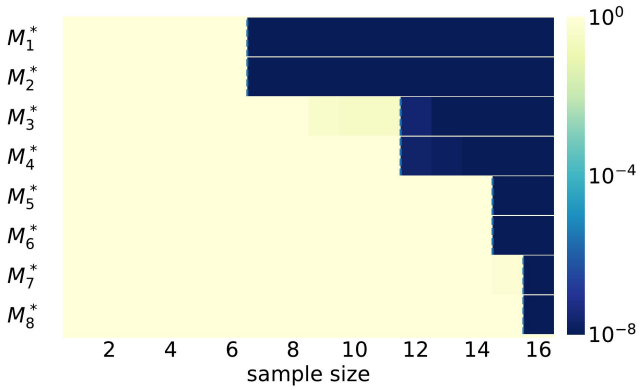


Figure 10.2: Average test error (color) versus the number of training samples (abscissa) for different target functions (ordinate).

To investigate whether this model can really learn low-rank solutions under over-parameterization, we conducted experiments on a 4×4 matrix completion task. From the experiments, we find that if the matrix rank of the target matrix $\boldsymbol{M}^*$ is the same, then the minimum sample size required to recover it is the same (the matrix rank is the dimension of the column space of the matrix). Figure 10.2 shows the error of fitting 8 target matrices using sample sizes from 1 to 16. The abscissa is the number of samples, and the ordinate is different target matrices. $\operatorname{rank}(\boldsymbol{M}_{2k-1}^*) = \operatorname{rank}(\boldsymbol{M}_{2k}^*) = k$, $k = 1, 2, 3, 4$. Here, rank denotes the matrix rank, and all $\boldsymbol{M}_i^*$ are 4×4 square matrices. The color represents the average test error (Euclidean distance between the trained matrix and the target matrix), and each test error is averaged over 50 trials of random initialization. The dashed line in the figure indicates the position where the average test error transitions sharply.

From the figure, we can see that when the rank of the target matrix is 1, 2, 3, or 4, the number of samples required for successful recovery of the matrix is 7, 12, 15, and 16, respectively. We note that these sample sizes exactly correspond to the optimistic sample sizes given by Eq. (10.14). This optimistic sample complexity has an intuitive interpretation: it equals the number of degrees of freedom that a rank-r matrix possesses. Specifically, for a $d \times d$ matrix of rank r, the number of degrees of freedom is $2rd - r^2$. Take a rank-1 4×4 matrix as an example. To uniquely determine

this matrix, we first need to determine the 4 entries of its first row (corresponding to 4 degrees of freedom), and then determine the proportional relationship between each of the remaining three rows and the first row (corresponding to 3 degrees of freedom). So there are $4 + 3 = 7$ degrees of freedom in total. This can be generalized: for any $d \times d$ matrix of rank r, the number of degrees of freedom is $2rd - r^2$. In the experiments shown in Figure 10.2, we verified that the theoretically derived optimistic sample complexity is attainable in practice.

10.2.3 Neural network models

First, we consider a two-layer fully-connected neural network with m neurons whose activation function is $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. The neural network is represented by $f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{i=1}^{m} a_i \tanh(\boldsymbol{w}_i^{\mathrm{T}} \boldsymbol{x})$, where $\boldsymbol{x} \in \mathbb{R}^d, \boldsymbol{\theta} = (a_i \in \mathbb{R}, \boldsymbol{w}_i \in \mathbb{R}^d)_{i=1}^m$. The network has $M = m(d+1)$ parameters in total. By computation, the optimistic sample size is related to the intrinsic width $k(f^*)$ of the target function f^*.

Definition 2. *Intrinsic width $k(f^*)$: defined as the minimum width of a neural network that can represent f^*; that is, f^* can be represented by a neural network of width $k(f^*)$, but cannot be represented by any narrower neural network.*

For an f^* that can be represented by a neural network of width m, its intrinsic width satisfies $0 \leqslant k(f^*) \leqslant m$. The optimistic sample size of f^* is

$$R_{\mathrm{NN}_m}(f^*) = k(f^*)(d+1), \tag{10.15}$$

A way of choosing $\boldsymbol{\theta}'$ that attains this optimistic sample size is: first pick $k(f^*)$ neurons to represent f^*, and then set the a and $\boldsymbol{w}$ of all other neurons to 0; in this case the model rank corresponding to $\boldsymbol{\theta}'$ is exactly $k(f^*)(d+1)$. The optimistic sample size $R_{\mathrm{NN}_m}(f^*)$ grows linearly with the intrinsic width. Therefore, for a function with intrinsic width $k(f^*) \ll m$, when the parameters are initialized using optimistic initialization, we can recover it under a sample size smaller than the parameter count.

Next, we consider a two-layer tanh-CNN with weight sharing (see Figure 10.6), represented by

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{i=1}^{m_{\mathrm{C}}} \sum_{j=1}^{d+1-s} a_{ij} \tanh\left(\sum_{\alpha=1}^{s} x_{j+s-\alpha} K_{i;\alpha} \right), \tag{10.16}$$

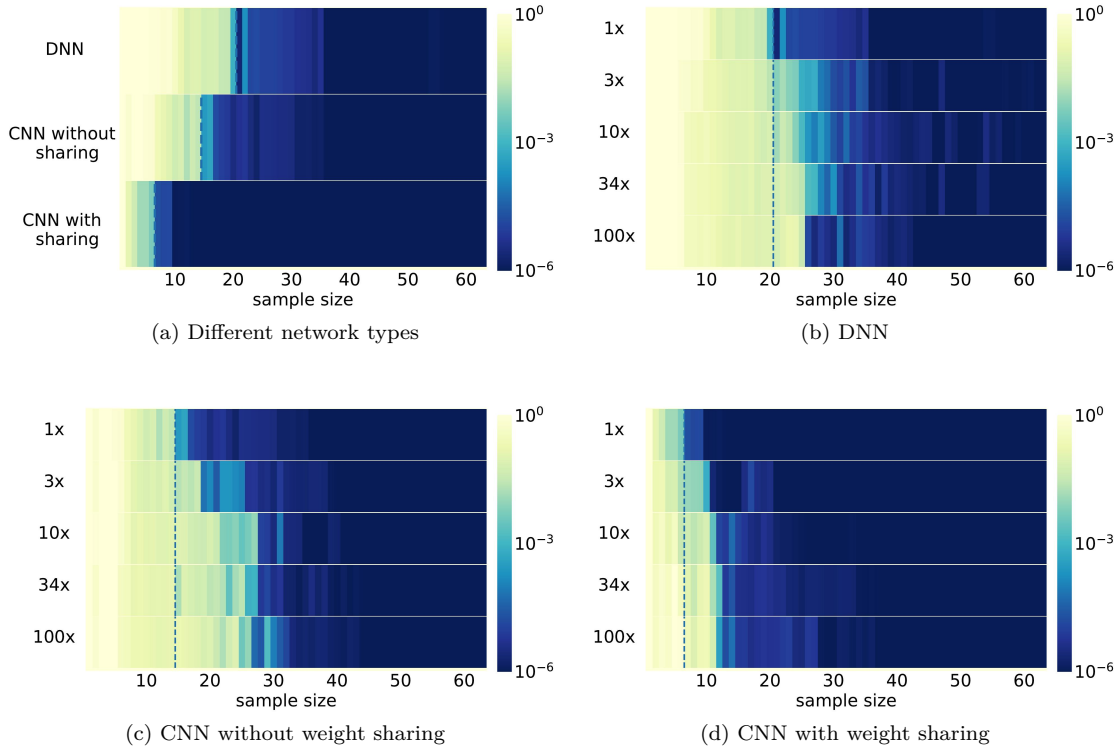
where $\boldsymbol{x} = [x_1, \cdots, x_d]^{\mathrm{T}} \in \mathbb{R}^d$, K_i is the convolution kernel (also called a neuron), s is the kernel size, a_{ij} is the output weight, and the convolution is 1D. For each i, K_i has s parameters, $(a_{ij})_{i=1}^{d+1-s}$ has $d+1-s$ parameters in total, so for each i, the expression $\sum_{j=1}^{d+1-s} a_{ij} \tanh\left(\sum_{\alpha=1}^{s} x_{j+s-\alpha} K_{i;\alpha}\right)$ has $d+1$ parameters. Therefore the model has $m_{\mathrm{C}}(d+1)$ parameters in total. Here m_{C} is the number of convolution kernels, analogous to the width of a fully-connected neural network. Similar to the intrinsic width defined in the fully-connected case, we define the intrinsic kernel number as:

Definition 3. *Intrinsic kernel number $k_{\mathrm{C}}(f^*)$: defined as the minimum number of convolution kernels of a convolutional network that can represent f^*. That is, f^* can be represented by a convolutional network with $k_{\mathrm{C}}(f^*)$ kernels, but cannot be represented by any convolutional network with fewer kernels.*

Note that if the convolutional network has weight sharing, then those neurons sharing weights are regarded as a single convolution kernel. In other words, one kernel can contain multiple neurons with the same weight. If weights are not shared, then there is only one neuron per kernel. Similar to the fully-connected case, the optimistic sample size is a function of the intrinsic kernel number $k_{\mathrm{C}}(f^*)$ of the target function f^*. Specifically, for any f^* that can be represented by an m_{C}-kernel CNN, the optimistic sample size is

$$R_{\mathrm{CNN}_{m_{\mathrm{C}}}}(f^*) = k_{\mathrm{C}}(f^*)(d+1). \tag{10.17}$$

Therefore, a function with intrinsic kernel number $k_{\mathrm{C}}(f^*) \ll m_{\mathrm{C}}$ can be recovered by a convolutional network under over-parameterization.



(a) Different network types

(b) DNN

(c) CNN without weight sharing

(d) CNN with weight sharing

Figure 10.3: Average test error (color) for neural networks of different widths (ordinate) and sample sizes (abscissa) when fitting the target function Eq. (10.18).

Figure 10.3 shows the average test errors of three different architectures of neural networks when fitting the same target function. In the figure, the ordinate indicates the network width and the abscissa indicates the number of samples. The target function used is:

$$f^*(\boldsymbol{x}) = \boldsymbol{W}^{*[2]} \tanh(\boldsymbol{W}^{*[1]}\boldsymbol{x}), \tag{10.18}$$

where $\boldsymbol{W}^{*[2]} = [1, 1, 1]$ and $\boldsymbol{W}^{*[1]} = \begin{bmatrix} 0.6 & 0.8 & 1 & 0 & 0 \\ 0 & 0.6 & 0.8 & 1 & 0 \\ 0 & 0 & 0.6 & 0.8 & 1 \end{bmatrix}$, $\boldsymbol{x} \in \mathbb{R}^4$. Figure 10.3 contains four subfigures showing the experimental results for neural networks of different architectures.

- **(a)**: comparison of three neural network architectures: two-layer tanh-CNN with 1 kernel; two-layer tanh-CNN with 1 kernel without weight sharing; and a two-layer fully-connected tanh-NN with width 3. In (b)-(d), these networks are referred to as 1x of their respective architectures.
- **(b)**: two-layer fully-connected tanh-NN with width $3N$, where $N = 1, 3, 10, 34, 100$, labeled as Nx.
- **(c)**: two-layer tanh-CNN with N kernels but without weight sharing.
- **(d)**: two-layer tanh-CNN with N kernels.

In all experiments, the network parameters were initialized from a normal distribution with mean 0 and variance 10^{-20}, and trained using gradient descent with fine-tuned learning rates. The input data of both the training and test datasets were sampled from a standard normal distribution, and the outputs were computed using the target function. The size of the training dataset varies, while the test dataset size is fixed at 1000. In each experimental setting, the learning rate is tuned between 0.05 and 0.5 to optimize generalization performance.

There are many phenomena to observe. First, in (a), we see that when recovering the target function (10.18), the tanh-CNN with weight sharing requires the fewest samples, followed by the tanh-CNN without weight sharing, while the tanh-NN requires the most samples. Second, in (b), (c), and (d), we find that the neural network has a strong ability to recover the target function under over-parameterization. For example, in the 100x row of (b), the network has 700 parameters in total, but only about 25 samples are needed to recover the target function. Third, we find that in (b), (c), and (d), no matter how wide the network is, the sample size required to recover the target function is around the dashed line, and seems to be little related to the actual width of the network.

Note that in this experiment, the parameters are initialized near $\mathbf{0}$ with small initialization, not under optimistic initialization. That is, even when the initialization is not near the target set, the neural network can still recover the target function when the sample size is near the optimistic sample size. This indicates that the actual performance of the network is close to its performance under optimistic initialization.

10.2.4 The role of hyperparameter tuning in nonlinear models

Neural networks have many hyperparameters that can be tuned, such as the variance of the Gaussian distribution used for initialization, the learning rate, and so on. On the one hand, if these hyperparameters are chosen poorly, the generalization error of the network may be very large. For example, in Figure 10.4, we use a neural network with four hidden layers to fit the blue star-shaped data points. Each layer has 500 neurons, and the activation function is Tanh. The difference between the two figures is that the variance of the Gaussian distribution used for parameter initialization differs. The initialization variance in the left figure is small, while that in the right figure is large. We find that if the initialization variance is large, then the learned solution will be highly

oscillatory, resulting in poor generalization error. On the other hand, even if the hyperparameters are very well tuned, the recovery ability always has an upper limit. For example, in the matrix factorization problem above, to recover a 4×4 matrix of rank 1, no matter how well the hyperparameters are chosen, at least 7 samples are required to recover the target matrix. In the earlier neural network experiments, even if the initialization is very small and the condensation is very strong, we still need a certain sample size to recover the target function.

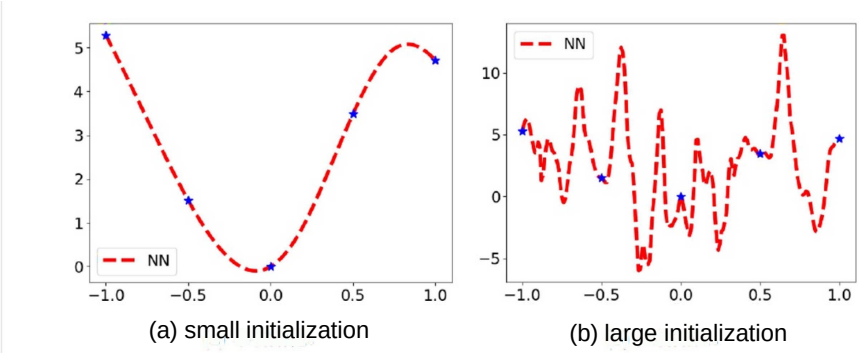


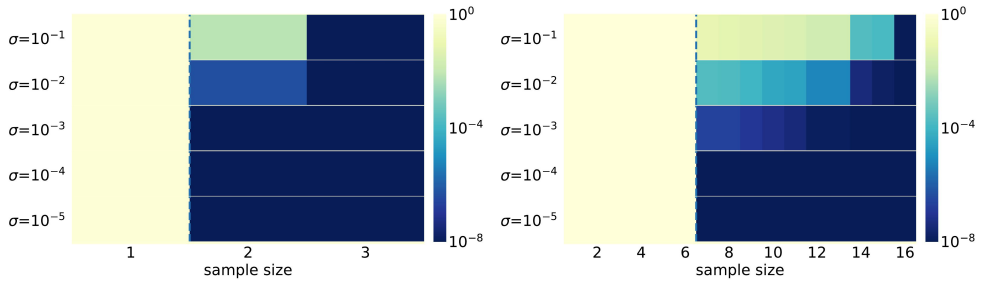
Figure 10.4: Results of a 1D fitting example.

To illustrate more carefully how hyperparameters affect the recovery ability of the model, in Figure 10.5 we show, for both the simple task of this section and the matrix factorization task, the relationship between the average test error (color) and the number of training samples (abscissa) under different initialization standard deviations (ordinate). Figure (a) shows the experimental results of fitting $1 + x$ using $f_{\mathrm{NL}}(x, \theta)$. Figure (b) is the result of fitting a rank-1 matrix using the matrix factorization model. The yellow dashed line indicates the transition to the optimistic sample size. Each test error is averaged over 50 trials of random initialization. We find that in Figure (a), if the initialization standard deviation is greater than 10^{-2}, then 3 sample points are needed to recover the target function. But if the initialization standard deviation is less than 10^{-3}, only 2 sample points are needed to recover the target function. A similar phenomenon occurs in Figure (b): under small initialization, the sample size required for recovery is smaller. However, even when the initialization is very small, the sample size required for recovery has a lower bound that cannot be broken.

10.3 Analyzing neural network architecture design: does the optimistic sample size increase?

Analyzing neural networks from the perspective of the optimistic sample size, we observe two important properties.

The first property is the **free expressive power from width**. In fully-connected and convolutional networks, the optimistic sample size of a target function depends only on the intrinsic width (or intrinsic kernel number) of the target function, and is independent of the width of the network. This shows that for two-layer fully-connected networks and convolutional networks, increasing the width (or the number of kernels) can increase their expressive power without increasing the opti-



(a) Fitting $1 + x_1$ with f_{NL} (b) Matrix factorization, target rank 1

Figure 10.5: Average test error (color) versus the number of training samples (abscissa) under different initialization standard deviations (ordinate).

mistic sample size. Since the sample size actually needed for "recovery" is close to the optimistic sample size, we may regard the sample size required for "recovery" as essentially unchanged.

The second property is the **expensive expressive power from connections**. In Table 10.1, we show the optimistic sample sizes of the same target function under different neural network architectures. In the table, $m_{\text{null}} = |\{(l, i, j)|a^*_{lij} = 0\}|$ denotes the number of zero output weights in the target function, and $\mathcal{F}_k^{\text{CNN}}$ denotes the set of all target functions that can be represented by a weight-sharing CNN with k convolution kernels. Let us give an example to compute the difference of optimistic sample sizes under different architectures. Suppose the target function f^* satisfies $f \in \mathcal{F}_k^{\text{CNN}} \backslash \mathcal{F}_{k-1}^{\text{CNN}}$, and all output weights of f^* are not equal to 0, i.e., $m_{null} = 0$. The kernel size is $s = 3$, and the input image size is 28×28, i.e., $d = 28$. In this case, the optimistic sample sizes of f^* in CNN, CNN (without weight sharing), and fully-connected network are $709k$, $6760k$, and $530660k$, respectively. We see that the sample sizes required to recover the same target function under different architectures differ enormously. When the target function can be represented by a CNN, the optimistic sample size of the CNN is much smaller than that of the other two architectures. Figure 10.6 illustrates the architectures of these three networks; from left to right: a two-layer fully-connected network, a convolutional network without weight sharing, and a convolutional network with weight sharing. The convolutions in this illustration are 1D. In the figure, we can see that going from a weight-sharing convolutional network to a fully-connected network adds many connections between neurons. When the target function can be represented by a weight-sharing CNN, these additional connections are unnecessary. Although these unnecessary connections enhance the representational ability of the neural network, they do so at the cost of sample efficiency. This property is called the **expensive expressive power from connections**.

In general, we refer to model properties obtained from the optimistic estimate as their optimistic properties. Above, we have revealed two optimistic properties of two-layer neural networks: the free expressive power from width and the expensive expressive power from connections. These properties suggest two architectural design principles for neural networks: (i) freely adding neurons/kernels, and (ii) limiting the connections between neurons. These two principles agree with common practice: people would rather expand the width (or kernel number) than add connections to increase the expressive power of a neural network.

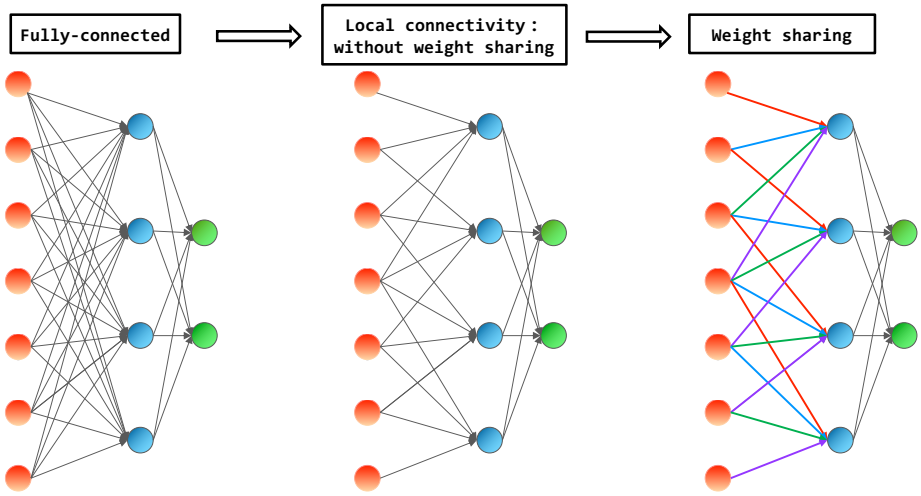


Figure 10.6: Schematic illustration of different network structures.

f^*	CNN	CNN (without weight sharing)	Fully-connected network
$\{0\}$	0	0	0
$\mathcal{F}_1^{\text{CNN}} \backslash \{0\}$	$s^2 + (d+1-s)^2$	$(s^2+1)(d+1-s)^2 - (s^2+1)m_{\text{null}}$	$(d^2+1)(d+1-s)^2 - (d^2+1)m_{\text{null}}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\mathcal{F}_k^{\text{CNN}} \backslash \mathcal{F}_{k-1}^{\text{CNN}}$	$k(s^2 + (d+1-s)^2)$	$k(s^2+1)(d+1-s)^2 - (s^2+1)m_{\text{null}}$	$k(d^2+1)(d+1-s)^2 - (d^2+1)m_{\text{null}}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\mathcal{F}_m^{\text{CNN}} \backslash \mathcal{F}_{m-1}^{\text{CNN}}$	$m(s^2 + (d+1-s)^2)$	$m(s^2+1)(d+1-s)^2 - (s^2+1)m_{\text{null}}$	$m(d^2+1)(d+1-s)^2 - (d^2+1)m_{\text{null}}$

Table 10.1: Optimistic sample sizes of three architectures.

Chapter 11

Flatness of Solutions

Neural networks are powerful machine learning tools, yet their internal workings resemble a complex topographical map. Because neural networks have a large number of parameters and are highly nonlinear, their learning process searches for an optimal solution within a loss landscape filled with "peaks" and "valleys." These "valleys" are minima that represent the different performance of a neural network on training data. However, different minima can perform very differently: some yield excellent models, while others perform only modestly. How can we distinguish good minima from bad ones? An important perspective is to look at the surrounding "terrain"—that is, the geometry of the loss landscape.

Imagine the loss landscape as a range of rolling hills. Some valleys are broad and flat, like wide-open plains; others are narrow and steep, as if plunging into a canyon. Near a flat valley, the loss function changes gradually. Even when the parameters are perturbed slightly, the model's performance does not fluctuate dramatically, so the model usually performs better. Conversely, near a steep valley, the loss function is highly sensitive to changes in the parameters, so even tiny perturbations can cause a large drop in performance, and the model's behavior is naturally unstable.

Therefore, studying the "flatness" of local minima becomes a key to understanding the performance of neural networks. A flat minimum is like a solid "stronghold" that helps the model remain robust on new data, whereas a sharp minimum is like a "danger zone" that easily leads to poor performance.

Popular Introduction

What is the flatness of solutions?

The flatness of a solution refers to how dramatically the loss function changes near the solution found by a neural network. Imagine the loss function as a range of rolling hills, with the "solution" being a low point in a valley. If the terrain around this low point is gentle (like a broad valley floor), then it is a "flat solution," and the loss function changes mildly; if the surrounding terrain is steep (like a sharp pit), then it is a "sharp solution," and the loss function changes drastically. A flat solution usually means more stable performance of the neural network, because even if the parameters change slightly, the loss does not fluctuate much.

What factors affect the flatness of solutions?

- **Batch size** Small-batch training tends to converge to flat solutions, while large-batch training tends to converge to sharp solutions.
- **Noise structure of stochastic gradient descent** The magnitude of the noise in stochastic gradient descent has an inverse relationship with the curvature of the loss function. This property allows stochastic gradient descent to escape regions of high curvature more quickly, and makes it more inclined to select flat solutions.
- **Dropout** The implicit regularization of Dropout drives the neural network to converge to flatter solutions.

11.1 Definition of Flatness of Solutions

The flatness of a solution describes how dramatically the loss function changes near the solution found by the neural network. So how do we precisely measure this magnitude of change? We begin our analysis with the simple quadratic function $y = ax^2$. As shown in Fig. 11.1(a), if we perturb the function slightly near $x = 0$, the value of $y = x^2$ changes relatively gently, while the value of $y = 5x^2$ changes more drastically. Therefore, for a one-dimensional quadratic function, the magnitude of the quadratic coefficient directly characterizes the flatness of the solution. From a mathematical viewpoint, the quadratic coefficient is exactly the second derivative at this point.

When we extend to a function with high-dimensional input, the second derivative becomes the Hessian matrix. How then can we use the Hessian matrix to characterize the flatness of the solution? Take the two-dimensional function $f(x, y) = 5x^2 + y^2$ as an example. As shown in Fig. 11.1(b), the function value changes significantly faster along the x-axis than along the y-axis, indicating that the curvature is steeper along the x-axis. Mathematically, the Hessian matrix quantifies this difference. For this function, the Hessian matrix is:

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} 10 & 0 \\ 0 & 2 \end{bmatrix}. \tag{11.1}$$

The eigenvalues of the Hessian matrix are $\lambda_1 = 10$ and $\lambda_2 = 2$, corresponding respectively to the curvatures along the x-axis and the y-axis. The fact that $\lambda_1 > \lambda_2$ indicates that the curvature along the x-axis is larger, so the magnitude of the eigenvalues of the Hessian matrix can be used to characterize the flatness in different directions. More generally, each eigenvalue is the curvature of the function along its corresponding eigendirection.

One method is to define flatness through the change of the loss function along a fixed direction (Feng and Tu, 2021). Specifically, consider the change of the loss function at a minimum $\boldsymbol{\theta}_t$ along a fixed direction $\boldsymbol{p}_i$, defined as:

$$L_i(\delta\boldsymbol{\theta}) = L(\boldsymbol{\theta}_t + \delta\boldsymbol{\theta}\boldsymbol{p}_i). \tag{11.2}$$

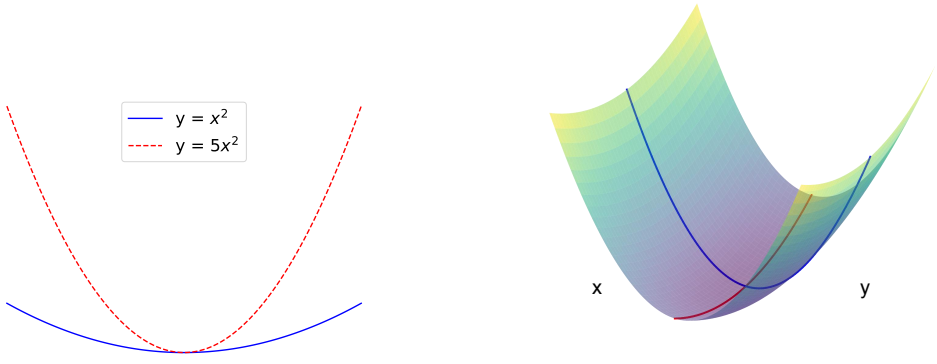
Based on this, the flatness of the loss function along the direction $\boldsymbol{p}_i$, denoted F_i, is defined as:

$$F_i \equiv \theta_i^r - \theta_i^l, \tag{11.3}$$

where $\theta_i^l < 0$ and $\theta_i^r > 0$ are respectively the closest points on the left and right sides of the loss function L_i along the $\boldsymbol{p}_i$ direction satisfying $L_i(\theta_i^l) = L_i(\theta_i^r) = e \cdot L_0$, with e being the base of the

natural logarithm and L_0 the value of the loss function at the minimum $\boldsymbol{\theta}_t$. The larger F_i is, the flatter the loss function is along the $\boldsymbol{p}_i$ direction.

Another method is to characterize flatness via the largest eigenvalue of the Hessian matrix. For different minima, the largest eigenvalue of the Hessian matrix $\lambda_{\max}$ reflects the magnitude of the curvature along the steepest direction at that point. By comparing $\lambda_{\max}$ of different minima, we can effectively compare the flatness of the solutions. The smaller $\lambda_{\max}$, the flatter the loss landscape at that minimum. Some studies have also shown that using the sum of all eigenvalues, i.e., the trace, can be a more effective way to characterize flatness.



(a) $y = ax^2, a = 1\ 5$ (b) $f(x, y) = 5x^2 + y^2$

Figure 11.1: Illustration of quadratic functions.

The flatness of solutions inspires a way to understand the generalization ability of models. Specifically, as shown in Fig. 11.2, when we only find a solution approximately—for example, due to insufficient data, noise, or insufficient training—then at a flat minimum the loss function changes gently in its neighborhood, i.e., the function surface is relatively flat, so the generalization ability of a flat solution remains good. However, at a sharp minimum, the loss function changes drastically in its neighborhood, i.e., the function surface is steep, and the sharp solution is very sensitive to small changes in the parameters, so its generalization performance is poor.

11.2 Effect of Batch Size on the Flatness of Solutions

In training deep neural networks, it has been observed that varying the batch size used in stochastic gradient descent leads to significant differences in the generalization performance of the model. With a smaller batch size, the model tends to generalize better, achieving higher accuracy on the test data; while with a larger batch size, the model's generalization performance on the same test data drops significantly. As listed in Table 11.1, results for six network architectures are shown, where F1 and F2 are fully connected networks and the remaining four are convolutional networks. Using a small batch (batch size of 256) clearly yields better generalization than using a large batch (10% of the whole dataset).

It is puzzling that, regardless of whether a large or small batch is used, the model performs

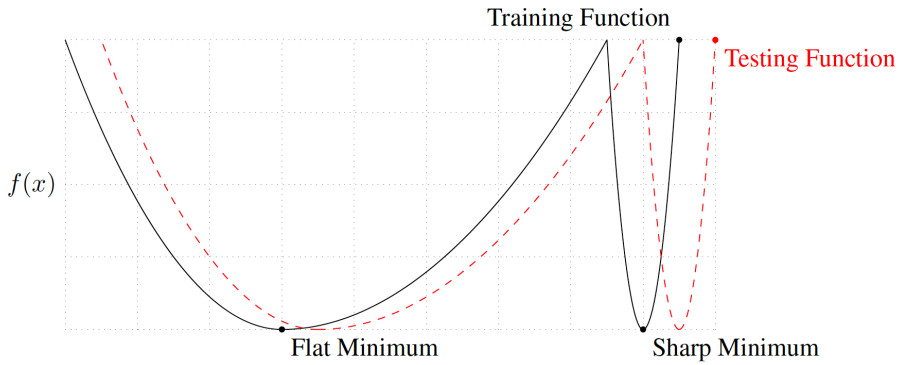


Figure 11.2: Conceptual sketch of the effect of flat and sharp solutions on generalization ability. Image source: Keskar et al. (2016).

Table 11.1: Performance of ADAM with small-batch and large-batch variants on the six networks listed. Source: Keskar et al. (2016).

Network	Training accuracy		Test accuracy	
	Small batch	Large batch	Small batch	Large batch
F1	99.66% ± 0.05%	99.92% ± 0.01%	98.03% ± 0.07%	97.81% ± 0.07%
F2	99.99% ± 0.03%	98.35% ± 2.08%	64.02% ± 0.2%	59.45% ± 1.05%
C1	99.89% ± 0.02%	99.66% ± 0.2%	80.04% ± 0.12%	77.26% ± 0.42%
C2	99.99% ± 0.04%	99.99% ± 0.01%	89.24% ± 0.12%	87.26% ± 0.07%
C3	99.56% ± 0.44%	99.88% ± 0.30%	49.58% ± 0.39%	46.45% ± 0.43%
C4	99.10% ± 1.23%	99.57% ± 1.84%	63.08% ± 0.5%	57.81% ± 0.17%

well on the training set, with very similar training loss values, yet the test-set performance differs significantly. Keskar et al. (2016) conducted an in-depth study and analysis of this phenomenon.

Keskar et al. (2016) found that large-batch training methods tend to converge to sharp minima of the loss function, while small-batch training methods are more likely to converge to flat minima. As shown in Fig. 11.3, the results for a VGG network on CIFAR100 are presented. In parameter space, the loss values and accuracies corresponding to $\theta = \alpha\theta_L + (1-\alpha)\theta_S$ are considered, where θ_L and θ_S are the solutions from large-batch and small-batch training, respectively. The figure shows that the neighborhood of the small-batch solution appears flatter, while the neighborhood of the large-batch solution is clearly sharper.

11.3 Effect of Stochastic Gradient Descent on the Flatness of Solutions

Batch size has an important impact on the flatness of solutions, so what about the effect of stochastic gradient descent itself on the flatness of solutions?

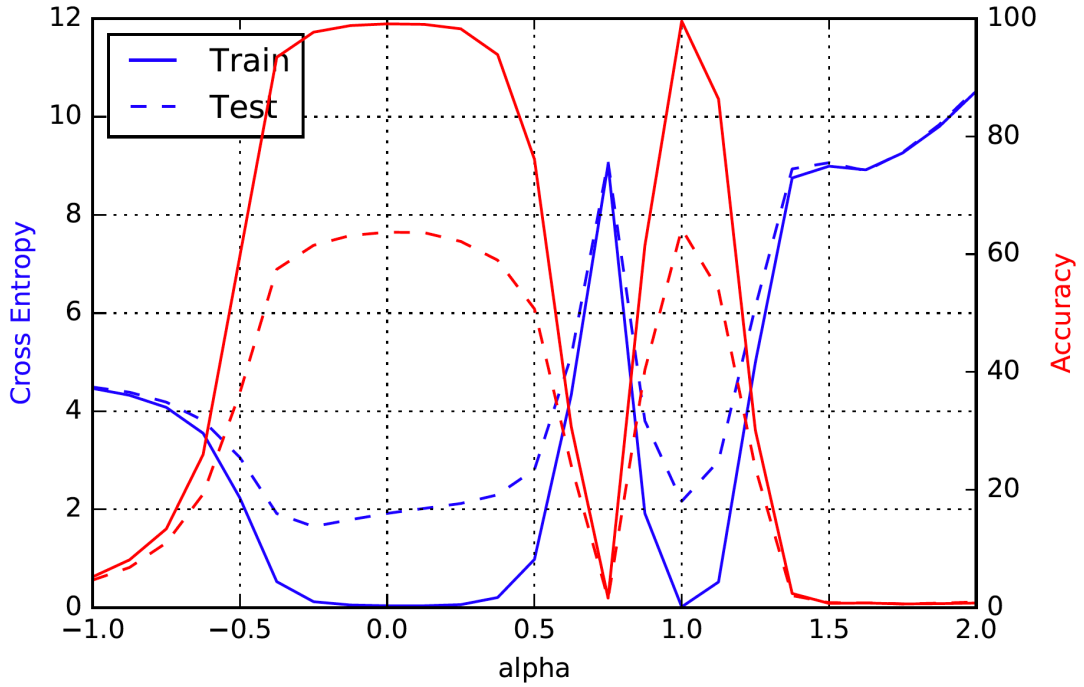


Figure 11.3: One-dimensional parameter curve (the left vertical axis corresponds to cross-entropy loss, and the right vertical axis to classification accuracy; solid lines denote the training set and dashed lines the test set); $\alpha = 0$ corresponds to the small-batch minimum, and $\alpha = 1$ corresponds to the large-batch minimum. Image source: Keskar et al. (2016).

11.3.1 The Importance of the Noise Structure of Stochastic Gradient Descent

In the optimization of deep learning models, stochastic gradient descent (SGD) is a widely used optimization algorithm. The noise structure of stochastic gradient descent has an important impact on the model's generalization performance. The study of Zhu et al. (2018) shows that different random noise structures lead to significant differences in model performance, and that the noise of stochastic gradient descent has a very special structure that is strongly correlated with the Hessian matrix of the loss landscape.

The update rule of stochastic gradient descent can be written as:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) + \eta \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \Sigma_{sgd}^t) \tag{11.4}$$

where η is the learning rate and Σ_{sgd}^t is the covariance matrix of the SGD noise, which has an anisotropic structure. Gradient Langevin dynamics (GLD) can be viewed as a variant of stochastic gradient descent, with the update rule:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) + \eta \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma_t^2 I) \tag{11.5}$$

where the noise term follows an isotropic Gaussian distribution.

Fig. 11.4 shows a performance comparison of several different optimization methods on the SVHN and CIFAR-10 datasets, with the noise structures of these methods summarized in Table 11.2. Stochastic gradient descent achieves the best test accuracy, while other optimization methods such as GD or GLD variants with different noise structures have significantly lower test accuracy than stochastic gradient descent. This indicates that the special noise structure of stochastic gradient descent is the key to its better generalization performance compared with other optimization methods. Zhu et al. (2018) found that Σ^t_{sgd} is to some extent aligned with the Hessian matrix H^t of the loss landscape; their results show that $H^t\Sigma^t_{sgd}$ is significantly larger than the product of H^t and an isotropic covariance matrix (with eigenvalues equal to the average of the eigenvalues of Σ^t_{sgd}).

Dynamics	Noise ϵ_t	Remarks
GLD constant	$\epsilon_t \sim \mathcal{N}(0, \varrho_t^2 I)$	ϱ_t is a tunable constant.
GLD dynamic	$\epsilon_t \sim \mathcal{N}(0, \sigma_t^2 I)$	σ_t shares the same magnitude with Σ^t_{sgd}.
GLD diagonal	$\epsilon_t \sim \mathcal{N}(0, \mathrm{diag}(\Sigma^t_{sgd}))$	$\mathrm{diag}(\Sigma^t_{sgd})$ is the diagonal of Σ^t_{sgd}

Table 11.2: Noise structures of different optimization methods. Table source: Zhu et al. (2018).

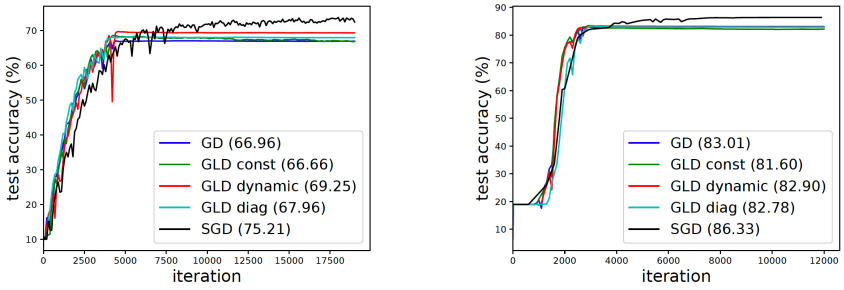


Figure 11.4: Performance comparison of different optimization dynamics on the SVHN (left) and CIFAR-10 (right) datasets. Image source: Zhu et al. (2018).

11.3.2 Relationship between SGD Noise and the Flatness of Solutions

Why is the noise structure of stochastic gradient descent so special? Feng and Tu (2021) further found that the variance of stochastic gradient descent noise has an inverse relationship with the curvature of the loss function. We adopt the flatness of solutions defined in Eq. 11.3.

Let $\boldsymbol{p}_i$ be the eigendirection corresponding to the i-th eigenvalue of Σ^t_{sgd}, and let σ_i^2 characterize the variance of the noise in the direction $\boldsymbol{p}_i$. As shown in Fig. 11.5, the experiment is conducted on a simple fully connected neural network with two hidden layers, each containing 50 neurons, used for a classification task on the MNIST database. B is the batch size and α is the learning rate. The variance σ_i^2 of the stochastic gradient descent noise and the flatness F_i of the loss function exhibit a power-law inverse relationship:

$$\sigma_i^2 \sim F_i^{-\psi} \tag{11.6}$$

where ψ is close to 4 under various hyperparameter settings. This indicates that in directions where the loss function is flatter, the variance of the stochastic gradient descent noise is smaller; while in directions where the loss function is steeper, the variance of the stochastic gradient descent noise is larger. This adaptive coupling between the SGD noise and the curvature of the loss function enables stochastic gradient descent to escape sharp minima more quickly, making it more inclined to converge to broad, flat minima, which may be one of the reasons for its excellent generalization performance.

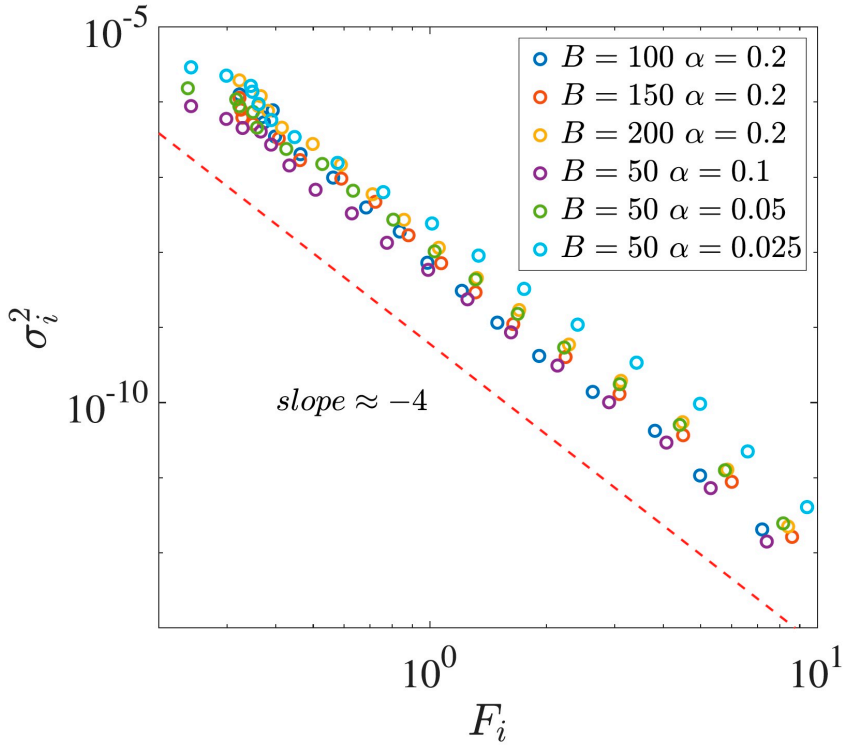


Figure 11.5: The inverse relationship between the variance of stochastic gradient descent noise and the flatness of the loss function. Image source: Feng and Tu (2021).

11.3.3 Theoretical Analysis of the Implicit Regularization of Stochastic Gradient Descent

Smith et al. (2020) analyzed from the perspective of an effective gradient flow and found that stochastic gradient descent is equivalent to adding a regularization term to the gradient flow. Suppose $L(\boldsymbol{\theta})$ is the original loss function, $\nabla L(\boldsymbol{\theta})$ is its gradient, η is the learning rate, m is the number of minibatches per epoch, and $\widehat{L}_i(\boldsymbol{\theta})$ is the loss function corresponding to the i-th minibatch. The

new regularization term can be expressed as:

$$\frac{\eta}{4m}\sum_{i=0}^{m-1}||\nabla\widehat{L}_i(\boldsymbol{\theta})-\nabla L(\boldsymbol{\theta})||^2. \tag{11.7}$$

Since each minibatch is randomly sampled, $\mathbb{E}(\nabla\widehat{L}_i(\boldsymbol{\theta}))=\nabla L(\boldsymbol{\theta})$. This is equivalent to finding a solution during training that minimizes the variance of the gradients across different minibatches.

In fact, this can be understood in the same framework as Dropout discussed earlier. If during training some parts are randomly dropped and the remaining parts compensate for them, then the training process will make the compensating parts and the dropped parts as similar as possible. For Dropout, what is dropped is the output of neurons, so the outputs of other neurons in the same layer must compensate, and as a result training drives the outputs of neurons in the same layer toward each other, i.e., the condensation phenomenon. For stochastic gradient descent, what is dropped is the gradient of the loss on some samples, and the gradients of the losses on the other samples compensate (through averaging), so the training makes the discrepancies between the gradients from different batches as small as possible. A flat solution can achieve this, so this regularization term also provides an explanation for why stochastic gradient descent tends to find flat solutions.

11.4 Effect of Dropout on the Flatness of Solutions

In this section, we study the flatness of the loss landscape at the minima found by dropout.

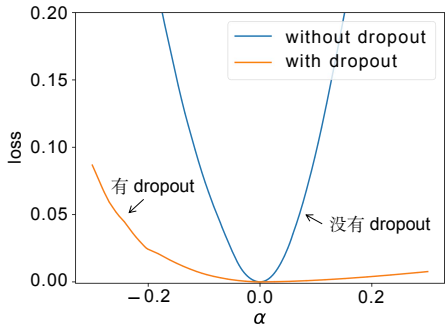


Figure 11.6: Experimental results of training a two-layer ReLU neural network with and without dropout. Image source: Zhang and Xu (2022).

To fairly compare the flatness between different models, we adopt the method used in Li et al. (2017), as follows. We use the function $L(\alpha)=R_S\left(\boldsymbol{\theta}+\alpha\boldsymbol{d}\right)$ to obtain the loss landscape around the minimum, i.e., we vary the value of α along the direction $\boldsymbol{d}$ and observe how the value of the loss function changes.

We first verify the effect of dropout on flatness with a one-dimensional simple example. We use a ReLU fully connected neural network of width 1000 to fit the target function,

$$f(x)=\frac{1}{2}\sigma(-x-\frac{1}{3})+\frac{1}{2}\sigma(x-\frac{1}{3}),$$

Table 11.3: Effect of dropout on model accuracy.

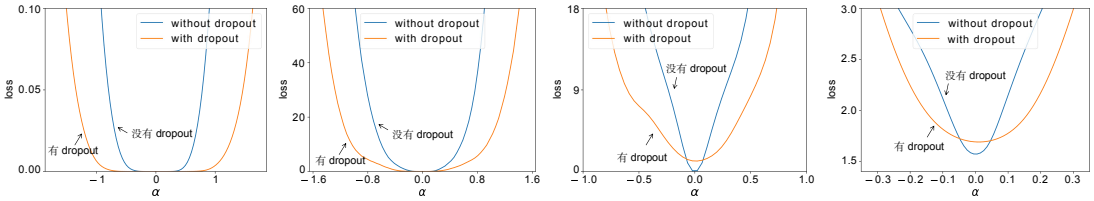
Network	Dataset	With dropout	Without dropout
FNN	MNIST	98.7%	98.1%
VGG-9	CIFAR-10	60.6%	59.2%
ResNet-20	CIFAR-100	54.7%	34.1%
Transformer	Multi30k	49.3%	34.7%

where $\sigma(x) = \text{ReLU}(x)$. We train the network with Adam at a learning rate of 1×10^{-4}. We use large initialization parameters, $\boldsymbol{\theta} \sim N\left(0, \frac{1}{m^{0.2}}\right)$, where $m = 1000$ is the width of the hidden layer.

From Fig. 11.6, it is easy to see that for the network trained with dropout, the loss landscape around its minimum is flatter. In the following, we focus on more complex tasks and compare the flatness and generalization ability of networks trained with and without dropout layers.

For all network architectures shown in Fig. 11.7, dropout improves the generalization of the network and finds flatter minima. In Fig. 11.7(a, b), for networks trained with and without dropout layers, the training loss values are both close to zero, yet their flatness and generalization abilities still differ. In Fig. 11.7(c, d), due to the complexity of the datasets (i.e., CIFAR-100 and Multi30k) and the network architectures (i.e., ResNet-20 and Transformer), networks with dropout struggle to achieve zero training error, but networks with dropout layers still find flatter minima and achieve better generalization. The accuracies of the different networks are listed in Table 11.3.

The detailed settings are as follows: Fig. 11.7(a) shows an FNN trained on the MNIST dataset. The test accuracy of the model with the dropout layer is 98.7%, while the test accuracy of the model without the dropout layer is 98.1%. Fig. 11.7(b) shows a VGG-9 network trained on the CIFAR-10 dataset, using the first 2048 examples as the training set. The test accuracy of the model with the dropout layer is 60.6%, while the test accuracy of the model without the dropout layer is 59.2%. Fig. 11.7(c) shows a ResNet-20 network trained on the CIFAR-100 dataset, using all examples as the training set. The test accuracy of the model with the dropout layer is 54.7%, while the test accuracy of the model without the dropout layer is 34.1%. Fig. 11.7(d) shows a Transformer trained on the Multi30k dataset, using the first 2048 examples as the training set. The test accuracy of the model with the dropout layer is 49.3%, while the test accuracy of the model without the dropout layer is 34.7%.



(a) Loss landscape of FNN (b) Loss landscape of VGG-9 (c) Loss landscape of ResNet-20 (d) Loss landscape of Transformer

Figure 11.7: One-dimensional visualization of the loss landscapes of solutions for different network architectures obtained with and without dropout layers. Image source: Zhang and Xu (2022).

11.5 Edge of Stability Phenomenon

Training stability is a key consideration in the training process of neural networks. Unstable training usually prolongs the training time and may even affect the model's convergence. In flat regions of parameter space, the function value changes gently around the parameters, so a larger learning rate can be used to accelerate training; in steep regions, the function value changes drastically, so only a small learning rate can be used to ensure stability. So how can we quantitatively describe the relationship between flatness and learning rate?

We first consider a simple quadratic function:

$$f(x) = \frac{1}{2}\lambda x^2 \tag{11.8}$$

where $\lambda > 0$ is a constant. If we use gradient descent (GD) to optimize this function, the update rule is:

$$x_{t+1} = x_t - \eta \nabla f(x_t) = (1 - \eta\lambda)x_t \tag{11.9}$$

where η is the learning rate. To ensure convergence, we require $|1 - \eta\lambda| < 1$, i.e.,

$$0 < \eta < \frac{2}{\lambda} \tag{11.10}$$

This inequality gives the stability condition for GD: the learning rate cannot exceed $2/\lambda$, otherwise the optimization process will diverge.

As mentioned in Section 11.1, we can use the largest eigenvalue of the Hessian matrix to characterize the flatness of the loss landscape near a local minimum of a neural network. Analogously, the stability condition for GD becomes:

$$0 < \eta < \frac{2}{\lambda_{\max}}. \tag{11.11}$$

If the learning rate exceeds this threshold, the optimization process becomes unstable, exhibiting severe oscillations. In neural network training, the sharpness often stays at the edge $2/\eta$, a phenomenon known as the "Edge of Stability" (EoS).

In a 2018 study, Lei Wu, Chao Ma, and Weinan E (Wu et al., 2018) systematically investigated for the first time the Edge of Stability phenomenon in the training of deep neural networks. Table 11.4 shows the experimental results of the means and standard deviations of the sharpness at the end of training across different datasets and different learning rates. Each experiment was repeated 5 times, each with an independent random initialization. The fourth row of the table shows the maximum possible sharpness predicted by theory. They observed that at the end of training, in most cases the learning rate and $2/\lambda_{\max}$ are very close, indicating that the optimization process is at the edge of stability.

In a 2020 study, Cohen et al. (Cohen et al., 2020) further revealed the universality of the Edge of Stability phenomenon. They conducted experiments on multiple datasets and network architectures. The results are shown in Fig. 11.8, where the horizontal axis represents the training step and the vertical axis represents the sharpness, with the dashed line marking $2/\eta$. On three different architectures, one can observe that the sharpness rises to $2/\eta$ (marked by horizontal dashed lines of the corresponding colors), and then fluctuates around this value or stays slightly above it.

The Edge of Stability phenomenon has important implications for both the theory and practice of deep learning. First, it shows that the optimization process of deep neural networks is not always

η	0.01	0.05	0.1	0.5	1	5
FashionMNIST	53.5 $\pm$ 4.3	39.3 $\pm$ 0.5	19.6 $\pm$ 0.15	3.9 $\pm$ 0.0	1.9 $\pm$ 0.0	0.4 $\pm$ 0.0
CIFAR10	198.9 $\pm$ 0.6	39.8 $\pm$ 0.2	19.8 $\pm$ 0.1	3.6 $\pm$ 0.4	-	-
prediction $2/\eta$	200	40	20	4	2	0.4

Table 11.4: Sharpness of the solutions obtained by the GD algorithm at different learning rates. Table source: Wu et al. (2018).

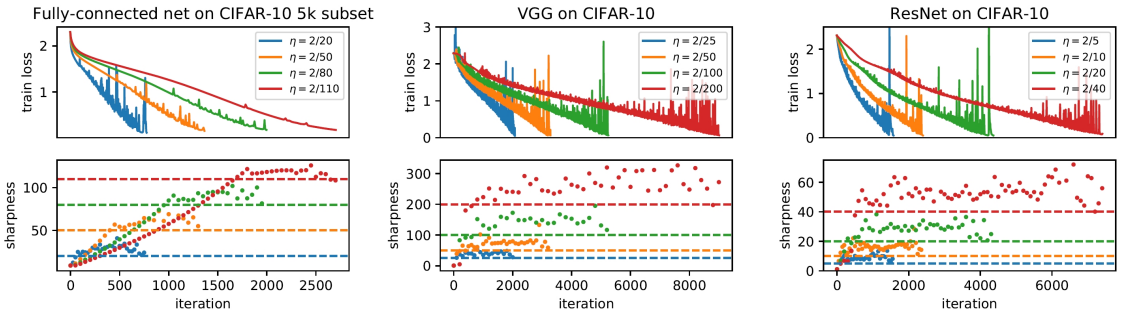


Figure 11.8: Gradient descent typically operates at the edge of stability. Image source: Cohen et al. (2020), Figure 1.

smooth, but is often in a dynamically critical state. This state is neither monotonically convergent like traditional convex optimization, nor fully divergent, but exhibits complex dynamics between the two. Second, the Edge of Stability phenomenon reminds us to be especially careful in choosing the learning rate. On the one hand, the learning rate should not be too small, otherwise the training speed will be very slow and the solution found may be sharp; on the other hand, the learning rate should not be too large, otherwise it will cause instability. Achieving a balance between the two requires a deeper understanding of the structure of the problem and tuning according to the specific situation. The Edge of Stability phenomenon is a clearly characterized phenomenon that reveals the complexity of the optimization process in deep learning.

Chapter 12

Anchor Functions: A Class of Simple Functions for Studying Language Models

In recent years, large language models (LLMs) have achieved tremendous success and have been playing an increasingly important role in natural language processing. These models have been widely applied to many tasks such as machine translation, text summarization, question answering, and sentiment analysis.

At present, language models based on the Transformer architecture Vaswani et al. (2017) have become an important paradigm. By using self-attention to capture both short- and long-range dependencies between words, Transformer models support parallel computation and handle longer text sequences better than traditional recurrent neural networks (RNNs) and long short-term memory networks (LSTMs).

The GPT series of models Brown et al. (2020); OpenAI (2023) have demonstrated the remarkable capabilities of language models on tasks such as text generation, dialogue, and question answering. By continuously scaling up model size and training data, large language models such as GPT have exhibited near-human-level language understanding and generation abilities, attracting wide attention from both academia and industry.

Despite these impressive achievements, our understanding of the inner workings of language models is still relatively limited. How exactly do language models learn and represent linguistic knowledge, and how do they transfer and generalize on downstream tasks? These questions remain to be further explored. Therefore, an in-depth study of the theoretical foundations and optimization mechanisms of Transformer-based language models is of great significance for advancing the development and application of language models.

This chapter discusses in detail a class of simple functions for studying language models—*anchor functions*—and analyzes their effects on and potential applications to language models through a variety of language-like task experiments. The contents are organized as follows: Section 1 discusses the challenges of studying Transformer-based language models, especially those concerning unknown tasks, high computational and memory demands, and the difficulty of interpreting the reasoning mechanism. Section 2 describes the characteristics of language tasks, focusing on logical inference

relations and the concept of "anchor–key-value" pairs in natural language, providing an entry point for studying Transformer models. Section 3 proposes a research approach based on anchor functions and language-like tasks, drawing on experimental methods from biology and physics by designing simplified experimental settings to explore the behavioral properties of language models. Section 4 introduces in detail the definitions of anchor functions and language-like tasks, illustrating with concrete examples how they can be used to study language models, with particular focus on the construction and analysis of single-anchor and multi-anchor functions. Section 5 discusses data partitioning and model generalization, proposing three methods: modulo-remainder data partitioning, anchor-based data partitioning, and composite anchor partitioning; it also discusses the concepts of data generalization and task generalization. Section 6 presents a variety of experimental results based on anchor functions and discusses the model's behavior and mechanisms on identity-learning tasks, reading-comprehension tasks, classification tasks, composite tasks, working-memory tasks, synonym tasks, forward–backward recitation tasks, statistical-output tasks, and multi-anchor tasks. Section 7 studies the mechanism of the identity-learning task: by analyzing a simplified two-layer model, it reveals the inner mechanism of Transformer models in shift and broadcast operations and discusses its generality in large language models. Through these contents, this chapter explores in depth the inner workings of large language models when handling complex language tasks, and proposes the anchor-function perspective as an important reference for future research.

Popular Science Section

What difficulties does current large-model research face?

In studying Transformer-based large language models, researchers face several challenges. The main difficulties include unknown tasks, high computational and memory demands, and the limited interpretability of the reasoning mechanism. Large language models are usually trained via next token prediction; although this approach performs excellently on a variety of language tasks, the training objective is not directly tied to specific tasks, which makes it difficult to explain the model's performance on any given task. In addition, training high-quality large language models requires large amounts of data and computational resources, which poses a huge challenge for academic research teams. Finally, understanding the reasoning process of large language models is highly intricate: even when the model produces the correct output, it is hard to explain its decision process.

What are anchor functions and language-like tasks? What is the motivation?

Anchor function: An anchor function is a mapping from a language sequence to its label, determined by the anchors appearing in the sentence. Depending on the number and combination of anchors, many different forms of anchor functions can be constructed. For example, for the sentence "My name is xxx, what is my name?", the anchor is "name is", the key token is "xxx", and the anchor function is the mapping from this sentence to the answer "xxx". Anchor functions help researchers understand the behavior of language models when handling specific patterns.

Language-like tasks: Language-like tasks are simplified tasks designed around anchor functions, used to simulate certain patterns that frequently appear in natural language processing. Through these tasks, researchers can analyze a model's performance at different levels of difficulty

and examine its generalization ability and internal mechanisms. For example, the task "$3x$ to x" can simulate using the anchor "3" to locate the number that follows it; such tasks help researchers better understand and explain the model's behavior.

Research motivation: By studying anchor functions and language-like tasks, researchers hope to address the following issues:

- Explore the behavioral properties of models in simplified experimental settings and reveal their inner working mechanisms.
- Design reasonable experiments and dataset partitioning schemes to comprehensively evaluate a model's generalization ability.
- Provide better understanding and interpretation of the reasoning process of language models, helping to improve and optimize their performance in real-world applications.

12.1 Challenges in studying Transformer-based language models

In the course of deeply studying and understanding Transformer-based large language models, we face many challenges. These challenges greatly increase the difficulty of Transformer research, especially for academic teams with limited resources. We elaborate on these challenges below.

12.1.1 Unknown tasks

In training large language models, we typically adopt the next token prediction approach, namely using a given sequence of tokens to predict the next token (see Fig. 12.1). This training objective seems unrelated to any specific language task, yet in practice language models trained in this way achieve excellent performance on a wide variety of language tasks. This phenomenon prompts the question: why can next token prediction, which looks so simple, endow language models with such powerful linguistic abilities?

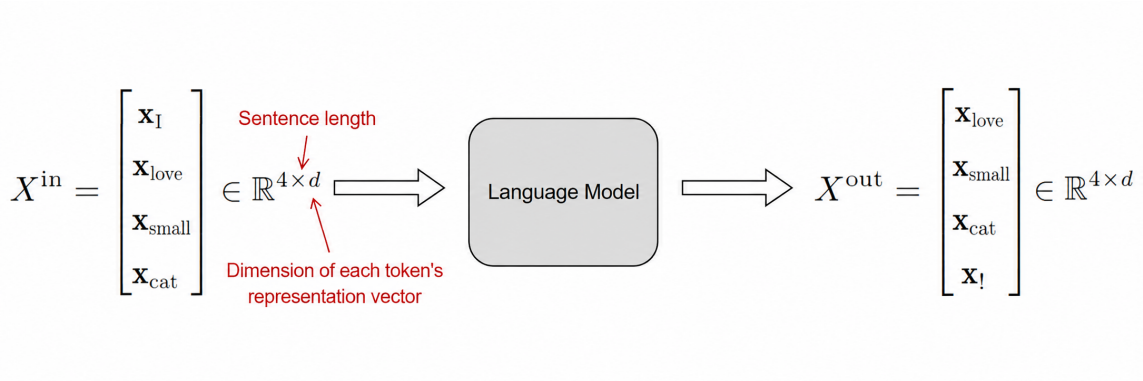


Figure 12.1: Illustration of the next token prediction paradigm.

The traditional machine learning paradigm is usually task-specific: for each task, one designs a dedicated model architecture and training objective. For example, to perform sentiment analysis, one might design a dedicated classification model. However, faced with a complex cognitive system like language, it is hard to enumerate all possible tasks and devise an optimal solution for each. Instead, next token prediction provides a general way of modeling language, allowing the model to learn the inherent regularities and representations of language autonomously from massive corpora. This training method does not depend on any specific task; rather, it attempts to capture the statistical properties and structural patterns of language itself.

However, this general training approach also brings new challenges: since the training objective is not directly tied to a specific task, it is difficult to analyze the training process in order to explain the model's performance on downstream tasks. What exactly has the model learned during training? Which linguistic knowledge and abilities are acquired first? These questions are hard to answer directly from the next token prediction training process. Moreover, the lack of task-specific evaluation metrics during training makes it difficult to directly optimize and improve the model's performance on specific tasks.

Next token prediction, as a general way of training language models, allows the model to acquire the inherent regularities of language from massive corpora, yielding strong abilities of language understanding and generation. At the same time, this non-task-driven training method poses new challenges for analyzing and optimizing the model. How to better understand and guide the learning process of language models is an important topic we cannot avoid when exploring their mysteries.

12.1.2 High computational and memory demands

Training a high-quality language model usually requires massive amounts of data and enormous computational resources, which is a huge challenge for academic research teams.

- The GPT-3 model uses 175 billion parameters and was trained on 499 billion tokens Brown et al. (2020). Training such a huge model requires expensive hardware and long training times, which is unaffordable for academic labs with limited funding.

12.1.3 Difficulty of interpreting the reasoning mechanism

Understanding how language models accomplish various tasks and explaining their decision process is a major challenge in Transformer research.

- A Transformer model typically contains hundreds of millions of parameters, and its inference process involves complex nonlinear transformations and high-dimensional vector computations. Even if the model outputs the correct result based on the input, it is hard to explain precisely how the model reasons its way to this result step by step.
- Some existing work tries to understand how Transformers work by visualizing attention. However, attention scores cannot fully reflect the model's reasoning process. We still need more fine-grained analytical tools and methods.

12.2 Characteristics of language tasks

Language is humanity's most important communication tool, and its unique structure and features provide rich clues for studying the mysteries of language. Most language tasks possess certain clear

logical structures, for example:

- In natural language, there are typically many logical inference relations. For example, "All teachers at a certain school are professors; Xiao Ming is a teacher at this school; therefore Xiao Ming is a professor." This is in fact a classical syllogistic reasoning pattern of the form "all A are B, C is A, therefore C is B". Beyond syllogisms, natural language also features higher-order logical inference chains whose reasoning involves multiple steps, requiring the conclusions obtained earlier to be applied step by step, forming a complete logical chain.
- Natural language also contains a large number of "anchor–key" pairs. For example, if you ask a large language model "My name is xxx, what is my name?", then no matter what your name is, it will output it correctly. In this task, "name is" is an anchor, telling the model to pay attention to the information that immediately follows. "xxx" is the key token corresponding to the anchor; generally, regardless of what token is filled in, it refers to the "name" attribute of "me" in the sentence. This anchor–key correspondence constitutes a basic unit of linguistic organization and provides an important basis for understanding language structure.

Such language tasks with clearly defined structures provide a good entry point for studying Transformers. We can understand a Transformer's learning and inference mechanisms by analyzing whether its inference modes are consistent with the inherent logical structure of these tasks, revealing its inner workings and biases. Such insights not only help us improve language models but can also provide useful inspirations and references for applying Transformers to other domains. Although language tasks possess the favorable analytical features mentioned above, due to their inherent complexity, studying Transformers directly on real corpora still faces enormous challenges. Therefore, we need to take a different route and explore the behavioral properties of Transformers in simplified settings.

12.3 Research approach

In scientific research, when we try to understand a complex phenomenon, a common strategy is to first experiment and verify in simple, controlled settings, and then gradually extend to more complex, realistic scenarios. This approach has been widely used in biology, physics, and other fields, and has achieved great success.

A well-known example is Hodgkin and Huxley's use of giant squid (Loligo) axons in their study of neuron electrophysiology. They chose the giant squid because its axon can be up to 1 mm in diameter, far larger than the axons of most animals, which made it possible to insert electrodes and measure changes in membrane potential. By experimenting on this special model organism, Hodgkin and Huxley successfully formulated a mathematical model of neuronal membrane potential changes (i.e., the HH model), making a pioneering contribution to our understanding of how the nervous system works. Their work also won the Nobel Prize in Physiology or Medicine.

Another example is using mice for diabetes research. Mice have become an ideal model for diabetes research for several reasons: first, mice share a high degree of physiological and genetic similarity with humans, which makes findings in mice more likely to translate to humans; second, mice have short life cycles, allowing researchers to observe the onset and progression of disease within a relatively short time, accelerating research; third, mice are easier to breed and manage, greatly reducing research costs; finally, mouse genes can be easily modified by genetic engineering techniques, enabling the creation of specific gene-knockout or knock-in models for studying the role

of particular genes in the development of diabetes. These advantages make mice an indispensable model organism in diabetes research.

Our earlier studies on the frequency principle, the condensation phenomenon, and the like, which were mentioned in previous chapters, also adopted this idea of reducing complexity. These works first discovered and summarized experimental phenomena on some artificially constructed simple functions, and then extended them to real datasets. By analyzing system behavior in a restricted setting, we can more accurately grasp the essential laws of phenomena.

The above cases suggest that, in studying language models, we can adopt a similar approach. Rather than directly analyzing model behavior on large-scale, complex datasets and tasks, it is better to first design some simple, controlled tasks and observe the model's properties and patterns through experiments. However, the construction of simple functions in previous chapters mainly targeted regression problems (fitting continuous functions) and cannot be directly applied to discrete language modeling tasks. To study language models, we need a new approach and design new simplified functions, in order to observe Transformers' sensitivity and biases toward different linguistic features. By examining model behavior in controlled experimental environments, we hope to discover general laws underlying language processing, providing entry points for theoretical breakthroughs. This research paradigm—starting from phenomena and using simple functions to explore unknown territory—is exactly the phenomenon-driven theoretical research methodology we advocate.

12.4 Anchor functions and language-like tasks

As we have introduced earlier, language data has many characteristics; here we focus on the "anchor–key" pair feature. Based on this property, we propose a class of language-like task patterns called *anchor functions*. Anchor functions are a class of specially designed functions used to simulate certain patterns that frequently appear in natural language processing.

We use a simple example to introduce the concepts of anchor functions and language-like tasks. For the task of asking for a name mentioned earlier, we can design a language-like task called "3x to x": given a fixed-length sequence consisting of random numbers from 20∼100 together with a single 3 (3 appears exactly once), e.g., (43, 33, 23, 3, 20, 89, 44, 24, 56), we set its label to be the number after 3, which in this example is 20. In this task, the anchor is 3, the key token is the number after 3, and the anchor function is the mapping from this sequence to the number after 3. This language-like task corresponds well to the name-asking task, but in comparison it is more concise, deterministic, and independent, and can be trained and studied on small Transformer networks.

In the following parts, we will describe in detail the definitions of anchor functions and language-like tasks, and the advantages of using anchor functions to study language models.

12.4.1 Anchor functions

An anchor function refers to a mapping from a language sequence to its label, whose functional expression is determined by the anchors appearing in the sentence. Depending on the number and combination of anchors, we can construct many different forms of anchor functions. Fig. 12.2 intuitively shows examples of single-anchor with a single key token, single-anchor with multiple key tokens, and multi-anchor with a single key token. By now, the reader should have an initial understanding of anchor functions; below, we give a strict mathematical formulation.

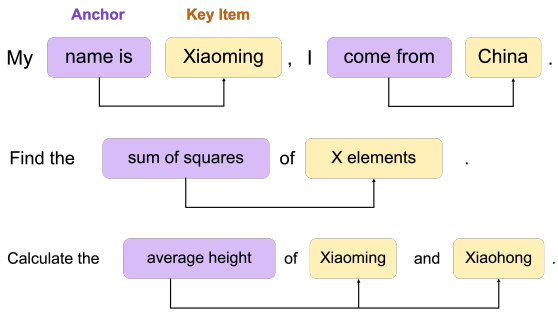


Figure 12.2: Examples of anchor function tasks.

Single-anchor functions

Single-anchor functions are the simplest class of anchor functions, corresponding to anchors with independent operation rules. For example, in the "$3x$ to x" task mentioned earlier, the anchor is "3", and the corresponding single-anchor function is:

$$f_3(x_1, \ldots, x_n) = x_{i+1}, \quad \text{where} \quad x_i = 3. \tag{12.1}$$

The single-anchor function f_a we define is a mapping from $\mathbb{R}^{n\times d}$ to $\mathbb{R}^d$, where a is the anchor, n is the number of tokens in the sentence (i.e., the sentence length), and d is the dimension of each token. In deep learning, each token is typically represented by a one-hot vector, i.e., $\boldsymbol{x} \in \{0,1\}^d$, so d also denotes the vocabulary size. A sentence of length n is represented as $X = (\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n) \in \mathbb{R}^{n\times d}$. In each sentence X, among $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_{n-1}$ there is exactly one element equal to the specified prompt anchor $\boldsymbol{a} \in \mathbb{R}^d$. Assuming $\boldsymbol{x}_i = \boldsymbol{a}$, the single-anchor function then satisfies $f_a(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n) = \boldsymbol{x}_{i+1}$. More strictly, a single-anchor function can be written in the following form:

$$f_a(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n) = \sum_{i\in\Omega} \boldsymbol{x}_{i+1}\delta(\boldsymbol{x}_i, \boldsymbol{a}), \tag{12.2}$$

where Ω is the set of all possible positions at which the anchor may appear, and $\delta(\boldsymbol{x}_i, \boldsymbol{a})$ is the Kronecker δ function, equal to 1 when $\boldsymbol{x}_i = \boldsymbol{a}$ and 0 otherwise.

More generally, we can also apply further operations to the output of the anchor function, e.g., "$3x$ to x+1", meaning output the number after "3" plus 1. The position the anchor attends to is also variable: the anchor can attend to the two tokens that follow, the one that precedes, or even more positions. Hence we can define a more general single-anchor function:

$$f_a(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n) = \sum_{i\in\Omega} g_a(X_{I_{\boldsymbol{a},i}})\delta(\boldsymbol{x}_i, \boldsymbol{a}), \tag{12.3}$$

where g_a denotes the further operation applied to the identified tokens, and $I_{\boldsymbol{a},i}$ denotes the index set of tokens attended to, jointly determined by the anchor and its position. We call these attended tokens (i.e., $X_{I_{\boldsymbol{a},i}}$) the **key tokens of anchor** a.

Remark 19. *Usually the operation of an anchor is insensitive to its position. For instance, in the name-asking context, the answer is always the name that follows the anchor "name is"; that is, the anchor's operation is translation-invariant to a certain extent. When designing anchor functions,*

we also focus on translation-invariant anchor functions. To reflect translation invariance, $I_{\boldsymbol{a},i}$ can be further rewritten as $I_{\boldsymbol{a}} + i$.

Multi-anchor functions

Building on single-anchor functions, we can also construct multi-anchor functions $f_A(X) : \mathbb{R}^{n\times d} \to \mathbb{R}^d$ in which several anchors work together. Here $A = \{\boldsymbol{a}_1, \boldsymbol{a}_2, \cdots, \boldsymbol{a}_t\}$ is the set of anchors. Suppose in the input sequence X these anchors appear at the set of positions J; then the multi-anchor function can be written as

$$f_A(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n) = \sum_{J\in\Omega} g_A(X_{I_{A,J}})\delta(X_J, A), \tag{12.4}$$

where Ω is the family of all possible position sets that the anchor set may occupy.

Remark 20. *A simple multi-anchor function is a two-anchor composite function: each anchor has its own operation rule; denoting the operation rule of anchor $\boldsymbol{a}_i$ by $g_{\boldsymbol{a}_i}$, the two-anchor function is the composition of these simple anchor functions:*

$$g_A(X_{I_{A,J}}) = g_a(g_b(X_{I_{A,J}})). \tag{12.5}$$

In what follows we will focus on examples of such two-anchor composite functions.

Compared with anchors and their effects in real language data, anchor functions have clear definitions and outputs, and we can conveniently control the distribution and complexity of the input data. This helps us analyze the model's performance at different difficulty levels and examine the limits of its generalization. At the same time, since the task objective is well-defined, it is also easier to interpret the model's behavior.

12.4.2 Language-like tasks

Based on different anchor functions, we can design a variety of language-like tasks to simulate various real language scenarios. Fig. 12.3 lists several language-like tasks: the first row contains tasks for single-anchor functions; the second row contains tasks for multi-anchor functions; and the third row contains sequence-understanding tasks and anchor-function tasks with non-deterministic mappings. These tasks all have concrete contextual meaning. For example, the identity-learning task is the "$3x$ to x" task mentioned earlier, corresponding to the scenario of finding the answer from the preceding context; the composite task is a multi-anchor task constructed from two anchors, corresponding to complex thinking scenarios; the goal of the reading-comprehension task is to find content that previously appeared in the sentence based on a prompt word; the forward–backward recitation task investigates the difference between a large model's ability to find subsequent content from preceding text and to find preceding content from subsequent text… The definitions of these language-like tasks and their correspondence to real contexts will be elaborated upon when we discuss the experimental results later.

Compared with real language tasks, these language-like tasks have many advantages. First, their rules are simple and clear, easy to understand and implement. Researchers can quickly design and construct tasks that meet specific needs, without having to deal with the noise and uncertainty in real language data. Second, the data generation process of language-like tasks is fully controlled, allowing precise control over the split between training and test sets, and therefore

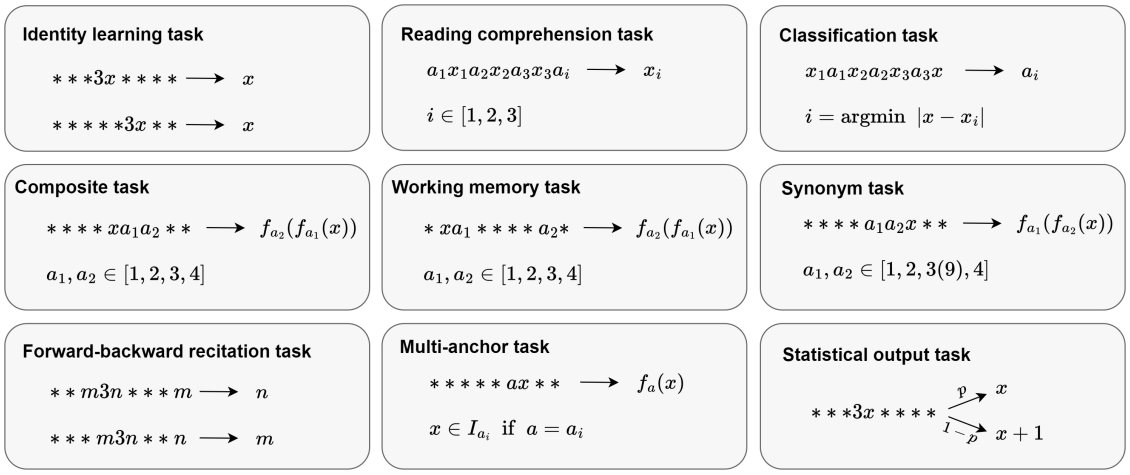


Figure 12.3: Examples of anchor function tasks.

a more accurate assessment of the model's generalization ability. In addition, since the tasks are simple, the training data required is relatively small and the model size is also relatively small; for phenomenon-driven studies, the required computational resources are modest, allowing researchers to iterate and optimize models quickly.

More importantly, through language-like tasks we can gain a deep understanding of the internal mechanisms of large language models when they handle real language problems. Although language-like tasks are relatively simplified, they still capture some key features of real language processing, such as context dependence, semantic composition, and reasoning. We can interpret how the model arrives at its output step by step by visualizing neuron activations, attention weights, and the like. This is of great significance for understanding the working mechanisms of deep neural networks such as Transformers. This research paradigm, going from simple to complex and from controlled to realistic, helps us gradually uncover the essence of large language models.

12.5 Data partitioning and model generalization

To fully exploit the advantages of anchor functions and language-like tasks, sensible experimental settings and dataset partitioning strategies are crucial. In the following part, we discuss in detail various methods of dataset partitioning and a deeper understanding of model generalization.

12.5.1 Splitting training and test sets

In this section, we propose several data partitioning methods suitable for language-like tasks, with the following two goals:

- Cleanly partitioning training and test sets.
- Using this partition to further verify whether the Transformer architecture learns the essence of the language task at the logical-structural level.

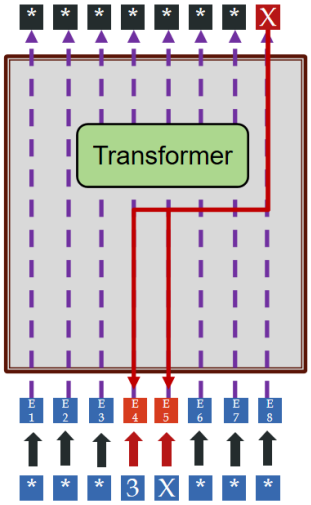


Figure 12.4: Forms in which parameters are updated when a sequence is fed into a Transformer.

Taking the "$3x$ to x" task as an example, a natural idea is to partition by key tokens: the key tokens of sequences in the training set lie in one interval, and those in the test set lie in another interval, with other tokens in the sentence chosen at random. However, this partition is actually problematic. Fig. 12.4 gives an intuitive understanding: on the one hand, for a "$3x$ to x" sequence, only the anchor "3" and the key token "x" are associated with the label "x", so during back-propagation, the embedding vectors of "3" and the key token "x" are updated once as anchor and key token, while "x" at other positions is updated once as noise. If some "x_0" has never appeared as a key token, then its embedding vector is only ever updated as noise; on the other hand, the embedding matrix responsible for encoding and the projection matrix responsible for projecting to the output have no inherent matching relationship: in the untrained state, "x" passing through embedding and projection does not output "x". These two reasons together cause the network to be unable to generalize on sequences with this "x_0" as the key token. Therefore, we need to partition training and test sets sensibly: ensure that the two sets are completely disjoint, while also ensuring that every possible "$3x$" combination has appeared in the training set, and similarly for other tasks.

In this section, we propose three methods for splitting training and test sets, as shown in Fig. 12.5. Each of the three methods has its own characteristics and can help us better assess the model's generalization ability. We describe each partitioning method in detail below.

Modulo-remainder data partitioning

To ensure that every token has appeared as a key token while completely separating training and test sets, we design a modulo-remainder data partitioning method. Consider a task with input sequences of length n. The range of values of all tokens x is I. Define $\Gamma_i = \{kn + i | k \in \mathbb{Z}\}$ to be the set of all integers with remainder i modulo n. The modulo-remainder partition is as follows:

- For an input sequence in the training set, if the token at the i-th position is the key token, then its value x is taken from $x \in I \backslash \Gamma_i$, i.e., x does not have remainder i modulo n.

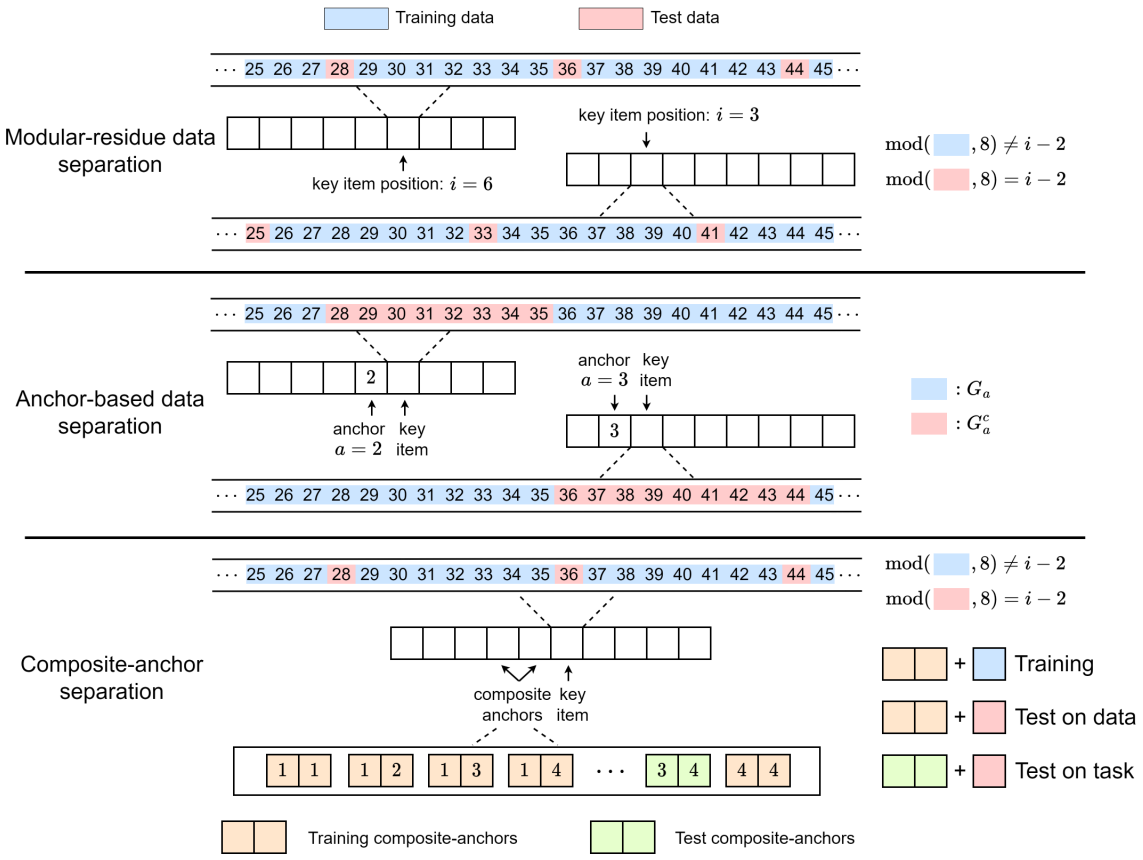


Figure 12.5: Schematic illustration of the three ways to split training and test sets.

- For an input sequence in the test set, if the token at the i-th position is the key token, then its value x is taken from $x \in I \cap \Gamma_i$, i.e., x has remainder i modulo n.

This partition completely separates training and test sets; in subsequent training on language-like tasks, the test set defined by this partition serves as a most basic metric for evaluating a Transformer's generalization. However, the criteria for evaluating model generalization are not unique. To prevent the model from cheating its way to memorizing all "$3x$" combinations (which cannot happen in real large models), we propose below two more sophisticated partitioning methods—anchor-based data partitioning and composite anchor partitioning—for a more comprehensive evaluation of Transformer models.

Anchor-based data partitioning

Suppose a language-like task dataset consists of sentences constructed from multiple single-anchor functions, with exactly one anchor appearing in each sentence. Denote the anchor set by $A = \{a_j\}_{j=1}^{J}$. We can then consider the following anchor-based partitioning method.

Denote the set of all tokens by I. In the training set, the key tokens of anchor a_j belong to a set $G_{a_j} \subsetneq I$, with the sets satisfying $\cup G_{a_j} = I$. In test sequences, the key tokens of a_j belong to $G_{a_j}^c = I \backslash G_{a_j}$.

The core idea here is that, once "x" has been trained as a key token for several anchors, the model can accurately estimate its behavior as a key token for the other anchors.

Composite anchor partitioning

Suppose a language-like task dataset consists of sentences constructed from multiple multi-anchor functions. For example, taking ["1", "2", "3", "4"] as single anchors, their pairwise combinations form 16 composite anchors; the task is "abx to $g_a(g_b(x))$" (called the composite task). Then we can use the language-like tasks corresponding to 15 of these composite anchors for training and use the remaining 1 composite-anchor task for testing.

This partition can be used to examine the model's generalization ability to new anchor combinations, reflecting its compositional reasoning ability. By cleverly designing anchor combinations, we can build test sets of varying difficulty and comprehensively evaluate the model's performance.

The three dataset partitioning methods introduced in this section provide us with tools to study anchor functions and evaluate model generalization from multiple angles. In subsequent experiments, we will flexibly use these methods to deeply investigate the behavioral properties and inner mechanisms of Transformer models.

12.5.2 Data generalization and task generalization

The way the dataset is partitioned naturally gives rise to two notions of generalization: data generalization and task generalization. The former describes the model's ability to generalize to unseen data within seen tasks, while the latter concerns the model's ability to generalize to unseen tasks.

Data generalization and task generalization are two important dimensions for evaluating model performance, reflecting the model's ability to adapt and extrapolate at different levels. We use the composite task "abx to $g_a(g_b(x))$" as an example to discuss the characteristics of these two kinds of generalization and how they are realized. In this task, both anchors "a" and "b" are taken from the set ["1", "2", "3", "4"], and their pairwise combinations form 16 anchor combinations. In this example, we can examine generalization with respect to the key token "x", as well as the model's generalization on the remaining one anchor-combination task after training on 15 of them.

Data generalization. Data generalization mainly refers to the model's performance on test sets partitioned by the key token "x". The partition can use the modulo-remainder data partitioning or anchor-based data partitioning mentioned above. For this test set, the anchor function (i.e., the task) has already appeared in the training set. Data generalization investigates whether, when faced with new data samples, the model can fully exploit the knowledge and rules learned during training to make correct predictions or outputs on unseen samples. For example, after the model has learned addition, it should be able to output the correct sum for any pair of summands. This requires the model not just to memorize the training-set content, but to grasp the general rules behind the task and form robust representations and computational abilities.

Task generalization. Task generalization mainly refers to the model's performance on test sets partitioned by anchor pairs "ab". One way to construct the partition is to select a set of anchor combinations "ab" as the test set and use other anchor combinations as the training set—this is the composite-anchor partition mentioned earlier. Although some anchor combinations in the test data

are new (do not appear in the training set), the model can use existing composite anchors in the training set to infer unseen composite-anchor functions. Task generalization investigates whether, when faced with brand-new tasks, the model can leverage its learned knowledge and skills—via composition, analogy, and so on—to quickly adapt and solve new problems. For example, after the model has trained on many sentiment-analysis tasks for food, travel, music, and so on, it should also achieve high accuracy on the sentiment-analysis task for movie reviews. This requires the model to possess some level of abstraction and reasoning ability, to establish connections between different tasks, extract common features, and flexibly apply them to new scenarios.

Introducing the concepts of data generalization and task generalization provides an important perspective for comprehensively evaluating model performance. By designing reasonable dataset partitioning schemes and constructing test sets with appropriate difficulty and broad coverage, we can objectively measure model performance along these two dimensions, identify its strengths and limitations, and provide a basis for further improvement and optimization.

In the following sections, we will use a series of experiments based on anchor-function language-like tasks to investigate in depth the characteristics of Transformer models in terms of data generalization and task generalization, reveal their inner working mechanisms, and discuss possible directions for improvement.

12.6 Experimental results and discussion

In this section, we use Transformer models to learn various anchor-function language-like tasks shown in Fig. 12.3 and explore the internal mechanisms and output preferences under different tasks. Unless otherwise stated, we use a 4-layer decoder-only Transformer to learn the tasks. Each decoder layer contains 4 attention heads. For detailed settings, please refer to 12.8.3. Below we focus on the real conversational scenarios these language-like tasks simulate, and on the characteristics of the Transformer model when learning these data.

12.6.1 Identity-learning task

The identity-learning task is the "$3x$ to x" task mentioned earlier. It simulates the task of finding the keyword that represents a name in a sentence. For example, for the input "He is from China, his name is Mike, he likes reading", the anchor in this sentence is "name is" and "Mike" is the key value (or key token) corresponding to the anchor.

The anchor function for the identity-learning task is defined in Eq. (12.2), where the prompt anchor appears exactly once in the input sequence. The function's output is the number immediately following the specific anchor. In our experiments, we use "3" as the anchor.

To highlight the superiority of the Transformer architecture, we compare the generalization ability of a fully connected neural network (FNN), an LSTM, and a Transformer on the identity-learning task at different data sizes. The parameter counts of these three models are 416 million, 5.3 million, and 5.6 million, respectively. Detailed settings of the three models can be found in the appendix. For each experimental setting, we run experiments with 10 random seeds. Each experiment trains for 4000 epochs. As shown in Fig. 12.6, the scatter points are the averages over 10 runs, and the shaded regions are the maxima and minima over 10 independent runs. We can see that the Transformer exhibits excellent generalization with only 600 training data, while the LSTM requires about 2200 training data to reach a similar level of generalization. On the other hand, the FNN struggles to generalize with limited data.

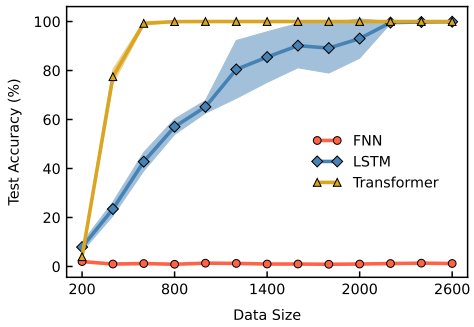


Figure 12.6: Performance comparison of FNN, LSTM, and Transformer.

The identity-learning task is a basic research task for anchor functions. In Section 12.7 below, we will further study in detail the mechanism by which the Transformer model achieves good generalization on the identity-learning task.

12.6.2 Reading-comprehension task

In everyday reading comprehension, when faced with a question, people usually first identify the key information in the question—i.e., the anchor and the key token—and then search the given text for expressions similar to the anchor. The answer to the question can usually be found in the content near the expression in the main text that is similar to the anchor. We set up the following task to simulate this reading-comprehension task.

We define different anchor values $a_i \in \{1, 2, \cdots, 8\}$ and key tokens $x_i \in [20, 100]$. The input sequence follows the pattern $(a_1, x_1, a_2, x_2, a_3, x_3, a_4, x_4, a)$, where a_i are anchors and for $i \neq j$ we require $a_i \neq a_j$. The target output of the sequence is x_k, where the index k satisfies $a_k = a$. In this task, the train/test split is done via the "modulo-remainder data partitioning" method. After training for 1000 epochs on a total of 1000 training points, the model achieves 100% accuracy on the test set.

12.6.3 Classification task

The classification task is a variant of the reading-comprehension task that focuses on content that is similar—rather than identical—to the anchor. In this task, we define different anchor values $a_i \in \{1, 2, \cdots, 8\}$ and key tokens $x_i \in [20, 100]$. The input sequence follows the pattern $(x_1, a_1, x_2, a_2, x_3, a_3, x_4, a_4, x)$, where a_i are anchors and for $i \neq j$ we require $a_i \neq a_j$. The output label is $a = a_k$, where $k = \text{argmin}_{i \in \{1,2,3,4\}} |x_i - x|$. In this task, the train/test split is done via the "modulo-remainder data partitioning" method. We use a total of 30000 data points for training with batch size 50 and train for 4000 epochs. At this point the model achieves 100% accuracy on the test set.

Compared with the reading-comprehension task, the classification task has higher complexity, mainly because the model needs to encode the ordinal relations between data in order to characterize the distance between any two pieces of data. As shown in Fig. 12.7, we use t-SNE Van der Maaten and Hinton (2008) to visualize the embedding-space vectors of different tokens in the trained model.

It is clear that the embedding vectors capture the ordinal relations between tokens, which is crucial in the classification task.

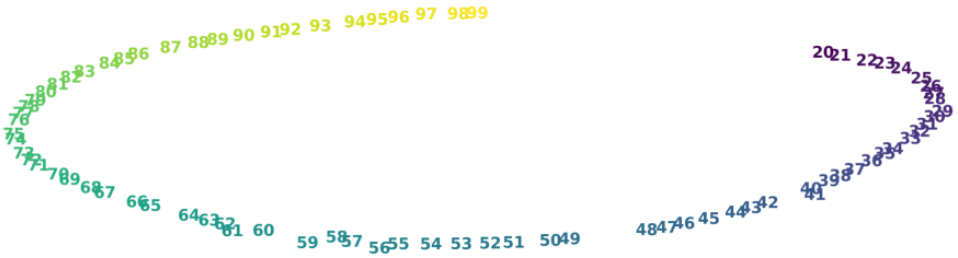


Figure 12.7: t-SNE distribution of the trained token embeddings of the numbers 20 to 99.

12.6.4 Composite task

The composite task is a natural extension of the single-anchor task. In each input sequence, there are always two anchors indicating a composite function. A basic question arises: in a composite task, can the model learn the mapping of each single anchor making up the composite function? In this task, we use modulo-remainder data partitioning together with composite-anchor partitioning to split the data. This allows us to study the Transformer's ability to generalize over both data and tasks.

First, we define four anchors, each representing a specific mapping. We then pair these anchors to create 16 different composite-anchor functions. We choose 15 of these pairs to build the training set. The model's generalization is tested on the test set generated by all 16 composite-anchor functions. Notably, the model's generalization on the 15 composite mappings used for training reflects its generalization over data, while its generalization on composite mappings absent from the training set reflects its generalization over tasks.

As shown in the left of Fig. 12.8, we denote the function of single anchor a by f_a and combine each pair of single anchors to obtain the two-anchor composite function f_{a_1,a_2} for the anchor pair $\{a_1, a_2\}$. The grey anchor pairs in the "composite output" section do not appear in the training set. On the right of Fig. 12.8, we illustrate the behavior of a trained 10-layer model with three input

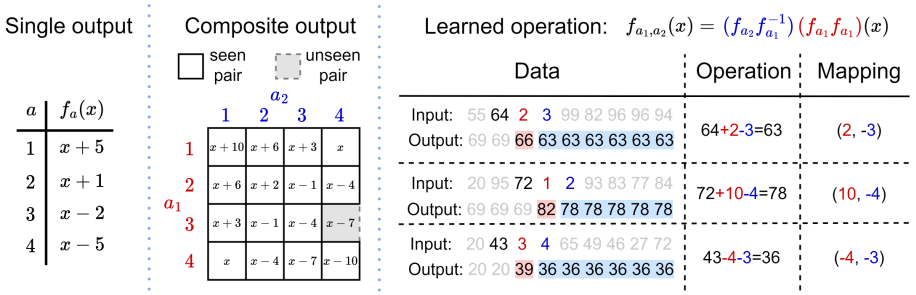


Figure 12.8: Composite-task setup and the regularities in the model's outputs.

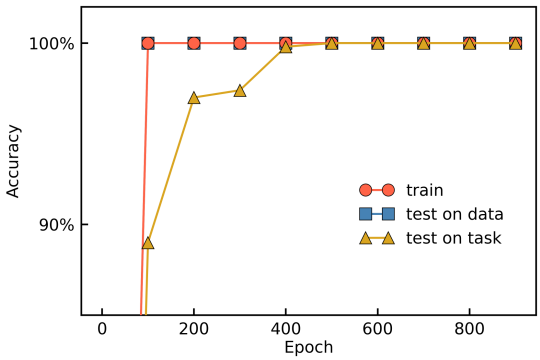


Figure 12.9: Training and test accuracy on the composite task.

examples. We find that the network learns a specific composite-mapping structure. For the first anchor a_1, the network's output at that anchor's position is $(f_{a_1} f_{a_1})(x)$, where x is the key token. For the second anchor, the network's output at that anchor's position is $(f_{a_2} f_{a_1}^{-1})(f_{a_1} f_{a_1})(x)$, which cancels the extra f_{a_1} operation at the position of anchor a_1.

We find that the model size can significantly affect the post-training output pattern. For the 10-layer model, as shown on the right of Fig. 12.8, after the second anchor is input, the network outputs the correct answer and continues to output the correct answer for the subsequent input tokens. For shallower models, however, the above phenomenon does not appear stably, hinting at a link between model size and stability.

In addition, we observe that the task test accuracy after training is generally lower than the data test accuracy (which always reaches 100%). This shows that task generalization is more difficult. To verify this, Fig. 12.9 shows one learning process. In this training process, the accuracy on the training data and on the test data reaches 100% in just 100 epochs, while the generalization over tasks reaches 100% only after 400 epochs. The model's convergence on data generalization is much faster than on task generalization.

12.6.5 Working-memory task

The composite task shows the model's ability to learn the operation of each single anchor from a composite anchor. The second anchor processes the output of the first anchor rather than directly processing the key token. If there are some irrelevant tokens between the two anchors, this sequential processing requires the model to have a working-memory ability. Specifically, we set the first anchor $a_1 \in \{1, 2\}$ and the second anchor $a_2 \in \{3, 4\}$. As shown in Fig. 12.10, the output of the first anchor is an intermediate result obtained by the first anchor operating on the key token. The model memorizes this intermediate result by retaining this value in the outputs at the irrelevant tokens. Once the model sees the second anchor, it combines the intermediate result with the second anchor to output the final correct answer. Notably, since this problem is multi-solution (adding a constant k to a_1 and subtracting the constant k from a_2 leaves the composite result unchanged), the intermediate result learned by the network may differ from the single-anchor operation we specified.

The working-memory task reveals the Transformer model's ability to handle complex tasks that require temporarily storing and maintaining information. The model can not only learn the operation of single anchors but also retain intermediate results as needed until producing the final

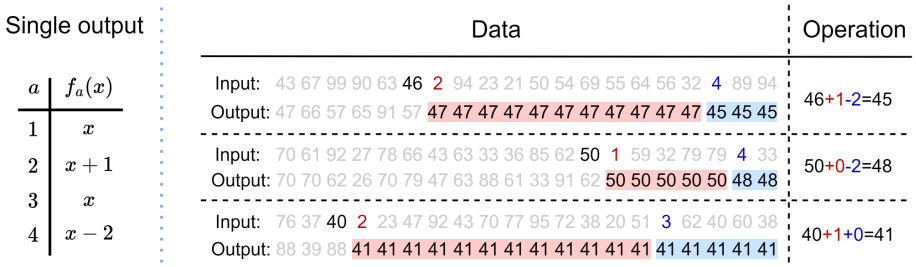


Figure 12.10: Working-memory task setup and the regularities in the model's outputs.

output. This ability is significant for understanding and simulating human cognitive processes in reasoning, problem solving, and similar tasks.

12.6.6 Synonym task

The synonym task is an extension of the composite task, designed to study a language model's ability to model synonymy between two anchors. The composite anchors in the original dataset are drawn from: {(1,9), (2,9), (9,1), (9,2), (1,2), (1,4), (2,1), (4,1), (2,4), (4,2)}. The training data for each composite anchor are equal in number. However, as shown by the cross-marked dotted line in Fig. 12.11(a), even as we continually increase the amount of training data, the Transformer network still struggles to learn the mappings for composite anchors such as (4,9) and (9,4).

We introduce a synonym anchor "3" which performs the same function as anchor "9". To see whether the Transformer can model the function of anchors (4,9) and (9,4) via the composite anchors (4,3) and (3,4), we add data containing the synonym anchor "3" to the training set. Specifically, we add the composite anchors {(1,3), (3,1), (2,3), (3,2), (4,3), (3,4)} to the original dataset. With total training data of M, the number of training data for each composite anchor containing "9" (such as (1,9)) is fixed at 100, totaling 400, while the number of training data for the other composite anchors is $(M - 400)/12$. As shown by the diamond-marked dotted line in Fig. 12.11(a), once the total amount of data exceeds a certain value, the network can accurately predict the outputs of (4,9) and (9,4). In Fig. 12.11(b), we show the specific learning processes under two data sizes (3000 and 4000). It is clear that the learning of (4,9) and (9,4) is slightly later than that of (4,3) and (3,4). These results show that introducing synonyms significantly affects the Transformer's task generalization ability.

The synonym task demonstrates the potential of Transformer models for learning semantic relations between words. By introducing synonym anchors, the model can transfer learned knowledge to new combinations, achieving stronger generalization. This finding provides new insights for using language models to handle practical tasks such as synonym recognition and word-sense disambiguation.

12.6.7 Forward–backward recitation task

It is generally observed that reciting the next line of a poem (the forward task) is much easier than reciting the previous line (the backward task). Building on this experience, we tested LLMs and found that they also find the forward task easier than the backward one. For example, we tested a forward–backward recitation task containing 20 poems with GPT-4. On the forward task, GPT-4's

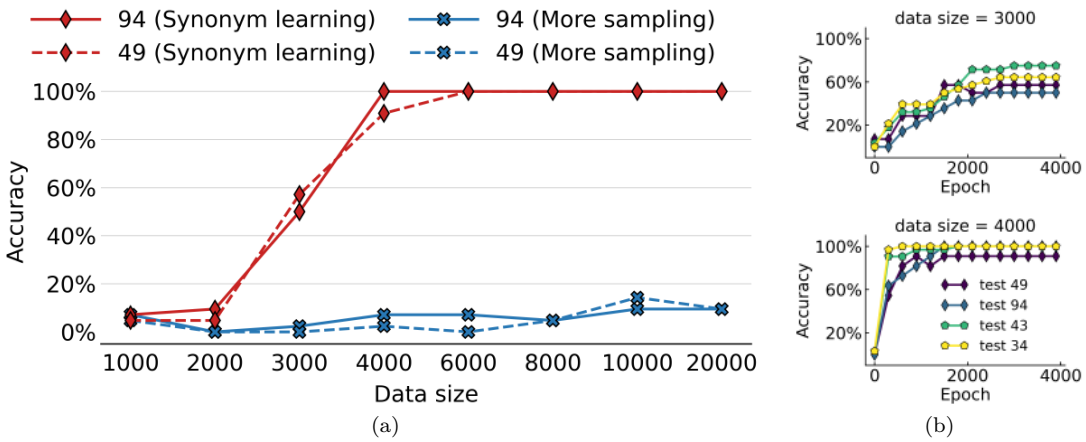


Figure 12.11: Comparing the effect of two data-augmentation methods on task generalization (a) and the training behavior of the model under different data sizes (b).

accuracy was 95%, whereas on backward recitation it was only 5%. This bears some similarity to the cognitive mechanism of human language understanding and memory. Motivated by this observation, we design an idealized forward–backward recitation task to study this asymmetry in LLMs. The input sequence follows the pattern "$(*,*,m,3,n,*,*,*,n)$" or "$(*,*,m,3,n,*,*,*,m)$", where "3" is the anchor, the other tokens are randomly chosen from $[20, 100]$, "$*$" denotes a token at that position that is irrelevant to the target value, and "m,3,n" can be at any position. The output of "$(*,*,m,3,n,*,*,*,m)$" is "n" (backward task), while the output of "$(*,*,m,3,n,*,*,*,n)$" is "m" (forward task). As shown in Fig. 12.12, where the first two columns are epoch=400 and the last two are epoch=4000, with the first and third columns being the forward task and the second and fourth being the backward task, the Transformer's generalization on the forward task is consistently better than on the backward task.

The forward–backward recitation task reveals an interesting bias of Transformer models in processing sequential information. Although the data structure is symmetric, due to the presence of the mask, the model's performance on the forward and backward tasks differs significantly. By further studying the differences in learning rate and generalization ability on these two kinds of data, we can better understand the inner workings of language models and explore similarities and differences with human language processing.

12.6.8 Statistical-output task

Statistical output is a common scenario in which the output of the sequence follows a probability distribution. For example, when surveying the dietary preferences of a group of people, suppose 80% like apples and 20% like oranges; the model may then tend to output only "apple" and ignore the wishes of the minority. To simulate this scenario, we design a task based on the identity-learning anchor function.

To simulate statistical-output scenarios, we design two ways of constructing the dataset:

- Dataset S is constructed in the same way as the identity-learning task, then duplicated five

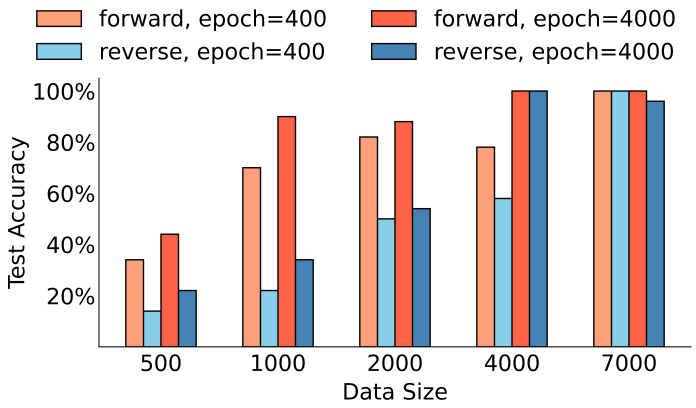


Figure 12.12: Accuracy of forward and backward prediction in the forward–backward recitation task as a function of training data size.

times, with the labels of the last copy modified to $x+1$. These five copies merge to form the first type of dataset.

- Dataset S is constructed in the same way as the identity-learning task, and 20% of the data have their labels randomly changed to $x+1$. The newly obtained dataset is called the second type of dataset.

As shown in Fig. 12.13(a), for the first type of data, the training accuracy can only reach 80% (red). This is because our way of generating the data makes some inputs identical but with different target values. On data with label x, the network can reach 100%, while on data with label $x+1$, the network's accuracy is 0%. As shown in Fig. 12.13(b), for the second type of dataset, since each data point has a unique label, the Transformer can fully learn the training set; on never-seen test data, the network has an 80% probability of outputting x and a 20% probability of outputting $x+1$.

We then check whether the network has learned the output probability distribution. For input x, we can separately record the probabilities of outputting x and $x+1$, i.e., the softmax outputs at the output layer. As shown in Fig. 12.13(c), for the first type of data, as epochs increase, the probability of outputting x approaches 80% and that of $x+1$ approaches 20%, both consistent with the experimental setup.

The statistical-output task demonstrates the ability of Transformer models to learn and represent probability distributions. By designing appropriate datasets, we can guide the model to learn statistical relations between inputs and outputs, rather than only deterministic mappings. This ability is important for practical tasks dealing with uncertainty and producing diverse outputs, such as dialogue generation and machine translation.

12.6.9 Multi-anchor task

Under modulo-remainder data partitioning, only the positions of the (anchor, key-token) pair (i.e., (a, x)) differ between training and test sets. One might argue that the network has only learned translation invariance and not the operation. To verify learning of the operation, we design the following multi-anchor task.

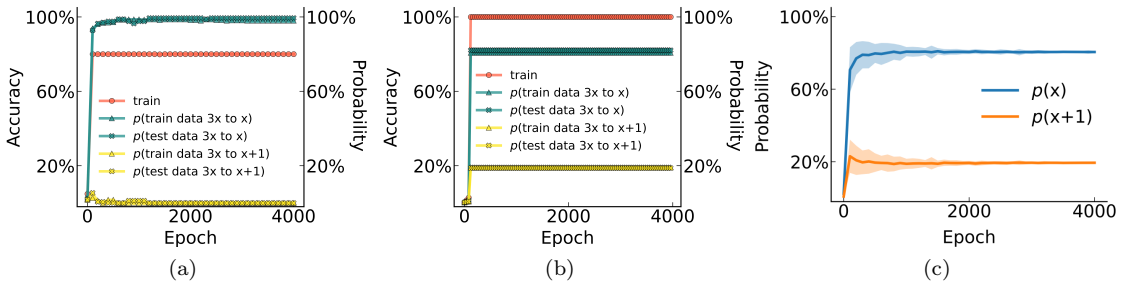


Figure 12.13: Prediction accuracy of the Transformer on the statistical-output task.

We use ten different anchors $a \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ and define the anchor function $f_a(x) = x + a$, where x is the key token of the anchor. For the test set, the key tokens of anchor a belong to $G_a^c = [12 + 8a, 12 + 8(a + 1) - 1]$; for the training set, the key tokens of anchor a belong to $G_a = [20, 100] \backslash G_a^c$, i.e., anchor-based data partitioning. Our focus is whether, after training, the Transformer's predictive ability generalizes to sentences with anchor–key-token combinations unseen in training.

We examine the Transformer's predictive ability on datasets built with different anchors. The test set is $G_a^c = [12 + 8a, 12 + 8(a + 1) - 1]$ and the training set is $G_a = [20, 100] \backslash G_a^c$. As shown in Fig. 12.14, in most cases the model exhibits good generalization on anchor-partitioned data, meaning the network has learned the operation rather than translation invariance. For boundary cases (anchors "1", "2", "10"), due to the dataset construction, fewer data points have the anchor at the boundary, which may explain why the accuracy at the boundary does not reach 100%.

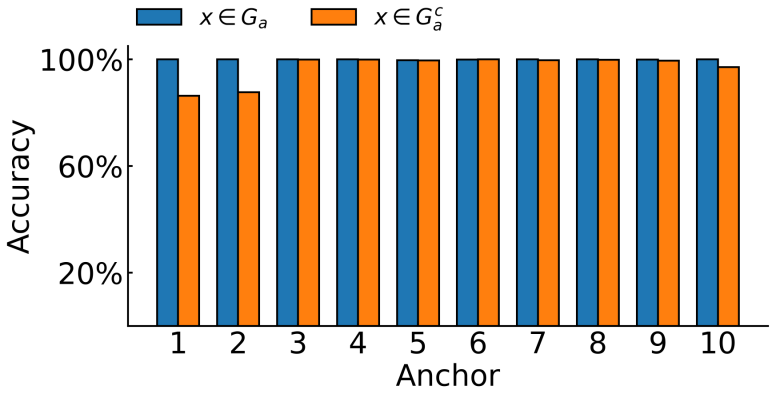


Figure 12.14: Accuracy after training on the multi-anchor task.

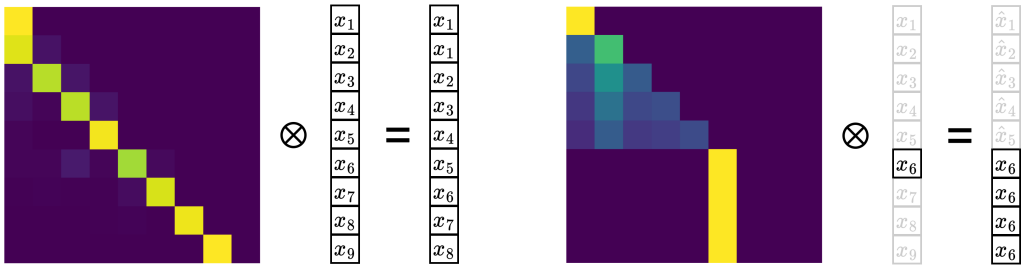
The multi-anchor task further verifies the property that Transformer models learn the operation itself rather than relying purely on positional information. Through carefully designed data partitioning strategies, we can investigate the model's generalization performance on brand-new anchor–key-token combinations, providing a more comprehensive evaluation of its learning. At the same time, this task reveals the impact of boundary cases on model learning, offering useful insights

for optimizing dataset construction and model training.

12.7 Mechanistic study of the identity-learning task*

Through the previous experiments, we have seen that Transformer models exhibit excellent performance on a variety of anchor-function tasks. However, the inner workings of the model still merit further exploration. In this section, we focus on analyzing the detailed mechanism of a two-layer Transformer on the identity-learning task, in order to obtain a deeper understanding. This section first briefly explains the mechanism of the identity-learning task in a two-layer model, identifying two key operations: shift and broadcast. We then find a highly simplified two-layer model that can accomplish the identity-learning task. Shift and broadcast operations also play a key role in this simplified model. Finally, we show that these two operations are widespread in large language models (LLMs).

12.7.1 A brief explanation of the two-layer model



(a) First layer, shift operation

(b) Second layer, broadcast operation

Figure 12.15: Attention maps of the first and second layers.

In the identity-learning task, the attention maps of the two-layer model primarily serve two purposes: shift and broadcast. As shown in Fig. 12.15(a), the attention map of the first layer performs a shift operation: the sequence (x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9) is shifted by the first-layer attention map to (x_1, x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8). Thanks to the residual connection, the shifted vectors are fused with the unshifted vectors, integrating information between the anchor and the key token. Notably, this is a one-position shift because the key token is designed to be the token immediately following the anchor. Once the information of the anchor and the key token has been fused, the subsequent fully connected structure generates the correct label based on this fused information.

The second layer's attention map performs a broadcast operation. As shown in Fig. 12.15(b), the sequence (x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9) is transformed into ($\hat{x_1}$, $\hat{x_2}$, $\hat{x_3}$, $\hat{x_4}$, $\hat{x_5}$, x_6, x_6, x_6, x_6), where $\hat{x_i}$ is a linear combination of $x_1, \ldots, x_i$. This broadcast operation transmits the correct label to the last position, i.e., the output position. The first-layer attention map is primarily based on positional encoding, while the second-layer attention map is primarily based on the anchor's position in the input sequence.

This example illustrates two important operations that the attention structure provides in language models: shift and broadcast. We also find that these two operations are common in large language models (LLMs) (see the experiments in later sections). Hence, anchor functions can serve as a benchmark for understanding Transformer-based language models.

12.7.2 Mechanism of the simplified two-layer model

In this section, we adopt a simplified two-layer model that removes structures including masking, layer normalization, residual connections, linear transformations, and fully connected networks, in order to study the mechanism of the identity-learning task. We first give the definition of the simplified model.

Let $X^{(1)} \in \mathbb{R}^{n \times d_m}$ denote the input sequence after word and positional embedding, i.e., $X^{(1)} = X^{\text{em}} + X^{\text{pos}}$. We choose d_m equal to the vocabulary size d in order to simplify the linear operators in the Transformer and remove the projection layer. The final output is computed as follows:

$$\text{Attn}^{(1)} = \text{softmax}\left(\frac{X^{(1)}W^{Q(1)}W^{K(1)T}X^{(1)T}}{\sqrt{d}}\right), \quad X^{(2)} = \text{Attn}^{(1)}X^{(1)}W^{V(1)},$$
$$\text{Attn}^{(2)} = \text{softmax}\left(\frac{X^{(2)}W^{Q(2)}W^{K(2)T}X^{(2)T}}{\sqrt{d}}\right), \quad X^{(\text{out})} = \text{Attn}^{(2)}X^{(2)}W^{V(2)}.$$

It can be written simply as

$$X^{(\text{out})} = \text{Attn}^{(2)}\text{Attn}^{(1)}X^{(1)}W^{V(1)}W^{V(2)}.$$

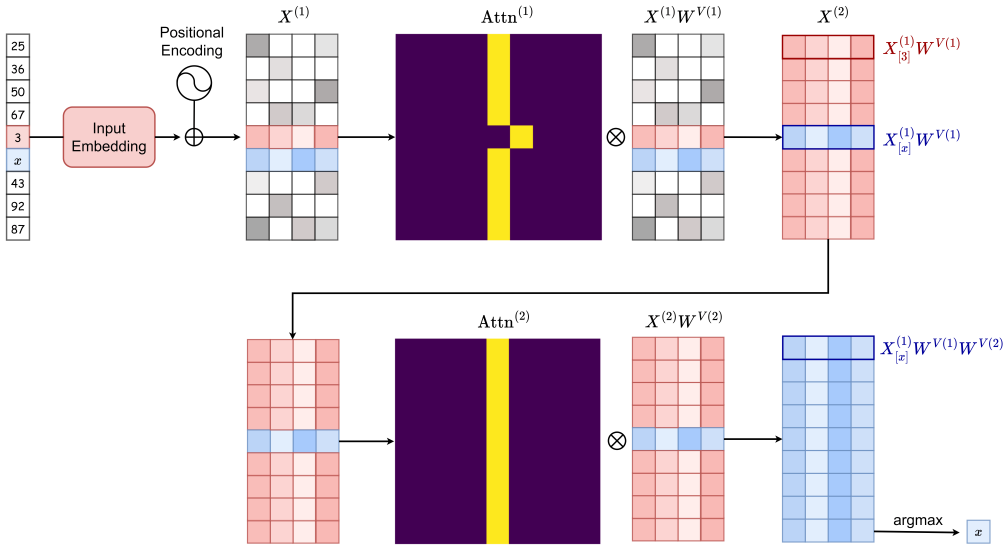


Figure 12.16: Mechanism of the simplified two-layer Transformer model on the identity-learning task.

It is worth noting that $\text{Attn}^{(l)}$ fuses information from $X^{(l)}$. Therefore, the formula above is not a simple linear transformation.

Fig. 12.16 illustrates the mechanistic interpretation of this model: the first layer's attention shifts the information of the key token "x" to the position of the anchor "3" and broadcasts the anchor's information to other positions. Specifically, the first layer's attention matrix turns the output vector into $X^{(1)}_{[3]}W^{V(1)}$ or $X^{(1)}_{[x]}W^{V(1)}$, where [3] and [x] denote the positions of "3" and "x" in the input sequence. Therefore, the row vectors of the matrix input to the second layer come in only two types: those containing the anchor's information and those containing the key token's information. This structure also facilitates the construction of the second-layer attention map $\text{Attn}^{(2)}$. The attention matrix $\text{Attn}^{(2)}$ performs a broadcast operation, i.e., broadcasting the vector containing the key token's information to all positions. Specifically, the second layer's attention matrix turns all output vectors into $X^{(1)}_{[x]}W^{V(1)}W^{V(2)}$. The matrix $W^{V(1)}W^{V(2)}$ plays the role of classification. The final prediction is obtained via the function $\text{argmax}(\cdot)$.

In the simplified model above, $\text{Attn}^{(1)}$ plays an important role in information transfer and can be viewed as a combination of shift attention and broadcast attention.

12.7.3 Shift and broadcast in Llama2-7B

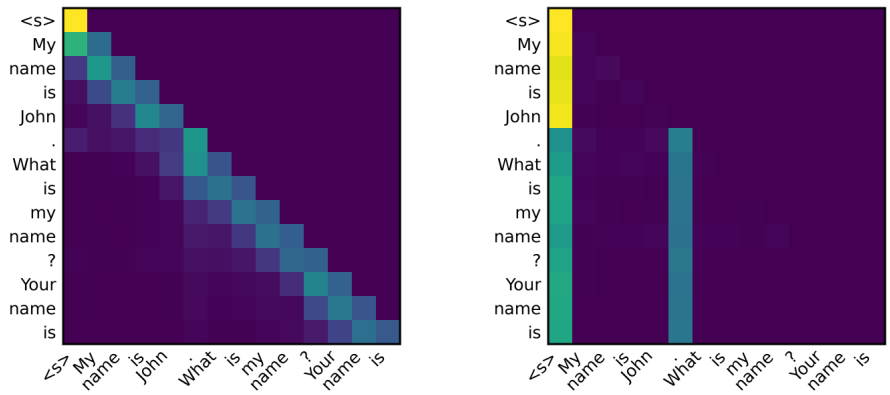
We find that the shift and broadcast operations of attention, which play an important role in learning anchor functions, are also very common in large language models (LLMs). We use Llama2-7B Touvron et al. (2023) as an example to demonstrate these two operations.

Similar to the identity-learning task, we use "My name is John. What is my name? Your name is" as the input to Llama2-7B and observe the attention maps at different layers. Fig. 12.17(a, b) shows two heads in Llama2-7B. The left one is a head in the second layer, exhibiting a clear shift operation; the second one is a head in the 29th layer, exhibiting a clear broadcast operation. The appendix provides various attention maps for many heads. Both operations are quite common. Furthermore, based on observations over all heads, we note that shallow-layer attention tends to perform shift operations, while deep-layer attention tends to perform broadcast operations.

12.7.4 Discussion of the shift and broadcast operations

In the language-like task of identity learning, we find that the Transformer's attention structure exhibits two patterns: shift and broadcast. The shift operation is task-dependent and does not depend on the input sequence; it arises because the distance between the anchor and the key token is fixed, and the attention structure learns this positional relationship and performs the shift operation. The broadcast operation, on the other hand, mainly depends on the input sequence; it can simplify the network, for example by ignoring irrelevant tokens and providing working memory for intermediate results. From the perspective of complexity, the broadcast operation enables a neural network to use low-complexity functions to accomplish a task.

Shift and broadcast are just two of the basic operations performed by Transformer attention matrices. Other operations of the attention matrix also exist; for example, induction heads associate the current token with similar tokens from previous context Elhage et al. (2021); Olsson et al. (2022). Future studies of various anchor functions will reveal more and more attention-structure patterns and basic operations of other components.



(a) The 23rd head of the 2nd layer

(b) The 17th head of the 29th layer

Figure 12.17: Example attention maps from Llama2-7B.

12.8 Experimental setup

This section describes the specific setup of our experiments on various anchor-function tasks, including model architecture, loss function, optimization algorithm, and hyperparameters. Through reasonable experimental design, we can comprehensively evaluate the model's performance on different tasks and deeply analyze its inner mechanisms.

12.8.1 Model architecture

We adopt a Transformer-based architecture to learn the various tasks. Fig. 12.18 shows a generic block of the Transformer network.

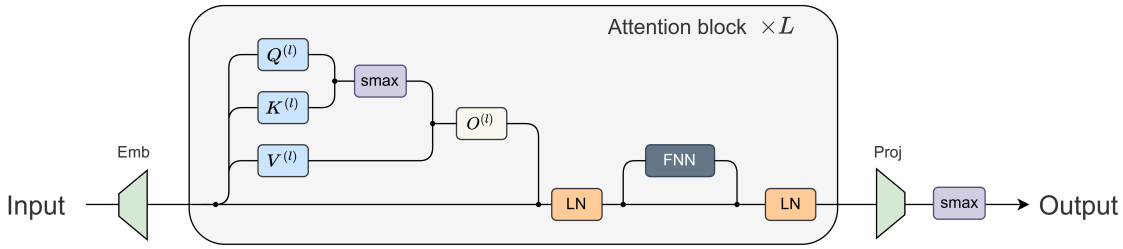


Figure 12.18: Schematic of the decoder-only Transformer model.

12.8.2 Loss function

In this study, we use the Transformer architecture, which takes as input a sequence of length $n = 9$ and then outputs a matrix $X^{(\text{out})} \in \mathbb{R}^{n \times d}$, where d is the vocabulary size. For the i-th

input sequence, the loss is the cross-entropy of the softmaxed output at the last token (denoted by $\mathbf{x}_n^{(\text{out},i)} \in \mathbb{R}^d$). The total loss over the entire training dataset is computed as:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{d} \boldsymbol{y}_c^{(i)} \log(\mathbf{x}_{n,c}^{(\text{out},i)}),$$

where $\boldsymbol{y}^{(i)}$ is the one-hot label of the i-th sequence, and N denotes the size of the training dataset.

12.8.3 Hyperparameter settings

Unless otherwise specified, in the subsequent tasks the hyperparameters used for training the Transformer network, as well as the LSTM and DNN networks used as comparison experiments, are as follows:

Transformer: We use a 4-layer decoder-only Transformer network to learn the various tasks. Each decoder layer has 4 heads. The chosen optimizer is AdamW. The batch size is set to 100. The initial learning rate is 2e-5. During the first 400 epochs, a warm-up learning rate scheduling strategy is used to gradually increase the learning rate to 2e-4, and then a cosine annealing strategy is used to decay the learning rate back to 2e-5. The network is trained for a total of 4000 epochs.

LSTM: We use a 4-layer LSTM to learn the various tasks. The chosen optimizer is AdamW. The batch size is set to 100. The initial learning rate is 2e-5. During the first 400 epochs, a warm-up learning rate scheduling strategy is used to gradually increase the learning rate to 2e-4, and then a cosine annealing strategy is used to decay the learning rate back to 2e-5. The network is trained for a total of 4000 epochs.

DNN: We use a 4-layer DNN to learn the various tasks. The chosen optimizer is AdamW. The batch size is set to 10. The initial learning rate is 2e-5. During the first 400 epochs, a warm-up learning rate scheduling strategy is used to gradually increase the learning rate to 2e-4, and then a cosine annealing strategy is used to decay the learning rate back to 2e-5. The network is trained for a total of 4000 epochs.

Chapter 13

The Effect of Complexity Control on the Reasoning Ability of Language Models

From the previous chapters, we have learned that large-scale Transformer models have demonstrated unprecedented capabilities, even hailed as the "sparks of artificial general intelligence". Despite this enormous success, they still exhibit significant gaps in reasoning ability, particularly when handling novel scenarios systematically. For instance, language models often struggle to consistently apply logical rules or extend knowledge to unfamiliar contexts, which highlights their limitations in systematic reasoning—a key aspect of human cognition. A central challenge in systematic reasoning is *compositional reasoning*, the ability to generalize from known concepts to novel combinations. Humans demonstrate remarkable proficiency in this domain; for example, once a child learns the concept of "jumping", they can effortlessly understand extensions such as "jumping backward" or "jumping twice around a cone". This compositional skill is integral to human reasoning and has long been a focal point of debate in the neural network community. Although modern models achieve outstanding performance on a variety of tasks, their ability to perform compositional reasoning, especially under out-of-distribution (OOD) scenarios, remains limited.

This raises critical open questions about how to accurately interpret Transformers' capabilities on compositional tasks: do Transformers genuinely learn the compositional primitives within the data, or do they primarily rely on memorizing input–output mappings? When they fail on compositional tasks, are these failures systematic, or do they reflect a deeper absence of reasoning structure? Addressing these questions is essential to understanding their mechanisms and limitations. Moreover, an understanding of these issues can inform strategies for enhancing model generalization, especially in OOD scenarios. In the chapter on anchor functions, we constructed composite tasks and conducted some preliminary explorations, finding that Transformers possess the capability of learning unseen anchor pairs.

In this chapter, we further analyze the intrinsic mechanisms by which the model learns composite tasks. The results show that model complexity has a significant impact on how the model learns. There are multiple ways to control complexity; here we focus on parameter initialization and the weight decay coefficient. For example, when the initialization scale is small or the weight decay

coefficient is large, the model tends toward inference, i.e., learning a "Reasoning-based Solution"; when the initialization scale is large and the weight decay coefficient is small, the model tends toward memorization, i.e., reciting training data without generalization ability.

Popular Science

- **What is model complexity?** Model complexity refers to the complexity of the solution obtained through training. In addition to directly inspecting the complexity of the model output, model complexity is typically measured by several indicators; in this chapter we focus on the following two:
  - **Number of effective neurons:** the number of condensed directions of the model's weight vectors; the larger this number, the higher the model complexity. Chapters 7 and 8 introduced the condensation phenomenon in neural networks, in which different weight vectors converge to the same direction. This phenomenon implies that in the nonlinear regime, the number of effective neurons can more effectively characterize model complexity.
  - **Norm of model parameters:** the norm of the trainable parameter vector. With the same number of model parameters, a larger parameter norm corresponds to higher model complexity.
- **What is complexity control?** Complexity control refers to adjusting certain settings during neural network training to influence the "complexity" of the solution the model eventually learns. In this chapter, we focus on two approaches:
  - **Parameter initialization scale:** the magnitude of the initial values assigned to model parameters (e.g., weights) at the start of training. A small scale means the initial values are close to zero; a large scale means the initial values are relatively large. Chapter 8 revealed that a smaller initialization scale more readily induces the condensation phenomenon, reducing the number of effective neurons and thus lowering model complexity.
  - **Weight decay:** a regularization technique that limits the size of the parameter norm by adding a penalty term to the loss function, preventing the model from becoming overly complex. The larger the weight decay coefficient, the stronger the constraint on the parameter norm, and the stronger the control over model complexity.

  By controlling these two factors, we can guide the model to learn different types of solutions.
- **Why does complexity control matter?** Model complexity is closely related to its learning strategy and generalization ability. Excessive complexity may lead the model to merely "memorize" the training data (memorization), making it difficult to generalize to unseen data (especially OOD data). Insufficient complexity, on the other hand, may prevent the model from fully learning the complex patterns in the data. Appropriate complexity control helps the model strike a balance: fitting the training data while also learning the underlying rules (reasoning), and thereby generalizing better.
- **How does complexity control affect the way a model learns?** The research in this chapter finds that:

  - **Strong complexity control (small initialization scale + large weight decay):** tends to guide the model toward a "**Reasoning-based Solution**". The model attempts to identify the basic rules or operational units (primitives) underlying the data, and solves problems by composing these primitives. Such solutions are typically simpler, more structured, and exhibit better OOD generalization.
  - **Weak complexity control (large initialization scale + small weight decay):** tends to guide the model toward a "**Memory-based Solution**". The model is more likely to directly memorize specific mappings between inputs and outputs rather than understand the underlying compositional rules. Although such solutions perform well on training data and similar ID test data, they typically perform poorly under OOD scenarios.

- **What are Reasoning-based and Memory-based Solutions?**
  - **Reasoning-based Solution:** the model learns the basic components of the task (e.g., individual arithmetic operations) and can flexibly apply these basic components to perform reasoning when encountering new combinations. Such a solution reflects the model's understanding of the task structure.
  - **Memory-based Solution:** the model primarily memorizes specific input–output pairs seen during training. When encountering unseen new combinations, the model struggles to handle them correctly due to its lack of understanding of the underlying rules.

- **How does complexity control affect the model's generalization ability (especially OOD generalization)?** Since OOD tasks typically require the model to understand and apply latent rules to new combinations, models that can learn Reasoning-based Solutions typically exhibit better OOD generalization. By adopting strong complexity control (small initialization, large weight decay), we can encourage the model to learn such Reasoning-based Solutions and thus improve its performance on OOD tasks.

13.1 Introduction

Within the human body lies an extraordinarily precise "blueprint of life" that determines the color of our eyes, the curliness of our hair, our ability to metabolize lactose, and even influences our athletic talents. Each key "instruction" in this blueprint is what we call a gene. Analogously, for large language models, their "genes" are the various hyperparameters, such as the number of layers, the width, the initialization scheme, and the training method. Different hyperparameters and their combinations ultimately shape different model behaviors.

In biology, after centuries of research, scientists have established correspondences between numerous genes and the human traits they influence. As shown in the left column of Fig. 13.1, genes such as SLC24A5 and HERC2 significantly affect skin color, eye color, and other traits, respectively. Continuing the analogy, investigating how different "model genes" (hyperparameters) affect a model's various abilities is equally a critical question, as it bears directly on whether we can design high-performing models that meet our intended goals.

Currently, reasoning ability is one of the core metrics for evaluating large language models. It determines whether a model can think effectively and perform logical deduction like humans, and thereby solve complex problems. For example, imagine feeding the astronomical data accumulated by Tycho Brahe into a model for training:

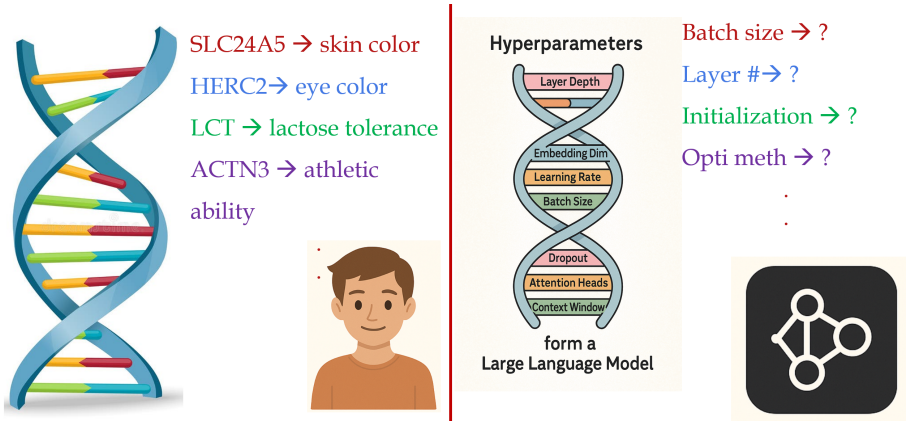


Figure 13.1: The genes of the human body and the "genes" of large models.

- Under what conditions does the model merely memorize this data?
- Under what conditions can the model, like Kepler, derive the three laws of planetary motion from the data?
- And under what conditions can the model develop higher-order reasoning abilities, like Newton, deriving the law of universal gravitation from the laws of motion?

This raises a key question: does there exist a particular "model gene" (hyperparameter), whose different settings or states fundamentally determine the level of reasoning ability the model eventually exhibits—whether it remains a "Tycho model" that merely memorizes data, becomes a "Kepler model" that discovers regularities, or a "Newton model" that constructs universal theories? In this chapter, through the example of composite functions, we show that the initialization scale and the weight decay coefficient are precisely such crucial genes that determine this key capability of large models, and we systematically investigate and analyze the underlying mechanisms.

13.2 Definitions

We will study the effect of complexity control on the reasoning ability of models via anchor functions. Before doing so, we introduce the key definitions used in this chapter. Fig. 13.3 briefly illustrates our definitions.

13.2.1 Two-Anchor Composite Functions

A two-anchor composite function applies two sequential operations to a specific token. This setup allows us to dissect whether the model learns these operations as discrete, composable rules, or merely memorizes specific input–output mappings. A two-anchor composite function $f(X) : \mathbb{R}^n \to \mathbb{R}$ is defined as:

$$f(x_1, \ldots, x_n) = g\left(g(x_{i-1}; x_i); x_{i+1}\right), \tag{13.1}$$

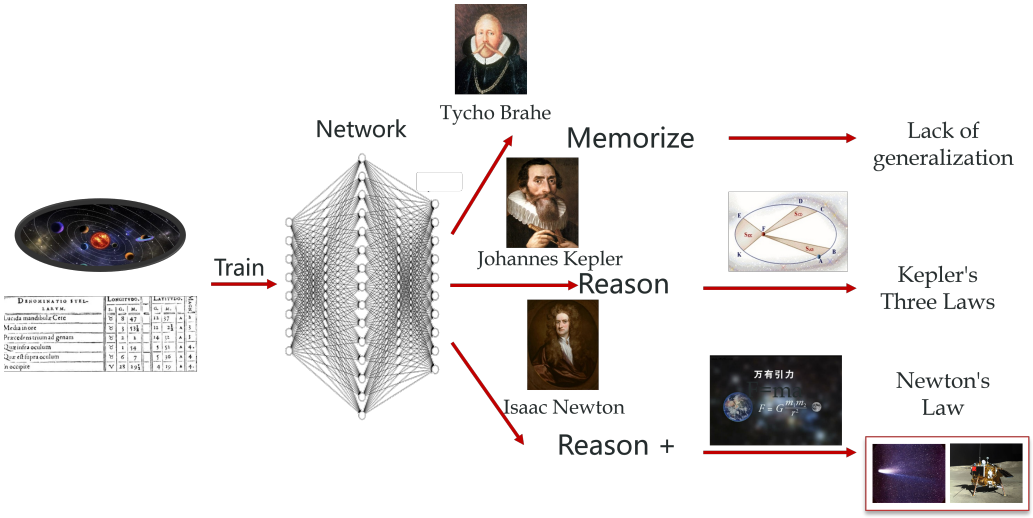


Figure 13.2: How can we obtain models with different levels of capability by training neural networks on astronomical data?

where $x_i, x_{i+1} \in A$. Here, the input sequence $X = (x_1, \ldots, x_n)$ contains n tokens. We specify an anchor set $A = \{a_1, a_2, \ldots, a_J\}$, in which each token $a_k \in A$ corresponds to a function $g(x; a_k)$. In each X, there is exactly one pair of consecutive elements belonging to A, e.g., $x_i, x_{i+1} \in A$. We refer to the token immediately preceding the anchor pair as the **key token**. To simplify notation, we denote the two-anchor composite function as $f(x_{i-1}; x_i, x_{i+1})$ to emphasize the anchor pair (x_i, x_{i+1}) and the key token x_{i-1}.

In this chapter, we set the anchor set $A = \{a, b, c, d\}$. Each anchor corresponds to a specific function:

$$\begin{aligned} g(x; a) &= x + 5, \quad g(x; b) = x + 1, \\ g(x; c) &= x - 2, \quad g(x; d) = x - 8. \end{aligned} \tag{13.2}$$

Example: Suppose we have an input sequence $X = (23, a, b, 43, 46, 74, 54, 44, 72)$. In this sequence, the second and third tokens (i.e., tokens a and b) belong to the anchor set A, and thus form an anchor pair (a, b). The token 23 immediately preceding the anchor pair is the key token.

For this input sequence X, the two-anchor composite function is computed as follows:

$$\begin{aligned} f(X) &= f(23; a, b) = g(g(23; a); b) \\ &= g(23 + 5; b) = g(28; b) = 28 + 1 = 29. \end{aligned}$$

13.2.2 Data Generation

Input data: In this study, we construct the input dataset using four anchors (i.e., a, b, c, d) together with numerical tokens sampled from 20 to 100. For each sequence, two anchors are selected (with repetition allowed) to form an anchor pair, and the sequence consists of this anchor pair together with other randomly sampled numerical tokens. The token immediately preceding the anchor pair is designated as the key token, and the remaining tokens are regarded as **noise tokens** that are

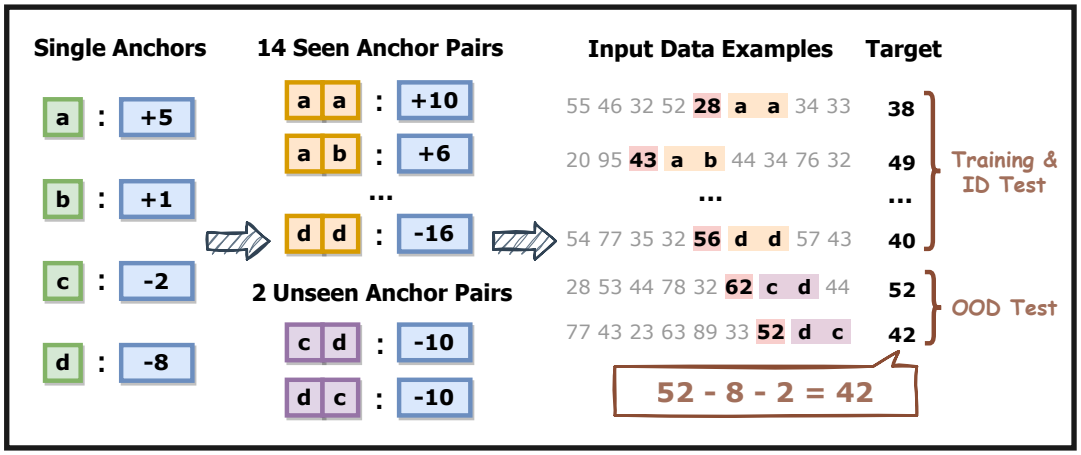


Figure 13.3: Experimental setup for compositional tasks. Left: each single anchor (i.e., a, b, c, d) corresponds to a specific arithmetic operation. Middle: during training, 14 out of the 16 possible anchor pairs appear in the training set, while the remaining anchor pairs (c, d) and (d, c) are held out as unseen tasks (not appearing during training). Right: an input sequence contains an anchor pair, a key token immediately preceding the anchor pair, and noise tokens that are irrelevant to the target. We construct mutually exclusive training and ID test sets from data generated by the 14 seen anchor pairs, while data from the remaining 2 unseen anchor pairs are used to form the OOD test set.

irrelevant to the target. The noise tokens are primarily used to separate the training set from the ID test set. The four anchors yield 16 anchor pairs, of which (c, d) and (d, c) are designated as **unseen anchor pairs**. The training dataset is built from the remaining 14 **seen anchor pairs**.

Training target: the output of the two-anchor composite function applied to the key token, i.e., the corresponding output value of the two-anchor composite function.

Dataset split: we split the dataset into three categories: the training set, the ID test set, and the OOD test set. For the training set and the ID test set, anchor pairs are drawn from the 14 seen anchor pairs, excluding (c, d) and (d, c). For the OOD test set, anchor pairs are drawn from (c, d) or (d, c).

13.2.3 Initialization and Regularization Parameters

- **Initialization rate:** the initialization rate, denoted γ, controls the scale of parameter initialization within the model. Specifically, model parameters are initialized by sampling from a normal distribution $\mathcal{N}\left(0, \left(\frac{1}{d_{\text{in}}^{\gamma}}\right)^2\right)$, where d_{in} denotes the input dimension. A larger γ leads to a smaller initialization scale.
- **Weight decay coefficient:** the weight decay coefficient is the hyperparameter of L_2 regularization during training, denoted λ. It controls the regularization strength by adding $\lambda \|W\|_2^2$ to the loss function, where W denotes the weights of the model.

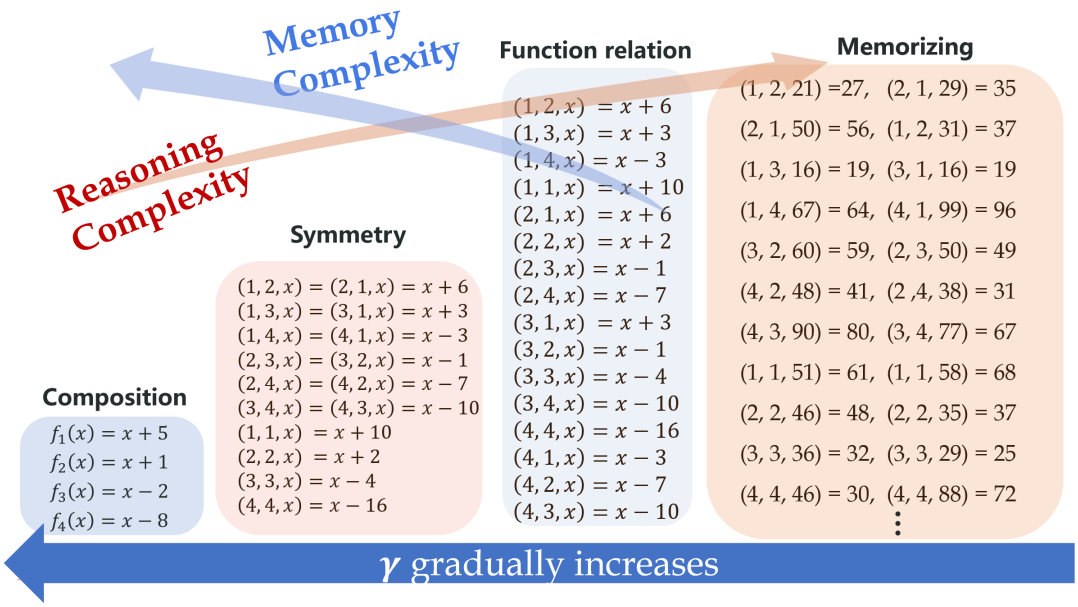


Figure 13.4: Compositions of four single-anchor functions can form mappings with the same output but different complexities.

13.2.4 Hierarchical Structure of the Data

As shown in Fig. 13.4, the two-anchor composite functions generated as described in Section 13.2.2 admit forms with different reasoning and memorization complexities: they may compose all combination mappings purely from minimal single-anchor functions; they may memorize some symmetric structure; they may memorize the function relation corresponding to every two-anchor pair; or they may merely memorize the output of a key token under a specific anchor pair. As the initialization rate γ gradually decreases (i.e., the initialization scale gradually increases), the memorization complexity of the data gradually increases, while the reasoning complexity gradually decreases.

13.2.5 Generalization

The dataset split naturally introduces the following two notions of generalization:

- **In-Distribution (ID) generalization:** the generalization ability on the ID test set, in which all anchor pairs have appeared in the training set.
- **Out-of-Distribution (OOD) generalization:** the generalization ability on the OOD test set, in which the anchor pairs have not appeared in the training set.

13.2.6 Model Architecture and Basic Experimental Setup

For basic analysis and to simplify the exploration of mechanisms, we first adopt a single-head Transformer model. Subsequent experiments extend these findings to the multi-head architecture

of GPT-2 to ensure the generality of our conclusions. Below we only introduce the single-head attention architecture; the multi-head case is a natural extension of the single-head model.

The input sequence is represented as a one-hot vector X^{in}. The token embedding X^{em} and the input $X^{(1)}$ of the first Transformer block are computed as:

$$X^{\text{em}} = X^{\text{in}} W^{\text{em}}, X^{(1)} = X^{\text{em}} + X^{\text{pos}}, \tag{13.3}$$

where X^{pos} is a trainable positional vector. For the l-th layer, Q, K, V are defined as:

$$Q^{(l)} = X^{(l)} W^{Q(l)}, K^{(l)} = X^{(l)} W^{K(l)}, V^{(l)} = X^{(l)} W^{V(l)}. \tag{13.4}$$

The attention matrix $\text{Attn}^{(l)}$ at the l-th layer and its subsequent output $X^{\text{qkv}(l)}$ are computed as:

$$\begin{aligned} \text{Attn}^{(l)} &= \text{softmax}\left(\frac{Q^{(l)} K^{(l)T}}{\sqrt{d_k}}\right) \quad (\quad), \\ X^{\text{qkv}(l)} &= \text{Attn}^{(l)} V^{(l)}. \end{aligned} \tag{13.5}$$

The output of the attention layer at the l-th layer is:

$$\begin{aligned} X^{\text{ao}(l)} &= \text{LN}(X^{(l)} + X^{\text{qkv}(l)} W^{\text{attn},l}), \\ X^{(l+1)} &:= X^{\text{do}(l)} = \text{LN}(\text{MLP}(X^{\text{ao}(l)}) + X^{\text{ao}(l)}). \end{aligned} \tag{13.6}$$

where "LN" denotes Layer Normalization. The final output is obtained by projecting the output of the last layer $X^{\text{do}(L)}$ through a linear projection layer, followed by a softmax operation and an argmax to obtain the predicted token.

We use $f_{\boldsymbol{\theta}}(x; a_1, a_2; \boldsymbol{n})$ to denote the neural network output for a sequence with key token x, anchor pair (a_1, a_2), and noise token sequence $\boldsymbol{n}$. In this case, the input sequence is uniquely determined by the values and positions of the noise tokens in $\boldsymbol{n}$.

For the basic experimental setup, we use the cross-entropy loss on the last token of the sequence, and optimize the model using the Adam optimizer with weight decay.

13.3 Phase Partition of Solutions for Composite Functions

In this section, we explore the distinct characteristics of Transformer-based models (specifically GPT-2) when learning compositional tasks with the ID and OOD datasets defined in Section 13.2.2. Our investigation focuses on understanding how variations in the initialization scale and the weight decay coefficient affect the model's generalization ability.

First, we evaluate the impact of different initialization scales on the ID and OOD performance of the model, while fixing the weight decay coefficient at 0.01, as shown in Fig. 13.5. The horizontal axis represents the initialization rate γ, and the vertical axis represents the accuracy achieved on ID data (blue) and OOD data (red). According to ID and OOD generalization ability, we classify different initialization settings into three phases:

- **Phase 1 (poor ID and poor OOD generalization):** the model relies mainly on memorization, with limited ID and OOD generalization. This corresponds to very large or very small initialization scales (very small or very large γ).

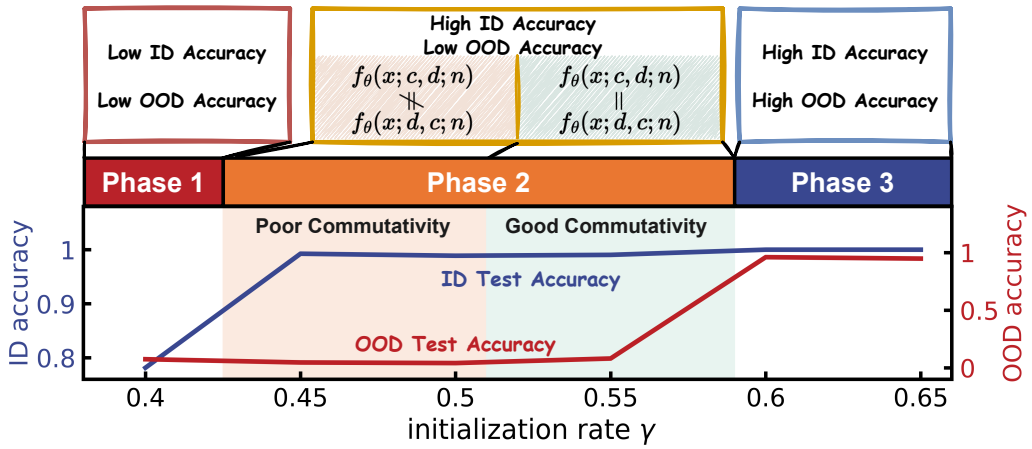


Figure 13.5: ID and OOD generalization of GPT-2 models on the compositional task when the weight decay coefficient is fixed at 0.01. The horizontal axis represents the initialization rate γ, which corresponds to the standard deviation $(1/d_{\text{in}})^{\gamma}$ of the normal distribution used for parameter initialization. The vertical axis represents the accuracy on ID (blue) and OOD (red) data. Different phases are classified according to their ID and OOD generalization ability.

- **Phase 2 (good ID but poor OOD generalization):** the model learns the composite anchor mappings (i.e., the overall effect of specific anchor pairs), achieving good ID generalization but lacking an abstract understanding of individual anchor operations, leading to poor OOD generalization. This typically occurs at moderate to large initialization scales (small to moderate γ).
- **Phase 3 (good ID and good OOD generalization):** the model learns the mapping of individual anchors, understanding the task structure from a higher level, which enables strong OOD generalization on unseen anchor pairs. This typically requires a small initialization scale (large γ).

To further elucidate the model's reasoning ability, we examine its learning of **commutativity**, a fundamental property of the addition task. Specifically, we evaluate whether the model's output remains consistent when switching the unseen anchor pair from (d, c) to (c, d) while keeping the key token and noise tokens fixed. (Note: we are not concerned with the relation between the output and the target value here. For instances with high OOD accuracy, the model naturally satisfies commutativity since the output matches the target.) Within Phase 2, commutativity varies depending on the initialization rate. For larger initialization rate γ (small parameter initialization scale), the model treats symmetric anchor pairs as a unified entity, and therefore exhibits good commutativity. Formally, for most key tokens x and noise sequences $\boldsymbol{n}$, $f_{\boldsymbol{\theta}}(x; c, d; \boldsymbol{n}) = f_{\boldsymbol{\theta}}(x; d, c; \boldsymbol{n})$ holds. In contrast, for smaller initialization rate γ (large parameter initialization scale), the model treats each anchor pair as an independent mapping, resulting in poor commutativity.

In addition to the initialization scale, we also investigate the role of the weight decay coefficient in shaping the model's generalization ability, as shown in Fig. 13.6. This analysis examines the combined effect of varying the initialization rate γ and the weight decay coefficient on OOD performance, represented by the colors in the heatmap. The striped region indicates poor ID

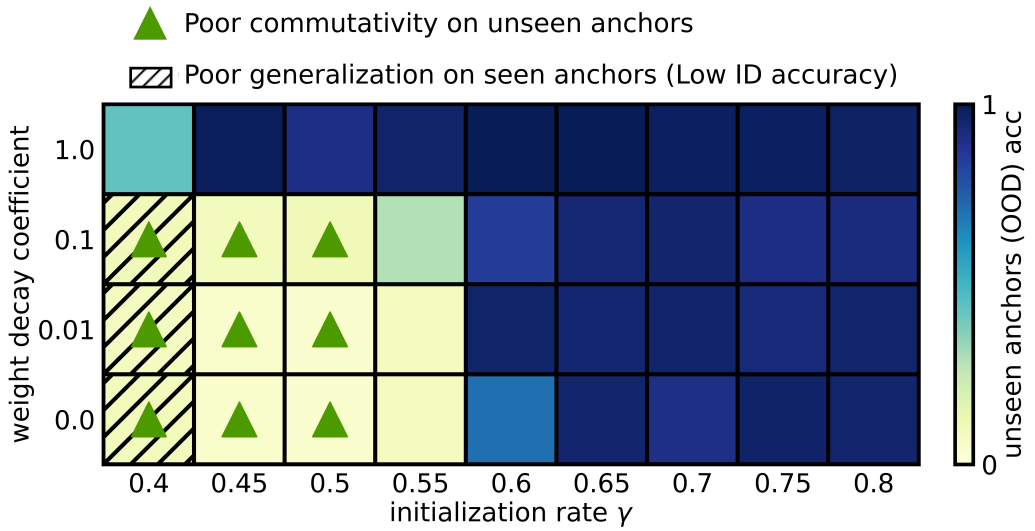


Figure 13.6: Heatmap showing the OOD generalization of GPT-2 models on the compositional task, with the accuracy on unseen anchor pairs represented by color intensity. The horizontal axis is the same as in Fig. 13.5, and the vertical axis represents the weight decay coefficient. Each setting reflects the average result of three independent trials. The striped region indicates poor ID generalization (ID accuracy $< 90\%$). The green triangles highlight cases of poor commutativity (commutativity probability $< 70\%$) when switching unseen anchor pairs (c, d) and (d, c).

generalization (ID accuracy $< 90\%$), while the green triangles indicate cases with commutativity probability below 70% when switching the unseen anchor pair from (d, c) to (c, d).

This pattern remains consistent across all trials, with results averaged over three random seeds. We conclude that as the initialization rate and the weight decay coefficient gradually increase (i.e., complexity control is strengthened), the model undergoes a transition from basic memorization to a deeper understanding of the task structure, thereby enhancing reasoning ability and mastering the underlying rules in complex generalization tasks.

We treat the adjustment of the initialization rate and the weight decay coefficient as forms of **complexity control** on the model, as the values of these parameters significantly affect the model's complexity characteristics. Consequently, models with different complexity levels exhibit markedly different internal mechanisms. In the subsequent sections, we will further explore the mechanisms and complexity differences of models in different phases.

13.4 Mechanism Analysis of Different Phases via Attention Masking Strategy

In this section, we analyze the mechanisms behind different phases by selectively **masking** specific information circuits. This approach allows us to isolate and inspect the contributions of the key token and the anchor pair, systematically study the output space, and reveal the internal mechanisms of the model in different phases. Specifically: i) We study the distribution patterns of different anchor pairs in the output space by masking the information of the key token (Fig. 13.7A, 13.8). ii) We explore the distribution patterns of the key token and individual anchors in the output space by masking the information of the second anchor (Fig. 13.7B, 13.9).To this end, we mask specific circuits in the model to exclude information of certain tokens from the output (masked activation), allowing us to study the model's behavior when only the other input information is present. Masking is implemented by disabling specific circuits, as indicated by the dashed gray lines in Fig. 13.7A and 13.7B, while retaining the normal circuits (solid blue lines) for the unmasked tokens. To preserve the structure of the normal circuits, the masking is applied on the attention matrix after the softmax operation. For simplicity, we use a two-layer single-head Transformer model for the analysis.

13.4.1 Mechanism Analysis via Masking the Key Token

Based on the masking strategy described above, we analyze the mechanisms behind different phases of the model by visualizing the output embeddings using t-SNE. Fig. 13.8 shows how the model organizes the representations of the key token and the anchor pair under different conditions. For this analysis, we randomly sample 50 data points per anchor pair to evaluate the model's effectiveness in distinguishing different mappings and whether it respects task-specific properties such as commutativity. To enhance clarity, we use similar colors with different shades to denote symmetric anchor pairs (e.g., (c, d) and (d, c)), indicating their potential equivalence under the commutativity operation.

The three phases observed in Fig. 13.8 are realized by adjusting the initialization rate (γ), which controls the standard deviation of parameter initialization. In **Phase 1** (left panel of Fig. 13.8), the representations of different anchor pairs are disorganized, with significant overlap among clusters. This indicates that the model fails to capture the distinct mappings associated with each anchor pair,

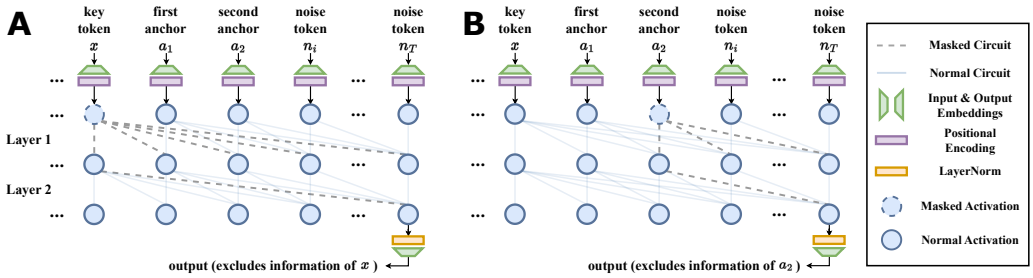


Figure 13.7: **(A, B)** Masking strategies applied to the Transformer model for analyzing the contributions of specific tokens. **(A)** The key token (x) is masked, removing its information from the model output, allowing analysis of the output space distribution influenced by the anchor pair (a_1, a_2) and the noise tokens. **(B)** The second anchor (a_2) is masked to isolate the interaction between the key token (x) and the first anchor (a_1). Dashed gray lines indicate masked circuits, while solid blue lines indicate normal circuits. Hollow nodes depict masked activations, while solid nodes depict normal activations.

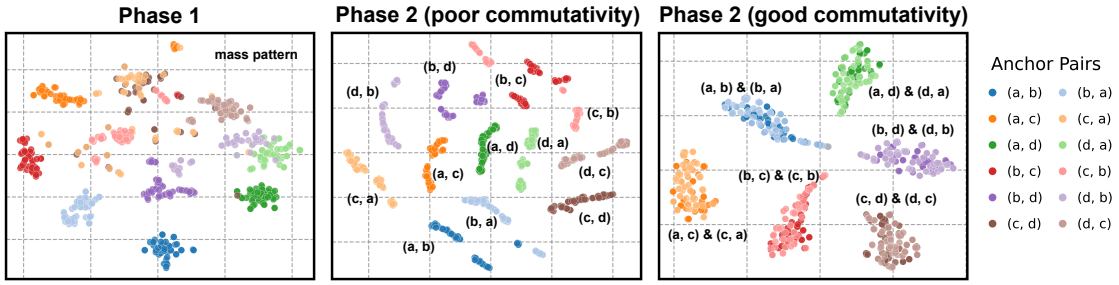


Figure 13.8: Principal Component Analysis (PCA) is applied to the model outputs to visualize the model's representations of different anchor pairs. 50 data points are sampled per anchor pair. Symmetric anchor pairs (e.g., (c, d) and (d, c)) are shown in similar colors with different shades to indicate their equivalence. The three phases are realized by adjusting the initialization rate (γ).

leading to entangled representations and poor generalization. The lack of clear structure reflects the model's reliance on memorization rather than learning the compositional rules underlying the task. Consequently, the model performs poorly on both ID and OOD data, indicating its inability to generalize within the training distribution.

In **Phase 2** (middle and right panels of Fig. 13.8), the model begins to capture the structural relations between the key token and the anchor pairs, as manifested by the emergence of clear and well-separated clusters. In the middle panel, the model treats symmetric anchor pairs (e.g., (c, d) and (d, c)) as independent clusters, failing to recognize their equivalence. This lack of abstraction leads to poor commutativity despite improved ID generalization. In contrast, the right panel illustrates a more advanced model state, in which symmetric pairs successfully merge into a single cluster. This structural improvement not only achieves good commutativity and enhanced generalization, but also suggests a transition toward a simpler, rule-based approach to fitting the data. The transition from disorganized to highly structured clusters highlights the model's tendency, in later phases, to adopt lower-complexity and more systematic solutions.

Note that Phase 3 is a special case of solutions with good commutativity. Therefore, it is infeasible to analyze the difference between Phase 2 with good commutativity and Phase 3 solely from the perspective of composite anchors. In the next subsection, we focus on analyzing these differences at the single-anchor level to reveal the underlying mechanism by which the model learns the task.

13.4.2 Mechanism Analysis via Masking the Second Anchor

To investigate the mechanism behind the single-anchor operation and its consistency with the single-anchor function $g(\cdot;\cdot)$ defined in Eq. (13.2), we analyze the distribution pattern of the key token and a single anchor in the output space by masking the information of the second anchor, as described in Fig. 13.7B. In Fig. 13.9, the color represents the pairwise cosine similarity between the outputs of the masked model (with only the contributions of the key token and the first anchor isolated). Each group of four blocks corresponds to inputs that produce the same value of $g(x; a_1)$, and the zoomed-in panel on the right provides detailed input values of x and a_1 for these selected blocks.

In the left panel of Fig. 13.9 (corresponding to **Phase 2**), the output embeddings with the same value of $g(x; a_1)$ exhibit low correlation, as shown by the sparse red diagonal and generally low cosine similarities. This observation indicates that even though the model captures the symmetric relations, it still relies on memorizing the composite mappings to fit the data. In contrast, the middle panel represents **Phase 3**, showing significantly higher correlations among inputs with the same value of $g(x; a_1)$, as shown by the more prominent red regions on the diagonal blocks. This indicates that the model computes the single-anchor mappings step by step, following the defined function $g(\cdot;\cdot)$ to produce the final output. The zoomed-in view on the right highlights consistent cosine similarity patterns across input values within each block, further confirming the step-by-step reasoning mechanism in Phase 3.

These results illustrate the fundamental difference between the two phases: Phase 2 relies on memorizing composite mappings, while Phase 3 demonstrates a reasoning-based strategy by computing single-anchor mappings step by step. This transition highlights the model's ability in Phase 3 to leverage compositional rules to generalize on OOD data, rather than relying on memorization, and indicates a transition toward simpler and more structured data-fitting strategies.

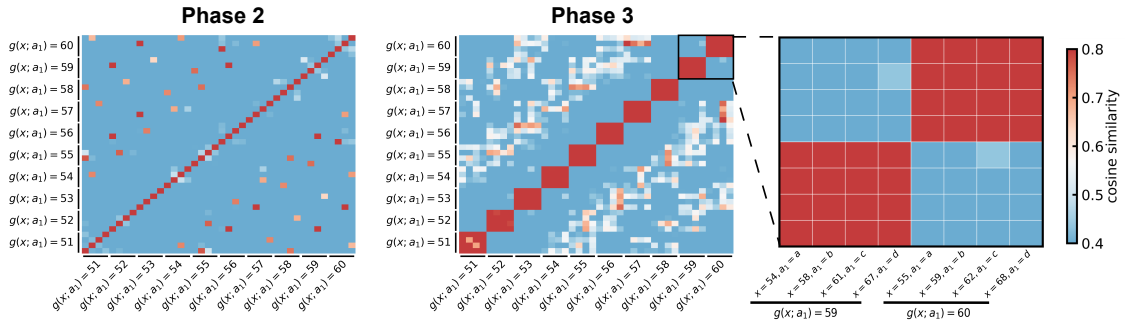


Figure 13.9: Cosine similarity matrix between the model outputs after masking the second anchor (a_2). Each group of four blocks corresponds to inputs with the same value of $g(x; a_1)$. The right panel provides a zoomed-in view of selected blocks, showing the detailed values of x and a_1 for the corresponding inputs. The color scale represents the cosine similarity values. Different phases correspond to different values of the initialization rate γ.

13.5 Model Complexity: A Key Factor in Phase Transitions

Based on the mechanism analysis in the previous section, we hypothesize that the **complexity preferences** of the model play a key role in driving phase transitions. In this section, we examine this phenomenon from two key perspectives: the **condensation** of input weights, and the **structured organization** of the token embedding matrix.

Condensation occurs when the input weights of neurons concentrate along a few isolated directions, effectively reducing the model's complexity and approximating it as a network of smaller scale. However, parameters such as the token embedding matrix, which need to encode distinctions among different input tokens, resist condensation and may form a systematic, rule-based structure. We examine the complexity of the model in different phases from these two perspectives.

13.5.1 Condensation of Input Weights

To study the parameter condensation phenomenon, we analyze the cosine similarity between the input weights of neurons in the first-layer query weight matrix ($W^{Q(1)}$). The similarity between the i-th and j-th neurons is computed as:

$$\frac{W^{Q(1)}[i,:] \cdot W^{Q(1)}[j,:]}{\|W^{Q(1)}[i,:]\|_2 \|W^{Q(1)}[j,:]\|_2},$$

and the results are visualized in Fig. 13.10 for each phase.

In **Phase 1**, the input weights show no significant condensation, and the cosine similarities among neurons are low. This indicates that the neurons are independently oriented in weight space, corresponding to a high-complexity state. In **Phase 2**, we observe mild condensation: neurons begin to cluster along a finite number of directions, leading to a slight increase in similarity. Finally, in **Phase 3**, the input weights are strongly clustered along a few isolated directions, forming distinct neuron groups with high intra-group similarity. This pronounced condensation reflects a significant reduction in complexity, allowing the model to approximate a lower-dimensional structure while maintaining task performance.

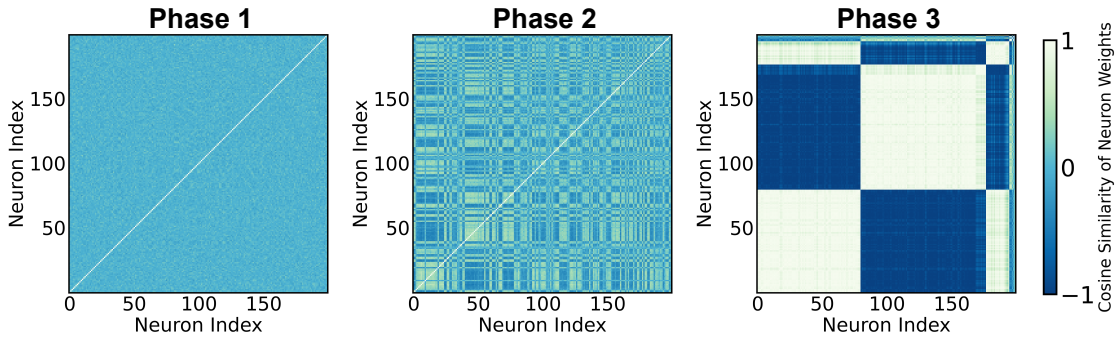


Figure 13.10: Cosine similarities between the input weights of neurons in the first-layer query weight matrix ($W^{Q(1)}$) for each phase. The horizontal and vertical axes both represent neuron indices. The matrices are computed under the setting where the weight decay coefficient is fixed at 0.01 and the initialization rate (γ) is set to 0.2, 0.5, and 0.8 to correspond to the settings of Phase 1, Phase 2, and Phase 3, respectively.

Fig. 13.11 more systematically illustrates the cosine similarity matrices of input weights of parameter matrices at different layers under different initialization scales. It reveals that as the initialization scale increases, the neuron condensation phenomenon becomes more pronounced, indicating a reduction in model complexity.

13.5.2 Structured Organization of the Token Embedding Matrix

To explore the structural properties of the model's token embedding matrix, we use PCA to visualize the embedding space, as shown in Fig. 13.12. Each number represents the position of the embedding vector corresponding to a specific token after PCA dimensionality reduction.

In **Phase 1** and **Phase 2**, the embedding vectors do not exhibit any clear structure, with the positions of tokens scattered irregularly in the dimension-reduced space. This lack of organization indicates that the model does not encode systematic relations among tokens. However, in **Phase 3**, the PCA visualization reveals a highly structured pattern, with token embeddings arranged in an ordered and regular manner. This structure reflects the model's tendency to encode relations in a systematic, rule-based manner, enabling it to generalize effectively while maintaining low complexity.

It is worth noting that the relative ordering of the token embedding matrix in Phase 3 does not correspond to a simple numerical magnitude ordering of the tokens. This ordering may stem from the definitions of the four single anchors, where the difference between any two single-anchor operations can be obtained via additions of the basic elements 3 and 4. This arrangement is consistent with the numbers being ordered in the embedding space at intervals of 3 (green arrows) and 4 (blue arrows) from two directions.

13.5.3 The Relation Between Condensation and Reasoning

For the case of very small parameter initialization, when training begins, parameters cluster along a few directions; as training progresses, more clustering directions gradually emerge, and the model's

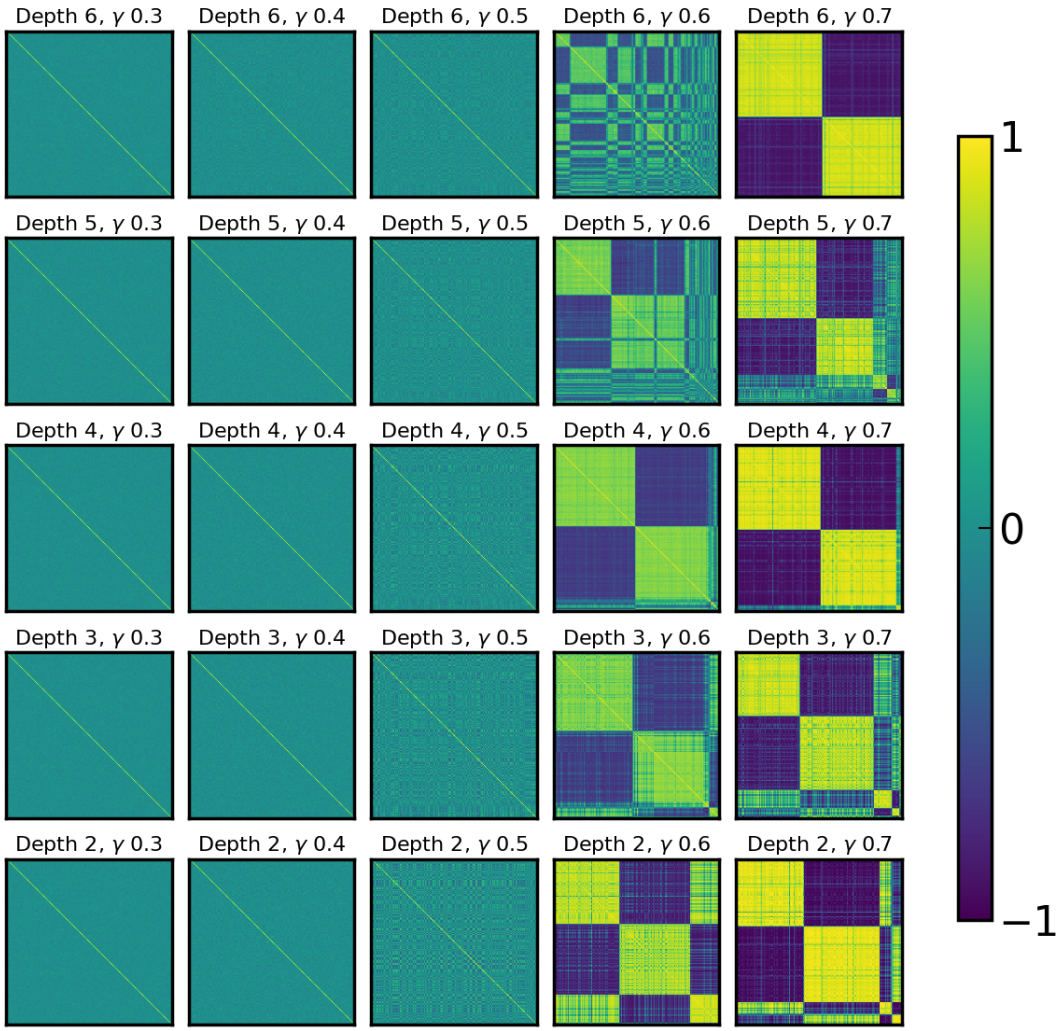


Figure 13.11: Cosine similarity matrices of the neurons of $W^{Q(1)}$ across different depths and initialization scales. Each subplot corresponds to a different depth (2 to 6 layers, arranged from bottom to top) and initialization rate γ (0.3 to 0.7, from left to right). The colors represent the cosine similarities between neurons, with warmer tones indicating higher similarity. Neurons are grouped by cosine similarity > 0.7 to highlight the condensation property.

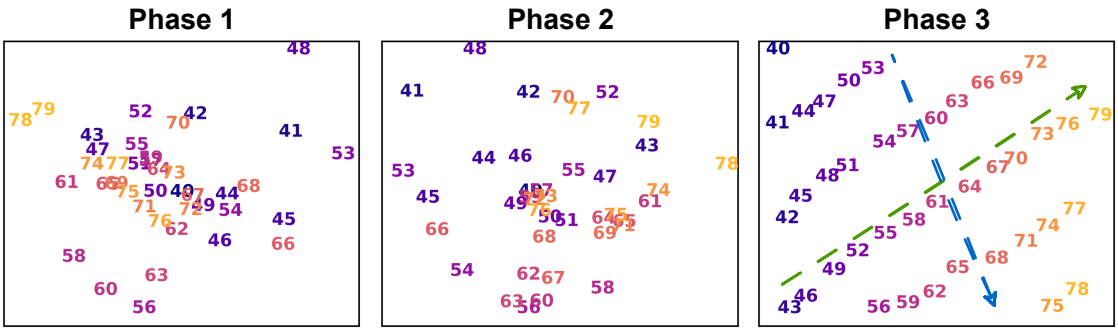


Figure 13.12: PCA visualization of the token embedding vectors under the same settings as in Fig. 13.10. Each number corresponds to a specific token, with its position representing the dimension-reduced embedding obtained by PCA.

complexity slowly grows, allowing it to memorize more. But increasing complexity incurs a training cost. Once the model finds that memorizing 4 mappings is enough to fit the training set, learning stops, and therefore it prefers solutions that involve reasoning.

If the initialization is larger, the model is already capable of memorizing ten functions at the start, but it cannot easily memorize more; nor will it pay the cost of seeking the more fundamental principle. The solution it learns is therefore the symmetric solution.

If we continue to increase the initialization, the model is initially capable of memorizing 16 functions but not more, so the model learns these 16 function relations.

Under even larger initialization, the complexity is high enough that the model can memorize the mapping from every input sequence to its output. As a result, it fails to learn the arithmetic rules and cannot even guarantee a perfect reproduction of the seen anchor combinations.

Experiments show that complexity control can be achieved by adjusting the initialization rate and the weight decay coefficient. Here we want to emphasize the importance of γ. Recalling the knowledge related to the phase diagram analysis, we know that γ controls the dynamical regime of the model. When the model width is not too narrow, networks with different widths but the same γ exhibit similar dynamical behavior, that is, they can learn similar solutions. This is especially important when training large models, since hyperparameter tuning is rarely performed on very large networks. Making large and small models exhibit similar dynamical behavior allows results obtained on small models to transfer to large ones. For the parameter governing initialization weights, γ can keep the behavior of networks of different scales as similar as possible, whereas adjusting the standard deviation of the initialization distribution cannot achieve this at all.

13.6 Further Validation on Real-World Tasks

We validate the performance of models with different initialization scales and weight decay settings on a series of compositional and reasoning tasks. Below, we introduce each task and its corresponding results.

Compositional diffusion task: Concept Graphs

The Concept Graphs dataset provides a synthetic and structured framework designed to evaluate

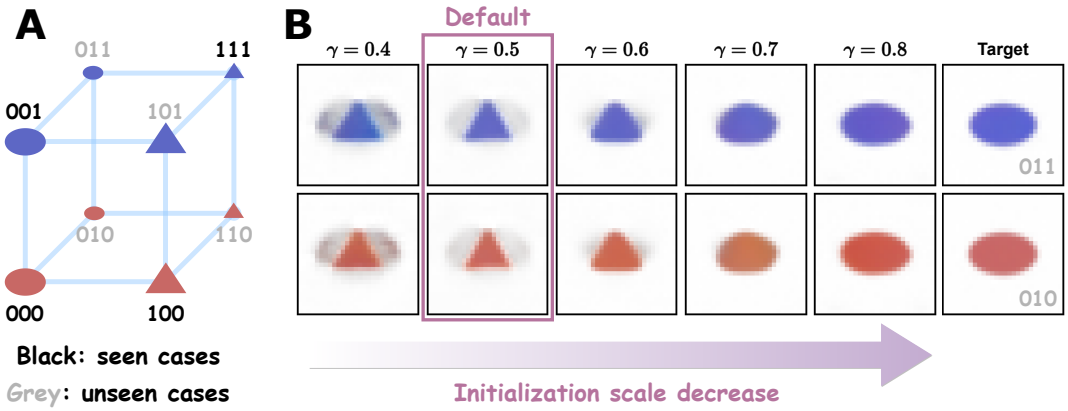


Figure 13.13: **(A)** Structure of the Concept Graphs dataset. The dataset is organized into a cubic framework based on three core conceptual variables—color, shape, and size—each taking specific values (e.g., color is red or blue, shape is circle or triangle, size is large or small). Black nodes represent the sparse subset of training data used to identify the interesting failure mode. **(B)** The impact of different initialization rates on the compositional generalization of cases 011 and 010. As the initialization rate increases (corresponding to a decreasing initialization scale), the model transitions from exhibiting the interesting failure mode to achieving robust compositional generalization. The purple box highlights the default initialization setting used in prior work.

compositional generalization in generative models. The dataset is built on three core conceptual variables—color, shape, and size—each taking specific values (e.g., color is red or blue, shape is circle or triangle, size is large or small), and organizes the relations among different conceptual combinations in the form of a cube, as shown in Fig. 13.13A. Prior work Okawa et al. (2024) identified a notable failure case of compositional generalization, which they called the "**interesting failure mode**". This failure arises when the model is trained on a sparse subset of the data (specifically the black nodes in Fig. 13.13A: 000, 100, 001, and 111). When tested on an unseen tuple (e.g., 010, representing a small red circle), the model incorrectly generates a small triangle instead of the expected small circle. This indicates that the model fails to achieve compositional generalization under such conditions.

Building on our analysis of how initialization affects the model's ability to learn compositional tasks, we trained models for this task with different initialization scales. We find that when the initialization scale is small, the "interesting failure mode" disappears, and the model achieves robust compositional generalization. As shown in Fig. 13.13B, we gradually increase the initialization rate (i.e., decrease the initialization scale) from left to right. The model's outputs on the two unseen conceptual tuples (010 and 011) gradually transition from triangles to circles, consistent with the target shape. The purple box highlights the initialization setting used in prior work, referred to as the default initialization. Note that, to faithfully reproduce the behavior observed in prior work at $\gamma = 0.5$ (the default setting), we used a uniform distribution for parameter initialization in this experiment, $\boldsymbol{\theta} \sim \mathcal{U}(-d_{\text{in}}^{-\gamma}, d_{\text{in}}^{-\gamma})$ (PyTorch's default `kaiming_uniform` distribution, which was also adopted in prior work). Consistent results were also observed when using a normal distribution for initialization.

Compositional tasks: SCAN and COGS

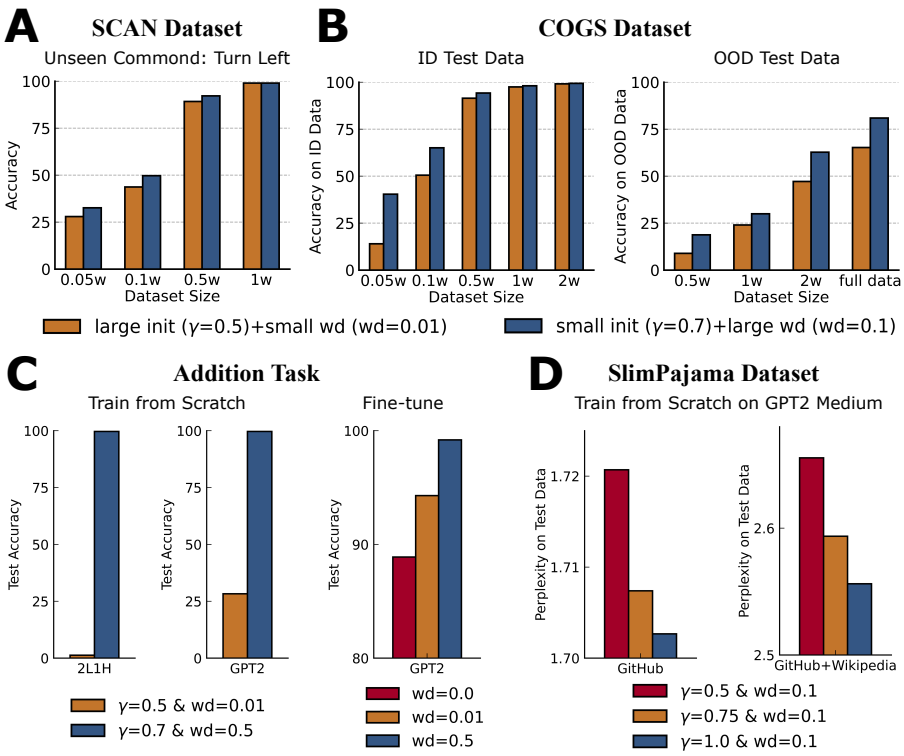


Figure 13.14: **(A, B)** Performance comparison of models with different initialization scales and weight decay coefficients on compositional tasks. **(A)** For the SCAN task, we evaluate the generalization ability on composite commands containing the "turn left" command. **(B)** For the COGS task, we evaluate (left panel) ID and (right panel) OOD generalization after training on the same dataset. Small initialization with large weight decay (blue) consistently outperforms large initialization with small weight decay (orange) across different tasks and data scales. Parameters are initialized following a zero-mean normal distribution with standard deviation $d_{in}^{-\gamma}$. **(C)** Performance comparison of models with different initialization scales and weight decay coefficients on the addition task. We use a case-based reasoning intervention experiment, with the test set $a, b \in [400, 600]$ and the remaining data as the training set. For the from-scratch training part, parameters are initialized following a zero-mean normal distribution with standard deviation $d_{in}^{-\gamma}$. For the fine-tuning part, we use the pre-trained weights of the GPT-2 model provided by Hugging Face as the starting point of fine-tuning. **(D)** Performance comparison of models with different initialization scales and weight decay coefficients on the SlimPajama dataset. For the SlimPajama dataset, the GPT-2 Medium model trained on GitHub and GitHub+Wikipedia data consistently achieves lower perplexity with a small initialization scale. Parameters are initialized following a zero-mean normal distribution with standard deviation $d_{in}^{-\gamma}$.

SCAN and COGS are classical compositional tasks with more natural-language variations. For the SCAN dataset, we choose the "Generalizing composition across primitive commands" task, in which the "turn left" command appears only in single-command mappings and is trained together with other composite commands. We evaluate the model's generalization ability on composite commands containing the "turn left" command. For the COGS dataset, we evaluate ID and OOD generalization after training on the same set. The ID test uses data with the same compositional patterns but different primitives, while the OOD test uses data that follows different compositional rules.

As shown in Fig. 13.14A, 13.14B, we demonstrate the generalization performance of models with different initialization scales and weight decay coefficients across various data scales. Small initialization with large weight decay (blue) consistently outperforms large initialization with small weight decay (orange) across different task types and data scales. Notably, on the COGS task, even when ID generalization (with 20k training data) of both settings reaches over 99%, the difference in OOD generalization remains significant.

Real-world task: Legal document reasoning

In this experiment, we train a Transformer model with 180 million parameters using different initializations, fitting a dataset of 40 billion tokens via next-token prediction. We then take an unseen legal text to test the model's reasoning ability. As shown in the left panel of Fig. 13.15, with a large initialization scale (small initialization rate γ), the pre-trained model with high complexity cannot perform meaningful next-token prediction on unseen test data. With a moderate initialization rate, as shown in the middle panel of Fig. 13.15, the model with moderate complexity (the commonly used initialization scale) demonstrates basic grammatical knowledge (e.g., "finds that") and lexical knowledge (e.g., "Jessica"), as well as the induction head mechanism (Olsson et al., 2022) (e.g., the repeated phrase "September 12, 2024"). When we further increase the initialization rate, as shown in the right panel of Fig. 13.15, the low-complexity model (i.e., small initialization scale) further captures complex semantic reasoning, successfully predicting "Plaintiff Michael Anderson" based on the commonsense reasoning that, in the context of "Anderson v. Carter", the plaintiff is generally listed first, even though the full name "Michael Anderson" does not appear earlier. Moreover, the model accurately predicts "required medical treatment" under the context that Anderson was assaulted by Carter.

Real-world task: Addition task and the SlimPajama dataset

Unlike conventional addition tasks, we use a **case-based reasoning intervention experiment** to study the generalization of rule learning in the addition task. Specifically, we consider the setting: $a+b=c$, where $a, b \in [0, 999]$. We use $a, b \in [400, 600]$ as the test set, and the remaining data as the training set. This construction prevents the model from simply imitating training data similar to the test set. We train a simple 2-layer 1-head model and a GPT-2 model. As shown in Fig. 13.14C, regardless of model size and learning mode, a small initialization scale (or a large weight decay coefficient) typically leads to good rule generalization, whereas a large initialization scale (or a small weight decay coefficient) fails to generalize perfectly.

For the SlimPajama dataset, we use two data compositions: the GitHub part and the GitHub+Wikipedia part. We train a GPT-2 Medium model on both datasets for 40B tokens with different initializations. As shown in Fig. 13.14D, for both data compositions, a smaller initialization scale consistently achieves lower perplexity.

Reasoning task: PrOntoQA

PrOntoQA is a synthetic multi-step reasoning dataset in which each data point specifies hierarchical relations among objects and requires the model to determine whether a multi-step reasoning

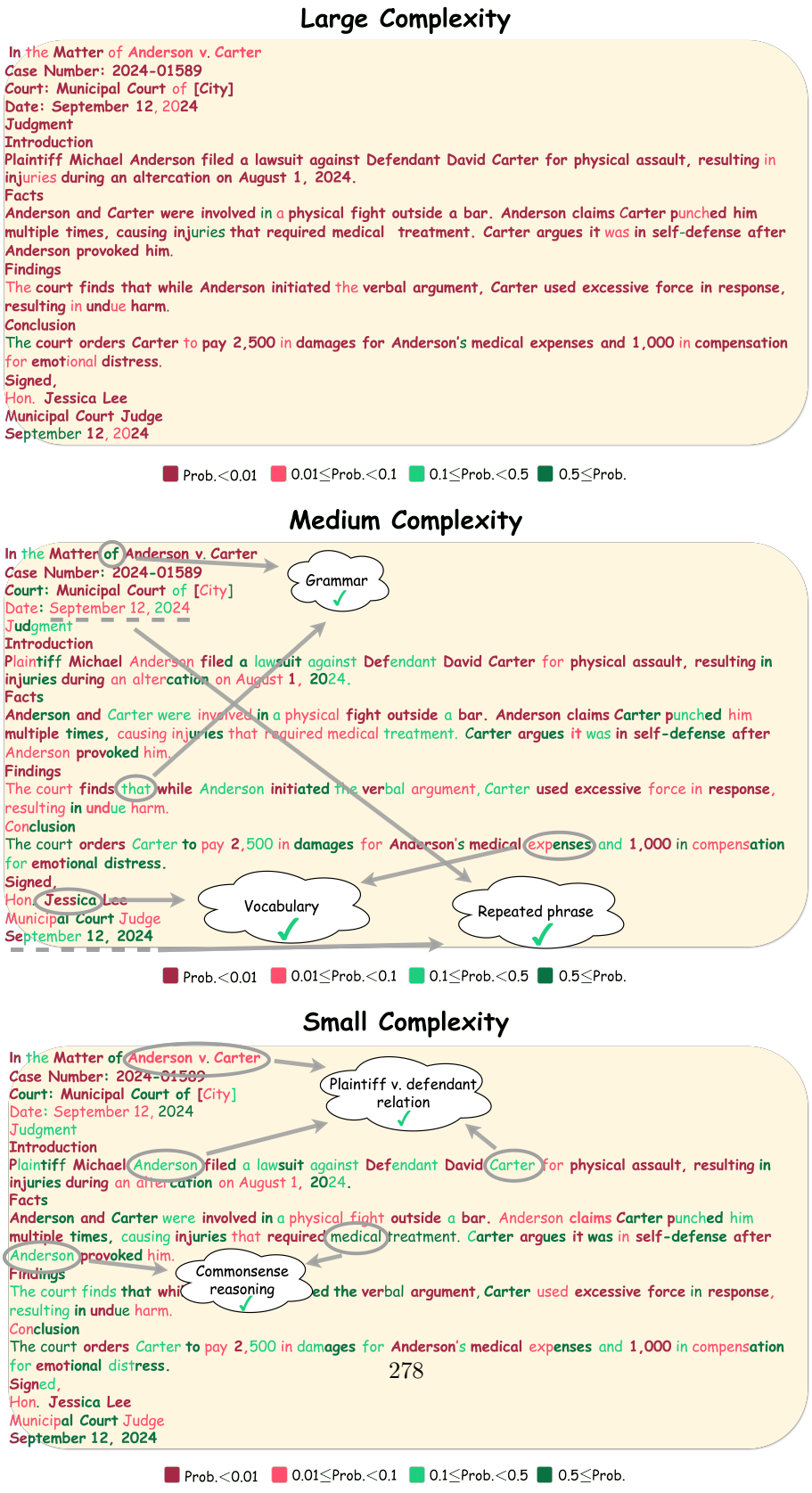


Figure 13.15: From top to bottom: the next-token prediction accuracy on different tokens of the same legal document, after training a Transformer model with initialization rates $\gamma = 0.1, 0.5, 1.0$, respectively. Red indicates inaccurate prediction, while green indicates accurate prediction. The experiments are from Hang et al. (2025).

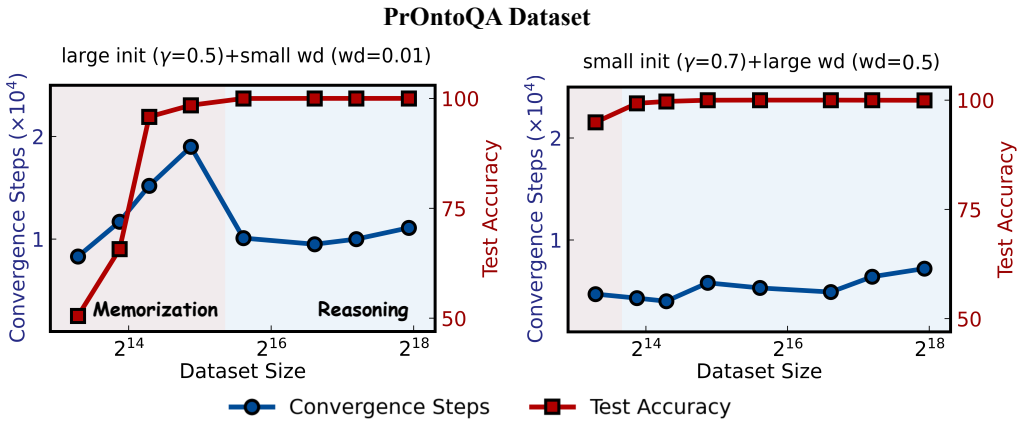


Figure 13.16: Performance comparison of models with different initialization scales and weight decay coefficients on PrOntoQA. Left: convergence steps and test accuracy for large initialization ($\gamma = 0.5$) and small weight decay (WD $= 0.01$). Right: convergence steps and test accuracy for small initialization ($\gamma = 0.7$) and large weight decay (WD $= 0.1$). Parameters are initialized following a zero-mean normal distribution with standard deviation $d_{in}^{-\gamma}$.

chain is correct. During testing, we only evaluate the accuracy of the model in judging the hierarchical relations. Therefore, the model's random-guess accuracy is 50%.

Fig. 13.16 illustrates the convergence speed and generalization error of models with large initialization (and small weight decay coefficient, top panel) and small initialization (and large weight decay coefficient, bottom panel) with respect to the data scale. For the model with large initialization (small weight decay coefficient), we observe an interesting phenomenon: as the data scale increases, the convergence speed first decreases and then increases. When the data scale is small, the model tends to fit the data by memorization. Therefore, as the data scale increases, the training difficulty grows (i.e., the training speed slows down) and the model's generalization ability deteriorates. As the data scale further grows, the model, limited by its complexity, can no longer memorize all the data, and thus shifts toward fitting the data via reasoning. This leads to faster fitting and better generalization. In contrast, the model with small initialization (large weight decay coefficient) inherently prefers to fit the data via reasoning, leading to faster convergence and better generalization than the model with large initialization (small weight decay coefficient) at the same data scale.

Pre-training task: Scaling Law

In large model training, the Scaling Law reveals the mathematical relations among model performance, data scale, and parameter scale, and has fundamentally changed the paradigm of large model development. We obtain different scaling laws to study how complexity control improves the performance of large language models. We use three different sets of complexity hyperparameters: (1) $\gamma = 1, \lambda = 1$; (2) $\gamma = 0.5, \lambda = 0.1$; and (3) $\sigma = 0.02, \lambda = 0.1$, where $\sigma = 0.02$ means that all parameters are initialized from a normal distribution with mean 0 and standard deviation 0.02. For each set of complexity hyperparameters, we analyze its generalization performance through the following experimental designs:

- **Data-scale experiments**: fix the number of model parameters at 800 million (800M), and

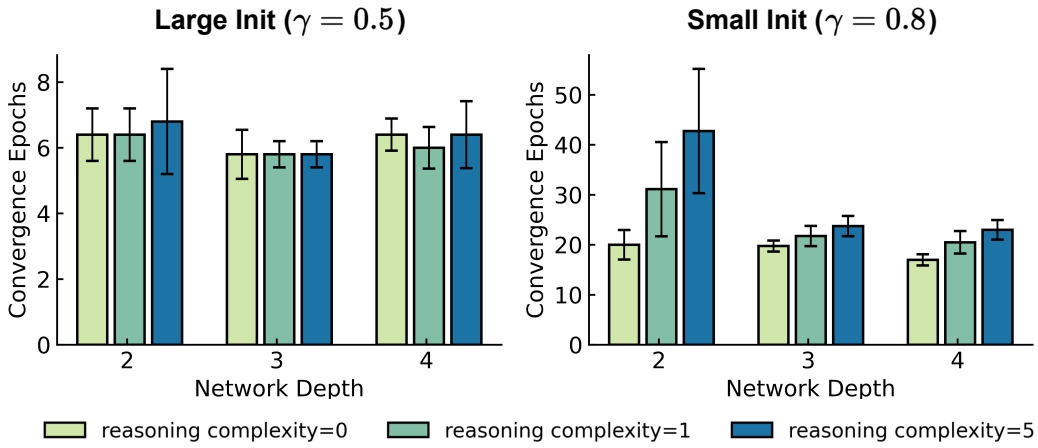


Figure 13.17: Training speed under different initialization scales when facing data with different complexities. Horizontal axis: model depth. Vertical axis: the number of epochs required for the training accuracy of the model to reach 100%. Color: reasoning complexity. For each setting of initialization, depth, and reasoning complexity of the data, we run 9 random trials and take the average of the required number of epochs. Left panel: large initialization (initialization rate $\gamma = 0.5$). Right panel: small initialization (initialization rate $\gamma = 0.8$).

vary the training data from 200 million to 1.2 billion tokens (200M–1.2B);

- **Parameter-scale experiments**: fix the training data at 1 billion tokens (1B), and vary the number of model parameters from 50 million to 800 million (50M–800M).

Fig. 13.18 shows the relations between the model test error and the data scale (left) and the model parameter scale (right) under different complexity hyperparameter settings. We can see that controlling the model complexity can effectively reduce the test error of large models. The intercept of the test error trajectory under low complexity is significantly smaller than that under high complexity, demonstrating that complexity control can effectively improve the generalization ability and reasoning performance of large models.

13.7 Predictions on the Mechanisms Behind Different Solutions

As data complexity increases, neural networks typically require more training steps to fit the data. Based on our analysis of the mechanisms behind different solutions, we can predict that models with different initialization scales will exhibit different levels of difficulty when learning data of different complexities. To create datasets with different complexity levels, we deliberately modify certain composite mappings while ensuring that anchor pairs with a symmetric relation remain consistent (i.e., $f(x; a_i, a_j) = f(x; a_j, a_i) \neq g(g(x; a_i); a_j)$). We define the **reasoning complexity** as the number of anchor-pair groups that do not satisfy the compositional rule (note: two anchor pairs with a symmetric relation are considered to belong to the same group). This reasoning complexity

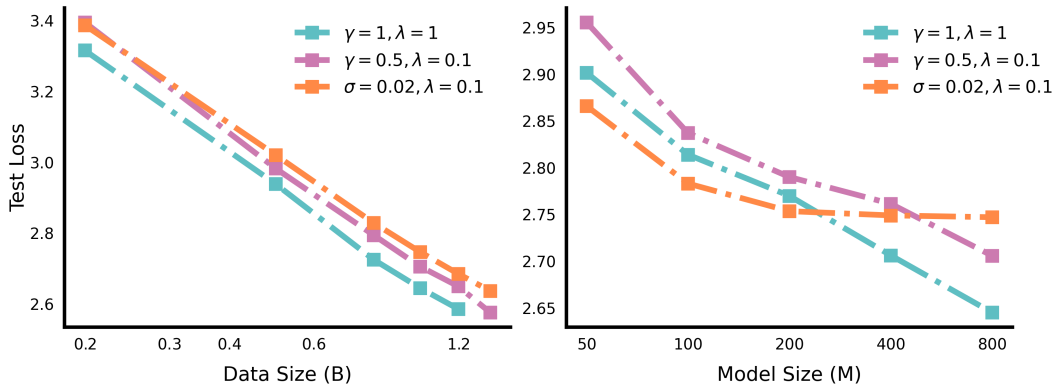


Figure 13.18: Scaling Law under different complexity hyperparameter settings. Left: with the model size fixed at 800 million, the relation between test error and data scale; right: with the data scale fixed at 1 billion tokens, the relation between test error and parameter scale.

captures the diversity of data in the dataset that violates the reasoning rules and requires memory-based mappings. A high reasoning complexity indicates the existence of more complex relations and patterns that cannot be easily captured by simple reasoning rules. Consequently, the model must rely more on memorizing specific data mappings rather than applying general reasoning principles.

It can be predicted that models with small initialization (i.e., those that prefer to fit data with as low complexity as possible) should require more training steps to fit data with high reasoning complexity, while models with large initialization (i.e., those that easily memorize symmetric-solution mappings) should require similar training steps to fit data with different reasoning complexities.

To measure the number of training steps required to fit the data, we use the number of epochs required for the training accuracy of the model to reach 100%. We conduct 9 random trials for each setting.

As shown in Fig. 13.17, models that tend to learn data via memory-based solutions use roughly the same number of training steps for data with different reasoning complexities (left panel of Fig. 13.17), while models that tend to learn data via reasoning-based solutions indeed use more training steps for data with higher reasoning complexity (right panel of Fig. 13.17). This prediction strongly supports our analysis of why different initialization scales lead to different solutions.

We further show the fitting speed of different anchor pairs when the reasoning complexity is 1. In this experiment, we simultaneously set the anchor pairs (a, b) and (b, a) to the same mapping that violates the compositional rule. As shown in Fig. 13.19, we present the number of epochs required for the 16 kinds of anchor pairs to reach 60% accuracy on their corresponding training data. We highlight in red the four kinds of anchor pairs (a, b), (b, a), (a, a), and (b, b) (since (a, b) and (b, a) violate the compositional rule, the above four kinds of anchor pairs are significantly affected). It can be observed that the highlighted anchor pair types exhibit significantly slower training speed, further validating that the presence of mappings that violate the compositional rule slows down the training of models with small initialization.

From another perspective, this also allows us to detect mappings in a dataset that violate the compositional rule (typically noisy data). For a set of data with an unknown distribution, we can distinguish among different types of data to some extent by studying the convergence speed of different types of data using a small-initialization model.

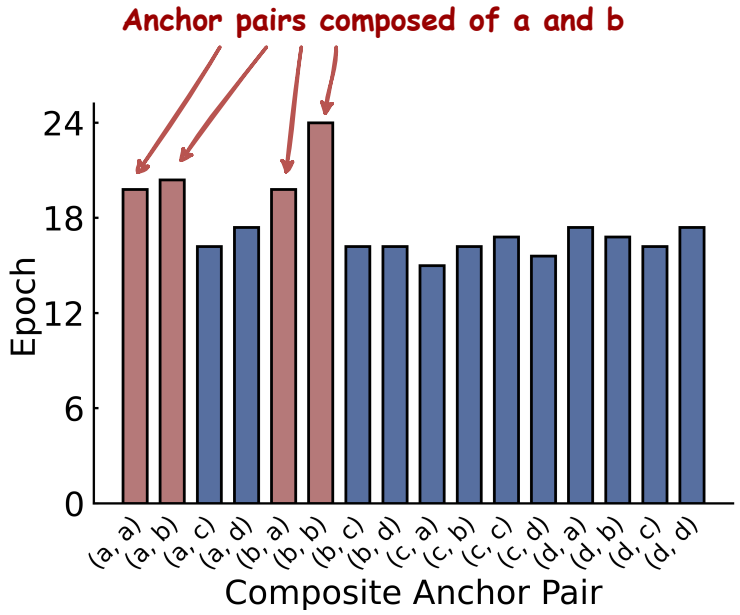


Figure 13.19: The number of epochs required for different anchor pairs to reach 60% accuracy on their corresponding training data. The anchor pairs (a, b), (b, a), (a, a) and (b, b) are highlighted in red, as they are significantly affected by the non-reasoning solutions (a, b) and (b, a).

Chapter 14

More Phenomena in Deep Neural Networks

In the preceding chapters of this book, we have focused on phenomena such as the Frequency Principle (F-Principle, also known as spectral bias) and condensation. In this chapter we will guide you through another series of fascinating phenomena in deep learning. These phenomena not only showcase the complexity and potential of deep learning, but also challenge our traditional understanding of learning, intelligence, and optimization.

Popular Introduction

Phenomenon-driven research is becoming an increasingly important paradigm in deep learning. The underlying reason is that deep neural networks, as highly nonlinear and strongly correlated high-dimensional complex systems, are very difficult to analyze purely by deduction, that is, by starting from their mathematical formulations and dynamical equations and deriving macroscopic regularities from them. The history of science tells us that, when faced with such complex systems, systematic experimental observation and inductive summarization of phenomena are often the first and most crucial step in building understanding. From a history-of-science perspective, current theoretical research on deep learning is still in a "pre-Newtonian era" (Fei-Fei Li, Stanford). At this stage, systematic experimental study and phenomenological summary are indispensable. Such work will lay the foundation for our entry into the "Newtonian era".

To give an intuitive picture, we can carry out a thought experiment. Suppose that before the birth of thermodynamics we had already known the microscopic dynamical equations governing an ideal gas, namely that a large number of molecules move randomly in air, obeying Newton's laws of motion and colliding elastically. When considering the motion of molecules at the scale of Avogadro's number ($\sim 6.02 \times 10^{23}$) inside a container, how could we ever deduce the ideal gas law $PV = nRT$ from $O(10^{23})$ differential equations? How could we deduce the principle of entropy increase from such deterministic equations? It is hard to imagine that such mathematical deduction could happen without being guided by experimental phenomena like the following: temperature, pressure, and volume are important macroscopic quantities and there is a clear dependence among them; although the dynamical equations are reversible, in experiments molecules confined to half

of the space rapidly fill the entire space, while molecules filling the entire space almost never spontaneously gather back into half of it.

Scaling law The performance of large language models grows as a power law of the model size, the data size, and the compute budget.

Densing law The density of large language models grows as a power law of the release time.

In-context learning in large language models In-context learning is an important property of large language models, allowing the model to rapidly adapt to and execute new tasks given only a task description and a small number of demonstrations, without any additional adjustment of the model weights.

Chain-of-thought in large language models Chain-of-thought refers to a method that guides the model to produce intermediate reasoning steps instead of directly outputting the answer; this approach can significantly improve the model's performance on complex tasks.

In the preceding chapters of this book, we have focused on phenomena such as the Frequency Principle and condensation. In this chapter we will guide you through another series of fascinating phenomena in deep learning. These phenomena not only showcase the complexity and potential of deep learning, but also challenge our traditional understanding of learning, intelligence, and optimization.

Grokking Grokking refers to a phenomenon in which a neural network, during a long training process, first reaches a state of very small training error, and then after some time abruptly undergoes a grokking phase. In this phase, the model's performance rapidly jumps from an overfitting state to one with good generalization.

The lottery ticket phenomenon The lottery ticket phenomenon states that a large neural network contains sparse subnetworks which already possess a good structure at initialization and can perform well after training.

Double Descent The double descent phenomenon describes how, as model complexity or training data size increases, the test error may exhibit two successive decreasing trends.

Neural collapse Neural collapse refers to a special state arising in the late stage of training of deep neural networks, in which the feature representations of samples from the same class cluster together while features from different classes form an equiangular configuration.

Mode connectivity Mode connectivity research finds that different local optima in the optimization landscape of neural networks are typically connected with each other.

14.1 Scaling law

The scaling law for large models describes a series of empirical regularities for how model performance, training loss, and reasoning ability change as the model size, the amount of data, and the available compute grow. It was first proposed by OpenAI in Kaplan et al. (2020), and scaling laws have since provided important guidance for the design and training of large models.

Specifically, scaling laws identify three main variables that affect model performance:

Model size The model size N refers to the number of non-embedding parameters of the neural network, and is typically related to the model's depth and width. Studies show that as the number of parameters increases, the model's loss function follows a power law:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N},$$

where N_c is a scaling-law constant.

This power-law relation implies that increasing the model size improves the model's performance: given a sufficiently large training set, every doubling of the parameter count decreases the test loss by a factor of 2^{α_N}.

However, the performance gain from increasing the model size suffers from diminishing returns. As the model becomes larger, each additional increment of parameters produces a smaller improvement in test loss. This observation indicates that, although increasing the model size is an effective way to improve performance, it may not be the most efficient, especially when compute resources are limited.

Dataset size The amount of data used to train the model also has a significant impact on the final performance. For a fixed model size, the larger the data set, the more the loss function follows a power law:

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D},$$

The scaling law indicates that when the data is insufficient, even increasing the model parameters yields only limited improvement.

This power-law relation implies that enlarging the training dataset improves performance: every doubling of the dataset size decreases the test loss by a factor of 2^{α_D}.

However, similarly to the model size, the performance gain from increasing the dataset size is subject to diminishing returns. As the dataset grows, every doubling of the number of tokens yields a smaller improvement in model performance.

Compute budget Beyond model size and dataset size, compute is another key factor that determines the performance of a language model. Measuring compute in petaflop/s-days (PF-days), with a constrained compute budget, training a dataset large enough on the optimally sized model and using a sufficiently small batch size (so as to use the compute most efficiently), we obtain the relation between test loss and compute:

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}.$$

This power-law relation implies that increasing the compute budget yields a continuous improvement in performance: every doubling of the budget decreases the test loss by a factor of 2^{α_C}.

Likewise, the performance gain from increasing the compute budget is subject to diminishing returns, meaning that even though raising the compute budget improves the model's performance, it is likely not the optimal approach when resources are limited.

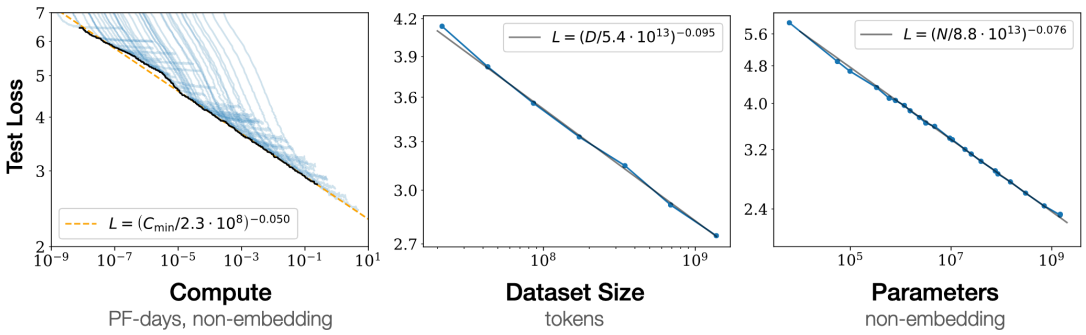


Figure 14.1: Left: scaling law with respect to compute; middle: scaling law with respect to dataset size; right: scaling law with respect to the number of parameters. Taken from Figure 1 of Kaplan et al. (2020).

14.2 Densing law for large models (densing law)

The scaling law characterizes how the performance of models with the same architecture depends on the number of parameters, the amount of data, and the compute budget, but how can we compare the training quality of *different* models? Xiao et al. (2024) proposed the densing law for large models in order to compare the training quality of different models. In that paper, the density of a model M with performance S_M is defined as

$$\rho(M) = \frac{\hat{N}(S_M)}{N_M},$$

where N_M is the parameter count of the model and $\hat{N}(S_M)$ is the effective parameter count of the model, i.e. the minimum number of parameters that a reference model needs in order to reach the same performance S_M as model M.

How to compute the effective parameter count $\hat{N}(S_M)$? In Xiao et al. (2024), the authors first take the conditional loss $\mathcal{L}$ on the answer tokens as the training objective and fix the data size $D_0 = 1\text{T}$, training to obtain the power-law relation of $\mathcal{L}$ with respect to the parameter count N of the reference model and the data size D_0:

$$\mathcal{L} = aN^{-\alpha} + bD_0^{-\beta}. \tag{14.1}$$

Then they compute, for several open-source models, the performance S on downstream tasks together with the loss $\mathcal{L}$ of the model on the test set of the task, and fit the relationship by a sigmoid function:

$$S = \frac{c}{1 + e^{-\gamma(\mathcal{L}-l)}} + d. \tag{14.2}$$

This finally gives the relation between the parameter count of the reference model and the model performance. Given a model performance S_M, one inverts Eq. 14.2 to obtain L, and then Eq. 14.1 yields the effective parameter count:

$$\hat{N}(S_M) = \left(\frac{\hat{\mathcal{L}}(S_M) - bD_0^{-\beta}}{a} \right)^{-\frac{1}{\alpha}},$$

where $\hat{\mathcal{L}}(S_M) = l - \frac{1}{\gamma} \ln\left(\frac{c}{S_M - d} - 1\right)$ is obtained by inverting the sigmoid.

Once the effective parameter count of a model can be computed, its density can be obtained directly from the definition. Fitting the maximum density of models in different periods against the model release time, Xiao et al. (2024) obtains the densing law for large models shown in Fig. 14.2:

$$\ln(\rho_{\max}) = A \cdot t + B,$$

where $\rho_{\max}$ is the maximum density of all large models at time t, and t is the number of days elapsed since the release of Llama-1.

At the same time, Xiao et al. (2024) fits $A \approx 0.007$, indicating that the density of large models roughly doubles every three months. In other words, it takes only about three months for a new model to achieve the same level of performance as an old one using half as many parameters. This law deeply reveals the astonishing speed at which large-model technology is advancing.

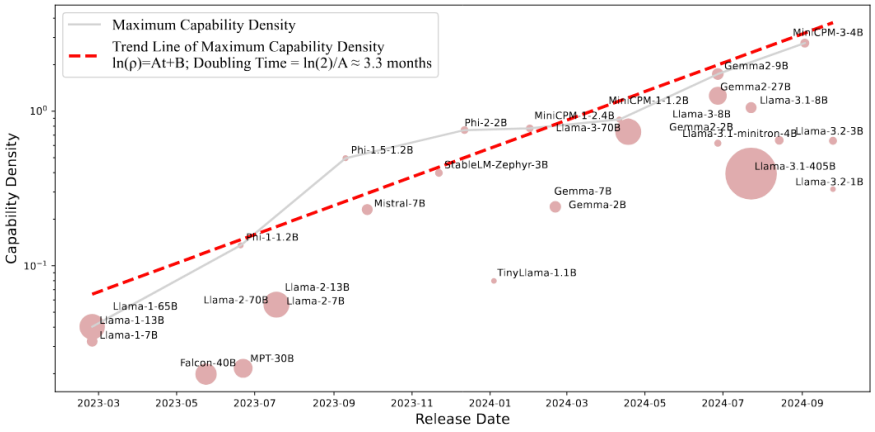


Figure 14.2: The densing law for large models. Taken from Figure 1 of Xiao et al. (2024).

Xiao et al. (2024) also gives several corollaries:

1. Exponential decrease of inference cost: keeping the performance fixed, the compute required for inference halves every 3–4 months.

2. Moore's law × densing law: on a chip at the same price level, the number of "effective parameters" that can be run doubles every 88 days.

3. Acceleration after ChatGPT: after the release of ChatGPT, the slope of density growth has increased from 0.0048 to 0.0073.

4. Compression ≠ density improvement: common pruning/distillation often *decreases* the density.

5. A shift toward "density-optimal training": future training objectives should shift from "maximum performance" to "maximum density", so as to realize a green and sustainable Scaling Law.

14.3 In-context learning in large language models

In 2020, OpenAI discovered through GPT-3, a model with 175 billion parameters, that large language models exhibit an astonishing ability called **in-context learning** (Brown et al., 2020). In-context learning (ICL) refers to the ability of a language model to learn to perform a task given only a few examples, without any additional training or tuning of the model. In this section, we will explain in-context learning intuitively through some examples, and discuss several research findings demonstrating its strong performance.

Let us first illustrate in-context learning with an example. Suppose we have a chatbot and we want it to learn to perform sentiment analysis on user reviews, i.e. to decide whether a review is positive or negative. The traditional approach is to collect a large amount of review data and let the bot train on these data. With in-context learning, however, we only need to give the bot a few examples, and it can learn to do sentiment analysis.

Concretely, we can provide the bot with a few demonstration examples, each consisting of a review and its corresponding sentiment (positive or negative), as shown in Fig. 14.3. By reading these examples, the bot can roughly learn how to judge the sentiment polarity of a review. When we then give the bot a new review, it can predict, based on the examples it has seen, whether the sentiment of this new review is positive or negative.

Beyond sentiment analysis, in-context learning can be applied to many other tasks. For instance, in image segmentation, as illustrated in Fig. 14.4, given a natural-language task description and a single example, one provides an image to be processed, and the model is required to output its segmentation result.

In OpenAI's 2020 paper (Brown et al., 2020), through extensive experiments, the authors found that in-context learning achieves impressive results on a wide variety of natural-language processing tasks. For example, the 175-billion-parameter GPT-3 model achieved performance close to the state-of-the-art on the SuperGLUE benchmark with only 32 examples. On the COPA and ReCoRD datasets, the in-context learning performance of GPT-3 also reached a level comparable to that of the best fine-tuned models at the time.

These examples show that in-context learning allows a language model to learn a new task without large amounts of training data, using only a small number of demonstrations. This way of learning is very similar to how humans learn—we, too, often learn a new skill from a few examples, without needing to see large amounts of data. Why this in-context learning phenomenon occurs is still an open question.

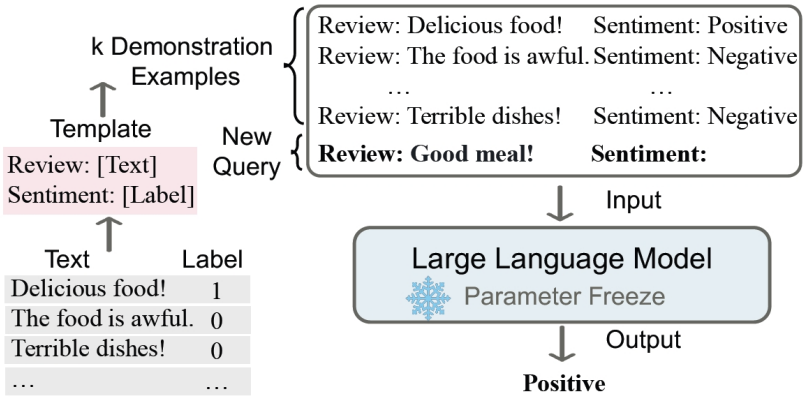


Figure 14.3: Illustration of in-context learning on a sentiment analysis task. ICL requires a demonstration context containing several examples written in a natural-language template. With the demonstration and a query as input, a large language model is responsible for making the prediction. Taken from Dong et al. (2022).

14.4 Chain-of-thought in large language models

In large language models, chain-of-thought (CoT) is a new prompting method (Wei et al., 2022) that significantly improves the performance of language models on complex reasoning tasks by having the model generate a sequence of intermediate reasoning steps. Specifically, CoT prompting works by inserting chain-of-thought demonstrations into a few-shot prompt, thereby inducing the language model to produce a similar reasoning process. Fig. 14.5 shows an example of a chain-of-thought prompt.

Experiments show that on arithmetic, commonsense, and symbolic reasoning tasks, chain-of-thought prompting can substantially improve the performance of language models. For example, on the GSM8K math word problem dataset, the PaLM 540B model prompted with only 8 chain-of-thought exemplars surpassed the accuracy of fine-tuned GPT-3 and set a new state of the art on this task.

More intriguingly, chain-of-thought reasoning is a new ability that emerges as the model scales up. CoT prompting actually hurts the performance of small models, and only on sufficiently large models (e.g. tens of billions of parameters or more) does it bring performance gains. This indicates that chain-of-thought reasoning is a unique capability of large language models.

Chain-of-thought in large language models showcases the great potential of prompting methods. Through carefully designed prompts, we can induce a language model to display step-by-step reasoning abilities similar to those of humans. This paves the way for developing more intelligent and interpretable applications of language models.

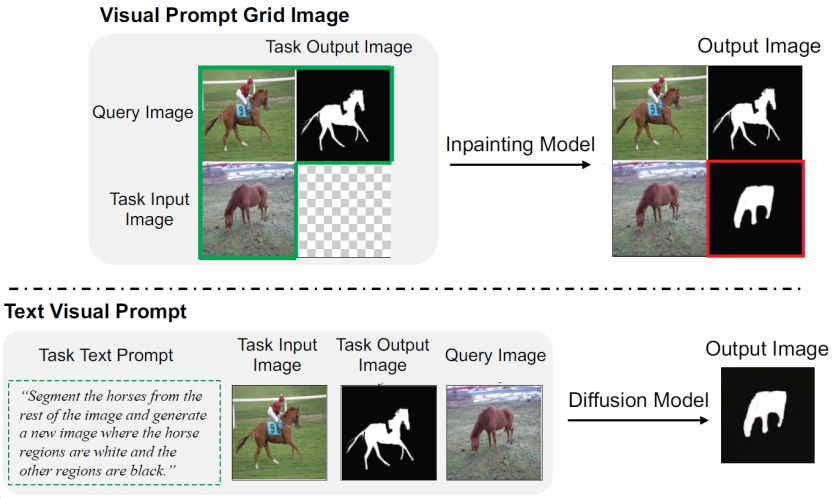


Figure 14.4: Pure-image and text-augmented prompts in visual in-context learning. Taken from Dong et al. (2022).

14.5 The grokking phenomenon

In 2022, Power et al. in Power et al. (2022) discovered that when training a decoder-only Transformer on algorithmic datasets, a striking grokking phenomenon appears, as shown in Fig. 14.6: during the training process the training error first drops to a very small value, and only after a long time does the test error start to decrease. Subsequently, Barak et al. (2022) and Bhattamishra et al. (2022) observed the grokking phenomenon in the sparse parity function task; Liu et al. (2022) observed it on the MNIST, IMDb, and QM9 datasets.

14.6 The lottery ticket phenomenon

In 2019, Frankle and Carbin (2018) proposed the Lottery Ticket Hypothesis: a randomly initialized dense neural network contains a subnetwork—obtained by masking some of the parameters of the original network so that they are not used—such that, when this subnetwork is trained in isolation with the same initialization for the same number of iterations, it can match the test accuracy of the original network.

Formally, for a randomly initialized feedforward network $f(x;\theta)$ with initial parameters $\theta = \theta_0 \sim D_\theta$, when training with stochastic gradient descent (SGD), suppose f reaches a minimum validation loss l at iteration j, and its test accuracy at this point is a. Now apply a binary mask $m \in \{0,1\}^{|\theta|}$ to the network parameters, where each entry of m is 1 with probability p and 0 with probability $1-p$, giving $f(x; m \odot \theta)$ ($\odot$ denotes elementwise multiplication), initialized with the same θ_0. If we train this subnetwork on the same training set with SGD (with m fixed), then there exists a mask m such that f reaches a minimum validation loss $l' \leq l$ at some iteration $j' \leq j$, with test accuracy $a' \geq a$, while the number of 1s in m is much smaller than $|\theta|$.

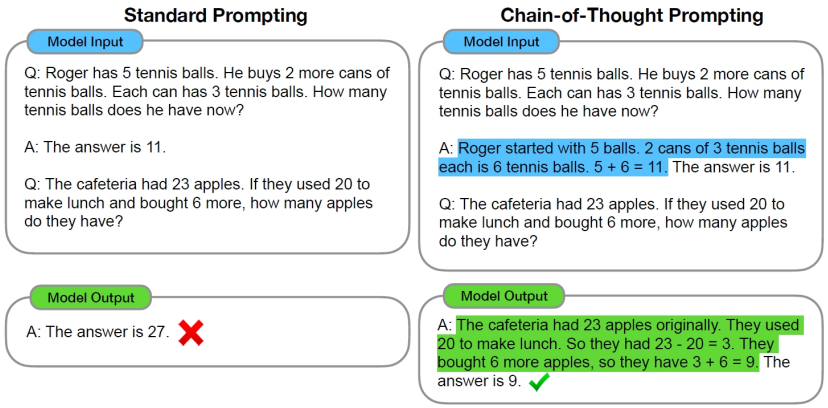


Figure 14.5: An example of chain-of-thought prompting. The figure shows the standard prompt (left) and the chain-of-thought prompt (right) on a math word problem. Taken from Wei et al. (2022).

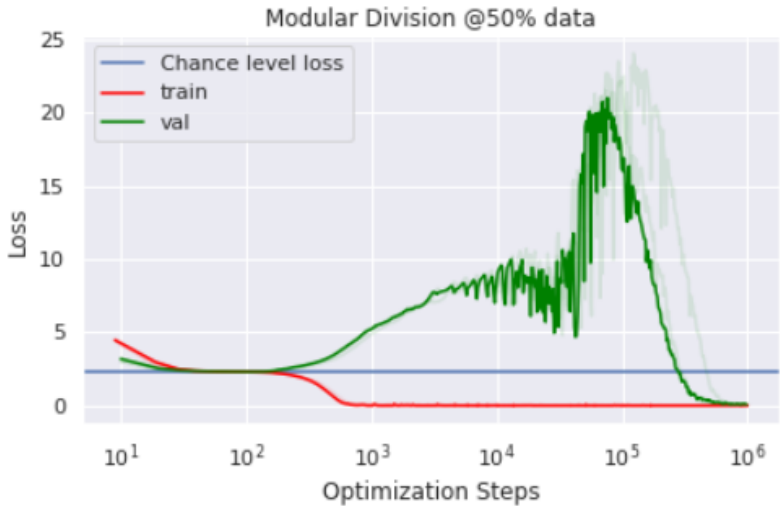


Figure 14.6: The grokking phenomenon on the modular division algorithmic dataset. Taken from Power et al. (2022).

Experiments show that a method called iterative pruning can effectively find such subnetworks (each curve in Fig. 14.7 is the average over 5 trials; different colors in the legend indicate different fractions of weights remaining in the pruned network; the error bars indicate the minimum and maximum across any single trial). Concretely, the method first trains the whole network, then removes a fraction of the weights with the smallest magnitudes, resets the remaining weights to their initial values, and repeats this process for several rounds. If too many weights are pruned, performance drops significantly. Moreover, if the remaining weights are randomly re-initialized, the performance also drops drastically. This indicates that the initialization is crucial for the winning ticket.

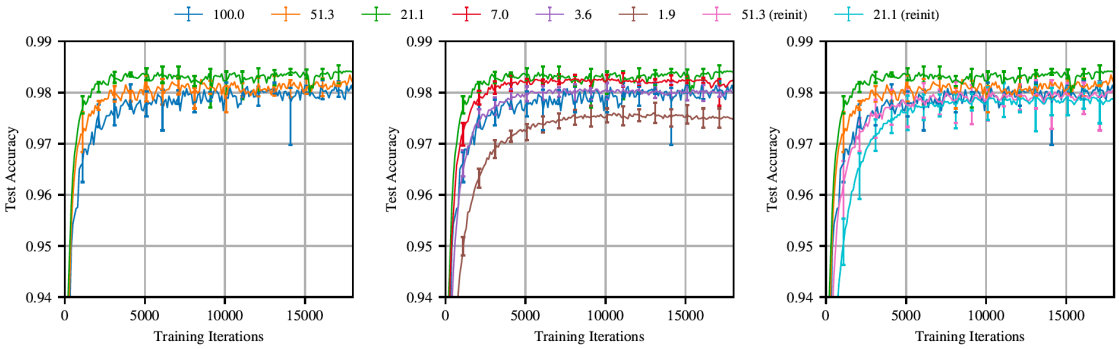


Figure 14.7: Test accuracy over training for LeNet with iterative pruning. (Figure from Figure 3 of Frankle and Carbin (2018).)

The lottery ticket phenomenon indicates that, even though deep learning models have a huge number of parameters, a small subset may already play a decisive role in the network's performance. The lottery ticket phenomenon also raises several interesting questions: Why do neural networks tend to contain winning tickets? How are they formed during random initialization and optimization? Beyond pruning, are there more efficient methods for discovering winning tickets? These questions await further theoretical and empirical investigation.

14.7 The double descent phenomenon in neural networks

In a 2019 study Belkin et al. (2019), Belkin et al. experimented with two-layer neural networks of various sizes on the MNIST and CIFAR datasets and observed the double descent phenomenon, namely that as the model grows the test error first decreases, then increases, and then decreases again. Nakkiran et al. Nakkiran et al. (2021) used the deeper convolutional network ResNet-18, and after adding label noise the double descent phenomenon became very pronounced, as shown in Fig. 14.8. Nakkiran et al. (2021) also found that double descent can be observed with respect to the number of optimization steps. In their experiments, without added label noise the double descent phenomenon is almost invisible.

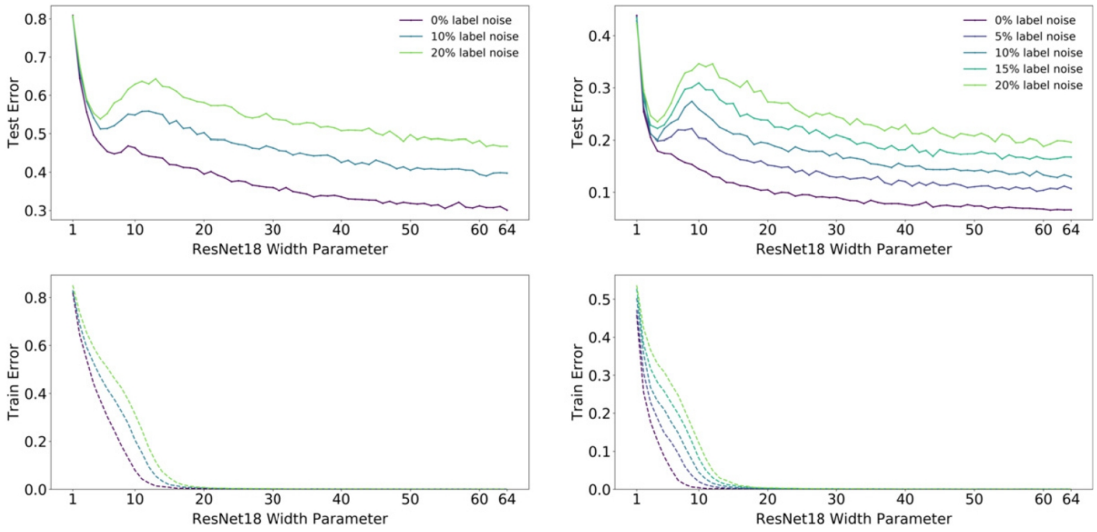


Figure 14.8: Double descent of ResNet-18 on CIFAR-10 and CIFAR-100 (with label noise). Taken from Nakkiran et al. (2021).

14.8 The neural collapse phenomenon

In 2020, Papyan et al. (2020) discovered a phenomenon called "neural collapse". Their paper characterizes this phenomenon in four ways. In this section we introduce it through two of these.

One feature of neural collapse is that, as training proceeds, the variability of samples within the same class in feature space gradually decreases and eventually collapses to the class center. Formally, we can characterize this phenomenon using the within-class covariance matrix Σ_W of the last-layer features:

$$\Sigma_W = \text{Ave}_{i,c}\left[(h_{i,c} - \mu_c)(h_{i,c} - \mu_c)^T\right] \to 0 \tag{14.3}$$

where $h_{i,c}$ denotes the last-layer feature of the i-th sample of class c, and μ_c denotes the mean feature of class c, i.e. the class center. The convergence of Σ_W to 0 means that the features of all within-class samples collapse to the class center.

An intuitive explanation of this phenomenon is that, as training proceeds, the neural network learns to extract the most discriminative features. These features can distinguish different classes very well, while ignoring subtle within-class differences. Hence the samples of the same class become more and more "compact" in feature space and eventually collapse to a single point.

Another feature of neural collapse is that the class centers of the last-layer features converge to points whose mutual angles are maximized (an Equiangular Tight Frame, ETF). For example, the four vertices of a regular tetrahedron form four vectors emanating from the origin in three-dimensional space whose pairwise angles are maximized.

14.9 The mode connectivity phenomenon

In deep learning, if we train the same network architecture on the same dataset multiple times with different random initializations, we obtain multiple distinct weight vectors $w_1, w_2, ..., w_n$, all of which are local minima of the loss function. The mode connectivity discovered by Garipov et al. (2018) states that there exists a continuous path from w_1 to w_2 such that, at every point w on the path, the corresponding model achieves a high test accuracy that is close to the accuracies of the models at w_1 and w_2.

Fig. 14.9 shows an example of mode connectivity. The figure plots contour lines of the cross-entropy loss of ResNet-164 on the CIFAR-100 dataset. The horizontal and vertical axes correspond to two different weight vectors w_1 and w_2, both obtained from independent training runs.

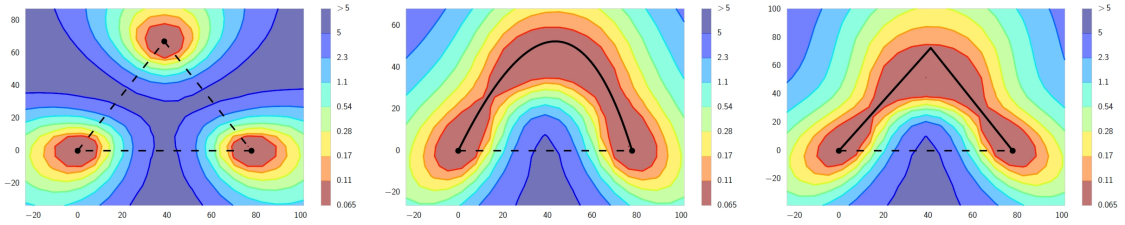


Figure 14.9: Mode connectivity of ResNet-164 on CIFAR-100. Taken from Garipov et al. (2018).

The middle and right panels in the figure show that near some local minima, namely between w_1 and w_2, there exists a curved path along which the value of the loss function remains essentially constant (represented by the color). This indicates that along this path from w_1 to w_2, the accuracy of the model stays essentially unchanged. By contrast, the straight-line path between w_1 and w_2 (not drawn in the figure) has substantially higher loss values. Similar mode connectivity phenomena are widely observed in other network architectures and datasets.

Mode connectivity is an interesting phenomenon in deep learning. It indicates that, although the weight space of a neural network is extremely complex and full of a huge number of local minima, these local minima are not isolated: they can be connected by simple curves along which the corresponding models have almost identical generalization performance.

Chapter 15

Solving Differential Equations with Neural Networks

Neural network technology has not only achieved enormous success in natural language processing and image recognition, but its applications to scientific problems are equally remarkable. In many scientific domains, researchers use neural networks to simulate and predict the behavior of complex systems. For example, in molecular dynamics simulations, neural networks are used to establish the mapping relation between molecular configurations and potential energy functions. By learning from large amounts of molecular structure data, neural networks can accurately capture this complex nonlinear relation, thereby accelerating molecular dynamics simulations and providing powerful tools for studying the properties and evolution of molecular systems. Another canonical example is AlphaFold, a deep-learning-based protein structure prediction system. Knowing the three-dimensional structure of a protein is crucial for understanding its biological function, but experimental determination of protein structures is typically time- and labor-consuming. By training a neural network to learn the relation between protein sequence and structure, AlphaFold successfully predicts the three-dimensional structure of a large number of proteins, bringing a revolutionary breakthrough to the life sciences.

Behind these successful cases lies the powerful learning capability of neural networks. In many complex scientific tasks, the underlying mappings often exhibit highly nonlinear and complex high-dimensional structures. Faced with such situations, neural networks can automatically learn and capture these complex mappings from large amounts of data, making them powerful tools for solving such problems. In particular, in the application of artificial intelligence to science (AI for Science), researchers widely use neural networks to model and parameterize complex scientific problems, which has become a major hotspot of current research. By training neural networks to learn the intrinsic laws of complex systems, scientists can more efficiently and accurately simulate and predict various physical, chemical, and biological processes, accelerating the pace of scientific discovery.

In this chapter, we start from solving differential equations with deep learning and introduce a series of applications and challenges of neural networks in scientific problems, demonstrating the potential and advantages of deep learning methods.

Differential equations are a fundamental tool for describing phenomena and solving problems across mathematics, physics, engineering, and many other disciplines. They are used to express how variables change over time or space, and serve as the core mathematical language for under-

standing and predicting the behavior of systems across many scientific and technological fields. From Newton's laws of motion to Maxwell's equations of electromagnetism, differential equations provide us with a powerful framework to describe and analyze the dynamic behavior of all kinds of natural and artificial systems. By building differential equation models, researchers can gain deep insight into the intrinsic mechanisms of a system, predict its future evolution, and explore how various factors influence the system's behavior. According to the type of derivatives, differential equations can be divided into two main categories: ordinary differential equations (ODEs) and partial differential equations (PDEs). Ordinary differential equations primarily handle the relation between a system's dependent variables and a single independent variable (usually time). ODEs have wide applications in describing lumped-parameter systems, population dynamics, chemical reaction kinetics, and more. In contrast, partial differential equations consider derivatives with respect to multiple independent variables (such as time and space), which enables them to describe the dynamic behavior of more complex distributed-parameter systems. PDEs play a key role in fluid dynamics, heat conduction, electromagnetism, quantum mechanics, and other areas. Through PDEs, we can study complex phenomena such as wave propagation, the onset of turbulence, and field distributions.

Solving differential equations is an important and challenging problem. For some simple differential equations, we can obtain exact solutions by analytical methods. However, for most practical problems, differential equations are often very complex and analytical solutions are hard to obtain. In such cases, numerical methods become an important means of solving differential equations. Traditional numerical algorithms, such as the Euler method, finite element method, and finite volume method, discretize time and space to convert differential equations into algebraic equation systems, which are then solved using computers. These methods have made remarkable progress over the past several decades and have driven the rapid development of computational mathematics.

Popular Science Section

Why use neural networks to solve differential equations?

Although traditional numerical algorithms have achieved remarkable success in solving differential equations, they still face many challenges. First, for highly nonlinear, multi-scale differential equations, traditional methods typically require very fine grid partitions and extremely small time steps, leading to high computational cost. Second, when handling problems with complex boundaries, the mesh generation process consumes substantial computational resources. In addition, for inverse problems, traditional algorithms require repeatedly solving the forward problem, which is computationally inefficient. Meanwhile, solving high-dimensional PDEs often suffers from the curse of dimensionality, causing computational complexity to grow exponentially. These difficulties have made certain complex differential-equation problems difficult to solve effectively for a long time. By contrast, neural-network-based methods leverage their powerful function-fitting ability: by training a neural network to approximate the solution of a differential equation, they do not require manually designed complex meshes or discretization schemes, and they can automatically learn the intrinsic structure and regularities of the problem, providing new possibilities for tackling these difficulties.

Methods for solving differential equations with neural networks

- **Parameterized-solution methods** Parameterized-solution methods mainly provide customized solutions for specific differential equations. Such methods build a parameterized neural network to represent the solution of the differential equation, and optimize the network parameters so that its output satisfies the conditions of the differential equation as closely as possible.
- **Parameterized-operator methods** Unlike parameterized-solution methods, parameterized-operator methods use a neural network to learn the entire operator mapping. Such methods have stronger generality and aim to parameterize, with a neural network, the operators of an entire family of differential equations. They are not restricted to solving a single differential equation but instead try to learn the generalized operator of differential equations, thus providing a unified framework for solving a large class of problems.

15.1 Example: Newton's Laws of Motion

In research at the intersection of artificial intelligence and science, the concept of a surrogate model is particularly important. First, we need to clearly identify the problem we wish to solve and convert it into an optimization problem. By solving this optimization problem, we can directly obtain the desired solution. For example, in physics, energy minimization is a very common and naturally occurring optimization problem, which makes our work relatively straightforward.

Take Newton's motion problem as an example. Suppose an object of mass m is subject, in a one-dimensional pipe, to a time-varying force $F = \sin(t) + t$. What is the relation between its position and time? Since the form of the force is rather complex, it is difficult to obtain a formula directly by theoretical means. Therefore, we assume that the position $s(t)$ approximately satisfies a polynomial form: $s(t) = a * t + b * t^2 + c * t^3 + d * t^4 + e$. Here a, b, c, d, e are parameters to be determined. How should we determine these parameters? According to Newton's second law:

$$m\frac{d^2s(t)}{dt^2} = F = \sin(t) + t, \tag{15.1}$$

assuming initial conditions $s(0) = 0$ and $\frac{ds(t)}{dt}|_{t=0} = 0$, we can deduce $a = 0, e = 0$. In fact, this is a second-order ordinary differential equation problem. Clearly, since the form of the force is rather complex, this polynomial form cannot exactly satisfy the above differential equation. So we settle for second best: find a set of parameters that comes closest. We want the error of the above equation to be as small as possible at each time point t. Taking the squared error as an example, we want $|m\frac{d^2s(t)}{dt^2} - (\sin(t) + t)|^2$ to be small. Suppose we only care about $t \in [0, T]$. Clearly, we cannot take all possible time points; a common approach is to randomly sample n points over this time interval and compute the error

$$L(b, c, d) = \sum_{i=1}^{n} |m\frac{d^2s(t_i)}{dt^2} - (\sin(t_i) + t_i)|^2. \tag{15.2}$$

In effect, this is a three-dimensional function in b, c, d, and we want to find a set of b, c, d that minimizes its value. Algorithms such as gradient descent can do this.

For problems where we cannot directly find a minimization form, we can look for equality forms related to the problem, such as evolution equations or energy conservation laws. By subtracting

the right-hand side from the left-hand side and squaring the result, we can convert it into a least-squares problem. This approach not only simplifies the complexity of the problem but also provides an effective route to a solution. For the problem above, in the case of initial values, we can also substitute $t = 0$ to obtain equations for the initial position and initial velocity, and then form two squared-error objectives, namely $|s(0) - 0|^2$ and $|\frac{ds(t)}{dt}|_{t=0} - 0|^2$, and sum the three objectives together for joint optimization. This way we do not need to determine a and e in advance, which is much more convenient for very complex situations.

When dealing with problems involving continuous integrals, we can convert them into approximate summation forms through discretization, which makes the computation much more tractable and efficient. Within this framework, the polynomial fitting mentioned above leads to many problems, especially for high-dimensional ones, and the role of neural networks becomes particularly crucial. Neural networks can be used to replace certain parts of the minimization problem; specifically, a neural network can directly serve as the function to be solved, or as a part of that function, helping us better capture complex nonlinear relations.

These neural-network-based methods provide a new perspective and toolkit for solving complex differential equations. They not only demonstrate the power of neural networks in capturing high-dimensional, nonlinear mappings, but also bring new solutions to the field of computational mathematics. For a more detailed introduction we refer the reader to Professor Weinan E's review article (E et al., 2021). In the following sections, we will introduce some basic methods and, starting from the phenomena, give a brief discussion of them.

Methods for solving differential equations based on neural networks can be roughly divided into two main categories. The first class of methods parameterizes the solution with a neural network, providing customized solutions for specific differential equations. Such methods build a parameterized neural network to represent the solution and optimize the network parameters so that the output satisfies the differential equation as closely as possible. These solution methods began to develop gradually in the 1990s, obtaining a least-squares loss by subtracting the right-hand side from the left-hand side of the equation (Dissanayake and Phan-Thien, 1994); however, influenced by the trough of deep learning, they were gradually forgotten. In recent years, with the resurgence of deep learning, this class of methods has once again attracted the attention of researchers. In 2017, this approach was renamed the Physics-Informed Neural Network (PINN) (Raissi et al., 2019). Similarly, various equivalent formulations of differential equations can be used to construct neural-network loss functions: for example, minimizing the Ritz variational form can be used to solve elliptic equations, and the method that constructs a loss in the Ritz variational form is called the Deep Ritz method (E and Yu, 2018), with a similar Deep Nitsche method (Liao and Ming, 2021).

The second class of neural-network methods for solving differential equations parameterizes the operator. These methods have stronger generality and aim to parameterize the operators of an entire class of differential equations with a neural network. They are not restricted to solving a single equation, but rather try to learn the generalized operator of differential equations, thereby providing a unified framework for solving a large class of problems. Examples include directly solving the mapping between source terms and solutions at discrete points (Khoo et al., 2017), DeepONet (Lu et al., 2019), and Fourier Neural Operator (Li et al., 2020). However, achieving the desired generalization ability with this class of methods is not easy.

15.2 Parameterized-Solution Methods

In this section, we introduce three parameterized-solution methods: the least-squares method, the variational method, and the weak-solution method.

15.2.1 Least-Squares Method

This subsection introduces the least-squares method for solving partial differential equations with neural networks, also known as the Physics-Informed Neural Network (PINN) method (Dissanayake and Phan-Thien, 1994; Raissi et al., 2019). Suppose the PDE we wish to solve has the form

$$\mathcal{N}(u(x,t)) = f(x,t), \quad (x,t) \in \Omega \times [0,T], \tag{15.3}$$

where $\mathcal{N}$ is a differential operator, $u(x,t)$ is the unknown function to be solved, $f(x,t)$ is the known source term, Ω is the spatial domain, and $[0,T]$ is the time interval.

We define the neural network solution as $u_\theta(x,t)$, used to approximate the solution $u(x,t)$. The loss function $L(\theta)$ can be expressed as

$$L(\theta) = L_{\text{PDE}}(\theta) + L_{\text{IC}}(\theta) + L_{\text{BC}}(\theta) + L_{\text{data}}(\theta), \tag{15.4}$$

where L_{PDE} quantifies how well the model satisfies the governing law of the PDE, L_{IC} and L_{BC} measure the fitting error of the model on the initial condition and the boundary condition respectively, and L_{data} measures the prediction deviation of the model at known data points in the spatial domain. The specific forms of each part are as follows.

$$L_{\text{PDE}}(\theta) = \frac{1}{N_{\text{PDE}}} \sum_{i=1}^{N_{\text{PDE}}} \left| \mathcal{N}(u_\theta(x_i,t_i)) - f(x_i,t_i) \right|^2. \tag{15.5}$$

Here, (x_i,t_i) are points randomly selected in the computational domain $\Omega \times [0,T]$, and N_{PDE} is the number of selected points.

$$L_{\text{IC}}(\theta) = \frac{1}{N_{\text{IC}}} \sum_{j=1}^{N_{\text{IC}}} \left| u_\theta(x_j,0) - u_0(x_j) \right|^2, \tag{15.6}$$

where $u_0(x_j)$ is the true value of the initial condition, $(x_j,0)$ are randomly selected points at the initial time, and N_{IC} is the number of selected points.

$$L_{\text{BC}}(\theta) = \frac{1}{N_{\text{BC}}} \sum_{k=1}^{N_{\text{BC}}} \left| u_\theta(x_k,t_k) - g(x_k,t_k) \right|^2. \tag{15.7}$$

Here, $g(x_k,t_k)$ is the true value of the boundary condition, (x_k,t_k) are randomly selected points on the boundary, and N_{BC} is the number of selected points.

$$L_{\text{data}}(\theta) = \frac{1}{N_{\text{data}}} \sum_{m=1}^{N_{\text{data}}} \left| u_\theta(x_m,t_m) - \hat{u}(x_m,t_m) \right|^2, \tag{15.8}$$

where $\hat{u}(x_m,t_m)$ is the value of the observation data, (x_m,t_m) are randomly selected points of observation data, and N_{data} is the number of observation data points. A key advantage of PINNs

is that the loss function can be evaluated at arbitrary points in the domain, with no need for a predefined mesh or grid. This makes handling complex geometries and high-dimensional problems efficient.

15.2.2 Variational Method

The core idea of the Deep Ritz variational method (E and Yu, 2018) is to use a deep neural network to approximately solve the minimization problem of an energy functional, thereby obtaining the solution of the PDE.

Taking the Poisson equation as an example, consider the equation of the form

$$-\Delta u(x) = f(x), \quad x \in \Omega, \tag{15.9}$$

where Δ is the Laplacian operator, $u(x)$ is the unknown function to be solved, $f(x)$ is the known source term, and Ω is the domain. We also need to consider boundary conditions, such as the Dirichlet boundary condition

$$u(x) = g(x), \quad x \in \partial\Omega, \tag{15.10}$$

where $g(x)$ is a function given on the boundary $\partial\Omega$. To solve this equation, we can define an energy functional

$$E(u) = \frac{1}{2}\int_{\Omega} |\nabla u|^2 \, dx - \int_{\Omega} f u \, dx. \tag{15.11}$$

Our goal is to find the function u that minimizes the energy functional $E(u)$ while satisfying the boundary conditions.

In the Deep Ritz method, we use a deep neural network $u_\theta(x)$ to approximate the solution $u(x)$. To ensure the network output satisfies the boundary condition, we can add a boundary-condition constraint to the loss function. By optimizing the network parameters θ, we minimize the loss function of the energy functional:

$$L(\theta) = E(u_\theta) + L_{\mathrm{BC}}(\theta) = E(u_\theta) + \frac{1}{N_{\mathrm{BC}}}\sum_{k=1}^{N_{\mathrm{BC}}} |u_\theta(x_k) - g(x_k)|^2 . \tag{15.12}$$

Here, $L_{\mathrm{BC}}(\theta)$ is the boundary-condition loss, x_k are randomly selected points on the boundary, and N_{BC} is the number of selected boundary points. For time-dependent problems, a loss term concerning the initial value can likewise be added to the loss function. If the problem also provides some measurement data, this can similarly be added to the loss function.

In practical computation, numerical integration methods such as Monte Carlo integration or adaptive integration techniques are typically used to approximate the above integrals.

The advantage of the Deep Ritz method is that the regularity requirements on the solution are reduced, the order of derivatives that need to be computed is lower, making it friendlier for training, while also effectively handling boundary conditions.

15.2.3 Weak-Solution Method

The Weak Adversarial Networks (WAN) (Zang et al., 2020) leverage the weak formulation of PDEs. Taking a second-order elliptic PDE as an example, consider the equation of the form

$$-\Delta u(x) = f(x), \quad x \in \Omega, \tag{15.13}$$

where Δ is the Laplacian operator, $u(x)$ is the unknown function to be solved, $f(x)$ is the known source term, and Ω is the domain. To solve this equation, we need to consider its weak form.

In the weak formulation, the goal is to find a weak solution that satisfies

$$\int_\Omega \nabla u(x) \cdot \nabla \phi(x)\, dx = \int_\Omega f(x)\phi(x)\, dx, \quad \forall \phi \in H_0^1(\Omega). \tag{15.14}$$

Here, ϕ is a test function, and $H_0^1(\Omega)$ is the appropriate Hilbert space.

In the WAN method, we parameterize the weak solution and the test function as a primal network and an adversarial network, respectively. Specifically, we define two neural networks:

- The primal network $u_\theta(x)$ is used to approximate the weak solution.
- The adversarial network $\phi_\eta(x)$ is used to approximate the test function.

By alternately updating the parameters of these two networks, WAN can effectively approximate the weak solution of the PDE. The loss function is constructed based on the weak-form condition:

$$L(\theta, \eta) = \left\| \int_\Omega \nabla u_\theta(x) \cdot \nabla \phi_\eta(x)\, dx - \int_\Omega f(x)\phi_\eta(x)\, dx \right\|^2, \tag{15.15}$$

To achieve this goal, the WAN method optimizes the parameters of the primal network and the adversarial network through adversarial training. Specifically, we can transform the problem into a max-min optimization problem:

$$\min_\theta \max_\eta L(\theta, \eta). \tag{15.16}$$

In this optimization, the parameters θ of the primal network are optimized to minimize the loss, while the parameters η of the adversarial network are optimized to maximize the loss. Such an adversarial training mechanism enables the networks to better capture the complexity of the problem, thereby improving the accuracy of the solution.

15.3 Parameterized-Operator Methods

Traditional methods and neural network methods that directly parameterize the solution share a common problem: they can only solve a single equation. For a new instance (with changed source term or boundary condition, when solving a family of equations), if we have to re-solve from scratch every time, the computational cost is enormous. A very direct idea is: can we learn the mapping between functions? For example, the input could be the initial-condition function, and the output is the solution function of the problem.

Of course, the foremost question we face is: how do we use a function as the input or output of a neural network? The simplest approach is to represent a function by sampling its values at a set of fixed grid points. Therefore, the input is the samples of the initial-condition function at the chosen grid points, namely a vector; the output to be approximated is the values of the target function at the chosen grid points, which is also a vector. What the neural network needs to learn is a mapping from a high-dimensional vector to another high-dimensional vector. Data can be obtained by solving many different initial conditions using traditional algorithms. This method was developed in 2017 Khoo et al. (2017).

For the output of the neural network, we can make the following further change. The neural network only outputs a value related to a position, and this position information together with the initial-condition function serves as the input of the network. The advantage is that we can obtain

the value of the target function at any point. The Deep Operator Network (DeepONet) adopts this approach Lu et al. (2019) and makes some structural innovations.

Another common form of neural network operator leverages the Green's function representation of differential equations. For example, if the solution of a differential equation can be expressed in terms of boundary conditions in the form

$$u(x) = \int_D G(x, y) f(y) \mathrm{d}y, \tag{15.17}$$

where $G(x, y)$ is the Green's function and $f(y)$ is the boundary-condition function, then we can learn a neural network to represent the Green's function, and quickly obtain an approximation of the target function via a discrete integral. The Fourier Neural Operator is precisely built on this idea Li et al. (2020).

The neural network operators mentioned above are all data-driven and do not exploit the specific information of the equation. In fact, the loss function can also include a governing-equation minimization term as used in PINN, which helps improve model performance; this line of exploration first appeared in Zhang et al. (2022).

15.4 Strengths and Limitations

In this section we discuss the strengths and limitations of neural-network methods for solving differential equations compared with traditional methods.

15.4.1 Strengths

Integration of mechanisms and data Deep learning methods can effectively integrate physical mechanisms with data-driven properties. By introducing penalty terms, constraints such as physical laws, boundary conditions, and initial conditions can be embedded directly into the loss function. This approach not only improves the predictive accuracy of the model but also ensures that the solutions are physically reasonable, thereby enhancing the credibility of the model.

The ability of neural networks to approximate high-dimensional functions Deep neural networks have powerful function approximation capabilities, able to handle complex functions in high-dimensional spaces. Compared with traditional numerical methods, deep learning can effectively capture nonlinear features in high-dimensional problems, providing more accurate solutions. This property is especially important for solving complex PDEs.

Mesh-free handling of high-dimensional and complex-boundary problems Deep learning methods do not rely on mesh discretization, which gives them significant advantages when handling high-dimensional and complex geometric boundaries. Traditional numerical methods typically require fine mesh partitioning, while deep learning methods can be evaluated at any point, avoiding the computational burden brought by mesh generation and optimization.

Advantages of automatic differentiation Deep learning frameworks typically include built-in automatic differentiation, allowing convenient computation of derivatives of the neural network. This makes deep learning methods particularly suitable for solving differential equations, because

we can directly use the network output to compute the residual of the PDE and efficiently update network parameters through backpropagation.

Parallel computing advantages The training of deep learning models can fully exploit the parallel computing capability of modern GPUs. This makes processing large datasets and training complex models much more efficient, significantly shortening solution times. It is especially important for applications that demand real-time or near real-time solutions (such as fluid simulation and climate modeling).

Fast solution of a class of problems Parameterized-operator methods can quickly solve a class of equations in the inference stage. For example, when boundary conditions change, the trained neural network operator does not need to be retrained and can directly and quickly produce the corresponding solution.

Solving inverse problems Some parameters in the equation that need to be determined can be directly treated as variables in the optimization problem, so that the solution of forward and inverse problems can use a unified formulation. Meanwhile, if a neural operator is used to solve a class of equations and traditional algorithms are applied to handle the inverse problem (which requires solving many forward problems), the neural network can quickly solve the forward problems within the algorithm. In the inverse-problem solving process, the parameters of the equation to be determined can be directly treated as variables of the optimization problem, so that the solutions of forward and inverse problems can be carried out in a unified form. In addition, by using a neural operator to solve specific types of equations, traditional algorithms can be effectively applied to handle inverse problems. In this process, the neural network can quickly solve forward problems, thus avoiding the long time traditionally required to solve a large number of forward problems and improving overall efficiency.

15.4.2 Limitations

Dependence on training data Deep learning methods require large amounts of training data to achieve good performance, and obtaining high-quality training data is not easy. In particular, for high-dimensional problems, obtaining a sufficient number of high-quality sample points in a high-dimensional space is very difficult.

Lack of theoretical guarantees Theoretical analyses of the convergence and stability of deep learning methods are still not fully developed, making it difficult to provide rigorous error estimates.

Satisfaction of physical constraints Solutions from deep learning may not strictly satisfy the physical constraints of the differential equation, and strategies are needed to enforce constraint conditions.

High demand for computational resources Training complex deep learning models requires substantial computational resources and time, with high hardware requirements.

Weak interpretability Deep learning models are often treated as black boxes, making it difficult to explain their internal mechanisms, which can be an obstacle in applications with strict interpretability requirements.

Low accuracy Limited by data quality and training methods, the accuracy of solutions from deep learning methods is typically lower than that of sophisticated traditional numerical schemes.

15.5 Different preferences of traditional algorithms and neural networks for low frequencies

In this section, we take the one-dimensional Poisson equation as an example to demonstrate the different preferences of traditional algorithms and neural networks for low frequencies.

The one-dimensional Poisson equation we consider is

$$-\Delta u(x) = g(x) \quad x \in \Omega = (-11), \tag{15.18}$$

where $g(x) = \sin(x) + 4\sin(4x) - 8\sin(8x) + 16\sin(24x)$, with boundary condition $u(x) = 0 \quad x = -1, 1$.

Our goal is to use a neural network $u_\theta(x)$ to approximate the reference solution $u_{ref}(x)$. To this end, we uniformly sample 1001 training points $\{x_i\}_{i=0}^n$ on the interval [01], with spacing h between adjacent points. To avoid directly computing high-order derivatives, we adopt the Deep Ritz method, that is, we use the variational form of the Poisson equation (15.19) as the loss function:

$$I_{\text{emp}} = \sum_{i=1}^{n-1} \left(\frac{1}{2} |\nabla_x u(x_i)|^2 - g(x_i) u(x_i) \right) h + \beta \left(u(x_0)^2 + u(x_n)^2 \right). \tag{15.19}$$

Here, β is a fixed weight coefficient used to handle the boundary condition, also called a penalty term. By continuously optimizing this loss function, we expect the neural network $u_\theta(x)$ to approximate the reference solution $u_{ref}(x)$.

During training, we focus on the convergence of the three peaks in the Fourier transform of $u_{ref}(x)$ (as in Fig. 15.1(a)). Figure 15.1(b) clearly shows the neural network's learning preference for different frequency components during training: low-frequency components converge faster than high-frequency components, which is consistent with the frequency principle.

By contrast, traditional numerical methods exhibit a completely different tendency when dealing with low frequencies. Take the finite-difference method as an example. We first discretize the Poisson equation to obtain a linear system, which can then be solved by various linear algebra methods, such as direct methods (e.g., Gaussian elimination or LU decomposition) and iterative methods (e.g., Jacobi iteration or conjugate gradient). In particular, iterative methods are especially suitable for solving large sparse linear systems arising from the discretization of differential equations, because they can refine the solution step by step and do not need to store the full matrix, making the computation more efficient. However, such methods often suffer from large low-frequency errors, as shown in Fig. 15.1(c): traditional methods have difficulty eliminating low-frequency errors. This frequency preference can be proved theoretically; see the advanced section of this subsection.

Based on the different preferences of traditional methods and neural networks for low-frequency components, an intuitive idea is to use a neural network in the early stage of training to quickly

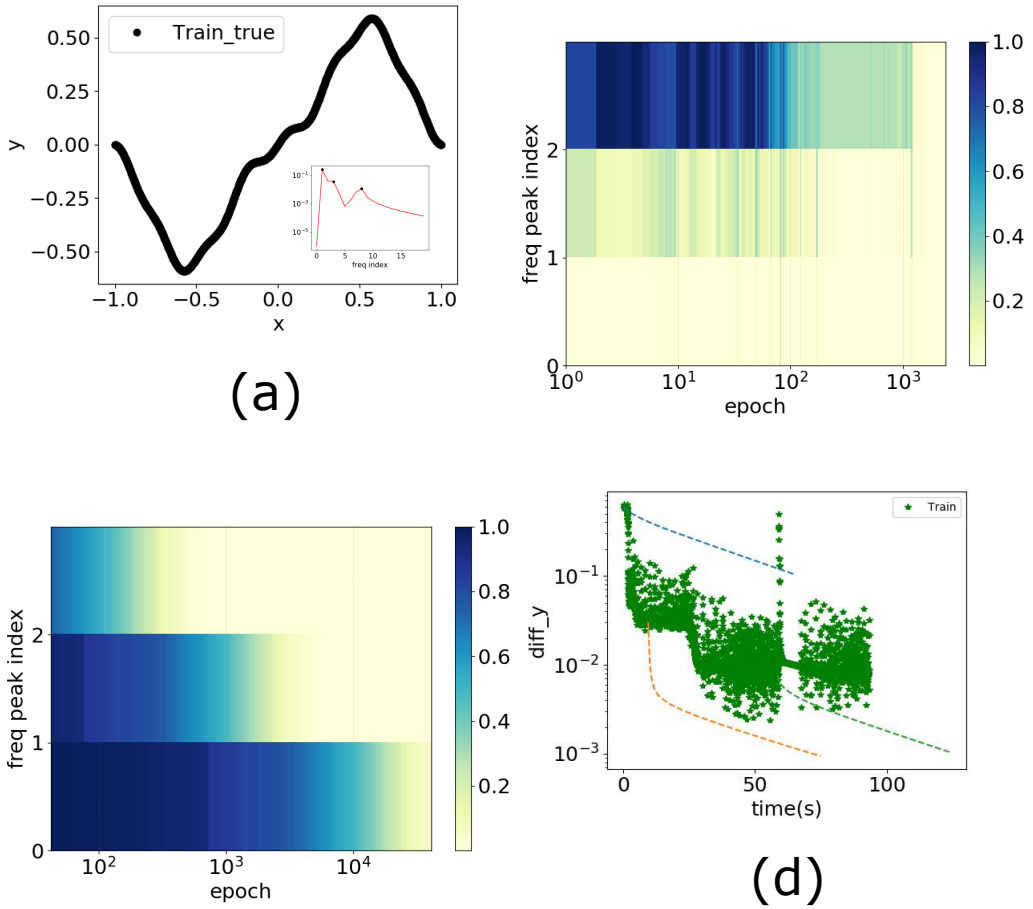


Figure 15.1: Different frequency-domain performance of neural networks and traditional algorithms in solving the Poisson equation. (a) $u_{\text{ref}}(x)$. Inset: $|\hat{u}_{\text{ref}}(k)|$ as a function of frequency k. Black dots mark the peaks of the frequency spectrum. (b, c) For several selected frequencies, $\Delta_F(k)$ on the training data at different epochs for the DNN (b) and the Jacobi method (c). (d) $\|h - u_{\text{ref}}\|_\infty$ at different running times.

learn the low-frequency features, and then switch to a traditional method in the later stage to more efficiently capture the higher-frequency features.

In the experiment, we first use a neural network to solve the Poisson equation in Eq.(15.18)), where $g(x) = \sin(x) + 4\sin(4x) - 8\sin(8x) + 16\sin(24x)$. To ensure that the neural network has already fit the low-frequency part well by the time training stops, we need to choose the number of training steps M reasonably. Next, we feed the discrete function values output by the neural network as new initial values into the Jacobi iteration method to continue the convergence computation. In the experiments we use $\|h - u_{\text{ref}}\|_\infty \triangleq \max_{x\in\Omega} |h(x) - u_{\text{ref}}(x)|$ to quantify the error.

Figure 15.1(d) shows the convergence of different methods. The green stars represent solving the equation using only the neural network; we can observe that the error $\|h - u_{\text{ref}}\|_\infty$ fluctuates after a few iterations. The dashed lines indicate that, after the neural network has been trained for some time, its result is used as the initial condition for the Jacobi iteration, which is then continued. If the stopping step M is too small, the result mainly depends on the Jacobi iteration, and the algorithm needs a longer time to converge because Jacobi is slow on the low-frequency part. Conversely, if M is too large (corresponding to the right dashed line in the figure), the result mainly depends on the neural network, and the higher-frequency part converges slowly. Choosing an appropriate value of M (corresponding to the orange dashed line in the figure) can combine the advantages of both methods: use the neural network to quickly fit the low-frequency part, and then use Jacobi to efficiently capture the higher-frequency details, thereby obtaining a converged solution faster.

This example brings us an important lesson: when solving certain problems, using a DNN alone may not be the best choice. This is because DNNs suffer from slow convergence on high-frequency components, which may affect both efficiency and accuracy. Combining the strengths of DNNs and traditional methods to design faster schemes is a promising direction for scientific computing problems; for example, Huang et al. (2020) use a neural network as the initial value of an iterative algorithm and accelerate the solution for many problems.

15.5.1 Multi-Scale Neural Network for Solving PDEs

The slow convergence of neural networks on high-frequency errors not only affects computational efficiency, but may also cause accuracy issues. High-frequency errors mainly contain small-scale features of physical phenomena, such as high-frequency electromagnetic fields in Maxwell's equations. If these high-frequency parts are not accurately resolved, the overall physical process obtained may lack key detailed information, thereby affecting our understanding and applications of the physical phenomena.

To overcome the slow learning of high-frequency features by neural networks, Chapter 5 proposes a multi-scale neural network (MscaleDNN); using this network can effectively learn high-frequency information. Below we use a two-dimensional problem in a porous medium as an example to illustrate the advantages of MscaleDNN.

Consider the Poisson equation (15.18) with the following source term:

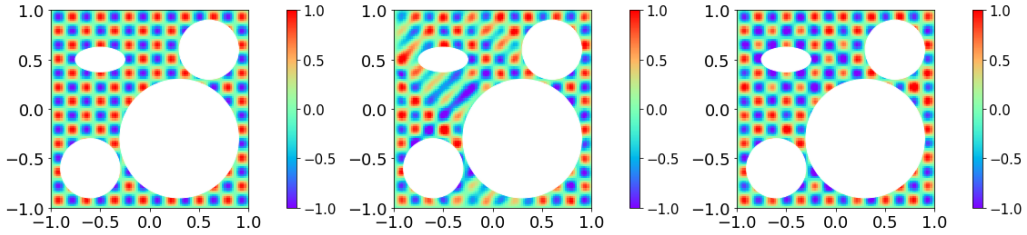
$$g(\boldsymbol{x}) = 2\mu^2 \sin \mu x_1 \sin \mu x_2 \quad \mu = 7\pi, \tag{15.20}$$

whose exact solution is

$$u(\boldsymbol{x}) = \sin \mu x_1 \sin \mu x_2. \tag{15.21}$$

We use the exact solution to give the boundary conditions. As shown in Fig. 1, MscaleDNN can

accurately capture the oscillatory features in the exact solution, whereas an ordinary neural network has difficulty fitting these high-frequency oscillations accurately.



(a) Exact solution (b) Ordinary neural network (c) MscaleDNN

Figure 15.2: A two-dimensional porous example. Figure adapted from Liu et al. (2020).

15.5.2 Summary

Traditional numerical algorithms often encounter the so-called "curse of dimensionality" when handling high-dimensional problems: as the dimensionality of the problem grows, the computational complexity increases exponentially, making data processing and manipulation extremely difficult. By contrast, neural networks have demonstrated significant advantages in dealing with high-dimensional problems, especially when fitting high-dimensional, nonlinear mappings with complex structures, where neural networks perform particularly well.

The powerful high-dimensional fitting ability of neural networks is one of their main attractions. When we need to solve PDEs with complex boundary conditions or special properties, deep learning can serve as a powerful tool to capture and learn the structures and patterns in the data. Compared with traditional methods, neural networks have the potential to simplify model complexity while maintaining high solution accuracy and computational efficiency, providing a novel strategy for solving PDEs.

Bibliography

M. L. Minsky, S. A. Papert, Perceptrons: expanded edition, 1988.

G. Cybenko, Approximation by superpositions of a sigmoidal function, Mathematics of control, signals and systems 2 (1989) 303–314.

K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, Neural networks 2 (1989) 359–366.

M. Leshno, V. Y. Lin, A. Pinkus, S. Schocken, Multilayer feedforward networks with a nonpolynomial activation function can approximate any function, Neural networks 6 (1993) 861–867.

D. Yarotsky, Error bounds for approximations with deep ReLU networks, Neural Networks 94 (2017) 103–114. URL: http://www.sciencedirect.com/science/article/pii/S0893608017301545. doi:10.1016/j.neunet.2017.07.002.

D. Yarotsky, Optimal approximation of continuous functions by very deep ReLU networks, in: S. Bubeck, V. Perchet, P. Rigollet (Eds.), Proceedings of the 31st Conference On Learning Theory, volume 75 of *Proceedings of Machine Learning Research*, PMLR, 2018, pp. 639–649. URL: http://proceedings.mlr.press/v75/yarotsky18a.html.

J. Lu, Z. Shen, H. Yang, S. Zhang, Deep network approximation for smooth functions, SIAM Journal on Mathematical Analysis 53 (2021) 5465–5506. doi:10.1137/20M134695X.

Z. Shen, H. Yang, S. Zhang, Deep network approximation characterized by number of neurons, Communications in Computational Physics 28 (2020) 1768–1811. doi:10.4208/cicp.OA-2020-0149.

Z. Shen, H. Yang, S. Zhang, Optimal approximation rate of ReLU networks in terms of width and depth, Journal de Mathématiques Pures et Appliquées 157 (2022) 101–135. URL: https://www.sciencedirect.com/science/article/pii/S0021782421001124. doi:10.1016/j.matpur.2021.07.009.

S. Zhang, J. Lu, H. Zhao, Deep network approximation: Beyond ReLU to diverse activation functions, Journal of Machine Learning Research 25 (2024) 1–39. URL: http://jmlr.org/papers/v25/23-0912.html.

D. Yarotsky, A. Zhevnerchuk, The phase diagram of approximation rates for deep neural networks, in: Advances in Neural Information Processing Systems, volume 33, Curran Associates, Inc., 2020, pp. 13005–13015. URL: https://proceedings.neurips.cc/paper/2020/file/979a3f14bae523dc5101c52120c535e9-Paper.pdf.

Z. Shen, H. Yang, S. Zhang, Deep network with approximation error being reciprocal of width to power of square root of depth, Neural Computation 33 (2021) 1005–1036. URL: https://doi.org/10.1162/neco_a_01364. doi:10.1162/neco_a_01364.

V. Maiorov, A. Pinkus, Lower bounds for approximation by MLP neural networks, Neurocomputing 25 (1999) 81–91. URL: http://www.sciencedirect.com/science/article/pii/S0925231298001118. doi:10.1016/S0925-2312(98)00111-8.

D. Yarotsky, Elementary superexpressive activations, in: M. Meila, T. Zhang (Eds.), Proceedings of the 38th International Conference on Machine Learning, volume 139 of *Proceedings of Machine Learning Research*, PMLR, 2021, pp. 11932–11940. URL: https://proceedings.mlr.press/v139/yarotsky21a.html.

Z. Shen, H. Yang, S. Zhang, Deep network approximation: Achieving arbitrary accuracy with fixed number of neurons, Journal of Machine Learning Research 23 (2022) 1–60. URL: http://jmlr.org/papers/v23/21-1404.html.

L. Wu, C. Ma, et al., How sgd selects the global minima in over-parameterized learning: A dynamical stability perspective, Advances in Neural Information Processing Systems 31 (2018).

B. T. Polyak, Some methods of speeding up the convergence of iteration methods, Ussr computational mathematics and mathematical physics 4 (1964) 1–17.

Y. Nesterov, A method of solving a convex programming problem with convergence rate o (1/k** 2), Doklady Akademii Nauk SSSR 269 (1983) 543.

J. Duchi, E. Hazan, Y. Singer, Adaptive subgradient methods for online learning and stochastic optimization., Journal of machine learning research 12 (2011).

D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, arXiv preprint arXiv:1412.6980 (2014).

S. J. Reddi, S. Kale, S. Kumar, On the convergence of adam and beyond, in: International Conference on Learning Representations, 2018.

Y. Zhang, C. Chen, N. Shi, R. Sun, Z.-Q. Luo, Adam can converge without any modification on update rules, Advances in neural information processing systems 35 (2022) 28386–28399.

X. Xie, P. Zhou, H. Li, Z. Lin, S. Yan, Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models, IEEE Transactions on Pattern Analysis and Machine Intelligence (2024).

H. Liu, Z. Li, D. Hall, P. Liang, T. Ma, Sophia: A scalable stochastic second-order optimizer for language model pre-training, arXiv preprint arXiv:2305.14342 (2023).

Y. A. LeCun, L. Bottou, G. B. Orr, K.-R. Müller, Efficient backprop, in: Neural networks: Tricks of the trade, Springer, 2012, pp. 9–48.

X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Proceedings of the thirteenth international conference on artificial intelligence and statistics, 2010, pp. 249–256.

K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, in: Proceedings of the IEEE international conference on computer vision, 2015, pp. 1026–1034.

M. V. Narkhede, P. P. Bartakke, M. S. Sutaone, A review on weight initialization strategies for neural networks, Artificial intelligence review 55 (2022) 291–322.

D. H. Wolpert, W. G. Macready, No free lunch theorems for optimization, IEEE transactions on evolutionary computation 1 (1997) 67–82.

L. Wu, Z. Zhu, W. E, Towards understanding generalization of deep learning: Perspective of loss landscapes, arXiv preprint arXiv:1706.10239 (2017).

S. Shalev-Shwartz, O. Shamir, S. Shammah, Failures of gradient-based deep learning, arXiv preprint arXiv:1703.07950 (2017).

M. Nye, A. Saxe, Are efficient deep representations learnable? (2018).

J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei, Scaling laws for neural language models, arXiv preprint arXiv:2001.08361 (2020).

L. Breiman, Reflections after refereeing papers for nips, The Mathematics of Generalization XX (1995) 11–15.

C. Zhang, S. Bengio, M. Hardt, B. Recht, O. Vinyals, Understanding deep learning requires rethinking generalization, in: 5th International Conference on Learning Representations, 2017.

R. A. DeVore, Nonlinear approximation, Acta numerica 7 (1998) 51–150.

A. Blum, J. Hopcroft, R. Kannan, High-Dimensional Space, Cambridge University Press, 2020, p. 4–28.

A. R. Barron, Universal approximation bounds for superpositions of a sigmoidal function, IEEE Transactions on Information theory 39 (1993) 930–945.

W. E, C. Ma, L. Wu, Barron spaces and the compositional function spaces for neural network models, arXiv preprint arXiv:1906.08039 (2019).

K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 770–778.

C. Szegedy, S. Ioffe, V. Vanhoucke, A. Alemi, Inception-v4, inception-resnet and the impact of residual connections on learning, in: Proceedings of the AAAI conference on artificial intelligence, volume 31, 2017.

G. Huang, Z. Liu, L. Van Der Maaten, K. Q. Weinberger, Densely connected convolutional networks, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2017, pp. 4700–4708.

S. Xie, R. Girshick, P. Dollár, Z. Tu, K. He, Aggregated residual transformations for deep neural networks, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2017, pp. 1492–1500.

Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, et al., Google's neural machine translation system: Bridging the gap between human and machine translation, arXiv preprint arXiv:1609.08144 (2016).

W. Xiong, L. Wu, F. Alleva, J. Droppo, X. Huang, A. Stolcke, The microsoft 2017 conversational speech recognition system, in: 2018 IEEE international conference on acoustics, speech and signal processing (ICASSP), IEEE, 2018, pp. 5934–5938.

Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, L. D. Jackel, Backpropagation applied to handwritten zip code recognition, Neural computation 1 (1989) 541–551.

A. Krizhevsky, I. Sutskever, G. E. Hinton, Imagenet classification with deep convolutional neural networks, in: Advances in neural information processing systems, 2012, pp. 1097–1105.

R. Girshick, J. Donahue, T. Darrell, J. Malik, Rich feature hierarchies for accurate object detection and semantic segmentation, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2014, pp. 580–587.

J. Long, E. Shelhamer, T. Darrell, Fully convolutional networks for semantic segmentation, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2015, pp. 3431–3440.

K. Fukushima, Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position, Biological cybernetics 36 (1980) 193–202.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Advances in neural information processing systems 30 (2017).

D. Bahdanau, K. Cho, Y. Bengio, Neural machine translation by jointly learning to align and translate, arXiv preprint arXiv:1409.0473 (2014).

Z. Lin, M. Feng, C. N. d. Santos, M. Yu, B. Xiang, B. Zhou, Y. Bengio, A structured self-attentive sentence embedding, arXiv preprint arXiv:1703.03130 (2017).

M. Wang, et al., Understanding the expressive power and mechanisms of transformer for sequence modeling, arXiv preprint arXiv:2402.00522 (2024).

J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, arXiv preprint arXiv:1810.04805 (2018).

A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al., Improving language understanding by generative pre-training (2018).

J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, Y. Liu, Roformer: Enhanced transformer with rotary position embedding, Neurocomputing 568 (2024) 127063.

D. P. Kingma, Auto-encoding variational bayes, arXiv preprint arXiv:1312.6114 (2013).

W. Cai, X. Li, L. Liu, A phase shift deep neural network for high frequency approximation and wave problems, SIAM Journal on Scientific Computing 42 (2020) A3285–A3312.

Z. Liu, W. Cai, Z.-Q. J. Xu, Multi-scale deep neural network (mscalednn) for solving poisson-boltzmann equation in complex domains, Communications in Computational Physics 28 (2020) 1970–2001.

M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, in: Advances in Neural Information Processing Systems, volume 33, Curran Associates, Inc., 2020, pp. 7537–7547.

B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, R. Ng, Nerf: Representing scenes as neural radiance fields for view synthesis, in: European Conference on Computer Vision, Springer, 2020, pp. 405–421.

V. Sitzmann, J. Martel, A. Bergman, D. Lindell, G. Wetzstein, Implicit neural representations with periodic activation functions, Advances in neural information processing systems 33 (2020) 7462–7473.

Y. Zhang, Z.-Q. J. Xu, T. Luo, Z. Ma, A type of generalization error induced by initialization in deep neural networks, in: Mathematical and Scientific Machine Learning, PMLR, 2020, pp. 144–164.

X.-A. Li, Z.-Q. J. Xu, L. Zhang, Subspace decomposition based dnn algorithm for elliptic type multi-scale pdes, Journal of Computational Physics 488 (2023) 112242.

B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, R. Ng, Nerf: Representing scenes as neural radiance fields for view synthesis, Communications of the ACM 65 (2021) 99–106.

A. D. Jagtap, G. E. Karniadakis, How important are activation functions in regression and classification? a survey, performance comparison, and future directions, Journal of Machine Learning for Modeling and Computing 4 (2023).

A. D. Jagtap, Y. Shin, K. Kawaguchi, G. E. Karniadakis, Deep kronecker neural networks: A general framework for neural networks with adaptive activation functions, Neurocomputing 468 (2022) 165–180.

R. Agarwal, N. Frosst, X. Zhang, R. Caruana, G. E. Hinton, Neural additive models: Interpretable machine learning with neural nets, arXiv preprint arXiv:2004.13912 (2020).

Z.-Q. J. Xu, Y. Zhang, T. Luo, Y. Xiao, Z. Ma, Frequency principle: Fourier analysis sheds light on deep neural networks, Communications in Computational Physics 28 (2020) 1746–1767.

J. Huang, H. Wang, H. Yang, Int-deep: A deep learning initialized iterative method for nonlinear problems, Journal of computational physics 419 (2020) 109675.

J. Chen, X. Chi, Z. Yang, et al., Bridging traditional and machine learning-based algorithms for solving pdes: the random feature method, J Mach Learn 1 (2022) 268–98.

A. Jacot, F. Gabriel, C. Hongler, Neural tangent kernel: Convergence and generalization in neural networks, in: Advances in neural information processing systems, 2018, pp. 8571–8580.

S. Mei, A. Montanari, P.-M. Nguyen, A mean field view of the landscape of two-layer neural networks, Proceedings of the National Academy of Sciences 115 (2018) E7665–E7671.

J. Sirignano, K. Spiliopoulos, Mean field analysis of neural networks: A central limit theorem, Stochastic Processes and their Applications 130 (2020) 1820–1852.

G. Rotskoff, E. Vanden-Eijnden, Parameters as interacting particles: long time convergence and asymptotic error scaling of neural networks, in: Advances in neural information processing systems, 2018, pp. 7146–7155.

W. E, C. Ma, L. Wu, A comparative analysis of optimization and generalization properties of two-layer neural network and random feature models under gradient descent dynamics, Science China Mathematics 63 (2020) 1235–1258.

H. Zhou, Q. Zhou, T. Luo, Y. Zhang, Z.-Q. Xu, Towards understanding the condensation of neural networks at initial training, Advances in Neural Information Processing Systems 35 (2022) 2184–2196.

N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, The journal of machine learning research 15 (2014) 1929–1958.

Z. Zhang, Z.-Q. J. Xu, Implicit regularization of dropout, arXiv preprint arXiv:2207.05952 (2022).

Y. Zhang, Z. Zhang, T. Luo, Z.-Q. J. Xu, Embedding principle of loss landscape of deep neural networks, arXiv preprint arXiv:2105.14573 (2021a).

Y. Zhang, Y. Li, Z. Zhang, T. Luo, Z.-Q. J. Xu, Embedding principle: a hierarchical structure of loss landscape of deep neural networks, arXiv preprint arXiv:2111.15527 (2021b).

Z. Bai, J. Zhao, Y. Zhang, Connectivity shapes implicit regularization in matrix factorization models for matrix completion, in: Proceedings of the Thirty-eighth Annual Conference on Neural Information Processing Systems (NeurIPS), volume 37, 2024, pp. 45914–45955.

Y. Zhang, Z. Zhang, L. Zhang, Z. Bai, T. Luo, Z.-Q. J. Xu, Optimistic estimate uncovers the potential of nonlinear models, arXiv preprint arXiv:2307.08921 (2023).

Y. Feng, Y. Tu, The inverse variance–flatness relation in stochastic gradient descent is critical for finding flat minima, Proceedings of the National Academy of Sciences 118 (2021).

N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, P. T. P. Tang, On large-batch training for deep learning: Generalization gap and sharp minima, arXiv preprint arXiv:1609.04836 (2016).

Z. Zhu, J. Wu, B. Yu, L. Wu, J. Ma, The anisotropic noise in stochastic gradient descent: Its behavior of escaping from sharp minima and regularization effects, arXiv preprint arXiv:1803.00195 (2018).

S. L. Smith, B. Dherin, D. Barrett, S. De, On the origin of implicit regularization in stochastic gradient descent, in: International Conference on Learning Representations, 2020.

H. Li, Z. Xu, G. Taylor, C. Studer, T. Goldstein, Visualizing the loss landscape of neural nets, arXiv preprint arXiv:1712.09913 (2017).

J. Cohen, S. Kaur, Y. Li, J. Z. Kolter, A. Talwalkar, Gradient descent on neural networks typically occurs at the edge of stability, in: International Conference on Learning Representations, 2020.

T. Brown, et al., Language models are few-shot learners, Advances in neural information processing systems 33 (2020) 1877–1901.

OpenAI, Gpt-4 technical report, 2023. arXiv:2303.08774.

L. Van der Maaten, G. Hinton, Visualizing data using t-sne., Journal of machine learning research 9 (2008).

H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al., Llama 2: Open foundation and fine-tuned chat models, arXiv preprint arXiv:2307.09288 (2023).

N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, N. DasSarma, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, C. Olah, Framework for investigating transformer circuits, https://transformer-circuits.pub/2021/framework/index.html, 2021.

C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, B. Mann, A. Askell, Y. Bai, A. Chen, et al., In-context learning and induction heads, arXiv preprint arXiv:2209.11895 (2022).

M. Okawa, E. S. Lubana, R. Dick, H. Tanaka, Compositional abilities emerge multiplicatively: Exploring diffusion models on a synthetic task, Advances in Neural Information Processing Systems 36 (2024).

L. Hang, J. Yao, Z. Bai, T. Chen, Y. Chen, R. Diao, H. Li, P. Lin, Z. Wang, C. Xu, et al., Scalable complexity control facilitates reasoning ability of llms, arXiv preprint arXiv:2505.23013 (2025).

C. Xiao, J. Cai, W. Zhao, G. Zeng, B. Lin, J. Zhou, Z. Zheng, X. Han, Z. Liu, M. Sun, Densing law of llms, arXiv preprint arXiv:2412.04315 (2024).

Q. Dong, L. Li, D. Dai, C. Zheng, Z. Wu, B. Chang, X. Sun, J. Xu, Z. Sui, A survey on in-context learning, arXiv preprint arXiv:2301.00234 (2022).

J. Wei, et al., Chain-of-thought prompting elicits reasoning in large language models, Advances in neural information processing systems 35 (2022) 24824–24837.

A. Power, Y. Burda, H. Edwards, I. Babuschkin, V. Misra, Grokking: Generalization beyond overfitting on small algorithmic datasets, arXiv preprint arXiv:2201.02177 (2022).

B. Barak, B. Edelman, S. Goel, S. Kakade, E. Malach, C. Zhang, Hidden progress in deep learning: Sgd learns parities near the computational limit, Advances in Neural Information Processing Systems 35 (2022) 21750–21764.

S. Bhattamishra, A. Patel, V. Kanade, P. Blunsom, Simplicity bias in transformers and their ability to learn sparse boolean functions, arXiv preprint arXiv:2211.12316 (2022).

Z. Liu, E. J. Michaud, M. Tegmark, Omnigrok: Grokking beyond algorithmic data, in: The Eleventh International Conference on Learning Representations, 2022.

J. Frankle, M. Carbin, The lottery ticket hypothesis: Finding sparse, trainable neural networks, arXiv preprint arXiv:1803.03635 (2018).

M. Belkin, D. Hsu, S. Ma, S. Mandal, Reconciling modern machine-learning practice and the classical bias–variance trade-off, Proceedings of the National Academy of Sciences 116 (2019) 15849–15854.

P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, I. Sutskever, Deep double descent: Where bigger models and more data hurt, Journal of Statistical Mechanics: Theory and Experiment 2021 (2021) 124003.

V. Papyan, X. Han, D. L. Donoho, Prevalence of neural collapse during the terminal phase of deep learning training, Proceedings of the National Academy of Sciences 117 (2020) 24652–24663.

T. Garipov, P. Izmailov, D. Podoprikhin, D. P. Vetrov, A. G. Wilson, Loss surfaces, mode connectivity, and fast ensembling of dnns, Advances in neural information processing systems 31 (2018).

W. E, J. Han, A. Jentzen, Algorithms for solving high dimensional pdes: from nonlinear monte carlo to machine learning, Nonlinearity 35 (2021) 278.

M. Dissanayake, N. Phan-Thien, Neural-network-based approximations for solving partial differential equations, communications in Numerical Methods in Engineering 10 (1994) 195–201.

M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, Journal of Computational Physics 378 (2019) 686–707.

W. E, B. Yu, The deep ritz method: A deep learning-based numerical algorithm for solving variational problems, Communications in Mathematics and Statistics 6 (2018) 1–12.

Y. Liao, P. Ming, Deep nitsche method: Deep ritz method with essential boundary conditions, Communications in Computational Physics 29 (2021) 1365–1384.

Y. Khoo, J. Lu, L. Ying, Solving parametric pde problems with artificial neural networks, arXiv preprint arXiv:1707.03351 (2017).

L. Lu, P. Jin, G. E. Karniadakis, Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators, arXiv preprint arXiv:1910.03193 (2019).

Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: Graph kernel network for partial differential equations, arXiv preprint arXiv:2003.03485 (2020).

Y. Zang, G. Bao, X. Ye, H. Zhou, Weak adversarial networks for high-dimensional partial differential equations, Journal of Computational Physics 411 (2020) 109409. URL: http://dx.doi.org/10.1016/j.jcp.2020.109409. doi:10.1016/j.jcp.2020.109409.

Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, arXiv preprint arXiv:2010.08895 (2020).

L. Zhang, T. Luo, Y. Zhang, W. E, Z. Q. J. Xu, Z. Ma, MOD-Net: A machine learning approach via model-operator-data network for solving pdes, Communications in Computational Physics 32 (2022) 299–335.